

The UNED logo is a dark green square with the letters 'UNED' in white, bold, sans-serif font.

UNED

The background image is a full-page photograph of a majestic glacier flowing down a steep, rocky mountain valley. The glacier is a mix of white and blue ice, with visible crevasses. The surrounding mountains are dark and rugged. In the foreground, there is a lush green forest of deciduous trees, and a river with white water rapids flows through the valley floor. The sky is a clear, vibrant blue.

Métodos de simulación y modelado

Alfonso Urquía Moraleda
Carla Martín Villalba

Métodos de simulación y modelado

ALFONSO URQUÍA MORALEDA
CARLA MARTÍN VILLALBA

UNIVERSIDAD NACIONAL DE EDUCACIÓN A DISTANCIA

MÉTODOS DE SIMULACIÓN Y MODELADO

Quedan rigurosamente prohibidas, sin la autorización escrita de los titulares del Copyright, bajo las sanciones establecidas en las leyes, la reproducción total o parcial de esta obra por cualquier medio o procedimiento, comprendidos la reprografía y el tratamiento informático, y la distribución de ejemplares de ella mediante alquiler o préstamos públicos.

© *Universidad Nacional de Educación a Distancia*
Madrid 2016

www.uned.es/publicaciones

© *Alfonso Urquía Moraleda, Carla Martín Villalba*

ISBN electrónico: 978-84-362-7132-4

Edición digital: septiembre de 2016

ÍNDICE

Prefacio

Organización de la Unidad Didáctica

Cómo utilizar el libro

Objetivos docentes

1. Introducción al modelado y la simulación

1.1. Introducción

1.2. Conceptos fundamentales

1.2.1. Sistema, experimento y modelo

1.2.2. Niveles en el conocimiento de los sistemas

1.2.3. Marco formal para el

1.3. Pasos en un estudio de simulación

1.3.1. Definición del objetivo

1.3.2. Hipótesis de modelado

1.3.3. Planteamiento del modelo

1.3.4. Diseño de los experimentos

1.3.5. Verificación y validación

1.4. Tipos de modelos y sus simuladores

1.4.1. Clasificaciones de los modelos matemáticos

1.4.2. Modelos de tiempo discreto

1.4.3. Modelos de eventos discretos

- 1.4.4. Autómatas celulares
- 1.4.5. Modelos basados en agentes
- 1.4.6. Modelos dinámicos en ecuaciones diferenciales ordinarias
- 1.4.7. Modelos híbridos
- 1.4.8. Modelos en derivadas parciales
- 1.5. Introducción al análisis de datos con R
 - 1.5.1. El espacio de trabajo
 - 1.5.2. Estructuras de datos
 - 1.5.3. Gráficos
 - 1.5.4. Manejo básico de los datos
 - 1.5.5. Valor NA (Not Available)
 - 1.5.6. Conversión del tipo de datos
 - 1.5.7. Control del flujo
 - 1.5.8. Funciones matemáticas y estadísticas
 - 1.5.9. Definición de funciones
 - 1.5.10. Paquetes
- 1.6. Lecturas recomendadas
- 1.7. Ejercicios de autocomprobación
- 1.8. Soluciones de los ejercicios

2. Modelado basado en principios físicos

- 2.1. Introducción
- 2.2. El lenguaje Modelica
 - 2.2.1. Paradigma del modelado físico
 - 2.2.2. Orientación a objetos
 - 2.2.3. Entornos de modelado
- 2.3. Fundamentos del modelado de sistemas físicos
 - 2.3.1. Interacción entre los componentes

- 2.3.2. Condiciones de contorno
- 2.3.3. Disipación de la energía
- 2.3.4. Intercambio de calor
- 2.3.5. Almacenamiento de la energía
- 2.3.6. Conversión reversible de la energía
- 2.4. Circuitos eléctricos
 - 2.4.1. Ejemplo 1: circuito rectificador
 - 2.4.2. Ejemplo 2: librería eléctrica
 - 2.4.3. Ejemplo 3: redeclaración del tipo de los componentes
- 2.5. Sistemas mecánicos
 - 2.5.1. Ejemplo 1: traslación en una dirección
 - 2.5.2. Ejemplo 2: traslación en el plano
 - 2.5.3. Ejemplo 3: vibración longitudinal de una varilla
- 2.6. Flujo de fluidos e intercambio de calor
 - 2.6.1. Ejemplo 1: flujo radial de calor en una tubería
 - 2.6.2. Ejemplo 2: conducción longitudinal del calor en una varilla
 - 2.6.3. Ejemplo 3: control del nivel y temperatura de un depósito
 - 2.6.4. Ejemplo 4: disipación del calor de un circuito
- 2.7. Lecturas recomendadas
- 2.8. Ejercicios de autocomprobación
- 2.9. Soluciones de los ejercicios
- 3. Simulación de modelos de tiempo continuo**
 - 3.1. Introducción
 - 3.2. Asignación de la causalidad computacional
 - 3.2.1. Clasificación de las variables
 - 3.2.2. Singularidad estructural
 - 3.2.3. Algoritmo de partición

- 3.2.4. Sistemas sobredeterminados e infradeterminados
- 3.2.5. Ejemplo: simulación de un circuito eléctrico
- 3.3. Sistemas DAE estructuralmente singulares
- 3.4. Índice de los sistemas DAE
 - 3.4.1. Definición de índice
 - 3.4.2. Dificultades asociadas al índice superior
- 3.5. Inicialización de sistemas DAE
 - 3.5.1. Ligaduras ocultas y reducción del índice
 - 3.5.2. El algoritmo de Pantelides
- 3.6. Lazos algebraicos
 - 3.6.1. Manipulación simbólica de los lazos algebraicos
 - 3.6.2. Solución de los lazos en la inicialización
 - 3.6.3. Tearing de los lazos algebraicos no lineales
- 3.7. Selección de las variables de estado
 - 3.7.1. Manipulación del sistema DAE
 - 3.7.2. Selección dinámica por el entorno de modelado
 - 3.7.3. Selección por el desarrollador del modelo
- 3.8. Solución numérica de sistemas DAE
 - 3.8.1. DASSL
 - 3.8.2. Integración inline
 - 3.8.3. Integración mixed-mode
- 3.9. Lecturas recomendadas
- 3.10. Ejercicios de autocomprobación
- 3.11. Soluciones de los ejercicios

4. Modelado y simulación de sistemas híbridos

- 4.1. Introducción
- 4.2. Especificación de los modelos híbridos

- 4.2.1. Formalismo OHM
- 4.2.2. Especificación formal y algoritmo de la simulación
- 4.2.3. Especificación formal y descripción en Modelica
- 4.2.4. Ejemplo 1: rebote de una pelota
- 4.2.5. Ejemplo 2: depósito con válvula de desagüe
- 4.2.6. Ejemplo 3: dos depósitos conectados mediante una válvula
- 4.3. Detección y ejecución de los eventos
 - 4.3.1. Eventos simultáneos
 - 4.3.2. Función de cruce
 - 4.3.3. Determinación del instante de disparo de los eventos
 - 4.3.4. Chattering
- 4.4. Modelos con estructura variable
 - 4.4.1. Ejemplo 1: interruptor ideal de flujo
 - 4.4.2. Ejemplo 2: tubería con sistema de desagüe
 - 4.4.3. Ejemplo 3: desagüe del líquido de un depósito
 - 4.4.4. Ejemplo 4: llenado de dos depósitos en paralelo
 - 4.4.5. Ejemplo 5: interruptor no ideal de flujo
- 4.5. Modelado de sistemas híbridos en Modelica
 - 4.5.1. Sentencia y cláusula if
 - 4.5.2. Ejemplo 1: interruptor ideal eléctrico
 - 4.5.3. Ejemplo 2: diodo ideal
 - 4.5.4. Cláusula when
 - 4.5.5. Ejemplo 3: dos depósitos conectados mediante una válvula
 - 4.5.6. Ejemplo 4: rebote de una pelota
 - 4.5.7. Ejemplo 5: fricción seca
 - 4.5.8. Tratamiento literal de las expresiones if
 - 4.5.9. Ejemplo 6: conducción de calor en una pared

- 4.6. Inicialización del modelo en Modelica
 - 4.6.1. Planteamiento de la inicialización
 - 4.6.2. Variables de tiempo continuo
 - 4.6.3. Ejemplo 1: péndulo plano
 - 4.6.4. Variables de tiempo discreto
 - 4.6.5. Ejemplo 2: lazo de control
 - 4.6.6. Modelo para la inicialización
- 4.7. Experimentación con modelos en Modelica
 - 4.7.1. Operaciones básicas
 - 4.7.2. Experimentos complejos
- 4.8. Rand Model Designer
 - 4.8.1. Modelado de sistemas físicos
 - 4.8.2. Ejemplo 1: diodo ideal
 - 4.8.3. Desarrollo de simulaciones interactivas
 - 4.8.4. Descripción de la vista del laboratorio virtual
 - 4.8.5. Ejemplo 2: sistema de cuatro tanques
- 4.9. Lecturas recomendadas
- 4.10. Ejercicios de autocomprobación
- 4.11. Soluciones de los ejercicios

5. Modelos en derivadas parciales

- 5.1. Introducción
- 5.2. Operadores diferenciales
 - 5.2.1. Gradiente
 - 5.2.2. Divergencia
 - 5.2.3. Rotacional
 - 5.2.4. Laplaciano
- 5.3. Tipos de PDE lineales de segundo orden

- 5.4. Condiciones iniciales y de frontera
- 5.5. Métodos de resolución
 - 5.5.1. Método de diferencias finitas
 - 5.5.2. Método de elementos finitos
- 5.6. Entornos de simulación de PDE
- 5.7. FlexPDE .
 - 5.7.1. Ficheros de trabajo
 - 5.7.2. Fichero con extensión .PDE
 - 5.7.3. Geometría
 - 5.7.4. Variables y ecuaciones
 - 5.7.5. Dominio del problema: regiones y parámetros
 - 5.7.6. Especificación de las condiciones de contorno
 - 5.7.7. Control de la precisión de la solución
 - 5.7.8. Salida gráfica
 - 5.7.9. Problemas en una dimensión
- 5.8. Ecuación de transferencia de calor
 - 5.8.1. Ejemplo 1: transmisión de calor en una varilla rectangular
 - 5.8.2. Ejemplo 2: transmisión de calor en una superficie cuadrada
- 5.9. Ecuación de ondas
 - 5.9.1. Ejemplo 1: cuerda fija en sus dos extremos
 - 5.9.2. Ejemplo 2: cuerda fija sólo en un extremo
- 5.10. Ecuación de Laplace
 - 5.10.1. Ejemplo 1: condensador 2D relleno con 2 dieléctricos
 - 5.10.2. Ejemplo 2: cable coaxial de sección exterior rectangular
- 5.11. Lecturas recomendadas
- 5.12. Ejercicios de autocomprobación
- 5.13. Soluciones de los ejercicios

Índice alfabético

Bibliografía

CÓDIGO

- 2.1. Movimiento de un péndulo simple en el plano.
- 2.2. Circuito de la Figura 2.15, con los parámetros de la Tabla 2.3.
- 2.3. Fragmento del package Modelica.SIunits de la MSL
- 2.4. Circuito de la Figura 2.15, usando los tipos de la MSL.
- 2.5. Modelo del circuito rectificador.
- 2.6. Librería eléctrica (1/3).
- 2.7. Librería eléctrica (2/3).
- 2.8. Librería eléctrica (3/3).
- 2.9. Modelo de los dos objetos deslizantes de la Figura 2.24.
- 2.10. Modelo de los tres objetos deslizantes de la Figura 2.26.
- 2.11. Modelo de la órbita terrestre.
- 2.12. Modelo de la vibración longitudinal de una varilla.
- 2.13. Transmisión radial de calor en una tubería.
- 2.14. Función para interpolación lineal y programa que la invoca.
- 2.15. Conducción de calor 1D mediante discretización espacial.
- 2.16. Librería para modelado de conducción de calor 1D en la varilla.
- 2.17. Conducción de calor 1D mediante división en volúmenes de control.
- 2.18. Control de nivel en el depósito de la Figura 2.40.
- 2.19. Control de nivel y temperatura del depósito de la Figura 2.42 (1/2).
- 2.20. Control de nivel y temperatura del depósito de la Figura 2.42 (2/2).
- 2.21. Librería para control de nivel y temperatura de un depósito (1/3).
- 2.22. Librería para control de nivel y temperatura de un depósito (2/3).
- 2.23. Librería para control de nivel y temperatura de un depósito (3/3).
- 2.24. Modelo del flujo de calor entre un circuito eléctrico y su entorno.
- 2.25. Flujo de calor entre el circuito y su entorno usando *inner* y *outer*.
- 2.26. Circuito mostrado en la Figura 2.49.
- 2.27. Circuito mostrado en la Figura 2.50.
- 2.28. Modelo de Lotka-Volterra.
- 2.29. Modelo del sistema mecánico mostrado en la Figura 2.46.
- 2.30. Modelo del sistema mostrado en la Figura 2.47.
- 2.31. Librería mecánica en una dimensión (1/5).

- 2.32. Librería mecánica en una dimensión (2/5).
- 2.33. Librería mecánica en una dimensión (3/5).
- 2.34. Librería mecánica en una dimensión (4/5).
- 2.35. Librería mecánica en una dimensión (5/5).
- 3.1. Fuente de líquido y dos depósitos de la Figura 3.6.
- 3.2. Péndulo plano en coordenadas rectangulares.
- 4.1. Modelo con disparo simultáneo de dos eventos.
- 4.2. Modelo para ilustrar la detección de los eventos en Dymola.
- 4.3. Caída vertical y rebote contra el suelo de una pelota.
- 4.4. Modelo con chattering de la caída vertical y rebote de una pelota.
- 4.5. Circuito de la Figura 4.26.
- 4.6. Circuito rectificador de la Figura 4.29 con diodo ideal.
- 4.7. Dos depósitos conectados mediante una válvula.
- 4.8. Rebote de la pelota con dos fases.
- 4.9. Librería *libFriccion* (1/3).
- 4.10. Librería *libFriccion* (2/3).
- 4.11. Librería *libFriccion* (3/3).
- 4.12. Pared con tres capas de la Figura 4.41.
- 4.13. Flujo de calor en una pared (material B dividido en N_{elemB} capas).
- 4.14. Ejemplo de inicialización de un modelo de tiempo continuo.
- 4.15. Tres formas alternativas de inicializar el modelo del péndulo.
- 4.16. Curva característica de la bomba de líquido (1/3).
- 4.17. Curva característica de la bomba de líquido (2/3).
- 4.18. Curva característica de la bomba de líquido (3/3).
- 4.19. Sistema de los cuatro tanques (1/2).
- 4.20. Sistema de los cuatro tanques (2/2).
- 4.21. Modelo de un gas perfecto en un recipiente calefactado.
- 4.22. Planta SISO y controlador con histéresis (1/2).
- 4.23. Planta SISO y controlador con histéresis (2/2).
- 4.24. Vaciado de un tanque a través de una tubería (1/4).
- 4.25. Vaciado de un tanque a través de una tubería (2/4).
- 4.26. Vaciado de un tanque a través de una tubería (3/4).
- 4.27. Vaciado de un tanque a través de una tubería (4/4).
- 5.1. Transmisión de calor en una varilla rectangular.
- 5.2. Transmisión de calor en una superficie cuadrada.
- 5.3. Cuerda homogénea fija en sus extremos.
- 5.4. Cuerda homogénea fija sólo en un extremo.
- 5.5. Condensador 2D relleno con 2 dieléctricos.
- 5.6. Cable coaxial de sección exterior rectangular.

- 5.7. Conducción de calor en un cuadrado metálico.
- 5.8. Cable coaxial de sección exterior circular.
- 5.9. Barra de hierro sometida a un salto escalón en la temperatura.
- 5.10. Barra de hierro del Ejercicio 5.5.
- 5.11. Conducción eléctrica y térmica en una barra de cobre.
- 5.12. Flujo de calor a través de una capa esférica.
- 5.13. Cilindro dieléctrico.

PREFACIO

El avance producido en las últimas décadas en las tecnologías del software y del hardware, y en los métodos matemáticos, así como el abaratamiento del hardware, han favorecido la popularización del uso del modelado y la simulación en todas las áreas de la Ciencia y la Ingeniería.

Así, durante la década de 1980 se desarrollaron algoritmos numéricos eficientes para la resolución de modelos dinámicos de ecuaciones algebraico diferenciales, lo que sentó la base para la aparición en la década siguiente de los lenguajes de modelado orientado a objetos de sistemas físicos. Los avances en la capacidad de computación permitieron el desarrollo en la década de 1990 del modelado de sistemas complejos basado en agentes y su popularización en la década de 2000. Los avances en computación paralela y visualización han facilitado la simulación de modelos dinámicos en derivadas parciales en dos y tres dimensiones espaciales, que hoy en día son empleados rutinariamente en áreas como la Mecánica de Fluidos, donde se emplea software de CFD (Computational Fluid Dynamics) para simular el flujo de fluidos y la transferencia de calor, la Física Fundamental, la Física de los Materiales, del Aire, etc. Estos mismos avances han favorecido asimismo la evolución producida desde los lenguajes de simulación de eventos discretos desarrollados en la década de 1970, hasta los entornos de simulación actuales, que facilitan la simulación y análisis de modelos estocásticos de eventos discretos extraordinariamente complejos de sistemas logísticos de fabricación, almacenamiento y distribución.

Todo ello ha conllevado una profunda transformación en la forma de trabajo de científicos e ingenieros, los cuales hoy en día comúnmente pasan más tiempo trabajando con el ordenador que experimentando en un laboratorio físico tradicional. Así, la experimentación con modelos matemáticos, que son resueltos con el ordenador aplicando métodos numéricos, es práctica cotidiana en áreas tecnológicas tales como las aplicaciones aeroespaciales, el desarrollo de nuevos materiales y de nuevos fármacos, el diseño de circuitos integrados, la nanotecnología, el control de procesos y el diseño industrial, por citar sólo algunas de ellas.

Existe abundante software de modelado y simulación que puede ser usado casi como una “caja negra”. Sin embargo, los métodos de análisis científico y de diseño en ingeniería están cambiando continuamente, afectando no sólo a la manera de estudiar los fenómenos, sino también al rango de aplicaciones abordadas. Trabajar en nuevas áreas de aplicación requiere de un buen conocimiento de los fundamentos del modelado y la simulación. Dicho conocimiento también es indispensable para dominar rápidamente el manejo de las nuevas herramientas que están continuamente apareciendo.

En modelado y simulación, el comportamiento de interés del sistema bajo estudio es representado mediante un modelo matemático, compuesto típicamente de ecuaciones y algoritmos. Para poder experimentar con el modelo, es necesario traducirlo a software que pueda ser ejecutado en un ordenador. El resultado de la simulación es un conjunto de datos que frecuentemente es voluminoso y debe ser representado gráficamente para poder ser interpretado.

Como una primera aproximación al planteamiento de qué conocimientos y habilidades en modelado y simulación necesita poseer un investigador o ingeniero, examinemos algunos de los requisitos que se derivan de los pasos seguidos en un estudio de simulación.

1. El primer paso es seleccionar la representación adecuada del sistema bajo estudio, realizando hipótesis adecuadas al problema que se desea resolver y planteando las ecuaciones y algoritmos del modelo.
2. El segundo paso es emplear los métodos numéricos adecuados para la resolución del modelo mediante el ordenador. Los algoritmos de los métodos numéricos no son específicos de una determinada disciplina. Los métodos numéricos para la resolución de ecuaciones en derivadas parciales, los sistemas de ecuaciones algebraico diferenciales o las ecuaciones diferenciales ordinarias han encontrado aplicación en todas las disciplinas de la ingeniería, así como en la física, la geología y otros campos.
3. La tercera tarea es realizar los cálculos de manera eficiente en el mundo en constante evolución de la computación. La eficiencia en la computación repercute en el grado posible de complejidad en los modelos y por tanto en el realismo en la representación de los sistemas, y en la utilidad que los resultados pueden tener en la aplicación abordada.
4. La cuarta tarea es evaluar la precisión de los resultados cuando no es posible obtener confirmación a partir de experimentos físicos. La precisión de la respuesta numérica predicha es un aspecto importante en la simulación, ya que

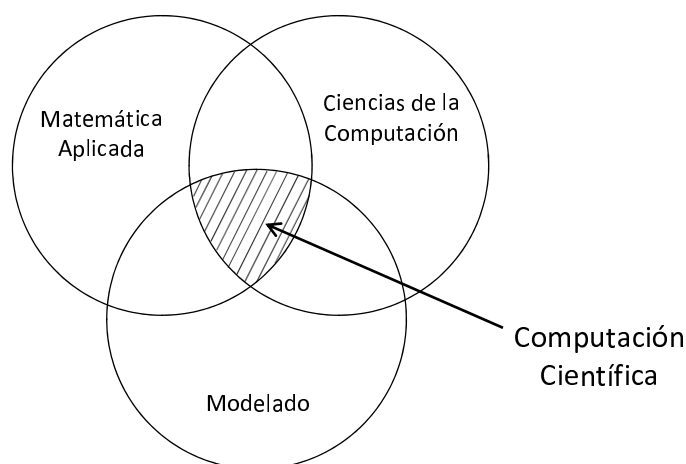


Figura 1: La computación científica está en la intersección de la matemática aplicada, las ciencias de la computación y el modelado. Por ello, el especialista en modelado y simulación necesita una sólida educación interdisciplinar.

el comportamiento observado en la simulación puede diferir significativamente del comportamiento del modelo, estando fuertemente influido por la forma en que se han discretizado las ecuaciones y por el algoritmo numérico empleado.

5. El análisis de los datos obtenidos al ejecutar la simulación completa el ciclo del proyecto de simulación. Entre los procedimientos más potentes de análisis se encuentra la inspección visual de la representación gráfica de los datos, para lo cual debe emplearse software específico de visualización.

La descripción anterior de los pasos típicos en un estudio de simulación pone de manifiesto que el modelado y la simulación requiere de conocimientos interdisciplinarios: requiere conocer el software y el hardware del ordenador, el ámbito científico o tecnológico en que se encuentra el sistema bajo estudio, y también los métodos numéricos. Como se representa en la Figura 1, la computación científica, que constituye la esencia de la simulación, se encuentra en la intersección entre la matemática numérica, la ciencia de la computación y el modelado. Surge por tanto la necesidad de integrar estos conocimientos, a fin de proporcionar la formación necesaria al científico o ingeniero en simulación.

El propósito de este texto es proporcionar una introducción didáctica y autocontenida, adecuada para su uso en el contexto de la educación a distancia, de los métodos de simulación y modelado. Con el propósito de que la lectura de la obra resulte amena y hacer evidente la aplicación de los conceptos expuestos, hemos incluido gran cantidad de ejemplos completamente desarrollados.

La selección del contenido se ha realizado de manera que se aborden los tres pasos fundamentales que van:

1. Desde el problema del mundo real, hasta su formulación como un modelo matemático.
2. Desde el modelo matemático, hasta su simulación con el ordenador.
3. Desde los datos obtenidos de la simulación, hasta la visualización y análisis de los mismos.

El planteamiento de modelos matemáticos útiles para la resolución de problemas reales es una disciplina compleja, que requiere de sólidos conocimientos y experiencia, y que tiene cierta componente de arte. En este texto se plantean modelos sencillos, presentados mediante ejemplos, en los dominios eléctrico, mecánico, de flujo de fluidos e intercambio de calor. Se emplea la descripción del flujo y almacenamiento de la energía como nexo común a todos los dominios, mostrando las analogías existentes entre ellos.

La selección de los tipos de modelos matemáticos considerados en el texto se ha realizado primando la amplitud frente a la profundidad. Así, se aborda la simulación de modelos estáticos, de tiempo discreto, de eventos discretos, de tiempo continuo e híbridos. Por ser conceptualmente más compleja, la descripción y simulación de los modelos de tiempo continuo representados mediante ecuaciones algebraicas y diferenciales (DAE), de los modelos híbridos y de los modelos en derivadas parciales (PDE) ocupa la mayor parte de la obra. Los modelos híbridos considerados están compuestos de sistemas de ecuaciones algebraico diferenciales y de eventos (DAE híbrido).

La simulación es una disciplina eminentemente aplicada. Consideramos por ello necesario ofrecer al lector conocimiento práctico sobre el manejo de herramientas y lenguajes de modelado. Con este fin hemos seleccionado lenguajes y herramientas que son representativas del estado del arte y que el lector pueda usar gratuitamente, bien porque son gratuitas, o bien porque aun siendo comerciales existen versiones gratuitas con capacidades limitadas.

Hemos escogido el lenguaje Modelica para la descripción de los modelos DAE híbridos. Se trata de un lenguaje de modelado orientado a objetos, de propósito general, que es ampliamente usado tanto en el ámbito académico como industrial. Se introduce el uso de dos entornos de modelado de Modelica: el entorno comercial Dymola (del cual existe una versión gratuita con capacidades limitadas) y el entorno de modelado gratuito OpenModelica. También se explican los fundamentos del entorno de modelado Rand Model Designer, el cual ofrece más flexibilidad que Modelica para la descripción de modelos híbridos con estructura variable y además

permite definir de manera rápida, simple y con poco esfuerzo interfaces de usuario interactivas para las simulaciones. La herramienta escogida para la simulación de modelos en derivadas parciales ha sido FlexPDE, por tratarse de una herramienta de propósito general y por disponer de un lenguaje para la descripción de los modelos.

Las herramientas anteriores permiten realizar algunos análisis de los datos obtenidos como resultado de la simulación, incluyendo su representación gráfica. Los análisis más elaborados deben realizarse exportando los datos a herramientas especializadas. Conscientes de que el análisis y visualización de los datos es un mundo es sí mismo, ofrecemos al lector una breve introducción al lenguaje de análisis de datos R. Se trata de un lenguaje gratuito, ampliamente usado, que permite realizar eficientemente análisis complejos de grandes volúmenes de datos.

ORGANIZACIÓN DE LA UNIDAD DIDÁCTICA

La Unidad Didáctica está estructurada en cinco temas.

En el **Tema 1** se introducen conceptos básicos del modelado y la simulación, y se describen los pasos de que habitualmente consta un estudio de simulación. Se describen también las características básicas de los siguientes tipos de modelos y sus simuladores: modelos de tiempo discreto, modelos de eventos discretos, autómatas celulares, modelos basados en agentes, modelos dinámicos en ecuaciones diferenciales ordinarias (ODE), modelos dinámicos híbridos y modelos de ecuaciones en derivadas parciales (PDE). Finalmente, se muestra cómo emplear el lenguaje R para realizar programas sencillos y representaciones gráficas sencillas de los datos.

En el **Tema 2** se describen los fundamentos del paradigma del modelado físico, del modelado modular y jerárquico, y del modelado orientado a objetos. Se ofrece también una introducción al lenguaje Modelica, describiéndose con qué motivación fue desarrollado y cuáles son sus principales capacidades, y a dos de sus entornos de modelado: Dymola y OpenModelica. Se describen las transformaciones sobre el modelo que realizan los entornos de modelado que soportan los lenguajes de modelado orientado a objetos.

También en el Tema 2 se explican conceptos básicos del modelado basado en la descripción del flujo de energía, mostrando las analogías entre las leyes de la física en los dominios eléctrico, mecánico, hidráulico y térmico. Se plantean modelos sencillos en dichos dominios y se muestra cómo describirlos en Modelica y simularlos. Las explicaciones sobre el lenguaje Modelica ofrecidas en este tema son suficientes para desarrollar librerías de modelos de tiempo continuo.

El **Tema 3** está dedicado a la simulación de modelos de tiempo continuo. Se describe cómo analizar si un sistema de ecuaciones algebraico diferenciales (DAE) es estructuralmente singular, cómo calcular y reducir su índice, y cómo asignar su causalidad computacional. Se muestra cómo plantear el algoritmo de la simulación de sistemas DAE sencillos y se abordan distintas cuestiones relacionadas con la simulación de este tipo de sistemas, como la resolución de sistemas DAE con índice superior, el análisis y solución de los lazos algebraicos, la selección de las variables de estado, y los métodos de integración de sistemas ODE y DAE.

En el **Tema 4** se aborda la descripción y simulación de modelos híbridos, que son aquellos que combinan el comportamiento de tiempo continuo con los eventos. Se describe un formalismo para la descripción de modelos híbridos y se muestra la relación entre dicho formalismo con el algoritmo de la simulación y con la descripción en lenguaje Modelica. Se describen los fundamentos de la simulación de modelos híbridos, incluyendo los procedimientos para la detección de los eventos, y para la gestión de la ejecución de eventos simultáneos y de las cadenas de eventos, y se muestra el procedimiento que emplean los entornos de modelado de Modelica para la detección y ejecución de los eventos, y para la simulación de los modelos con estructura variable. Las explicaciones dadas en este tema permiten desarrollar modelos híbridos en lenguaje Modelica y experimentar con ellos empleando el lenguaje de comandos de Modelica. Finalmente, se describen algunas de las capacidades del entorno de modelado Rand Model Designer para el modelado y simulación de modelos híbridos, y se comparan dichas capacidades con las ofrecidas por Modelica.

En el Tema 1 se abordaron algunos aspectos básicos de la simulación de PDE, como es por ejemplo reconocer y clasificar las PDE. También se introdujo el método de las líneas (MOL), que consiste en resolver las PDE transformándolas primeramente en un sistema ODE equivalente y a continuación aplicar métodos de resolución de sistemas ODE. En el **Tema 5** se profundiza en los métodos de simulación de los modelos en PDE. Se introducen las características principales de los campos escalares y vectoriales, y de los operadores diferenciales gradiente, divergencia, rotacional y laplaciano. Se describe qué son las condiciones iniciales y de frontera, y cuáles son las características básicas de los métodos de diferencias finitas y de elementos finitos.

Asimismo, en el Tema 5 se muestra cómo emplear el entorno de simulación FlexPDE para obtener la solución de sistemas de PDEs de una y de dos dimensiones, con condiciones iniciales y condiciones de contorno. En concreto se aplica el entorno FlexPDE para resolver ecuaciones de transferencia de calor, de ondas y de Poisson. Finalmente, se explica cómo derivar la ecuación de transferencia de calor en una

dimensión, la ecuación de ondas homogéneas en una dimensión y la ecuación de Poisson a partir de principios físicos.

CÓMO UTILIZAR EL LIBRO

Si la Unidad Didáctica va a emplearse como texto base en una asignatura en la cual se abordan conceptos tanto de modelado como de simulación, sería recomendable estudiar los temas en el orden en que aparecen en el texto. No obstante, una vez estudiados los dos primeros temas, es posible estudiar los Temas 3, 4 y 5 en cualquier orden.

También es posible emplear la Unidad Didáctica como texto base en dos asignaturas diferentes, en las cuales se aborden por separado los aspectos relacionados con el modelado de sistemas dinámicos y los relacionados con la simulación de sistemas. En este caso, se recomienda (véase la Figura 2):

- Emplear los Temas 1, 2 y 4 en la asignatura de modelado.
- Emplear los Temas 1, 3 y 5 en la asignatura de simulación.

Se recomienda al lector que instale en su propio ordenador algún entorno para el lenguaje R, algún entorno de modelado de Modelica (por ejemplo, Dymola u OpenModelica), y el entorno de simulación FlexPDE, y que los emplee para realizar por sí mismo los ejemplos y ejercicios planteados en el texto. En los Temas 1, 2 y 5 se dan orientaciones para la instalación y manejo de este software.

OBJETIVOS DOCENTES

Se plantea como objetivo docente fundamental que el alumno adquiriera las capacidades siguientes:

1. Discutir metodologías y herramientas software para el modelado matemático y la simulación por ordenador en el ámbito de la Ciencia y la Ingeniería.
2. Discutir los conceptos fundamentales, métodos numéricos y algoritmos para la simulación de modelos de tiempo discreto, tiempo continuo, eventos discretos e híbridos, de autómatas celulares, de modelos basados en agentes y de modelos en ecuaciones en derivadas parciales.

3. Plantear modelos sencillos, describirlos formalmente, codificarlos usando un lenguaje de modelado, verificarlos, validarlos y simularlos usando herramientas software de modelado y simulación, extraer conclusiones y documentarlas.

Al comienzo de cada tema se indican los objetivos docentes que deben alcanzarse una vez completado el estudio del tema y realizados los ejercicios prácticos de autocomprobación propuestos en el tema. Un buen procedimiento de autoevaluación consiste en repasar los objetivos docentes una vez completado el estudio del tema, analizando en cada caso si se ha alcanzado o no el objetivo.

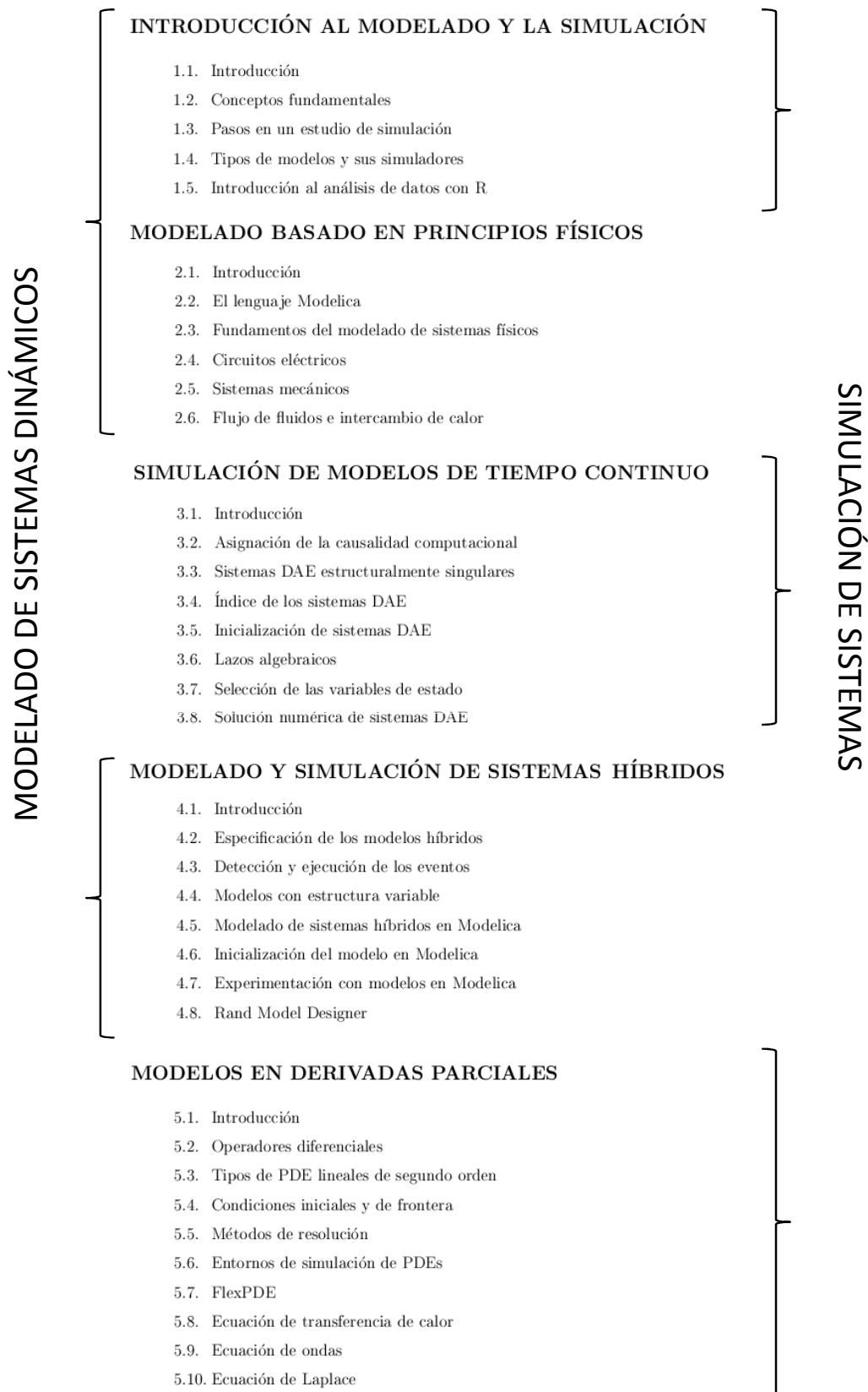


Figura 2: Propuesta de distribución del temario en dos asignaturas, una más directamente relacionada con el modelado de sistemas dinámicos y otra con la simulación de sistemas.

TEMA 1

INTRODUCCIÓN AL MODELADO Y LA SIMULACIÓN

- 1.1. Introducción
- 1.2. Conceptos fundamentales
- 1.3. Pasos en un estudio de simulación
- 1.4. Tipos de modelos y sus simuladores
- 1.5. Introducción al análisis de datos con R
- 1.6. Lecturas recomendadas
- 1.7. Ejercicios de autocomprobación
- 1.8. Soluciones de los ejercicios

OBJETIVOS DOCENTES

Una vez estudiado el contenido del tema y realizados los ejercicios prácticos, debería saber:

- Discutir el significado de conceptos básicos del modelado y la simulación.
- Describir y comparar las diferentes formas de estudiar un sistema y los diferentes tipos de modelos.
- Comparar y reconocer los distintos tipos de modelos matemáticos. Discutir y saber aplicar diferentes clasificaciones de los modelos matemáticos.
- Discutir una clasificación de los niveles en el conocimiento de los sistemas.
- Discutir un marco formal para el modelado y la simulación.
- Discutir los pasos de que típicamente consta un estudio de simulación.
- Discutir las características básicas de los siguientes tipos de modelos y sus simuladores: modelos de tiempo discreto, modelos de eventos discretos, autómatas celulares, modelos basados en agentes, modelos en ecuaciones diferenciales ordinarias, modelos híbridos y modelos en derivadas parciales.
- Emplear el lenguaje R para realizar programas sencillos y representaciones gráficas sencillas de los datos.

1.1. INTRODUCCIÓN

En este primer tema se definen algunos conceptos fundamentales como son *modelo*, *sistema*, *experimento* y *simulación*. Se describen cuatro tipos diferentes de modelos, entre los cuales se encuentra el empleado en simulación, que es el *modelo matemático*.

En la teoría de sistemas se diferencian dos aspectos fundamentales, ortogonales entre sí. Por una parte está el *conocimiento* del que se dispone acerca del comportamiento de un sistema. Por otra parte está la *especificación* de dicho conocimiento, que puede realizarse empleando uno u otro tipo de modelo matemático.

En este tema se muestra una clasificación en cuatro niveles del conocimiento que puede poseerse acerca de un sistema. Esta clasificación facilita la definición de las tres actividades fundamentales relacionadas con los sistemas: el *análisis*, la *inferencia* y el *diseño*.

Se describe asimismo un marco formal para el modelado y la simulación, en el cual se establecen unas entidades, como son el *sistema fuente*, el *modelo* y el *simulador*, y también la relación entre ellas. De este marco formal se deriva de forma natural la definición de conceptos básicos como son *marco experimental*, *simulador*, *validez del modelo*, *corrección del simulador*, *complejidad del modelo* y *simplificación del modelo*, entre otros.

Se explican también los pasos de los que típicamente consta un estudio de simulación. Estos pasos comprenden la definición del problema, la planificación del proyecto, la definición del sistema, las diferentes etapas en la definición del modelo y del experimento, la experimentación con el modelo, el análisis e interpretación de los resultados, y la documentación de las conclusiones.

A continuación se muestran varias posibles clasificaciones de los modelos matemáticos. También se describen las características básicas de los siguientes tipos de modelos y sus simuladores: modelos de tiempo discreto, modelos de eventos discretos, autómatas celulares, modelos basados en agentes, modelos en ecuaciones diferenciales ordinarias, modelos híbridos y modelos en derivadas parciales.

Finalmente, se introduce el uso de un lenguaje para el análisis de datos denominado *lenguaje R*. El análisis de los datos es una parte esencial en el planteamiento del modelo matemático y también en la toma de decisiones basada en los resultados de la simulación de los modelos.

1.2. CONCEPTOS FUNDAMENTALES

Una forma de aproximarse a una disciplina es entendiendo el significado de los términos que describen sus conceptos fundamentales. Con este propósito se explica a continuación el significado de términos que describen conceptos fundamentales en el ámbito del modelado y la simulación.

1.2.1. Sistema, experimento y modelo

El modelado matemático y la simulación por ordenador son una forma de adquirir conocimiento acerca del comportamiento de los sistemas. En este contexto, se considera que un **sistema** es *cualquier objeto cuyas propiedades se desean estudiar*. De acuerdo con esta definición, *cualquier fuente potencial de datos* es un sistema.

Una manera de conocer el comportamiento de un sistema es experimentar con él. De hecho, éste ha sido el método empleado durante siglos para avanzar en el conocimiento: plantear preguntas acerca del comportamiento de los sistemas y responderlas mediante experimentación. Un **experimento** es *el proceso de extraer datos de un sistema sobre el cual se ha ejercido una acción externa*.

Experimentar directamente con el sistema real presenta indudables ventajas. Sin embargo, en ocasiones no es posible. Quizá el motivo más evidente es que el sistema real aún no exista físicamente. Esto sucede por ejemplo en la fase de diseño de nuevos sistemas, cuando el ingeniero necesita predecir el comportamiento de los mismos antes de que sean construidos. Otro motivo es que la escala de tiempo del experimento lo haga inviable. Esto es habitual en los estudios geológicos, cosmológicos, de evolución de los ecosistemas y sociológicos, en los cuales los experimentos deberían tener una duración de cientos, miles o millones de años.

Incluso siendo posible experimentar directamente con el sistema real, en ocasiones es desaconsejable debido a su elevado coste económico. Otro posible motivo para no experimentar con el sistema real es que el experimento produzca perjuicio, incomodidad o sea peligroso.

Una alternativa a la experimentación con el sistema real consiste en realizar un modelo del sistema y experimentar con el modelo. En el sentido amplio del término, un **modelo** es *una representación de un sistema desarrollada para un propósito específico*.

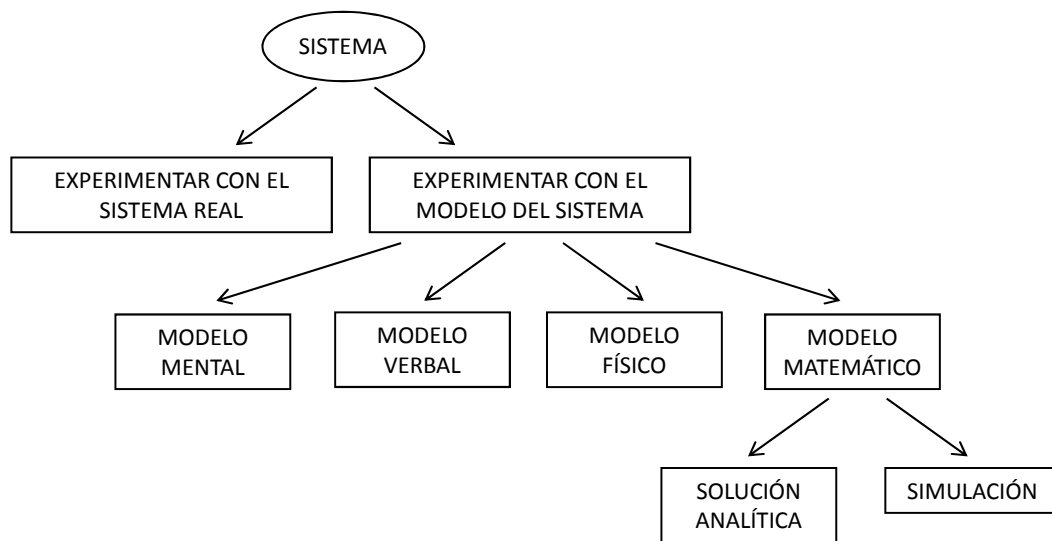


Figura 1.1: Formas de estudiar un sistema.

Experimentar con un modelo resulta en ocasiones menos costoso y más seguro que experimentar directamente con el sistema real. Otra ventaja de la experimentación con modelos es que, con un modelo adecuado, se pueden ensayar condiciones de operación extremas que pueden ser impracticables en el sistema real. Este es el caso cuando el experimento requiere modificar variables que en el sistema real o bien no están accesibles, o bien no pueden ser modificadas en el rango requerido.

Resumiendo lo anterior, en la Figura 1.1 se muestran las dos formas de conocer el comportamiento de un sistema: experimentando con el sistema real y experimentando con un modelo del sistema. Obsérvese también que en la figura se señalan cuatro tipos diferentes de modelo: mental, verbal, físico y matemático.

Al primer tipo, el **modelo mental**, pertenecen los modelos que empleamos continuamente en nuestra vida cotidiana para comprender y predecir el comportamiento de los sistemas. Considerar que alguien es “amable” constituye un modelo del comportamiento de esa persona. Este modelo nos ayuda a responder, por ejemplo, a la pregunta de cómo reaccionará si le pedimos un favor. También, mediante la intuición y la experiencia, desarrollamos modelos mentales de los sistemas técnicos. Por ejemplo, aprender a conducir un coche consiste parcialmente en desarrollar un modelo mental de las propiedades de la conducción del coche. Asimismo, un operario trabajando en determinado proceso industrial sabe cómo el proceso reacciona ante diferentes acciones: el operario, mediante el entrenamiento y la experiencia, ha desarrollado un modelo mental del proceso.

Otro tipo de modelo es el **modelo verbal**, en el cual el comportamiento del sistema es descrito mediante palabras. Por ejemplo, un modelo mental es: si se aprieta el freno, entonces la velocidad del coche se reduce. Los sistemas expertos son ejemplos de modelos verbales formalizados. Es importante diferenciar entre los modelos mentales y los verbales. Por ejemplo, usamos un modelo mental de la dinámica de la bicicleta cuando la conducimos. Sin embargo, no es sencillo convertirlo en un modelo verbal.

Además de los modelos mentales y verbales, existe otro tipo de modelos que tratan de imitar al sistema real. Son los **modelos físicos**, tales como las maquetas a escala que construyen los arquitectos, diseñadores de barcos o aeronaves, para comprobar las propiedades estéticas, aerodinámicas, etc.

Finalmente, el cuarto tipo de modelo es el **modelo matemático**. En los modelos matemáticos las relaciones entre las magnitudes de interés del sistema (distancias, velocidades, flujos, etc.) son descritas mediante relaciones matemáticas. La mayoría de las teorías sobre las Leyes de la Naturaleza son descritas empleando modelos matemáticos. Igualmente, en todas las ramas de la Ingeniería se emplean modelos matemáticos para describir el funcionamiento de los sistemas naturales y artificiales.

En algunos casos, las relaciones matemáticas que constituyen los modelos son sencillas y puede encontrarse una **solución analítica** del modelo. Sin embargo, en la mayoría de los casos los modelos no pueden resolverse analíticamente y deben estudiarse con ayuda del ordenador, aplicando métodos numéricos. Este *experimento numérico realizado sobre el modelo matemático* recibe el nombre de **simulación**. Éste es precisamente el objeto de estudio en este texto: los modelos matemáticos y su resolución mediante simulación empleando el ordenador.

1.2.2. Niveles en el conocimiento de los sistemas

En la sección anterior se ha hecho una breve introducción a los diferentes tipos de modelos matemáticos. En esta sección se ofrece una clasificación del conocimiento que puede poseerse de un sistema. Obsérvese que son dos aspectos ortogonales entre sí. El tipo de modelo más adecuado en cada caso no depende del sistema en sí, ni del nivel de conocimiento que se posea sobre él, sino de de las preguntas a las que se pretende contestar mediante la experimentación con el modelo.

Existen varias formas de clasificar el conocimiento que puede poseerse de un sistema. A principios de la década de 1970, G.J. Klir propuso una clasificación del conocimiento en cuatro niveles, que resulta muy adecuada en el ámbito del modelado

y la simulación. A continuación se indica qué tipo de conocimiento se posee acerca del sistema en cada nivel, de acuerdo con esta clasificación. A medida que se asciende de nivel, se conocen aspectos importantes del sistema que no se conocen en los niveles inferiores del conocimiento.

- **Nivel 0 - Fuente.** En este nivel identificamos la porción del mundo real a modelar y las maneras mediante las cuáles vamos a observarlo. Dicha porción del mundo real, que es nuestra fuente de datos, se denomina sistema fuente.
- **Nivel 1 - Datos.** En este nivel disponemos de una base de datos de medidas y observaciones de nuestro sistema fuente.
- **Nivel 2 - Generación.** En este nivel somos capaces de recrear estos datos usando una representación más compacta. Por ejemplo, mediante fórmulas matemáticas o algoritmos. Dado que un mismo conjunto de datos puede ser generado empleando diferentes fórmulas, algoritmos u otros procedimientos, haber determinado el procedimiento para reproducir los datos es un conocimiento que no teníamos al Nivel 1 (datos).
- **Nivel 3 - Estructura.** En este último nivel sabemos cómo recrear los datos observados en el Nivel 1 (datos) de una manera específica: en términos de componentes interconectados entre sí. Esto implica un conocimiento acerca de la estructura interna del sistema.

Los modelos constituyen una formalización del conocimiento en los Niveles 2 (generación) y 3 (estructura). Por otra parte, esta clasificación permite definir en qué consisten los tres tipos básicos de problemas relacionados con el conocimiento de los sistemas: análisis, inferencia y diseño. Cada uno de estos problemas implica un determinado movimiento entre los niveles de conocimiento del sistema.

- Al realizar el **análisis** de un sistema se intenta comprender el comportamiento del sistema, existente o hipotético, empleando para ello el conocimiento que se tiene de su estructura.
- En la **inferencia** sobre un sistema se intenta conocer la estructura del sistema a partir de las observaciones que pueden realizarse del mismo.
- En el **diseño** de un sistema se investigan diferentes estructuras alternativas para un sistema completamente nuevo o para el rediseño de uno ya existente.

Cuando nos movemos hacia niveles inferiores del conocimiento, como sucede en el caso del análisis de sistemas, no estamos generando conocimiento nuevo. Estamos únicamente haciendo explícito lo que ya está implícito en la descripción que tenemos. Por el contrario, la inferencia y el diseño son problemas que requieren ascender en los niveles de conocimiento. Al hacer inferencia disponemos de una base de datos del comportamiento del sistema fuente y tratamos de encontrar una representación del conocimiento al Nivel 2 (generación) o al Nivel 3 (estructura), que nos permita recrear los datos de que disponemos. Este proceso se denomina **construcción del modelo**.

En el caso de la inferencia, el sistema fuente existe. Sin embargo, en el caso del diseño el sistema fuente no existe y el objetivo es construir un sistema que se comporte de la manera deseada. Si el objetivo es llegar a construir el sistema, debe llegarse a un Nivel 3 (estructura) del conocimiento, puesto que la construcción se realizará mediante la interconexión de diferentes componentes tecnológicos.

Finalmente, el proceso denominado **ingeniería inversa** tiene elementos tanto de inferencia como de diseño. Para hacer ingeniería inversa de un sistema existente, en primer lugar se realiza un gran número de observaciones de él. A partir de estas observaciones, se infiere el comportamiento del sistema y se diseña una estructura alternativa que tenga ese comportamiento.

1.2.3. Marco formal para el modelado y la simulación

En esta sección se presenta un marco formal para el modelado matemático y la simulación, en el cual se definen cinco entidades (sistema fuente, base de datos del comportamiento, modelo, simulador y marco experimental) y dos relaciones entre ellas (relación de modelado y relación de simulación). En la Figura 1.2 se han representado esquemáticamente las cinco entidades y las dos relaciones. Se describen a continuación.

El **sistema fuente** es el entorno real o virtual que estamos interesados en modelar, el cual constituye una *fente de datos observables*, en la forma de trayectorias (observaciones indexadas en el tiempo) de variables. Esta entidad es conocida en el Nivel 0 (fuente) de conocimiento del sistema.

Los datos que se han recogido a partir de observaciones o experimentando con el sistema se denominan la **base de datos del comportamiento** del sistema. Estas observaciones son características del conocimiento en el Nivel 1 (datos). Los datos son observados o adquiridos a través de marcos experimentales de interés

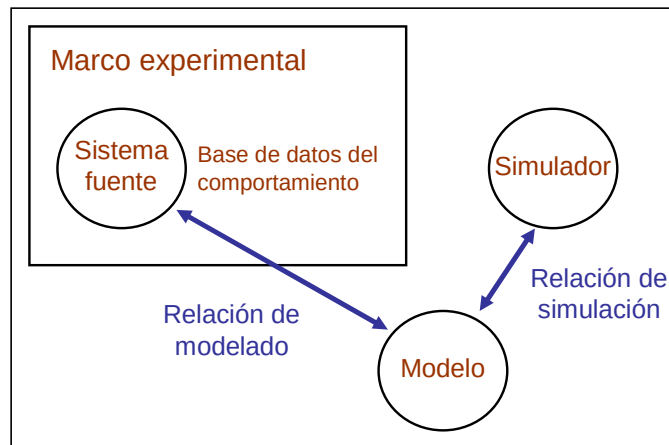


Figura 1.2: Entidades básicas del modelado y simulación, y su relación.

para quien realiza el modelo. Un **marco experimental** es una especificación de las condiciones bajo las cuales el sistema es observado o se experimenta con él. Puesto que el marco experimental *establece el conjunto de experimentos para los cuales el modelo es válido*, es la formulación operacional de los objetivos que motivan un proyecto de modelado y simulación.

Es importante establecer lo antes posible en el proceso de desarrollo del modelo cuáles son los objetivos del estudio, ya que los objetivos sirven para enfocar el modelo en aquellos aspectos del sistema que son relevantes para el propósito del estudio. En otras palabras, conocer los objetivos del estudio permite plantear los marcos experimentales adecuados. Los marcos experimentales trasladan los objetivos a condiciones de experimentación más precisas para el sistema fuente y sus modelos.

Una vez fijados los objetivos, presumiblemente existirá un nivel en la especificación del sistema que será el más adecuado para contestar la cuestión planteada. Cuanto más exigentes sean las preguntas, normalmente mayor es la resolución necesaria (nivel en la descripción del modelo) para contestarlas. Por ello, la elección de un nivel de abstracción apropiado repercute en la consecución de los objetivos.

Las otras dos entidades que componen el marco formal son el modelo y el simulador. En el contexto de este marco formal, se entiende que un **modelo** de un sistema es *un conjunto de instrucciones, reglas, ecuaciones o ligaduras para reproducir el comportamiento del sistema*. Se trata de una especificación del conocimiento acerca del comportamiento del sistema al Nivel 2 (generación) o al Nivel 3 (estructura).

Dado que un modelo es un conjunto de instrucciones, reglas, ecuaciones o ligaduras, es necesario disponer de un agente capaz de obedecer las instrucciones y

reglas, y de evaluar las ecuaciones, con el fin de generar el comportamiento descrito en el modelo. Este agente se denomina **simulador**. Así pues, puede definirse un simulador como *cualquier agente computacional (tal como un único procesador, una red de procesadores, la mente humana, o de manera más abstracta, un algoritmo) capaz de ejecutar el modelo para generar su comportamiento*.

La **relación de modelado**, denominada **validez**, se refiere a la relación entre el modelo, el sistema y el marco experimental. A menudo se piensa en la validez como el grado en el cual el modelo representa fielmente al correspondiente sistema. Sin embargo, resulta más práctico requerir que el modelo capture de forma fiel el comportamiento del sistema sólo hasta el punto demandado por los objetivos del estudio de simulación. De esta forma, el concepto de validez responde a la pregunta de si es posible distinguir entre el modelo y el sistema en el marco experimental de interés.

El tipo más básico de validez, la **validez replicativa**, se afirma si, para todos los posibles experimentos del marco experimental, el comportamiento del modelo y del sistema se corresponden dentro de una tolerancia aceptable. Formas más estrictas de validez son la validez predictiva y la validez estructural. En la **validez predictiva** no sólo requerimos validez replicativa, sino también la habilidad de predecir. Finalmente, para que exista **validez estructural** el modelo no sólo debe ser capaz de replicar los datos observados del sistema, sino también reproducir el comportamiento componente a componente.

La **relación de simulación**, denominada la **corrección** del simulador, es la relación entre el simulador y el modelo. Un simulador es correcto si ejecuta sin errores las instrucciones, reglas y ecuaciones que componen el modelo, generando correctamente el comportamiento descrito en el modelo.

La **complejidad** del modelo son los recursos requeridos por un determinado simulador para interpretar el modelo correctamente. Si bien la complejidad se mide relativa a un determinado simulador, a menudo las propiedades intrínsecas al modelo están fuertemente correlacionadas con su complejidad, la cual es prácticamente independientemente del simulador. El modelado exitoso puede verse como *la simplificación válida*. A menudo es necesario reducir la complejidad de los modelos (simplificar los modelos) para facilitar que sean ejecutados en un tiempo razonable en el simulador, de recursos limitados, de que disponemos.

En el proceso de la **simplificación** están implicados dos modelos: el modelo base y el modelo simplificado. El modelo base es “más capaz”, pero requiere más recursos para ser simulado que el modelo simplificado. En este contexto, “más capaz” significa que el modelo base es válido dentro de un conjunto de marcos experimentales (con

respecto a un sistema real) más amplio que el modelo simplificado. Sin embargo, el punto importante es que el modelo simplificado y el modelo base sean igualmente válidos dentro del marco experimental particular de interés.

1.3. PASOS EN UN ESTUDIO DE SIMULACIÓN

En la Figura 1.3 se muestra la secuencia de pasos de que consta típicamente un estudio de simulación. Se ofrecen a continuación algunas recomendaciones, que dan pie a introducir más conceptos básicos.

1.3.1. Definición del objetivo

El primer paso en un estudio de simulación consiste en definir explícitamente el objetivo del estudio. Algunos ejemplos de posibles objetivos son:

- Predecir el comportamiento del sistema bajo cierto conjunto de condiciones.
- Comparar el comportamiento de diferentes diseños alternativos del sistema.
- Analizar la sensibilidad de ciertas medidas del comportamiento del sistema frente a varios factores.
- Calcular qué valores de determinados parámetros del modelo proporcionan un valor óptimo de determinada respuesta.

El propósito del estudio de simulación tiene implicaciones decisivas en el diseño del modelo y del experimento, así como en el procedimiento de validación del modelo. Por ejemplo, si el propósito es evaluar el comportamiento de un sistema en términos absolutos, deberá existir un alto grado de correspondencia entre el comportamiento del modelo y del sistema. Por el contrario, si el propósito del estudio es comparar varios diseños, el modelo puede ser válido en un sentido relativo incluso cuando sus respuestas en un sentido absoluto difieran considerablemente de las del sistema real.

1.3.2. Hipótesis de modelado

Al plantear el modelo debe tenerse presente que la esencia del arte del modelado es identificar el pequeño conjunto de características o propiedades del sistema suficientes para servir los objetivos específicos del estudio.

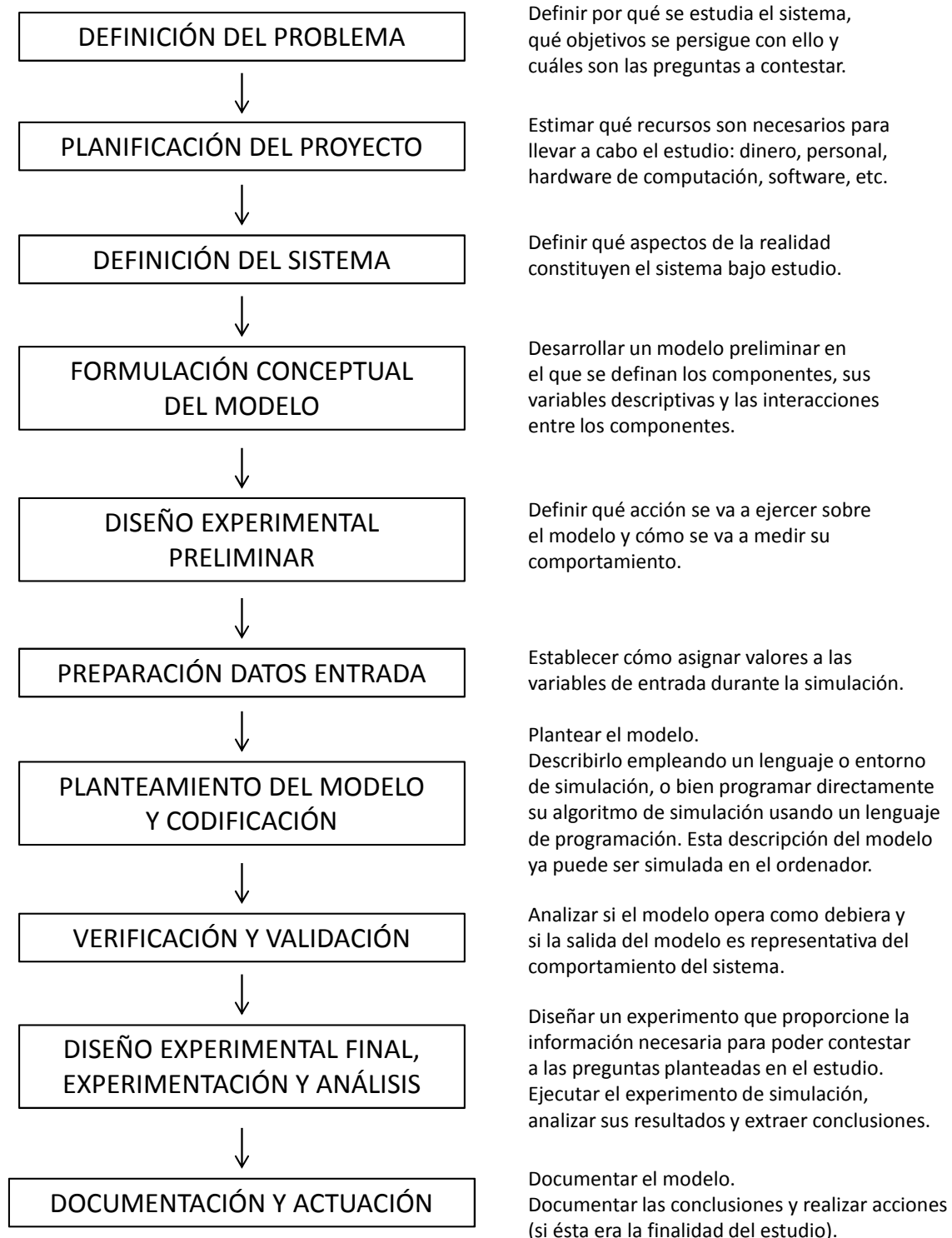


Figura 1.3: Pasos seguidos típicamente al realizar un estudio de simulación.

La tarea de modelado implica la búsqueda de un punto de equilibrio: el modelo debe representar los aspectos del sistema real con el grado de detalle requerido, pero de modo que sea lo más sencillo posible. No debe hacerse un modelo demasiado detallado, que incluya características o propiedades que contribuyan poco o nada a la comprensión del problema.

Una buena práctica consiste en realizar el modelo de manera iterativa: comenzar con un modelo muy simple, cuya complejidad puede posteriormente ir aumentándose fácilmente. Este modelo inicial puede construirse muy rápido y puede servir de punto de discusión sobre posteriores refinamientos en el modelado, entendiéndose por **refinamiento** del modelo *el aumento en su nivel de detalle*.

Añadiendo progresivamente los refinamientos al modelo, y comparando los resultados obtenidos con los del modelo más sencillo, puede estimarse el impacto de cada conjunto de refinamientos sobre la respuesta del modelo. En determinado punto de este proceso de aumento gradual de la complejidad del modelo, los refinamientos añadidos tienen un efecto pequeño, es decir, influyen despreciablemente en las conclusiones del estudio, con lo cual se concluirá que no es preciso incorporarlos.

La idea subyacente es que no todos los factores son igualmente importantes para determinar el comportamiento del sistema. Se trata de decidir qué factores son críticos (efectos de primer orden) y cuáles no (efectos de segundo orden). Omitir los detalles supone introducir deliberadamente “imperfecciones” y por tanto podría considerarse que es una mala práctica. Sin embargo, nada más lejos de la realidad: solamente omitiendo los factores de segundo orden puede construirse un modelo útil. El aumento en el nivel de detalle debe hacerse atendiendo al impacto que la complejidad añadida tiene sobre los resultados del estudio. Es decir, debe emplearse el esfuerzo de modelado en la representación de aquellos aspectos del sistema que más influencia tienen sobre los resultados del estudio.

El impacto de las hipótesis de modelado debe estimarse considerando el propósito específico del estudio de simulación. Abundando en la idea ya expuesta anteriormente, si el propósito es estimar de manera precisa el comportamiento del sistema, deben evaluarse las simplificaciones considerando su impacto sobre la medida del comportamiento. Por otro lado, si el propósito del estudio es comparar diferentes sistemas, debe evaluarse si la simplificación afecta a todos los modelos aproximadamente de la misma manera (los errores son aproximadamente iguales en magnitud). Por este motivo, estimar el comportamiento absoluto de un sistema normalmente requiere modelos considerablemente más detallados que comparar varios sistemas.

1.3.3. Planteamiento del modelo

Existen diferentes metodologías de modelado, las cuales están asociadas al tipo de modelo matemático que vaya a emplearse en el estudio. Una característica común a la mayoría de metodologías de modelado es que promueven que el modelo se desarrolle de manera modular y jerárquica. Es por ello que, sin entrar aún en este punto es detalles acerca de las diferentes metodologías de modelado, se exponen a continuación los fundamentos del diseño modular y jerárquico de modelos.

El **modelado modular** de un sistema se realiza siguiendo básicamente los tres pasos siguientes:

1. Analizar la estructura del sistema e identificar sus distintas partes.
2. Describir la interacción entre las partes.
3. Describir el comportamiento interno de cada parte independientemente de las demás.

Esta estrategia de **análisis por reducción** se basa en suponer que las partes individuales serán más fáciles de entender que el conjunto y que, entendiendo el comportamiento de cada parte y cómo interaccionan entre sí las partes, se puede inferir, y por tanto comprender, el comportamiento del conjunto.

El modelado modular facilita el planteamiento, desarrollo, verificación y validación de los modelos, ya que es más sencillo llevar a cabo estas tareas sobre pequeños submodelos que sobre modelos de grandes dimensiones. Además agiliza el desarrollo de los modelos, al permitir que diferentes especialistas trabajen independientemente en diferentes partes del modelo. Cabe esperar que un modelo construido modularmente sea más fácilmente actualizable si varía la realidad física que representa o las hipótesis bajo las cuales se ha realizado. Asimismo, la modularidad facilita la reutilización de los modelos en diferentes contextos, reduciéndose así los costes del modelado. Conceptos relacionados con la modularidad son la componibilidad y la abstracción.

- La **modularidad** es la capacidad de describir cada una de las partes del modelo de forma independiente.
- La **componibilidad** es la capacidad de describir y realizar el ensamblaje entre las partes del modelo.

- La **abstracción** es la posibilidad de usar cada parte del modelo sin conocer sus detalles internos.

La abstracción es un modo de afrontar el problema de modelar sistemas complejos, al posibilitar que varios especialistas puedan trabajar independientemente en diferentes partes del modelo del sistema sin tener que conocer todos los detalles del resto del mismo.

Una forma de facilitar la abstracción consiste en diferenciar, en cada parte del modelo, entre la interfaz y la descripción interna.

- La **interfaz** es la parte que describe la interacción del modelo con su entorno, a la vez que aísla éste de la descripción interna del modelo, de modo que pueden ser considerados separadamente.
- La **descripción interna** del modelo contiene la información acerca de la estructura y el comportamiento.

El **encapsulado de la información**, consistente en que sólo las variables pertenecientes a la interfaz sean accesibles desde el exterior del módulo por otros módulos (si bien todas las variables pueden ser observadas en la simulación del modelo). La **ocultación de la información** como criterio de diseño de los sistemas modulares facilita las modificaciones, las pruebas y el mantenimiento de los modelos.

A fin de facilitar su diseño y comprensión, el modelo puede, además de modularmente, realizarse de forma **jerárquica**, es decir, progresando de menor a mayor nivel de detalle en su descripción: el modelo se divide en submodelos, que a su vez pueden dividirse en sub-submodelos y así sucesivamente. Esta descripción del modelo en diferentes niveles de detalle facilita además su mantenimiento y reutilización.

La descripción modular y jerárquica del modelo conduce a dos tipos de submodelos: los primitivos y los compuestos.

- Los **submodelos primitivos o atómicos** no están compuestos por otros submodelos y describen mediante ecuaciones el comportamiento de los componentes del proceso.
- Los **submodelos compuestos o moleculares** están constituidos por otros submodelos, primitivos o compuestos, y las conexiones entre ellos.

1.3.4. Diseño de los experimentos

Puede observarse en la Figura 1.3 que el diseño de los experimentos se realiza en dos pasos diferentes del estudio de simulación.

- **Diseño experimental preliminar.** Tan pronto como sea posible, deben seleccionarse qué medidas del comportamiento del sistema van a usarse en el estudio, qué factores van a variarse y qué niveles de cada uno de estos factores van a investigarse. Esta información es relevante para el diseño del modelo.
- **Diseño experimental final.** Una vez el modelo ha sido desarrollado, verificado y validado, se revisa el diseño experimental teniendo en cuenta el conocimiento que se ha ganado durante el proceso de diseño, construcción, verificación y validación del modelo.

1.3.5. Verificación y validación

La verificación y validación del modelo son la comprobación de las relaciones de simulación (corrección) y modelado (validez) definidas en la Sección 1.2.3.

- La **verificación** consiste en comprobar que no se han cometido errores al describir el modelo usando un lenguaje de modelado o al programar el algoritmo de su simulación usando un lenguaje de programación.
- La **validación** consiste en comprobar que el modelo es una aproximación “adecuada” (conforme a los objetivos del estudio de simulación) del sistema.

Si bien la verificación y validación del modelo son análisis conceptualmente distintos, en la práctica aparecen relacionados: cuando los resultados de la simulación parecen “extraños” o erróneos, debe investigarse si es debido a equivocaciones al codificar el modelo o a que las hipótesis de modelado no son adecuadas al objetivo del estudio.

Existen varios **procedimientos de verificación**, entre los cuales cabe destacar los siguientes.

- **Verificar manualmente** la ejecución del algoritmo de la simulación, ejecutando la simulación durante un periodo de tiempo corto y comprobando manualmente los resultados obtenidos.

- **Comprobar submodelo a submodelo**, tratando de verificar individualmente que cada submodelo produce los resultados esperados para todos los posibles tipos de entradas.
- **Comparar con soluciones conocidas**. Para ello se ajusta el modelo de modo que represente un sistema de solución conocida y se compara ésta con los resultados de la simulación.
- **Realizar tests de sensibilidad**. Puede para ello modificarse el valor de un parámetro del modelo, dejando los demás fijos, con el fin de medir la sensibilidad de la respuesta del modelo respecto a ese parámetro. La comparación de la sensibilidad observada en las simulaciones, con la que sería de esperar en el sistema real, puede proporcionar pistas útiles.

La **validación** del modelo es un proceso continuado durante su diseño, desarrollo y uso. La confianza en la validez del modelo va acumulándose según éste va demostrando su utilidad para el propósito específico para el cual ha sido desarrollado. No debe perderse de vista que el objetivo del ingeniero dedicado al modelado es la realización de modelos útiles, en un tiempo razonable y con un coste razonable. Por este motivo, más que preguntarse en qué medida se ajusta el comportamiento simulado al comportamiento real del sistema, es más adecuado preguntarse en qué medida las diferencias entre el modelo y el sistema son lo suficientemente significativas como para afectar a las conclusiones derivadas del uso del modelo.

1.4. TIPOS DE MODELOS Y SUS SIMULADORES

El planteamiento del modelo implica decidir qué tipo de modelo matemático resulta más adecuado para el propósito del estudio. Con el fin de proporcionar una primera visión general de diferentes tipos de modelos matemáticos, se describen a continuación algunas clasificaciones de los mismos. Esto nos permitirá también seguir introduciendo conceptos básicos y terminología. Seguidamente se hará una primera aproximación a los tipos de modelos cuyo planteamiento y simulación se aborda en este texto.

1.4.1. Clasificaciones de los modelos matemáticos

Los modelos matemáticos se clasifican atendiendo a diferentes criterios. Dos criterios ampliamente empleados son que contenga o no variables aleatorias, y que

el tiempo juegue o no un papel relevante. Estos dos criterios dan lugar a las dos clasificaciones siguientes.

– *Determinista o estocástico.*

- Un modelo es **determinista** cuando todas sus variables de entrada son deterministas, lo que equivale a decir que el valor de cada una de ellas es conocido en cada instante. Las **variables de entrada** al sistema son aquellas que describen la interacción que el entorno ejerce sobre el sistema, entendiendo por **entorno** la parte del universo que no pertenece al sistema.
- Por el contrario, se dice que un modelo es **estocástico** cuando alguna de sus variables de entrada es aleatoria. Puesto que las variables del modelo calculadas a partir de variables aleatorias son también aleatorias, el comportamiento de los modelos estocásticos debe ser analizado empleando técnicas estadísticas.

– *Estático o dinámico.*

- Los **modelos dinámicos** son aquellos en los cuales interviene el tiempo.
- Aquellos modelos en los cuales el tiempo no juega ningún papel se denominan **modelos estáticos**.

Las dos clasificaciones anteriores son ortogonales entre sí, ya que se realizan atendiendo a diferentes aspectos del modelo. Así, pueden plantearse modelos estáticos deterministas y modelos estáticos estocásticos, como también pueden plantearse modelos dinámicos deterministas y modelos dinámicos estocásticos.

Por su parte, los **modelos dinámicos** pueden clasificarse, atendiendo a en qué instantes de tiempo pueden cambiar sus variables, en los cuatro tipos indicados a continuación (véase la Figura 1.4).

– *Modelos de tiempo discreto.*

– *Modelos de eventos discretos.*

– *Modelos de tiempo continuo.*

– *Modelos híbridos.*

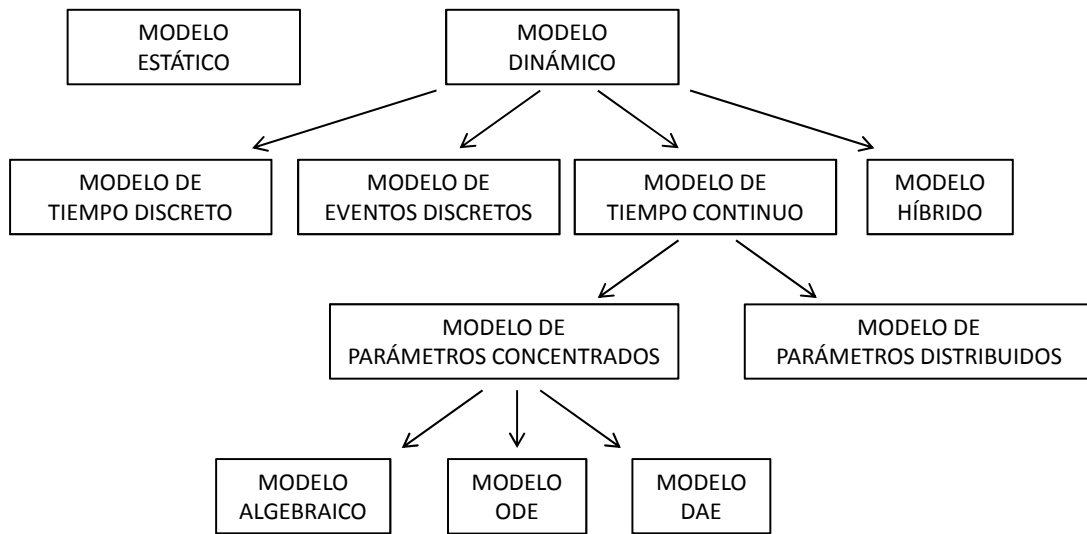


Figura 1.4: Algunas clasificaciones de los modelos matemáticos.

En los dos primeros tipos de modelo, el valor de las variables sólo puede cambiar en instantes específicos, permaneciendo constante el resto del tiempo. A este tipo de variables se las denomina **variables de tiempo discreto**. Los cambios instantáneos en el valor de las variables se denominan **eventos**. Las ecuaciones que permiten calcular en el instante actual el valor de las variables de tiempo discreto, a partir del valor de dichas variables en instantes pasados, se denominan **ecuaciones en diferencias**.

La diferencia entre los modelos de tiempo discreto y de eventos discretos es que en aquellos los eventos se producen en instantes de tiempo predefinidos y a intervalos constantes de tiempo, mientras que en los modelos de eventos discretos estas dos condiciones, en general, no se satisfacen.

Los **modelos de tiempo continuo** se caracterizan por el hecho de que el valor de sus variables puede cambiar de manera continua a lo largo del tiempo. A este tipo de variables se las denomina **variables de tiempo continuo**.

Obsérvese que la clasificación en *variable de tiempo continuo* o *variable de tiempo discreto* se realiza atendiendo a si la variable pueden cambiar o no de manera continua en el tiempo, no atendiendo al conjunto de posibles valores de la variable. Así, tanto las variables de tiempo continuo como las de tiempo discreto pueden ser de tipo real. Sin embargo, las variables de tipo entero, Booleano o cadena de caracteres (*string*) sólo pueden ser de tiempo discreto.

Los **modelos de tiempo continuo** pueden a su vez clasificarse atendiendo a si contienen o no derivadas respecto a las coordenadas espaciales.

- **Modelos de parámetros concentrados**, que están descritos mediante ecuaciones algebraicas y ecuaciones diferenciales ordinarias en las cuales la derivada es únicamente respecto al tiempo.
- **Modelos de parámetros distribuidos**, en los hay ecuaciones en las que hay derivadas respecto a las coordenadas espaciales y también posiblemente respecto al tiempo.

La clasificación anterior de los modelos dinámicos se fundamenta en la siguiente **clasificación de las ecuaciones**.

- Las **ecuaciones algebraicas** son aquellas en las cuales ninguna de las variables aparece derivada.
- Una **ecuación diferencial ordinaria** es aquella en la que aparecen derivadas respecto a una única variable independiente, la cual puede ser el tiempo o alguna coordenada espacial.
- Cuando en una ecuación aparecen derivadas respecto a más de una variable independiente (una o varias coordenadas espaciales y posiblemente el tiempo), se dice que se trata de una **ecuación en derivadas parciales**.

Los **modelos de parámetros concentrados** admiten la siguiente clasificación, en función de los tipos de ecuaciones que en ellos intervienen.

- **Modelos algebraicos** son aquellos modelos compuestos únicamente por ecuaciones algebraicas.
- Se denominan **modelos dinámicos en ecuaciones diferenciales ordinarias**, o abreviadamente modelos dinámicos ODE (del inglés *Ordinary Differential Equations*), a aquellos modelos compuestos únicamente por ecuaciones diferenciales ordinarias. En el contexto del modelado de sistemas dinámicos, al emplearse el término **modelo ODE** se sobreentiende que en las ecuaciones diferenciales ordinarias las derivadas son únicamente respecto al tiempo. Los modelos ODE se clasifican a su vez en **ODE explícito** y **ODE implícitos**, dependiendo de si es posible o no despejar a un lado de la igualdad las derivadas respecto al tiempo.

Tabla 1.1: Clasificación de los modelos dinámicos de parámetros concentrados.

Algebraico		$\mathbf{f}(\mathbf{x}, t) = \mathbf{0}$
ODE	explícito	$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, t)$
	implícito	$\mathbf{f}(\dot{\mathbf{x}}, \mathbf{x}, t) = \mathbf{0}$
DAE	semi-explícito	$\begin{cases} \dot{\mathbf{x}}_1 = \mathbf{f}_1(\mathbf{x}_1, \mathbf{x}_2, t) \\ \mathbf{f}_2(\mathbf{x}_1, \mathbf{x}_2, t) = \mathbf{0} \end{cases}$
	implícito	$\begin{cases} \mathbf{f}_1(\dot{\mathbf{x}}_1, \mathbf{x}_1, \mathbf{x}_2, t) = \mathbf{0} \\ \mathbf{f}_2(\mathbf{x}_1, \mathbf{x}_2, t) = \mathbf{0} \end{cases}$

- Los **modelos de ecuaciones algebraico diferenciales** o abreviadamente modelos DAE (del inglés *Differential-Algebraic Equations*), son aquellos en los que intervienen ecuaciones algebraicas y también ecuaciones diferenciales ordinarias con derivadas únicamente respecto al tiempo. Los modelos DAE pueden ser **DAE semi-explícito**, cuando es posible despejar las derivadas, o bien **DAE implícito**, cuando no es posible despejarlas. En los modelos DAE no aparecen derivadas respecto a las coordenadas espaciales.

En la Tabla 1.1 se resume la forma en que pueden intervenir los diferentes tipos de variables en los modelos dinámicos de parámetros concentrados y la clasificación a que ello da lugar. Se ha empleado la notación siguiente: $\mathbf{x}$ representa el vector de variables del modelo $(x_1, \dots, x_n)$, t representa el tiempo y $\dot{\mathbf{x}}$ representa el vector de las derivadas respecto al tiempo de las variables $\mathbf{x}$, es decir $(\frac{dx_1}{dt}, \dots, \frac{dx_n}{dt})$. En general, la derivada respecto al tiempo de una variable x se representa indistintamente como $\frac{dx}{dt}$ o $\dot{x}$. Por otra parte, obsérvese que el vector de variables $\mathbf{x}$ se ha descompuesto en la descripción de los modelos DAE en dos vectores, $\mathbf{x}_1$ y $\mathbf{x}_2$, que representan respectivamente las variables que aparecen derivadas respecto al tiempo y las que no.

Los **modelos dinámicos híbridos** (véase la Figura 1.4) son aquellos que tienen una parte de tiempo continuo, y una parte de tiempo discreto o eventos discretos. Este tipo de modelo tiene variables tanto de tiempo continuo, como de tiempo discreto. Normalmente, al referirse a los modelos dinámicos híbridos suele indicarse de qué tipo es su parte de tiempo continuo, denominándolos, por ejemplo, **ODE híbrido** o **DAE híbrido**. En el primer caso, la parte de tiempo continuo está descrita por ecuaciones ODE con derivadas únicamente respecto al tiempo. En el segundo caso, está descrita además mediante ecuaciones algebraicas.

1.4.2. Modelos de tiempo discreto

Los modelos de tiempo discreto son el tipo de modelo más fácil de entender de manera intuitiva, ya que sus variables van cambiando de valor únicamente en instantes de tiempo equiespaciados. Para simular este tipo de modelo, el **reloj de la simulación** (que *indica el valor del tiempo simulado*) avanza saltando un cierto intervalo de tiempo denominado **paso de avance en el tiempo**, que es constante a lo largo de la simulación.

Si el instante inicial de la simulación es t_0 y el paso de avance en el tiempo es Δt , entonces el reloj de la simulación va saltando sucesivamente a los instantes $t_0, t_1, t_2, \dots$, donde $t_{i+1} = t_i + \Delta t$ para $i = 0, 1, 2, \dots$.

En cada uno de esos instantes, el modelo se encuentra en un estado, recibe unas entradas y genera unas salidas. El modelo permite calcular, a partir de su estado actual y de sus entradas actuales, cuáles son sus salidas actuales y cuál será su estado en el siguiente instante de tiempo.

Una forma de representar el comportamiento de un modelo de tiempo discreto es indicando su función de transición de estado y su función de salida. Sean x_i la entrada, q_i el estado e y_i la salida del modelo en el instante t_i , donde el índice temporal i toma los valores $0, 1, 2, \dots$.

La **función de transición de estado**, δ , describe cómo se calcula el estado siguiente a partir del estado y la entrada actuales.

$$q_{i+1} = \delta(q_i, x_i) \quad \text{para } i : 0, 1, 2, \dots \quad (1.1)$$

La **función de salida**, λ , describe cómo se calcula la salida actual a partir del estado y la entrada actuales.

$$y_i = \lambda(q_i, x_i) \quad \text{para } i : 0, 1, 2, \dots \quad (1.2)$$

Las funciones de transición de estado y de salida permiten calcular la trayectoria del estado ($q_1, q_2, \dots$) y la trayectoria de la salida ($y_0, y_1, y_2, \dots$), conocidos el estado inicial del modelo (q_0) y la trayectoria de entrada ($x_0, x_1, x_2, \dots$).

El siguiente algoritmo es un **simulador** para un modelo de tiempo discreto que está descrito mediante las funciones de transición de estado y de salida. El algoritmo calcula las trayectorias del estado y de salida del modelo, a partir de la trayectoria de entrada y del estado inicial.

```

iFin = 9           índice temporal final
x(0) = 1, ..., x(9) = 0   trayectoria de entrada
q(0) = 0           estado inicial
i = 0              inicialización del índice temporal
while ( i <= iFin ) {
    y(i)   =  $\lambda( q(i), x(i) )$ 
    q(i+1) =  $\delta( q(i), x(i) )$ 
    i      = i + 1
}

```

1.4.3. Modelos de eventos discretos

En los modelos de eventos discretos los instantes en que se producen los eventos no tienen necesariamente que estar equiespaciados en el eje temporal. El tiempo que transcurre entre eventos consecutivos puede ser cualquiera, siempre que se satisfaga que el número de eventos en cualquier intervalo finito de tiempo sea finito. Una consecuencia de esto es que el algoritmo de simulación de eventos discretos debe mantener un **calendario de eventos** donde ir registrando el instante de activación de los eventos planificados para instantes futuros.

Una manera de definir los modelos de eventos discretos es especificando qué tipos de eventos pueden ocurrir y cuál es la relación causal entre ellos. Un evento puede producir un cambio en el estado del modelo, planificar eventos para instantes futuros y cancelar la planificación de eventos.

A continuación se describe una manera sencilla de programar, empleando un lenguaje de programación imperativo (C, FORTRAN, etc.), el algoritmo de la simulación de un modelo de eventos discretos descrito mediante **planificación de eventos**. El programa podría constar de las rutinas siguientes:

- **Rutina de inicialización.** Asigna valores iniciales a las variables de estado, inicializa los acumuladores estadísticos y planifica eventos añadiéndolos al calendario de eventos. Los **acumuladores estadísticos** son variables intermedias a partir de las cuales se calculan las variables de salida de la simulación.
- **Rutina de tiempo.** Determina cuál es el evento más inminente de los planificados en el calendario de eventos y avanza el reloj de la simulación hasta ese instante.
- **Rutinas de eventos.** Son las rutinas, una para cada tipo de evento, que realizan el flujo de acciones asociado al evento. Entre estas acciones puede

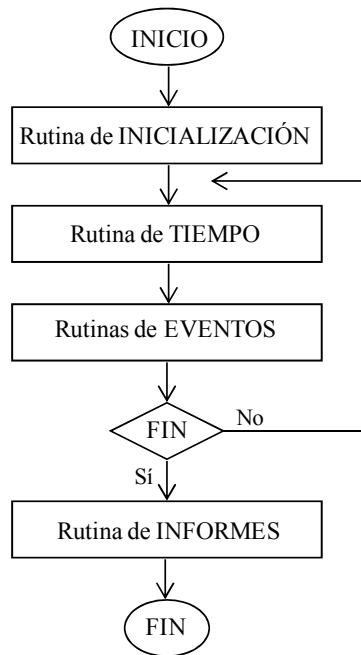


Figura 1.5: Flujo de la simulación de un modelo orientado a la planificación de eventos.

estar modificar el valor de las variables y acumuladores estadísticos, así como añadir o quitar eventos del calendario de eventos.

- **Rutina de informes.** Al finalizar la simulación, calcula y muestra el valor de las variables de salida.

El programa principal controla el flujo de control de la simulación, tal como se muestra en la Figura 1.5.

1. Comienza la simulación.
2. El programa principal pasa el control a la **rutina de inicialización**, para que ésta inicialice el reloj de la simulación, las variables de estado, el calendario de eventos y los acumuladores estadísticos. Se activa el evento “Inicio de la Simulación”. Como parte de las acciones asociadas a la ejecución de este evento, se planifican determinados eventos para su ejecución en instantes futuros. Estos eventos son añadidos al calendario de eventos, ordenados de menor a mayor instante de ejecución.
3. Una vez ejecutadas las acciones de inicialización, el programa principal transfiere el control a la **rutina de tiempo**. El reloj de la simulación es avanzado hasta el instante de ejecución del primer evento del calendario, que es el más

inminente, el cual es entonces borrado del calendario. A continuación, se activa dicho evento.

4. El programa principal transfiere el control a la rutina correspondiente al tipo de evento activado. Las acciones realizadas por la **rutina de eventos** dependen de cada caso, pero en general la rutina actualizará las variables de estado, los acumuladores estadísticos y añadirá nuevos eventos al calendario de eventos. También puede eliminar eventos del calendario de eventos.

Existe un evento especial, denominado “Finalización de la Simulación”, que se activa cuando se satisfacen las condiciones fijadas para la finalización de la simulación. Por ejemplo, que se alcance determinado valor del tiempo, que el estado del sistema satisfaga determinadas condiciones, etc. Una de las acciones asociadas al evento de finalización de la simulación es el cálculo de las variables de salida de la simulación a partir del valor de los acumuladores estadísticos.

5. Si el evento ejecutado es “Finalización de la Simulación”, el programa principal transfiere el control a la rutina generadora de informes. En caso contrario, el programa principal devuelve el control a la rutina de tiempo.
6. La **rutina generadora de informes** muestra el valor de las variables de salida de la simulación.
7. Finaliza la simulación.

Esta forma de describir los modelos de eventos discretos y de programar su algoritmo de simulación es conceptualmente sencilla cuando se aplica a modelos sencillos. Sin embargo, este procedimiento implica describir globalmente el comportamiento del sistema completo, lo cual resulta excesivamente complejo y es propenso a errores cuando se aplica a la realización de modelos de grandes dimensiones, en los que hay que definir muchos tipos de eventos y la relación causal entre ellos.

Existen formalismos de modelado de eventos discretos que no presentan estos inconvenientes. Un ejemplo es **DEVS** (Discrete EVent system Specification), que estando basado en la planificación de los eventos, permite describir el modelo de manera modular y jerárquica.

Otro ejemplo es la metodología de **modelado orientado a los procesos**, que pretende facilitar la descripción de los modelos permitiendo que ésta se realice de manera más próxima al razonamiento humano. Consiste en tomar el punto de vista de las entidades y describir su circulación a través del sistema y las acciones que van realizando.

La práctica de esta metodología de la orientación de los procesos comenzó a aplicarse en la década de 1970, gracias a la aparición de **lenguajes de simulación** de propósito general para modelos de eventos discretos. Los compiladores de estos lenguajes de simulación traducen automáticamente la descripción orientada a los procesos del modelo a una descripción orientada a la planificación de eventos escrita en algún lenguaje de programación. En última instancia, el código ejecutable de la simulación siempre está orientado a la planificación de los eventos. Algunos de estos lenguajes de simulación se usan todavía en la actualidad, como es el caso de GPSS, SIMSCRIPT, SLAM, SIMAN, etc.

Hoy en día el modelado orientado a los procesos suele realizarse empleando **entornos de simulación**, que son una capa software construida sobre un lenguaje de simulación a fin de facilitar la descripción del modelo mediante interfaces de usuario muy intuitivas, con menús, diálogos, etc. Estas herramientas son ampliamente empleadas en la simulación de modelos estocásticos de sistemas logísticos de fabricación, almacenamiento y distribución. Entre las más populares se encuentran AnyLogic, Arena, AutoMod, Enterprise Dynamics, ExtendSim, FlexSim, Plant Simulation, ProModel, Simio, SIMUL8, Witness, etc.

Estos entornos de simulación permiten al usuario construir el modelo instanciando módulos predefinidos (pinchando y arrastrando el icono desde la librería de módulos a la ventana de edición del modelo) y conectándolos gráficamente. La animación y otras capacidades gráficas permiten visualizar la evolución del modelo durante la simulación. La interfaz gráfica de usuario del entorno de simulación también permite acceder a los niveles inferiores en la descripción del modelo: a la descripción de partes del modelo usando el lenguaje de simulación e incluso al lenguaje de programación. Además, algunos entornos proporcionan facilidades para el modelado estadístico de las entradas aleatorias al modelo, para definir experimentos y problemas de optimización sobre el modelo, y para analizar estadísticamente los resultados.

Con el fin de ilustrar esta metodología de modelado, supongamos que se desea modelar el funcionamiento de una oficina de atención al público en la cual trabaja un empleado. La estructura lógica del modelo es sencilla. Si llega un nuevo cliente y el empleado está ocupado, el cliente se pone al final de la cola con disciplina FIFO. Si el empleado está libre, el cliente es atendido inmediatamente. Cuando el empleado termina de atender a un cliente, éste se marcha y comienza a ser atendido el primer cliente de la cola. Si la cola está vacía, el empleado permanece libre hasta la llegada de un nuevo cliente.

Para modelar este sistema emplearemos el entorno de simulación Arena. Tomaremos el punto de vista de los clientes y describiremos su circulación a través del

sistema. Los pasos que sigue un cliente en la oficina son los siguientes: (1) llego a la oficina; (2) me pongo al final de la cola; (3) espero hasta ser primero de la cola y a que el empleado esté libre (si tengo suerte, el tiempo de espera será cero); (4) el empleado me atiende durante el tiempo que requiero; y (5) pasado dicho tiempo, abandono la oficina.

El modelo puede describirse de manera sencilla, ya que Arena dispone de un módulo predefinido que permite describir el proceso de llegada, otro módulo que permite describir el proceso de atención al cliente con su cola, y otro módulo que permite describir que los clientes abandonan la oficina. En la Figura 1.6a se muestra el diagrama de módulos del modelo con la animación correspondiente a un cierto instante de la ejecución de la simulación. El diagrama del modelo está compuesto por la instanciación y conexión de los tres tipos de módulo anteriormente indicados. La conexión entre los módulos define el flujo de los clientes en la oficina. En el preciso instante de la simulación mostrado en la figura habían llegado ya 109 clientes, 3 esperaban en la cola, uno estaba siendo atendido por el empleado y 105 había abandonado la oficina (véanse los números que hay cerca de los módulos en la Figura 1.6a).

Haciendo doble clic sobre cada módulo del diagrama se abre un menú de configuración. En la Figura 1.6b se muestra el menú de configuración del módulo que describe la llegada de clientes (módulo de tipo Create) y en la Figura 1.6c el menú del módulo que describe el proceso de atención al cliente (módulo de tipo Process).

Arena calcula por defecto determinadas medidas estadísticas del comportamiento del sistema, como son el tiempo medio, máximo y mínimo en las colas, la ocupación de los recursos, etc. y permite al usuario definir sus propio cálculos. Al finalizar el experimento de simulación Arena genera automáticamente informes en los cuales se muestra esta información.

El análisis en profundidad de las metodologías y el software de simulación de modelos estocásticos de eventos discretos está fuera del alcance de este texto. Dejamos por tanto la explicación en este punto.

1.4.4. Autómatas celulares

El **autómata celular** es un tipo de modelo con dos propiedades características. La primera es que posee una estructura regular: está compuesto de componentes iguales, conectados de acuerdo a un cierto patrón espacial. La segunda es que el

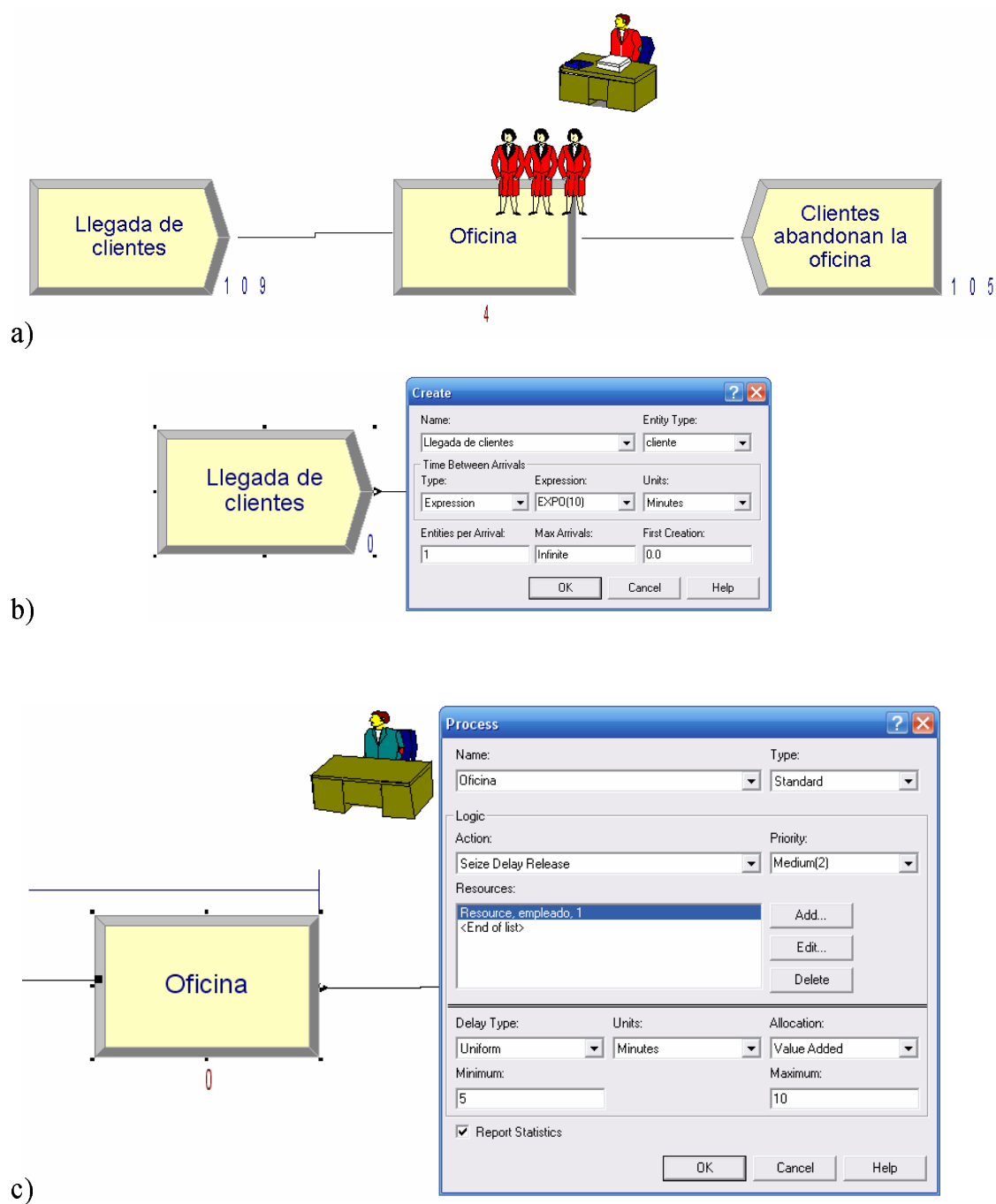


Figura 1.6: Modelado en Arena de la oficina atendida por un empleado: a) diagrama de módulos del modelo; y b) y c) propiedades de los módulos que definen los procesos de llegada de los clientes y de atención a los clientes, respectivamente.

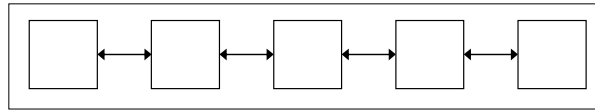


Figura 1.7: Espacio celular unidimensional.

comportamiento de todos los componentes está regido por el mismo conjunto de reglas.

Este tipo de modelo fue empleado originalmente por von Neumann y Ulam para describir la autorreproducción de sistemas biológicos. De ahí proviene su denominación. Cada uno de los componentes iguales que compone el autómata celular se denomina una **célula** y el conjunto de células se denomina **espacio celular**. La distribución espacial de las células puede ser un mallado unidimensional, bidimensional o multidimensional, conectado de manera uniforme. Las células que influyen sobre una célula en particular, denominadas sus **vecinas**, son a menudo aquellas situadas más cerca en el sentido geométrico.

Puede definirse un autómata unidimensional conectando las células en fila, como se muestra en la Figura 1.7, de modo que cada célula tenga conectada una célula a su izquierda y otra a su derecha. Supongamos que cada célula de este autómata unidimensional puede estar en dos estados, 0 y 1, y que recibe como entrada los estados de las células vecinas. En este caso, hay 8 posibles combinaciones de los valores de las 2 entradas y del estado de la célula. La **tabla de transición de estados** de la célula, que describe el nuevo estado de la célula en función de sus entradas y del estado actual, tendrá la forma mostrada a continuación.

Entrada izquierda actual	Estado actual	Entrada derecha actual	Estado siguiente
0	0	0	?
0	0	1	?
0	1	0	?
0	1	1	?
1	0	0	?
1	0	1	?
1	1	0	?
1	1	1	?

La cuarta columna contiene interrogaciones. Dado que dicha columna tiene 8 filas y que la célula puede estar en 2 estados, hay $2^8 = 256$ posibles tablas de transición de estados. Cada una de estas posibles tablas representa una **regla**. Hay, por tanto, 256 posibles reglas. El criterio para designar las reglas es el siguiente. La cuarta

columna puede interpretarse como un número binario de 8 bits, de modo que la primera fila sea el bit menos significativo y la octava fila el más significativo. Se obtiene el nombre de la regla convirtiendo este número de decimal.

Para simular el autómata hay que asignar valor inicial al estado de cada célula del espacio celular y aplicar el algoritmo simulador, que calcula en cada instante el estado de cada célula a partir del estado en el instante anterior de esa célula y de sus vecinas. Por ejemplo, en la Figura 1.8 se muestra la tabla de transición de estados del autómata con regla 90 y un ejemplo de su simulación. Para facilitar la visualización del resultado de la simulación, el estado 1 se ha representado mediante un aspa y el estado 0 mediante un guión.

Los autómatas celulares unidimensionales muestran comportamientos muy interesantes y diversos. Básicamente pueden distinguirse cuatro tipos diferentes de comportamiento. En algunos autómatas cualquier dinámica se extingue rápidamente. Otros tienen enseguida un comportamiento periódico. Hay otros que muestran un comportamiento caótico. Finalmente, está el tipo más interesante de autómatas: aquellos autómatas cuyo comportamiento no es periódico ni predecible, pero que muestran patrones regulares interesantes.

Un ejemplo interesante de autómata bidimensional es el denominado **Juego de la Vida de Conway**. El juego tiene lugar en un espacio celular bidimensional, cuyo tamaño puede ser finito o infinito. Cada celda está acoplada a aquellas que se encuentran más próximas a ella, tanto lateral como diagonalmente. Esto significa que, para una célula situada en el punto $(0, 0)$, sus células vecinas laterales están situadas en los puntos $(0, 1)$, $(1, 0)$, $(0, -1)$ y $(-1, 0)$, y sus células vecinas diagonales están situadas en $(1, 1)$, $(-1, 1)$, $(1, -1)$ y $(-1, -1)$, como se muestra en la Figura 1.9a. En la Figura 1.9b se muestran las células vecinas de una célula situada en la posición (i, j) .

El estado de cada célula puede tomar dos valores: 1 (viva) y 0 (muerta). Cada una de las células puede sobrevivir (está viva y permanece viva), nacer (su estado pasa de 0 a 1) y morir (su estado pasa de 1 a 0) a medida que el juego progresa. Las reglas, tal como fueron definidas por Conway, son las siguientes:

1. Una célula permanece viva si tiene en su vecindad 2 ó 3 células vivas.
2. Una célula muere debido a superpoblación si hay más de 3 células vivas en su vecindad.
3. Una célula muere a causa del aislamiento si hay menos de 2 células vivas en su vecindad.

Entrada izquierda actual	Estado actual	Entrada derecha actual	Estado siguiente
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

i=0	-----X-----
i=1	-----X-X-----
i=2	-----X--X-----
i=3	-----X-X-X-X-----
i=4	-----X-----X-----
i=5	-----X-X---X-X-----
i=6	-----X--X--X--X-----
i=7	-----X-X-X-X-X-X-X-----
i=8	-----X-----X-----
i=9	-----X-X-----X-X-----
i=10	-----X--X-----X--X-----
i=11	-----X-X-X-X-----X-X-X-X-----
i=12	-----X-----X-----X-----X-----
i=13	-----X-X---X-X---X-X---X-X-----
i=14	-----X--X--X--X--X--X--X--X-----
i=15	-----X-X-X-X-X-X-X-X-X-X-X-X-X-X-----
i=16	-----X-----X-----
i=17	-----X-X-----X-X-----
i=18	-----X--X-----X--X-----
i=19	-----X-X-X-X-----X-X-X-X-----
i=20	-----X-----X-----X-----X-----
i=21	-----X-X---X-X---X-X---X-X-----
i=22	-----X--X--X--X--X--X--X--X-----
i=23	-----X-X-X-X-X-X-X-X-X-X-X-X-X-X-----
i=24	-----X-----X-----X-----X-----
i=25	-----X-X-----X-X-----X-X-----X-X-----
i=26	-----X--X-----X--X-----X--X-----X--X-----
i=27	-----X-X-X-X-----X-X-X-X-----X-X-X-X-----X-X-X-X-----
i=28	-----X-----X-----X-----X-----X-----X-----X-----
i=29	-----X-X-----X-X-----X-X-----X-X-----X-X-----X-X-----

Figura 1.8: Autómata celular con regla 90: tabla de transición de estados (arriba) y simulación de un espacio celular con esa regla compuesto por 61 células (abajo).

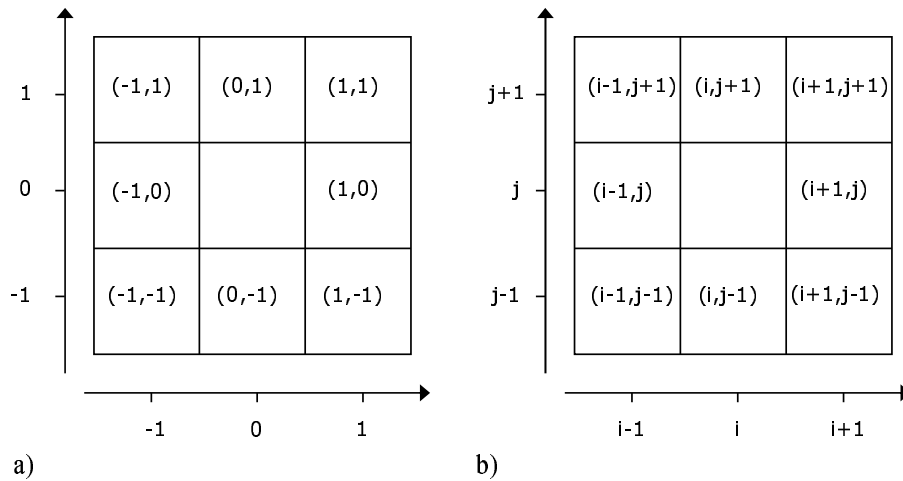


Figura 1.9: Células vecinas en el *Juego de la Vida* a la situada en: a) $(0,0)$; y b) (i,j) .

4. Una célula muerta vuelve a la vida si hay exactamente 3 células vivas en su vecindad.

El espacio celular recubre una región finita. Por ejemplo, el cuadrado ocupado por $N \times N$ células, de modo que el autómata está compuesto por N^2 células. Obsérvese que debe especificarse el comportamiento de las células del borde del espacio, ya que no poseen todos sus vecinos. Una posibilidad es asumir que las células situadas en los bordes de la región mantienen un estado constante: por ejemplo, todas están muertas. Otra solución es envolver el espacio de manera toroidal. Es decir, interpretar el índice N también como 0.

El modelo del Juego de la Vida evoluciona sobre una base de tiempo discreto. El tiempo avanza a pasos: 0, 1, 2, ... El juego comienza con cierta configuración de células vivas. A medida que la simulación va avanzando, la forma de los grupos de células vivas va cambiando. La idea del juego es encontrar nuevos patrones y estudiar su comportamiento. Por ejemplo, se muestran algunos patrones en la Figura 1.10.

Un algoritmo para simular el autómata sería el siguiente. Se almacena el estado inicial de las células en una estructura de datos. En cada instante de tiempo se inspeccionan todas las células, aplicando la función de transición de estado a cada una de ellas, y salvando el siguiente estado en una segunda copia de la estructura de datos. Una vez calculado el siguiente estado, éste se convierte en el estado actual y se avanza el reloj de la simulación un paso.

Este algoritmo que calcula el nuevo estado de todas las células en cada paso de tiempo. Una forma de optimizarlo consiste en calcular únicamente el nuevo estado de aquellas células cuyo nuevo estado pueda ser diferente del actual. Si el

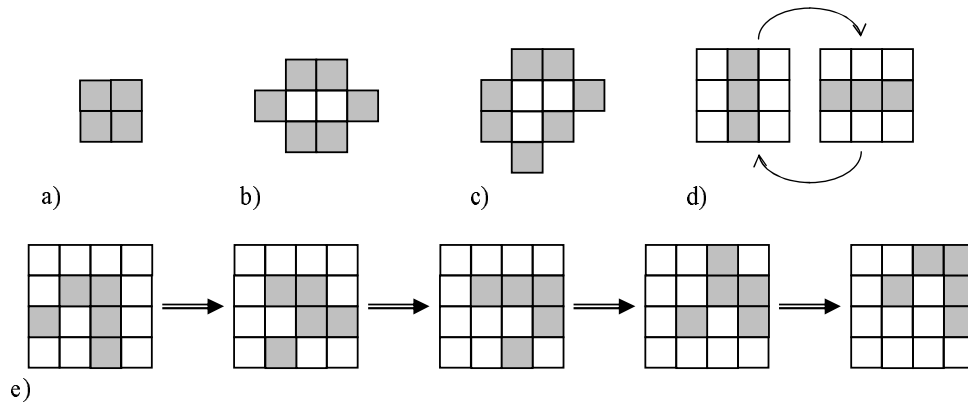


Figura 1.10: Algunos patrones que aparecen en el *Juego de la Vida de Conway*. Los patrones a), b) y c) son estables, ya que no cambian. El patrón d) es oscilante. En e) se muestra un ciclo de patrones que se mueve.

estado de las células vecinas no cambia en el instante actual, entonces el estado de la célula en cuestión no cambiará en el siguiente instante. Obsérvese que en esta versión optimizada del algoritmo es necesario llevar un registro de qué células son susceptibles de cambiar de estado en la transición siguiente.

La versión optimizada del algoritmo sigue la lógica siguiente. En una transición de estado deben señalarse aquellas células cuyo estado cambia. A continuación, se establece el conjunto de todas las células vecinas de aquellas. Este conjunto contiene todas las células que pueden posiblemente cambiar su estado en el siguiente paso de tiempo. El estado de las células no pertenecientes a este conjunto permanecerá inalterado en el siguiente paso de tiempo. De esta forma se determina el conjunto de células que posiblemente cambiarán su estado. Para saber si cada una de ellas efectivamente cambia de estado, es necesario evaluar su función de transición de estado.

Los autómatas celulares se emplean para describir sistemas cuyo comportamiento está determinado por la interacción entre sus componentes. Algunas aplicaciones son el estudio del crecimiento de cristales, la propagación de incendios forestales y de manchas en el mar, las reacciones químicas y el transporte en medios fluidos y sólidos, el tráfico de vehículos en las ciudades, el comportamiento de colonias de seres vivos, y el crecimiento de corales y conchas, por citar sólo algunos ejemplos.

1.4.5. Modelos basados en agentes

El modelado basado en agentes es una metodología de modelado computacional desarrollada para el estudio de los sistemas complejos. En este contexto, los **sistemas complejos** son aquellos que están compuestos de múltiples elementos individuales interactuantes, de manera que el comportamiento agregado del sistema no es una reproducción a escala sin más del comportamiento individual, sino que surgen “fenómenos emergentes”.

Estos comportamientos y propiedades emergentes no pueden ser deducidas únicamente a partir de las propiedades individuales de los elementos, dado que su origen se encuentra también en las interacciones entre los elementos. Las propiedades emergentes globales, a su vez, realimentan a cada elemento individual, influyendo sobre sus propiedades y comportamiento.

Un ejemplo de propiedad emergente es la autoorganización observada en algunos sistemas, en los cuales aparecen patrones globales que son debidos a la interacción entre los componentes. En los sistemas complejos, el orden puede surgir sin necesidad de que exista un director o coordinador centralizado. De hecho, en ocasiones se observan patrones emergentes, aparentemente muy complejos, que son generados por la aplicación repetida al nivel individual de reglas muy simples.

A consecuencia de ello, al modelar sistemas complejos es importante distinguir entre el comportamiento al nivel micro, del individuo, y el comportamiento al nivel macro, global del sistema, ya que el comportamiento en ambos niveles puede ser muy diferente. Por ejemplo, la emergencia de patrones y el orden observado al nivel de sistema es, en ocasiones, el resultado de la aleatoriedad en el comportamiento de los múltiples elementos individuales que interaccionan en el sistema. En muchos sistemas complejos los procesos aleatorios individuales son esenciales para la creación del orden observado al nivel de sistema.

El paso del conocimiento en un nivel, al conocimiento en el otro, plantea los dos posibles problemas siguientes:

- **Comprensión integral.** Conocido cómo se comportan los elementos individuales, el objetivo es tratar de obtener el comportamiento agregado.
- **Comprensión diferencial.** Conocido el comportamiento agregado, el objetivo es encontrar qué reglas de comportamiento individual, lo más simples posible, generan dicho comportamiento.

El modelado basado en agentes consiste en describir el comportamiento de los elementos individuales en términos de agentes y de sus interacciones. Los **agentes** son objetos computacionales autónomos, con sus propiedades (variables de estado) y reglas de comportamiento individuales, cambiantes en el tiempo. Entre las propiedades de los agentes se encuentra típicamente su apariencia gráfica, de manera que ésta, descriptiva del estado del agente, pueda visualizarse en la pantalla del ordenador.

Las interacciones entre los agentes pueden ser muy complejas. Suponen un intercambio de información, en base al cual los agentes pueden actualizar su estado o realizar acciones basadas en una toma de decisión. Las reglas de comportamiento son individuales para cada agente y pueden actualizarse a lo largo de la simulación. Por otra parte, el empleo de números pseudoaleatorios en los algoritmos de las reglas de decisión y las acciones, permite modelar el comportamiento aleatorio al nivel individual.

Los agentes individuales pueden llevar un registro histórico de sus interacciones y cambiar sus reglas de comportamiento en base a los eventos pasados. Esto posibilita el desarrollo de modelos con agentes adaptativos, que puedan aprender y modificar su comportamiento como resultado de la interacción con otros agentes.

Los modelos basados en agentes permiten reproducir comportamientos que son característicos de los sistemas complejos, donde se observa que:

1. Reglas individuales sencillas pueden generar comportamientos agregados muy complejos.
2. La aleatoriedad en el comportamiento individual puede resultar en un comportamiento global determinista.
3. Los sistemas pueden autoorganizarse, dando lugar a patrones complejos de comportamiento, sin necesidad de que exista un controlador o árbitro central.

En la actualidad no existe un lenguaje de modelado estándar para modelos basados en agentes. Las diferentes herramientas proporcionan sus propios mecanismos para la definición del modelo. Entre las herramientas gratuitas más populares se encuentran NetLogo, Swarm, Repast y MASON. AnyLogic es una herramienta comercial (con una versión gratuita) ampliamente usada, que permite combinar el uso de tres formalismos: modelado basado en agentes, dinámica de sistemas y simulación de eventos discretos.

De los anteriores, el entorno de simulación **NetLogo** es probablemente el que cuenta con una mayor comunidad de usuarios. Fue desarrollado a finales de la década de 1990. Su lenguaje para la descripción de los modelos proviene de Logo, un lenguaje de programación diseñado para ser usado por niños. Igual que en Logo, donde el programador dirige una tortuga para dibujar figuras geométricas, en NetLogo el agente móvil prototipo es una tortuga. El lenguaje de NetLogo fue diseñado para ser leído de manera sencilla, como si fuera pseudocódigo. Este enfoque hace que el aprendizaje de NetLogo resulte sencillo. A continuación, se describen muy brevemente algunas de las características de NetLogo.

En NetLogo existen los tipos predefinidos de agentes siguientes:

- **Tortugas** (*turtles*): agentes móviles, que pueden desplazarse por el entorno.
- **Baldosas** (*patches*): agentes que no pueden moverse y que ocupan un espacio/área en el mundo.
- **Enlaces** (*links*): agentes conexión, que enlazan dos o más tortugas.

Las tortugas son objetos puntuales. Aunque se le puede asociar una forma geométrica para visualizarlas, cada tortuga está contenida en aquella baldosa a la que pertenecen las coordenadas del centro de la tortuga. Un baldosa puede contener varias tortugas.

Además, existe un agente de alto nivel denominado **observador** (*observer*), mediante el cual el usuario del modelo controla la simulación. El observador envía comandos a los agentes, pidiéndoles que realicen cálculos o acciones. También, mediante el observador, el usuario del modelo puede especificar la perspectiva desde la cual quiere observar la evolución del modelo. Por ejemplo, la vista de la simulación puede estar centrada en un agente específico o puede ser la tradicional vista de pájaro.

En NetLogo se distingue entre variables globales y variables de agente. Una variable global tiene un único valor, con independencia del agente que esté accediendo a ella. Todos los agentes pueden acceder a la variable. En contraste, una variable de agente tiene un valor diferente para cada agente de cierto tipo. Los diferentes tipos de agentes tienen determinadas variables predefinidas, pudiendo el programador declarar nuevas variables.

En NetLogo hay predefinidas acciones para los agentes. Por ejemplo, acciones predefinidas para las tortugas son moverse hacia delante y hacia atrás en el entorno,

cambiar de dirección, morirse y crear una copia de sí mismo. El programador puede definir nuevas acciones.

Los agentes pueden agruparse en estructuras de datos, por ejemplo para facilitar solicitarles que realicen una determinada acción. Dos estructuras de datos son el conjunto de agentes (*agentset*) y la lista (*list*). Los conjuntos pueden contener tortugas, baldosas o enlaces, pero no puede mezclar diferentes tipos de agentes. El desarrollador del modelo puede crear sus propios conjuntos y también pedir a un conjunto que ejecute ciertos comandos. En NetLogo los elementos de un conjunto están ordenados aleatoriamente. Por ello, si se pide repetidas veces que un conjunto ejecute un comando, el orden en que los agentes del conjunto ejecutarán el comando será cada vez diferente. Por el contrario, los agentes almacenados en una lista están ordenados. Si se pide que una lista ejecute un comando, los agentes de la lista lo ejecutarían en el mismo orden en que están almacenados en la lista.

En NetLogo pueden definirse comandos (*commands*) e informes (*reporters*), mediante los cuales se puede pedir a los agentes que realicen acciones o cálculos, respectivamente. Hay un buen número de comandos e informes definidos por defecto (véase la lista completa en el NetLogo Dictionary que acompaña a NetLogo) y también es posible crear otros.

Dejamos en este punto esta breve exposición de conceptos sobre NetLogo. El lector interesado en profundizar en la materia puede consultar el manual de usuario y los ejemplos de código proporcionados junto con NetLogo.

De la discusión anterior podría deducirse que el espacio sobre el que se mueven los agentes debe ser una superficie o volumen discretizado espacialmente, por ejemplo, una superficie cubierta de baldosas. Esto no es correcto. Si bien hay modelos basados en agentes en los cuales el espacio es discreto, también hay modelos en los cuales el espacio es un continuo bidimensional o tridimensional, o consiste en datos proporcionados por un sistema de información geográfica (GIS).

Respecto a la simulación de los modelos basados en agentes, las diferentes herramientas emplean sus propias estrategias para el avance en el tiempo y para la ejecución de las acciones simultáneas. Dos estrategias para el avance del reloj de la simulación son:

- **Ejecución síncrona.** Existe un único reloj de la simulación y un paso de avance en el tiempo. Cuando el reloj hace un “tic”, todos los agentes invocan sus reglas. Si las condiciones de las reglas son satisfechas, se produce el comportamiento.

- **Ejecución asíncrona.** Sólo los agentes que posiblemente van a realizar una acción son invocados. En lugar de tener un paso en el tiempo constante, la simulación va saltando de un evento al siguiente.

Respecto al empleo de modelos basados en agentes, debe tenerse en cuenta que el modelado basado en agentes típicamente requiere de una gran cantidad de recursos computacionales. Simular miles o millones de agentes es computacionalmente intensivo. Es el precio que debe pagarse por obtener y almacenar información al nivel del individuo. Debe por tanto sopesarse si el objetivo del estudio requiere del uso de este tipo de modelo, o bien resulta más adecuado emplear un modelo basado en ecuaciones. El empleo de modelos basados en agentes está justificado, por ejemplo, cuando existe una gran heterogeneidad entre los agentes y esta heterogeneidad afecta al comportamiento global del sistema.

Otra cuestión práctica es la siguiente. Normalmente, cuanto mayor es el nivel de detalle del modelo, mayor es también el número de sus parámetros. Puesto que la descripción en los modelos basados en agentes se realiza al nivel del individuo, en dichos modelos quedan expuestos parámetros que quedarían ocultos, debido a las hipótesis de modelado simplificadoras realizadas, en otros modelos basados en ecuaciones que describieran directamente el comportamiento agregado del sistema. Encontrar valores adecuados para estos parámetros puede ser la tarea más laboriosa y delicada del proceso de construcción del modelo.

Por contra, dado que el modelo basado en agentes describe detalles del comportamiento al nivel del individuo, no requiere la descripción del comportamiento al nivel agregado. Esta es una ventaja, por ejemplo, en el ámbito de las Ciencias Sociales, donde frecuentemente es más sencillo modelar el comportamiento de los individuos que el de los grupos sociales, siendo de hecho comúnmente el objetivo del estudio estimar éste a partir de aquel.

1.4.6. Modelos dinámicos en ecuaciones diferenciales ordinarias

Los modelos de tiempo continuo se caracterizan por el hecho de que en un intervalo finito de tiempo sus variables pueden cambiar de valor infinitas veces (piénsese, por ejemplo, en el volumen de líquido almacenado en un depósito). Ningún otro tipo de modelo comparte esta propiedad.

En cualquier intervalo de tiempo de longitud mayor que cero existen infinitos instantes de tiempo. Dado que es imposible calcular el valor de las variables del modelo en infinitos instantes de tiempo, la simulación de los modelos de tiempo

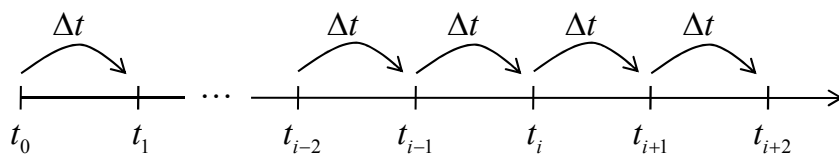


Figura 1.11: Discretización temporal.

continuo se realiza aplicando algoritmos que calculan el valor de las variables del modelo únicamente en determinados instantes de tiempo. Se dice que se realiza una **discretización temporal**.

La resolución numérica de las ecuaciones diferenciales ordinarias se realiza aplicando **métodos de integración numérica**. Consideremos, por ejemplo, la ecuación diferencial ordinaria explícita siguiente:

$$\frac{dx}{dt} = f(x, t) \quad (1.3)$$

la cual describe que la variación en el tiempo de la variable x , es decir, la derivada de x respecto al tiempo, es una función f de la propia variable x y del tiempo t . El valor de la variable x será calculado en instantes específicos predefinidos $t_0, t_1, t_2, \dots$, donde t_0 es el instante inicial de la simulación (véase la Figura 1.11). La distancia entre dos instantes consecutivos se denomina **paso de integración**: $\Delta t = t_i - t_{i-1}$ con $i = 1, 2, \dots$. Como la variable x es de tiempo continuo, su evolución suele representarse interpolando linealmente entre los valores calculados.

Supongamos que se ha ejecutado la simulación hasta el instante t_i . Se conoce por tanto $x_0, \dots, x_i$, que son los valores de x en los instantes $t_0, \dots, t_i$. El **método de integración** tiene, en general, la forma siguiente:

$$x_{i+1} = F\{f(x_{i+1}), f(x_i), f(x_{i-1}), \dots, x_i, x_{i-1}, x_{i-2}, \dots\} \quad (1.4)$$

lo cual simplemente indica que el valor de la variable en el instante $i + 1$ (es decir, x_{i+1}) se calcula a partir de los valores de la variable en instantes pasados $x_i, x_{i-1}, x_{i-2}, \dots$, y del valor de la derivada en ese mismo instante, $f(x_{i+1})$, y en instantes pasados $f(x_i), f(x_{i-1}), f(x_{i-2}), \dots$.

La diferencia entre los diferentes métodos de integración radica en la forma de F . Asimismo, ésta sirve de base para la clasificación de los métodos.

- **Método explícito o implícito.** El método de integración es explícito si el valor $f(x_{i+1})$ no interviene en la función F . En caso contrario se dice que el método es implícito. Por ejemplo, en la Tabla 1.2 se muestran las versiones explícita e implícita del **método de Euler**.

Tabla 1.2: Métodos de integración de Euler.

Explícito	$x_{i+1} = x_i + \Delta t \cdot f(x_i)$
Implícito	$x_{i+1} = x_i + \Delta t \cdot f(x_{i+1})$

- **Métodos de paso simple o múltiple.** Los métodos de paso simple son aquellos en los cuales intervienen en F únicamente el valor de la variable y su derivada en un intervalo de integración, es decir, x_i , $f(x_{i+1})$ y $f(x_i)$. Los métodos de paso múltiple, también llamados multipaso, son aquellos en los cuales intervienen en F también valores de la variable o de la derivada anteriores al instante i .
- **Orden del método.** El orden de un método de integración viene determinado por el mayor grado del polinomio $x(t)$ que es representado de manera exacta por x_i .

Supongamos, por ejemplo, que f tiene valor constante c . La solución exacta de la ecuación

$$\frac{dx}{dt} = c \quad (1.5)$$

es

$$x = x_0 + c \cdot t \quad (1.6)$$

que es un polinomio de grado uno en t . Un método de integración de orden uno daría resultados precisos. Sin embargo, el mismo método de integración introduciría errores si $f = a + b \cdot t$. La solución exacta a la ecuación

$$\frac{dx}{dt} = a + b \cdot t \quad (1.7)$$

es un polinomio de segundo orden

$$x = x_0 + a \cdot t + \frac{1}{2} \cdot b \cdot t^2 \quad (1.8)$$

Un método de integración de segundo orden daría en este caso resultados precisos.

Queda fuera del alcance de este texto ofrecer una descripción completa y rigurosa de los diferentes métodos de integración. Simplemente indicar que existen básicamente dos maneras de obtener expresiones adecuadas para F .

1. Aproximar x mediante los primeros términos de una serie de Taylor. El número de términos del desarrollo en serie se corresponde con el orden del algoritmo. Este planteamiento del método da lugar, por ejemplo, a los **métodos de Runge-Kutta**. En la Tabla 1.3 se muestran algunos de estos métodos. Obsérvese que son métodos explícitos, de paso simple.
2. Aproximar f mediante un polinomio, usando para ello los valores de f calculados en instantes pasados y, en el caso de los métodos implícitos, también en el instante actual. Métodos de integración de este tipo son los **métodos de Adams-Bashforth** y de **Adams-Moulton**, los cuales se muestran en la Tabla 1.4.

Cuanto mayor es el **orden del método** de integración, más precisos son los resultados obtenidos y más rápida es la simulación, ya que pueden emplearse mayores valores del paso de integración. Los métodos de integración más comúnmente empleados en Ingeniería tienen orden cuatro y cinco. Una regla práctica es la siguiente: emplear algoritmos de integración de orden k -ésimo cuando se quiera obtener una precisión de k decimales.

El **tamaño del paso** de integración Δt debe escogerse alcanzando un compromiso entre precisión y carga computacional. Cuanto menor sea el valor de Δt , menor es el error que se comete en el cálculo de las variables del modelo, pero mayor es el tiempo de ejecución de la simulación.

Un procedimiento para estimar el error cometido al escoger un determinado valor de Δt es comparar los resultados obtenidos usando ese valor y los obtenidos usando un valor menor, por ejemplo, $\frac{\Delta t}{2}$. Si la diferencia entre ambos resultados es aceptable para los propósitos del estudio de simulación que se está realizando, entonces el valor Δt es adecuado. En caso contrario, se comparan los resultados obtenidos usando $\frac{\Delta t}{2}$ y $\frac{\Delta t}{4}$. Si el error es aceptable, se emplea $\frac{\Delta t}{2}$. Si el error es demasiado grande, se investigan los valores $\frac{\Delta t}{4}$ y $\frac{\Delta t}{8}$, y así sucesivamente.

Este procedimiento de ajuste del tamaño del paso es conceptualmente muy sencillo, sin embargo no es eficiente desde el punto de vista computacional. Por ese motivo, los métodos numéricos de paso variable no emplean este procedimiento.

Tabla 1.3: Algunos métodos de integración de Runge-Kutta.

Runge-Kutta 1^{er} orden (Euler explícito)	$x_{i+1} = x_i + \Delta t \cdot f(x_i, t_i)$
Runge-Kutta 2^o orden	$k_1 = \Delta t \cdot f(x_i, t_i)$ $k_2 = \Delta t \cdot f(x_i + k_1, t_i + \Delta t)$ $x_{i+1} = x_i + \frac{1}{2} \cdot (k_1 + k_2)$
Runge-Kutta 4^o orden	$k_1 = \Delta t \cdot f(x_i, t_i)$ $k_2 = \Delta t \cdot f\left(x_i + \frac{k_1}{2}, t_i + \frac{\Delta t}{2}\right)$ $k_3 = \Delta t \cdot f\left(x_i + \frac{k_2}{2}, t_i + \frac{\Delta t}{2}\right)$ $k_4 = \Delta t \cdot f(x_i + k_3, t_i + \Delta t)$ $x_{i+1} = x_i + \frac{k_1}{6} + \frac{k_2}{3} + \frac{k_3}{3} + \frac{k_4}{6}$

Tabla 1.4: Métodos de integración basados en aproximaciones polinómicas. Por concisión, $f(x_i, t_i)$ se representa como f_i .

Orden	Adams-Bashforth
1	$x_{i+1} = x_i + \Delta t \cdot f_i$
2	$x_{i+1} = x_i + \frac{\Delta t}{2} \cdot (3 \cdot f_i - f_{i-1})$
3	$x_{i+1} = x_i + \frac{\Delta t}{12} \cdot (23 \cdot f_i - 16 \cdot f_{i-1} + 5 \cdot f_{i-2})$
4	$x_{i+1} = x_i + \frac{\Delta t}{24} \cdot (55 \cdot f_i - 59 \cdot f_{i-1} + 37 \cdot f_{i-2} - 9 \cdot f_{i-3})$
Orden	Adams-Moulton
1	$x_{i+1} = x_i + \Delta t \cdot f_{i+1}$
2	$x_{i+1} = x_i + \frac{\Delta t}{2} \cdot (f_{i+1} + f_i)$
3	$x_{i+1} = x_i + \frac{\Delta t}{12} \cdot (5 \cdot f_{i+1} + 8 \cdot f_i - f_{i-1})$
4	$x_{i+1} = x_i + \frac{\Delta t}{24} \cdot (9 \cdot f_{i+1} + 19 \cdot f_i - 5 \cdot f_{i-1} + f_{i-2})$

Tabla 1.5: Método de Runge-Kutta-Fehlberg (4°-5° orden).

	$k_1 = \Delta t \cdot f(x_i, t_i)$ $k_2 = \Delta t \cdot f\left(x_i + \frac{k_1}{4}, t_i + \frac{\Delta t}{4}\right)$ $k_3 = \Delta t \cdot f\left(x_i + \frac{3}{32} \cdot k_1 + \frac{9}{32} \cdot k_2, t_i + \frac{3}{8} \cdot \Delta t\right)$ $k_4 = \Delta t \cdot f\left(x_i + \frac{1932}{2197} \cdot k_1 - \frac{7200}{2197} \cdot k_2 + \frac{7296}{2197} \cdot k_3, t_i + \frac{12}{13} \cdot \Delta t\right)$ $k_5 = \Delta t \cdot f\left(x_i + \frac{439}{216} \cdot k_1 - 8 \cdot k_2 + \frac{3680}{513} \cdot k_3 - \frac{845}{4104} \cdot k_4, t_i + \Delta t\right)$ $k_6 = \Delta t \cdot f\left(x_i - \frac{8}{27} \cdot k_1 + 2 \cdot k_2 - \frac{3544}{2565} \cdot k_3 + \frac{1859}{4104} \cdot k_4 - \frac{11}{40} \cdot k_5, t_i + \frac{\Delta t}{2}\right)$
4° orden	$x_{i+1} = x_i + \frac{25}{216} \cdot k_1 + \frac{1408}{2565} \cdot k_3 + \frac{2197}{4104} \cdot k_4 - \frac{k_5}{5}$
5° orden	$x_{i+1} = x_i + \frac{16}{135} \cdot k_1 + \frac{6656}{12825} \cdot k_3 + \frac{28561}{56430} \cdot k_4 - \frac{9}{50} \cdot k_5 + \frac{2}{55} \cdot k_6$

Un procedimiento más eficiente es emplear dos algoritmos de integración, uno de mayor precisión (mayor orden) que el otro, y de modo que el de menor precisión esté embebido en el de mayor. La diferencia entre los resultados obtenidos con el algoritmo de mayor orden y el de menor orden puede considerarse una estimación del error cometido con el algoritmo de menor orden.

La estrategia anterior puede emplearse para programar un algoritmo de integración de paso variable: a cada paso de integración se estima el error cometido y si excede el máximo permitido se vuelve atrás, reduciendo el tamaño del paso de integración. El algoritmo incluye también una estrategia para aumentar el tamaño del paso de integración. Por ejemplo, se dobla su valor cuando el error está por debajo de cierto límite durante un determinado número de pasos de integración consecutivos.

Por ejemplo, el **método de Runge-Kutta-Fehlberg** (RKF45) es un método de cuarto orden embebido en uno de quinto orden (véase la Tabla 1.5). El error cometido al aplicar el método de cuarto orden se obtiene restando las funciones paso de ambos métodos. El método pueden programarse con paso variable: el algoritmo de integración va estimando el error cometido, y reduce o aumenta en consecuencia el tamaño del paso de integración.

Los **métodos BDF** (*Backward Differentiation Formula*) tiene la siguiente función de paso:

$$x_{i+1} = \sum_{k=0}^{q-1} \alpha_k \cdot x_{i-k} + \beta_0 \cdot f(x_{i+1}, t_{i+1}) \cdot \Delta t \quad (1.9)$$

donde q es el orden del método, y α_k , β_0 son constantes que dependen del orden y que han sido seleccionadas de manera que el algoritmo resultante tenga buenas propiedades. Al inicio de la simulación sólo se conoce el valor inicial, x_0 , con lo cual el algoritmo arranca con la función de paso de orden uno. Al irse calculando la solución en sucesivos instantes de tiempo, puede irse incrementando el orden. Puesto que el método ajusta también el paso de integración, se trata de un método de paso y orden variables.

1.4.7. Modelos híbridos

Los modelos híbridos combinan la descripción de comportamiento de tiempo continuo y de eventos discretos. La parte de tiempo continuo es modelada empleando ecuaciones algebraicas y diferenciales ordinarias, y la parte de eventos discretos empleando ecuaciones en diferencias.

La simulación de modelos híbridos se realiza empleando algoritmos específicos, que combinan la resolución numérica del modelo de tiempo continuo, con la detección y ejecución de los eventos discretos. El estado de un modelo híbrido evoluciona mediante el cambio continuo de sus variables de estado de tiempo continuo, y mediante cambios instantáneos en su estado total, de tiempo continuo y discreto, llamados **eventos**. La descripción de un evento consta de dos partes:

1. La **condición de disparo**, también llamada **condición de activación** o simplemente **condición** del evento. Dependiendo de su condición de activación, los eventos se clasifican en eventos en el estado y en el tiempo.
 - **Eventos en el estado** son aquellos cuya condición es función de variables de tiempo continuo del sistema.
 - **Eventos en el tiempo** son aquellos cuya condición no depende de variables de tiempo continuo. La condición puede depender del tiempo y de variables de tiempo discreto.
2. La **acción** a realizar cuando se dispara el evento. Ésta típicamente consiste en producir un cambio instantáneo en el estado total del modelo o en cambiar el conjunto de ecuaciones que describen el comportamiento del modelo. Se dice que un modelo tiene **estructura variable** cuando su descripción matemática puede cambiar durante la simulación (esto se explicará con detalle en el Tema 4).

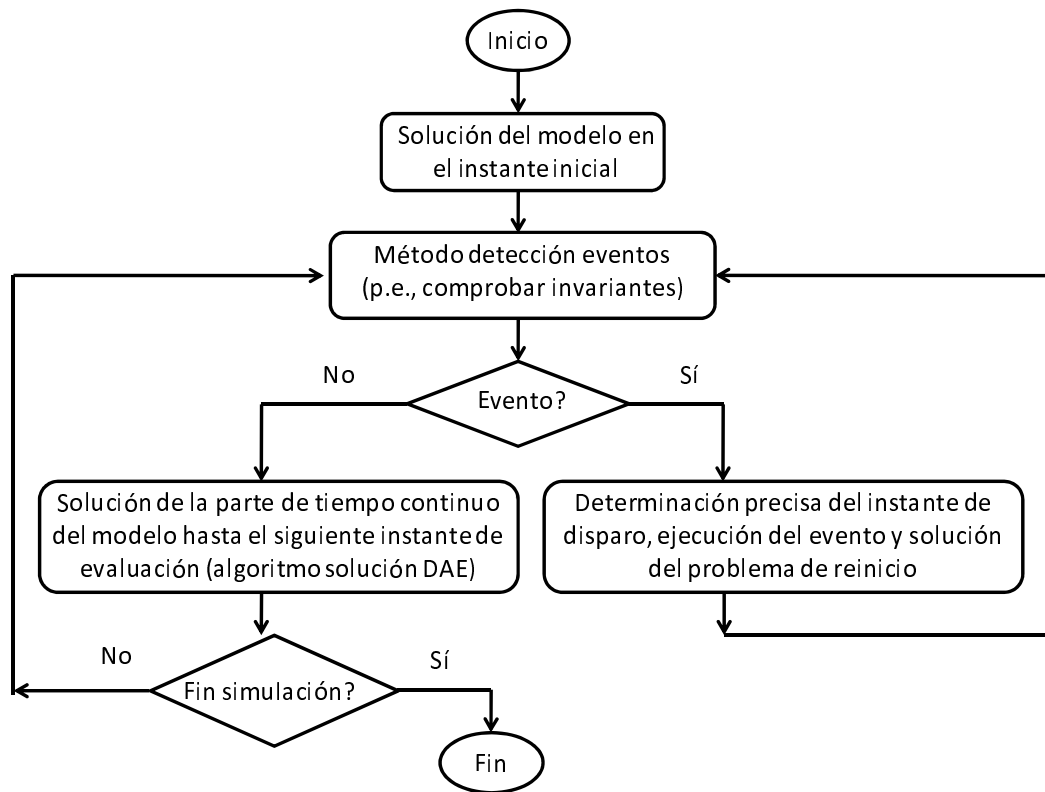


Figura 1.12: Algoritmo para la simulación de modelos híbridos.

Los instantes en que se satisfacen las condiciones de los eventos en el tiempo pueden planificarse con antelación. La gestión de este tipo de evento puede realizarse empleando un calendario de eventos, como se explicó en la Sección 1.4.3.

Por el contrario, en general no es posible planificar los eventos en el estado. La detección de los eventos en el estado debe realizarse durante la resolución del problema de tiempo continuo. Para ello, la condición de disparo de cada evento en el estado se expresa como una expresión booleana denominada **función invariante**, de tal manera que en el preciso instante en que se satisface la condición de evento la función invariante asociada pasa de valer *true* a valer *false*. Así pues, durante la solución del problema continuo se examinan las funciones invariantes. Si alguna pasa de *true* a *false* se suspende la solución del problema continuo y se ejecuta el evento correspondiente.

En la Figura 1.12 se muestra un algoritmo para la simulación de modelos híbridos. Puede observarse que el problema de la simulación de modelos híbridos se descompone tres subproblemas:

1. Resolución del problema de tiempo continuo.
2. Detección de los eventos y estimación del instante de disparo.

3. Solución del problema de reinicio.

La **resolución del problema de tiempo continuo** se realiza aplicando métodos numéricos de integración, y de resolución de sistemas simultáneos de ecuaciones lineales y no lineales. El objetivo es calcular la evolución de las variables de tiempo continuo. Durante la resolución del problema de tiempo continuo, el valor de las variables de tiempo discreto es conocido y se mantiene constante.

La resolución del problema de tiempo continuo va avanzando a saltos en el eje temporal, cuya longitud viene determinada por el tamaño del paso del método de integración Δt . Una vez evaluadas las variables de tiempo continuo en el instante t_i , se **evalúan los invariantes**. Si alguno de ellos vale *false*, es indicación de que la condición de disparo del evento asociado se verifica en algún instante del intervalo definido por el último paso de integración, $(t_{i-1}, t_i]$. Se inicia entonces un proceso iterativo para determinar de manera precisa en qué instante, dentro del último paso de integración, se produce el cambio en el valor del invariante. Este proceso se denomina **localización del evento**.

Una vez acotado dentro de un pequeño intervalo (la longitud máxima de dicho intervalo es un parámetro numérico que determina la precisión en la localización de los eventos) el instante en que se produce el cambio en el valor del invariante, se retrocede el reloj de la simulación hasta el extremo derecho de dicho intervalo y en ese instante de tiempo se resuelve el modelo, incluyendo las ecuaciones que describen las acciones asociadas al evento o eventos disparados. La ejecución de un evento conlleva en general cambios en los valores de variables de tiempo continuo y de tiempo discreto. Se denomina a esto la **solución del problema de reinicio**.

A continuación se vuelven a evaluar los invariantes. En ocasiones la solución del problema de reinicio puede no satisfacer (hacer true) uno de los invariantes, con lo cual se dispararía el evento correspondiente. De esta manera, se van disparando eventos en secuencia hasta que todos los invariantes son satisfechos, reiniciándose entonces la solución del problema continuo, tomando como valor inicial el estado tras el evento.

En el Tema 4 se explicará con más detalle el algoritmo de simulación de modelos híbridos, con lo cual en este punto el objetivo es únicamente entender el concepto general. Dejamos ahora de lado los aspectos relativos a la simulación, a fin de explicar muy brevemente algunos aspectos relativos al modelado.

Existe gran variedad de entornos (herramientas software) y lenguajes desarrollados para el modelado de sistemas híbridos. En general, pueden englobarse en uno de los dos tipos siguientes:

1. **Entornos de simulación**, que facilitan el **modelado basado en diagramas de bloques**. Este paradigma de modelado permite realizar una descripción modular y jerárquica de la representación matemática del modelo. El desarrollador del modelo debe manipularlo y expresarlo de manera que quede definido explícitamente qué variable debe evaluarse de cada una de las ecuaciones del modelo. Dos ejemplos de entorno de simulación para modelos híbridos son Matlab/Simulink y Scilab/Scicos.
2. **Lenguajes de modelado orientado a objetos**, que facilitan describir el modelo simplemente escribiendo sus ecuaciones. Las herramientas software que soportan estos lenguajes, denominadas **entornos de modelado**, son las encargadas de decidir qué variable se evalúa de cada ecuación, y de ordenar y manipular el modelo de manera que éste pueda ser resuelto numéricamente. El hecho de que el desarrollador del modelo quede liberado de realizar esas tareas agiliza enormemente el desarrollo y modificación de los modelos, y posibilita su reutilización. Dos ejemplos de lenguajes de modelado orientado a objetos son Modelica y EcosimPro.

En este texto nos enfocaremos en la metodología del modelado orientado a objetos, empleando el lenguaje Modelica. Volveremos sobre ello en el Tema 2.

1.4.8. Modelos en derivadas parciales

Las ecuaciones en derivadas parciales (PDE) se emplean en la descripción de procesos que suelen estar distribuidos en el espacio y en el tiempo. Es decir, problemas donde existe una función desconocida cuyos valores se desea calcular y esta función depende de más de una variable. Las variables de las que depende la función las denominamos variables independientes.

Por ejemplo, supongamos que deseamos conocer la distribución de temperatura a lo largo de una varilla. Nuestra función desconocida es la temperatura (T) y esta temperatura va a depender del instante de tiempo (t) y de la posición (x, y, z). Para obtener la temperatura, $T(t, x, y, z)$, tendremos que resolver una PDE.

Por otro lado, cuando esta función desconocida depende de una única variable, el problema se describe mediante ODE. En el caso de los sistemas dinámicos, esta variable independiente es el tiempo y los problemas se describen mediante ODE en el tiempo.

Las PDE se usan, por ejemplo, para describir la propagación del sonido, la propagación del calor, procesos de electrostática, electrodinámica, dinámica de fluidos, elasticidad, etc. Algunos de estos problemas serán estudiados en el Capítulo 5. En esta sección se define la notación matemática que se va a emplear a lo largo de este libro para expresar las ecuaciones diferenciales ordinarias y parciales. Se describen las características, los tipos (lineal o no lineal, homogénea o inhomogénea) y el grado de las PDE. También se explica un método numérico para resolver PDE: el método de las líneas.

Una **ecuación diferencial** para la función $u(x_1, x_2, \dots, x_n)$ se puede expresar de la forma general siguiente:

$$F\left(x_1, \dots, x_n, u, \frac{\partial}{\partial x_1}u, \dots, \frac{\partial}{\partial x_n}u, \frac{\partial^2}{\partial x_1 \partial x_1}u, \dots, \frac{\partial^2}{\partial x_1 \partial x_2}u, \dots\right) = 0 \quad (1.10)$$

donde F es una función de u y de las variables independientes $x_1, \dots, x_n$.

También es muy frecuente usar una notación de subíndices para indicar las derivadas parciales. Esto es, siguiendo la notación de subíndices las derivadas parciales de $u(x, y)$ se expresan del siguiente modo:

$$u_x \equiv \frac{\partial u}{\partial x} \quad (1.11)$$

$$u_y \equiv \frac{\partial u}{\partial y} \quad (1.12)$$

$$u_{xy} \equiv \frac{\partial^2 u}{\partial y \partial x} = \frac{\partial}{\partial y} \left(\frac{\partial u}{\partial x} \right) \quad (1.13)$$

Para especificar que la derivada parcial de la función $u(x, y)$ se está evaluando en un punto (x_0, y_0) , vamos a emplear la siguiente notación a lo largo del libro:

$$\frac{\partial u(x_0, y_0)}{\partial x} \quad (1.14)$$

$$\frac{\partial u(x_0, y_0)}{\partial y} \quad (1.15)$$

$$\frac{\partial^2 u(x_0, y_0)}{\partial y \partial x} \quad (1.16)$$

También es común encontrar la siguiente notación para expresar el punto donde se evalúan las derivadas:

$$\left. \frac{\partial u(x, y)}{\partial x} \right|_{x_0, y_0} \quad (1.17)$$

$$\left. \frac{\partial u(x, y)}{\partial y} \right|_{x_0, y_0} \quad (1.18)$$

$$\left. \frac{\partial^2 u(x, y)}{\partial y \partial x} \right|_{x_0, y_0} \quad (1.19)$$

Dependiendo del número de variables independientes que tenga la función desconocida u , la ecuación será diferencial ordinaria (ODE) o diferencial parcial (PDE). El orden de la derivada más alta en una ecuación diferencial será el orden de la ecuación. La mayor parte de las ecuaciones que se emplean en ciencia e ingeniería son de primer y de segundo orden. A continuación se dan ejemplos de ambos tipos de ecuaciones.

En una **ecuación diferencial ordinaria** (ODE), la función desconocida u tiene una única variable independiente y sólo existen derivadas simples en la ecuación. Las soluciones de las ODE son funciones de la variable independiente. La solución general de una ODE de orden m contendrá m constantes arbitrarias. Para obtener la solución de la ODE necesitaremos especificar condiciones iniciales para así determinar el valor de dichas constantes. En consecuencia, el número de condiciones iniciales dependerá del grado de la ODE. Así, en una ODE de orden m necesitaremos especificar m condiciones iniciales. Las siguientes ecuaciones son ODE:

$$\frac{du}{dt} = 2 \cdot u \quad (1.20)$$

$$\frac{d^2 u}{dx^2} + 6 \cdot \frac{du}{dx} + 5 \cdot u = 0 \quad (1.21)$$

La Ec. (1.20) es una ODE de primer orden cuya variable independiente es t , es decir, $u = u(t)$. La solución general de esta ecuación es $u(t) = C_1 \cdot e^{2 \cdot t}$, donde C_1 es una constante arbitraria. Par obtener el valor de C_1 debemos proporcionar una condición inicial. Esta condición inicial puede ser el valor de u en $t = 0$, es decir, $u(t = 0)$.

La Ec. (1.21) es una ODE de segundo orden cuya variable independiente es x , es decir, $u = u(x)$. La solución general de esta ecuación es $u(x) = C_1 \cdot e^{-x} + C_2 \cdot e^{-5x}$, donde C_1 y C_2 son constantes arbitrarias. Para determinar el valor de estas dos constantes debemos dar dos condiciones iniciales.

En una **ecuación diferencial parcial**, también denominada **ecuación en derivadas parciales** (PDE), la función desconocida u tiene dos o más variables independientes. A continuación se dan tres ejemplos de PDE.

$$\frac{\partial u(x, t)}{\partial x} = 0 \quad (1.22)$$

$$\frac{\partial^2 u(x, t)}{\partial x \partial t} = 3 \quad (1.23)$$

$$\frac{\partial^2 u(x, y)}{\partial x^2} + u(x, y) \cdot \frac{\partial u(x, y)}{\partial y} = 0 \quad (1.24)$$

La Ec. (1.22) es una PDE de primer orden con variables independientes t y x . La solución general de esta ecuación puede obtenerse integrando respecto a x . Realizando la integración obtenemos $u(x, t) = f(t)$, donde $f(t)$ es cualquier función arbitraria de t . Obsérvese que mientras la solución general de ODE contienen constantes arbitrarias, la solución general de PDE contienen funciones arbitrarias de las variables independientes.

La Ec. (1.23) es una PDE de segundo orden con variables independientes t y x . La solución general de esta ecuación puede obtenerse integrando respecto a x y a t . Integrando con respecto a t tratando a x como una constante obtenemos $\frac{\partial u}{\partial x} = 3 \cdot t + f(x)$, donde $f(x)$ es una función arbitraria de x . Después integramos con respecto de x tratando a t como constante y obtenemos $u(x, t) = 3 \cdot x \cdot t + F(x) + g(t)$. Donde $F(x) = \int f(x) \cdot dx$ y donde $g(t)$ es una función arbitraria de t . Como $f(x)$ es una función arbitraria de x , también lo es $F(x)$.

La Ec. (1.24) es una PDE de segundo orden con variables independientes x e y .

Una solución de una PDE generalmente no es única, de tal forma que se tienen que proporcionar condiciones adicionales de contorno capaces de definir la solución de forma única. Por ejemplo, en el caso de la PDE (1.22) la función $f(t)$ puede determinarse si u se especifica sobre la línea $x = 0$. En el Capítulo 5 se explicarán con más detalles qué son y cuáles son los tipos de las condiciones de contorno más comunes.

Para que la ecuación diferencial sea **lineal**, tanto la función desconocida u como sus derivadas deben aparecer sin multiplicarse, no han de estar elevadas a potencias distintas de 1 ó 0, y no deben aparecer como argumentos de funciones no lineales (funciones trigonométricas, exponencial, logaritmos, etc.)

Por ejemplo, la forma general de una PDE lineal de segundo orden de dos variables independientes es la siguiente:

$$\begin{aligned} a_{11}(x, y) \cdot \frac{\partial^2 u}{\partial x^2} + a_{12}(x, y) \cdot \frac{\partial^2 u}{\partial x \partial y} + a_{22}(x, y) \cdot \frac{\partial^2 u}{\partial y^2} \\ + a_1(x, y) \cdot \frac{\partial u}{\partial x} + a_2(x, y) \cdot \frac{\partial u}{\partial y} + a_0(x, y) \cdot u = f(x, y) \end{aligned} \quad (1.25)$$

Una ecuación diferencial lineal es **homogénea** si la función cero es una solución de dicha ecuación. En caso contrario, se dice que la ecuación es **inhomogénea**.

Las ecuaciones homogéneas cumplen una propiedad importante: dadas dos soluciones u y v de la ecuación diferencial lineal homogénea, una combinación lineal de las soluciones es también solución de la ecuación. Es decir,

$$\alpha \cdot u + \beta \cdot v \quad (1.26)$$

donde α y β son cualquier número real, es solución de la ecuación.

Podemos expresar la ecuación diferencial lineal agrupando todos los términos que involucran la función desconocida de la ecuación diferencial lineal en el lado izquierdo de la ecuación y todos los términos que no involucran la función desconocida en el lado derecho. Para una ecuación expresada de este modo, la ecuación es homogénea si y sólo si el lado derecho de la ecuación es idéntico a cero.

Por ejemplo, para que la PDE (1.25) sea homogénea $f(x, y)$ ha de ser cero. La ecuación

$$\frac{\partial u(x, y)}{\partial x} - 3 \cdot u(x, y) = \text{sen}(2 \cdot \pi \cdot x) \quad (1.27)$$

es una PDE lineal, de primer orden, no homogénea.

Un método para resolver numéricamente una PDE es el **método de las líneas** (MOL). Este método consiste en convertir la PDE en un conjunto equivalente de

ecuaciones diferenciales ordinarias, que son entonces resueltas empleando métodos numéricos para ODE.

Por ejemplo, consideremos la siguiente PDE:

$$\frac{\partial u(x, t)}{\partial t} - b \cdot \frac{\partial^2 u(x, t)}{\partial x^2} = f(x) \quad \text{en } \Omega = \left\{ x, t : \begin{array}{l} 0 \leq x \leq 1 \\ 0 \leq t \leq T \end{array} \right\} \quad (1.28)$$

donde Ω es el dominio donde está definida la PDE. Esta PDE tiene una condición inicial (en $t = 0$) dependiente de x , y condiciones de contorno (en $x = 0$ y $x = 1$) dependientes del tiempo:

$$\begin{aligned} u(x, t = 0) &= u_0(x) & u(x = 0, t) &= \alpha_1(t) \\ u(x = 1, t) &= \alpha_2(t) \end{aligned} \quad (1.29)$$

donde $u_0(x)$ es una función conocida de x , y α_1 y α_2 son funciones conocidas dependientes de t . Veamos cómo llegar al conjunto de ODE equivalente aplicando el MOL.

1. El primer paso consiste en realizar una discretización espacial del dominio Ω . Para ello, definimos en el dominio Ω los $N + 1$ puntos x_i siguientes:

$$x_i = i \cdot \Delta x \quad \text{con} \quad \begin{cases} i &= 0, 1, \dots, N \\ \Delta x &= \frac{1}{N} \end{cases} \quad (1.30)$$

Se desean conocer los valores de la función $u(x, t)$ en los puntos x_i . Denominamos a estos valores $u_i(t)$, donde $u_i(t) = u(x_i, t)$, e $i = 0, \dots, N$.

2. El segundo paso consiste en eliminar las derivadas espaciales de la ecuación. Para ello, se sustituye la derivada parcial de u de segundo orden respecto a x por una aproximación en diferencias finitas. Escogemos la siguiente aproximación:

$$\frac{d^2 u(x_i)}{dx^2} = \frac{u_{i+1} - 2 \cdot u_i + u_{i-1}}{\Delta x^2} + O(\Delta x^2) \quad (1.31)$$

Reemplazando la derivada espacial de u por la anterior aproximación, obtenemos las $N - 1$ ecuaciones siguientes:

$$\frac{du_i}{dt} = b \cdot \underbrace{\frac{u_{i-1}(t) - 2 \cdot u_i(t) + u_{i+1}(t)}{\Delta x^2}}_{\frac{\partial^2 u(x,t)}{\partial x^2}} + f_i \quad \text{con } i = 1, \dots, N-1 \quad (1.32)$$

con las condiciones de frontera:

$$u_0 = \alpha_1(t) \quad u_N = \alpha_2(t) \quad (1.33)$$

Esto puede ser escrito en forma matricial como

$$\begin{aligned} \frac{d}{dt} \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{N-2} \\ u_{N-1} \end{pmatrix} &= \frac{b}{\Delta x^2} \cdot \begin{pmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & 0 & \cdots & 0 \\ & & \ddots & \ddots & \ddots & \\ 0 & \cdots & 0 & 1 & -2 & 1 \\ 0 & & \cdots & 0 & 1 & -2 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{N-2} \\ u_{N-1} \end{pmatrix} \\ &+ \begin{pmatrix} f_1 + \frac{b \cdot \alpha_1(t)}{\Delta x^2} \\ f_2 \\ \vdots \\ f_{N-2} \\ f_{N-1} + \frac{b \cdot \alpha_2(t)}{\Delta x^2} \end{pmatrix} \end{aligned} \quad (1.34)$$

es decir,

$$\frac{d\mathbf{u}(t)}{dt} = \mathbf{A} \cdot \mathbf{u}(t) + \mathbf{s}(t) \quad (1.35)$$

Hemos pasado de tener un sistema descrito por una ecuación en derivadas parciales con derivada parcial de primer orden en t y derivada espacial de segundo orden en x , a tener un sistema de $N-1$ ecuaciones diferenciales ordinarias con derivadas en t . Este sistema puede ser resuelto por cualquier método de integración para problemas de valor inicial como los mostrados en la Sección 1.4.6.

La estabilidad numérica de la integración depende de los autovalores de la matriz $\mathbf{A}$ que resulta de la discretización espacial. Para este ejemplo, los autovalores varían entre 0 y $-(4 \cdot \alpha / \Delta x^2)$. A medida que el valor de Δx tiende a cero, el sistema puede tener autovalores con órdenes de magnitud muy diferentes, pudiendo llegar a ser rígido (*stiff*).

Existe multitud de entornos, tanto comerciales como gratuitos, que facilitan la simulación de PDE. Algunos de ellos son COMSOL Multiphysics, FlexPDE, Partial Differential Equation Toolbox de Matlab, aunque se podrían haber escogido otros. En el Capítulo 5 se describen los métodos numéricos para resolver PDE que emplean habitualmente estos entornos y se describen sus características básicas. Se ha escogido el entorno comercial FlexPDE para resolver sistemas de PDE, ya que tiene una versión gratuita de capacidades limitadas, pero amplias, y es de propósito general, es decir, permite resolver problemas pertenecientes a múltiples dominios. Este entorno se describirá en el Capítulo 5, mostrándose múltiples ejemplos.

1.5. INTRODUCCIÓN AL ANÁLISIS DE DATOS CON R

El análisis de los datos es una parte esencial en un proyecto de simulación. Por una parte es preciso analizar los datos del sistema bajo estudio, con el fin de modelar su comportamiento. Por otra parte es preciso analizar los datos obtenidos como resultado de la simulación del modelo, con el fin de extraer conclusiones.

Por ello, los entornos de simulación empleados en Ingeniería a menudo facilitan el análisis de datos. Algunas de estas herramientas permiten aplicar una gran variedad de técnicas para el análisis y modelado de los datos. Sin embargo, lo más habitual es que las capacidades para análisis de datos ofrecidas por los entornos de simulación se limiten a la realización de unas determinadas representaciones gráficas básicas de los datos. En este caso, el software de modelado y simulación debe frecuentemente usarse en combinación con alguna herramienta especializada en el análisis y modelado de los datos.

Existe una amplia variedad de herramientas especializadas en el análisis y modelado de los datos. En esta sección se presenta una de ellas: el lenguaje R; así como una de las plataformas software que soporta el lenguaje, denominada RGui. Se trata de software gratuito y de código abierto, con versiones para los sistemas operativos Windows, Mac OS X y Linux. Puede descargarse del Comprehensive R Archive Network (CRAN), que está ubicado en <https://cran.r-project.org/>.

R es un lenguaje interpretado, en el cual se distingue entre letras mayúsculas y minúsculas. Es posible ir introduciendo los comandos uno a uno en la línea de comandos de la consola de RGui, tras el símbolo del sistema ($>$), o bien ejecutar un conjunto de comandos escritos en un fichero *script*. En la Figura 1.13 se muestra la consola de RGui en Windows.

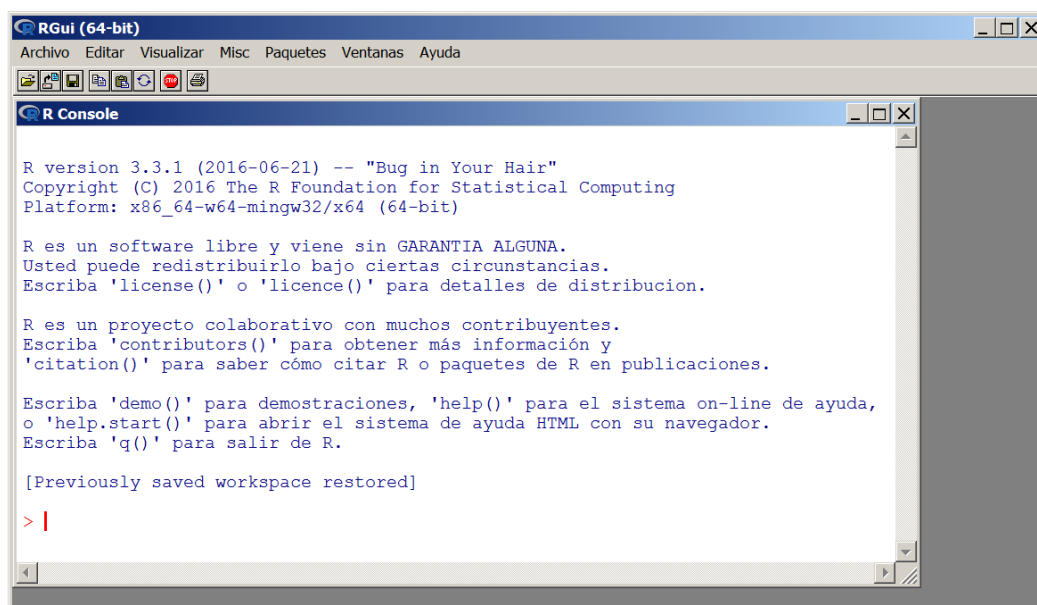


Figura 1.13: Consola de RGui en Windows, en el momento de arrancar.

El lenguaje R tiene una gran variedad de tipos de datos, incluyendo vectores, matrices, data frames y listas. La mayor parte de la funcionalidad se consigue mediante el uso de funciones, tanto las proporcionadas por el lenguaje como las definidas por el propio usuario. Algunas funciones básicas están disponibles por defecto. Otras se encuentran en paquetes, que deben ser descargados e instalados para poder usar las funciones que contienen. A la fecha de escritura de este texto, en el sitio web <https://cran.r-project.org/web/packages/> hay disponibles para su descarga más de 8700 paquetes.

Las sentencias consisten en funciones y asignaciones. R usa el símbolo `<-` para las asignaciones, en lugar del típico símbolo `=`. Por ejemplo,

```
> x <- rnorm(10)
```

crea un objeto de tipo vector llamado `x`, que contiene 10 observaciones de la distribución normal estándar.

Los comentarios son precedidos por el símbolo `#`. El intérprete de R ignora todo el texto que aparezca tras el símbolo `#`. Para finalizar una sesión debe ejecutarse la función `q()`.

La función `c()`, cuyo nombre proviene del término inglés *combine*, convierte sus argumentos en un vector o una lista. Por ejemplo, supongamos que se realizan 5 réplicas independientes de la simulación del modelo de la oficina descrita en la Sección 1.4.3, obteniéndose los siguientes cinco valores del tiempo medio de espera

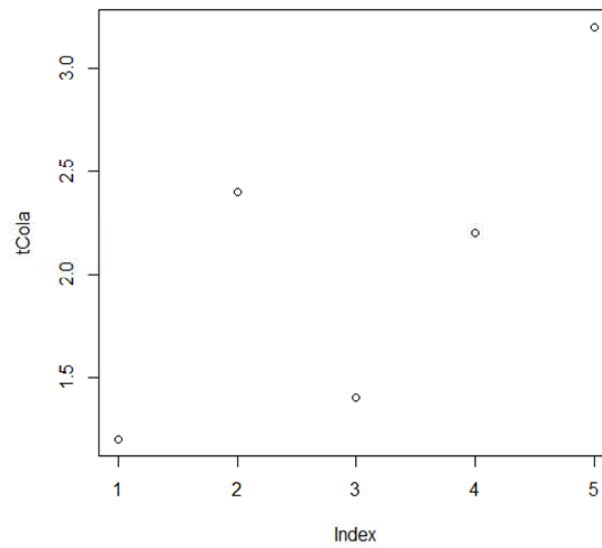


Figura 1.14: Ejemplo de uso de la función *plot*.

en cola: 1.2, 2.4, 1.4, 2.2 y 3.2 minutos. La primera de las siguientes dos sentencias crea un objeto del tipo vector llamado `tCola`, cuyos componentes son esos cinco valores. La segunda sentencia dibuja el valor del componente del vector frente al índice. En la Figura 1.14 se muestra el gráfico.

```
> tCola <- c(1.2, 2.4, 1.4, 2.2, 3.2)
> plot(tCola)
```

La función `help()` permite acceder a la documentación de las funciones. Por ejemplo,

```
> help(plot)
```

abre una ventana en la cual se muestra la documentación para la función `plot`.

R incluye demos que ilustran algunas de sus capacidades para la representación gráfica. Por ejemplo,

```
> demo(graphics)
```

arranca una de ellas. Otras dos demos relacionadas con las capacidades gráficas se arrancan mediante: `demo(persp)` y `demo(image)`. Puede obtenerse la lista completa de demos ejecutando:

```
> demo()
```

1.5.1. El espacio de trabajo

El **espacio de trabajo** es el espacio de memoria en el cual se guardan todos los objetos creados durante la sesión. Es posible salvar el espacio de trabajo al finalizar la sesión (RGui pregunta antes de cerrarse si debe hacerlo), de modo que automáticamente sea vuelto a cargar la siguiente vez que se arranque RGui. Puede obtenerse una lista de todos los objetos del espacio de trabajo mediante:

```
> ls()
```

La función `rm()` elimina del espacio de trabajo los objetos que se le pasan como argumento. Por ejemplo,

```
> rm(x)
```

elimina del espacio de trabajo el objeto llamado `x`. La siguiente sentencia elimina todos los objetos del espacio de trabajo

```
> rm(list = ls())
```

Mediante el paso de argumentos a la función `ls()`, se pueden filtrar los objetos atendiendo, por ejemplo, a que su nombre obedezca un determinado patrón. Puede accederse a la documentación de la función ejecutando `help("ls")`.

El **directorio de trabajo** es donde RGui salva por defecto los resultados y también de donde intenta por defecto leer los ficheros. Para saber cuál es el directorio de trabajo hay que ejecutar

```
> getwd()
```

Es recomendable separar en diferentes directorios los diferentes proyectos. La función `setwd()` permite modificar el directorio de trabajo, pasando como argumento a la función el nombre del nuevo directorio escrito entre comillas. Por ejemplo:

```
> setwd("F:/Simulacion")
```

La función `save.image` salva el espacio de trabajo completo a un fichero. Para salvar únicamente algunos objetos puede emplearse `save()`. La función `load()` carga en la sesión actual el espacio de trabajo almacenado en el fichero que se le pasa como argumento. Si se desea acceder a un fichero que no se encuentra en el directorio de trabajo, es necesario especificar el path completo en la llamada.

RGui guarda memoria de los comandos ejecutados durante la sesión. Con las flechas del teclado es posible moverse por la historia de comandos. Las funciones

`savehistory()` y `loadhistory()` permiten salvar a fichero los comandos de la sesión y cargar dicha historia desde un fichero.

Es posible escribir los comandos en un fichero de texto y cargar este **fichero de comandos** desde RGui, de manera que se ejecuten en secuencia todos los comandos escritos en él. Por convenio, el nombre del fichero con los comandos (también llamado *fichero script*) se escribe con extensión `.R`. La función `source()` permite cargar y ejecutar el fichero. Por ejemplo,

```
> source("F:/Simulacion/script1.R")
```

carga y ejecuta el fichero de comandos llamado *script1.R*. En la práctica suele resultar más cómodo trabajar empleando ficheros de comandos que hacerlo escribiendo directamente las sentencias en la consola de R.

1.5.2. Estructuras de datos

El primer paso en el análisis de los datos es crear una estructura de datos que contenga los datos a estudiar. Los datos pueden ser cargados en la estructura de datos bien manualmente o bien pueden ser importados desde una fuente externa. El lenguaje R tiene básicamente cinco tipos diferentes de estructura de datos: vector, matriz, array, data frame y lista. Los vectores, matrices y arrays contienen números y tienen una, dos o más dimensiones.

Un data frame es una tabla bidimensional, pero más general que una matriz, dado que en el data frame el tipo de los datos almacenados en una columna puede diferir del tipo de los datos almacenados en otra. Los datos almacenados en una misma columna deben ser del mismo tipo: cadena de caracteres, numérico o lógico (TRUE, FALSE).

Las sentencias siguientes crean un data frame llamado `exp1` con tres columnas (las columnas del data frame se denominan sus *variables*):

```
> numOper <- c(1,1,2,2)
> horario <- c("std", "nuevo", "std", "nuevo")
> tCola   <- c(12.3, 6.1, 5.2, 2.5)
> exp1    <- data.frame(numOper, horario, tCola)
```

Escribiendo el nombre de un objeto, R muestra su valor. En este caso, escribiendo `exp1` se obtiene:

```
> exp1
```

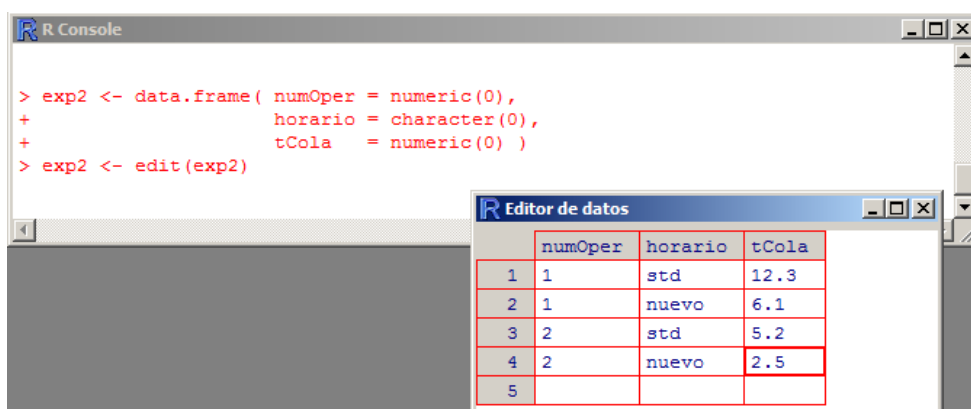


Figura 1.15: Ejemplo de uso de la función `edit` para editar el contenido de un data frame.

```

numOper horario tCola
1      1      std 12.3
2      1    nuevo 6.1
3      2      std 5.2
4      2    nuevo 2.5

```

Otra forma de introducir o modificar los datos del data frame es mediante la función `edit()`. Para ello, en primer lugar debe crearse el objeto vacío, especificando el nombre y tipo de las variables. A continuación, se invoca el editor de texto mediante la función `edit()` y se introducen los datos empleando dicho editor. Por ejemplo:

```

> exp2 <- data.frame( numOper = numeric(0),
+                     horario = character(0),
+                     tCola   = numeric(0) )
> exp2 <- edit(exp2)

```

En la Figura 1.15 se muestra el aspecto de la ventana de edición una vez se han introducido en ella los datos.

Otra forma de introducir datos en el data frame es importarlos desde un fichero de texto. Para ello puede emplearse la función `read.table()`. Supongamos que el contenido del fichero *data.txt* es:

```

numOper horario tCola
1      std    12.3
1      nuevo  6.1
2      std    5.2
2      nuevo  2.5

```

Obsérvese que en la primera fila se han escrito los nombres de las variables. En las siguientes filas están los valores separados por un delimitador, que en este caso es el espacio en blanco (puede usarse cualquier otro símbolo como delimitador).

Mediante las dos siguientes sentencias se crea un data frame llamado `exp3`, se cargan en él valores contenidos en el fichero `data.txt` y a continuación se muestra el contenido de `exp3`. Si el fichero con los datos no se encuentra en el directorio de trabajo, debe especificarse el path completo. Mediante `header=TRUE` se indica que la primera fila del fichero contiene los nombres de las variables.

```
> exp3 <- read.table("data.txt", header=TRUE)
> exp3
  numOper horario tCola
1      1      std  12.3
2      1    nuevo   6.1
3      2      std   5.2
4      2    nuevo   2.5
```

Puede accederse a una variable del data frame indicando su número de orden. La primera variable es la columna 1, la segunda la 2 y así sucesivamente. Otra posibilidad es escribir el nombre del data frame, seguido del símbolo `$` y del nombre de la variable. Por ejemplo, las dos siguientes sentencias son equivalentes:

```
> exp1[,3]
[1] 12.3  6.1  5.2  2.5
> exp1$tCola
[1] 12.3  6.1  5.2  2.5
```

De forma similar puede accederse a los elementos de cada variable. Por ejemplo, mediante estas dos sentencias se accede al cuarto elemento de la variable `tCola` (los índices en R comienzan en el valor 1):

```
> exp1$tCola[4]
[1] 2.5
> exp1[4,3]
[1] 2.5
```

Las funciones mostradas en la Tabla 1.6 facilitan la manipulación de los datos. La función `pretty()` se emplea a menudo para dibujar gráficos. Los siguientes son algunos ejemplos de uso de estas funciones:

```
> x <- seq(1,10,2)
> x
[1] 1 3 5 7 9
```

Tabla 1.6: Algunas funciones de R.

Función	Significado
<code>length(x)</code>	Devuelve la longitud del objeto <code>x</code> .
<code>seq(desde,hasta,paso)</code>	Devuelve una secuencia.
<code>rep(x,n)</code>	Repite <code>x</code> <code>n</code> veces.
<code>pretty(x,n)</code>	Divide <code>x</code> en aproximadamente <code>n</code> intervalos iguales, de modo que los puntos de división tengan valores redondeados. Devuelve los puntos de división.

```

> length(x)
[1] 5
> y <- rep(x,2)
> y
[1] 1 3 5 7 9 1 3 5 7 9
> pretty(c(1,5),10)
[1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0

```

1.5.3. Gráficos

Por su potencia y sencillez, la representación gráfica de los datos es una herramienta fundamental para su análisis. En ocasiones representar gráficamente los datos permite detectar patrones o anomalías que difícilmente pueden ser detectadas mediante análisis estadísticos numéricos.

R proporciona excelentes recursos para la construcción de gráficos. La construcción del gráfico se realiza ejecutando varias sentencias, cada una de las cuales va añadiendo nuevas características al gráfico y definiendo su aspecto. Se muestra un ejemplo a continuación. Supongamos un data frame llamado `datos` cuyo contenido es:

```

> datos
  x    y
1 1.2 2.4
2 2.3 6.4
3 1.4 2.8
4 3.7 10.1
5 1.2 2.0
6 0.6 1.0

```

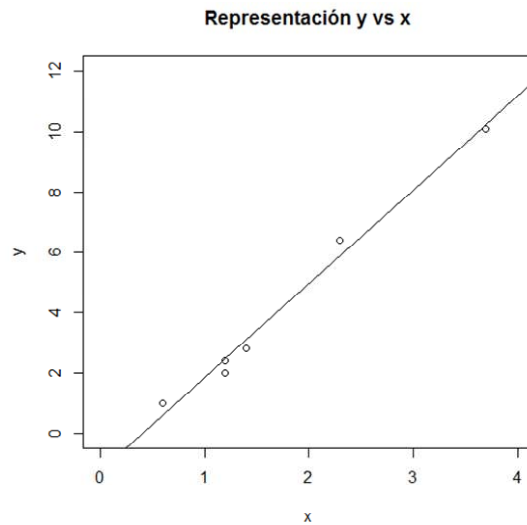


Figura 1.16: Ejemplo de construcción de un gráfico X-Y.

La primera de las siguientes tres sentencias dibuja la variable x frente a y . En la sentencia se especifican las etiquetas de ambos ejes (`xlab`, `ylab`) y el rango de valores de cada eje (`xlim`, `ylim`). La segunda sentencia añade la línea correspondiente al ajuste de los datos y la tercera añade el título.

```
> plot(datos$x, datos$y, xlab="x", ylab="y", xlim=c(0,4), ylim=c(0,12))
> abline( lm(datos$y ~ datos$x) )
> title("Representación y vs x")
```

La gráfica obtenida se muestra en la Figura 1.16. Es posible configurar otras características del gráfico, como son su tamaño, los símbolos usados para representar los valores y su color, el tipo y color de las líneas, el tamaño, color y fuente del texto de los ejes, del título y del subtítulo, las marcas de los ejes, etc. Asimismo, es posible añadir etiquetas al gráfico y superponer varios gráficos.

Otros tipos de gráficos útiles para el análisis de los datos son el histograma y el boxplot. Las funciones `hist()` y `boxplot()` dibujan el histograma y el boxplot de los datos pasados como argumento.

La función `dev.new()` abre una nueva ventana gráfica sobre la cual es posible construir un nuevo gráfico.

La función `par()` permite asignar valor a los parámetros del gráfico. Puede usarse para asignar valor al parámetro `mfrow`, que define el número de filas y columnas de gráficos que van a representarse en la ventana de dibujo. Por ejemplo,

```
> par( mfrow(2,3) )
```

prepara la ventana de dibujo para que aloje 6 gráficos, dispuestos formando una matriz de 2 filas y 3 columnas.

A continuación se muestra un ejemplo. Se crea un objeto de tipo vector llamado `x`, en el cual se guardan 500 observaciones independientes de una distribución exponencial de media 1, y se representa de tres maneras diferentes: gráfico X-Y, histograma y boxplot. Las tres gráficas obtenidas se muestran en la Figura 1.17.

```
> x <- rexp(500)
> par(mfrow=c(1,3))
> plot(x)
> hist(x)
> boxplot(x)
```

La función `hist()` crea un **histograma**. La opción `freq=FALSE` crea el histograma basándose en la densidad de probabilidad. La opción `breaks` controla el número de clases.

El **boxplot** es una herramienta de análisis gráfico muy útil cuando se desea comparar la distribución de diferentes grupos de datos. En la Figura 1.18 se muestra nuevamente el boxplot que aparece en la Figura 1.17, indicando en esta ocasión cómo ha sido construido a partir de los datos.

Con el fin de ilustrar el uso de boxplots en la comparación de grupos de datos, consideremos un data frame en el cual hay una variable numérica que contiene los datos y una variable categórica que describe a qué grupo pertenece cada dato. En concreto, supongamos que el data frame se llama `datos`, que la variable que contiene los datos se llama `x` y que la variable que describe a qué grupo pertenece cada dato se llama `cat`. La siguiente sentencia dibuja un gráfico con un boxplot por cada grupo.

```
> boxplot(x ~ cat, data=datos, varwidth=TRUE,
          main="Ejemplo boxplots en paralelo",
          xlab="grupos", ylab="valor a comparar")
```

La fórmula `x ~ cat` indica que se debe dibujar un boxplot de la variable numérica `x` por cada valor de la variable `cat`. Mediante `data=datos` se indica que ambas variables forman parte del data frame `datos`. Finalmente, `varwidth=TRUE` hace que la anchura del boxplot de cada categoría sea proporcional al número de datos de esa categoría.

Supongamos que el resultado de ejecutar la sentencia anterior es el mostrado en la Figura 1.19. Se observa que el grupo B tiene un comportamiento diferente que los grupos A y C. En general, cuando las cajas de dos grupos no solapan hay una fuerte

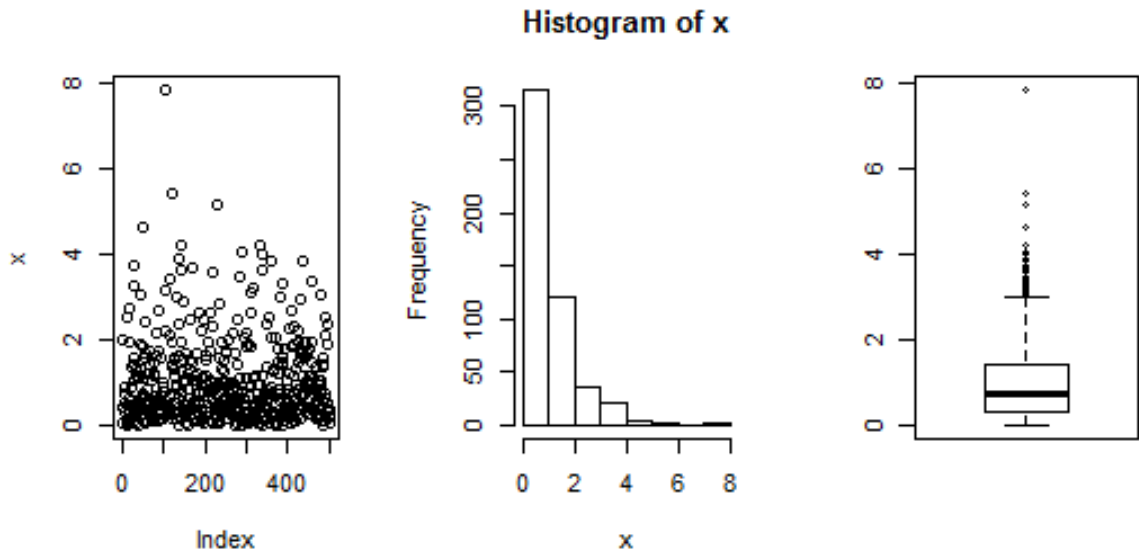


Figura 1.17: Ejemplo de construcción de un gráfico X-Y, un histograma y un boxplot.

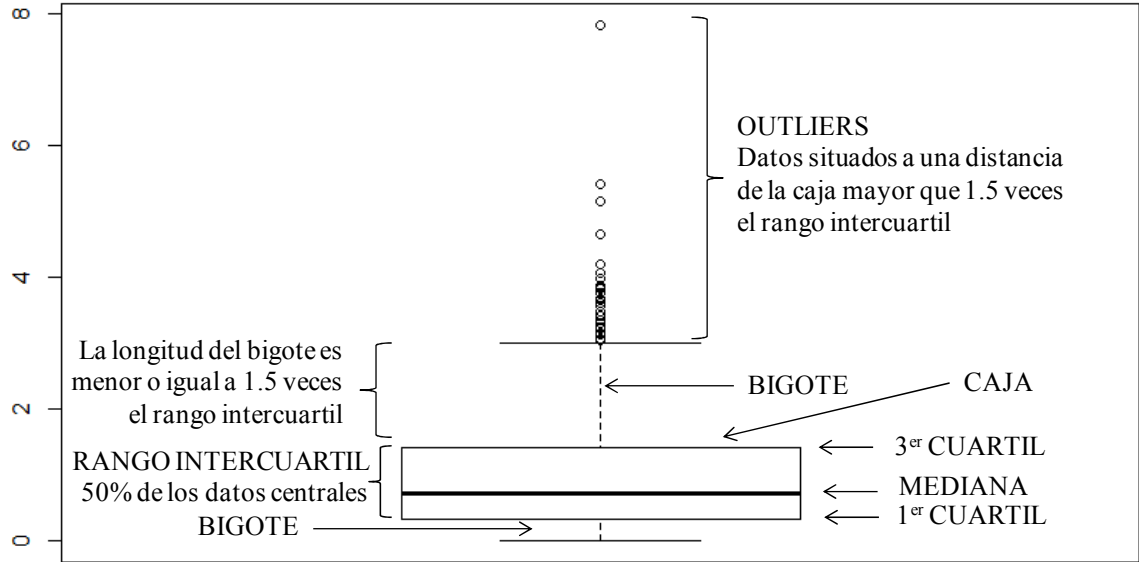


Figura 1.18: Construcción del boxplot de un grupo de datos.

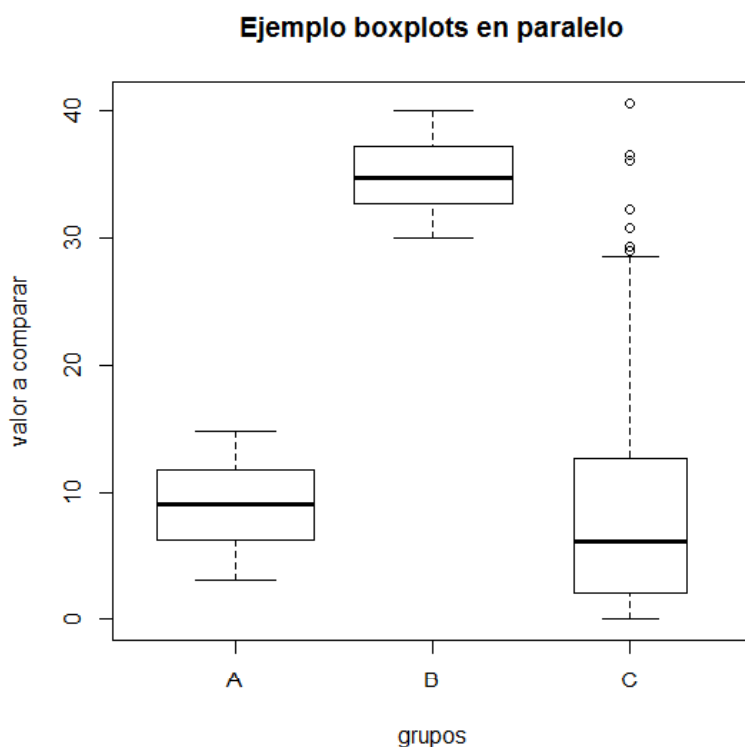


Figura 1.19: Análisis de las diferencias entre grupos mediante boxplots.

evidencia de que sus medianas son diferentes. Por otro lado, también se observa que la dispersión de los datos del grupo C es mayor que la de los datos de los otros dos grupos.

La anchura del boxplot del grupo C es menor (aproximadamente la mitad) que la de los grupos A y B. Esto indica que el número de datos pertenecientes al grupo C es menor: aproximadamente la mitad que el número en cada uno de los otros dos grupos.

1.5.4. Manejo básico de los datos

Sentencias del tipo

```
> variable <- expresion
```

permiten crear nuevas variables y transformar las existentes. La expresión puede contener operadores y funciones. También es posible añadir nuevas variables a un data frame. Por ejemplo, si el data frame `data` tiene dos variables `x` e `y`

```
> data <- data.frame(x=c(1,2),y=c(10,20))
```

```
> data
  x  y
1 1 10
2 2 20
```

mediante la sentencia

```
> data$media <- (data$x+data$y)/2
```

se crea una nueva variable llamada `media` en el data frame. Cada elemento de la nueva variable es la media de los correspondientes elementos de las otras dos variables.

```
> data
  x  y media
1 1 10  5.5
2 2 20 11.0
```

Una sentencia del tipo

```
> variable[condicion] <- expresion
```

sólo realiza la asignación si la condición vale TRUE. Por ejemplo, puede incluirse una nueva variable en el data frame que indique si la media es mayor o menor que 10.

```
> data$criterio10 [data$media < 10] <- "menor"
> data$criterio10 [data$media == 10] <- "igual"
> data$criterio10 [data$media > 10] <- "mayor"
> data
  x  y media criterio10
1 1 10  5.5      menor
2 2 20 11.0      mayor
```

Igualmente,

```
> variable1 <- variable2[condicion]
```

crea el objeto `variable1` y copia en él los elementos de `variable2` que satisfacen la condición. Por ejemplo:

```
> x1 <- data$x[data$criterio10 == "mayor"]
> x1
[1] 2
```

Las expresiones lógicas en R pueden tomar dos valores: TRUE y FALSE. Los operadores de comparación son `<`, `<=`, `>`, `>=`, `==` y `!=`. Las operaciones lógicas se representan: `!x` (not x), `x | y` (x or y), `x & y` (x and y).

La función `order()` permite ordenar los elementos de una estructura de datos. Puede por ejemplo aplicarse a un data frame de la forma siguiente (se ordenan las filas de `exp3` tomando como criterio el valor de la variable `tCola`):

```
> exp3
  num0per horario tCola
1      1      std  12.3
2      1     nuevo   6.1
3      2      std   5.2
4      2     nuevo   2.5
> exp3[order(tCola),]
  num0per horario tCola
4      2     nuevo   2.5
3      2      std   5.2
2      1     nuevo   6.1
1      1      std  12.3
```

Las funciones `merge()`, `cbind()` y `rbind()` permiten realizar la unión de los datos de dos data frames.

1.5.5. Valor NA (Not Available)

El valor `NA` significa Not Available (No Disponible). Se emplea para indicar que no se dispone de ese dato. Supongamos por ejemplo un vector de 5 elementos definido de la forma:

```
> x <- c(1:5)
> x
[1] 1 2 3 4 5
```

Si se asigna valor a un elemento del vector que no existe, R aumenta automáticamente el tamaño del vector rellenando con valores `NA`. Por ejemplo:

```
> x[8] <- 8
> x
[1] 1 2 3 4 5 NA NA 8
```

La función `is.na()` acepta un objeto como argumento y devuelve un objeto del mismo tamaño con las entradas reemplazadas por `TRUE` si el elemento es `NA` y por `FALSE` si no lo es. Por ejemplo:

```
> x
[1] 1 2 3 4 5 NA NA 8
```

```

> is.na(x)
[1] FALSE FALSE FALSE FALSE FALSE  TRUE  TRUE FALSE
> x[ !is.na(x) ]
[1] 1 2 3 4 5 8

```

Si alguno de los operandos es NA, el resultado de la operación aritmética es también NA. Lo mismo sucede cuando se pasa el valor NA como argumento a una función. Esto debe ser tenido en cuenta al operar sobre conjuntos de datos: los valores NA deben ser excluidos del análisis. Con este fin, la mayor parte de las funciones tienen un parámetro llamado `na.rm`, tal que si se le asigna el valor TRUE los valores NA son eliminados de los datos pasados como argumento a la función. Por ejemplo, la función `sum` realiza la suma de los datos contenidos en el objeto que es pasado como argumento (puede consultarse su documentación ejecutando `help(sum)`):

```

> x
[1] 1 2 3 4 5 NA NA 8
> sum(x)
[1] NA
> sum(x, na.rm=TRUE)
[1] 23

```

La función `na.omit()` devuelve un objeto en el cual se han eliminado todos los valores NA. Cuando se pasa como argumento un data frame, el objeto devuelto es un data frame en el cual se han eliminado todas las filas que contienen algún valor NA.

1.5.6. Conversión del tipo de datos

R proporciona funciones para identificar el tipo de dato de un objeto y convertirlo a un tipo diferente. Las funciones cuyo nombre tiene la forma `is.tipodato()` devuelven TRUE o FALSE dependiendo de que el objeto pasado como argumento sea o no del tipo de dato. Por el contrario, las funciones cuyo nombre es de la forma `as.tipodato()` realizan la conversión del argumento al tipo de dato. En la Tabla 1.7 se muestran las funciones disponibles.

Tabla 1.7: Funciones para la comprobación y conversión de tipos de datos.

Comprobación	Conversión
<code>is.numeric()</code>	<code>as.numeric()</code>
<code>is.character()</code>	<code>as.character()</code>
<code>is.vector()</code>	<code>as.vector()</code>
<code>is.matrix()</code>	<code>as.matrix()</code>
<code>is.data.frame()</code>	<code>as.data.frame()</code>
<code>is.factor()</code>	<code>as.factor()</code>
<code>is.logical()</code>	<code>as.logical()</code>

1.5.7. Control del flujo

R tiene sentencias para controlar el flujo de ejecución. Las sentencias *for* y *while* permiten definir bucles, mientras que las sentencias *if-else* y *switch* permiten especificar que una sentencia se ejecute sólo cuando se satisface determinada condición.

La sentencia *for* tiene la siguiente sintaxis:

```
> for (var in secuencia) sentencia
```

Si el cuerpo del bucle consta de varias sentencias, debe escribirse entre llaves (`{ }`). La función `length()` devuelve el número de elementos del objeto que se le pasa como argumento. El siguiente ejemplo ilustra el uso de dicha función y del bucle *for*.

```
> x <- c(1,2,3,5,7,11)
> y <- numeric(0)
> k <- numeric(0)
> for (i in 1:length(x)) {
  y[i] <- i*x[i]
  k[i] <- y[i]/2
}
> y
[1] 1 4 9 20 35 66
> k
[1] 0.5 2.0 4.5 10.0 17.5 33.0
```

La sentencia *while* ejecuta el cuerpo del bucle mientras la condición sea verdadera. Por ejemplo,

```
> i <- 4
> y <- numeric(0)
> while ( i > 0 ) {
  y <- c(y, i*i)
```

```

        i <- i - 1
    }
> y
[1] 16  9  4  1

```

La sentencia *if-else* permite definir que una o varias sentencias sean ejecutadas sólo si se satisface determinada condición. La sintaxis es:

```

> if (condición) sentencia
> if (condición) sentencia1 else sentencia2

```

La sentencia *ifelse* es una forma compacta de *if-else*. La sintaxis es:

```

ifelse(condición, sentencia1, sentencia2)

```

Por ejemplo,

```

> nota <- 7.5
> ifelse( nota < 5, print("suspense"), print("aprobado") )
[1] "aprobado"
[1] "aprobado"
> calificacion <- ifelse ( nota < 5, "suspense", "aprobado" )
> calificacion
[1] "aprobado"

```

1.5.8. Funciones matemáticas y estadísticas

R proporciona las funciones matemáticas más habituales, tales como `abs(x)`, `sqrt(x)`, `ceiling(x)` (entero más pequeño no menor que x), `floor(x)` (mayor entero no mayor que x), `trunc(x)` (entero formado truncando a cero los decimales de x), `round(x,digits=n)` (redondea x al número de dígitos decimales especificado), `signif(x,digits=n)` (redondea x al número de dígitos especificado, incluyendo dígitos enteros más decimales), `sin(x)`, `cos(x)`, `tan(x)`, `asin(x)`, `acos(x)`, `atan(x)`, `sinh(x)`, `cosh(x)`, `tanh(x)`, `asinh()`, `acosh()`, `atanh()`, `log(x,base=n)`, `log(x)` (logaritmo en base e), `log10(x)` (logaritmo en base 10) y `exp(x)`.

También R proporciona las funciones estadísticas más habituales. Se muestran algunas de ellas en la Tabla 1.8.

La función `quantile(x,probs)` acepta como argumentos el vector numérico x del cual se desean calcular los cuantiles y un vector numérico `prob` con probabilidades en $[0, 1]$. Por ejemplo, los percentiles 0.25 y 0.75 de x pueden calcularse de la forma:

```
> y <- quantile(x, c(0.25,0.75))
```

Asimismo, R proporciona funciones de probabilidad que permiten generar datos y calcular valores de la probabilidad. El nombre de las funciones de probabilidad de R sigue el convenio siguiente. El nombre de la función comienza por una de las cuatro letras siguientes:

d	Densidad
p	Función de distribución
q	Función cuantil
d	Generación aleatoria de observaciones

y a continuación se escribe el nombre abreviado correspondiente a la distribución de probabilidad. En la Tabla 1.9 se muestran los nombres de algunas de las distribuciones de probabilidad de R.

Por ejemplo, las sentencias mostradas a continuación dibujan la densidad de probabilidad de la distribución normal, tal como se muestra en la Figura 1.20.

```
> x <- pretty(c(-3,3),60)
> y <- dnorm(x)
> plot(x,y,type="l",xlab="x",ylab="f(x)",yaxs="i",
      main="Densidad de probabilidad N(0,1)")
```

Veamos algunos otros ejemplos. El área que queda a la izquierda de $z = 2$, bajo la densidad de probabilidad $N(0,1)$ (véase nuevamente la Figura 1.20), es:

```
> pnorm(2)
[1] 0.9772499
```

El valor del percentil 90 de la distribución $N(50,10)$ es:

```
> qnorm(0.9, mean=50, sd=20)
[1] 75.63103
```

La sentencia siguiente genera 6 observaciones independientes de la distribución $N(50,10)$:

```
> rnorm(6, mean=50, sd=20)
[1] 42.79969 60.27908 70.79518 31.82367 51.92393 50.23364
```

Cada vez que R genera observaciones de una distribución de probabilidad emplea una semilla diferente, con lo cual los valores generados son diferentes. Si se desea reproducir los mismos resultados, es necesario especificar la semilla mediante la función `set.seed()`. A continuación se muestra un ejemplo.

Tabla 1.8: Algunas funciones estadísticas de R.

Función	Descripción
<code>mean(x)</code>	Media
<code>median(x)</code>	Mediana
<code>sd(x)</code>	Desviación estándar
<code>var(x)</code>	Varianza
<code>quantile(x,probs)</code>	Cuantiles
<code>min(x)</code>	Mínimo
<code>max(x)</code>	Máximo

Tabla 1.9: Algunas distribuciones de probabilidad de R.

Distribución	Nombre	Distribución	Nombre
Beta	<code>beta</code>	Geométrica	<code>geom</code>
Binomial	<code>binom</code>	Lognormal	<code>lnorm</code>
Binomial negativa	<code>nbinom</code>	Normal	<code>norm</code>
Cauchy	<code>cauchy</code>	Poisson	<code>pois</code>
Chi-cuadrado	<code>chisq</code>	T	<code>t</code>
Exponencial	<code>exp</code>	Uniforme	<code>unif</code>
Gamma	<code>gamma</code>	Weibull	<code>weibull</code>

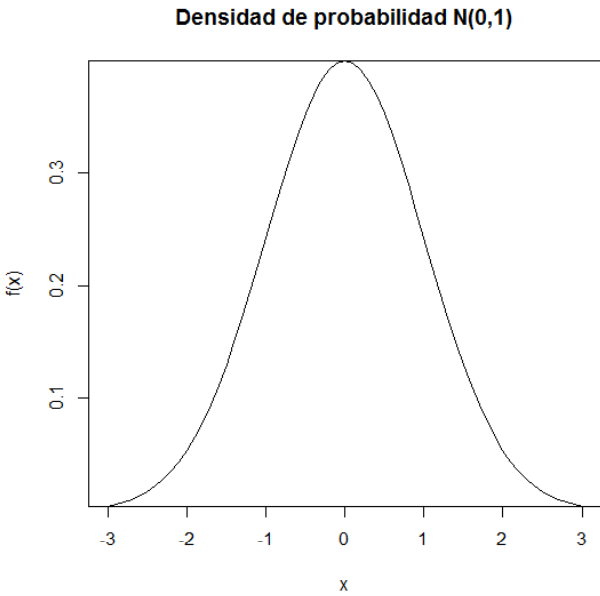


Figura 1.20: Densidad de probabilidad de la distribución normal estándar.

```

> rnorm(6, mean=50, sd=20)
[1] 38.50520 39.06736 38.71096 32.19924 40.45615 30.03227
> rnorm(6, mean=50, sd=20)
[1] 34.47492 51.28918 69.18988 47.79429 39.77981 31.77609
> set.seed(1234)
> rnorm(6, mean=50, sd=20)
[1] 25.858685 55.548585 71.688824 3.086046 58.582494 60.121118
> set.seed(1234)
> rnorm(6, mean=50, sd=20)
[1] 25.858685 55.548585 71.688824 3.086046 58.582494 60.121118

```

1.5.9. Definición de funciones

El usuario puede definir sus propias funciones. La sintaxis para ello es la siguiente:

```

> nombreFunción <- function(arg1, arg2, ...) {
    sentencias
    return(objeto)
}

```

Los objetos creados dentro del cuerpo de la función son locales a la función. El objeto devuelto por la función puede ser cualquier tipo de dato. La función se invoca de la misma forma que las demás funciones definidas en R: especificando su nombre y escribiendo a continuación los argumentos entre paréntesis, separados por coma.

1.5.10. Paquetes

Los paquetes son colecciones de funciones en R, datos y código compilado. Cuando se instala un entorno de desarrollo de R, hay un cierto número de paquetes que son proporcionados por defecto. Para poder usar otros paquetes, es preciso descargarlos (por ejemplo, de <https://cran.r-project.org/web/packages/>), instalarlos y cargarlos en la sesión. El entorno de desarrollo de R típicamente proporciona menús que facilitan estas tareas.

El directorio donde se almacenan los paquetes se denomina la *librería*. La función `.libPaths()` muestra dónde está ubicada la librería. El comando `library()` muestra qué paquetes están almacenados en la librería y también permite cargar un paquete en la sesión actual. Por ejemplo, ejecutando `library("Matrix")` se añade el paquete *Matrix* a la sesión actual. El comando `search()` muestra qué paquetes están cargados en la sesión y, por tanto, están listos para ser usados.

1.6. LECTURAS RECOMENDADAS

Las definiciones de sistema y modelo dadas en la Sección 1.2.1 han sido extraídas de (Cellier 1991). En este excelente libro pueden encontrarse otras definiciones de estos términos, así como una discusión más elaborada acerca de en qué situaciones resulta imposible o inconveniente experimentar con el sistema real, y las ventajas que tiene en estos casos el empleo de modelos.

La clasificación de los modelos en cuatro tipos (mental, verbal, físico y matemático) está explicada con más detalle en (Ljung & Torkel 1994). En la Sección 1.4.1 se presentaron algunas clasificaciones de los modelos matemáticos. En (Cellier 1991, Ljung & Torkel 1994) puede encontrarse una discusión más extensa acerca de las distintas clasificaciones de los modelos matemáticos.

En la Sección 1.2.2 se describe una clasificación en cuatro niveles del conocimiento que puede poseerse acerca de un sistema. Puede encontrarse una descripción más detallada en (Klir 1985, Zeigler et al. 2000). En el texto (Zeigler et al. 2000) se propone una clasificación similar, que está enfocada al modelado y la simulación. La clasificación de Zeigler consta de 5 niveles de conocimiento: (0) marco de observación; (1) comportamiento de entrada/salida; (2) función de entrada/salida; (3) transición de estado; y (4) componentes acoplados.

El marco formal para el modelado y la simulación descrito en la Sección 1.2.3 está extraído de (Zeigler et al. 2000), donde puede encontrarse una explicación más detallada. En ese texto puede encontrarse también una descripción detallada de DEVS, un formalismo de modelado que fue mencionado en la Sección 1.4.3.

En la Sección 1.4.4 se han introducido algunos conceptos básicos acerca de los autómatas celulares. En (Wolfram 1986) se investigan sistemáticamente todas las posibles funciones de transición en autómatas celulares unidimensionales. Puede encontrarse una descripción más detallada del Juego de la Vida en (Gardner 1970). En los textos (Kier et al. 2005, Schiff 2008) se muestran diferentes aplicaciones de los autómatas celulares al modelado de sistemas físicos, químicos y biológicos.

En (Tyszer 1999) se explica en uso de C++ en la programación de simulaciones de eventos discretos. Este texto complementa las explicaciones dadas sobre la programación de modelos basados en planificación de eventos.

Los pasos que típicamente se siguen en un estudio de simulación están descritos con mayor detalle en (Pedgen et al. 1995, Law & Kelton 2000). Se recomienda acudir a esas referencias para ampliar las explicaciones de la Sección 1.3. Asimismo, en (Bratley et al. 1987) y (Hoover & Perry 1989) puede encontrarse abundante

información acerca de lenguajes de simulación como pueden ser Simscript, GPSS, SIMAN y Simula.

Los textos (Railsback & Grimm 2012) y (Wilensky & Rand 2015) son una excelente introducción al modelado basado en agentes usando el entorno de simulación NetLogo. El texto (Borshchev 2013) es una referencia muy completa y didáctica sobre el uso de la herramienta de simulación AnyLogic, la cual facilita la descripción de modelos combinando los tres formalismos siguientes: dinámica de sistemas, modelado de eventos discretos y modelado basado en agentes.

En (Gockenbach 2011) se proporciona una buena clasificación de las ecuaciones diferenciales, multitud de ejemplos de PDE y una buena descripción de los métodos numéricos más comúnmente usados para resolverlas. Se pueden encontrar más ejemplos de aplicación del método de las líneas para la resolución de PDE en el libro (Silebi & Schliesser 1992).

Finalmente, existe gran cantidad de documentación en Internet y excelentes referencias sobre el lenguaje R y también sobre el lenguaje S, en el cual se basa R. Excelentes textos sobre análisis estadístico en S y S-PLUS son (Venables & Ripley 1997) y (Becker et al. 1988). Los dos siguientes textos sobre R son excepcionales por la claridad de sus explicaciones: (Kabacoff 2011) y (Maindonald & Braun 2010). En (Lawrence & Verzani 2012) se describe el empleo de diferentes paquetes (gWidgets, RGtk2, qtbase y tcltk) para programar interfaces de usuario en R.

1.7. EJERCICIOS DE AUTOCOMPROBACIÓN

Ejercicio 1.1

Tomando como base las explicaciones dadas en las Secciones 1.2 – 1.4, defina el significado de los conceptos listados en la tabla siguiente.

sistema	experimento
modelo	modelo mental
modelo verbal	modelo físico
modelo matemático	simulación
análisis	inferencia
diseño	sistema fuente
base de datos del comportamiento	marco experimental
simulador	relación de modelado (validez)
validez replicativa	validez predictiva
validez estructural	relación de simulación (corrección)
complejidad del modelo	simplificación del modelo
refinamiento del modelo	modelado modular
análisis por reducción	modularidad
componibilidad	abstracción
interfaz del modelo	descripción interna del modelo
encapsulado de la información	ocultación de la información
modelado jerárquico	modelo primitivo o atómico
modelo compuesto o molecular	verificación
validación	test de sensibilidad
modelo determinista	modelo estocástico
variable de entrada	modelo dinámico
modelo estático	variable de tiempo discreto
evento	ecuación en diferencias
variable de tiempo continuo	ecuación algebraica
ecuación diferencial ordinaria	ecuación en derivadas parciales
modelo algebraico	modelo dinámico ODE
modelo DAE	modelo híbrido

Ejercicio 1.2

Plantee un ejemplo de conocimiento de un sistema en cada uno de los cuatro niveles de la clasificación de G.J. Klir descrita en la Sección 1.2.2.

Para ese mismo sistema, plantee un ejemplo de análisis del sistema, inferencia sobre el sistema y diseño del sistema. Indique el nivel en el conocimiento del sistema que se precisa, según la clasificación de Klir, para realizar cada una de estas tres actividades.

Ejercicio 1.3

Cuando un modelo de tiempo discreto tiene un número finito de estados y sus entradas pueden tomar un número finito de posibles valores, una forma sencilla de especificar el comportamiento del modelo es mediante su *tabla de transición/salidas*. En ella se escriben todas las posibles combinaciones de valores de los estados y las entradas, y para cada una de éstas se indica el valor de la salida y del siguiente estado. Este tipo de representación se emplea frecuentemente para describir el comportamiento de los circuitos digitales. Se muestra a continuación un ejemplo sencillo.

Consideremos el siguiente modelo de un biestable de tipo D. El modelo tiene dos estados, representados mediante 0 y 1, una entrada y una salida. La entrada puede tomar dos posibles valores, representados mediante 0 y 1. Hay cuatro combinaciones entre los estados y los posibles valores de la entrada. Estas cuatro combinaciones se escriben en las dos primeras columnas de la tabla. El número de combinaciones determina el número de filas de la tabla. Para cada una de estas combinaciones, se escribe en la tercera columna el valor del estado siguiente. En la cuarta columna se escribe el valor de la salida actual del sistema.

Estado actual	Entrada actual	Estado siguiente	Salida actual
0	0	0	0
0	1	1	0
1	0	0	1
1	1	1	1

La primera fila de la tabla indica que cuando el estado actual es 0 y la entrada es 0, entonces la salida del sistema es 0 y el estado en el siguiente instante de tiempo

es 0. La segunda fila indica que cuando el estado actual es 0 y la entrada actual es 1, entonces la salida actual es 0 y el estado siguiente es 1, y así sucesivamente. Obsérvese que en el biestable D la salida es igual al estado actual y el estado siguiente es igual a la entrada actual.

Para un estado inicial del modelo y una trayectoria de entrada que usted escoja arbitrariamente, calcule las trayectorias de estado y de salida.

En lugar de empleando la tabla anterior, describa el comportamiento del sistema mediante las funciones de transición de estado y de salida (véase la Sección 1.4.2).

Ejercicio 1.4

Ejecute manualmente el algoritmo que realiza la simulación del modelo representado por la *tabla de transición/salidas* siguiente:

Estado actual	Entrada actual	Estado siguiente	Salida actual
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

para la *trayectoria de entrada* siguiente:

$$x(0)=0 \quad x(1)=1 \quad x(2)=0 \quad x(3)=1 \quad x(4)=1 \quad x(5)=0 \quad x(6)=0 \quad x(7)=1 \quad x(8)=0 \quad x(9)=1$$

Realice dos réplicas de la simulación. En la primera considere que el estado inicial es $q(0) = 0$ y en la segunda considere que es $q(0) = 1$. A la vista de los resultados, explique por qué este sistema se denomina “contador binario”. Finalmente, escriba el código en lenguaje R que permite simular el comportamiento del modelo para esa trayectoria de entrada y la condición inicial $q(0) = 0$.

Ejercicio 1.5

Consideremos nuevamente el modelo de la oficina de atención al público, en la cual trabaja un único empleado, cuyo funcionamiento fue descrito en la Sección 1.4.3. Realice las tareas siguientes:

1. Describa la estructura lógica del modelo, indicando para ello las acciones asociadas a los dos eventos siguientes: llegada de un nuevo cliente y finalización de la atención del empleado a un cliente.
2. Ponga un ejemplo del funcionamiento del sistema, dibujando la evolución del estado y de la salida correspondiente a un proceso de llegada de clientes arbitrario que usted escoja como ejemplo.
3. El objetivo de la simulación es estimar el tiempo medio de espera de los clientes en la cola y el número medio de clientes en la cola. Indique cómo calcularía cada uno de estos estadísticos.
4. Describa el modelo siguiendo la metodología de la planificación de eventos. Para ello, escriba el diagrama de flujo de las rutinas de inicialización y de eventos del modelo, en las cuales debe realizarse el cálculo de los dos estadísticos anteriores.

Ejercicio 1.6

Como se explicó en la Sección 1.4.1, los modelos estáticos son aquellos en los cuales el tiempo no juega ningún papel. Los modelos estáticos pueden ser deterministas y estocásticos. Este segundo tipo se emplea en la realización de las *simulaciones de Monte Carlo*. Reciben este nombre las simulaciones que emplean números aleatorios para resolver problemas estáticos. A continuación de muestra un ejemplo.

El método de Monte Carlo permite estimar el valor de la integral definida de una función real monovaluada $g(x)$ entre dos valores reales finitos a y b :

$$I = \int_a^b g(x) \cdot dx \quad (1.36)$$

Para ello, se define una variable aleatoria Y de la forma siguiente:

$$Y = (b - a) \cdot g(X) \quad (1.37)$$

donde la variable aleatoria X está distribuida uniformemente en el intervalo $[a, b]$.

Puede demostrarse que el valor esperado de la variable aleatoria Y , que representaremos μ_Y , es igual al valor de la integral definida I que se desea calcular.

La media $\bar{Y}(n)$ de n observaciones independientes de Y es una estimación de μ_Y y por tanto de I . $\bar{Y}(n)$ puede calcularse de la Ecuación (1.38), donde $\{x_1, \dots, x_n\}$ son n observaciones independientes de la variable aleatoria X .

$$\bar{Y}(n) = \frac{b-a}{n} \cdot \sum_{i=1}^n g(x_i) \quad (1.38)$$

Realice un programa en lenguaje R que aplique la técnica anteriormente descrita para estimar el valor de la integral definida siguiente:

$$I = \int_0^{\pi} \sin(x) \cdot dx \quad (1.39)$$

Esta integral posee primitiva analítica: $-\cos(x)$. Por ello es posible comparar el resultado obtenido aplicando la técnica de Monte Carlo con el resultado analítico exacto, que es $I = 2$.

Ejecute 5 veces el programa para cada uno de los tres tamaños de muestra siguientes: $n = 50$, 500 y 5000 . Con el fin de comparar la distribución de las estimaciones obtenidas para cada tamaño de muestra, dibuje un gráfico con un boxplot para cada tamaño de muestra, en el cual se representen las correspondientes cinco estimaciones.

Ejercicio 1.7

Escriba en lenguaje R un simulador para el autómata celular unidimensional con Regla 90 mostrado en la Figura 1.8. Realice el programa de manera que pueda configurarse de manera sencilla el número de células, el estado inicial y la función de transición de estados. Ejecútelo para el mismo número de células (61) y el mismo estado inicial mostrados en la Figura 1.8.

Ejercicio 1.8

En la Sección 1.4.4 se describió el autómata bidimensional denominado el *Juego de la Vida de Conway*. Escriba un programa en lenguaje R que ejecute el Juego de la Vida para un autómata 10×10 , imponiendo que las células del borde del espacio permanezcan muertas (en el estado 0) durante toda la simulación.

Ejecute el programa tomando como configuración inicial los patrones mostrados en la Figura 1.10 y compruebe que el programa reproduce los comportamientos mostrados en la figura.

Ejercicio 1.9

Programa en R el algoritmo de la simulación de un modelo que describe el movimiento vertical de un objeto de masa m , que está sometido a una fuerza F cuyo valor es la suma de dos términos: la fuerza gravitatoria (primer sumando) y una fuerza armónica (segundo sumando).

$$F = m \cdot g + F_0 \cdot \sin(\omega \cdot t) \quad (1.40)$$

El significado de las variables que intervienen en la ecuación anterior es el siguiente:

F	Fuerza total aplicada sobre el objeto
m	Masa del objeto
g	Aceleración gravitatoria
F_0	Amplitud de la fuerza armónica
ω	Frecuencia de la fuerza armónica
t	Tiempo

La fuerza F aplicada sobre el objeto hace que éste adquiera una aceleración a , que viene determinada por la ecuación siguiente:

$$a = \frac{F}{m} \quad (1.41)$$

Las variables m , g , F_0 y ω son parámetros del modelo. Es decir, su valor es constante y conocido. La fuerza F y la aceleración a son variables cuyo valor depende del tiempo.

La Ec. (1.40) puede emplearse para calcular el valor de la fuerza en un determinado instante de tiempo t . Conocido el valor de la fuerza en dicho instante, puede emplearse la Ec. (1.41) para calcular la aceleración del objeto en ese mismo instante.

Las Ecs. (1.42) y (1.43) describen la relación entre la aceleración (a) del objeto, su velocidad (v) y su posición (x). La derivada respecto al tiempo de la velocidad es la aceleración. La derivada respecto al tiempo de la posición es la velocidad.

$$\frac{dv}{dt} = a \quad (1.42)$$

$$\frac{dx}{dt} = v \quad (1.43)$$

En este modelo, la posición (x) y velocidad (v) del objeto son variables de estado. Se calculan integrando numéricamente sus derivadas, esto es, $\frac{dx}{dt}$ y $\frac{dv}{dt}$ respectivamente. Emplearemos para ello el método de integración de Euler explícito.

El algoritmo de la simulación del modelo se muestra en la Figura 1.21. Puede verse el valor que debe asignarse a cada parámetro y los valores iniciales de las variables de estado. Obsérvese que $\frac{dx}{dt}$ es la variable $derx$ del algoritmo y $\frac{dv}{dt}$ es la variable $deriv$. La condición de finalización de la simulación es que el objeto toque el suelo. Es decir, que se satisfaga la condición $x < 0$.

Ejecute el algoritmo de la simulación y dibuje gráficas mostrando la evolución de la posición, velocidad y aceleración del objeto.

Con el fin de clarificar la aplicación del algoritmo mostrado en la Figura 1.21, a continuación se muestran los primeros pasos de su ejecución.

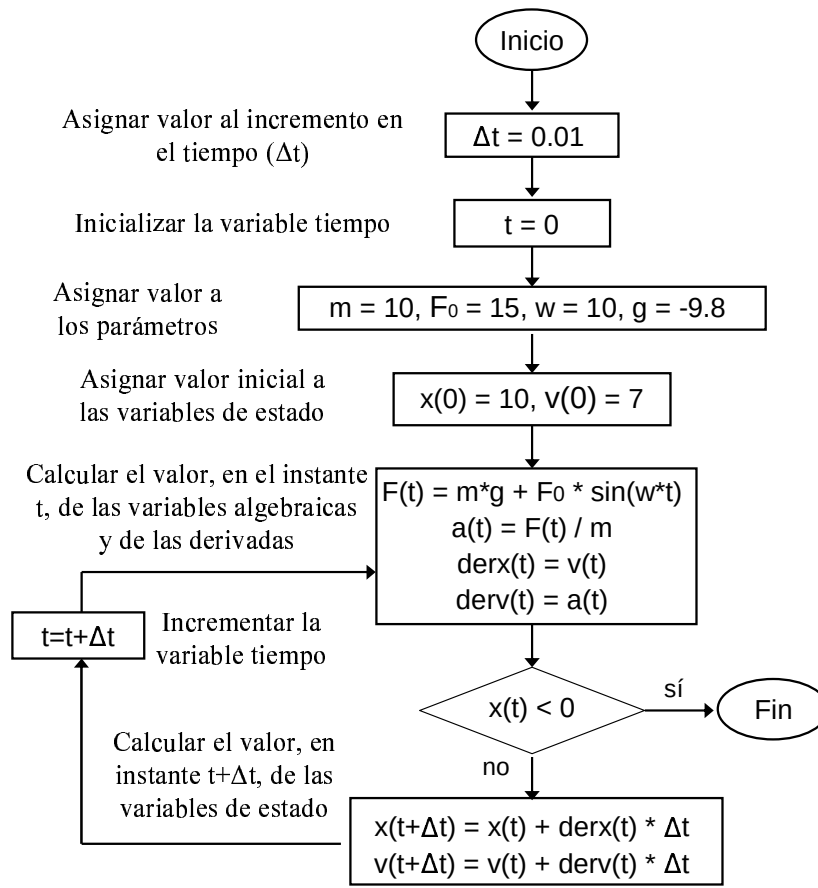


Figura 1.21: Algoritmo de la simulación del modelo que describe la caída de un objeto.

$t = 0$ $m = 10, F_0 = 15, \omega = 10, g = -9.8$
 $x(0) = 10, v(0) = 7$
 $F(0) = m \cdot g + F_0 \cdot \sin(\omega \cdot t) \rightarrow F(0) = 10 \cdot (-9.8) + 15 \cdot \sin(10 \cdot 0) = -98$
 $a(0) = F(0)/m \rightarrow a(0) = -98/10 = -9.8$
 $derx(0) = v(0) \rightarrow derx(0) = 7$
 $deriv(0) = a(0) \rightarrow deriv(0) = -9.8$
 $x(0.01) = x(0) + derx(0) \cdot \Delta t \rightarrow x(0.01) = 10 + 7 \cdot 0.01 = 10.07$
 $v(0.01) = v(0) + deriv(0) \cdot \Delta t \rightarrow v(0.01) = 7 + -9.8 \cdot 0.01 = 6.902$
 $t = 0.01$ $F(0.01) = m \cdot g + F_0 \cdot \sin(\omega \cdot t) \rightarrow F(0.01) = 10 \cdot (-9.8) + 15 \cdot \sin(10 \cdot 0.01) = -96.5025$
 $a(0.01) = F(0.01)/m \rightarrow a(0.01) = -96.5025/10 = -9.65025$
 $derx(0.01) = v(0.01) \rightarrow derx(0.01) = 6.902$
 $deriv(0.01) = a(0.01) \rightarrow deriv(0.01) = -9.65025$
 $x(0.02) = x(0.01) + derx(0.01) \cdot \Delta t \rightarrow x(0.02) = 10.07 + 6.902 \cdot 0.01 = 10.139$
 $v(0.02) = v(0.01) + deriv(0.01) \cdot \Delta t \rightarrow v(0.02) = 6.902 + -9.65025 \cdot 0.01 = 6.8055$
 $t = 0.02$ Y así sucesivamente ...

Ejercicio 1.10

En la Figura 1.22 se muestra un objeto de masa M que puede deslizar con rozamiento sobre el suelo y que se encuentra unido a una pared inmóvil mediante un muelle.

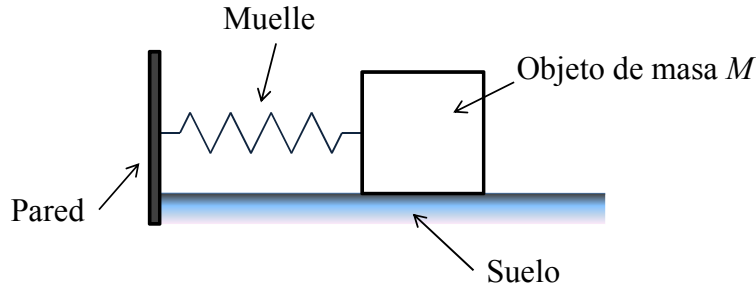


Figura 1.22: Sistema mecánico.

La evolución en el tiempo de la velocidad v y la posición x del objeto viene descrita por las ecuaciones siguientes:

$$M \cdot \frac{dv}{dt} = -K \cdot (x - L_0) - b \cdot v \quad (1.44)$$

$$\frac{dx}{dt} = v \quad (1.45)$$

donde K es la constante de proporcionalidad del muelle, L_0 es su elongación natural, y b es la constante de rozamiento entre el objeto y el suelo. Se supone que el valor de cada una de estas magnitudes, así como la masa del objeto, permanecen constantes en el tiempo.

Aplicando el método de integración de Euler explícito a las dos ecuaciones anteriores se obtiene:

$$M \cdot \frac{v_{i+1} - v_i}{h} = -K \cdot (x_i - L_0) - b \cdot v_i \quad (1.46)$$

$$\frac{x_{i+1} - x_i}{h} = v_i \quad (1.47)$$

donde x_i , v_i representan la posición y la velocidad del objeto en el instante de tiempo $t = i \cdot h$, y donde h es el tamaño del paso de integración.

Manipulando las expresiones anteriores puede obtenerse la posición x_{i+1} y la velocidad v_{i+1} en el instante $(i+1) \cdot h$, a partir de la posición x_i y la velocidad v_i en el instante $i \cdot h$, conocidas las magnitudes constantes K , L_0 , M y b , y el valor del paso de integración h .

$$v_{i+1} = v_i - \frac{K \cdot h}{M} \cdot (x_i - L_0) - \frac{b \cdot h}{M} \cdot v_i \quad (1.48)$$

$$x_{i+1} = x_i + h \cdot v_i \quad (1.49)$$

para $i = 0, \dots, N-1$.

Realice un programa en lenguaje R que simule el modelo anterior durante $N = 10000$ pasos de integración de tamaño $h = 0.01$ s. Ejecute la simulación asignando valores arbitrarios positivos a los parámetros K , M y b . Suponga que la elongación natural del muelle es $L_0 = 1$ m y que inicialmente la masa se encuentra en reposo en la posición $x_0 = 1.2$ m. El programa debe representar gráficamente la evolución de la posición de la masa y su velocidad.

Ejercicio 1.11

Indique para cada una de las siguientes ecuaciones si es ODE o PDE, su orden, si es lineal o no lineal y, en caso de ser lineal, si es homogénea o no homogénea (inhomogénea).

$$\frac{dx}{dt} + t \cdot x = 0 \quad (1.50)$$

$$\frac{\partial u}{\partial t} - \frac{\partial^2 u}{\partial x^2} = (1+t) \cdot \sin(x) \quad (1.51)$$

$$\frac{\partial w}{\partial t} + w \cdot \frac{\partial w}{\partial x} = 0 \quad (1.52)$$

$$\frac{\partial v}{\partial t} + 3 \cdot \frac{\partial v}{\partial x} = x + 3 \cdot t \quad (1.53)$$

$$\frac{d^2 \theta}{dt^2} + \sin(\theta) = 0 \quad (1.54)$$

$$\rho(x, y) \cdot \frac{\partial^2 u}{\partial t^2} - T \cdot \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = f(x, t) \quad (1.55)$$

$$\frac{d^2 x}{dt^2} - 2 \cdot \frac{dx}{dt} + t \cdot x = 0 \quad (1.56)$$

$$\frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} + u \cdot \frac{\partial u}{\partial x} - \frac{\partial^3 u}{\partial x^2 \partial t} = 0 \quad (1.57)$$

1.8. SOLUCIONES DE LOS EJERCICIOS

Solución al Ejercicio 1.1

En la tabla siguiente se indica la sección y la página del texto en la cual puede encontrarse la definición de cada uno de los conceptos indicados en el enunciado del ejercicio.

Concepto	Sección / Página
sistema	§1.2.1/34
experimento	§1.2.1/34
modelo	§1.2.1/34, §1.2.3/39
modelo mental	§1.2.1/35,
modelo verbal	§1.2.1/36
modelo físico	§1.2.1/36
modelo matemático	§1.2.1/36
simulación	§1.2.1/36
análisis	§1.2.2/37
inferencia	§1.2.2/37
diseño	§1.2.2/37
sistema fuente	§1.2.3/38
base de datos comportamiento	§1.2.3/38
marco experimental	§1.2.3/38
simulador	§1.2.3/40
relación de modelado (validez)	§1.2.3/40
validez replicativa	§1.2.3/40
validez predictiva	§1.2.3/40
validez estructural	§1.2.3/40
relación de simulación (corrección)	§1.2.3/40
complejidad del modelo	§1.2.3/40
simplificación del modelo	§1.2.3/40
refinamiento del modelo	§1.3.2/43
modelado modular	§1.3.3/44
análisis por reducción	§1.3.3/44
modularidad	§1.3.3/44

Concepto	Sección / Página
componibilidad	§1.3.3/44
abstracción	§1.3.3/45
interfaz del modelo	§1.3.3/45
descripción interna del modelo	§1.3.3/45
encapsulado de la información	§1.3.3/45
ocultación de la información	§1.3.3/45
modelado jerárquico	§1.3.3/45
modelo primitivo o atómico	§1.3.3/45
modelo compuesto o molecular	§1.3.3/45
verificación	§1.3.5/46
validación	§1.3.5/46
test de sensibilidad	§1.3.5/47
modelo determinista	§1.4.1/48
modelo estocástico	§1.4.1/48
variable de entrada	§1.4.1/48
modelo dinámico	§1.4.1/48
modelo estático	§1.4.1/48
variable de tiempo discreto	§1.4.1/49
evento	§1.4.1/49
ecuación en diferencias	§1.4.1/49
variable de tiempo continuo	§1.4.1/49
ecuación algebraica	§1.4.1/50
ecuación diferencial ordinaria	§1.4.1/50
ecuación en derivadas parciales	§1.4.1/50
modelo algebraico	§1.4.1/50
modelo dinámico ODE	§1.4.1/50
modelo DAE	§1.4.1/51
modelo híbrido	§1.4.1/51

Solución al Ejercicio 1.2

Consideremos, por ejemplo, que el sistema es un *colector solar*. Se trata de un dispositivo que forma parte de los calentadores solares y que aprovecha la energía de la radiación solar para calentar un fluido (por ejemplo, una mezcla de agua y glicol) que circula por unos conductos situados dentro del panel. El fluido caliente circula desde el panel hasta su punto de uso (por ejemplo, los radiadores del sistema de calefacción de una vivienda), retornando luego al panel.

- En el Nivel 0 del conocimiento se identifica la porción del mundo que vamos a modelar y las maneras mediante las cuáles vamos a observarlo. Esta porción del sistema real, que en nuestro caso será el panel solar térmico, constituye el sistema fuente. La forma en que vamos a observar el colector será midiendo la radiación solar incidente, la temperatura de entrada del fluido y su temperatura de salida.
- En el Nivel 1 disponemos de una base de datos de medidas del sistema. En este caso, de las temperaturas de entrada y salida del fluido que se han medido para un cierto valor medido de la radiación incidente.
- En el Nivel 2 hemos sido capaces de correlacionar el incremento en la temperatura del fluido con la intensidad de la radiación incidente. Esta correlación la hemos obtenido ajustando los parámetros de una fórmula a los datos experimentales.
- Finalmente, en el Nivel 3 conocemos la estructura del sistema y somos capaces de entender su comportamiento a partir del comportamiento y la interacción de sus componentes. Sabemos que el panel solar térmico está compuesto por una caja rectangular, dentro del cual está el sistema captador de calor. Una de las caras de esta caja está cubierta de un vidrio muy fino y las otras cinco caras son opacas y están aisladas térmicamente. Dentro de la caja, expuesta al sol, se sitúa una placa metálica, que está tratada para que aumente su absorción del calor, y a la cual están soldados los conductos por los que circula el fluido transportador del calor.

Planteemos ahora un ejemplo de análisis, inferencia y diseño de nuestro sistema, consistente en un colector solar.

El *análisis* del sistema consistiría en intentar comprender su funcionamiento basándose para ello en el conocimiento que se tiene de su estructura. A partir del

análisis de su estructura, podemos comprender que el colector solar funciona aprovechando el efecto invernadero. El vidrio deja pasar la luz visible del sol, que incide en la placa metálica calentándola. Sin embargo, el vidrio no deja salir la radiación infrarroja de baja energía que emite la placa metálica caliente. A consecuencia de ello, y a pesar de las pérdidas por transmisión (el vidrio es mal aislante térmico), el volumen interior de la caja se calienta por encima de la temperatura exterior.

En la *inferencia*, disponemos de una base de datos del comportamiento del sistema fuente y tratamos de encontrar una representación del conocimiento al Nivel 2 (generación) o al Nivel 3 (estructura), que nos permita recrear los datos de que disponemos. Por ejemplo, hemos observado experimentalmente que el rendimiento de los colectores mejora cuanto menor sea la temperatura de trabajo. El modelo que describe la estructura del sistema debería reproducir este comportamiento: a mayor temperatura dentro de la caja (en relación con la exterior), mayores serán las pérdidas por transmisión en el vidrio. También, a mayor temperatura de la placa captadora, más energética será su radiación, y más transparencia tendrá el vidrio a ella, disminuyendo por tanto la eficiencia del colector.

En el *diseño* de un sistema se investigan diferentes estructuras alternativas para un sistema completamente nuevo o para el rediseño de uno ya existente. Como parte del diseño del colector solar podrían investigarse el efecto sobre la absorción de calor de diferentes longitudes de los conductos, diferente volumen interno de la caja, diferentes pinturas aplicadas sobre la capa metálica, diferentes condiciones de operación (por ejemplo, caudal de fluido), etc.

Solución al Ejercicio 1.3

Conocido el estado inicial del modelo, la tabla de transición/salidas permite calcular las trayectorias de estado y de salida correspondientes a una trayectoria de entrada. A continuación se muestra un ejemplo del funcionamiento del biestable D. Se comprueba que la salida sigue a la entrada, pero retrasada un paso en el tiempo.

Tiempo	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9
Trayectoria de entrada	1	0	1	0	1	0	1	0	1	0
Trayectoria de estado	0	1	0	1	0	1	0	1	0	1
Trayectoria de salida	0	1	0	1	0	1	0	1	0	1

La tabla de transición/salidas es útil para representar el comportamiento de modelos sencillos, con un número pequeño de estados y unas entradas con un número pequeño de posibles valores.

La información representada mediante una tabla de transición/salidas puede ser descrita de forma más compacta empleando las funciones de transición de estado y de salida. Por ejemplo, el comportamiento del biestable D (la salida es igual al estado actual y el estado siguiente es igual a la entrada actual) puede representarse de la forma siguiente:

$$\delta(q, x) = x \quad (1.58)$$

$$\lambda(q, x) = q \quad (1.59)$$

Solución al Ejercicio 1.4

Para la trayectoria de entrada dada en el enunciado del ejercicio y con un estado inicial $q(0) = 0$, se obtiene las trayectorias de los estados y de salida siguientes:

q(0)=0	q(1)=0	q(2)=1	q(3)=1	q(4)=0	q(5)=1	q(6)=1	q(7)=1	q(8)=0	q(9)=0
x(0)=0	x(1)=1	x(2)=0	x(3)=1	x(4)=1	x(5)=0	x(6)=0	x(7)=1	x(8)=0	x(9)=1
q(1)=0	q(2)=1	q(3)=1	q(4)=0	q(5)=1	q(6)=1	q(7)=1	q(8)=0	q(9)=0	q(10)=1
y(0)=0	y(1)=0	y(2)=0	y(3)=1	y(4)=0	y(5)=0	y(6)=0	y(7)=1	y(8)=0	y(9)=0

Para la misma trayectoria de entrada, pero con estado inicial $q(0) = 1$, se obtiene:

q(0)=1	q(1)=1	q(2)=0	q(3)=0	q(4)=1	q(5)=0	q(6)=0	q(7)=0	q(8)=1	q(9)=1
x(0)=0	x(1)=1	x(2)=0	x(3)=1	x(4)=1	x(5)=0	x(6)=0	x(7)=1	x(8)=0	x(9)=1
q(1)=1	q(2)=0	q(3)=0	q(4)=1	q(5)=0	q(6)=0	q(7)=0	q(8)=1	q(9)=1	q(10)=0
y(0)=0	y(1)=1	y(2)=0	y(3)=0	y(4)=1	y(5)=0	y(6)=0	y(7)=0	y(8)=0	y(9)=1

El estado del sistema almacena el bit menos significativo de la suma binaria de los valores de entrada. La salida es el acarreo.

Existen varias maneras de escribir un programa en lenguaje R que simule el comportamiento del sistema para la trayectoria de entrada dada. Se muestra a continuación un posible programa. Lo más cómodo es escribir el programa en un fichero de texto y cargarlo desde la consola de R empleando la función `source()`.

```

x <- c(0,1,0,1,1,0,0,1,0,1) # Trayectoria de entrada
t <- c(0:(length(x)-1))      # Tiempo
q <- 0                        # Estado inicial
y <- numeric(0)              # Salida
# Bucle de la simulación
for (i in 1:length(x)) {
  q[i+1] <- if ( q[i]==0 & x[i]==0 ) 0
             else if ( q[i]==0 & x[i]==1 ) 1
             else if ( q[i]==1 & x[i]==0 ) 1
             else 0
  y[i] <- if ( q[i]==0 & x[i]==0 ) 0
           else if ( q[i]==0 & x[i]==1 ) 0
           else if ( q[i]==1 & x[i]==0 ) 0
           else 1
}
# Salida de resultados
q <- q[1:length(x)]
resultados <- data.frame(t,x,q,y)

```

Una vez ejecutado el código anterior, el resultado está almacenado en el data frame `resultados`. Su contenido es el siguiente:

```

> resultados
  t x q y
1 0 0 0 0
2 1 1 0 0
3 2 0 1 0
4 3 1 1 1
5 4 1 0 0
6 5 0 1 0
7 6 0 1 0
8 7 1 1 1
9 8 0 0 0
10 9 1 0 0

```

Solución al Ejercicio 1.5

La estructura lógica del modelo es la siguiente (véase la Figura 1.23):

- Si llega un nuevo cliente y el empleado está ocupado, el cliente se pone al final de la cola con disciplina FIFO. Si el empleado está libre, el cliente es atendido inmediatamente.
- Si el empleado termina de atender a un cliente, éste se marcha y comienza a ser atendido el primer cliente de la cola. Si la cola está vacía, el empleado permanece libre hasta la llegada de un nuevo cliente.

En la parte superior de la Figura 1.24 se muestra un esquema de la estructura del modelo, que está compuesto por una cola y un proceso con un recurso: el empleado. Se muestran los eventos de entrada y de salida al sistema. Los eventos de entrada, ordenados sobre el eje temporal, se denominan *trayectoria de entrada*, mientras que los eventos de salida ordenados sobre el eje temporal se denominan *trayectoria de salida*. Cada evento de entrada (llegada de un cliente a la oficina) y de salida (un cliente que abandona la oficina después de ser atendido) se representa mediante una flecha vertical situada sobre el eje temporal, en el instante en que se produce el evento.

Los procesos de llegada y de atención al cliente se modelan de la forma siguiente. Supongamos que los clientes llegan de uno en uno y que el intervalo de tiempo que transcurre entre llegadas sucesivas está distribuido exponencialmente, con media 10 minutos. Supongamos también que el tiempo que tarda el empleado en atender a un cliente es una variable aleatoria distribuida uniformemente en el intervalo $[5, 10]$ minutos.

Para describir el estado del sistema, hacen falta al menos dos variables de estado: la fase del empleado y el número de clientes que están en cola. Los valores posibles de estas variables son los siguientes. La fase del empleado puede tomar dos posibles valores: {libre, ocupado}. El número de clientes que esperan en la cola puede tomar valores enteros positivos, incluido el cero. En la Figura 1.24 se muestra un ejemplo de cómo evoluciona el estado del modelo a medida que los clientes van llegando y abandonando la oficina.

El objetivo de la simulación es estimar el tiempo medio de espera en cola y el número medio de clientes que esperan en la cola. Los cálculos a realizar son los siguientes.

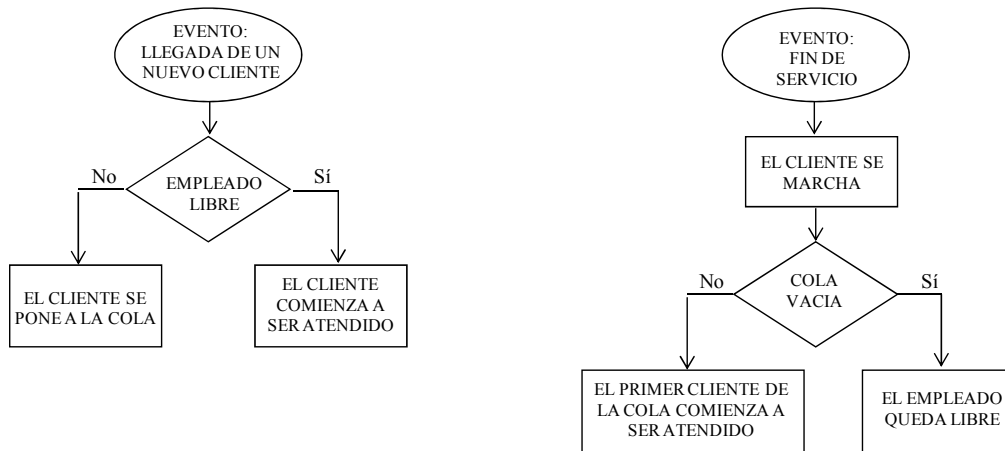


Figura 1.23: Esquema de la estructura lógica del modelo.

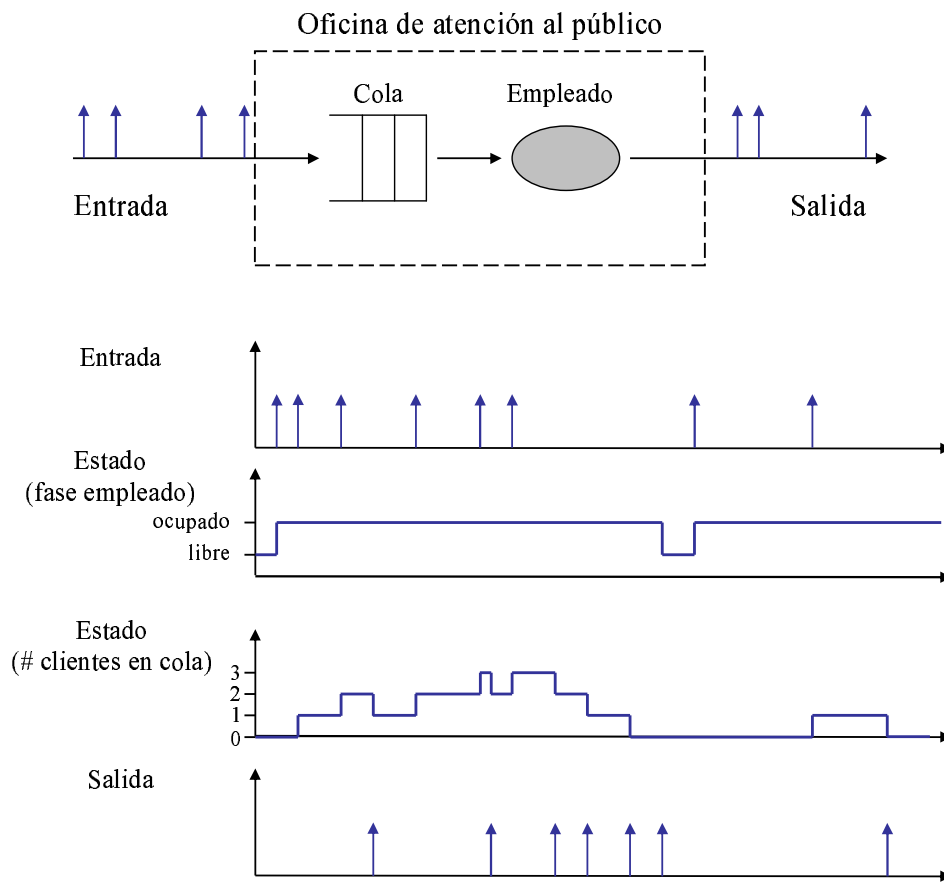


Figura 1.24: Ejemplo del comportamiento del modelo de la oficina.

- El *tiempo medio de espera de los clientes en la cola*, se define de la forma siguiente:

$$\hat{d}(n) = \frac{1}{n} \sum_{i=1}^n D_i \quad (1.60)$$

donde n es el número total del clientes que han abandonado la cola y D_i es el tiempo de espera en la cola del cliente i . Para calcularlo es preciso llevar registro a lo largo de la simulación tanto del número de clientes que han abandonado la cola hasta ese momento, como de la suma de sus tiempos de espera.

- El *número medio de clientes en la cola* se define de la forma siguiente:

$$\hat{q}(T) = \frac{1}{T} \int_0^T Q(\tau) \cdot d\tau \quad (1.61)$$

donde T es el tiempo que ha durado la simulación y $Q(\tau)$ es el número de clientes que había en la cola en el instante τ .

Obsérvese que si se representa el número de clientes en cola Q en función del tiempo, la integral es igual al área bajo la curva entre los instantes inicial y final de la simulación. Dicho área debe irse calculando a lo largo de la simulación, para lo cual se emplea el acumulador estadístico R . Inicialmente R vale cero. Se actualiza el valor de R cada vez que cambia el tamaño de la cola y en el instante final la simulación. Para ello, se suma al valor actual de R el producto del número de clientes en cola antes del cambio por el tiempo transcurrido desde el anterior cambio. Es preciso definir un segundo acumulador estadístico, t_{evento} , que almacene el instante en que se produjo el cambio en el tamaño de la cola anterior al actual.

La descripción del modelo siguiendo la metodología de la planificación de eventos se realiza describiendo la condición de disparo y las acciones asociadas a cada tipo de evento. En este caso hay cuatro tipos de eventos: inicio de la simulación, llegada a la oficina de un nuevo cliente, fin de servicio a un cliente y final de la simulación. Los flujos de acciones asociados a los tres primeros tipos se muestran en la Figura 1.25.

Obsérvese que la fase del empleado se representa mediante la variable de estado E , que puede tomar los valores 0 (libre) y 1 (ocupado). El modelo tiene otras dos variables de estado además de E . Estas son: Q (número de clientes en cola) y t_{evento} (instante en que se produjo el último evento).

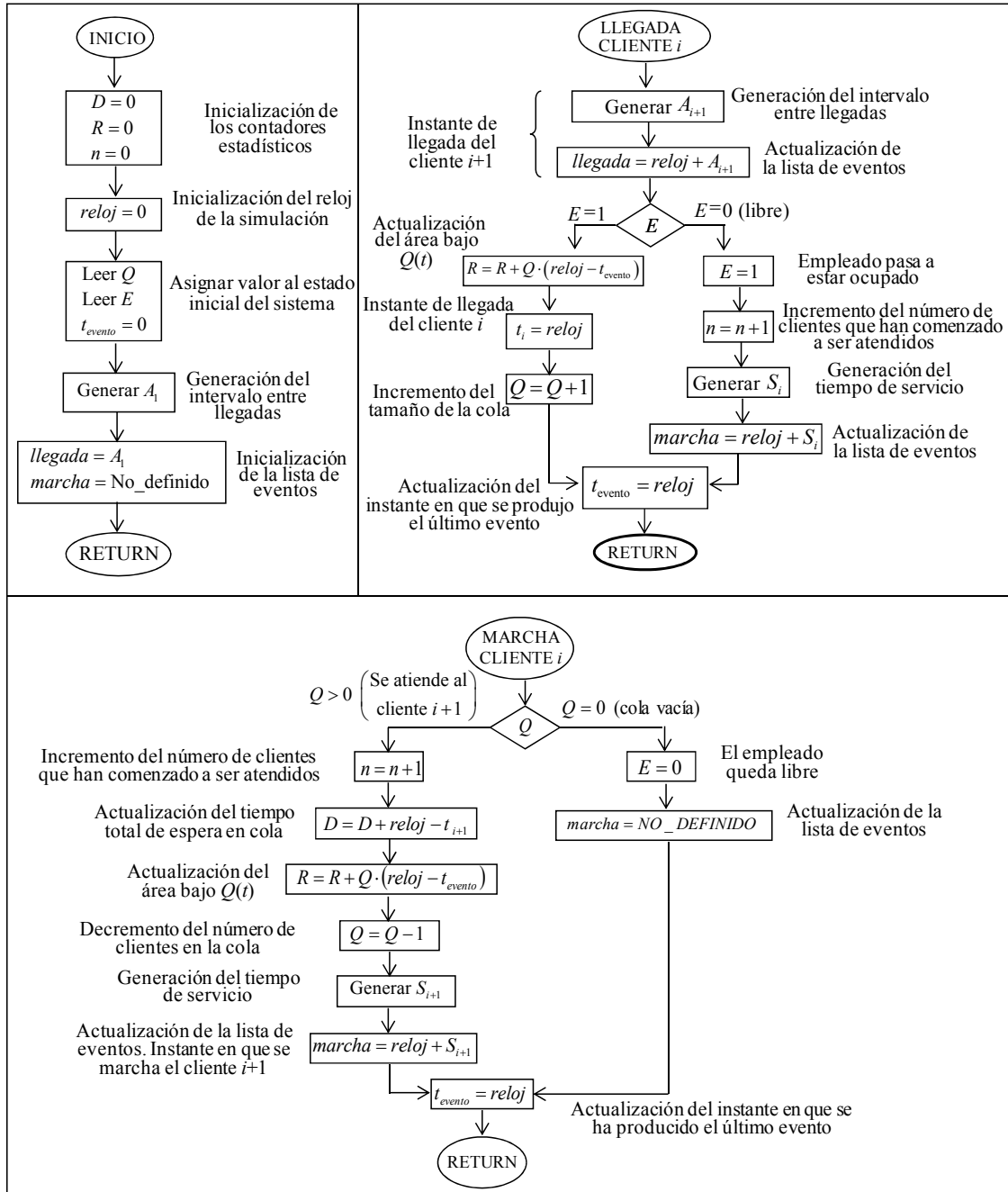


Figura 1.25: Flujos de acciones asociadas a los eventos.

El calendario de eventos guarda dos valores: *llegada* (instante en que está planificada la llegada del próximo cliente) y *marcha* (instante en que está planificado que el cliente que está siendo atendido abandone el sistema).

El modelo tiene tres contadores estadísticos: D (suma de los tiempos de espera en cola de todos los clientes que la han abandonado), n (número de clientes que han comenzado a ser atendidos) y R (área bajo Q).

Solución al Ejercicio 1.6

A continuación se muestra una forma de realizar el programa.

```
# Función para la estimación de la integral
estimaI <- function(n,a,b) {
  x <- runif(n,a,b)
  I <- (b-a)*sum( sin(x) )/n
  return (I)
}

# Extremos del intervalo de integración
a <- 0
b <- 3.14159265358979323
# Número de réplicas
nReplicas <- 5
# -----
# Tamaño de la muestra 50
I_50 <- numeric(0)
for (rep in c(1:nReplicas))
  I_50[rep] <- estimaI(50,a,b)
# -----
# Tamaño de la muestra 500
I_500 <- numeric(0)
for (rep in c(1:nReplicas))
  I_500[rep] <- estimaI(500,a,b)
# -----
# Tamaño de la muestra 5000
I_5000 <- numeric(0)
for (rep in c(1:nReplicas))
  I_5000[rep] <- estimaI(5000,a,b)
# -----
# Representación resultados
I <- data.frame(I_50, I_500, I_5000)
boxplot(I)
abline(h=2, lty=3) # Dibuja linea horizontal
```

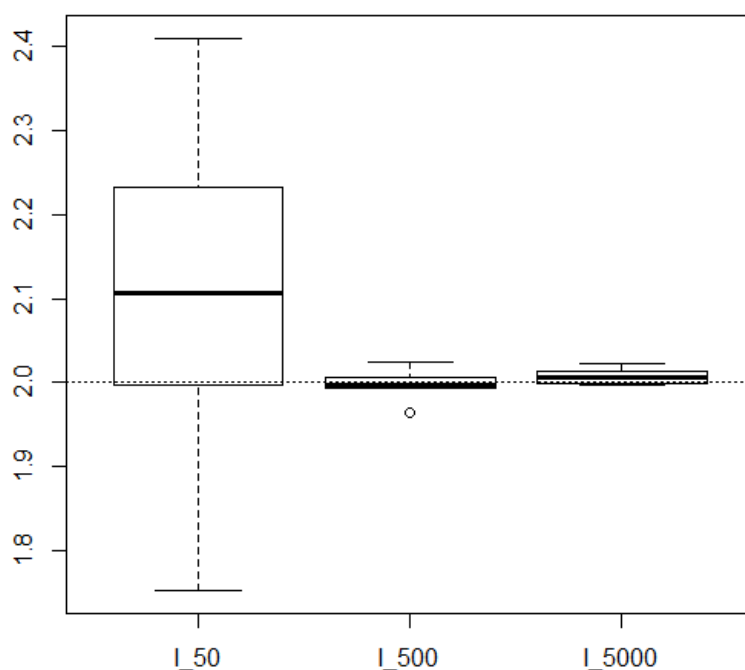


Figura 1.26: Boxplots de las estimaciones de la integral para cada tamaño de muestra.

El resultado obtenido de la ejecución del programa se encuentra almacenado en el data frame `I`. En la Figura 1.26 se muestran los boxplots. Puede observarse que al aumentar el tamaño de muestra los resultados tienden hacia el valor verdadero y su variabilidad disminuye.

Obsérvese también que en general se obtendrán diferentes resultados si se repite la ejecución del programa, ya que R usa en cada caso secuencias diferentes de números pseudoaleatorios.

```
> I
      I_50      I_500      I_5000
1 2.231706 1.964716 2.013596
2 2.106927 1.992722 1.997176
3 2.410476 2.006765 2.006508
4 1.751520 2.024015 2.022142
5 1.996247 1.996353 1.998435
```

Solución al Ejercicio 1.7

Se muestra a continuación el programa. Se crea una matriz llamada `evolAutom`, en cuyas filas se va almacenando el estado del autómatas en los sucesivos instantes de tiempo. En la primera fila se almacena el estado inicial, en la segunda fila el estado en el siguiente instante de tiempo y así sucesivamente. Una vez realizada la simulación, se emplea la función `matrixplot()` para visualizar el contenido de `evolAutom`. Para poder usar esa función es necesario tener cargado el paquete `VIM` (es posible descargarlo de Internet, instalarlo y cargarlo desde la propia plataforma R). En la Figura 1.27 se muestra el resultado de la ejecución del programa.

```
# Estado inicial
estadoInicial <- c( rep(0,30), 1, rep(0,30) );
# -----
# Tabla de transición de estados
# -----
tabla <- numeric(0)
# Estado      Entrada  Estado  Entrada
# siguiente   izqda    actual   drcha
tabla[1] <- 0 #      0      0      0
tabla[2] <- 1 #      0      0      1
tabla[3] <- 0 #      0      1      0
tabla[4] <- 1 #      0      1      1
tabla[5] <- 1 #      1      0      0
tabla[6] <- 0 #      1      0      1
tabla[7] <- 1 #      1      1      0
tabla[8] <- 0 #      1      1      1
# -----
# Número de réplicas
numReplicas <- 30
# -----
# Simulación
evolAutom <- matrix( 0, nrow=numReplicas , ncol=length(estadoInicial) )
evolAutom[1,] <- estadoInicial
for (rep in c(2:numReplicas)) {
  for (cel in c(2:(length(estadoInicial)-1))) {
    evolAutom [rep,cel] <- tabla[ 4*evolAutom[rep-1,cel-1]+
                                2*evolAutom[rep-1,cel]  +
                                evolAutom[rep-1,cel+1]+ 1 ]
  }
}
# La función matrixplot está definida en el paquete VIM
matrixplot(evolAutom, labels=FALSE)
```

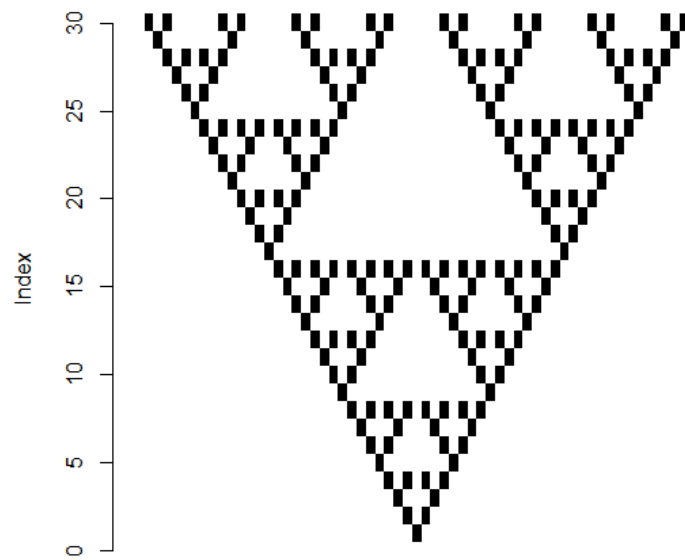


Figura 1.27: Evolución del autómata celular unidimensional con Regla 90.

Solución al Ejercicio 1.8

A continuación se muestra una posible forma de programar el autómata del Juego de la Vida. No es el algoritmo más eficiente posible, pero sí probablemente el más sencillo. En la Figura 1.28 se muestra el resultado de la ejecución.

```
# El espacio celular tiene NxN células
N <- 10 # Debe satisfacerse N > 2
# Estado inicial
estadoInicial <- matrix ( c(
  0,0,0,0,0,0,0,0,0,0,
  0,0,0,0,0,0,0,0,0,0,
  0,0,0,0,0,0,0,0,0,0,
  0,0,0,0,0,0,0,0,0,0,
  0,0,0,0,0,0,0,0,0,0,
  0,0,0,0,0,0,0,0,0,0,
  0,0,0,0,0,0,0,0,0,0,
  0,0,1,1,0,0,0,0,0,0,
  0,1,0,1,0,0,0,0,0,0,
  0,0,0,1,0,0,0,0,0,0,
  0,0,0,0,0,0,0,0,0,0 ),
  nrow = N, ncol = N, byrow = TRUE )
# Número de instantes de evaluación
numEval <- 11
# Se invierte el orden de las filas debido a que matrixplot
# dibuja en la parte superior la fila de mayor índice
estado <- estadoInicial [nrow(estadoInicial ):1, ]
nuevoEstado <- estado
par(mfrow=c(3,4), cex=0.6, mai=c(0.1,0.1,0.3,0.1))
for ( t in 0:numEval ) {
  # Muestra el estado actual
  # La función matrixplot está definida en el paquete VIM
  matrixplot( estado, labels = FALSE, axes = FALSE, ylab = "",
    xlim = 0.5+c(0,N), ylim= 0.5+c(0,N),
    interactive = F, main = paste("t = ",t) )
  abline( h=0.5+(0:N), v=0.5+(0:N) )
  # Calcula el nuevo estado
  for( j in 2:(N-1) ) {
    for ( i in 2:(N-1) ) {
      vecinasVivas <- estado[i-1,j-1] + estado[i,j-1] + estado[i+1,j-1] +
        estado[i-1,j] + estado[i+1,j] +
        estado[i-1,j+1] + estado[i,j+1] + estado[i+1,j+1]
      if ( vecinasVivas < 2 | vecinasVivas > 3 ) {
        nuevoEstado[i,j] <- 0
      } else if ( vecinasVivas == 3 ) {
        nuevoEstado[i,j] <- 1
      }
    }
  }
  estado <- nuevoEstado
}
```

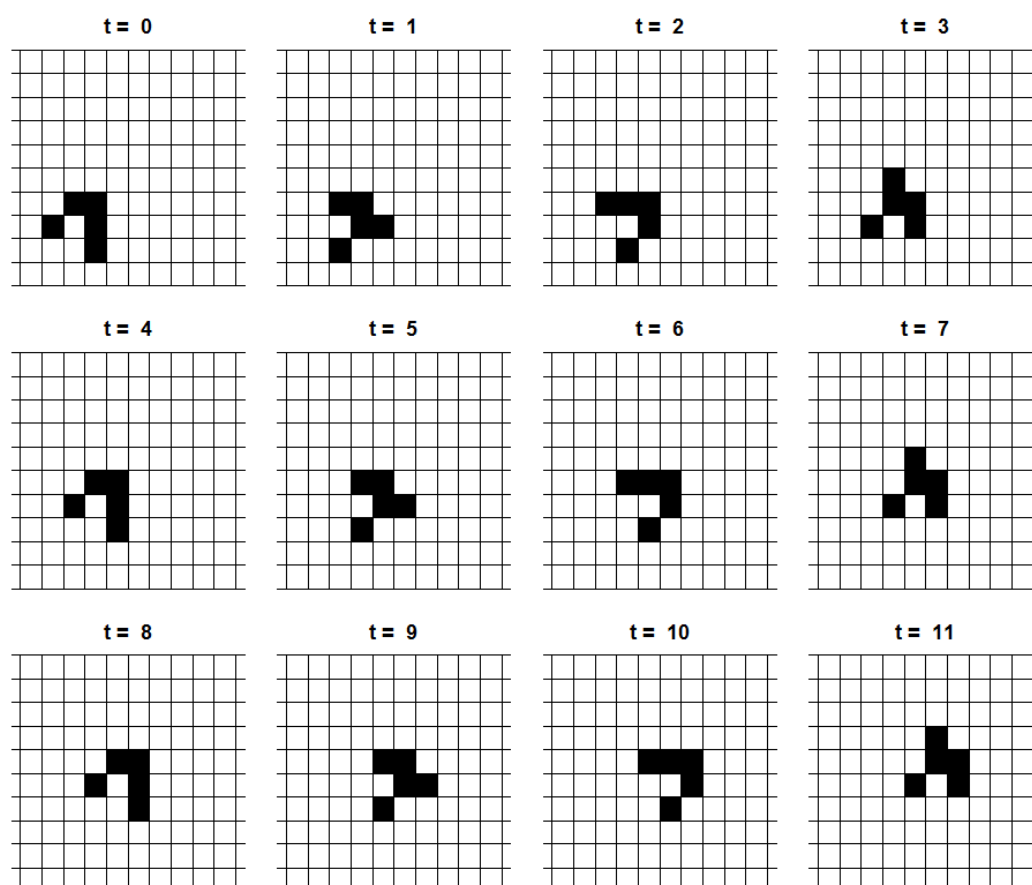


Figura 1.28: Ejemplo de evolución del autómata celular del Juego de la Vida.

Solución al Ejercicio 1.9

```

deltaT <- 0.01 # Paso en el tiempo
t <- 0 # Inicialización del tiempo
# Parámetros
m <- 10
F0 <- 15
w <- 10
g <- -9.8
# Valores iniciales de las variables de estado
x0 <- 10
v0 <- 7
# Inicialización de las variables de estado
x <- x0
v <- v0
# Data frame donde se guardan los resultados
resultado <- data.frame(
  t = numeric(),
  x = numeric(),
  v = numeric(),
  a = numeric() )
# Bucle de la simulación
while ( x > 0 ) {
  F <- m*g + F0*sin(w*t)
  a <- F/m
  derx <- v
  derv <- a
  resultado <- rbind(resultado, c(t,x,v,a))
  x <- x + derx*deltaT
  v <- v + derv*deltaT
  t <- t + deltaT
}
names(resultado) <- c("t","x","v","a")
# Configuración de la representación gráfica
par( mfrow = c(1,2), cex = 1.2 )
# Para poder usar minor.tick es necesario instalar el paquete Hmisc
# x vs t
plot(resultado$t, resultado$x, lty = 1, type = "l",
  yaxt = "n", xlab = "t [s]", ylab = "x [m]", lwd = 1.5 )
axis(2, las = 2)
minor.tick(nx = 5, ny = 4, tick.ratio = 0.5)
abline( v = seq(0,2.5,0.1), h = seq(0,13,0.5), lwd = 0.5 )
# v,a vs t
plot(resultado$t, resultado$v, lty = 1, type = "l",
  yaxt = "n", xlab = "t [s]", ylab = "", lwd = 1.5 )
axis(2, las = 2)
minor.tick(nx = 5, ny = 5, tick.ratio = 0.5)
lines(resultado$t, resultado$a, lty = 2, type = "l", lwd = 1.5)
abline(v = seq(0,2.5,0.1), h = seq(-16,8,1), lwd = 0.5)
legend("topright", inset = 0.05,
  box.lwd = 0, box.col = "white", bg = "white",
  c("v [m/s]", "a [m/s2]"), lty = c(1,2), lwd = c(1.5,1.5) )

```

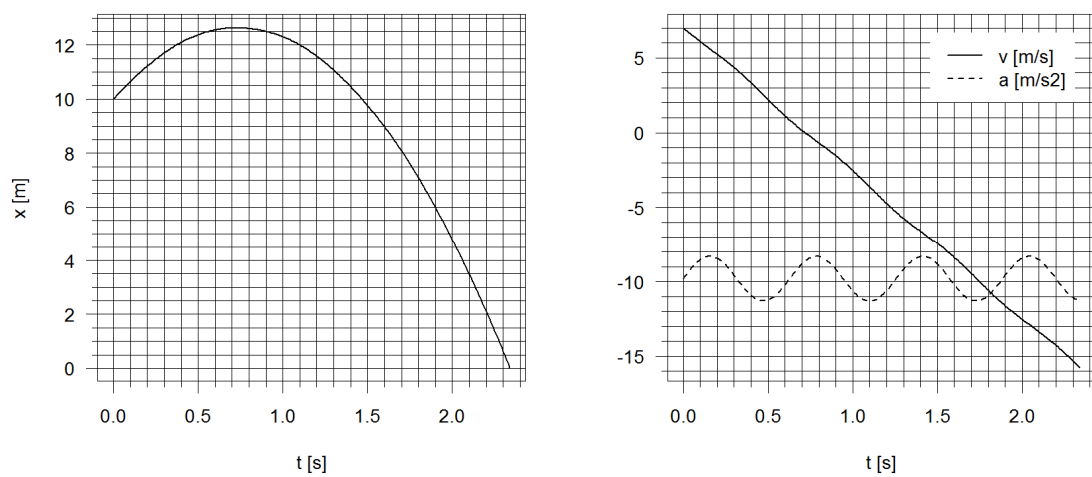


Figura 1.29: Resultado de la ejecución del algoritmo mostrado en la Figura 1.21.

En la Figura 1.29 se muestran las gráficas obtenidas al simular el script anterior.

Solución al Ejercicio 1.10

```

N <- 10000 # Número de pasos de integración
h <- 0.01  # Tamaño del paso
# Parámetros
K <- 5
L0 <- 1
M <- 10
b <- 0.3
# Valor inicial
x0 <- 1.2
v0 <- 0
# Inicialización
x <- numeric(0)
v <- numeric(0)
x[1] <- x0
v[1] <- v0
# Bucle de la simulación
for ( i in 1:N ) {
  v[i+1] <- v[i] - K*h/M*(x[i] - L0) - b*h/M*v[i]
  x[i+1] <- x[i] + h*v[i]
}
t <- seq(0,N*h,h)
# Representación gráfica
# Para poder usar minor.tick es necesario instalar el paquete Hmisc
# Configuración de la representación gráfica
par( mfrow = c(2,1) )
# x vs t
par( mai = c(0.3,1.2,0.4,0.1), cex = 1.3 )
plot(t, x, lty = 1, type = "l", xlim = c(0,N*h),
      ylim = c(0.8,1.2), xaxt = "n",
      yaxt = "n", xlab = "", ylab = "x [m]", lwd = 2 )
axis(2, las = 2)
minor.tick(nx = 2, ny = 5, tick.ratio = 0.5)
abline(v = seq(0,N*h,1), h = seq(0.7,1.3,0.02), lwd = 0.5, lty = 3)
abline(h = L0, lwd = 1, lty = 1)
title("Sistema de muelle y masa", cex = 1.2)
# v vs t
par( mai=c(1.1,1.2,0,0.1), cex = 1.3 )
plot(t, v, lty = 1, type = "l", xlim = c(0,N*h), ylim = c(-0.15,0.15),
      yaxt = "n", xlab = "t [s]", ylab = "v [m/s]", lwd = 2)
axis(2, las = 2)
minor.tick(nx = 2, ny = 5, tick.ratio = 0.5)
abline(v = seq(0,N*h,1), h = seq(-0.2,0.2,0.01), lwd = 0.5, lty=3)
abline(h = 0, lwd = 1, lty=1)

```

En la Figura 1.30 se muestran las dos gráficas obtenidas al simular el script anterior.

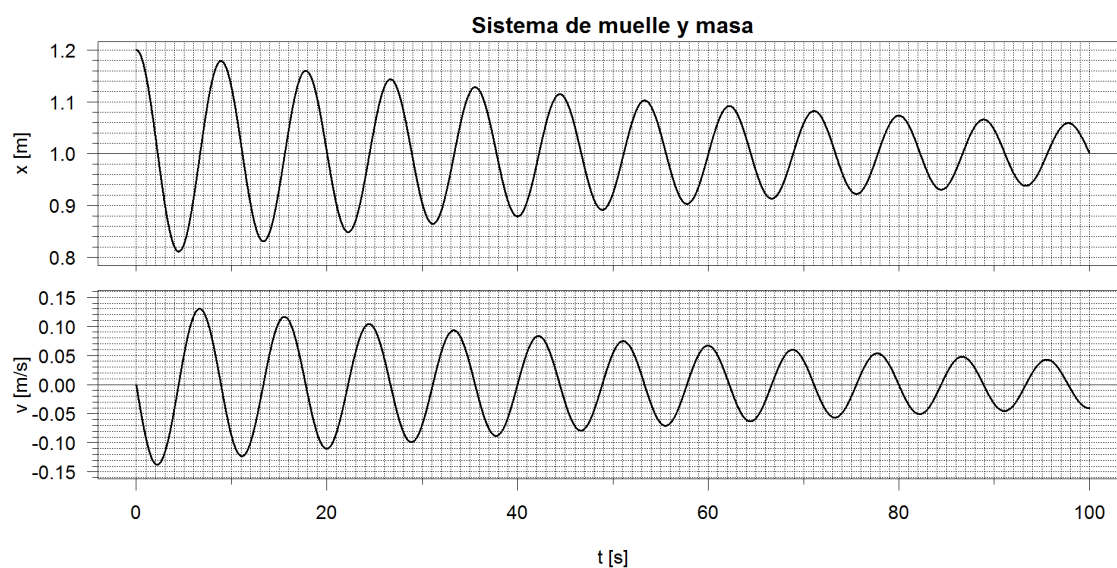


Figura 1.30: Simulación del sistema de muelle y masa.

Solución al Ejercicio 1.11

$$\frac{dx}{dt} + t \cdot x = 0$$

ODE de 1^{er} orden, lineal y homogénea

$$\frac{\partial u}{\partial t} - \frac{\partial^2 u}{\partial x^2} = (1 + t) \cdot \sin(x)$$

PDE de 2^o orden, lineal e inhomogénea

$$\frac{\partial w}{\partial t} + w \cdot \frac{\partial w}{\partial x} = 0$$

PDE de 1^{er} orden, no lineal y homogénea

$$\frac{\partial v}{\partial t} + 3 \cdot \frac{\partial v}{\partial x} = x + 3 \cdot t$$

PDE de 1^{er} orden, lineal e inhomogénea

$$\frac{d^2 \theta}{dt^2} + \sin(\theta) = 0$$

ODE de 2^o orden, no lineal y homogénea

$$\rho(x, y) \cdot \frac{\partial^2 u}{\partial t^2} - T \cdot \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = f(x, t)$$

PDE de 2^o orden, lineal e inhomogénea

$$\frac{d^2 x}{dt^2} - 2 \cdot \frac{dx}{dt} + t \cdot x = 0$$

ODE de 2^o orden, lineal y homogénea

$$\frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} + u \cdot \frac{\partial u}{\partial x} - \frac{\partial^3 u}{\partial x^2 \partial t} = 0$$

PDE de 3^{er} orden, no lineal y homogénea

TEMA 2

MODELADO BASADO EN PRINCIPIOS FÍSICOS

- 2.1. Introducción
- 2.2. El lenguaje Modelica
- 2.3. Fundamentos del modelado de sistemas físicos
- 2.4. Circuitos eléctricos
- 2.5. Sistemas mecánicos
- 2.6. Flujo de fluidos e intercambio de calor
- 2.7. Lecturas recomendadas
- 2.8. Ejercicios de autocomprobación
- 2.9. Soluciones de los ejercicios

OBJETIVOS DOCENTES

Una vez estudiado el contenido del tema y realizados los ejercicios prácticos, debería saber:

- Discutir los fundamentos del paradigma del modelado físico, del modelado modular y jerárquico, y del modelado orientado a objetos, y aplicar este último al diseño de librerías de modelos, de manera que se favorezca la reutilización de los modelos por diferentes desarrolladores y en diferentes contextos.
- Discutir con qué propósito se desarrolló el lenguaje Modelica y cuáles son sus principales capacidades.
- Discutir los conceptos básicos del modelado de sistemas físicos basado en la descripción del almacenamiento, transformación y flujo de la energía. Discutir analogías entre las leyes de la física en los dominios eléctrico, mecánico, hidráulico y térmico.
- Realizar modelos matemáticos, basados en principios físicos, de sistemas multidominio.
- Desarrollar modelos y librerías de modelos de tiempo continuo en lenguaje Modelica, sacando el máximo partido de las capacidades del lenguaje.
- Discutir qué transformaciones sobre el modelo realizan los entornos de modelado que soportan los lenguajes de modelado orientado a objetos.
- Emplear un entorno de modelado de Modelica para ejecutar experimentos de simulación sobre el modelo y analizar los resultados. Interpretar la información proporcionada por el entorno de modelado durante la traducción, el depurado y la simulación del modelo.

2.1. INTRODUCCIÓN

A comienzos de la década de 1990 se desarrollaron los primeros lenguajes de modelado orientado a objetos y aparecieron las primeras herramientas software (entornos de modelado) para la simulación de modelos descritos empleando estos lenguajes. El objetivo era facilitar la descripción de modelos híbridos de sistemas físicos, en los cuales típicamente aparecen interrelacionados fenómenos de diferentes dominios: eléctrico, mecánico, hidráulico, térmico, químico, etc. Se trata, por tanto, de lenguajes de modelado de propósito general, no ligados a un dominio específico.

Durante los primeros años, el uso de los lenguajes de modelado era minoritario y estaba restringido al ámbito académico. La existencia de un elevado número de lenguajes de modelado contribuyó a la dispersión del esfuerzo en el desarrollo de herramientas y de librerías. Librerías similares eran desarrolladas por diferentes desarrolladores, sin que hubiera posibilidad de reutilizar el código debido a que estaban escritas en diferentes lenguajes de modelado. Igualmente, las mejoras en las capacidades de cada entorno de modelado únicamente resultaban ventajosas para la pequeña comunidad de usuarios que empleaba el lenguaje soportado por dicho entorno.

Con el fin de proponer un lenguaje de modelado estándar, que permita el intercambio de modelos y la compatibilidad entre entornos de modelado, se estableció en el año 1996 un grupo de diseño que estaba integrado por personas que ya habían intervenido anteriormente en el desarrollo de otros lenguajes de modelado, así como por personas con experiencia en el desarrollo de modelos en el ámbito académico y de la industria. El lenguaje propuesto, que se denominó **Modelica**, incorporó características de lenguajes de modelado previamente existentes, tales como ALLAN, Dymola, NMF, ObjectMath, Omola, SIDOPS+ y Smile.

Desde en el año 1997, se han ido sucediendo sucesivas versiones de Modelica. Un hito importante fue la creación en el año 2000 de la **Modelica Association**: una asociación sin ánimo de lucro cuyo objetivo es el desarrollo y difusión del lenguaje Modelica. En su sitio web

<http://www.modelica.org>

se publican las especificaciones del lenguaje, así como artículos científicos, manuales, librerías de modelos, enlaces para la descarga de software, etc. Entre las librerías de modelos gratuitas cabe destacar la **Modelica Standard Library** (MSL), que es desarrollada y mantenida por la Modelica Association.

Este esfuerzo estandarizador, que resultó en la propuesta del lenguaje Modelica, supuso un impulso importante para la popularización del modelado orientado a objetos, el cual hoy en día es ampliamente usado en el ámbito académico e industrial. Actualmente hay disponibles múltiples librerías gratuitas y comerciales escritas en Modelica. La posibilidad de componer modelos, reutilizando componentes de diferentes librerías, es uno de los puntos fuertes de Modelica. Asimismo, existe una variedad de herramientas gratuitas y comerciales que soportan el lenguaje Modelica o parte de él. Algunas de estas herramientas son CATIA, CyModelica, Dymola, JModelica.org, LMS Imagine.Lab AMESim, MapleSim, MathModelica, Modelicac, MWorks, SimulationX, OpenModelica, Scicos y Wolfram SystemModeler.

2.2. EL LENGUAJE MODELICA

En esta sección se describen los conceptos fundamentales del paradigma del modelado físico y del modelado orientado a objetos, ya que constituyen la base conceptual sobre la que se asientan todos los lenguajes de modelado orientado a objetos. También se ofrece una introducción a algunas de las características más sobresalientes de Modelica y se muestra el uso de dos de los entornos de modelado que soportan Modelica: Dymola y OpenModelica.

En las siguientes secciones de este tema se irán introduciendo por medio de ejemplos otras capacidades de Modelica para la descripción de modelos de tiempo continuo. A ese respecto, el objetivo propuesto en este tema es enseñar al lector a desarrollar modelos y librerías de modelos de tiempo continuo en Modelica.

2.2.1. Paradigma del modelado físico

Modelica es un lenguaje de modelado concebido para facilitar la aplicación de un paradigma de modelado denominado el paradigma del modelado físico (no confundir con los *modelos físicos* explicados en la Sección 1.2.1, que son maquetas a escala del sistema real y nada tienen que ver con la metodología para la descripción de modelos matemáticos que ahora nos ocupa).

El **paradigma del modelado físico** es una metodología para el modelado de los sistemas físicos, que propone plantear el modelo de la forma siguiente:

1. Definir la estructura del sistema y dividir el sistema en partes.
2. Definir la interacción entre las partes.

3. Definir el comportamiento interno de cada parte, independientemente de las demás, mediante ecuaciones: en términos de balances de materia, energía y momento, y de relaciones constitutivas.

El modelo que típicamente se obtiene mediante la aplicación de este paradigma es un modelo de tiempo continuo, descrito mediante ecuaciones algebraicas y diferenciales (modelo DAE). Los lenguajes concebidos para dar soporte a este paradigma de modelado permiten al desarrollador del modelo describir el comportamiento del sistema mediante ecuaciones. Estos lenguajes de modelado, entre los cuales se encuentra Modelica, se denominan **lenguajes basados en ecuaciones**.

Una **ecuación** expresa una relación de igualdad entre dos expresiones,

$$\text{expresión}_1 = \text{expresión}_2$$

en las cuales intervienen una o varias variables del modelo. Al simular el modelo, la ecuación debe emplearse para evaluar una de esas variables.

En los lenguajes basados en ecuaciones, la forma en que el desarrollador del modelo escribe una ecuación no indica qué variable debe ser evaluada de ella. Asimismo, el orden de escritura de las ecuaciones del modelo no indica el orden en que deben ser evaluadas al simular el modelo.

Para ilustrar este punto, supongamos que se está modelando una resistencia de valor conocido (R). La caída de tensión (v) entre los bornes de la resistencia y la corriente que circula a través de ella (i) satisfacen una relación lineal: la ley de Ohm. Dicha ecuación puede escribirse de diferentes maneras, por ejemplo de una de las seis siguientes:

$$\begin{array}{lll} v = i \cdot R & i = v/R & R = v/i \\ v/i = R & v - i \cdot R = 0 & 0 = v - i \cdot R \end{array}$$

Si la descripción del modelo se realiza empleando un lenguaje basado en ecuaciones, la forma en que se escriba la ecuación es indiferente, ya que el hecho de escribirla de una u otra manera no aporta información adicional alguna.

La **causalidad computacional** indica qué ecuación del modelo se emplea en la simulación para evaluar cada variable, o equivalentemente, qué variable es evaluada de cada ecuación del modelo.

Dado que en el modelado basado en ecuaciones la manera de escribir una ecuación no indica su causalidad computacional, el modelado basado en ecuaciones también se denomina **modelado acausal**.

Cuando se emplean lenguajes de modelado basados en ecuaciones, la asignación de la causalidad computacional, así como la manipulación simbólica y ordenación de las ecuaciones del modelo, no están a cargo del desarrollador del modelo, sino del entorno de modelado, el cual dispone de algoritmos para realizarlas de manera automática. Esto no sólo facilita enormemente el desarrollo de los modelos, sino que también facilita la reutilización de los modelos en diferentes contextos.

La causalidad computacional de cada ecuación no sólo depende de ella misma, sino también de las demás ecuaciones del modelo. Se dice por ello que *la causalidad computacional es una propiedad global del modelo*.

Continuando con el ejemplo anterior, si la resistencia está conectada a un generador de tensión, la causalidad del modelo es diferente que si se encuentra conectada a un generador de corriente. Se muestran seguidamente los dos modelos:

$$\left. \begin{array}{l} [v] = f_v(time) \\ v = [i] \cdot R \end{array} \right| \begin{array}{l} [i] = f_i(time) \\ [v] = i \cdot R \end{array}$$

La primera ecuación es la relación constitutiva del generador (de tensión en un caso y de corriente en el otro) y la segunda ecuación es la relación constitutiva de la resistencia. La causalidad computacional se ha señalado escribiendo entre corchetes la variable evaluada de cada ecuación. Obsérvese que la ecuación que describe la resistencia es la misma en ambos casos, sin embargo la causalidad computacional de dicha ecuación es diferente.

Además de mediante ecuaciones, Modelica permite describir el modelo empleando **algoritmos**. Un algoritmo es una secuencia ordenada de asignaciones. Una **asignación** tiene la forma siguiente:

variable := expresión

La causalidad computacional de una asignación está explícita en la forma en que ésta se escribe: la variable a evaluar de la asignación es la variable escrita en el lado izquierdo. Su valor se calcula evaluando la expresión escrita en el lado derecho.

Cuando se describe el modelo mediante algoritmos, es el desarrollador del modelo quien decide qué asignación se emplea para evaluar cada variable y en qué orden deben calcularse las variables. El entorno de modelado ni ordena ni manipula las asignaciones: las ejecuta en el mismo orden en que aparecen escritas en el algoritmo. Puesto que el desarrollador del modelo debe indicar explícitamente la causalidad computacional, el modelado mediante asignaciones se denomina **modelado causal**.

Además de algoritmos, Modelica permite definir **funciones**, que pueden ser invocadas desde cualquier parte del modelo. El lenguaje ofrece dos posibilidades:

- Los algoritmos escritos en lenguaje Modelica pueden ser reutilizados encapsulándolos en funciones, cuyo cuerpo es el propio algoritmo y que están dotadas de una interfaz bien definida: unos argumentos de entrada (entradas computacionales) y unos argumentos de salida (salidas computacionales), que son calculados del algoritmo.
- En el cuerpo de las funciones en Modelica, puede embeberse llamadas a funciones escritas en C y Fortran.

Modelica permite definir en el modelo variables de tiempo discreto y facilita la descripción de **eventos**. La acción asociada a un evento puede:

- Producir un cambio instantáneo en el valor de las variables de estado de tiempo discreto y de tiempo continuo.
- Modificar la descripción matemática de la parte de tiempo continuo del modelo.

En todo momento, las ecuaciones y algoritmos activos expresan relaciones entre las variables, que deben ser satisfechas concurrentemente. El conjunto de ecuaciones activas puede estar compuesto de ecuaciones de tiempo continuo (durante la solución del problema de tiempo continuo), y de una mezcla de ecuaciones de tiempo continuo y discreto, (durante la ejecución de los eventos). El orden en que deben ser evaluadas las ecuaciones es determinado automáticamente por el entorno de modelado, de manera que en todo momento debe quedar unívocamente definido de qué ecuación o algoritmo debe ser evaluada cada variable.

2.2.2. Orientación a objetos

El **modelado orientado a objetos** está fundamentado en los principios del modelado modular y jerárquico, es decir, en

- la reutilización de los modelos mediante composición,
- la distinción entre la interfaz del modelo y su descripción interna, y

- el encapsulado de la información,

e incorpora conceptos nuevos, tales como

- el concepto de clase de modelo y
- la reutilización mediante herencia de las clases de modelos.

Modelica proporciona pleno soporte a la metodología del modelado orientado a objetos, facilitando la reutilización de las clases de modelos mediante composición y herencia. Se describen a continuación algunas de las capacidades básicas ofrecidas por el lenguaje.

- *Modularidad y jerarquía.* En la definición de las clases de modelos, Modelica permite distinguir entre la interfaz y la descripción interna del modelo, y encapsular la información. Las variables que no pertenecen a la interfaz pueden ser declaradas **variables protegidas**, de manera que no sea posible acceder a ellas desde fuera de la clase.

Para facilitar la descripción de la conexión entre componentes, Modelica permite agrupar las variables de la interfaz en **conectores** y proporciona la sintaxis para definir la conexión entre conectores.

Asimismo, establece reglas (inspiradas en el principio de conservación de la energía) para relacionar las variables de los conectores en el punto de conexión. A tal fin, las variables de los conectores se clasifican en dos tipos: *across* y *through*.

- Las **variables across**, también denominadas **variables esfuerzo** (*effort*), son aquellas que valen lo mismo en el punto de conexión.
 - Las **variables through**, también llamadas **variables flujo** (*flow*), son aquellas cuya suma vale cero en el punto de conexión.
- *Herencia.* Modelica soporta **herencia múltiple** y la definición de **clases parciales**, que incluyen propiedades generales de la clase pero que no pueden instanciarse. Esto facilita la descripción de características comunes, que son compartidas por varias clases.
 - *Parametrización.* Modelica permite modificar el valor de los **parámetros** de una clase al instanciarla y heredarla. Los parámetros pueden ser variables no dependientes del tiempo. También es posible modificar la clase de los objetos

Tabla 2.1: Clases del lenguaje Modelica.

Nombre	Aplicación
<code>type</code>	Permite extender los tipos de variables predefinidos en el lenguaje.
<code>connector</code>	Define conectores (esto es, grupos de variables de la interfaz de los componentes) con el fin de facilitar la descripción de la conexión entre componentes. No puede contener ecuaciones.
<code>model</code>	Se emplea para definir las clases de modelos.
<code>block</code>	Es igual que la clase <code>model</code> , pero en la clase <code>block</code> las variables de la interfaz deben tener la causalidad computacional explícitamente definida.
<code>record</code>	Permite definir conjuntos de variables y de parámetros. No puede tener ecuaciones. Su finalidad es facilitar la parametrización de los modelos.
<code>function</code>	Permite la definición de funciones, es decir, el encapsulado de código algorítmico que puede ser reutilizado en la definición del modelo y también del experimento.
<code>package</code>	Facilita la definición de librerías de clases. Un <code>package</code> es una clase que únicamente puede contener otras clases.

que forman parte de las clases compuestas y que son heredados. Esto simplifica la experimentación con los modelos, al permitir al desarrollador que modifique la clase de los objetos, en lugar de obligarle a reescribir el modelo y sus componentes.

- *Anotaciones.* Modelica permite incluir **anotaciones** en los modelos, mediante las cuales se especifica la representación gráfica del icono y el diagrama del modelo, la documentación del modelo, etc. La capacidad de incorporar esta información gráfica en los modelos posibilita su composición mediante un editor gráfico de modelos.

La definición de los modelos, las librerías, las magnitudes físicas, los conjuntos de datos y las funciones se realiza mediante las **siete clases** mostradas en la Tabla 2.1, las cuales constituyen la columna vertebral del lenguaje Modelica.

2.2.3. Entornos de modelado

Los modelos descritos en Modelica son traducidos por el entorno de modelado en un **modelo plano**, con toda la estructura (jerarquía y composición) y herencia

deshecha. Este modelo plano está compuesto por la declaración de las variables, ecuaciones algebraicas, ecuaciones diferenciales ordinarias, algoritmos y eventos.

Puesto que las ecuaciones no contienen información acerca de su causalidad computacional, el entorno de modelado debe asignarles la causalidad computacional. Resultado de este análisis, en el que debe considerarse el modelo completo, se obtiene el orden en que deben evaluarse las ecuaciones y los algoritmos, y qué variable debe evaluarse de cada ecuación. Este análisis, mediante el cual se asigna la causalidad computacional y se ordena el modelo, se denomina **partición**.

Aquellas ecuaciones en que la variable a evaluar interviene linealmente pueden ser manipuladas simbólicamente, a fin de despejar dicha variable. El entorno de modelado puede realizar dichas manipulaciones simbólicas. Además, debe plantear la resolución numérica de las ecuaciones en que la variable a evaluar interviene no linealmente. Una vez realizadas estas y otras operaciones de análisis y manipulación simbólica del modelo, el entorno de modelado genera el código del algoritmo de la simulación escrito en algún lenguaje de programación, que generalmente es el lenguaje C.

El **algoritmo de la simulación** del modelo es esencialmente el representado en la Figura 1.12. Consta de los pasos siguientes:

1. La parte de tiempo continuo se resuelve usando algoritmos numéricos para la resolución numérica de sistemas DAE.
2. Si durante la integración se satisface la condición de disparo de algún evento, se detiene la integración y se calcula de manera precisa el instante de disparo del evento.
3. En el instante del evento, se resuelve el conjunto de ecuaciones de tiempo continuo y ecuaciones discretas.
4. Una vez ejecutadas las acciones del evento, se comprueban de nuevo las condiciones de disparo de los eventos. Si uno o varios eventos han sido disparados, se ejecutan inmediatamente. En caso contrario, se reinicia la integración de la parte de tiempo continuo.

Además de traducir el modelo escrito en Modelica a código ejecutable, los entornos de modelado típicamente facilitan la edición gráfica del modelo, la experimentación con el modelo y la representación gráfica de los resultados de la simulación.

A fin de ilustrar dichas capacidades, se introducen seguidamente los conceptos básicos de manejo del entorno de modelado **Dymola**. Si bien Dymola es un entorno

de modelado comercial, su versión gratuita de demostración, que está limitada a modelos de hasta 10 variables de estado, es suficiente para describir y simular los modelos descritos en este texto.

En las Figuras 2.1 y 2.2 se señalan algunos elementos de la interfaz de usuario de Dymola. Obsérvese que la interfaz gráfica de usuario de Dymola tiene dos ventanas principales: la ventana *Modeling*, mostrada en la Figura 2.1, que sirve para definir y comprobar el modelo, y la ventana *Simulation*, mostrada en la Figura 2.2, en la cual se configura el experimento, se traduce el modelo, se simula y se representan los resultados. Los dos botones que hay en la parte inferior derecha permiten cambiar de una ventana a la otra.

Dymola traduce el modelo escrito en Modelica a un programa en lenguaje de programación C llamado *dsmode.c*. Al ejecutar la simulación, Dymola compila dicho programa, generándose un fichero ejecutable llamado *dymosim.exe*. Para ello, Dymola necesita emplear un compilador de C. Puesto que dicho compilador no se distribuye con Dymola, el usuario debe tener uno instalado. En la versión para Windows de Dymola 2015 los compiladores soportados son Microsoft Visual Studio / Visual C++ Express Edition y MinGW GCC. Una vez instalado el compilador, debe configurarse Dymola para que lo use. En la Figura 2.3 se muestra cómo hacerlo. Se recomienda en todo caso consultar la documentación, a la que puede accederse pulsando *Help/Documentation*.

Para completar esta breve introducción al manejo de Dymola, veremos cómo editar, comprobar, traducir y simular un modelo que describe el movimiento de un péndulo simple en el plano. Los pasos a seguir son básicamente los siguientes (en el pie de las Figuras 2.4 a 2.8 se amplía esta información):

1. *Creación de un nuevo modelo.* Para definir un nuevo modelo, seleccionar *File/New/Model* en la ventana *Modeling*, y asignar nombre al modelo. En este ejemplo el nombre será *pendulo* (véase la Figura 2.4). Puesto que no vamos a incluir este modelo dentro de ninguna librería de modelos Modelica (package), el modelo estará situado en el nivel jerárquico superior.
2. *Escritura del código Modelica del modelo.* Pulsando el botón *Modelica Text* (véase la Figura 2.5), aparece una ventana en la cual puede teclearse la descripción Modelica del nuevo modelo *pendulo*. Ésta se muestra en Código 2.1. Puede observarse que el modelo contiene dos partes: primeramente se declaran las variables y seguidamente, a continuación de la palabra reservada **equation**, se escriben las ecuaciones.

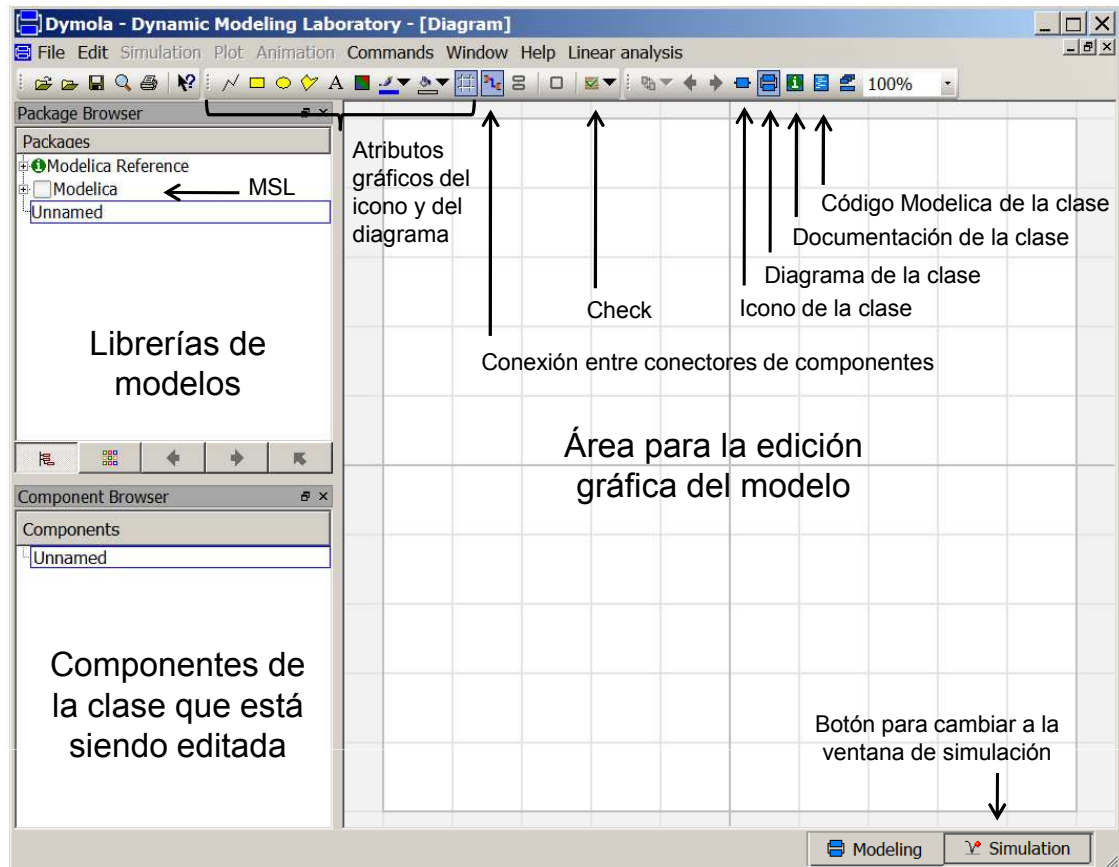


Figura 2.1: Ventana para la edición del modelo de Dymola 2015. La ventana está dividida en tres áreas: área para navegar en las librerías de modelos (ventana *packages*, parte superior izquierda), área donde se muestran los componentes de la clase que está siendo editada (ventana *components*, parte inferior izquierda) y área para la edición del modelo (parte derecha). La *Modelica Standard Library* (MSL) aparece cargada por defecto. En la parte superior hay una regleta con botones, los cuales permiten abrir modelos guardados en fichero, salvar modelos a fichero, definir atributos gráficos, conectar modelos, comprobar (*check*) sintácticamente los modelos y contabilizar el número de ecuaciones e incógnitas, mostrar y modificar el icono, diagrama, documentación y código Modelica de la clase que está siendo editada, etc. En la parte inferior derecha de la ventana hay dos botones, *Modeling* y *Simulation*, que permiten cambiar a la ventana para modelado (la mostrada en esta figura) y a la ventana para simulación.

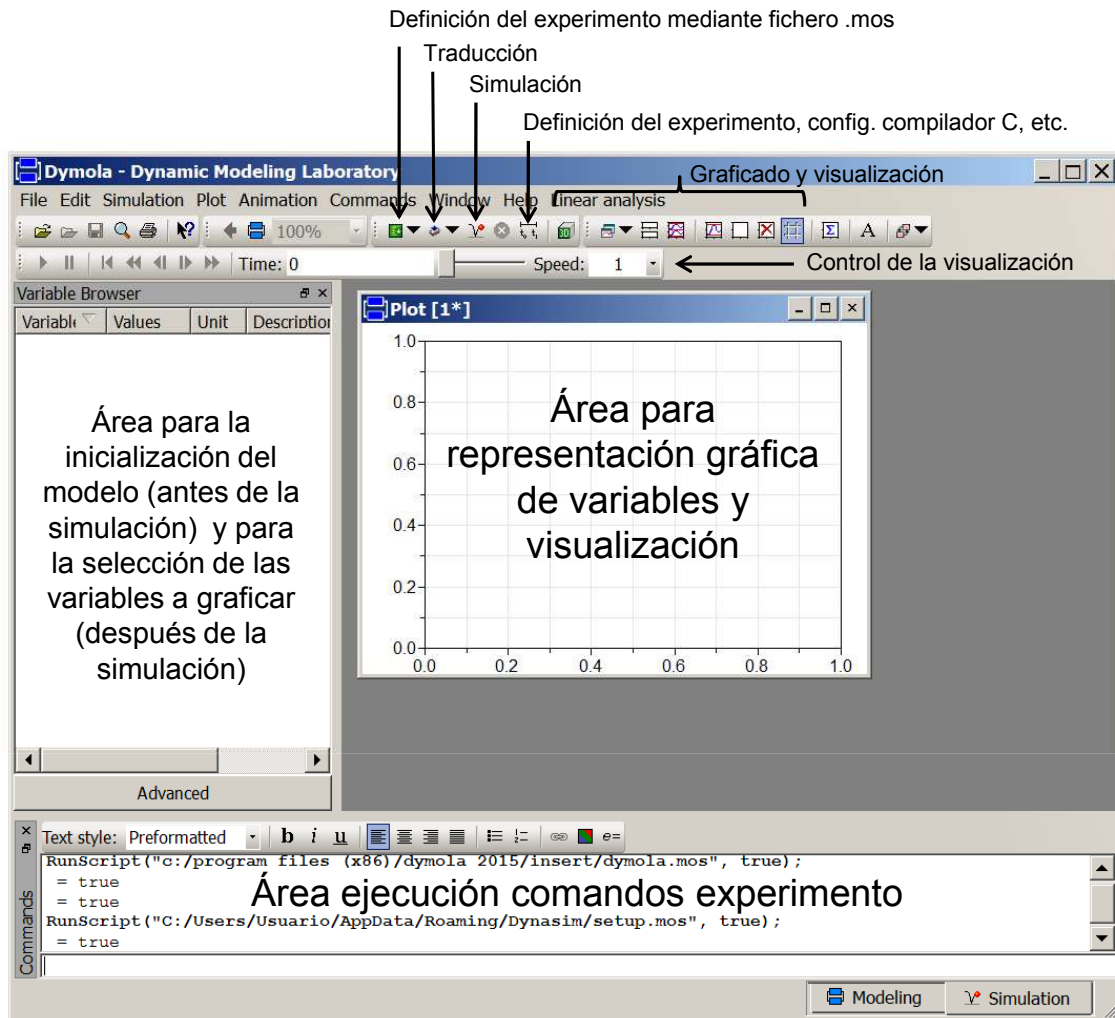


Figura 2.2: Ventana para la simulación de Dymola 2015. Está dividida en tres áreas. En la ventana de la parte izquierda se completa, antes de ejecutar la simulación, la inicialización del modelo. También se seleccionan las variables a representar gráficamente durante o tras la simulación. En la ventana de la derecha se muestran los gráficos y las animaciones. La ventana inferior está a su vez dividida en dos: en la parte inferior se escriben los comandos del experimento, mientras que en la parte superior Dymola va mostrando el resultado de la ejecución de los comandos experimentales, tanto los tecleados en la ventana inferior como los definidos mediante los botones y menús. Los comandos del experimento pueden también escribirse en un fichero de texto con extensión *.mos* (*Modelica script*), que puede ser seleccionado y ejecutado pulsando el botón indicado en la figura.

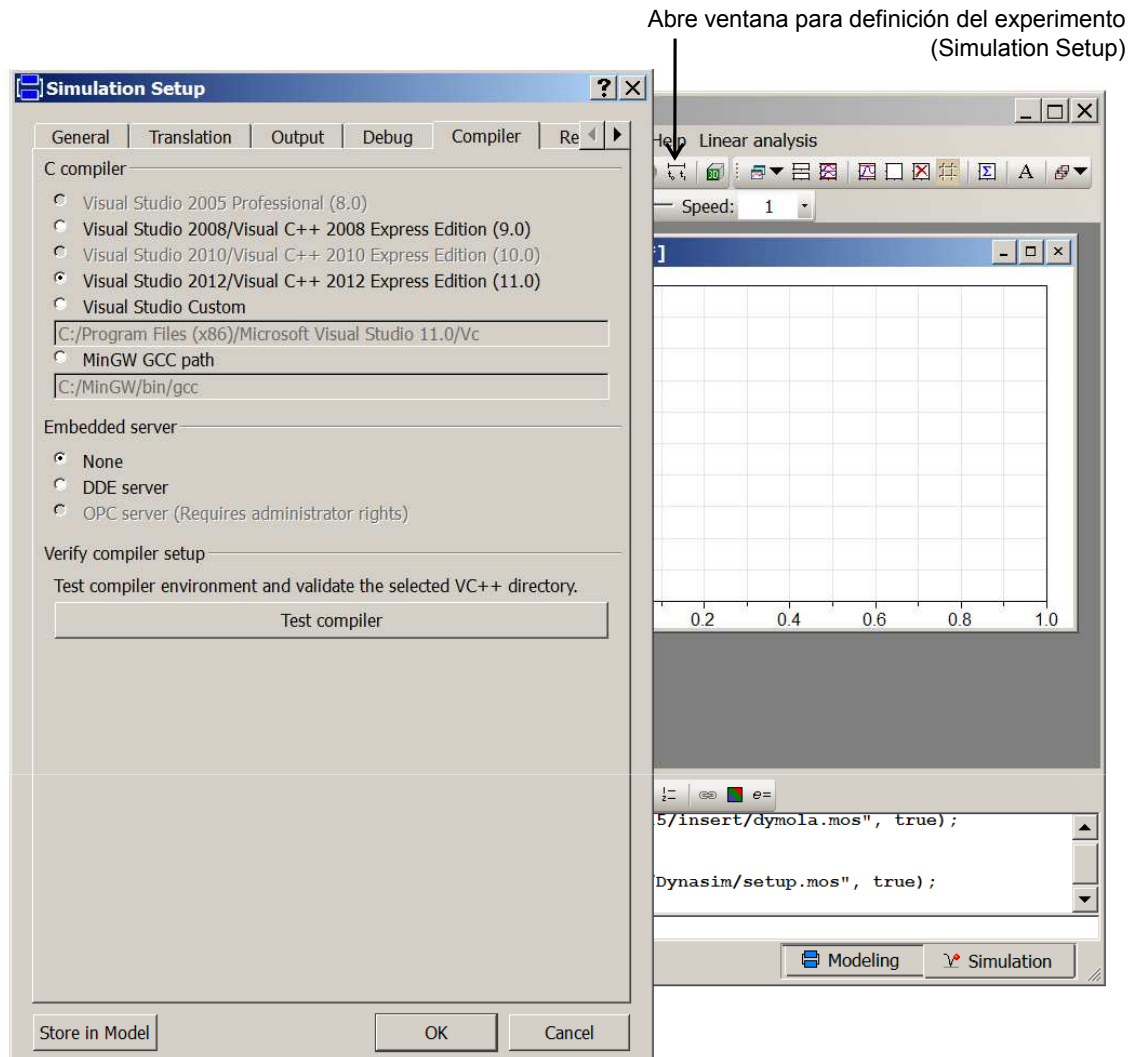


Figura 2.3: Pulsando el botón para la definición del experimento y seleccionando la lengüeta *Compiler*, se muestra la ventana en la cual se indica a Dymola dónde está instalado el compilador de C. Puede comprobarse que Dymola pueda usar correctamente el compilador pulsando el botón *Test compiler*.


```

model pendulo
  constant Real g = 9.81 "Aceleración gravitatoria";
  parameter Real L = 1 "Longitud";
  Real phi(start=0.1, fixed=true) "Ángulo";
  Real w(start=0, fixed=true) "Velocidad angular";
equation
  der(phi) = w;
  // Segunda Ley de Newton
  L*der(w) = -g*sin(phi);
end pendulo;

```

Código 2.1: Movimiento de un péndulo simple en el plano.

El modelo consta de dos variables reales de tiempo continuo, **phi** y **w**, que representan el ángulo y la velocidad angular de péndulo, un parámetro de tipo real, **L**, que representa la longitud del péndulo, y una constante de tipo real, **g**, que representa la aceleración gravitatoria.

Los parámetros y constantes mantienen su valor fijo durante toda la simulación. La diferencia entre ambos es que el valor de los **parámetros** puede ser modificado al definir el experimento, cosa que no es posible hacer con las **constantes**. El valor de ambos, $L = 1$ y $g = 9.81$, se especifica en la declaración. El valor del parámetro podrá ser modificado al definir el experimento.

Las derivadas de las dos variables de tiempo continuo son evaluadas de las dos ecuaciones. Las variables son calculadas integrando numéricamente sus derivadas. El valor inicial de las variables es indicado en la declaración (se asigna al atributo *start* y se indica que el atributo *fixed* vale true): el ángulo inicialmente vale 0.1 radianes y la velocidad angular es nula.

Pueden añadirse comentarios en el código del modelo. Los comentarios asociados a una declaración se escriben entre comillas dobles ("), antes del punto y coma. Por ejemplo:

```

  constant Real g = 9.81 "Aceleración gravitatoria";
  parameter Real L = 1 "Longitud";
  Real phi(start=0.1, fixed=true) "Ángulo";

```

Cuando el comentario deba incluir una comilla, debe escribirse \" para distinguirlo de la comilla de cierre del comentario. Por ejemplo:

```

  parameter Real href = 2 "Referencia: \"setpoint\" de la altura";

```

Estos comentarios serán mostrados por el editor gráfico de modelos del entorno de modelado. Por ejemplo, en el caso del parámetro, en la ventana de variables en la que se permite al usuario cambiar su valor.

Modelica permite también incluir en el modelo comentarios que serán ignorados por el editor del modelo. Todo el texto que se escriba en una línea a continuación de `//` será ignorado. Cuando los comentarios ocupan varias líneas, pueden englobarse entre los símbolos `/*` y `*/`.

Los comentarios al código son escritos entre comillas dobles (`"`), en una única línea precedidos de `//`, o en varias líneas comprendidos entre los símbolos `/*` y `*/`. Por ejemplo,

```
/*
  Segunda Ley de Newton
*/
// Segunda Ley de Newton
L*der(w) = -g*sin(phi); // Segunda Ley de Newton
```

La parte del modelo escrita a continuación de la palabra reservada *equation* se denomina la **sección equation**. En ella se escriben las ecuaciones que describen el comportamiento dinámico del modelo.

```
equation
  der(phi) = w;
  // Segunda Ley de Newton
  L*der(w) = -g*sin(phi);
```

La forma en la que se expresa la ecuación y el orden en el cual se escriben las ecuaciones es indiferente, puesto que será el entorno de modelado quien decida automáticamente qué incógnita se evalúa de cada ecuación y quien manipule simbólicamente (cuando esto sea posible) cada ecuación para despejar la incógnita a evaluar.

Obsérvese que la derivada respecto al tiempo de la variable `phi` se representa `der(phi)` y la derivada respecto al tiempo de `w` se representa `der(w)`.

3. *Comprobación del modelo.* Pulsando el botón *Check* (véase nuevamente la Figura 2.5), Dymola comprueba que el modelo que está siendo editado no contenga errores sintácticos, y además muestra el número de ecuaciones y de incógnitas del modelo. Como se explicará en el Tema 3, se contabilizan como incógnitas las variables dependientes del tiempo. No se consideran incógnitas ni las constantes, ni los parámetros.
4. *Traducción del modelo.* Pulsando el botón *Simulation*, se cambia a la ventana de simulación de Dymola. La traducción del modelo se realiza pulsando el botón *Translate* (véase la Figura 2.6). Al hacerlo, Dymola genera el código C del algoritmo de la simulación y lo compila. Dymola muestra información en la

ventana de mensajes sobre el número de incógnitas, el número de ecuaciones, el número y tamaño de los sistemas simultáneos de ecuaciones (los cuales deben ser resueltos empleando métodos numéricos iterativos), etc.

5. *Inicialización del modelo.* Una vez concluida la compilación, Dymola muestra en la ventana izquierda, denominada *Variable Browser*, las variables a las que hay que asignar valor para inicializar el modelo (véase nuevamente la Figura 2.6). Éstas son el ángulo, la velocidad angular y la longitud del péndulo. Dymola muestra los valores asignados en el modelo y permite modificarlos. Estos valores permiten resolver el modelo en el instante inicial de la simulación. El valor de la longitud del péndulo se mantendrá constante durante toda la simulación. La evolución en el tiempo del ángulo y la velocidad angular se calculará resolviendo las dos ecuaciones del modelo.
6. *Definición del experimento.* Pulsando el botón *Setup* (véase la Figura 2.7), se abre la ventana *Simulation setup*. En la lengüeta *General*, el usuario puede indicar el instante inicial y final de la simulación, cada cuánto tiempo o el número de veces en que Dymola debe almacenar el valor de las variables de salida, y también puede seleccionar el método de integración y su tolerancia. Asignamos al instante final de la simulación el valor 5.
7. *Ejecución de la simulación y análisis de los resultados.* La simulación se inicia pulsando el botón *Simulate* (véase la Figura 2.8). Dymola muestra en la ventana *Variable Browser* las variables de salida. Pulsando sobre ellas, tanto durante la simulación como al terminar ésta, se obtiene su representación gráfica. Dymola guarda también los resultados en fichero.

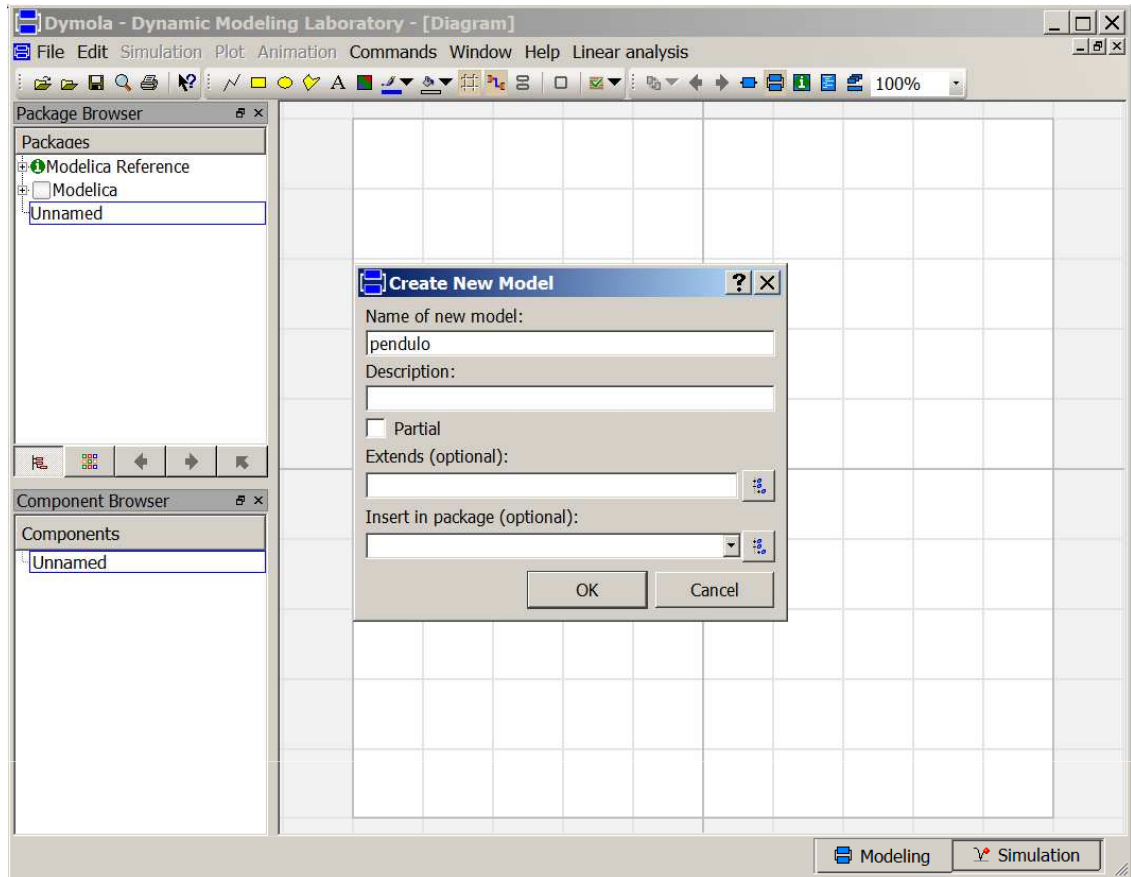


Figura 2.4: Puede definirse una nueva clase de modelo pulsando *File/New/Model*. Al hacerlo, se abre una ventana (como la mostrada en la figura) donde se puede indicar el nombre de la nueva clase (en la casilla *Name of new model*) y un breve comentario acerca de su finalidad (en la casilla *Description*). Señalando el cuadro *Partial* se indica que esta nueva clase podrá ser heredada, pero no instanciada, es decir, se indica que se trata de una clase parcial. Si el nuevo modelo hereda de otros modelos, en la casilla *Extends* pueden indicarse las superclases. Asimismo, si el nuevo modelo debe ser parte de una librería ya existente, en la casilla *Insert in package* puede indicarse en qué paquete de la librería debe incluirse. En este ejemplo dejaremos sin rellenar todas las casillas salvo la primera, en la que hemos indicado el nombre del modelo. Al pulsar el botón OK, Dymola muestra una segunda ventana pidiendo que confirmemos que efectivamente el nuevo modelo no debe ser incluido dentro de ninguna librería. Así debe ser en este caso, con lo cual aceptaremos la confirmación.

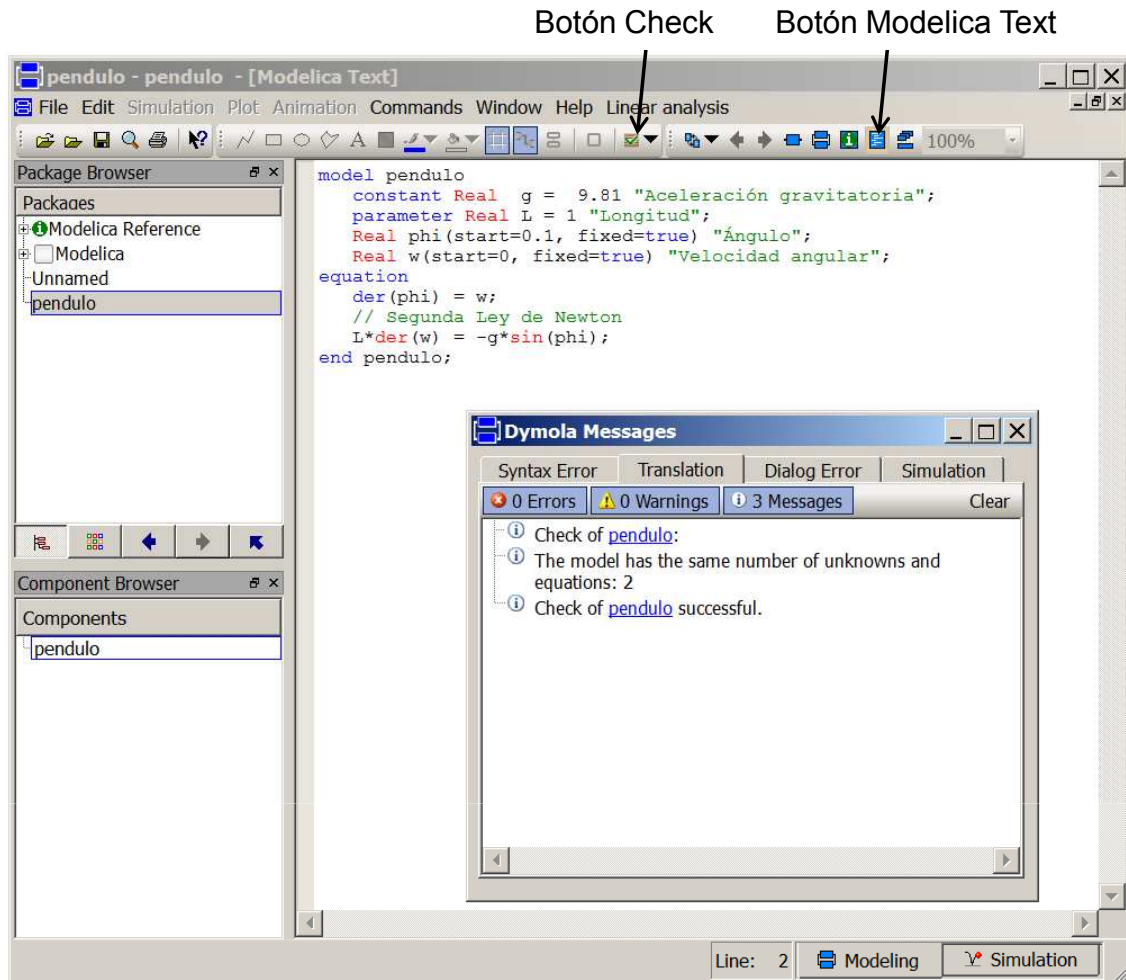


Figura 2.5: Pulsando el botón *Modelica text* se muestra la ventana de edición de código Modelica, con el código Modelica del modelo. Se ha tecleado la descripción en Modelica del modelo, que es la mostrada en Código 2.1. Para facilitar al usuario la inspección del código, Dymola permite ocultar una parte de él, como puede ser la declaración de componentes y las anotaciones (esto puede configurarse en *Edit/Options*). Para indicar a Dymola que debe mostrar el código Modelica completo, debe pulsarse con el botón derecho sobre la ventana de edición de código y seleccionar *Expand*.

Pulsando el botón *Check*, Dymola comprueba que el modelo no tenga errores y colorea la sintaxis del código. Además, muestra en la ventana de mensajes el número de incógnitas y de ecuaciones. Este modelo tiene dos incógnitas: las variables reales de tiempo continuo ϕ y w . Para que un modelo descrito mediante ecuaciones pueda ser simulado, es condición necesaria (pero no suficiente, como se verá en el Tema 3) que tenga el mismo número de ecuaciones que de incógnitas, ya que cada incógnita debe ser evaluada de una ecuación y cada ecuación debe emplearse para evaluar una incógnita.

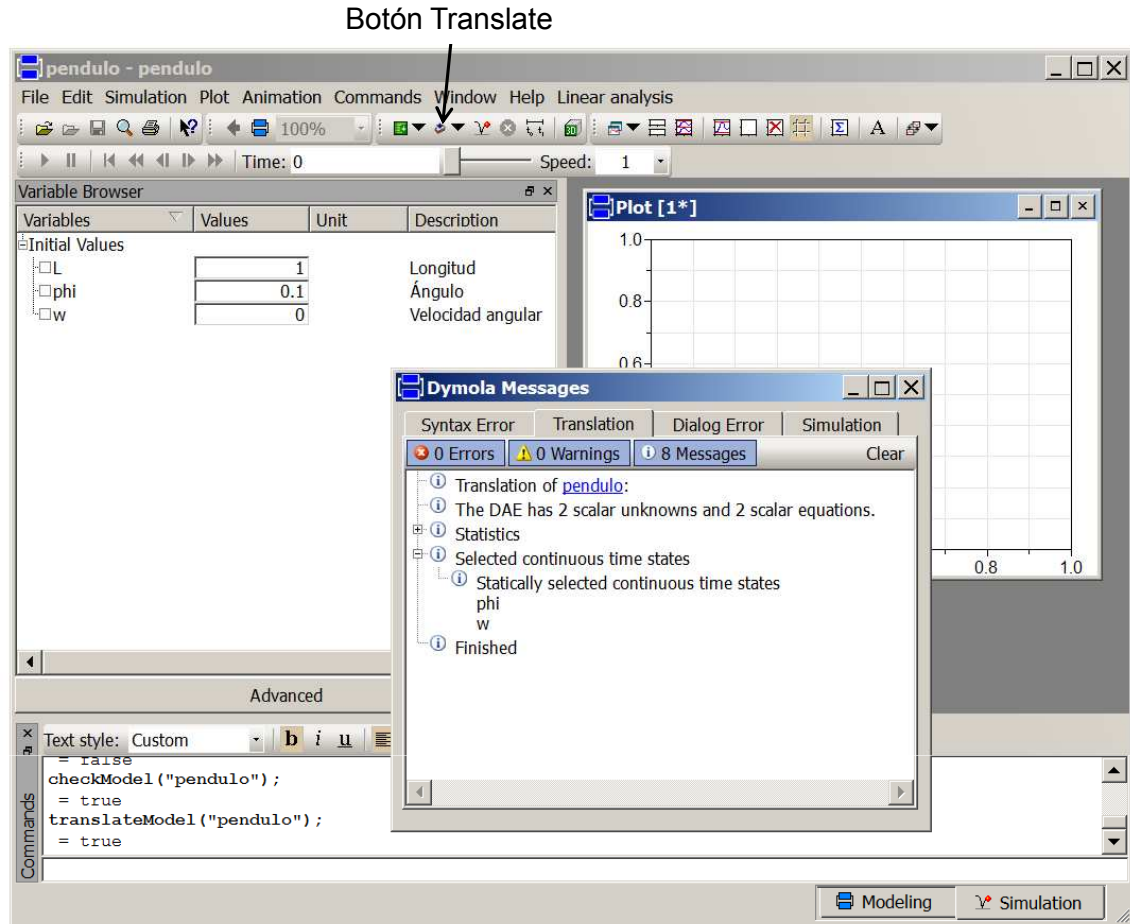


Figura 2.6: Cuando el usuario pulsa el botón *Translate*, Dymola realiza la traducción del modelo seleccionado en la ventana de modelado. Obsérvese que el nombre de dicho modelo, *pendulo* en este caso, se muestra como título de la ventana. Entre los ficheros generados en la traducción, cabe destacar *dymosim.exe*, que es el código ejecutable del algoritmo de la simulación, y *dsin.txt*, que contiene la definición del experimento. Ésta incluye la información necesaria para inicializar el modelo, el instante inicial y final de la simulación, información sobre los métodos numéricos, la lista completa de variables del modelo, etc. Dymola guarda estos ficheros en su directorio de trabajo, el cual puede modificarse mediante *File/Change directory*.

En la ventana de mensajes se muestra información referente a la traducción. Por ejemplo, Dymola indica qué variables de tiempo continuo ha seleccionado como variables de estado. En este caso, las variables de estado son *phi* y *w*.

En la ventana de la izquierda, llamada *Variable Browser*, se muestran las variables a las que el usuario debe asignar valor para inicializar el modelo. Los valores mostrados son los indicados en el modelo, ofreciéndose al usuario la posibilidad de modificarlos. Si el usuario los modifica, Dymola actualizará con los nuevos valores el fichero *dsin.txt*.

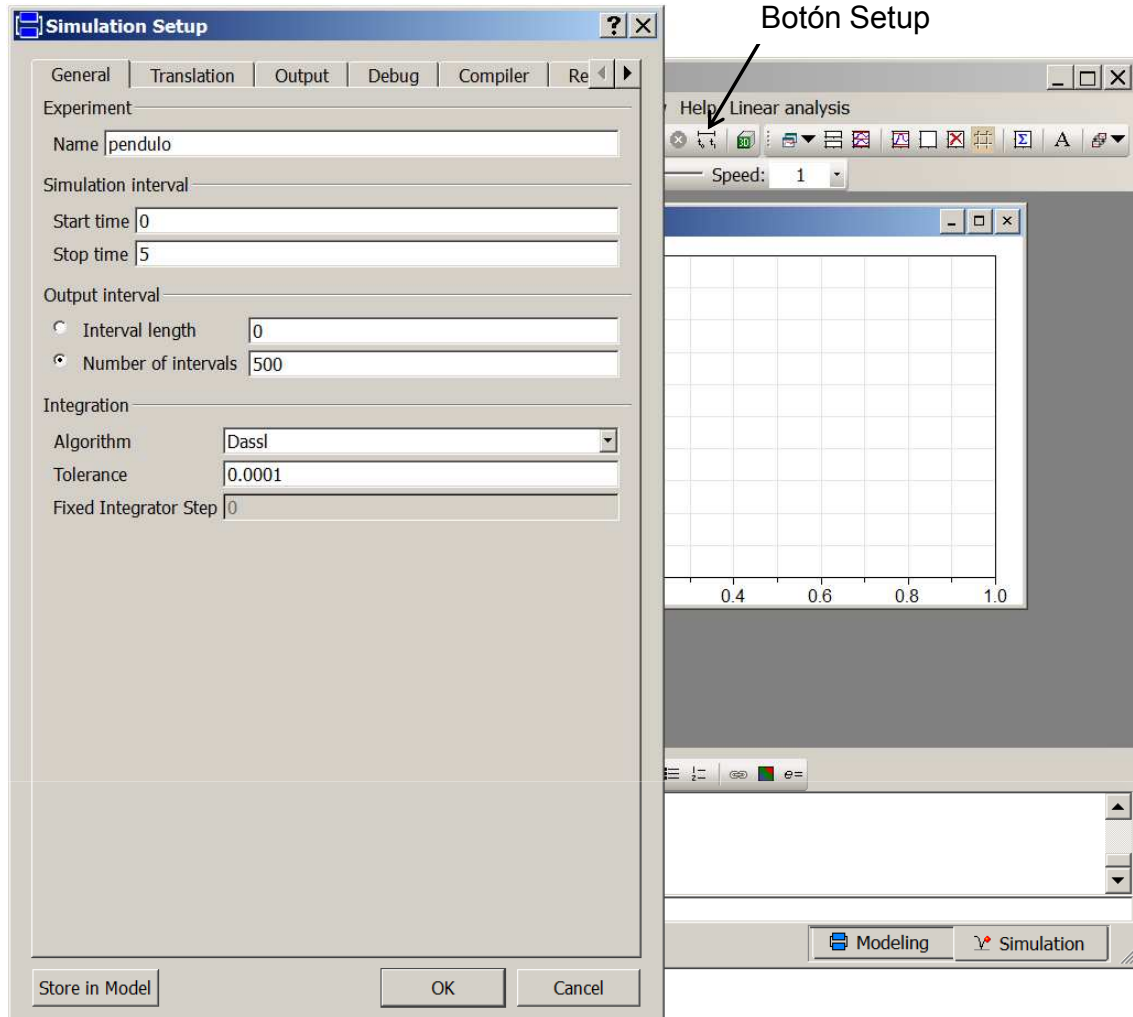


Figura 2.7: En la lengüeta *General* puede indicarse el instante inicial y final de la simulación, y la longitud o el número de intervalos de comunicación. También puede seleccionarse el algoritmo de integración y su tolerancia.

En la lengüeta *Translation* puede indicarse a Dymola que genere información adicional durante la traducción. Por ejemplo, puede solicitarse que genere un fichero que contenga el modelo plano con la causalidad computacional señalada. Esto resulta en ocasiones útil para depurar el modelo.

En la lengüeta *Output* puede indicarse qué variables debe almacenar Dymola durante la simulación. El valor de las variables seleccionadas, que se denominan variables de salida, se guarda en los instantes de comunicación y también en los instantes de los eventos.

En la lengüeta *Debug* se puede indicar a Dymola que escriba durante la simulación, en la ventana de mensajes, información sobre la solución numérica del modelo. Esto es útil para analizar los errores numéricos producidos al simular el modelo.

En la lengüeta *Realtime* puede indicarse cómo debe sincronizarse, durante la ejecución de la simulación, el tiempo simulado con el tiempo real. Si no se indica que la simulación deba sincronizarse con el tiempo real, Dymola ejecuta la simulación lo más rápido posible.

Si se pulsa el botón *Store in Model*, Dymola añade al código Modelica del modelo una anotación con la información del experimento.

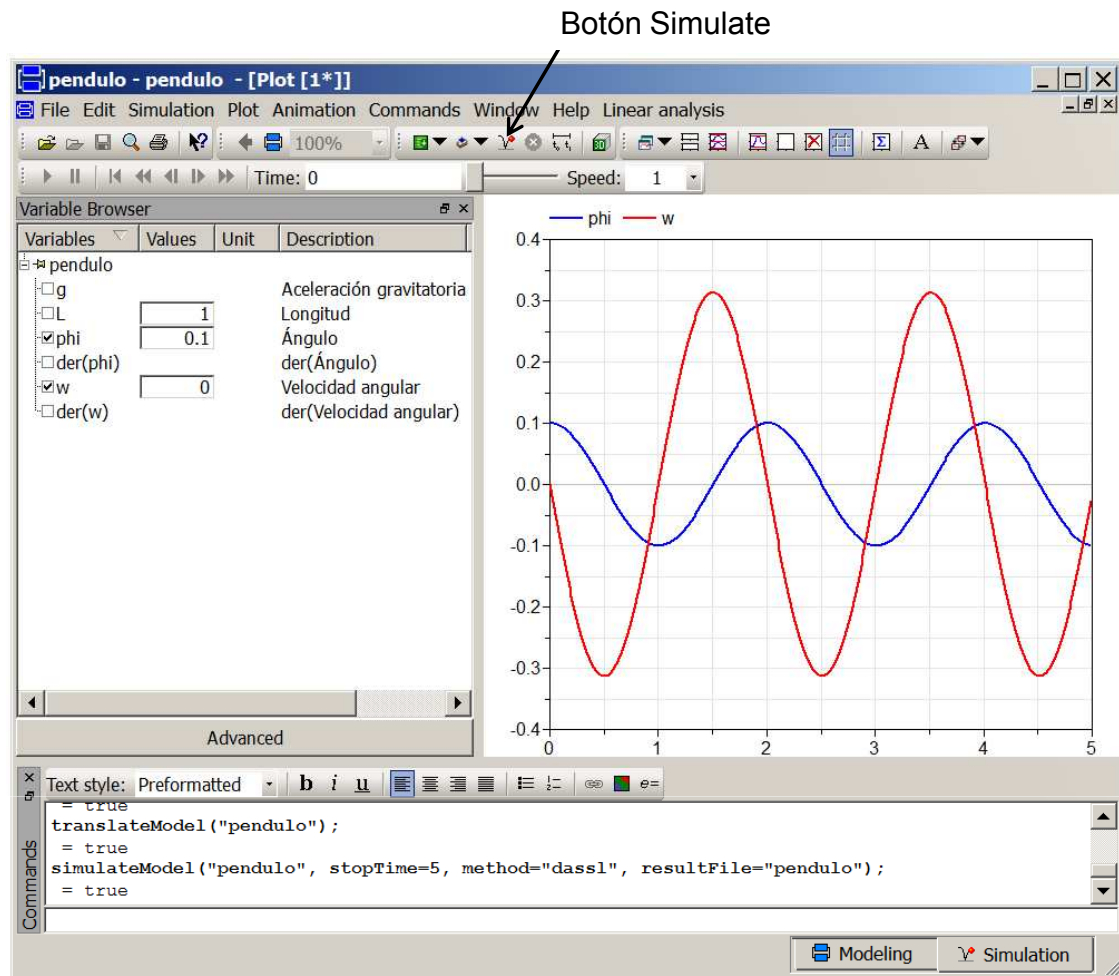


Figura 2.8: Cuando el usuario pulsa el botón *Simulate*, Dymola ejecuta el fichero *dymosim.exe* pasándole como argumento el fichero *dsin.txt*.

Si se produce error durante la simulación, Dymola lo indica en la ventana de mensajes. En caso contrario, muestra estadísticas sobre la ejecución del algoritmo de integración y los métodos iterativos para la resolución de los sistemas simultáneos de ecuaciones, y también indica el número de eventos en el tiempo y en el estado ejecutados durante la simulación. Esta información es en ocasiones útil para saber qué parte de la resolución numérica del modelo está consumiendo mayor tiempo de CPU.

Dymola muestra en la ventana *Variable Browser* las variables de salida. Señalando una o varias de estas variables, se obtiene su representación gráfica frente al tiempo. Si se pulsa sobre una variable y a continuación se pulsa el botón derecho del ratón, aparece un menú en el cual puede indicarse que esa, en lugar del tiempo, sea la variable independiente en los gráficos.

Dymola almacena los resultados de la simulación en un fichero binario en el formato de Matlab, cuyo nombre coincide con el nombre del modelo y tiene extensión *.mat*. Si en la lengüeta *Output* de la ventana *Simulation Setup* se señala *Textual data format*, también se almacenan los resultados en un fichero de texto con extensión *.txt*. Dymola guarda estos ficheros en su directorio de trabajo.

A continuación vamos a describir los conceptos básicos para el manejo del entorno de modelado **OpenModelica**, de manera análoga a como se describió Dymola. Para ello vamos a explicar los pasos necesarios para editar, comprobar y simular el ejemplo del péndulo mostrado en Código 2.1, que fue escogido para describir Dymola, empleando ahora el entorno OpenModelica.

El entorno OpenModelica ha sido desarrollado por el Open Source Modelica Consortium (OSMC), se puede descargar gratuitamente y existen versiones para Windows, Linux y Mac. El núcleo de este entorno es el compilador de código Modelica, llamado OpenModelica Compiler, que convierte el código Modelica a código C. El entorno tiene, además del compilador de código Modelica, varias herramientas que sirven de interfaz entre el compilador y el usuario. Las herramientas son el editor OpenModelica Connection Editor (OMEdit), el Interactive OpenModelica Shell (OMShell), OpenModelica Notebook (OMNotebook), DrControl, OpenModelica Equation Model Debugger, OMOptim, Modelica Development Tooling (MDT), OpenModelica Python Interface (OMPython). Nosotros vamos a emplear como interfaz de usuario el OpenModelica Connection Editor (OMEdit).

Una vez instalado el entorno OpenModelica en un ordenador con sistema operativo Windows, para abrir OMEdit simplemente hay que ejecutar OMEdit.exe, que se encuentra en el directorio *bin* de la instalación. Una vez ejecutado, se abre la ventana mostrada en la Figura 2.9, que tiene tres pestañas llamadas *Welcome*, *Modeling* y *Plotting* (véase la parte inferior derecha de la ventana).

- En la **pestaña Welcome** se muestran las últimas noticias sobre OpenModelica y los ficheros de Modelica que han sido abiertos más recientemente.
- La **pestaña Modeling** permite la edición de modelos.
- La **pestaña Plotting** permite visualizar los resultados de la simulación.

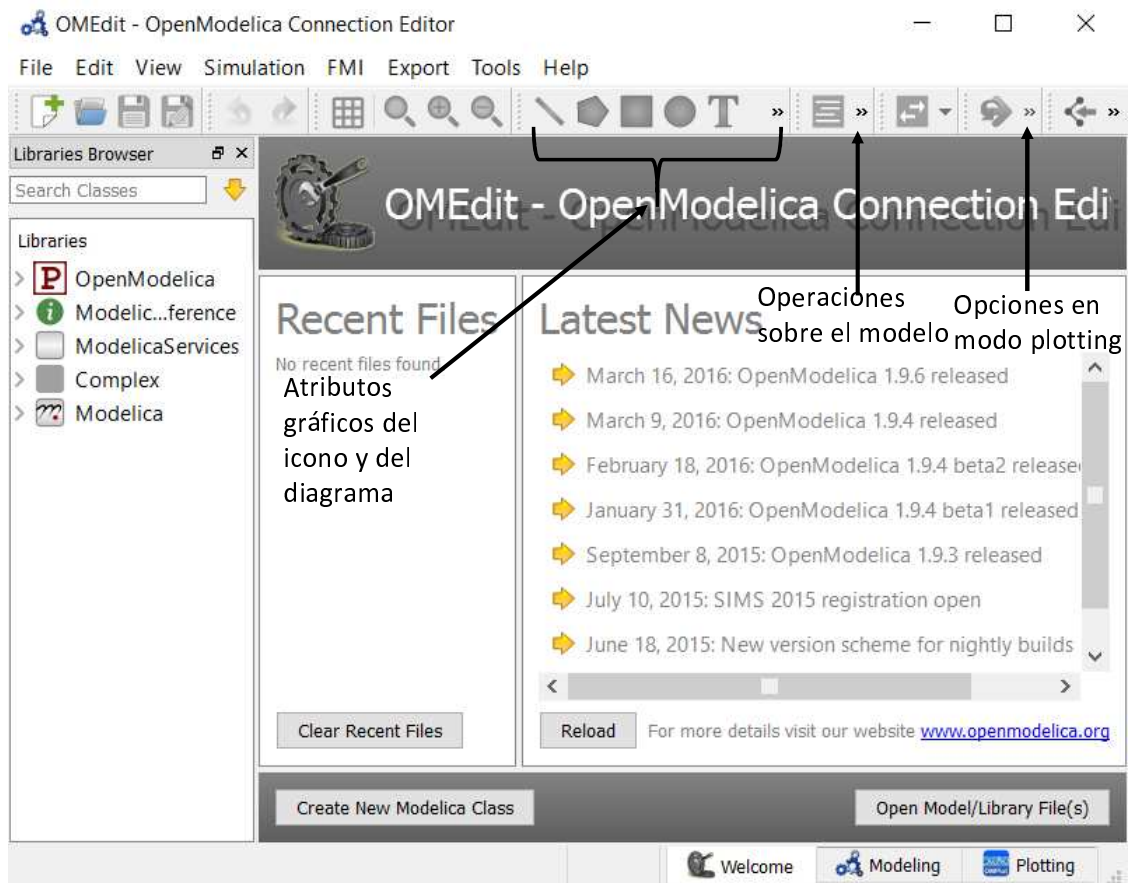


Figura 2.9: Ventana obtenida al ejecutar OMEdit.exe. La ventana está dividida en tres áreas: área para navegar en las librerías de modelos (ventana *Libraries Browser*, parte superior izquierda), área donde se muestran los ficheros más recientes (ventana *Recent Files*, parte central) y área donde se muestran las últimas noticias sobre OpenModelica (parte derecha). Aparecen varias librerías cargadas por defecto, entre ellas la *Modelica Standard Library* (MSL). En la parte superior hay una regleta con botones, los cuales permiten abrir modelos guardados en fichero, salvar modelos a fichero, definir atributos gráficos, realizar operaciones sobre el modelo como comprobar (*check*) sintácticamente los modelos, simular el modelo, etc.

A continuación se describen brevemente los pasos para editar, comprobar y simular el ejemplo del péndulo con OMEdit.

1. *Creación de un nuevo modelo.* Para definir un nuevo modelo hay que seleccionar *File / New Modelica Class*, y a continuación asignar un nombre al modelo. En este caso, el modelo se llamará *pendulo* (véase la Figura 2.10).
2. *Escritura del código Modelica del modelo.* Pulsando el botón *Text View* (véase la Figura 2.11), aparece una ventana en la cual puede teclearse la descripción Modelica del nuevo modelo. La descripción en Modelica del modelo del péndulo se muestra en el Código 2.1.
3. *Definición del experimento.* Pulsando el botón *Simulation Setup* (véase la Figura 2.11), se abre la ventana *Simulation Setup* (véase la Figura 2.12). En la lengüeta *General*, el usuario puede indicar el instante inicial y final de la simulación, cada cuánto tiempo o el número de veces en que OMEdit debe almacenar el valor de las variables de salida, y también puede seleccionar el método de integración y su tolerancia. Asignamos al instante final de la simulación el valor 5.
4. *Ejecución de la simulación y análisis de los resultados.* La simulación se inicia pulsando el botón *Simulate* (véase la Figura 2.11). OMEdit muestra en la ventana *Variables Browser* las variables del modelo (véase la Figura 2.13). Pulsando sobre ellas se obtiene su representación gráfica. También se guardan los resultados en fichero.

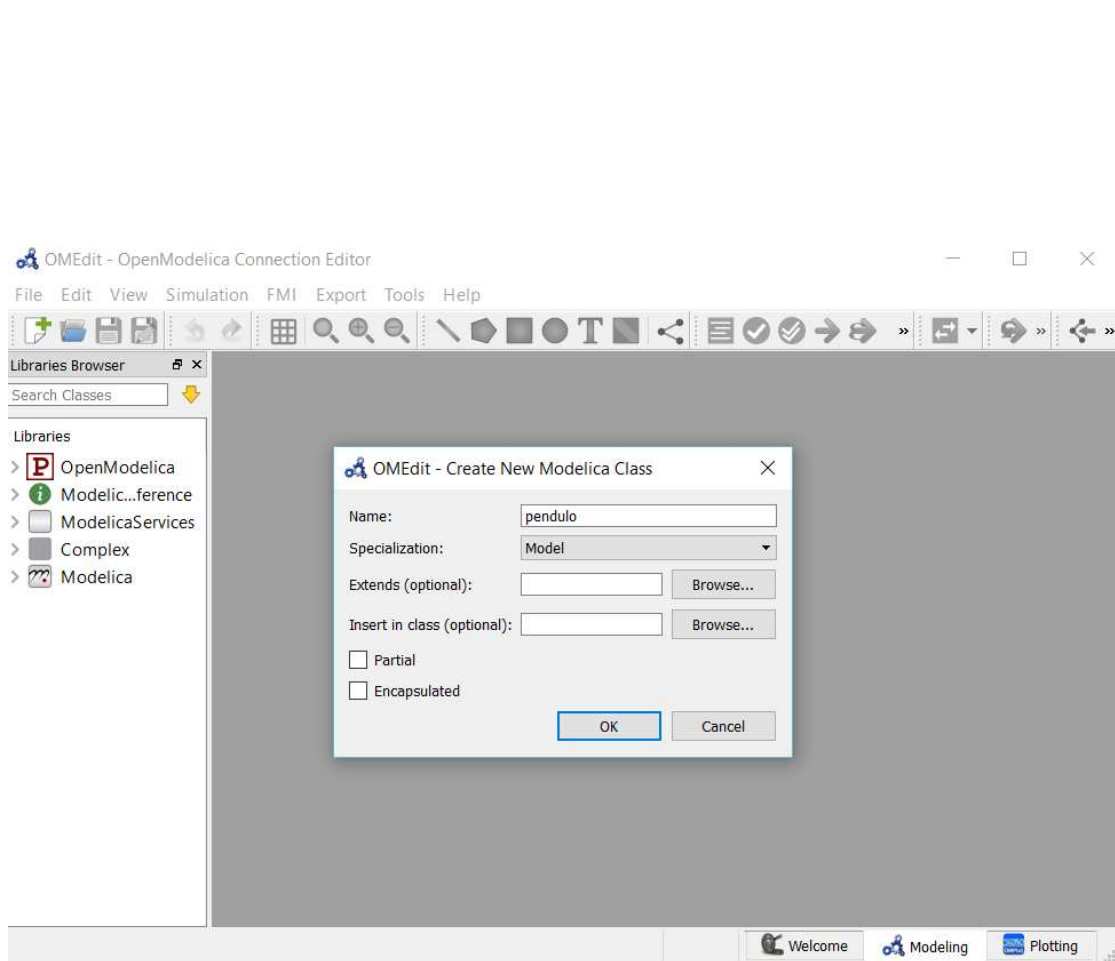


Figura 2.10: Puede definirse una nueva clase pulsando *File /New Modelica Class*. Al hacerlo, se abre una ventana como la mostrada en la figura. En la casilla *Name* se debe indicar el nombre de la nueva clase. En la casilla *Specialization* se debe indicar el tipo de la clase (model, package, record, type, etc.) Señalando el cuadro *Partial* se indica que esta nueva clase podrá ser heredada, pero no instanciada, es decir, se indica que se trata de una clase parcial. Si el nuevo modelo hereda de otros modelos, en la casilla *Extends* pueden indicarse las superclases. Asimismo, si el nuevo modelo debe ser parte de una librería ya existente, en la casilla *Insert in class* puede indicarse dónde debe incluirse.

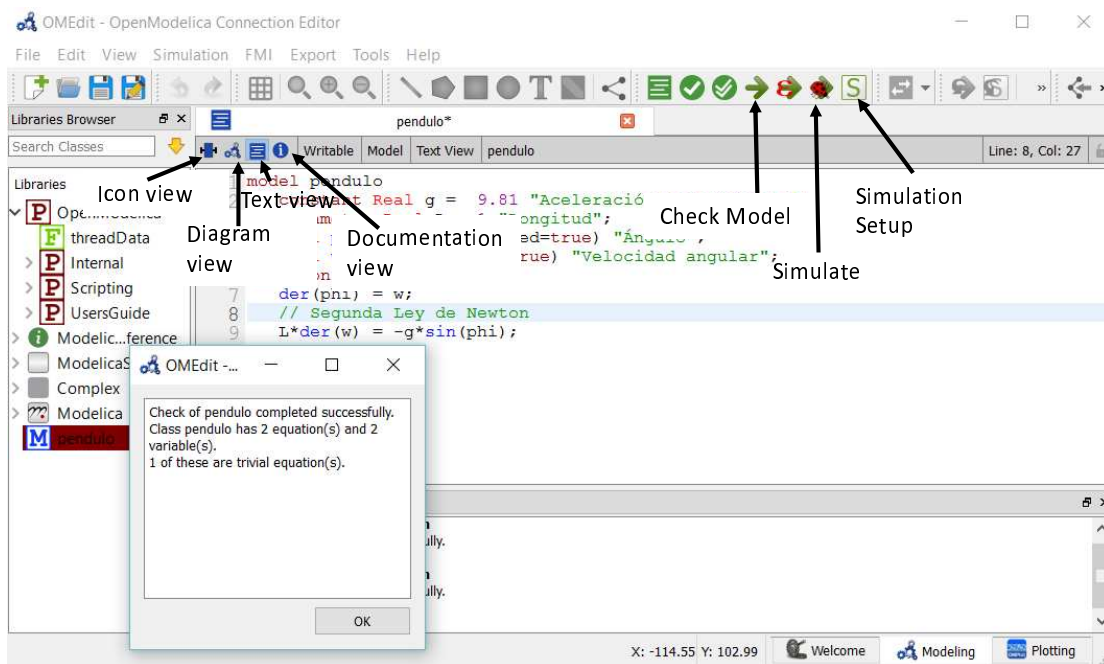


Figura 2.11: Pulsando el botón *Text View* se muestra la ventana de edición de código Modelica. En esta ventana se teclea la descripción en Modelica del modelo, que es la mostrada en el Código 2.1. En esta ventana se puede mostrar el icono, el diagrama del modelo o su documentación pulsando, respectivamente a los botones *Icon View*, *Diagram View* o *Documentation View*. Pulsando el botón *Check Model*, OpenModelica comprueba que el modelo no tenga errores. Además, muestra una ventana informando del número de variables y de ecuaciones del modelo.

OMEdit - Simulation Setup - pendulo

Simulation Setup - pendulo

General Output Simulation Flags Archived Simulations

Simulation Interval

Start Time: 0

Stop Time: 1

☒ Number of Intervals: 500

☐ Interval: 0.002

Integration

Method: dassl

Tolerance: 1e-6

DASSL Options

Jacobian: Colored Numerical

☒ Root Finding

☒ Restart After Event

Initial Step Size:

Maximum Step Size:

Maximum Integration Order: 5

Compiler Flags (Optional):

Number of Processors: 8 Use 1 processor if you encounter problems during compilation.

☐ Build Only

☐ Launch Transformational Debugger

☐ Launch Algorithmic Debugger

☐ Save simulation settings inside model

Simulate Cancel

Figura 2.12: En la lengüeta *General* puede indicarse el instante inicial y final de la simulación, y la longitud o el número de intervalos de comunicación. También puede seleccionarse el algoritmo de integración y su tolerancia.

En la lengüeta *Output* puede indicarse el formato del fichero donde se van a almacenar los resultados. También se puede seleccionar si se quieren almacenar las variables de tipo *protected*, si se quiere almacenar ó no el valor de las variables en los instantes de tiempo de los eventos y si se quiere almacenar ó no las variables en intervalos de tiempo equidistantes.

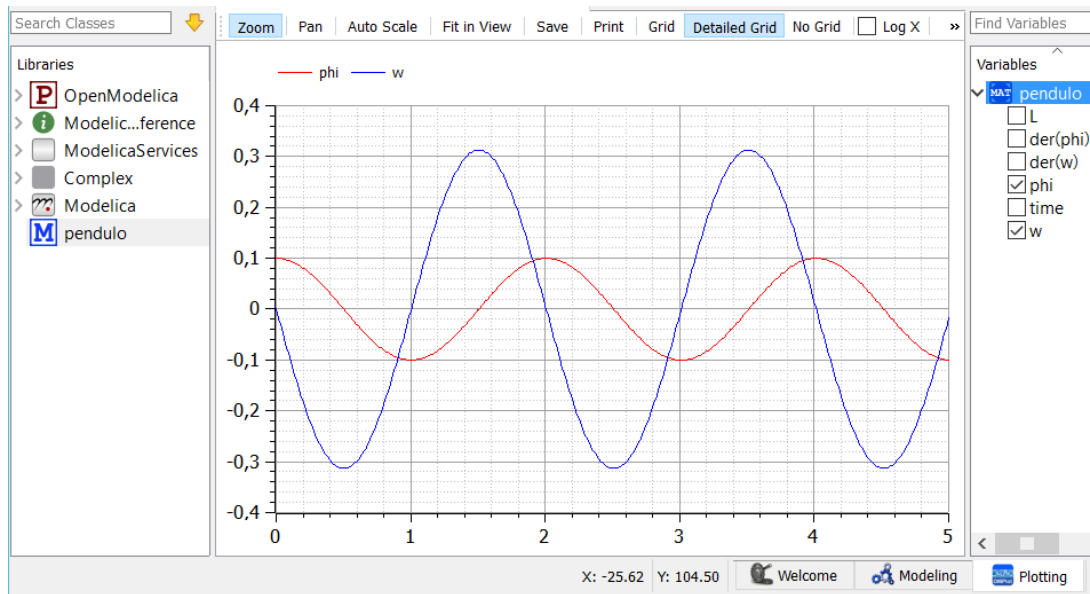


Figura 2.13: Cuando el usuario pulsa el botón *Simulate*, se realiza la simulación del modelo. Si se produce error durante la simulación, OMEdit lo indica en la ventana de mensajes.

OMEdit muestra en la ventana *Variables Browser* las variables de salida. Señalando una o varias de estas variables, se obtiene su representación gráfica frente al tiempo. OMEdit almacena los resultados de la simulación en un fichero binario en el formato de Matlab, cuyo nombre coincide con el nombre del modelo añadiendo el sufijo *_res* y la extensión *.mat*. En este caso, el nombre del fichero es *pendulo_res.mat*. OMEdit guarda el fichero de salida en su directorio de trabajo. El directorio de trabajo se puede cambiar en la ventana *Options* que se muestra al pulsar *Tools / Options*.

2.3. FUNDAMENTOS DEL MODELADO DE SISTEMAS FÍSICOS

La metodología para el modelado de sistemas físicos descrita en la sección anterior, y que es facilitada por los lenguajes de modelado basados en ecuaciones, consiste básicamente en describir el modelo siguiendo los tres pasos siguientes:

1. Definir la estructura del sistema y dividir el sistema en sus componentes.
2. Definir la interacción entre los componentes en términos de variables que son agrupadas en conectores, y clasificadas en cada conector como across y through dependiendo de que deban ser igualadas en el punto de conexión o que su suma en el punto de conexión deba ser igualada a cero.
3. Definir el comportamiento interno de cada componente, independientemente de los demás, en términos de balances de materia, energía y momento, y de relaciones constitutivas.

En esta sección se realiza una primera aproximación al modelado de sistemas físicos aplicando la metodología anterior. Primeramente se aborda el modelado de la interacción entre los componentes, lo cual implica seleccionar las variables de los conectores. A continuación, se ofrece una introducción al modelado de algunos fenómenos físicos.

Una característica común de los sistemas en los diferentes dominios es que la energía fluye a través de ellos, transformándose y almacenándose. Al describir el comportamiento de los sistemas desde el punto de vista de la energía, se hacen patentes las analogías entre los diferentes dominios, empleándose conceptos comunes tales como fuente de energía, comportamiento resistivo y almacenador, transformadores y giradores, etc. Estos conceptos, útiles para plantear hipótesis sobre el comportamiento de los sistemas, pueden representarse mediante **componentes primitivos**. En general, el modelo de un sistema no será un único concepto, sino que consistirá en un conjunto de conceptos interconectados que pueden ser representados mediante la conexión de un conjunto de componentes primitivos.

2.3.1. Interacción entre los componentes

El motivo por el cual cada variable de un conector debe declararse como de tipo across o de tipo through es que eso permite que, si se escogen adecuadamente las variables, la conexión entre los componentes satisfaga las leyes de conservación de la

Tabla 2.2: Variables de los conectores en diferentes dominios, seleccionadas de manera que el producto (*across* $\times$ *through*) tenga unidades de potencia.

Dominio	Across	Through
Eléctrico	Voltaje (V)	Corriente (A)
Mecánica traslación	Velocidad (m/s)	Fuerza (N)
Mecánica rotación	Velocidad angular (rad/s)	Torque (N·m)
Hidráulica	Presión (N/m ²)	Caudal volumétrico (m ³ /s)
Térmico	Temperatura (K)	Flujo de entropía (W/K)
Químico	Potencial químico (J/mol)	Flujo molar (mol/s)

materia, la energía y el momento. Conviene no obstante indicar que esta clasificación de las variables en across y through es una hipótesis de modelado (cuya validez depende del objetivo del estudio), no es una propiedad intrínseca de las magnitudes de la física.

En la Tabla 2.2 se muestra una de las posibles formas de escoger las variables de los conectores de modo que el producto de la variable across y through del conector tenga unidades de potencia. Mediante esta selección se garantiza que al conectar varios conectores del mismo tipo se satisfará la **ley de conservación de la energía**.

Supongamos que se desea modelar un circuito eléctrico compuesto por la conexión de componentes tales como generadores, resistencias, condensadores, inducciones, diodos, transistores, etc. Los terminales de conexión de los componentes electrónicos se denominan **pin**es. Cuando dos o más pines se conectan entre sí, el punto de conexión se denomina **nodo** del circuito. Consideremos un nodo de conexión entre cuatro pines de otros tantos componentes, a los que se ha llamado C_1 , C_2 , C_3 y C_4 . La conexión se muestra en la Figura 2.14. Los pines se han representado mediante rectángulos rellenos.

Seleccionando las variables como se indica en la Tabla 2.2, el pin se modela mediante un conector compuesto de dos variables: el voltaje y la corriente. La primera es una variable across del conector y la segunda una variable through. Por lo general se aplica el siguiente criterio de signos para las variables through: *se consideran positivas cuando tienen sentido entrante al componente*. Así pues, cada pin es modelado mediante dos variables: el voltaje en ese punto (u) y la corriente eléctrica entrante al componente (i).

La descripción mediante un lenguaje de modelado de la conexión entre estos cuatro pines sería traducida automáticamente por el entorno de modelado en cuatro

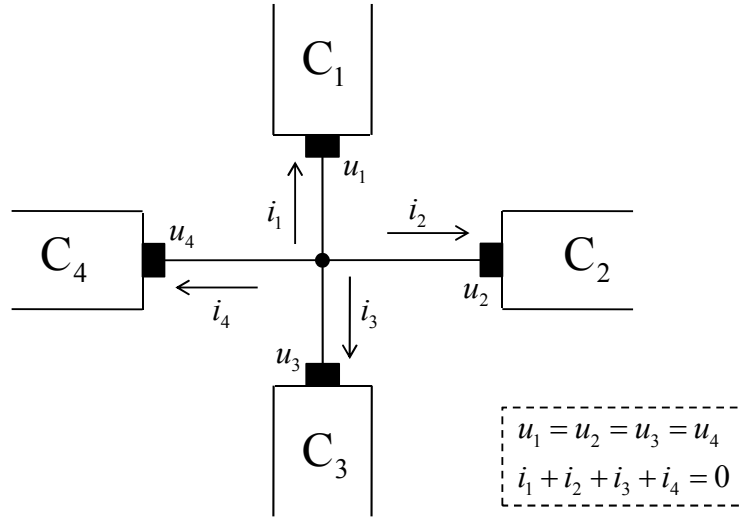


Figura 2.14: Conexión entre cuatro componentes (C_1 , C_2 , C_3 y C_4). Los conectores (rectángulos rellenos) están compuestos por una variable across (u) y una variable through (i).

ecuaciones: tres ecuaciones estableciendo la igualdad entre las variables across ($u_1 = u_2 = u_3 = u_4$) y una ecuación estableciendo que la suma de las variables through es igual a cero ($i_1 + i_2 + i_3 + i_4 = 0$).

En la Tabla 2.2 se indican posibles selecciones de las variables de los conectores en otros dominios. Obsérvese que las citadas variables across corresponden a magnitudes físicas que determinan si existe equilibrio, mientras que las variables through corresponden a magnitudes físicas que establecen el equilibrio.

Existen otras formas de seleccionar las variables de los conectores de manera que la conexión garantice la conservación de la energía. Por ejemplo, en el dominio térmico es más frecuente emplear el flujo de calor como variable through, en lugar del flujo de entropía. El flujo de calor tiene unidades de potencia, con lo cual la ecuación para la conexión para esa variable through describe la conservación de la energía.

2.3.2. Condiciones de contorno

La energía en un sistema cerrado se conserva, pudiéndose únicamente almacenar, transportar y transformar. Así pues, el concepto de **fuentes de energía** surge al describir las **condiciones de contorno** del sistema bajo estudio, es decir, la influencia del **entorno** (de todo aquello no perteneciente a nuestro sistema) sobre nuestro sistema. Una fuente puede actuar cediendo energía a nuestro sistema y

también extrayendo energía de él. Asimismo, el flujo de energía proporcionado por la fuente puede depender del estado de nuestro sistema, o bien puede ser independiente de él. En el primer caso se trata de una **fente modulada** y en el segundo de una **fente ideal**.

Un tipo de fuente que se encuentra muy frecuentemente en los modelos es aquella que impone el valor de determinada magnitud del conector. Estas fuentes se clasifican en **fuentes de esfuerzo** y **fuentes de flujo** dependiendo de que dicha magnitud fijada por la fuente sea variable across o through del conector.

Por ejemplo, si se escoge el conector en el dominio eléctrico de manera que la tensión sea variable across y la corriente sea through, entonces una fuente de esfuerzo impondría el valor de la caída de tensión entre los pines y una fuente de flujo impondría la corriente en los pines. Análogamente, si el conector del dominio hidráulico se escoge de manera que la presión es variable across y el caudal volumétrico es variable through, entonces la fuente de esfuerzo impondría la caída de presión entre sus conectores y la fuente de flujo el caudal volumétrico.

2.3.3. Disipación de la energía

Como se ha indicado anteriormente, la energía puede transportarse, almacenarse y convertirse. Una forma de conversión de la energía es la disipación en forma de calor. Así, cuando fluye corriente eléctrica a través de una resistencia o cuando se produce fricción, parte de la energía eléctrica o mecánica se convierte de manera irreversible en calor. Este fenómeno puede describirse empleando componentes primitivos. Si se seleccionan las variables de los conectores como se indica en la Tabla 2.2, la relación constitutiva de estos componentes es una relación algebraica, lineal o no lineal, entre las variables across y through de sus conectores. Por analogía con la resistencia eléctrica, estos componentes se denominan **componentes resistivos**.

- Un componente primitivo resistencia eléctrica describe la relación algebraica existente entre la caída de tensión y la corriente.
- Una resistencia hidráulica describe la relación algebraica entre el caudal volumétrico y la caída de presión debida a la fricción.
- Una resistencia en el dominio mecánico describe la relación algebraica entre las velocidades relativas de dos cuerpos y la fuerza de fricción entre ellos.

2.3.4. Intercambio de calor

Si en el dominio térmico se selecciona la temperatura como variable across del conector y el flujo de calor como variable through, el componente resistivo describiría el flujo de calor que se establece debido a la diferencia de temperaturas, ya sea por **conducción**, **convección** o **radiación**. La relación constitutiva que describe estos mecanismos de **transporte de energía** es una relación algebraica entre las temperaturas y el flujo de calor. Se trata por tanto de un **componente resistivo**.

2.3.5. Almacenamiento de la energía

Los diferentes fenómenos de almacenamiento de la energía pueden describirse mediante componentes primitivos **almacenadores de energía**. Dos componentes básicos almacenadores en el dominio eléctrico son el condensador y la inducción. Obsérvese que, si se escogen las variables de los conectores como se indica en la Tabla 2.2, en la relación constitutiva de estos componentes interviene la derivada de la magnitud across o through.

En el **dominio eléctrico**, las relaciones constitutivas del condensador y la inducción eléctricas son, respectivamente:

$$C \cdot \frac{du}{dt} = i \quad (2.1)$$

$$L \cdot \frac{di}{dt} = u \quad (2.2)$$

De forma análoga pueden modelarse fenómenos de almacenamiento de energía en otros dominios. En el **dominio mecánico**, se almacena energía cinética en un objeto puntual de masa m constante, sobre el que se aplica una fuerza F (ley de Newton):

$$m \cdot \frac{dv}{dt} = F \quad (2.3)$$

y se almacena energía potencial en un muelle al comprimirlo:

$$\frac{1}{k} \cdot \frac{dF}{dt} = v \quad (2.4)$$

La expresión anterior se obtiene de la ley de Hooke, según la cual la fuerza ejercida por un muelle es proporcional a la diferencia entre su elongación (x) y su elongación natural (x_0):

$$F = k \cdot (x - x_0) \quad (2.5)$$

Obsérvese que la Ec. (2.4) se obtiene derivando respecto al tiempo la Ec. (2.5), asumiendo que el factor de proporcionalidad k es constante.

En el **dominio hidráulico**, la presión (p) en la base de un depósito es función del caudal volumétrico (Q) entrante al depósito:

$$\frac{A}{\rho \cdot g} \cdot \frac{dp}{dt} = Q \quad (2.6)$$

donde A es el área de la base del depósito, ρ es la densidad del líquido y g es la aceleración gravitatoria. El cambio en la presión es debido a la variación en el volumen de líquido acumulado, de la misma manera que en el condensador eléctrico la variación en la carga eléctrica acumulada produce una variación en la diferencia de potencial entre las armaduras del condensador.

La Ec. (2.6) puede deducirse teniendo en cuenta que la relación entre la presión (p) en la base del depósito y el volumen de líquido (V) en el depósito es:

$$p = \frac{\rho \cdot V \cdot g}{A} \quad (2.7)$$

y que la variación en el volumen de líquido (V) almacenado es igual al caudal volumétrico (Q):

$$\frac{dV}{dt} = Q \quad (2.8)$$

Derivando respecto al tiempo la Ec. (2.7) asumiendo que ρ y A son constantes y sustituyendo la Ec. (2.8), se obtiene la Ec. (2.6).

Asimismo, el líquido que circula por una conducción también almacena energía cinética. La variación en el caudal volumétrico del fluido que circula por una tubería es función de la diferencia de las presiones aplicadas en los extremos de la tubería (p):

$$\frac{\rho \cdot L}{S} \cdot \frac{dQ}{dt} = p \quad (2.9)$$

donde L es la longitud de la tubería y S el área de la sección de la tubería. Esta relación constitutiva se obtiene de aplicar la ley de Newton al líquido contenido en la tubería, teniendo en cuenta que la fuerza aplicada sobre él es igual a $p \cdot S$ y que su masa es igual a $\rho \cdot L \cdot S$:

$$\rho \cdot L \cdot S \cdot \frac{dv}{dt} = p \cdot S \quad (2.10)$$

Teniendo en cuenta que el flujo volumétrico está relacionado con la velocidad del líquido (v) y la sección de la tubería:

$$Q = v \cdot S \quad (2.11)$$

sustituyendo la Ec. (2.11) en la Ec. (2.10), se obtiene la Ec. (2.9).

En el **dominio térmico**, se produce acumulación de energía cuando la materia absorbe calor (Q), aumentando por ello su temperatura (T). Asumiendo que la masa (m), y las capacidades caloríficas a presión (C_p) y a volumen constante (C_v) son constantes, la relación constitutiva de este elemento almacenador es, en el caso de un gas:

$$m \cdot C_v \cdot \frac{dT}{dt} = Q \quad (2.12)$$

y en el caso de un sólido o un líquido:

$$m \cdot C_p \cdot \frac{dT}{dt} = Q \quad (2.13)$$

2.3.6. Conversión reversible de la energía

Los componentes primitivos considerados hasta este punto permiten describir condiciones de contorno, y almacenamiento, disipación y transporte de energía. Queda por considerar el modelado de la conversión reversible de la energía.

La conversión puede producirse entre formas idénticas de energía o entre dos formas distintas de energía. A continuación se muestran algunos ejemplos de fenómenos reversibles de conversión de energía.

- El transformador eléctrico, la palanca mecánica o dos ruedas dentadas que giran engranadas realizan conversión entre formas idénticas de energía.
- Un pistón convierte potencia mecánica (fuerza $\times$ velocidad) en potencia hidráulica (presión $\times$ caudal volumétrico).
- Un tacómetro convierte la potencia mecánica de rotación en potencia eléctrica.
- Una bomba transforma la potencia mecánica de rotación en potencia hidráulica.
- Una varilla dentada engranada a una rueda dentada convierte potencia mecánica de traslación en potencia mecánica de rotación.

El modelado de los conceptos anteriores puede realizarse empleando componentes con dos conectores, de manera que cada uno de ellos describa una forma de energía. Puesto que en el componente no se produce, almacena o disipa energía, deberá satisfacerse en todo momento que la potencia que entra por un conector sale por el otro.

Si se escogen las variables de los conectores de manera que el producto de la variable across y through de cada conector tenga unidades de potencia (por ejemplo, como se indica en la Tabla 2.2), entonces las relaciones constitutivas del componente deberán ser tales que se satisfaga:

$$e_1 \cdot f_1 = e_2 \cdot f_2 \quad (2.14)$$

donde e_1 y e_2 son las variables across del primer y segundo conector, y f_1 y f_2 las variables through.

El modelo más sencillo es aquel en el cual se establece una relación lineal entre las variables de los conectores. Dependiendo de si la relación lineal se establece entre las parejas $\{e_1 - e_2, f_1 - f_2\}$ o entre las parejas $\{e_1 - f_2, f_1 - e_2\}$, se dice que se trata de un elemento transformador o girador.

- Los elementos **transformadores** son aquellos que satisfacen:

$$e_1 = m \cdot e_2 \quad (2.15)$$

$$m \cdot f_1 = f_2 \quad (2.16)$$

con lo cual, en todo momento se verifica $e_1 \cdot f_1 = e_2 \cdot f_2$. Ejemplos de transformadores son una palanca mecánica, ruedas dentadas que giran engranadas, el transformador eléctrico, etc.

- Los elementos **giradores** son aquellos que satisfacen:

$$e_1 = r \cdot f_2 \quad (2.17)$$

$$r \cdot f_1 = e_2 \quad (2.18)$$

con lo cual, en todo momento se verifica $e_1 \cdot f_1 = e_2 \cdot f_2$. Un ejemplo de girador mecánico es un giróscopo, en el cual la fuerza en uno de los conectores es proporcional a la velocidad en el otro.

2.4. CIRCUITOS ELÉCTRICOS

En esta sección veremos cómo pueden aplicarse los conceptos anteriores al modelado de circuitos eléctricos usando el lenguaje Modelica.

2.4.1. Ejemplo 1: circuito rectificador

Consideremos el circuito mostrado en la Figura 2.15. Está compuesto por un generador sinusoidal de tensión, dos resistencias, un condensador y un diodo.

Los componentes electrónicos que componen el circuito real pueden modelarse con diferente nivel de detalle, empleando diferentes tipos de modelo matemático. En este caso describiremos los componentes empleando las relaciones constitutivas mostradas en la Figura 2.16. El generador, las resistencias y el condensador se consideran componentes ideales. El diodo es descrito empleando el modelo de Shockley. El valor de los parámetros para el circuito en concreto que se desea modelar se muestra en la Tabla 2.3. Las ecuaciones del modelo pueden plantearse siguiendo el procedimiento descrito a continuación.

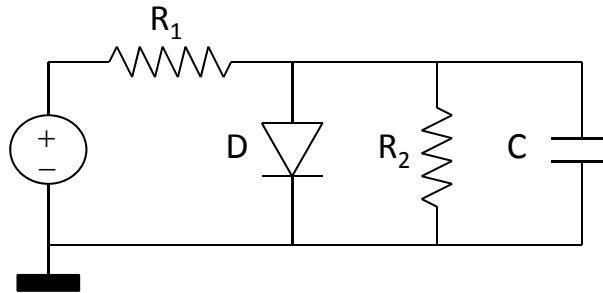


Figura 2.15: Circuito eléctrico.

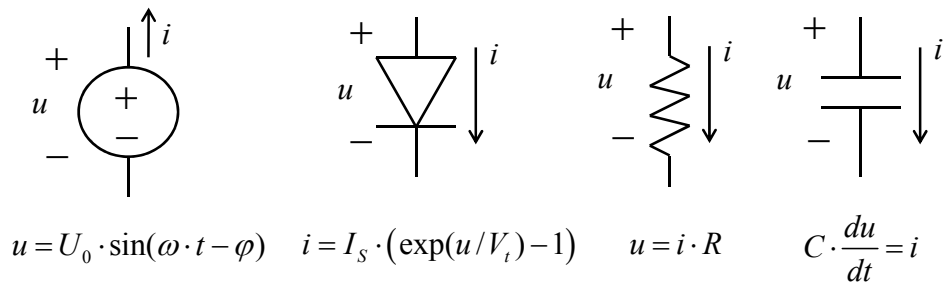


Figura 2.16: Relaciones constitutivas de los componentes.

Tabla 2.3: Parámetros del circuito mostrado en la Figura 2.15.

Componente	Parámetro	Valor
Generador de tensión	Amplitud	$U_0 = 5 \text{ V}$
	Frecuencia angular	$w = 200\pi \text{ rad/s (= 100 Hz)}$
	Desfase	$\varphi = 0 \text{ rad}$
Resistencias	Resistencia	$R_1 = 100 \text{ ohm}$
		$R_2 = 100 \text{ ohm}$
Condensador	Capacidad	$C = 10^{-6} \text{ F}$
Diodo	Corriente de saturación	$I_s = 10^{-9} \text{ A}$
	Tensión térmica	$V_t = 0.025 \text{ V}$

1. *Seleccionar un nodo como referencia para el voltaje.* Este nodo se denomina **nodo tierra**. El valor del voltaje de los demás nodos se calcula con referencia al voltaje del nodo tierra, al cual por convenio se le asigna el valor 0 V. Por ejemplo, si el voltaje de un nodo es +5 V, significa que ese nodo se encuentra a un voltaje 5 V más alto que el nodo tierra. En el circuito de la Figura 2.15 ya se especifica cuál es el nodo tierra, por tanto este primer paso ya está hecho.
2. *Asignar un nombre al voltaje en cada nodo del circuito.* Los nodos están representados mediante círculos rellenos en la Figura 2.17. En la figura también se muestran los nombres asignados al voltaje: u_0 (nodo tierra), u_1 , u_2 .

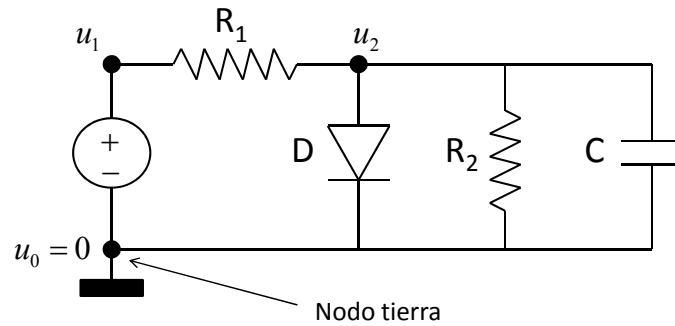


Figura 2.17: Se asignan nombres al voltaje en los nodos.

3. *Asignar nombre a las corrientes.* Para cada componente con dos conectores, se asigna nombre a la corriente que fluye a través del componente y se establece el criterio de signos para la corriente. En los componentes con más conectores, se asigna un nombre a la corriente que entra al componente a través de cada uno de sus conectores. En este circuito todos los componentes tienen dos conectores. Los nombres asignados a las corrientes son (véase la Figura 2.18): i_{gen} , i_{R1} , i_D , i_{R2} , i_C .

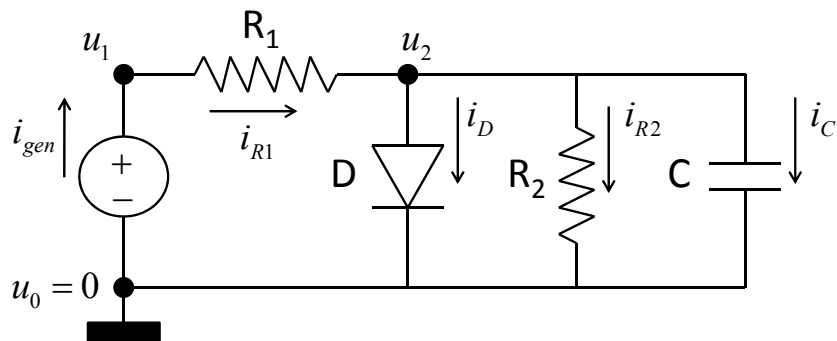


Figura 2.18: Se asignan nombres a las corrientes.

4. *Plantear la ecuación de conservación de la corriente en todos los nodos, salvo en el nodo tierra.* Dado que la carga eléctrica no se acumula en los nodos,

la suma total de corrientes que se dirigen hacia el nodo es igual a la suma de corrientes que salen de él. Las ecuaciones para los nodos u_1 y u_2 de este circuito son:

$$i_{gen} = i_{R1} \quad (2.19)$$

$$i_{R1} = i_D + i_{R2} + i_C \quad (2.20)$$

Obsérvese que plantear la ecuación en el nodo tierra no aporta información adicional: la ecuación obtenida es combinación lineal de las planteadas en los demás nodos. En este circuito, la ecuación en el nodo tierra:

$$i_{gen} = i_D + i_{R2} + i_C \quad (2.21)$$

es la misma que se obtiene sumando las Ecs (2.19) y (2.20), por ello no se incluye en el modelo.

5. *Escribir las relaciones constitutivas de los componentes.* Las relaciones constitutivas son las mostradas en la Figura 2.16. En todos los casos, u representa la caída de tensión entre bornes del componente, es decir, el voltaje del pin señalado con el signo “+” menos el voltaje del pin representado con el signo “-”. Suele aplicarse el convenio de considerar positiva la corriente que sale del pin positivo de los componentes activos y la que entra por el pin positivo de los componentes pasivos.

Las cinco ecuaciones mostradas a continuación son las relaciones constitutivas de los cinco componentes del circuito de la Figura 2.15.

$$u_1 - u_0 = U_0 \cdot \sin(w \cdot t + \varphi) \quad (2.22)$$

$$u_1 - u_2 = i_{R1} \cdot R_1 \quad (2.23)$$

$$i_D = I_s \cdot \left(\exp\left(\frac{u_2 - u_0}{V_t}\right) - 1 \right) \quad (2.24)$$

$$u_2 - u_0 = i_{R2} \cdot R_2 \quad (2.25)$$

$$C \cdot \frac{d}{dt}(u_2 - u_0) = i_C \quad (2.26)$$

Reemplazando el voltaje en el nodo tierra (u_0) por cero, se obtiene:

$$u_1 = U_0 \cdot \sin(w \cdot t + \varphi) \quad (2.27)$$

$$u_1 - u_2 = i_{R1} \cdot R_1 \quad (2.28)$$

$$i_D = I_s \cdot \left(\exp\left(\frac{u_2}{V_t}\right) - 1 \right) \quad (2.29)$$

$$u_2 = i_{R2} \cdot R_2 \quad (2.30)$$

$$C \cdot \frac{du_2}{dt} = i_C \quad (2.31)$$

Recopilando las ecuaciones anteriores, obtenemos que el modelo está compuesto por las dos ecuaciones de los nodos planteadas en el Paso 4, Ecs. (2.19) y (2.20), y por las cinco ecuaciones de los componentes planteadas en el Paso 5, Ecs. (2.27) – (2.31). Estas siete ecuaciones describen la evolución de las siete variables del modelo que dependen del tiempo: los voltajes (u_1 , u_2) y las corrientes (i_{gen} , i_{R1} , i_D , i_{R2} , i_C) del circuito. A los parámetros que intervienen en las relaciones constitutivas (U_0 , w , φ , R_1 , R_2 , C , I_s , V_t) se les asigna valor en la inicialización de la simulación y ese valor permanece constante durante toda la ejecución de la simulación. Los valores asignados son los indicados en la Tabla 2.3.

El modelo anterior puede ser descrito en lenguaje Modelica tal como se muestra en Código 2.2. Obsérvese lo siguiente:

- La declaración de las variables incluye especificar el tipo de la variable, su variabilidad (véase la Tabla 2.4), su nombre y su dimensión (la variable puede ser escalar, vectorial o una matriz). Opcionalmente, se puede asignar valor a los atributos asociados al tipo de la variable (véanse las Tablas 2.5 y Tabla 2.6).
- La relación constitutiva del generador es una función sinusoidal del tiempo. El tiempo es representado por una variable predefinida (está declarada por defecto en todos los modelos) del lenguaje llamada **time**.
- Se han empleado los atributos **start** y **fixed** en la declaración de u_2 para asignar valor inicial a esa variable. Este valor inicial, así como los valores asignados a los parámetros, podrán ser modificados en la definición del experimento.
- Al declarar las variables se han especificado sus unidades, asignando para ello valor al atributo **unit**.

En lugar de indicar una y otra vez que las unidades de las variables que representan corrientes y voltajes son amperios y voltios respectivamente, es posible definir

Tabla 2.4: Prefijos de variabilidad.

Prefijo	Significado
<i>constant</i>	La magnitud permanece constante durante la simulación. Representa una propiedad del modelo que en ningún caso será necesario cambiar.
<i>parameter</i>	La magnitud permanece constante durante la simulación, pero su valor puede ser cambiado al reutilizar los modelos y al definir el experimento.
<i>discrete</i>	Indica que la variable es de tiempo discreto. Este prefijo es opcional y generalmente no se usa, puesto que el tipo (las variables de tipo <i>Integer</i> , <i>Boolean</i> y <i>String</i> sólo pueden ser de tiempo discreto) y uso de la variable indica si es de tiempo continuo o discreto.
(sin prefijo)	Variables de tiempo continuo o discreto.

Tabla 2.5: Atributos de las variables.

Atributo	Significado
<i>quantity</i>	Significado físico de la variable.
<i>unit</i>	Unidades de la variable en las ecuaciones.
<i>min, max</i>	Valores mínimo y máximo que razonablemente acotan el valor de la variable. Si sale de ese rango es indicativo de que la simulación del modelo no reproduce el comportamiento del sistema modelado. Algunos entornos de modelado (por ejemplo, Dymola) ofrecen la opción de que se muestre un mensaje de aviso cuando el valor de la variable sale del intervalo definido por los atributos <i>min</i> y <i>max</i> . La sentencia <i>assert</i> también puede ser usada con este propósito.
<i>start, fixed</i>	El atributo <i>start</i> va asociado al atributo <i>fixed</i> . Cuando <i>fixed</i> vale <i>true</i> , el valor asignado a <i>start</i> representa el valor inicial de la variable. Cuando <i>fixed</i> vale <i>false</i> , el valor asignado a <i>start</i> se toma como valor inicial del proceso iterativo para la solución del modelo en el instante inicial de la simulación (inicialización).
<i>displayUnit</i>	Unidades empleadas para representar los resultados.

Tabla 2.6: Tipos predefinidos de variables y sus atributos.

	<i>Real</i>	<i>Integer</i>	<i>Boolean</i>	<i>String</i>
<i>quantity</i>	×	×	×	×
<i>unit</i>	×			
<i>min, max</i>	×	×		
<i>start, fixed</i>	×	×	×	×
<i>displayUnit</i>	×			

```

model circuito1
  Real i_gen(unit="A") "Corriente generador";
  Real i_R1(unit="A") "Corriente R1";
  Real i_R2(unit="A") "Corriente R2";
  Real i_C(unit="A") "Corriente condensador";
  Real i_D(unit="A") "Corriente diodo";
  Real u_1(unit="V") "Voltaje generador";
  Real u_2(start=0, fixed=true, unit="V") "Voltaje salida";
  // Parámetros del generador
  constant Real PI = 3.1415926536;
  parameter Real U0( unit="V") = 5;
  parameter Real frec( unit="Hz") = 100;
  parameter Real w( unit="rad/s") = 2*PI*frec;
  parameter Real phi( unit="rad") = 0;
  // Resistencias
  parameter Real R1( unit="ohm") = 100;
  parameter Real R2( unit="ohm") = 100;
  // Condensador
  parameter Real C( unit="F") = 1e-6;
  // Diodo
  parameter Real Is( unit="A") = 1e-9;
  parameter Real Vt( unit="V") = 0.025;

equation
  // Ecuaciones en los nodos
  i_gen = i_R1;
  i_R1 = i_D + i_R2 + i_C;
  // Relaciones constitutivas
  u_1 = U0 * sin( w * time + phi);
  u_1 - u_2 = i_R1 * R1;
  i_D = Is * ( exp(u_2 / Vt) - 1);
  u_2 = i_R2 * R2;
  C * der(u_2) = i_C;
end circuito1;

```

Código 2.2: Circuito de la Figura 2.15, con los parámetros de la Tabla 2.3.

nuevos tipos de variables, basándose en uno de los tipos básicos y asignando valor a los atributos. Por ejemplo, podrían definirse dos nuevos tipos llamados **Voltaje** y **Corriente** de la forma siguiente:

```

type Voltaje    = Real( unit = "V" );
type Corriente = Real( unit = "A" );

```

La clase **type** permite al desarrollador del modelo definir sus propios tipos de variables. Para ello debe indicarse el nombre del nuevo tipo, el tipo predefinido en el cual se basa el nuevo tipo y el valor de los atributos para el nuevo tipo.

La librería estándar de Modelica contiene un package llamado **Modelica.SIunits** en el cual se definen tipos de variables cuyas unidades son las del sistema internacional de unidades. Este paquete pretende ser un estándar respecto al tipo de las variables, con el objetivo de facilitar la reutilización de los modelos y el intercambio de modelos entre diferentes desarrolladores. Para ilustrar cómo se definen nuevos

```

package SIunits "Type definitions based on SI units, ISO 31-1992"
...
// Space and Time (chapter 1 of ISO 31-1992)
type Angle = Real(final quantity="Angle",final unit="rad",displayUnit="deg");
type Frequency = Real(final quantity="Frequency", final unit="Hz");
type AngularFrequency = Real(final quantity="AngularFrequency",final unit="s-1");
...
// Electricity and Magnetism (chapter 5 of ISO 31-1992)
type ElectricCurrent = Real(final quantity="ElectricCurrent", final unit="A");
type Current = ElectricCurrent;
type ElectricPotential = Real(final quantity="ElectricPotential", final unit="V");
type Voltage = ElectricPotential;
type Capacitance = Real(final quantity="Capacitance",final unit="F",min=0);
type Resistance = Real(final quantity="Resistance",final unit="Ohm");
...
end SIunits;

```

Código 2.3: Fragmento del package Modelica.SIunits de la MSL.

tipos, en Código 2.3 se muestra un fragmento de este package. Cuando se antepone *final* al nombre de un atributo, significa que el valor del atributo no puede ser cambiado al declarar variables de ese tipo.

Se usa la notación punto para acceder a las clases de un package. Así, el modelo mostrado en Código 2.2 puede equivalentemente describirse como se muestra en Código 2.4, donde se han usado tipos definidos en el package Modelica.SIunits. Obsérvese que, en lugar de declarar la constante π , se ha usado una constante llamada `pi` que está declarada en el package **Modelica.Constants** de la librería estándar de Modelica.

Simulando el modelo mostrado en el Código 2.4 (o indistintamente el mostrado en el Código 2.2) desde el instante 0 s hasta el instante 0.05 s, se obtiene la evolución del voltaje en los nodos mostrada en la Figura 2.19.

Añadiendo tres diodos al circuito de la Figura 2.15 y eliminando una de las resistencias, puede obtener el circuito rectificador mostrado en la Figura 2.20. Puede verse en la figura cómo se han aplicado los primeros pasos de la metodología de modelado: se ha seleccionado un nodo como referencia para el voltaje, y se ha asignado nombre al voltaje en los nodos y a las corrientes.

El modelo se compone de las ecuaciones de conservación de la corriente, planteadas en todos los nodos excepto el nodo tierra, y de las relaciones constitutivas de los componentes. El modelo está compuesto por las Ecs. (2.32) – (2.42). Se ha eliminado de las ecuaciones la tensión en el nodo tierra, sustituyendo su valor por cero. Las tres primeras ecuaciones representan las condición de conservación de la corriente en los nodos u_1 , u_2 y u_3 , respectivamente.

```

model circuito1
  Modelica.SIunits.Current i_gen "Corriente generador";
  Modelica.SIunits.Current i_R1 "Corriente R1";
  Modelica.SIunits.Current i_R2 "Corriente R2";
  Modelica.SIunits.Current i_C "Corriente condensador";
  Modelica.SIunits.Current i_D "Corriente diodo";
  Modelica.SIunits.Voltage u_1 "Voltaje generador";
  Modelica.SIunits.Voltage u_2(start=0, fixed=true) "Voltaje salida";
  // Parámetros del generador
  parameter Modelica.SIunits.Voltage U0 = 5;
  parameter Modelica.SIunits.Frequency freq = 100;
  parameter Modelica.SIunits.AngularFrequency w = 2*Modelica.Constants.pi*freq;
  parameter Modelica.SIunits.Angle phi = 0;
  // Resistencias
  parameter Modelica.SIunits.Resistance R1 = 100;
  parameter Modelica.SIunits.Resistance R2 = 100;
  // Condensador
  parameter Modelica.SIunits.Capacitance C = 1e-6;
  // Diodo
  parameter Modelica.SIunits.Current Is = 1e-9;
  parameter Modelica.SIunits.Voltage Vt = 0.025;

equation
  // Ecuaciones en los nodos
  i_gen = i_R1;
  i_R1 = i_D + i_R2 + i_C;
  // Relaciones constitutivas
  u_1 = U0 * sin( w * time + phi);
  u_1 - u_2 = i_R1 * R1;
  i_D = Is * ( exp(u_2 / Vt) - 1);
  u_2 = i_R2 * R2;
  C * der(u_2) = i_C;
end circuito1;

```

Código 2.4: Circuito de la Figura 2.15, usando los tipos de la MSL.

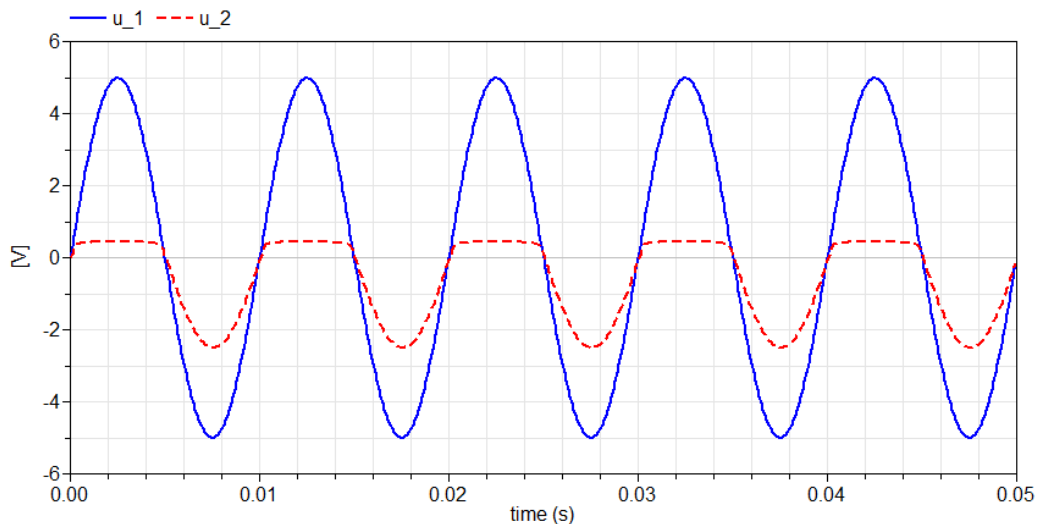


Figura 2.19: Voltajes del circuito de la Figura 2.15 obtenidos simulando el Código 2.4.

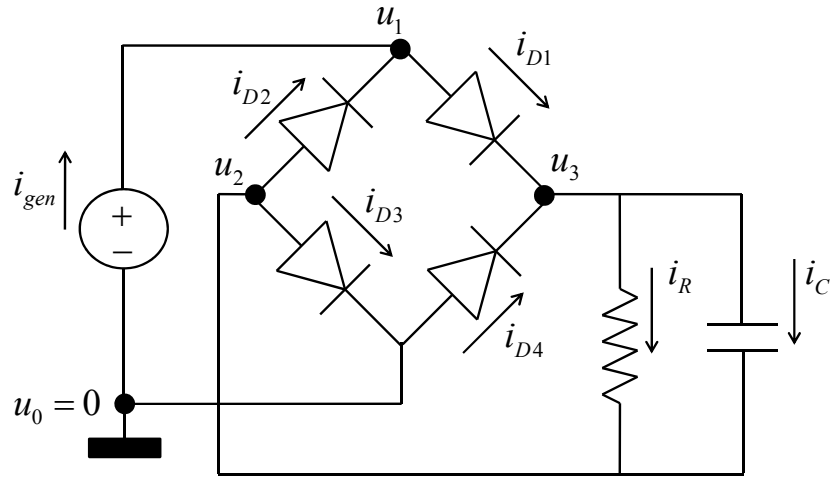


Figura 2.20: Circuito eléctrico rectificador.

$$i_{gen} + i_{D2} = i_{D1} \quad (2.32)$$

$$i_R + i_C = i_{D2} + i_{D3} \quad (2.33)$$

$$i_{D1} + i_{D4} = i_R + i_C \quad (2.34)$$

$$u_1 = U_0 \cdot \sin(w \cdot t + \varphi) \quad (2.35)$$

$$i_{D1} = I_s \cdot \left(\exp\left(\frac{u_1 - u_3}{V_t}\right) - 1 \right) \quad (2.36)$$

$$i_{D2} = I_s \cdot \left(\exp\left(\frac{u_2 - u_1}{V_t}\right) - 1 \right) \quad (2.37)$$

$$i_{D3} = I_s \cdot \left(\exp\left(\frac{u_2}{V_t}\right) - 1 \right) \quad (2.38)$$

$$i_{D4} = I_s \cdot \left(\exp\left(\frac{-u_3}{V_t}\right) - 1 \right) \quad (2.39)$$

$$u_3 - u_2 = i_R \cdot R \quad (2.40)$$

$$u_C = u_3 - u_2 \quad (2.41)$$

$$C \cdot \frac{du_C}{dt} = i_C \quad (2.42)$$

La descripción en lenguaje Modelica del circuito se muestra en el Código 2.5. Se ha simulado el modelo durante 0.05 s. En la Figura 2.21 se representan el voltaje del generador (u_1) y el voltaje entre los conectores del condensador (u_C).

```

model circRectificador
  Modelica.SIunits.Current i_gen "Corriente generador";
  Modelica.SIunits.Current i_R "Corriente R";
  Modelica.SIunits.Current i_C "Corriente condensador";
  Modelica.SIunits.Current i_D1 "Corriente diodo 1";
  Modelica.SIunits.Current i_D2 "Corriente diodo 2";
  Modelica.SIunits.Current i_D3 "Corriente diodo 3";
  Modelica.SIunits.Current i_D4 "Corriente diodo 4";
  Modelica.SIunits.Voltage u_1 "Voltaje generador";
  Modelica.SIunits.Voltage u_2 "Voltaje nodo u2";
  Modelica.SIunits.Voltage u_3 "Voltaje nodo u3";
  Modelica.SIunits.Voltage u_C(start=0, fixed=true) "Voltaje condensador";
  // Parámetros del generador
  parameter Modelica.SIunits.Voltage U0=5;
  parameter Modelica.SIunits.Frequency freq=100;
  parameter Modelica.SIunits.AngularFrequency w=2*Modelica.Constants.pi*freq;
  parameter Modelica.SIunits.Angle phi=0;
  // Resistencia
  parameter Modelica.SIunits.Resistance R=100;
  // Condensador
  parameter Modelica.SIunits.Capacitance C=1e-6;
  // Diodos
  parameter Modelica.SIunits.Current Is=1e-9;
  parameter Modelica.SIunits.Voltage Vt=0.025;

equation
  // Ecuaciones en los nodos
  i_gen + i_D2 = i_D1;
  i_R + i_C = i_D2 + i_D3;
  i_D1 + i_D4 = i_R + i_C;
  // Relaciones constitutivas
  u_1 = U0*sin(w*time + phi);
  i_D1 = Is*(exp((u_1 - u_3)/Vt) - 1);
  i_D2 = Is*(exp((u_2 - u_1)/Vt) - 1);
  i_D3 = Is*(exp(u_2/Vt) - 1);
  i_D4 = Is*(exp(-u_3/Vt) - 1);
  u_3 - u_2 = i_R*R;
  u_C = u_3 - u_2;
  C*der(u_C) = i_C;
end circRectificador;

```

Código 2.5: Modelo del circuito rectificador.

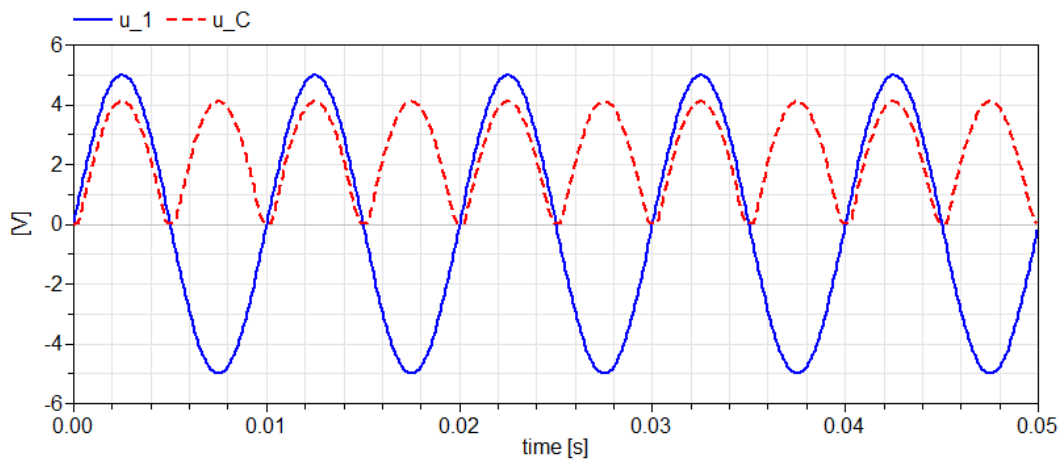


Figura 2.21: Resultado de la simulación del Código 2.5.

2.4.2. Ejemplo 2: librería eléctrica

Abordemos ahora el modelado del circuito desde la perspectiva de la orientación a objetos. En lugar de escribir las ecuaciones del modelo completo, describamos los modelos de los componentes electrónicos como clases, y seguidamente describamos el circuito mediante instanciación y conexión de sus componentes.

El primer paso consiste en definir el conector eléctrico. Para ello se emplea la clase **connector** de Modelica. La definición de una clase de conector llamada **Pin**, que describa el conector eléctrico, podría ser la siguiente:

```
connector Pin
  Modelica.SIunits.Voltage    u;
  flow Modelica.SIunits.Current i;
end Pin;
```

La declaración comienza con la palabra reservada *connector* seguida por el nombre de la clase, y finaliza por la palabra reservada *end* seguida del nombre de la clase y de punto y coma. La clase *connector* contiene las declaraciones de las variables que forman el conector. No está permitido incluir ecuaciones en este tipo de clase. Para distinguir las variables *across* de las *through*, se escribe la palabra reservada **flow** antes de la declaración de las variables *through*.

La **interfaz** de un modelo está compuesta por las variables que definen la interacción con el entorno y por aquellos parámetros que permiten adaptar el modelo a sus distintas aplicaciones.

Al desarrollar librerías de modelos, es una buena práctica definir la interfaz de los modelos en clases separadas, de tal modo que se fomente la reutilización de la definición de las interfaces por aquellos modelos que tienen la interfaz en común o parte de la interfaz en común. Esto facilita reconocer cuándo varios modelos tienen la misma interfaz, o parte de la interfaz en común.

Por ejemplo, los componentes eléctricos de dos terminales tienen parte de su interfaz en común: los dos pines. Aparte, cada componente tendrá determinados parámetros que no serán comunes al resto de los componentes, por ser propios de la relación constitutiva de cada componente. Puede definirse una clase compuesta de dos instancias del conector **Pin** de la forma siguiente:

```
partial model DosPines
  Pin  p, n;
protected
  Modelica.SIunits.Voltage u  "Voltaje entre pines (= p.u - n.u)";
```

```
equation
  u = p.u - n.u;
end DosPines;
```

Obsérvese que en lugar de *model*, en este caso se ha escrito **partial model**. Con ello se indica que esta clase debe ser completada con información adicional, antes de poder crear instancias de ella. En efecto, este modelo sólo describe la interfaz, siendo necesario especificar la relación constitutiva del componente.

En el modelo `DosPines` se ha declarado la variable `u`, que describe la diferencia entre el voltaje en el pin `p` (positivo) y en el pin `n` (negativo). Obsérvese que se emplea la notación punto para acceder a las variables de los conectores: `p.u` es la variable `u` del pin `p`, mientras que `n.u` es la variable `u` del pin `n`.

La variable `u` se ha declarado a continuación de la palabra reservada **protected**. Con ello se indica que esta variable no pertenece a la interfaz de la clase y por tanto no debería accederse directamente a ella desde fuera de la clase.

En los componentes de dos terminales tales como la resistencia, el condensador, la inducción y el diodo se satisface que la corriente que entra por uno de los pines es igual a la corriente que sale por el otro. En los componentes pasivos de dos terminales se considera que la corriente tiene signo positivo cuando fluye del pin positivo al pin negativo. Esta característica puede modelarse mediante una clase como la mostrada a continuación:

```
partial model UnPuerto
  extends DosPines;
protected
  Modelica.SIunits.Current i "Corriente del pin p al pin n";
equation
  i = p.i;
  i = -n.i;
end UnPuerto;
```

Al definir una clase puede indicarse que ésta es subclase de otras escribiendo el nombre de cada una de las superclases precedido de la palabra reservada **extends**. Así, por ejemplo, la sentencia

```
extends DosPines;
```

indica que la clase `UnPuerto` hereda la clase `DosPines`.

Modelica soporta herencia múltiple. Si la clase B hereda las clases A_1 , A_2 , $\dots$, A_N , se dice que A_1 , A_2 , $\dots$, A_N son **superclases** de B , quien a su vez es

una **subclase** de aquellas. Cuando la clase B hereda la clase A_i , es equivalente a copiar y pegar el código de la clase A_i en la definición de la clase B , pero con la ventaja añadida de que al heredar se pueden asignar valores a los parámetros de la superclase, con el fin de particularizar el comportamiento y la estructura definidos en la superclase.

Pueden definirse las clases que describen la resistencia, el diodo y el condensador heredando la clase `UnPuerto`.

```

model Resistencia "Resistencia ideal"
  extends UnPuerto;
  parameter Modelica.SIunits.Resistance R "Resistencia";
equation
  u = R*i;
end Resistencia;

model Diodo "Diodo ideal"
  extends UnPuerto;
  parameter Modelica.SIunits.Current Is = 1e-9 "Corriente saturación";
  parameter Modelica.SIunits.Voltage Vt = 0.025 "Tensión térmica";
equation
  i = Is * ( exp(u / Vt) - 1 );
end Resistencia;

model Condensador "Condensador ideal"
  extends UnPuerto;
  parameter Modelica.SIunits.Capacitance C "Capacidad";
equation
  C*der(u) = i;
end Condensador;

```

Obsérvese que en las clases `Resistencia` y `Condensador` no se ha asignado valor a los parámetros R y C , respectivamente. Cuando no se asigna valor a un parámetro, éste toma por defecto el valor 0 si es de tipo Real o Integer, y el valor *false* si es de tipo Boolean. Al utilizar la clase, es decir, al heredarla o instanciarla como componente de un modelo compuesto, y también al instanciarla para simularla (es decir, al definir el experimento), es posible modificar el valor de sus parámetros.

Como se señaló en el Paso 5 de la metodología de modelado de circuitos descrita en la Sección 2.4.1, el criterio de signos para la corriente que fluye a través de las fuentes es normalmente diferente al criterio de signos para los sistemas pasivos. En el caso de las fuentes suele considerarse que la corriente tiene signo positivo cuando fluye del pin negativo al positivo.

Modifiquemos la clase `UnPuerto` de manera que sea aplicable tanto a componentes pasivos como activos. Para ello, puede definirse un parámetro booleano en función del cual se adopte un criterio de signos u otro para la corriente.

```

partial model UnPuerto
  extends DosPines;
  parameter Boolean activo = false;
protected
  Modelica.SIunits.Current i "Corriente a través del componente";
equation
  if ( activo ) then
    i = n.i;
  else
    i = p.i;
  end if;
  p.i = - n.i;
end UnPuerto;

```

La **cláusula if** permite incluir condicionalmente ecuaciones en el modelo. Esto resulta útil cuando se desea poder seleccionar, entre diferentes hipótesis de modelado, cuál es la más apropiada al caso de uso en concreto. La condición de la cláusula *if* es una expresión booleana que puede ser dependiente o independiente del tiempo. La resolución numérica depende de ello:

- Si la condición de la *cláusula if* es susceptible de cambiar en tiempo de simulación, el entorno de modelado debe asociar un invariante a la condición tal como se explicó en la Sección 1.4.7.
- Si la condición de la *cláusula if* sólo depende de constantes y parámetros, entonces el valor de la condición es calculado en la inicialización del modelo y permanece constante durante toda la simulación. En este caso, el entorno de modelado puede decidir en la inicialización qué rama de la cláusula incluir en el modelo, eliminando del modelo las ramas cuya condición vale *false*.

En este caso la condición de la *cláusula if* es el parámetro **activo**, por lo que es independiente del tiempo. Al declarar el parámetro **activo** se le ha asignado un valor por defecto: *false*. Ese será el valor asignado al parámetro a menos que se modifique al instanciar la clase. En la siguiente clase, que describe una fuente sinusoidal de tensión, puede verse cómo puede redefinirse el valor de los parámetros de una clase al heredarla.

```

model VfuelleAC "Fuente tensión AC"
  extends UnPuerto ( activo = true );
  parameter Modelica.SIunits.Voltage U0;
  parameter Modelica.SIunits.Frequency frec;
  parameter Modelica.SIunits.Angle phi;
protected
  parameter Modelica.SIunits.AngularFrequency w =

```

```

2*Modelica.Constants.pi*frec;
equation
  u = U0 * sin( w * time + phi);
end VfuelleAC;

```

El componente eléctrico tierra, que posee un único pin, puede definirse de la manera siguiente:

```

model Tierra "Tierra"
  Pin p;
equation
  p.u = 0;
end Tierra;

```

Cuando se desarrollan clases para el modelado en un cierto dominio o aplicación, normalmente se estructuran formando una librería. De esta manera, se facilita la comprensión y el uso de las clases, a la vez que se evita que haya conflictos de nombres entre estas clases y otras a las que se haya dado en mismo nombre.

La clase **package** permite organizar las clases en librerías. Dado que es posible definir una clase *package* dentro de otra clase *package*, pueden crearse librerías estructuradas de forma compleja, con múltiples niveles jerárquicos.

Estructuremos los componentes definidos anteriormente en una librería. Para ello, definiremos en el nivel jerárquico superior un *package* llamado *LibElectrica*, que englobará la librería completa. Dentro de éste, definiremos otros tres *packages* (véase la Figura 2.22 y el Código 2.6–2.8):

1. En el *package Interfaces* incluiremos las clases que definen las interfaces de los componentes.
2. En el *package Componentes* incluiremos las clases que describen los componentes eléctricos.
3. En el *package Ejemplos* incluiremos el circuito ejemplo mostrado en la Figura 2.15, el cual vamos a describir instanciando y conectando los componentes de la librería.

Como puede observarse en el Código 2.8, la forma de referirse a una clase que se encuentra en una librería es mediante la **notación punto**. Por ejemplo, es posible referirse a la clase *Resistencia* desde fuera de la librería *LibElectrica* especificando todo el camino mediante la notación punto (véase la Figura 2.22): *LibElectrica.Componentes.Resistencia*.

```

encapsulated package LibElectrica

import SI = Modelica.SIunits;
import Modelica.Constants;

package Interfaces
  connector Pin
    SI.Voltage u;
    flow SI.Current i;
  end Pin;

  partial model DosPines
    Pin p, n;
  protected
    SI.Voltage u "Voltaje entre pines (= p.u - n.u)";
  equation
    u = p.u - n.u;
  end DosPines;

  partial model UnPuerto
    extends DosPines;
    parameter Boolean activo=false;
  protected
    SI.Current i "Corriente a través del componente";
  equation
    if (activo) then
      i = n.i;
    else
      i = p.i;
    end if;
    p.i = -n.i;
  end UnPuerto;
end Interfaces;

```

Código 2.6: Librería eléctrica (1/3).


```

package Componentes

model Resistencia "Resistencia ideal"
  extends Interfaces.UnPuerto;
  parameter SI.Resistance R "Resistencia";
equation
  u = R*i;
end Resistencia;

model Diodo "Diodo ideal"
  extends Interfaces.UnPuerto;
  parameter SI.Current Is=1e-9 "Corriente saturación";
  parameter SI.Voltage Vt=0.025 "Tensión térmica";
equation
  i = Is*(exp(u/Vt) - 1);
end Diodo;

model Condensador "Condensador ideal"
  extends Interfaces.UnPuerto;
  parameter SI.Capacitance C "Capacidad";
equation
  C*der(u) = i;
end Condensador;

model VfuelleAC "Fuente tensión AC"
  extends Interfaces.UnPuerto(activo=true);
  parameter SI.Voltage U0;
  parameter SI.Frequency freq;
  parameter SI.Angle phi;
protected
  parameter SI.AngularFrequency w=2*Constants.pi*freq;
equation
  u = U0*sin(w*time + phi);
end VfuelleAC;

model Tierra "Tierra"
  Interfaces.Pin p;
equation
  p.u = 0;
end Tierra;

end Componentes;

```

Código 2.7: Librería eléctrica (2/3).

```

package Ejemplos

model circuito1
  // Parámetros del generador
  parameter SI.Voltage U0=5;
  parameter SI.Frequency frec=100;
  parameter SI.Angle phi=0;
  // Resistencias
  parameter SI.Resistance R1=100;
  parameter SI.Resistance R2=100;
  // Condensador
  parameter SI.Capacitance C=1e-6;
  // Diodo
  parameter SI.Current Is=1e-9;
  parameter SI.Voltage Vt=0.025;

  // Componentes
  Componentes.Resistencia Resist1(R=R1);
  Componentes.Resistencia Resist2(R=R2);
  Componentes.Condensador Cond(C=C);
  Componentes.Diodo Diodo(Is=Is, Vt=Vt);
  Componentes.VfuenteAC VS(U0=U0, frec=frec, phi=phi);
  Componentes.Tierra tierra;
equation
  connect(VS.p, Resist1.p);
  connect(Resist1.n, Diodo.p);
  connect(Resist1.n, Resist2.p);
  connect(Resist1.n, Cond.p);
  connect(VS.n, tierra.p);
  connect(Diodo.n, tierra.p);
  connect(Resist2.n, tierra.p);
  connect(Cond.n, tierra.p);
end circuito1;

end Ejemplos;

end LibElectrica;

```

Código 2.8: Librería eléctrica (3/3).



Figura 2.22: Arquitectura de la librería eléctrica.

El algoritmo que emplea el entorno de modelado para **localizar una clase** conocido su camino es el siguiente:

1. Partiendo desde donde se hace la llamada, se asciende por el árbol jerárquico buscando el primer nombre. En el ejemplo anterior, éste sería **LibElectrica**.
2. Una vez se encuentra, se desciende por el árbol partiendo de ese punto, hasta localizar el resto del camino, que en el ejemplo anterior sería **Componentes.Resistencia**.

Obsérvese también que en la declaración del *package* (véase la primera línea del Código 2.6) se ha antepuesto la palabra reservada **encapsulated**. Cuando un *package* se encapsula no es posible referenciar directamente, desde dentro del *package*, clases externas al *package*. Con ello se facilita el mantenimiento de los modelos, ya que se minimiza el número de cambios que hay que hacer en el código cuando cambia la ubicación de los *packages* en la jerarquía de clases.

Para referenciar nodos externos al *encapsulated package*, debe importarse la dirección que se desea hacer visible, empleando para ello una sentencia **import**. A partir del nodo visible, se pueden referenciar clases que cuelguen de él empleando la notación punto. En el Código 2.6, que se han declarado las dos sentencias *import* siguientes:

```

import SI = Modelica.SIunits;
import Modelica.Constants;

```

donde puede verse que la sentencia *import* se usa de dos maneras diferentes:

1. Mediante la sentencia

```
import Modelica.Constants;
```

se hace visible el nodo `Constants`, con lo cual pueden referenciarse los nodos que cuelgue de él tomándole a él como primer nodo de la referencia. Por ejemplo, `Constants.pi` (véase el Código 2.7).

2. Mediante la sentencia

```
import SI = Modelica.SIunits;
```

se define una etiqueta, `SI`, que sirve de abreviatura para referirse el nodo visible `Modelica.SIunits`. De esta manera los tipos declarados en la librería estándar de Modelica pueden referenciarse de la forma `SI.Voltage`, `SI.Current`, etc.

Aparte de las dos anteriores, existe una tercera forma de usar la sentencia *import*. Puede verse en el ejemplo siguiente:

```
import Modelica.Math.*;
```

Esta sentencia da acceso a los nodos que cuelgan de `Modelica.Math` sin necesidad de especificar que el primer nodo de la referencia es `Math`. Esta forma de la sentencia *import* hay que emplearla con precaución, ya que puede haber conflicto de nombres entre el paquete local y el paquete importado, lo cual podría dar lugar a errores difíciles de depurar. No obstante, esta variante de *import* sí puede ser útil para establecer una notación estándar de funciones matemáticas, constantes, etc. Para evitar introducir dependencias ocultas, las sentencias *import*, en cualquiera de sus modalidades, no se heredan.

Puede observarse en el Código 2.8 que al instanciar las clases puede asignarse valor a sus parámetros. Por ejemplo, en la definición del modelo `circuito1` se ha creado un objeto de la clase `Resistencia`, al que se ha llamado `Resist1` y se ha asignado a su parámetro `R` el valor del parámetro `R1` del modelo `circuito1`, al cual se ha asignado por defecto el valor 100 ohm. Este valor de `R1` podrá posteriormente ser modificado al definir el experimento.

La conexión entre los componentes se realiza mediante sentencias **connect**. Cada sentencia *connect* tienen dos argumentos, cada uno de los cuales es el conector de un objeto. El entorno de modelado traduce cada sentencia *connect* en las correspondientes ecuaciones de igualdad entre las variables across de los conectores y las ecuaciones de suma igualada a cero para las variables through.

Cuando se deja un **connector sin conectar** a otros conectores de su mismo nivel jerárquico, el entorno de modelado añade automáticamente al modelo tantas ecuaciones como variables through tiene el conector. En cada ecuación se impone que la variable through valga cero.

Es posible realizar modelos compuestos conectando componentes que a su vez son modelos compuestos. No existe un límite definido para el número de niveles jerárquicos en la composición. Al instanciar un objeto pueden modificarse los parámetros de componentes que se encuentren varios niveles por debajo en la jerarquía, usando para ello la notación punto. A continuación se muestra un ejemplo:

```
model Resistencia
    parameter Modelica.SIunits.Resistance R;
    ...
end Resistencia;

model SubCirc1
    Resistencia R1(R=1), R2(R=10);
    ...
end SubCirc1;

model Circuito1
    SubCirc1 S2(R1.R=3, R2.R=30);
    ...
end Circuito1;
```

Obsérvese que la clase `SubCirc1` está compuesta de dos objetos de la clase `Resistencia`, a los que se ha llamado `R1` y `R2`. A su vez, la clase `Circuito1` contiene un objeto de la clase `SubCirc1`. Al definir este objeto, se modifica el valor del parámetro `R` de sus componentes `R1` y `R2`, empleando para ello la notación punto.

Como comentario final, indicar que típicamente hay dos maneras diferentes de almacenar en disco las librerías en Modelica. La más habitual consiste en almacenar toda la librería en un único fichero de texto, el cual debe tener extensión **.mo**. Así, por ejemplo, la librería eléctrica podría guardarse en un único fichero de texto llamado *LibElectrica.mo*.

Otra opción es almacenar la librería en una estructura jerárquica de ficheros y directorios. Esto debe hacerse de la forma indicada a continuación.

- Se crea un directorio por cada *package*, manteniendo la misma estructura jerárquica que tienen los *packages*. El nombre del directorio corresponde con el nombre del *package*.
- Dentro del directorio asociado a un *package* se crea un fichero llamado **package.mo**, en el cual se declara el *package*. Normalmente se incluye dentro de la declaración del *package* la *annotation* con la documentación del paquete. Las clases contenidas en el *package* se definen en ficheros separados, grabados en el directorio del *package*, con lo cual no se incluyen en el fichero *package.mo*.
- Cada clase se almacena en un fichero cuyo nombre coincide con el de la clase y su extensión es .mo. El fichero con la clase se graba en el directorio correspondiente al *package* en el cual está la clase.
- La primera sentencia de cada uno de estos ficheros y de cada fichero *package.mo* debe ser una sentencia **within**, que especifica el nombre completo de la clase dentro de la jerarquía de ficheros. Por ejemplo, dentro del fichero *Resistencia.mo* se define la clase **Resistencia**. En este caso, la primera sentencia del fichero *Resistencia.mo* debería ser:

```
within LibElectrica.Componentes;
```

lo cual indica que el camino de la clase dentro de la librería es:

```
LibElectrica.Componentes.Resistencia
```

En la variable de entorno **MODELICAPATH** pueden especificarse los directorios donde están las librerías, separados por el símbolo punto y coma. Por ejemplo, si se referencia

```
LibElectrica.Componentes.Resistencia
```

el entorno de modelado buscará **LibElectrica** en la lista de directorios de **MODELICAPATH**. Conviene tener en cuenta que si se encuentra **LibElectrica** en uno de los directorios y el resto de la búsqueda falla (el descenso a partir de ese nodo), entonces se aborta el proceso, es decir, no se buscará **LibElectrica** en otro directorio.

2.4.3. Ejemplo 3: redeclaración del tipo de los componentes

Los siguientes ejemplos en el dominio eléctrico ilustran otra de las capacidades del lenguaje Modelica: la parametrización de la clase de los objetos. Modelica permite que la clase de los objetos sea un parámetro, de modo que pueda redeclararse al instanciar y heredar. Veámoslo mediante un ejemplo.

Supongamos que se definen dos modelos diferentes de la resistencia eléctrica. En el primero, no se considera la dependencia de la resistencia respecto a la temperatura:

```
model Resistencia
  extends UnPuerto;
  parameter Real R(unit="Ohm");
equation
  v = R*i;
end Resistencia;
```

mientras que en el segundo modelo sí se considera:

```
model ResistenciaTemp
  extends UnPuerto;
  parameter Real R      (unit="Ohm");
  parameter Real RT     (unit="Ohm/degC") = 0;
  parameter Real Tref   (unit="degC")     = 20;
  Real Temp = 20;
equation
  v = ( R + RT*(Temp-Tref) ) * i;
end ResistenciaTemp;
```

Supongamos que se define un circuito eléctrico que contiene resistencias y también que, por las peculiaridades del estudio a realizar, en algunos casos convendrá que los objetos que representan las resistencias sean de la clase `Resistencia`, mientras que en otros casos convendrá que sean de la clase `ResistenciaTemp`.

La clase de los objetos que representan las resistencias puede definirse como un parámetro que sea modificado al instanciar el modelo del circuito. Para ello, se definen los componentes resistencia como componentes reemplazables:

```
model Circuito
  replaceable Resistencia R1(R=100), R2(R=200);
  Resistencia R3(R=300), R4(final R=400);
equation
  ...
end Circuito;
```

De esta manera, puede redeclararse la clase de los objetos R1 y R2. Por ejemplo, en el siguiente modelo,

```
model CircuitoModificado
  extends Circuito (
    redeclare ResistenciaTemp R1(RT=0.1),
    redeclare ResistenciaTemp R2 );
end CircuitoModificado;
```

los objetos R1 y R2 pasan a ser de la clase `ResistenciaTemp`.

Puede imponerse que la clase reemplazada y las reemplazantes tengan una superclase en común, que típicamente será la clase donde se define la interfaz o una subclase de ella. Puede imponerse que las clases que reemplacen a `Resistencia` sean subclases de `UnPuerto`.

```
model Circuito
  replaceable Resistencia R1(R=100) extends UnPuerto;
  replaceable Resistencia R2(R=200) extends UnPuerto;
  Resistencia R3(R=300), R4(final R=400);
equation
  ...
end Circuito;
```

Por ejemplo, si se desea reemplazar la resistencia R1 por un condensador, puede reemplazarse la clase del objeto:

```
model CircuitoModificado
  extends Circuito ( redeclare Condensador R1(C=0.003) );
end CircuitoModificado;
```

de esta manera, R1 pasa de ser de la clase `Resistencia` a ser de la clase `Condensador`.

La desventaja que tiene este procedimiento es que obliga a conocer que hay dos objetos de clase reemplazable en `Circuito`, y que sus nombres son R1 y R2. La necesidad de tener un conocimiento tan detallado acerca de la descripción interna de `Circuito` es contrario al principio de abstracción, según el cual las clases deben poder ser usadas sin necesidad de conocer sus detalles internos.

Para evitar esto, puede definirse una etiqueta, que almacene el nombre de la clase que deben tener los objetos. Asignando un nuevo valor a esa etiqueta se cambia la clase de los objetos. Veamos un ejemplo. En la siguiente definición del modelo `Circuito` se declara una etiqueta, `Dispositivo`, a la cual se asigna el valor por defecto `Resistencia`.


```

model Circuito
  replaceable model Dispositivo = Resistencia;
protected
  Dispositivo R1(R=100), R2(R=200);
  Resistencia R3(R=300), R4(final R=400);
equation
  ...
end Circuito;

```

Esto permite reemplazar en bloque todos los objetos cuya clase sea esa etiqueta, sin necesidad de ir refiriéndose a ellos uno a uno. Por ejemplo, la clase de los objetos puede redefinirse como **Condensador** de la forma siguiente:

```

model CircuitoModificado =
  Circuito ( redeclare model Dispositivo = Condensador );

```

Este mismo mecanismo para reemplazar la clase de los objetos puede aplicarse no sólo a modelos (es decir, a objetos `model`) sino también a conectores, tipos de variables, records, etc. La sintaxis de las sentencias `replaceable` y `redeclare` es análoga, indicando en cada caso si se trata de un `connector`, `type`, `record`, etc. Por ejemplo, para declarar un conector de clase reemplazable debe emplearse `replaceable connector` y para redeclararlo `redeclare connector`.

Veamos un ejemplo de **redeclaración de conectores**. Supongamos que se han definido dos clases de conectores,

<pre> connector ConecElec1 ... end ConecElec1; </pre>		<pre> connector ConecElec2 ... end ConecElec2; </pre>
---	--	---

y que se define un circuito cuya interfaz contiene dos conectores llamados `Pin1` y `Pin2`, cuya clase debe poder ser reemplazada. Se define una etiqueta, a la que arbitrariamente se llama `ConectorElectrico`, que representa la clase de los conectores y a la que se asigna el valor por defecto `ConecElec1`.

```

model circ1
  replaceable connector ConectorElectrico = ConecElec1;
  ConectorElectrico Pin1, Pin2;
  ...
equation
  ...
end circ1;

```

Puede definirse el modelo `circ2`, que extiende `circ1` redeclarando la clase de los conectores `Pin1` y `Pin2`, de la manera siguiente:

```

model circ2
  extends circ1 ( redeclare connector ConectorElectrico = ConecElec2 );
  ...
equation
  ...
end circ2;

```

Veamos un ejemplo de **redeclaración del tipo de variable**. Supongamos que se desea programar un modelo que describa la forma de onda exponencial. Esta forma de onda será posteriormente usada para describir dos modelos: un generador de voltaje y un generador de corriente eléctrica. Por ello, a priori no puede especificarse qué unidades tendrá la variable `signal`, que describe la forma de onda.

```

partial model EXP
  replaceable type SignalType = Real;
  parameter SignalType S1;
  parameter SignalType S2;
  parameter Modelica.SIunits.Time TD1(min=0);
  ...
protected
  parameter SignalType TRANS_INITIAL = S1;
  SignalType signal;
  Modelica.SIunits.Time timeStartTran;
equation
  ...
end EXP;

```

A partir de esta clase, pueden definirse el generador exponencial de tensión,

```

model Vsource
  extends EXP ( redeclare type SignalType=Modelica.SIunits.Voltage );
  ...
end Vsource;

```

y el generador exponencial de corriente eléctrica,

```

model Isource
  extends EXP ( redeclare type SignalType=Modelica.SIunits.Current );
  ...
end Isource;

```

2.5. SISTEMAS MECÁNICOS

En esta sección mostraremos ejemplos sencillos de modelado en el dominio mecánico, lo cual dará pie a explicar otras capacidades del lenguaje Modelica. Para

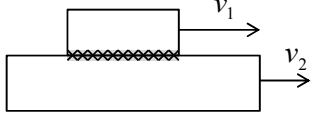
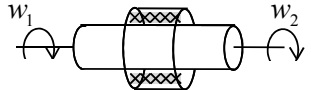
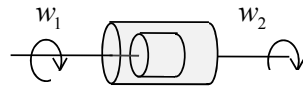
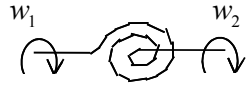
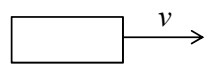
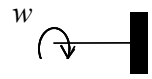
	TRASLACIÓN	ROTACIÓN ALREDEDOR DE UN EJE FIJO
COMPONENTES RESISTIVOS	Fricción seca 	Fricción seca 
	Fricción viscosa 	Fricción viscosa 
COMPONENTES ALMACENADORES DE ENERGÍA	Muelle 	Muelle de torsión 
	Masa (inercia) 	Masa (inercia rotacional) 

Figura 2.23: Disipación y almacenamiento de energía en el dominio de la mecánica de traslación y rotación entorno a un eje fijo.

plantear los modelos nos valdremos de los conceptos básicos de almacenamiento y disipación de la energía expuestos en la Sección 2.3. En la Figura 2.23 se han representado los componentes primitivos que emplearemos para representar los tres conceptos siguientes:

1. La *disipación de energía* debida a la fricción seca o viscosa. En estos componentes existe una relación algebraica entre la fuerza en los conectores y la velocidad relativa entre ellos.
2. El *almacenamiento de energía potencial* en un muelle o muelle de torsión. Existe una relación algebraica entre la fuerza en los conectores, y la distancia (muelle) o ángulo (muelle de torsión) entre ellos.
3. El *almacenamiento de energía cinética* en una masa que se desplaza o que gira alrededor de un eje fijo. Existe una relación algebraica entre la fuerza en el conector y la derivada respecto al tiempo de la velocidad (aceleración) del conector.

A continuación veremos cómo emplear estos conceptos para plantear modelos de algunos sistemas mecánicos sencillos y cómo describirlos en lenguaje Modelica.

2.5.1. Ejemplo 1: traslación en una dirección

Consideremos el sistema mostrado en la Figura 2.24. Está compuesto por el suelo, un objeto que desliza con rozamiento sobre el suelo, y un segundo objeto que desliza con rozamiento sobre el primero. Se supone que la fuerza de fricción es proporcional a la diferencia entre las velocidades de las dos superficies. Los objetos tienen masas constantes m_1 y m_2 . El objetivo es desarrollar un modelo que describa la evolución de la velocidad y la posición de los objetos.

En la Sección 2.4.1 se describió un procedimiento sistemático para modelar circuitos eléctricos. El primer paso era escoger un nodo del circuito como referencia para el voltaje. Se procede de manera análoga al modelar sistemas mecánicos, debiéndose escoger una referencia y un criterio de signos para la velocidad. En este caso escogeremos como referencia la velocidad del suelo, a la que asignaremos valor cero. Llamemos v_1 y v_2 a la velocidad del primer y segundo objeto respectivamente, medidas ambas respecto al suelo. Respecto al criterio de signos, establecemos que en nuestro modelo un objeto tendrá velocidad positiva cuando se desplace hacia la derecha y negativa cuando se desplace hacia la izquierda.

Para calcular la evolución de las velocidades podemos aplicar la segunda ley de Newton, según la cual la variación en el momento lineal experimentada por un cuerpo es igual a la suma de las fuerzas aplicadas sobre él.

$$\frac{d}{dt}(m \cdot v) = F \quad (2.43)$$

Cuando la masa del cuerpo permanece constante, la Ec. (2.43) puede escribirse de la forma siguiente:

$$m \cdot \frac{dv}{dt} = F \quad (2.44)$$

Particularicemos la Ec. (2.44) para cada uno de los dos cuerpos. Sobre el primer cuerpo actúa la fuerza de rozamiento con el suelo y la fuerza de rozamiento con el segundo objeto. Sobre el segundo objeto únicamente actúa la fuerza de rozamiento con el primer objeto.

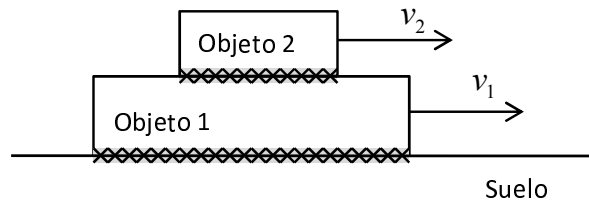


Figura 2.24: Dos objetos deslizantes con fricción.

```

model DosObjetosDeslizantes
  import SI = Modelica.SIunits;
  parameter SI.Mass m1=100 "Masa del objeto 1";
  parameter SI.Mass m2=10 "Masa del objeto 2";
  parameter SI.TranslationalDampingConstant b_s1=1.8
    "Coef. fricción suelo-obj.1";
  parameter SI.TranslationalDampingConstant b_12=1.6 "Coef. fricción ob.1-ob.2";
  SI.Position x1(start=0, fixed=true) "Posición del objeto 1";
  SI.Position x2(start=0, fixed=true) "Posición del objeto 2";
  SI.Velocity v1(start=0.1, fixed=true) "Velocidad del objeto 1";
  SI.Velocity v2(start=0, fixed=true) "Velocidad del objeto 2";
equation
  // Objeto 1
  der(x1) = v1;
  m1*der(v1) = -b_s1*v1 - b_12*(v1 - v2);
  // Objeto 2
  der(x2) = v2;
  m2*der(v2) = -b_12*(v2 - v1);
end DosObjetosDeslizantes;
    
```

Código 2.9: Modelo de los dos objetos deslizantes de la Figura 2.24.

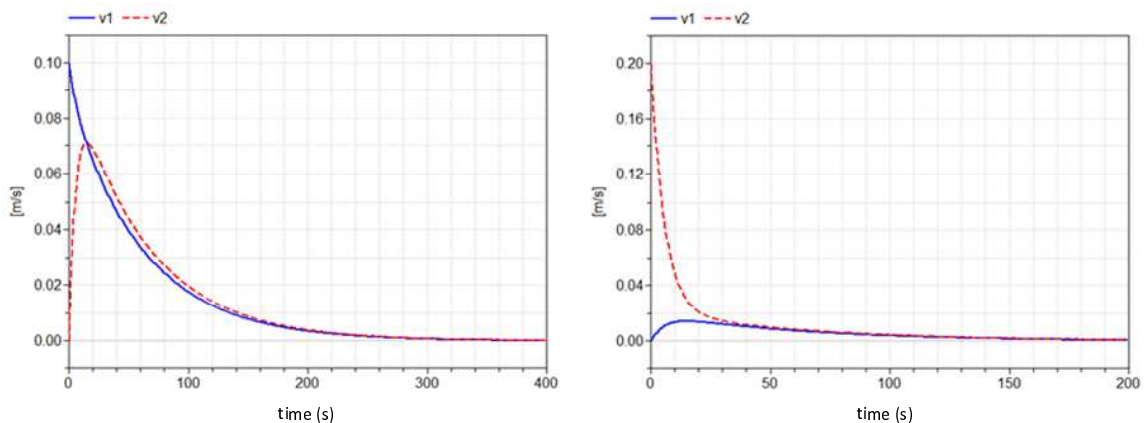


Figura 2.25: Simulación del Código 2.9 con dos condiciones iniciales diferentes: $v_1(0) = 0.1$ m/s, $v_2(0) = 0$ (izquierda); $v_1(0) = 0$, $v_2(0) = 0.2$ m/s (derecha).

$$m_1 \cdot \frac{dv_1}{dt} = \underbrace{-b_{s,1} \cdot v_1}_{\text{Fuerza ejercida por el suelo sobre el obj.1}} \underbrace{-b_{1,2} \cdot (v_1 - v_2)}_{\text{Fuerza ejercida por el obj.2 sobre el obj.1}} \quad (2.45)$$

$$m_2 \cdot \frac{dv_2}{dt} = \underbrace{-b_{1,2} \cdot (v_2 - v_1)}_{\text{Fuerza ejercida por el obj.1 sobre el obj.2}} \quad (2.46)$$

Los parámetros $b_{s,1}$ y $b_{1,2}$ son los coeficientes de proporcionalidad que relacionan la fuerza de rozamiento y la velocidad relativa entre las superficies.

Obsérvese que, conocida la velocidad de los objetos en el instante inicial de la simulación, las Ecs. (2.45) y (2.46) permiten calcular el valor de las velocidades en los sucesivos instantes de tiempo.

El siguiente paso en el desarrollo del modelo es incluir las ecuaciones que relacionan las posiciones de los objetos con sus velocidades. Para ello es preciso escoger la referencia y el criterio de signos de las coordenadas espaciales.

El criterio de signos para la posición suele escogerse en consonancia con el criterio de signos de la velocidad, de manera que el valor de la coordenada espacial del objeto aumente cuando la velocidad sea positiva y disminuya cuando la velocidad sea negativa. Puesto que la velocidad es positiva cuando el objeto se desplaza hacia la derecha, el valor de las coordenadas espaciales en nuestro modelo crecen al desplazarnos a la derecha. De esta manera, la relación entre las velocidades (v_1 , v_2) de los objetos y sus coordenadas espaciales (x_1 , x_2) es:

$$\frac{dx_1}{dt} = v_1 \quad (2.47)$$

$$\frac{dx_2}{dt} = v_2 \quad (2.48)$$

Conocida la posición de los objetos en el instante inicial de la simulación, las Ecs. (2.47) y (2.48) permiten calcular las posiciones en los sucesivos instantes de tiempo.

Asignar valor inicial a las posiciones conlleva seleccionar el origen de coordenadas para la posición de cada objeto. Esto debe hacerse de manera que se simplifique lo más posible el posterior análisis de los resultados de la simulación. Por ejemplo, se podría seleccionar un determinado punto del suelo como origen de coordenadas para

ambos objetos y medir la posición de cada objeto respecto a él. Otra posibilidad sería asignar valor cero a la posición inicial de ambos objetos. De esta forma, la coordenada espacial mide el desplazamiento del objeto respecto a su posición inicial.

Las Ecs. (2.45)–(2.48) constituyen el modelo del sistema. Conocidos los valores constantes de las masas (m_1 , m_2) y de los coeficientes de rozamiento ($b_{s,1}$, $b_{1,2}$), así como los valores iniciales de las posiciones y las velocidades, el modelo permite calcular las posiciones y las velocidades de los objetos en los sucesivos instantes de tiempo.

La descripción en lenguaje Modelica del modelo es el Código 2.9. En la Figura 2.25 se muestra la evolución de las velocidades de los objetos para dos condiciones iniciales diferentes. El comportamiento observado se explica considerando que el rozamiento con el suelo frena continuamente el Objeto 1 hasta detenerlo, y que el rozamiento entre los Objetos 1 y 2 hace que sus velocidades tiendan a igualarse.

En la primera simulación el Objeto 2 se encuentra inicialmente en reposo y el Objeto 1 en movimiento. La fuerza de rozamiento entre ambos acelera el Objeto 2, haciendo que su velocidad aumente. Cuando la velocidad de los dos objetos es la misma, la fuerza de rozamiento entre ellos es cero. Sin embargo, el suelo continúa frenando al Objeto 1, con lo cual la velocidad del Objeto 2 se hace mayor que la del Objeto 1. Entonces vuelve a actuar la fuerza de rozamiento entre ambos objetos, pero en este caso, en lugar de acelerar el Objeto 2, lo frena. En la segunda simulación se produce la situación inversa: el Objeto 1 está inicialmente en reposo y es acelerado por la fuerza de rozamiento con el Objeto 2.

Complicuemos ahora un poco el sistema a modelar, añadiendo algunos elementos. Consideremos el sistema mostrado en la Figura 2.26, que consiste en dos objetos de masa constante m_1 y m_2 , que deslizan con rozamiento sobre el suelo, y sobre los cuales desliza con rozamiento un tercer objeto de masa constante m_3 . Los Objetos 1 y 2 se encuentran unidos a las paredes laterales mediante muelles y están unidos entre sí mediante un amortiguador. Sobre el Objeto 3 actúa una fuerza sinusoidal conocida F . Las paredes están en reposo respecto al suelo. El modelo debe describir la evolución en el tiempo de la velocidad y posición de los objetos.

El primer paso consiste en especificar la referencia y el criterio de signos para las velocidades. Escogemos como velocidad de referencia la del suelo y las paredes, y asumiremos que tienen velocidad cero. Consideraremos que un objeto tiene velocidad positiva cuando se desplaza hacia la derecha. Adoptaremos el mismo criterio de signos para la fuerza: es positiva cuando está orientada hacia la derecha.

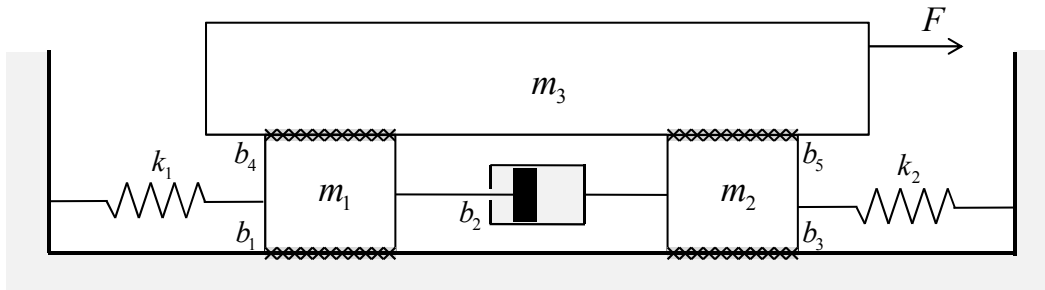


Figura 2.26: Tres objetos deslizantes con fricción y dos muelles.

```

model TresObjetosDeslizantes
  import SI = Modelica.SIunits;
  parameter SI.Mass m1=100 "Masa del objeto 1";
  parameter SI.Mass m2=10 "Masa del objeto 2";
  parameter SI.Mass m3=15 "Masa del objeto 2";
  parameter SI.TranslationalDampingConstant b1=1.8 "Obj.1 - Suelo";
  parameter SI.TranslationalDampingConstant b2=3.2 "Obj.1 - Obj.2";
  parameter SI.TranslationalDampingConstant b3=1.8 "Obj.2 - Suelo";
  parameter SI.TranslationalDampingConstant b4=1.6 "Obj.1 - Obj.3";
  parameter SI.TranslationalDampingConstant b5=1.6 "Obj.2 - Obj.3";
  parameter SI.TranslationalSpringConstant k1=1.2 "Muelle 1";
  parameter SI.TranslationalSpringConstant k2=1.2 "Muelle 2";
  parameter SI.Force F0=10 "Amplitud de la fuerza externa";
  parameter SI.AngularFrequency w=0.1 "Frecuencia de la fuerza externa";
  SI.Position x1(start=0, fixed=true) "Posición del objeto 1";
  SI.Position x2(start=0, fixed=true) "Posición del objeto 2";
  SI.Position x3(start=0, fixed=true) "Posición del objeto 3";
  SI.Velocity v1(start=0, fixed=true) "Velocidad del objeto 1";
  SI.Velocity v2(start=0, fixed=true) "Velocidad del objeto 2";
  SI.Velocity v3(start=0, fixed=true) "Velocidad del objeto 3";
  SI.Length e1(start=1, fixed=true) "Muelle 1: elong - elongNatural";
  SI.Length e2(start=0, fixed=true) "Muelle 2: elong - elongNatural";
  SI.Force F "Fuerza externa sobre Obj.3";
equation
  // Objeto 1
  der(x1) = v1;
  m1*der(v1) = -b1*v1 - b4*(v1 - v3) - b2*(v1 - v2) - k1*e1;
  // Objeto 2
  der(x2) = v2;
  m2*der(v2) = -b3*v2 - b5*(v2 - v3) - b2*(v2 - v1) + k2*e2;
  // Objeto 3
  der(x3) = v3;
  m3*der(v3) = -b4*(v3 - v1) - b5*(v3 - v2) + F;
  // Muelle 1
  der(e1) = v1;
  // Muelle 2
  der(e2) = -v2;
  // Fuerza externa aplicada sobre el Objeto 3
  F = F0*sin(w*time);
end TresObjetosDeslizantes;

```

Código 2.10: Modelo de los tres objetos deslizantes de la Figura 2.26.

La segunda ley de Newton permite relacionar el cambio en el momento lineal de cada uno de los objetos con las fuerzas que actúan sobre él.

$$m_1 \cdot \frac{dv_1}{dt} = \underbrace{-b_1 \cdot v_1}_{\text{Fricción con el suelo}} \underbrace{-b_4 \cdot (v_1 - v_3)}_{\text{Fricción con el obj.3}} \underbrace{-b_2 \cdot (v_1 - v_2)}_{\text{Fuerza del amortiguador}} \underbrace{-k_1 \cdot e_1}_{\text{Fuerza del muelle 1}} \quad (2.49)$$

$$m_2 \cdot \frac{dv_2}{dt} = \underbrace{-b_3 \cdot v_2}_{\text{Fricción con el suelo}} \underbrace{-b_5 \cdot (v_2 - v_3)}_{\text{Fricción con el obj.3}} \underbrace{-b_2 \cdot (v_2 - v_1)}_{\text{Fuerza del amortiguador}} \underbrace{+k_2 \cdot e_2}_{\text{Fuerza del muelle 2}} \quad (2.50)$$

$$m_3 \cdot \frac{dv_3}{dt} = \underbrace{-b_4 \cdot (v_3 - v_1)}_{\text{Fricción con el obj.1}} \underbrace{-b_5 \cdot (v_3 - v_2)}_{\text{Fricción con el obj.2}} \underbrace{+F}_{\text{Fuerza externa}} \quad (2.51)$$

donde e representa la diferencia entre la elongación actual del muelle (distancia entre sus extremos) y su elongación natural. El Muelle 1 tiene el extremo izquierdo fijo y el extremo derecho se mueve con velocidad v_1 . El Muelle 2 tiene el extremo derecho fijo y el extremo izquierdo se mueve con velocidad v_2 . Por tanto:

$$\frac{de_1}{dt} = v_1 \quad (2.52)$$

$$\frac{de_2}{dt} = -v_2 \quad (2.53)$$

La relación entre la posición de los objetos y sus velocidades viene dada por las ecuaciones siguientes:

$$\frac{dx_1}{dt} = v_1 \quad (2.54)$$

$$\frac{dx_2}{dt} = v_2 \quad (2.55)$$

$$\frac{dx_3}{dt} = v_3 \quad (2.56)$$

El modelo del sistema está compuesto por las Ecs. (2.49)–(2.56), más el valor constante conocido de las masas (m_1, m_2, m_3), los eficientes de los muelles (k_1, k_2), el coeficiente de fricción viscosa del amortiguador (b_2) y los coeficientes de fricción seca (b_1, b_3, b_4, b_5).

Para inicializar el modelo necesitamos conocer el valor inicial de las velocidades y posiciones de los tres objetos, y la elongación inicial de los muelles con respecto a su elongación natural. Tomaremos como origen de coordenadas para medir el desplazamiento de cada objeto su posición en el instante inicial, por ello el valor inicial de la posición de los tres objetos es cero. De esta manera, el valor de la variable x de cada objeto indica su desplazamiento respecto a su posición en el instante inicial de la simulación. La descripción en lenguaje Modelica del modelo se muestra en el Código 2.10.

2.5.2. Ejemplo 2: traslación en el plano

Los objetos de los dos sistemas mecánicos modelados anteriormente se mueven en una única dirección, por lo cual una vez fijados los sistemas de referencia basta con un número para especificar la posición de un objeto y un número para especificar su velocidad. En estos modelos las variables posición y velocidad son unidimensionales. Las variables unidimensionales se denominan **variables escalares**.

Consideremos ahora el modelado del movimiento de un objeto en el plano. En concreto, se trata de plantear un modelo sencillo del movimiento de la tierra alrededor del sol. Para ello, en el plano en el cual se mueve la tierra, definiremos un sistema de coordenadas rectangular, con el sol fijo en el origen. Representemos la posición y velocidad de la tierra mediante dos vectores de dos componentes, a los que llamaremos respectivamente $\mathbf{x}$ y $\mathbf{v}$ (véase la Figura 2.27).

La magnitud de la fuerza gravitatoria entre la tierra y el sol es igual al producto de la constante gravitatoria (G) por la masa del sol (M) y por la masa de la tierra (m), dividido por el cuadrado de la distancia que separa ambos cuerpos. Esta fuerza actúa sobre el sol, atrayéndolo hacia la tierra, y sobre la tierra, atrayéndola hacia el sol. La fuerza que actúa sobre la tierra tiene en todo momento la dirección de la línea que pasa por el sol y la tierra, y el sentido apuntando hacia el sol. Representemos esta fuerza mediante un vector de dos componentes al que llamaremos $\mathbf{F}$ (véase nuevamente la Figura 2.27).

La fuerza gravitatoria entre el sol y la tierra está descrita por la Ec. (2.57). La segunda ley de Newton permite relacionar la aceleración de la tierra y la fuerza gravitatoria, tal como se muestra en la Ec. (2.58). La Ec. (2.59) relaciona la velocidad de la tierra con su posición.

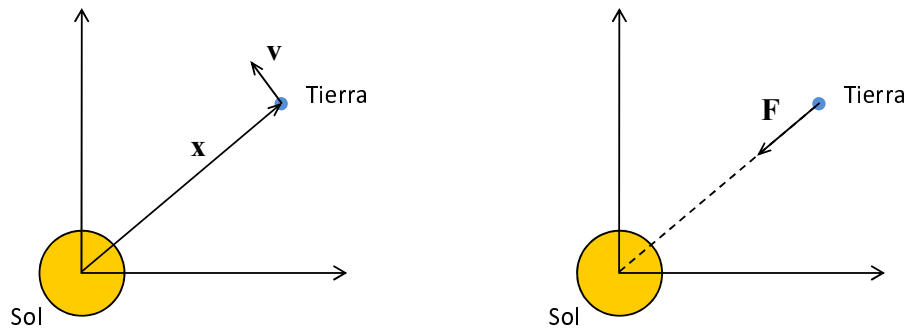


Figura 2.27: Posición y velocidad de la tierra (figura izquierda), y fuerza gravitatoria que actúa sobre la tierra (figura derecha).

```

model orbitaTerrestre
  import SI = Modelica.SIunits;
  SI.Position x[2](start={152.1e9,0}, fixed=true) "Posición tierra";
  SI.Velocity v[2](start={0,29.29e3}, fixed=true) "Velocidad tierra";
  SI.Force F[2] "Fuerza gravitatoria sobre la tierra";
  parameter SI.Mass m=5.976e24 "Masa de la tierra";
  parameter SI.Mass M=1.989e30 "Masa del sol";
equation
  // Fuerza gravitatoria que actúa sobre la tierra
  F = -(Modelica.Constants.G*m*M/(x*x))*(x/sqrt(x*x));
  // Posición y velocidad de la tierra
  m*der(v) = F;
  der(x) = v;
end orbitaTerrestre;
    
```

Código 2.11: Modelo de la órbita terrestre.

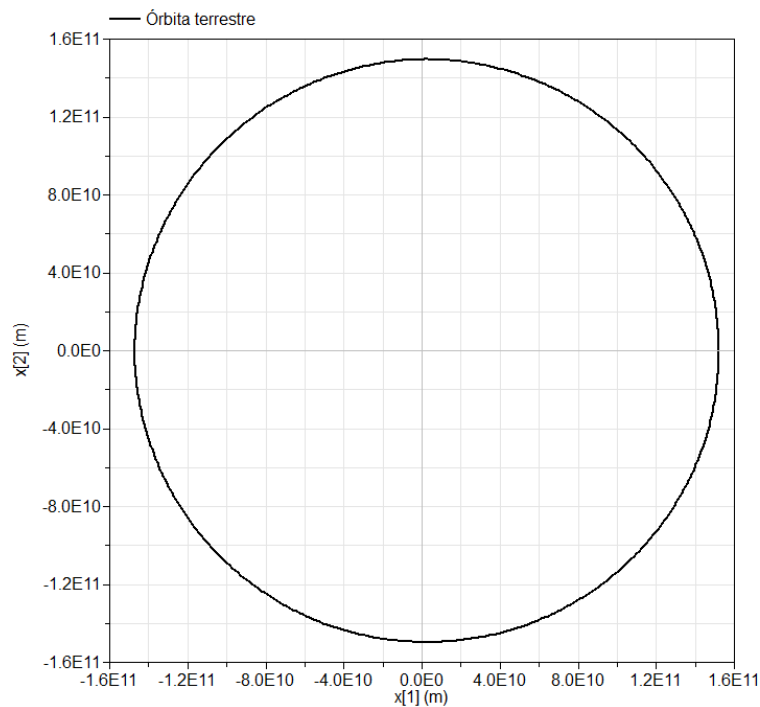


Figura 2.28: Posición de la tierra obtenida simulando el Código 2.11 durante un año.

$$\mathbf{F} = -G \cdot \frac{M \cdot m}{\mathbf{x} \cdot \mathbf{x}} \cdot \frac{\mathbf{x}}{|\mathbf{x}|} \quad (2.57)$$

$$m \cdot \frac{d\mathbf{v}}{dt} = \mathbf{F} \quad (2.58)$$

$$\frac{d\mathbf{x}}{dt} = \mathbf{v} \quad (2.59)$$

Para completar el planteamiento del modelo es preciso asignar valor a la masa de la tierra (m) y del sol (M). Supondremos que tienen los siguientes valores constantes: $m = 5.976 \cdot 10^{24}$ kg y $M = 1.989 \cdot 10^{30}$ kg. Para inicializar el modelo es preciso asignar valor a la posición inicial de la tierra y a su velocidad inicial. Supondremos que inicialmente la tierra se encuentra en la posición $\mathbf{x}(0) = \{152.1 \cdot 10^9, 0\}$ m y que se desplaza con velocidad $\mathbf{v}(0) = \{0, 29.29 \cdot 10^3\}$ m/s.

La descripción en Modelica de este modelo puede hacerse empleando variables escalares y escribiendo las ecuaciones en términos de los componentes de las magnitudes vectoriales. Otra posibilidad es emplear variables vectoriales para representar las magnitudes vectoriales y describir la relaciones entre ellas empleando ecuaciones vectoriales, tal como se muestra en las Ecs. (2.57)–(2.59).

La declaración de variables vectoriales y matriciales puede realizarse en Modelica indicando (indistintamente a continuación del tipo o del nombre de la variable) la dimensión y el número de componentes, por ejemplo,

<code>Real[2] x, v, F;</code> <code>Real[6,2,10] tabla;</code>		<code>Real x[2], v[2], F[2];</code> <code>Real tabla[6,2,10];</code>
---	--	---

donde las variables $\mathbf{x}$, $\mathbf{v}$ y $\mathbf{F}$ son **variables vectoriales** (su dimensión es uno) de dos componentes y $\mathbf{tabla}$ es una **variable matricial** de 3 dimensiones, con 6 componentes en la primera dimensión, 2 componentes en la segunda y 10 componentes en la tercera.

También es posible declarar las variables vectoriales o matriciales indicando sólo su dimensión, por ejemplo,

<code>Real[:] x, v, F;</code> <code>Real[:, :, :] tabla;</code>		<code>Real x[:], v[:], F[:];</code> <code>Real tabla[:, :, :];</code>
--	--	--

si bien en este caso **el entorno de modelado debe poder deducir el número de componentes al realizar la partición**. El entorno de modelado obtiene esta información a partir de la dimensión de las ecuaciones vectoriales o matriciales en

las que interviene la variable. En cualquier caso, ni el número de componentes, ni la dimensión de una variable vectorial o matricial puede modificarse durante la simulación.

Para referirse a un elemento de un vector o matriz, se especifica el valor de sus índices. Por ejemplo, `x[i]` es el componente *i*-ésimo del vector, **comenzando por uno**. Por ejemplo, si el vector `x` tiene dos componentes, estos son `x[1]` y `x[2]`.

Es posible especificar subvectores o submatrices indicando el **rango de los índices**. Por ejemplo, `A[i1:i2, j1:j2]` especifica una submatriz compuesta de las filas comprendidas entre `i1` e `i2` ambas inclusive, y las columnas comprendidas entre `j1` y `j2` ambas inclusive.

En la Tabla 2.7 se muestran algunas funciones predefinidas en el lenguaje Modelica que son útiles para trabajar con vectores y matrices.

Las ecuaciones en las cuales intervienen variables vectoriales y matriciales son interpretadas como ecuaciones vectoriales o matriciales, respectivamente. En la Tabla 2.8 pueden verse algunos ejemplos.

Las explicaciones anteriores sobre la declaración y uso de las variables vectoriales permiten describir el modelo de la órbita terrestre. Éste se muestra en el Código 2.11. Simulando el modelo anterior durante un año ($=3.1536e7$ s), se obtiene la trayectoria mostrada en la Figura 2.28.

Obsérvese en el Código 2.11 cómo se ha asignado el valor inicial de las variables vectoriales: asignando al atributo *start* el vector de valores iniciales. El vector se escribe englobando la secuencia de valores entre llaves y separando mediante coma los valores consecutivos. Además de la anterior, Modelica proporciona otras formas de asignar valores numéricos a vectores y matrices. En la Tabla 2.9 se muestran algunos ejemplos.

También puede emplearse una **expresión for con iteradores** para asignar valor a un vector o matriz. La sintaxis es:

```
{ expresión for iteradores }
```

donde deben definirse tantos iteradores como dimensión tenga el vector o la matriz. En la Tabla 2.10 se muestran algunos ejemplos.

Volviendo nuevamente al Código 2.11, obsérvese que en este caso la traducción de las ecuaciones del modelo a su descripción en Modelica como ecuaciones vectoriales ha sido trivial:

Tabla 2.7: Algunas funciones predefinidas en Modelica para vectores y matrices.

Invocación	Funcionalidad
<code>ndims(A)</code>	Número de dimensiones de A
<code>size(A, i)</code>	Número componentes de A en la dimensión i
<code>size(A)</code>	Vector de longitud <code>ndims(A)</code> , con el número de elementos de A en cada dimensión.
<code>diagonal(v)</code>	Genera una matriz cuadrada con el vector v en la diagonal.
<code>fill(s,n1,n2,...)</code>	Genera una matriz $n_1 \times n_2 \times \dots$ llena con el valor s .
<code>transpose(A)</code>	Permuta las dos primeras dimensiones de A .
<code>zeros(n1,n2,...)</code>	Genera una matriz $n_1 \times n_2 \times \dots$ y la llena con ceros.
<code>ones(n1,n2,...)</code>	Genera una matriz $n_1 \times n_2 \times \dots$ y la llena con unos.
<code>identity(n)</code>	Crea una matriz identidad $n \times n$.
<code>linspace(x1,x2,n)</code>	Crea un vector con n elementos, con valores equiespaciados entre x_1 y x_2 .
<code>min(A), max(A)</code>	Devuelve el valor mínimo/máximo de A .
<code>sum(A), product(A)</code>	Devuelve la suma/producto de los elementos A .
<code>cross(x,y)</code>	Producto vectorial de vectores de tres componentes x, y .

Tabla 2.8: Ejemplos de ecuaciones vectoriales y matriciales.

Fragmento de código Modelica	Comentario al código
<pre> Real A[3,2], B[2,4]; Real C[3,4]; equation C = A * B;</pre>	Como A , B y C son matrices, la ecuación $C = A * B$ es interpretada como una ecuación matricial.
<pre> Real x[3], y[3]; Real z; equation z = x * y;</pre>	El producto entre vectores es interpretado como producto escalar.
<pre> Real x[3], y[3]; Real z; equation z = cross (x,y);</pre>	La función <i>cross</i> realiza el producto vectorial de dos vectores de tres componentes (véase la Tabla 2.7).

Tabla 2.9: Ejemplos de asignación de valores a vectores y matrices.

parameter Real z[:] = {0.1, 0.3, 0.5, 0.7, 0.9}; parameter Real z[:] = 0.1 : 0.2 : 0.9;
parameter Integer puntos[:] = 2:2:10;
parameter Real x[2,3] = { {1,2,3} , {4,5,6} }; parameter Real x[2,3] = { 1:3 , 4:6 }; parameter Real x[2,3] = [1, 2, 3 ; 4, 5, 6];
parameter Real y[2,3,4] = { { {10,20,30,40},{50,60,70,80},{90,10,11,12} }, { {11,21,31,41},{51,61,71,81},{91,11,12,13} } };

Tabla 2.10: Ejemplos de uso de *expresión for con iteradores*.

Fragmento de código Modelica	Comentario al código
parameter Real x[:] = { i^2 for i in 1:5 };	Asigna a x el vector: $\{1, 4, 9, 16, 25\}$
parameter Real A[:,:] = { i^2 for i in 1:n, j in 1:m };	Asigna a A la matriz: $\begin{pmatrix} 1 & \dots & 1 \\ 4 & \dots & 4 \\ \dots & \dots & \dots \\ n^2 & \dots & n^2 \end{pmatrix}_{n \times m}$
parameter Real B[:,:] = { if i == j then i else 0 for i in 1:n, j in 1:n };	Asigna a B la matriz: $\begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 2 & 0 & \dots & 0 \\ 0 & 0 & 3 & \dots & 0 \\ \dots & & & \dots & \\ 0 & 0 & 0 & \dots & n \end{pmatrix}_{n \times n}$

$$\begin{aligned}
\mathbf{F} &= -G \cdot \frac{m \cdot M}{\mathbf{x} \cdot \mathbf{x}} \cdot \frac{\mathbf{x}}{|\mathbf{x}|} &\rightarrow & \text{F} = - \text{Modelica.Constants.G} * \text{m} * \text{M} / (\text{x} * \text{x}) * \\
& & & \text{x} / \text{sqrt}(\text{x} * \text{x}); \\
m \cdot \frac{d\mathbf{v}}{dt} &= \mathbf{F} &\rightarrow & \text{m} * \text{der}(\text{v}) = \text{F}; \\
\frac{d\mathbf{x}}{dt} &= \mathbf{v} &\rightarrow & \text{der}(\text{x}) = \text{v};
\end{aligned}$$

Sin embargo, en algunas ocasiones la traducción del modelo a ecuaciones vectoriales y matriciales no resulta tan directa. En estos casos, resulta más sencillo describir las ecuaciones en las que intervienen vectores y matrices mediante las *cláusulas for* e *if* de Modelica. Ambas cláusulas pueden ser usadas en *secciones equation* y *algorithm*.

Las **cláusulas for** son traducidas por el entorno de modelado antes de realizar la partición a conjuntos de ecuaciones o asignaciones, según se usen en una *sección equation* o *algorithm*. En general, la sintaxis de una *cláusula for* es:

```

for for_indices loop
  ...
end for;

```

En la Tabla 2.11 se muestra un ejemplo. La ecuación de la izquierda puede ser descrita mediante el código Modelica mostrado a la derecha. Obsérvese que los componentes del vector auxiliar **a** almacenan las sucesivas potencias de **x**.

La *cláusula for* también puede emplearse en una **sección algorithm**. En la Tabla 2.12 se muestran algunos ejemplos. En la columna izquierda de muestra código Modelica con ecuaciones vectoriales. En la columna derecha se muestran sus descripciones equivalentes mediante algoritmos.

Como se indicó en la Sección 2.2.1, las *secciones algorithm* están compuestas por asignaciones, que son evaluadas por el entorno de modelado en el mismo orden en que aparecen escritas. Una variable puede ser evaluada varias veces dentro de una misma *sección algorithm*. Sin embargo, una variable evaluada en una *sección algorithm* no puede ser reevaluada en otra *sección algorithm*.

Al realizar la partición, el entorno de modelado ordena cada *sección algorithm* como un conjunto indivisible, junto con el resto del modelo. El procedimiento que sigue para ello es básicamente el siguiente. Si de una *sección algorithm* se evalúan n variables, al hacer la partición sustituye dicha sección por un conjunto de n ecuaciones implícitas, todas dependientes de las n variables y de las demás variables usadas a la derecha de las asignaciones. Consideremos, por ejemplo, la *sección algorithm* siguiente:

```

algorithm
  x1 := g1(y1, ..., ym);

```


Tabla 2.11: Ejemplo de uso de *cláusula for* en sección *equation*.

Ecuación	Fragmento de código Modelica
$y = \sum_{i=0}^n c_{i+1} \cdot x^i$	<pre> parameter Real c[:]; Real x, y; parameter Integer n = size(c,1) - 1; Real a[n+1]; equation a[1] = 1; for i in 1:n loop a[i+1] = a[i] * x; end for; y = c * a; </pre>

Tabla 2.12: Ejemplos de ecuaciones vectoriales y sus algoritmos equivalentes.

Ecuación vectorial	Algoritmo equivalente
<pre> Real x[3], y[3]; Real z; equation y = z * x; </pre>	<pre> Real x[3], y[3]; Real z; algorithm for i in 1:size(x,1) loop y[i] := z * x[i]; end for; </pre>
<pre> Real x[3], y[3]; Real z; equation z = x * y; </pre>	<pre> Real x[3], y[3]; Real z; algorithm z := 0; for i in 1:size(x,1) loop z := z + y[i] * x[i]; end for; </pre>
<pre> Real x[3], y[4]; Real A[3,4]; equation x = A * y; </pre>	<pre> Real x[3], y[4], A[3,4]; algorithm for i in 1:size(A,1) loop x[i] := 0; for j in 1:size(A,2) loop x[i] = x[i] + A[i,j]*y[j]; end for; end for; </pre>

...
 $\mathbf{x}_n := \mathbf{g}_n(\mathbf{y}_1, \dots, \mathbf{y}_m);$

Las entradas computacionales son $y_1, \dots, y_m$, y las salidas computacionales son $x_1, \dots, x_n$. Al hacer la partición, esta *sección algorithm* es sustituida por las n ecuaciones siguientes:

$$\begin{aligned} f_1(x_1, \dots, x_n, y_1, \dots, y_m) &= 0 \\ \dots & \\ f_n(x_1, \dots, x_n, y_1, \dots, y_m) &= 0 \end{aligned} \tag{2.60}$$

Este lazo algebraico permanece junto durante la ordenación y puede, una vez ordenado el sistema completo, ser reemplazado por una llamada al algoritmo. Volvemos sobre ello en el Tema 3.

2.5.3. Ejemplo 3: vibración longitudinal de una varilla

Supongamos que se desea estudiar la vibración longitudinal de una varilla sometida en uno de sus extremos a una fuerza, mientras que el otro extremo se mantiene fijo. El sistema se muestra en la Figura 2.29, donde $\xi(x, t)$ representa el desplazamiento respecto a su posición natural de la sección de la barra situada en x , en el instante t . $F(t)$ es la fuerza externa aplicada.

Los parámetros físicos que caracterizan la varilla son su sección transversal (A), longitud (L), masa (M) y módulo de Young (E). Para realizar el modelo de la varilla, se divide ésta en n elementos iguales, con longitud $\Delta x = L/n$ y masa $\Delta m = M/n$. El momento lineal del i -ésimo elemento es:

$$p_i = \Delta m \cdot \frac{d\xi_i}{dt} \quad \text{con } i = 1, \dots, n \tag{2.61}$$

La fuerza que actúa sobre cada elemento debida al estrés se modela mediante dos muelles, uno situado a cada lado del elemento. De esta forma, el modelo está compuesto por n masas iguales y n muelles iguales, tal como se muestra en la Figura 2.30. Aplicando la ley de Newton a cada una de las masas se obtiene:

$$\frac{dp_i}{dt} = F_{i+1} - F_i \quad \text{con } i = 1, \dots, n-1 \tag{2.62}$$

$$\frac{dp_n}{dt} = F - F_n \tag{2.63}$$

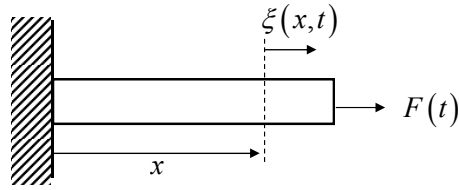
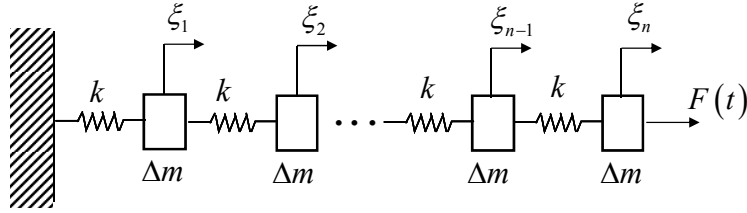


Figura 2.29: Varilla sometida a una fuerza en la dirección longitudinal.



$$\Delta m = \rho \cdot A \cdot \Delta x$$

$$k = \frac{E \cdot A}{\Delta x}$$

Figura 2.30: Modelo para describir la vibración longitudinal de la varilla.

donde F es la fuerza externa ejercida sobre el extremo de la varilla. Inicialmente, dicha la fuerza externa vale cero. En el instante 0.001 s, la fuerza aplicada pasa a valer 2000 N, manteniéndose constante.

$$F = \begin{cases} 0 & \text{si } t < 0.001 \text{ s} \\ 2000 & \text{si } t \geq 0.001 \text{ s} \end{cases} \quad (2.64)$$

La fuerza ejercida por los muelles se calcula de la forma siguiente:

$$F_1 = k \cdot \xi_1 \quad (2.65)$$

$$F_i = k \cdot (\xi_i - \xi_{i-1}) \quad \text{con } i = 2, \dots, n \quad (2.66)$$

donde:

$$k = \frac{E \cdot A}{\Delta x} \quad (2.67)$$

El modelo está descrito por las Ecs. (2.61) – (2.67). El valor de los parámetros físicos de la varilla se muestra en la Tabla 2.13.

```

record DatosVarilla
  import SI = Modelica.SIunits;
  parameter SI.Area seccion;
  parameter SI.Length longitud;
  parameter SI.Mass masa;
  parameter SI.Pressure modYoung;
end DatosVarilla;

package CatalogoVarillas

  record Var21 = DatosVarilla (
    seccion=8.636e-005,
    longitud=1,
    masa=0.233172,
    modYoung=6.9e10);

end CatalogoVarillas;

model varilla
  import SI = Modelica.SIunits;
  constant Integer n=100 "Número de elementos";
  DatosVarilla dVar;
  parameter SI.Length delta_x=dVar.longitud/n;
  parameter SI.Mass delta_m=dVar.masa/n;
  parameter SI.TranslationalSpringConstant k=dVar.modYoung*dVar.seccion/delta_x;
  SI.Position eps[n] "Desplazamiento";
  SI.Force F[n] "Fuerza muelles";
  SI.Momentum p[n] "Momento lineal";
  SI.Force Fext "Fuerza externa";
equation
  // Momento lineal elementos
  for i in 1:n loop
    p[i] = delta_m*der(eps[i]);
  end for;
  // Ley de Newton aplicada a elementos
  for i in 1:(n - 1) loop
    der(p[i]) = F[i + 1] - F[i];
  end for;
  der(p[n]) = Fext - F[n];
  // Fuerza muelles
  F[1] = k*eps[1];
  for i in 2:n loop
    F[i] = k*(eps[i] - eps[i - 1]);
  end for;
  // Fuerza externa aplicada
  Fext = if time < 1e-3 then 0 else 2e3;
end varilla;

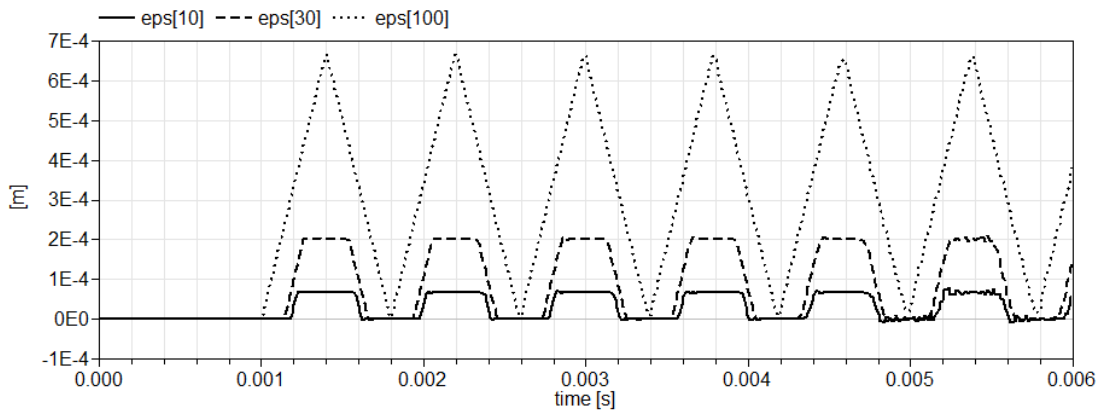
model Test
  extends varilla(dVar=CatalogoVarillas.Var21());
end Test;

```

Código 2.12: Modelo de la vibración longitudinal de una varilla.

Tabla 2.13: Parámetros físicos de la varilla.

Parámetro	Valor	Unidades
Sección transversal	$8.636E - 005$	m^2
Longitud	1.0	m
Masa	0.233172	kg
Módulo de elasticidad	$6.9E + 10$	Pa


 Figura 2.31: Desplazamiento de los elementos $i = 10, 30$ y 100 de la varilla.

La descripción en Modelica del modelo de la varilla se muestra en el Código 2.12. Obsérvese que en el modelo `varilla` se han empleado variables vectoriales para representar el desplazamiento y el momento lineal de las masas, así como la fuerza de los muelles, y que se han empleado *cláusulas for* para describir las ecuaciones que relacionan esas variables. Se ha simulado el modelo `Test` durante 0.006 segundos. En la Figura 2.31 se muestra el desplazamiento de los elementos $i = 10, 30$ y 100 .

Puede observarse en el Código 2.12 que se ha empleado una clase **record** llamada `DatosVarilla` para agrupar los parámetros físicos de la varilla. Esa es precisamente la finalidad de la clase *record*: agrupar valores de parámetros, de modo que de forma sencilla puedan ser asignados a un objeto. En la clase *record* no pueden incluirse ecuaciones, ni secciones *protected*.

La separación entre la descripción matemática del modelo y los valores numéricos de los parámetros es común en otros simuladores. Por ejemplo, en SPICE los modelos matemáticos de los dispositivos están incorporados en el propio simulador y para definir un dispositivo en concreto únicamente debe especificarse el valor de sus parámetros. Los fabricantes de dispositivos semiconductores (diodos, transistores, etc.) proporcionan los valores de los parámetros SPICE de los diferentes modelos de dispositivo que fabrican. De hecho, puesto que existen varios niveles de detalle en

la descripción de los dispositivos (nivel 1, nivel 2, etc.), el fabricante proporciona, para un mismo modelo de dispositivo, los parámetros del modelo Spice nivel 1, los del modelo nivel 2, etc.

La clase *record* de Modelica facilita esta separación entre las clases donde se define el modelo matemático y las clases donde se asigna valor a los parámetros. En el Código 2.12 se ha definido una clase *record* llamada `DatosVarilla`, donde se han declarado los parámetros físicos de la varilla. Dentro de la clase `varilla` se ha creado una instancia de la clase `DatosVarilla` llamada `dVar`.

Si van a estudiarse diferentes tipos de varillas, puede agruparse la declaración de sus parámetros en un *package*, al que en este caso se ha llamado `CatalogoVarillas`. Como puede observarse en el Código 2.12, sólo se ha definido un *record* dentro del *package* `CatalogoVarillas`, pero podrían haberse definido uno por cada tipo de varilla a estudiar.

Una vez definido, por una parte el modelo de la varilla, y por otra el conjunto de parámetros físicos de la varilla, pueden combinarse como se muestra en el modelo `Test`.

```
model Test
  extends varilla( dVar=CatalogoVarillas.Var21( ) );
end Test;
```

Es posible modificar el valor de uno o varios de los parámetros al realizar la asignación. Por ejemplo, si se hubiera definido el modelo `Test` de la forma:

```
model Test
  extends varilla( dVar=CatalogoVarillas.Var21(longitud=2) );
end Test;
```

la longitud de la varilla sería 2 m. Los demás parámetros tendrían los valores especificados en `CatalogoVarillas.Var21`: sección igual a $8.636 \times 10^{-5} \text{ m}^2$, masa igual a 0.233172 kg y módulo de Young igual a $6.9 \times 10^{10} \text{ Pa}$.

2.6. FLUJO DE FLUIDOS E INTERCAMBIO DE CALOR

En esta sección se muestran dos ejemplos de modelado de la conducción del calor en una dimensión. En el primero de ellos no se considera la inercia térmica de los materiales. En el segundo, sí se considera. Este segundo ejemplo servirá además para ilustrar la descripción de modelos con estructura regular mediante el empleo de vectores de componentes y *sentencias connect* definidas dentro de una *cláusula for*.

2.6.1. Ejemplo 1: flujo radial de calor en una tubería

Se pretende analizar mediante simulación la conducción radial en el estacionario en una tubería, con flujo de calor con el entorno por convección. El sistema se muestra en la Figura 2.32.

La tubería es cilíndrica y su longitud es $L = 1$ m. Por su interior circula vapor, que se encuentra a una temperatura constante $T_1 = 418$ K. El radio interno de la tubería es $r_1 = 0.025$ m y su radio externo es $r_2 = 0.035$ m. Por tanto, la pared de la tubería tiene un espesor de 0.01 m. La tubería está rodeada de aislante, cuyo espesor es 0.025 m. Se supone que la temperatura del entorno oscila a lo largo de 24 horas (=86400 s) entre 283 K y 300 K.

En la Tabla 2.14 se muestra el valor de la conductividad térmica de la pared de la tubería (κ_1) y del aislante (κ_2). También se muestra el valor del coeficiente de transferencia de calor por convección entre el vapor que circula por la tubería y ésta (h_1), y entre la superficie del aislante y el aire (h_2). El valor de este último depende de la diferencia de temperaturas entre la superficie del aislante y el aire del entorno, tal como se indica en la Tabla 2.14.

El modelo de la transmisión de calor en este sistema es el circuito térmico equivalente que se muestra en la Figura 2.32. Obsérvese la analogía entre la temperatura y el voltaje; el flujo de calor y la corriente eléctrica; y la resistencia eléctrica y la resistencia térmica. Llamando q al flujo de calor por segundo (unidades: W) desde el vapor interno a la tubería hasta el aire que rodea el aislante, el modelo está compuesto por las Ecs. (2.68) – (2.73), y por los valores de los coeficientes, conductividades y radios mostrados en la Tabla 2.14. Las temperaturas están expresadas en el modelo en grados Kelvin.

$$T_1 = 418 \quad (2.68)$$

$$T_1 - T_2 = q \cdot \frac{1}{h_1 \cdot 2 \cdot \pi \cdot r_1 \cdot L} \quad (2.69)$$

$$T_2 - T_3 = q \cdot \frac{\ln(r_2/r_1)}{2 \cdot \pi \cdot \kappa_1 \cdot L} \quad (2.70)$$

$$T_3 - T_4 = q \cdot \frac{\ln(r_3/r_2)}{2 \cdot \pi \cdot \kappa_2 \cdot L} \quad (2.71)$$

$$T_4 - T_5 = q \cdot \frac{1}{h_2 \cdot 2 \cdot \pi \cdot r_3 \cdot L} \quad (2.72)$$

$$T_5 = 291.5 - 8.5 \cdot \sin\left(\frac{2 \cdot \pi \cdot t}{86400}\right) \quad (2.73)$$

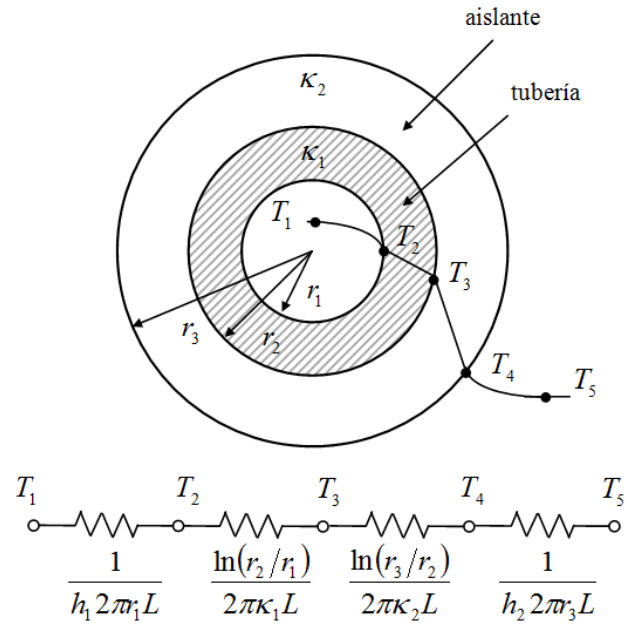


Figura 2.32: Esquema de la tubería y modelo de la transmisión de calor.

Tabla 2.14: Parámetros físicos del sistema mostrado en la Figura 2.32.

Componente	Parámetro	Valor	Unidades
Vapor/Tubería	Coef. convección	$h_1 = 11350$	$W/(m^2 \cdot K)$
Tubería	Radio interno	$r_1 = 0.025$	m
	Radio externo	$r_2 = 0.035$	m
	Conductividad térmica	$\kappa_1 = 45$	$W/(m \cdot K)$
Aislante	Radio externo	$r_3 = 0.06$	m
	Conductividad térmica	$\kappa_2 = 0.087$	$W/(m \cdot K)$
Aislante/Aire	Coef. convección	$h_2 = 1.32 \left(\frac{ T_4 - T_5 }{2 \cdot r_3} \right)^{0.25}$	$W/(m^2 \cdot K)$

El modelo puede describirse en lenguaje Modelica como se muestra en el Código 2.13. Las variables que describen la temperatura han sido declaradas del tipo *Modelica.SIunits.Temperature*. La declaración del tipo en la Modelica Standard Library version 3.2.1 es la siguiente:

```
type Temperature = ThermodynamicTemperature;
type ThermodynamicTemperature = Real (
  final quantity = "ThermodynamicTemperature",
  final unit = "K",
  min = 0.0,
  start = 288.15,
  nominal = 300,
  displayUnit = "degC" );
```

El atributo `unit` especifica las unidades empleadas para la variable en las ecuaciones del modelo, mientras que el atributo `displayUnit` indica las unidades que debe emplear el entorno de modelado al representar gráficamente los resultados de la simulación. Si se especifican diferentes unidades en estos atributos, algunos entornos de modelado realizarán automáticamente el cambio de unidades (conviene consultar la documentación del entorno de modelado). En este caso, la temperatura está expresada en grados Kelvin en el modelo y es representada en grados centígrados.

Las temperaturas obtenidas al simular el modelo durante 86400 s (el equivalente a un día), expresadas en grados centígrados, se muestran en la Figura 2.33. Obsérvese que el rango de temperaturas representado en el eje de ordenadas de ambas gráficas es diferente. Los mayores escalones térmicos se producen (como era de esperar) en el aislante, y entre la superficie del mismo y el aire del entorno.

Complicuemos un poco el modelo. Supongamos que el valor de la temperatura del entorno (T_5), en lugar de estar descrito por una función conocida del tiempo, debe ser leído de una tabla de datos meteorológicos que consta de dos columnas: en la primera se registra el instante del día en que se hace la medida y en la segunda el correspondiente valor medido de la temperatura, expresado en grados Kelvin. Los valores de la primera columna están ordenados crecientemente y expresados en segundos. Se toma como valor cero para el tiempo el inicio del periodo que va a ser estudiado mediante simulación.

Obsérvese que se plantea el problema siguiente. La tabla proporciona el valor de la temperatura únicamente en un conjunto determinado de instantes de tiempo. Sin embargo, para simular el modelo es necesario conocer la temperatura en un conjunto de instantes de tiempo que viene determinado por el algoritmo de la simulación del modelo. Una solución consiste en obtener mediante interpolación lineal de los datos

```

model calorTuberia
  import SI = Modelica.SIunits;
  import Modelica.Constants.pi;
  import Modelica.Math.log;

  parameter SI.Radius r1=0.025 "Radio interno tubería";
  parameter SI.Radius r2=0.035 "Radio externo tubería";
  parameter SI.Radius r3=0.06 "Radio externo aislante";
  parameter SI.Length L=1 "Longitud tubería";

  constant SI.ThermalConductivity k1=45 "Conductividad térmica tubería";
  constant SI.ThermalConductivity k2=0.087 "Conductividad térmica aislante";

  constant SI.CoefficientOfHeatTransfer h1=11350
    "Coef. convección vapor/tubería";
  SI.CoefficientOfHeatTransfer h2 "Coef. convección aislante/aire";

  parameter SI.Temperature T1=418 "Temperatura vapor";
  SI.Temperature T2 "Temperatura interna tubería";
  SI.Temperature T3 "Temperatura tubería/aislante";
  SI.Temperature T4 "Temperatura aislante/aire";
  SI.Temperature T5 "Temperatura aire ambiente";

  SI.HeatFlowRate q "Flujo calor por segundo del vapor al ambiente";
equation
  h2 = 1.32*(abs(T4 - T5)/(2*r3))^0.25;
  T1 - T2 = q/(h1*2*pi*r1*L);
  T2 - T3 = q*log(r2/r1)/(2*pi*k1*L);
  T3 - T4 = q*log(r3/r2)/(2*pi*k2*L);
  T4 - T5 = q/(h2*2*pi*r3*L);
  T5 = 291.5 - 8.5*sin(2*pi*time/86400);
end calorTuberia;

```

Código 2.13: Transmisión radial de calor en una tubería.

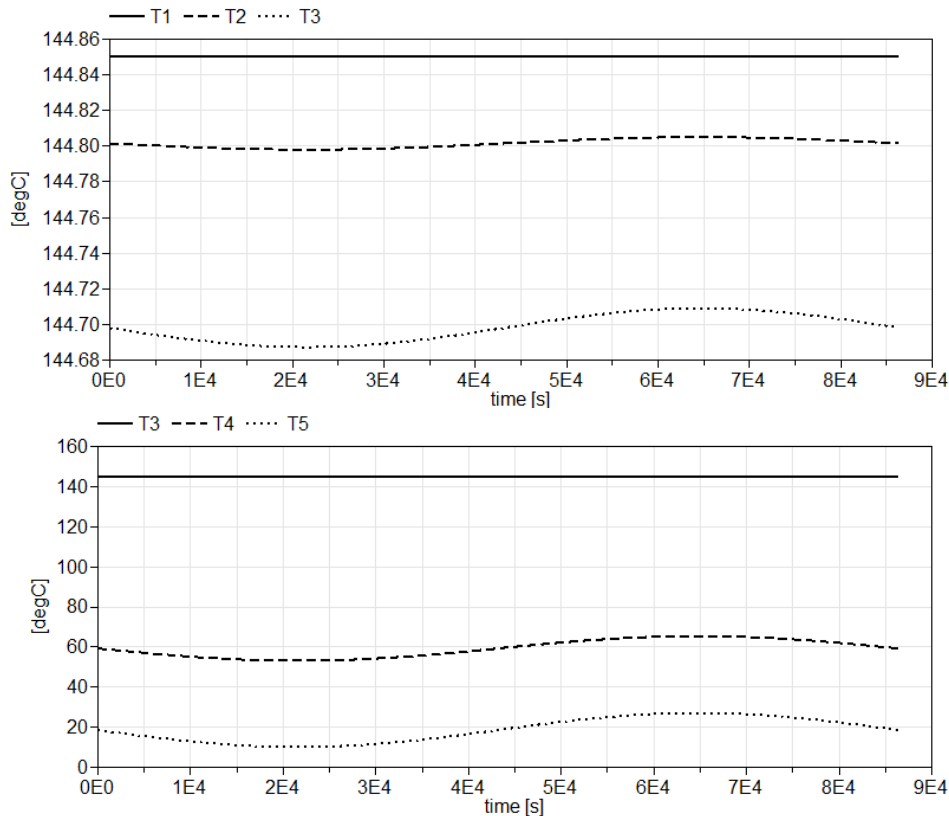


Figura 2.33: Resultado de simular el Código 2.13 durante 86400 s.

Tabla 2.15: Algunas funciones predefinidas en Modelica.

Invocación	Funcionalidad
<code>abs(v)</code>	Se expande a: <code>if v>=0 then v else -v</code>
<code>sign(v)</code>	Se expande a: <code>if v>0 then 1 elseif v<0 then -1 else 0</code>
<code>sqrt(v)</code>	Raíz cuadrada de v si $v \geq 0$. En caso contrario, se produce error
<code>div(x,y)</code>	Devuelve la parte entera del resultado de x/y
<code>rem(x,y)</code>	Devuelve el resto obtenido de la realizar división x/y

de la tabla el valor de la temperatura en aquellos instantes de tiempo para los cuales no se disponga de valor medido experimentalmente.

Para realizar el cálculo de la interpolación lineal emplearemos una función escrita en lenguaje Modelica. Esto nos permitirá además introducir a continuación el uso y definición de funciones en lenguaje Modelica.

Modelica proporciona un conjunto de **funciones predefinidas**. Algunas de las más comúnmente usadas se muestran en la Tabla 2.15. Los argumentos de estas funciones pueden ser variables escalares o vectoriales. En este último caso, se aplica la función a cada uno de los elementos del vector, devolviéndose un vector. Esta propiedad se denomina **vectorización de las funciones**. Por ejemplo,

`sqrt( {1,2,3} )`

es equivalente a

`{ sqrt(1),sqrt(2),sqrt(3) }`

Asimismo,

`rem ( {10,20,30}, {2,3,4} )`

es equivalente a

`{ rem(10,2) , rem(20,3), rem(30,4) }`

Además de las funciones predefinidas, Modelica proporciona la clase *function*, cuya finalidad es facilitar que el desarrollador del modelo programe sus propias funciones. Una **clase function** puede contener:

- La **interfaz de la función**, que está compuesta de la declaración de las variables públicas de la función. Cada una de estas variables debe ser clasificada como variable de entrada o de salida, para lo cual se antepone en la declaración

de la variable la palabra reservada **input** o **output**, respectivamente. Las variables de salidas son aquellas cuyo valor se calcula en la función.

- Las variables locales a la función, que son declaradas en la **sección protected** de la función. Estas variables son típicamente usadas en el cuerpo de la función para almacenar resultados intermedios.
- Una única **sección algorithm** o una **llamada a una función externa** escrita en lenguaje de programación (típicamente C o Fortran 77).

En la especificación del lenguaje Modelica se establece el requisito siguiente:

El desarrollador del modelo debe programar las funciones en Modelica de modo que **no tengan memoria interna**.

Supóngase un valor cualquiera de las entradas computacionales a la función. No tener memoria interna significa que siempre que se ejecute la función empleando esos valores de las entradas, debe obtenerse el mismo valor de las salidas computacionales. Es decir, si se ejecuta varias veces la función con las mismas entradas, debe obtenerse en todos los casos la misma salida.

La razón para imponer esta restricción es que en ocasiones los métodos numéricos empleados para simular el modelo requieren llamar repetidamente la función para un mismo valor de sus entradas. Por ejemplo, cuando se resuelve el modelo en el instante inicial, puede ser necesario evaluar varias veces el modelo (la razón será aparente cuando se expliquen las capacidades de Modelica para la inicialización del modelo). Puesto que todas estas ejecuciones de la función son conceptualmente la misma, debería obtenerse el mismo resultado en todos los casos.

Volvamos al problema que nos ocupa. Supongamos que se desea programar una función que interpole linealmente. En la Figura 2.34 se muestra la interfaz de la función y la forma en que se realiza la interpolación. La función tiene tres argumentos de entrada: el punto en el cual se desea interpolar y dos vectores, que definen los puntos entre los cuales debe interpolarse. La definición de la función y el modelo que la invoca se muestran en el Código 2.14. Del modelo de la transmisión de calor en la tubería, desde el que se invoca la función, sólo se muestran las diferencias respecto al Código 2.13. En la Figura 2.35 se muestra la evolución de la temperatura del entorno, calculada mediante interpolación.

La sentencia **assert** se emplea en la definición de la función (véase nuevamente el Código 2.14) para especificar expresiones lógicas que deben verificarse durante la simulación. La sintaxis es:

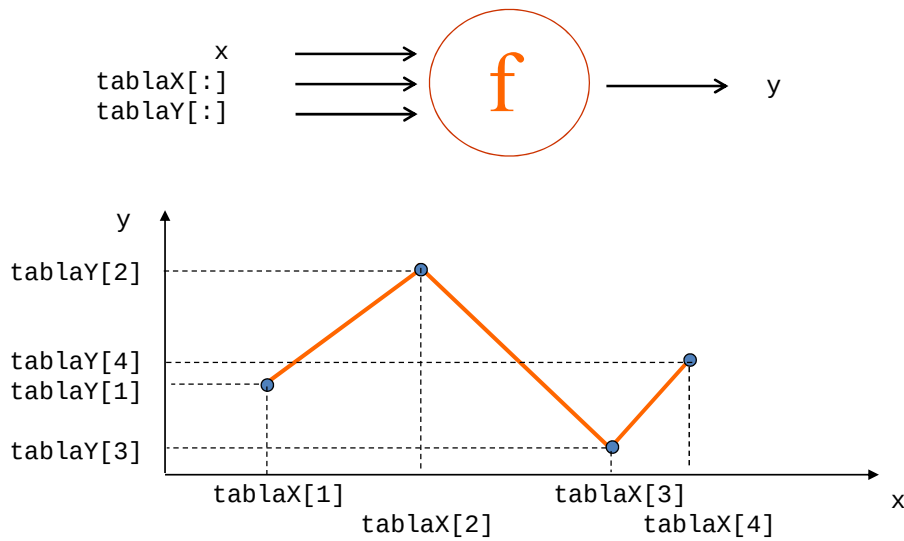


Figura 2.34: Interfaz de la función (arriba) y ejemplo de interpolación (abajo).

```
assert( expresion_logica, mensaje_de_error );
```

Si la expresión lógica se hace *false*, se aborta la simulación y se muestra en la ventana de mensajes el mensaje de error especificado en la sentencia. En la función se emplea para comprobar que las dos columnas de la tabla tienen el mismo número de filas y para comprobar que el punto en el cual se quiere interpolar está dentro del rango de valores de la primera columna de la tabla.

Prosiguiendo con la breve introducción a las funciones en Modelica, indicar que es posible definir funciones que devuelvan varios valores. Por ejemplo, a continuación se muestra una función que acepta dos argumentos (ángulo, radio), que son las coordenadas polares de un punto, y devuelve dos valores escalares (x , y), que son las coordenadas rectangulares del punto.

```
function circulo
  input Real angulo, radio;
  output Real x, y;
algorithm
  x := radio * cos(angulo);
  y := radio * sin(angulo);
end circulo;
```

Esta función puede ser invocada de la forma siguiente:

```
(x1,y1) = circulo(angulo1, radio1);
```

Al definir la función pueden asignarse valores por defecto a los argumentos de entrada. Por ejemplo, si en la función anterior se asigna valores por defecto

```
function circulo
```

```

function interpolacionLineal
    input Real x "Variable independiente";
    input Real tablaX[:] "Puntos de la variable independiente";
    input Real tablaY[:] "Puntos de la variable dependiente";
    output Real y "Resultado interpolado";
protected
    Integer n;
    Real pendiente;
algorithm
    n := size(tablaX, 1);
    assert(size(tablaX, 1) == size(tablaY, 1),
        "Error: tablaX y tablaY deben tener el mismo número de elementos");
    assert(x >= tablaX[1] and x <= tablaX[n],
        "Error: variable independiente fuera de rango");
    for i in 1:n - 1 loop
        if x >= tablaX[i] and x <= tablaX[i + 1] then
            pendiente := (tablaY[i + 1] - tablaY[i]) / (tablaX[i + 1] - tablaX[i]);
            y := tablaY[i] + pendiente * (x - tablaX[i]);
        end if;
    end for;
end interpolacionLineal;

model calorTuberiaInterp

    parameter SI.Time valTiempo[:] = 0:3600:86400;
    parameter SI.Temperature valTemp[:] = {291.5, 293.0, 295.5, 300.0, 302.5, 305.5,
        306.0, 306.5, 308.5, 310.0, 312.5, 313.5, 313.5, 312.5, 310.0, 309.0, 305.5, 300.5,
        298.5, 293.5, 290.5, 286.0, 280.5, 275.5, 273.0};

    ...
equation
    ...
    T5 = 291.5 - 8.5 * sin(2 * pi * time / 86400);
    T5 = interpolacionLineal(x=time, tablaX=valTiempo, tablaY=valTemp);
end calorTuberiaInterp;

```

Código 2.14: Función para interpolación lineal y programa que la invoca.

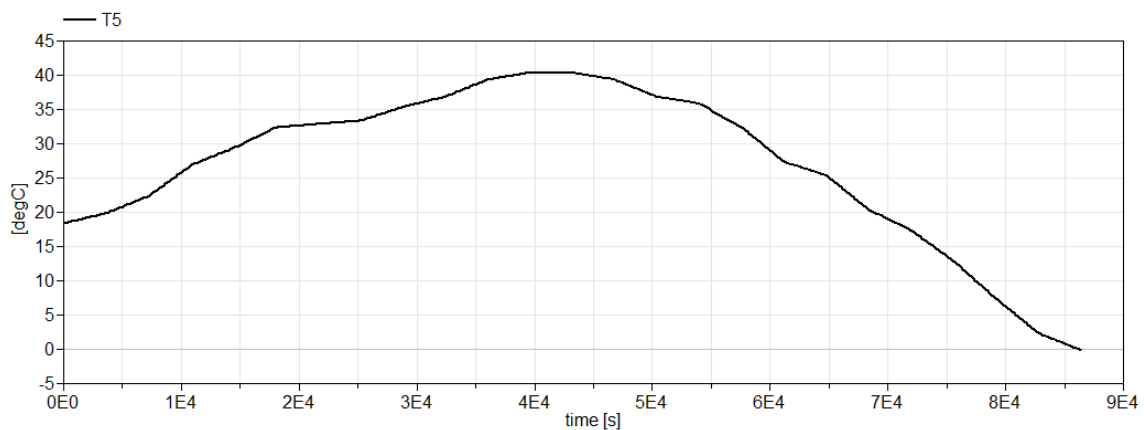


Figura 2.35: Temperatura del entorno calculada mediante interpolación.

```

input Real angulo = 0;
input Real radio  = 1;
output Real x, y;
algorithm
  ...

```

las invocaciones siguientes serían equivalentes:

```

(x1,y1) = circulo(0, 1);
(x1,y1) = circulo();
(x1,y1) = circulo(0);
(x1,y1) = circulo(radio=1);

```

Asimismo, Modelica permite encapsular **funciones externas**. Pueden definirse funciones en Modelica que encapsulen llamadas a funciones escritas en los lenguajes de programación C y Fortran 77. A continuación, se muestran varios ejemplos que ilustran cómo hacerlo.

- **Entradas escalares, una salida escalar.** Supongamos que en el fichero *tan2.c* está definida una función en C que acepta dos números de tipo *double* y devuelve un *double*. El prototipo de la función es:

```
double tan2(double, double);
```

La función en Modelica que realiza la llamada a esta función en C podría ser la siguiente:

```

function tan2
  input Real x1;
  input Real x2;
  output Real y;
  external "C" y = tan2(x1, x2);
  annotation(Include="#include<tan2.c>");
end tan2;

```

La invocación a la función puede formar parte de cualquier expresión, tanto en una ecuación como en una asignación. Por ejemplo:

```
z := tan2(x,y);
```

- **Entradas vectoriales, una salida vectorial.** Supongamos que en el fichero *join3vecFile.c* se encuentra la declaración de una función en C cuyo prototipo es:

```

void join3vec(double*, double*, double*, double*,
              size_t, size_t, size_t);

```

La función en Modelica que hace la llamada a esta función en C podría ser la siguiente:

```
function joinThreeVectors
  input Real v1[:], v2[:], v3[:];
  output Real vres[size(v1,1)+size(v2,1)+size(v3,1)];
  external "C" join3vec( v1, v2, v3, vres,
                        size(v1,1), size(v2,1), size(v3,1) );
  annotation(Include="#include<join3vecFile.c>");
end joinThreeVectors;
```

La invocación a la función puede formar parte de cualquier expresión, tanto en una ecuación, como en una asignación. Por ejemplo,

```
vjoined := joinThreeVectors(va, vb, vc);
```

que es traducida en la llamada siguiente a la función en C:

```
join3vec(&va, &vb, &vc, &vjoined, n1, n2, n3)
```

- **Varias salidas.** Supóngase que en el fichero *doubleResFile.c* está definida una función cuyo prototipo es:

```
double doubleresults(double, int, int*);
```

La función en Modelica que hace la llamada a esta función podría ser la siguiente:

```
function dualResults
  input Real x;
  input Integer y;
  output Real funcvalue;
  output Integer out1;
  external "C" funcvalue = doubleresults(x, y, out1);
  annotation(Include="#include<doubleResFile.c>");
end dualResults;
```

La función puede invocarse, por ejemplo, de la forma

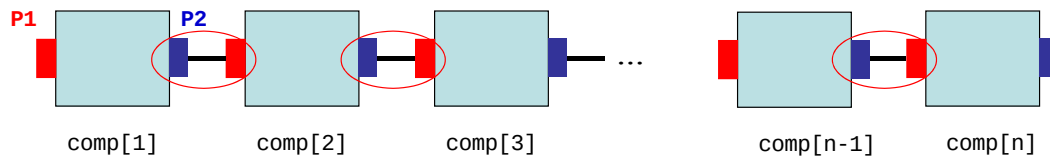
```
(z1, i2) := dualResults(2.4, 3);
```

que es traducida en la siguiente llamada a la función en C:

```
z1 = doubleresults(2.4, 3, &i2)
```

2.6.2. Ejemplo 2: conducción longitudinal del calor en una varilla

En ocasiones los modelos compuestos tienen una **estructura regular**. Es decir, la conexión entre ellos obedece a la aplicación repetida de ciertas reglas. Para



```

model ...
    parameter Integer n;
    Componente comp[n];
equation
    for i in 1:n-1 loop
        connect( comp[i].P2, comp[i+1].P1 );
    end for;
...
    
```

Figura 2.36: Modelo compuesto con estructura regular y código Modelica.

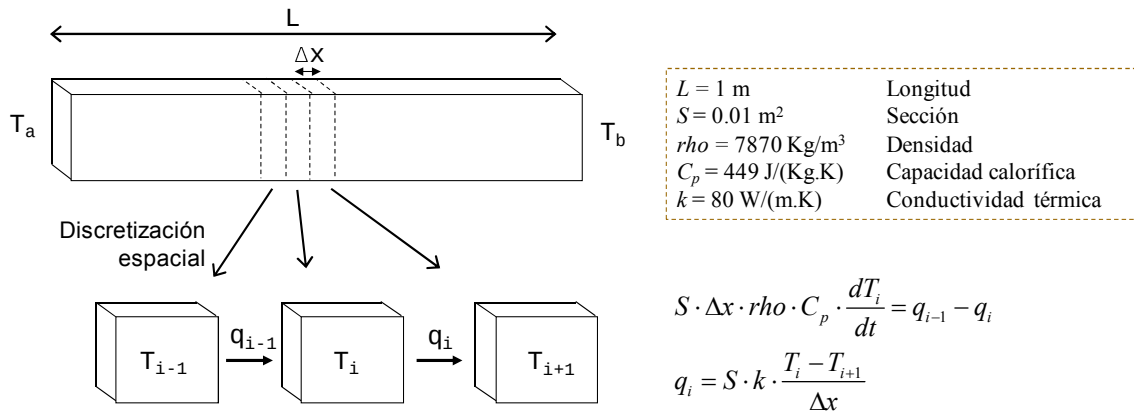


Figura 2.37: Discretización espacial de la varilla.

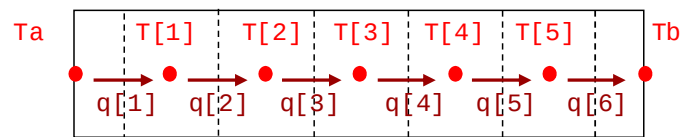


Figura 2.38: Modelado mediante discretización de la coordenada espacial.

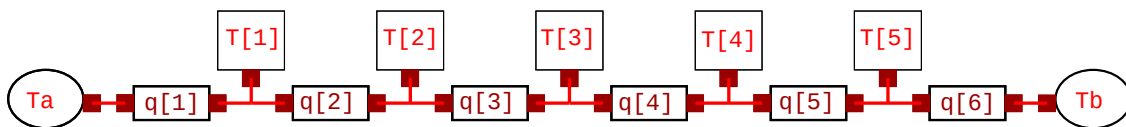


Figura 2.39: Modelado mediante descomposición en volúmenes de control.

facilitar la definición de este tipo de modelos, Modelica soporta la vectorización de los componentes y las conexiones.

Considérese, por ejemplo, el modelo mostrado en la parte superior de la Figura 2.36. Está compuesto de n objetos de una determinada clase llamada **Componente**. La clase tiene dos conectores llamados P1 y P2. En el modelo, se conecta el conector P2 del componente i con el conector P1 del componente $i + 1$.

Los componentes pueden definirse mediante un vector de n componentes, cada uno de los cuales es un objeto de la clase **Componente**. El número de componentes, n , es un parámetro, con lo cual puede ser fácilmente modificado. La conexión se describe mediante una *cláusula for*, empleando la notación punto para referirse a los conectores de cada componente del vector de componentes. Véase el código mostrado en la parte inferior de la Figura 2.36.

Para ilustrar esta capacidad del lenguaje Modelica, supongamos que se desea modelar el perfil de temperatura en una varilla. En concreto, se desea simular un salto escalón en la temperatura de uno de los extremos de la varilla (T_a), mientras la temperatura del otro extremo permanece constante $T_b = 300$ K.

Plantearémos, para ello, dos modelos diferentes. En el primero se realiza una discretización de las derivadas respecto a la coordenada espacial. Las variables temperatura se representen mediante un vector, y las ecuaciones del balance de energía y del flujo de calor son generadas empleando bucles *for*. El número de nodos de la discretización espacial es un parámetro entero, N . En la Figura 2.38 se muestra el caso $N = 5$. El modelo está descrito por el Código 2.15.

Obsérvese que se asigna valor inicial a la temperatura en una *sección initial equation*, en lugar de hacerlo empleando los atributos *start* y *fixed*. Las ecuaciones incluidas en la *sección initial equation* son válidas sólo en la inicialización del modelo. Así pues, el valor inicial de las variables $T[1], \dots, T[N-2]$ es igual a T_{step1} . La inicialización de modelos en Modelica se explicará con detalle en el Tema 4.

El segundo modelo se plantea dividiendo la varilla en volúmenes de control. En cada volumen de control se plantea el balance de energía. A través del plano de control que une dos volúmenes de control se produce flujo de calor. Para realizar este modelo, se programa una librería con los componentes siguientes: resistencia térmica, capacidad térmica y condición de contorno para la temperatura. A continuación, se modelado la varilla mediante la conexión de objetos de estas clases. En la Figura 2.39 se muestra el diagrama del modelo para el caso de $N = 5$ volúmenes de control. La descripción Modelica del modelo es el Código 2.16 y 2.17.

```

model conduccionCalorA
  parameter Integer N = 7;
  Real Ta      (unit="K");
  Real Tb      (unit="K");
  Real T[N-2] (unit="K");
  Real q[N-1] (unit="W");
  parameter Real L      (unit="m")      = 1;
  parameter Real S      (unit="m2")     = 0.01;
  parameter Real rho    (unit="kg/m3")  = 7870;
  parameter Real Cp     (unit="J/(kg.K)") = 449;
  parameter Real k      (unit="W/(m.K)") = 80;
  parameter Real Tstep1 (unit="K")      = 300;
  parameter Real Tstep2 (unit="K")      = 350;
  parameter Real tstep  (unit="s")      = 50;
  parameter Real Deltax = L / (N-1);
equation
  // Condiciones de contorno
  Ta = if time < tstep then Tstep1 else Tstep2;
  Tb = Tstep1;
  // Varilla
  for i in 1:N-2 loop
    S*Deltax*rho*Cp*der(T[i]) = q[i] - q[i+1];
  end for;
  q[1] = S*k*(Ta-T[1])/Deltax;
  for i in 2:N-2 loop
    q[i] = S*k*(T[i-1]-T[i])/Deltax;
  end for;
  q[N-1] = S*k*(T[N-2]-Tb)/Deltax;
initial equation
  for i in 1:N-2 loop
    T[i] = Tstep1;
  end for;
end conduccionCalorA;

```

Código 2.15: Conducción de calor 1D mediante discretización espacial.

```

package termico

connector flujoCalor
  Real T      (unit="K");
  flow Real q (unit="W");
end flujoCalor;

model resistenciaTermica
  flujoCalor p1, p2;
  parameter Real S      (unit="m2");
  parameter Real k      (unit="W/(m.K)");
  parameter Real Deltax (unit="m");
equation
  p1.q = S*k*(p1.T-p2.T)/Deltax;
  p2.q = -p1.q;
end resistenciaTermica;

model capacidadTermica
  parameter Real S      (unit="m2");
  parameter Real rho    (unit="kg/m3");
  parameter Real Cp     (unit="J/(kg.K)");
  parameter Real Deltax (unit="m");
  parameter Real Tinicial (unit="K");
  flujoCalor p;
equation
  S*Deltax*rho*Cp*der(p.T) = p.q;
initial equation
  p.T = Tinicial;
end capacidadTermica;

model fuenteT
  parameter Real Tstep1 (unit="K");
  parameter Real Tstep2 (unit="K");
  parameter Real tstep  (unit="s");
  flujoCalor p;
equation
  p.T = if time < tstep
    then Tstep1
    else Tstep2;
end fuenteT;

end termico;

```

Código 2.16: Librería para modelado de conducción de calor 1D en la varilla.

```

model conduccionCalorB
  parameter Integer N = 7;
  parameter Real L      (unit="m")      = 1;
  parameter Real S      (unit="m2")     = 0.01;
  parameter Real rho    (unit="kg/m3")  = 7870;
  parameter Real Cp     (unit="J/(kg.K)") = 449;
  parameter Real k      (unit="W/(m.K)") = 80;
  parameter Real Tstep1 (unit="K")      = 300;
  parameter Real Tstep2 (unit="K")      = 350;
  parameter Real tstep  (unit="s")      = 50;
  parameter Real Deltax = L / (N-1);
  termico.fuenteT Ta(Tstep1=Tstep1,Tstep2=Tstep2,tstep=tstep);
  termico.fuenteT Tb(Tstep1=Tstep1,Tstep2=Tstep1,tstep=tstep);
  termico.resistenciaTermica Rth[N-1] (S=fill(S,N-1), k=fill(k,N-1),
                                         Deltax=fill(Deltax,N-1));
  termico.capacidadTermica Cth[N-2] (S=fill(S,N-2), rho=fill(rho,N-2),
                                       Cp=fill(Cp,N-2), Deltax=fill(Deltax,N-2),
                                       Tinicial=fill(Tstep1,N-2));
equation
  connect (Ta.p, Rth[1].p1);
  for i in 1:N-2 loop
    connect (Rth[i].p2, Cth[i].p);
    connect (Cth[i].p, Rth[i+1].p1);
  end for;
  connect (Rth[N-1].p2, Tb.p);
end conduccionCalorB;

```

Código 2.17: Conducción de calor 1D mediante división en volúmenes de control.

2.6.3. Ejemplo 3: control del nivel y temperatura de un depósito

Supongamos que se desea modelar el sistema mostrado en la Figura 2.40, que está compuesto por un depósito, una fuente de líquido y un controlador de nivel. El líquido proporcionado por la fuente entra al depósito por la parte superior del mismo, y sale, por acción de la fuerza gravitatoria, a través de un orificio existente en la base del depósito.

El flujo de líquido suministrado por la fuente, que llamaremos F_{in} , es regulado mediante el voltaje de entrada a la fuente. Si el voltaje de entrada u_h es positivo, entonces F_{in} es proporcional a u_h . El factor de proporcionalidad es un parámetro denominado k_f . Si el voltaje u_h es negativo, entonces el flujo másico F_{in} es cero. La fuente, por tanto, puede introducir agua en el depósito, pero no puede extraerla.

El controlador de nivel es de tipo proporcional integral, conocido comúnmente como **controlador PI**. En el controlador se calcula la diferencia entre el valor de consigna y el valor actual de la variable controlada. A esta diferencia se la denomina error. La salida del controlador PI se calcula como la suma de dos términos. El primero es proporcional al error. El segundo es proporcional a la integral del error. En este caso, el error e y la salida u_h se calculan de la forma siguiente:

$$e = h_{ref} - h \quad (2.74)$$

$$u_h = \underbrace{k_P \cdot e}_{\text{Término proporcional}} + \underbrace{\frac{1}{k_I} \cdot \int_0^t e \cdot dt}_{\text{Término integral}} \quad (2.75)$$

Obsérvese que llamando I a la integral del error, las ecuaciones anteriores pueden escribirse equivalentemente de la forma siguiente:

$$e = h_{ref} - h \quad (2.76)$$

$$\frac{dI}{dt} = e \quad (2.77)$$

$$u_h = k_P \cdot e + \frac{1}{k_I} \cdot I \quad (2.78)$$

Las magnitudes físicas que intervienen en el modelo de muestran en la Tabla 2.16. En este modelo consideramos que la sección de la base del depósito (A) y del agujero

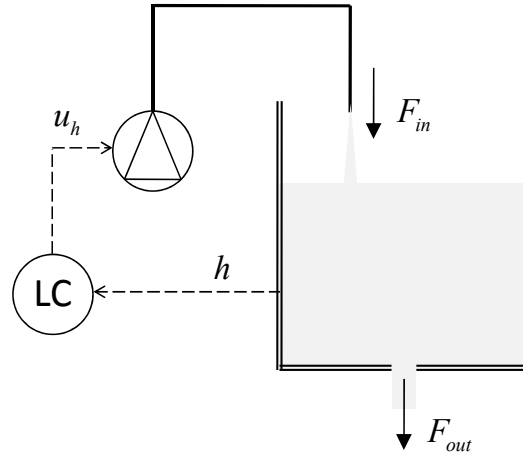


Figura 2.40: Control de nivel en un depósito de líquido. El sistema está compuesto por un depósito con un orificio en su base, un controlador de nivel (indicado “LC”, del inglés *Level Controller*) y una fuente de líquido.

Tabla 2.16: Magnitudes físicas relevantes del sistema mostrado en la Figura 2.40.

Símbolo	Significado	Unidades	Valor
a	Sección del agujero en el depósito	m^2	0.1
A	Sección de la base del depósito	m^2	2.0
e	Error calculado en el controlador	m	
F_{in}, F_{out}	Flujos máscos	kg/s	
g	Aceleración gravitatoria	m/s^2	9.81
h	Nivel de líquido en el depósito	m	
h_{ref}	Valor de consigna para el nivel	m	
I	Integral del error e	$\text{m}\cdot\text{s}$	
k_f	Parámetro de la fuente	$\text{kg}/(\text{s}\cdot\text{V})$	100
k_I	Parámetro integral del controlador	$\text{m}\cdot\text{s}/\text{V}$	15
k_P	Parámetro proporcional del controlador	V/m	2
m	Masa de líquido en el depósito	kg	
u_h	Voltaje de salida del controlador	V	
ρ	Densidad del líquido	kg/m^3	760

(a), así como los parámetros de proporcionalidad de la bomba y el controlador (k_c , k_f), y la densidad del líquido (ρ), son parámetros. Las variables del modelo (m , h , F_{out} , F_{in} , e , I , u_h , h_{ref}) son calculadas de las Ecs. (2.79) – (2.86). La aceleración gravitatoria (g) es una constante del modelo.

$$\frac{dm}{dt} = F_{in} - F_{out} \quad (2.79)$$

$$m = \rho \cdot A \cdot h \quad (2.80)$$

$$F_{out} = a \cdot \rho \cdot \sqrt{2 \cdot g \cdot h} \quad (2.81)$$

$$F_{in} = \text{máx}(0, k_f \cdot u_h) \quad (2.82)$$

$$e = h_{ref} - h \quad (2.83)$$

$$\frac{dI}{dt} = e \quad (2.84)$$

$$u_h = k_P \cdot e + \frac{1}{k_I} \cdot I \quad (2.85)$$

$$h_{ref} = \begin{cases} 5 \text{ m} & \text{si } t < 300 \text{ s} \\ 3 \text{ m} & \text{si } 300 \text{ s} \leq t < 600 \text{ s} \\ 7 \text{ m} & \text{si } t \geq 600 \text{ s} \end{cases} \quad (2.86)$$

La Ec. (2.79) expresa el balance de la masa de líquido en el depósito: la variación en la masa es igual al flujo másico entrante menos el saliente. La Ec. (2.80) expresa la relación entre la masa y el volumen ($A \cdot h$) del líquido, la cual viene determinada por su densidad. La Ec. (2.81) describe la dependencia del flujo másico de salida a través del agujero con el nivel del líquido. La Ec. (2.82) es la relación constitutiva de la fuente y las Ecs. (2.83) – (2.85) describen la ley de control del controlador PI. La evolución en el tiempo del valor de la consigna para el nivel viene dada por la Ec. (2.86).

Las condiciones iniciales quedan especificadas indicando el valor de la masa de líquido que hay inicialmente en el depósito e indicando que la integral del error vale inicialmente cero. La descripción en Modelica de este modelo se muestra en el Código 2.18. Se simula el modelo durante 1000 s. En la Figura 2.41 se muestra la evolución del valor de consigna para el nivel del líquido y el valor del nivel de líquido en el depósito.

Supongamos ahora se añade al sistema anterior un componente calefactor, que permite calentar el líquido contenido en el depósito, y un controlador de temperatura que acciona el calefactor. El sistema está representado en la Figura 2.42. El controlador de temperatura recibe el valor de la temperatura del líquido almacenado


```

model controlDeposito1
  import SI = Modelica.SIunits;
  import Modelica.Math.*;
  parameter SI.Area a=0.1 "Sección del agujero";
  parameter SI.Area A=2 "Sección del depósito";
  SI.Height e "href-h, calculado en el controlador";
  SI.MassFlowRate Fin "Flujo másico de entrada al depósito";
  SI.MassFlowRate Fout "Flujo másico de salida del depósito";
  constant SI.Acceleration g=9.81 "Aceleración gravitatoria";
  SI.Height h "Nivel en el depósito";
  SI.Height href "Consigna para el nivel";
  Real I(unit="m.s", start=0, fixed=true) "Integral del error";
  parameter Real kf(unit="kg/(s.V)") = 100 "Parámetro de la fuente";
  parameter Real kI(unit="m.s/V") = 15 "Parám. integral controlador";
  parameter Real kP(unit="V/m") = 2 "Parám. proporcional controlador";
  SI.Mass m(start=1e3, fixed=true) "Masa de líquido en el depósito";
  SI.Voltage uh "Voltaje de salida del controlador";
  parameter SI.Density rho=760 "Densidad del líquido";
equation
  der(m) = Fin - Fout;
  m = rho*A*h;
  Fout = a*rho*sqrt(2*g*h);
  Fin = max(0, kf*uh);
  e = href - h;
  der(I) = e;
  uh = kP*e + I/kI;
  href = if time < 300 then 5 else if time < 600 then 3 else 7;
end controlDeposito1;

```

Código 2.18: Control de nivel en el depósito de la Figura 2.40.

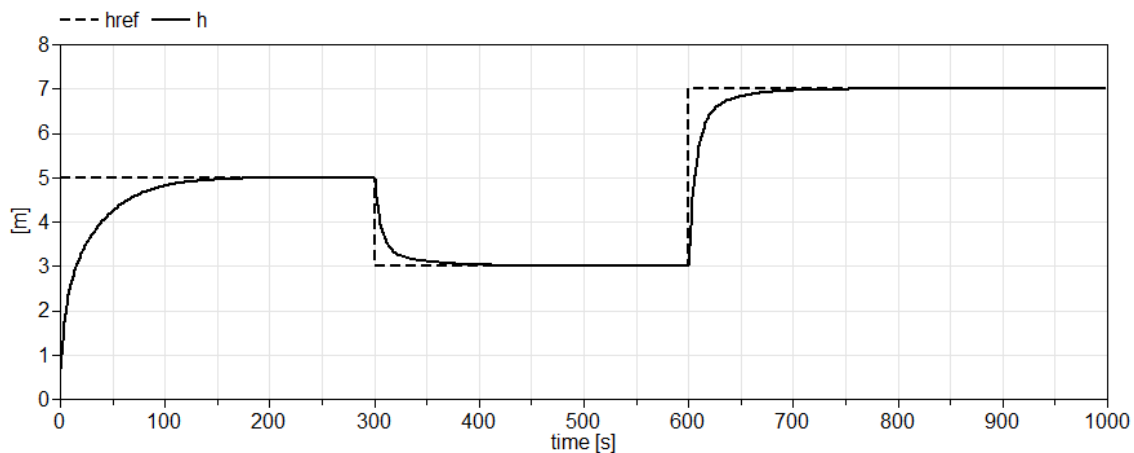


Figura 2.41: Resultado de la simulación del Código 2.18: valor de consigna para el nivel de líquido (h_{ref}) y valor del nivel de líquido (h).

en el depósito y calcula un voltaje u_T que es aplicado al calefactor. El flujo de calor cedido por el calefactor al líquido, que llamaremos Q , es proporcional a u_T .

Puede modelarse la evolución de la temperatura del líquido almacenado en el depósito planteando su balance de energía. Para ello, llamaremos T_{in} a la temperatura a la que se encuentra el líquido que entra en el depósito procedente de la fuente. Asimismo, supondremos que el líquido del depósito está perfectamente agitado, con lo cual se encuentra a temperatura uniforme (independiente de la coordenada espacial) que llamaremos T . Por tanto, el líquido que abandona el depósito a través del agujero de la base se encuentra a temperatura T .

El balance de energía del líquido del depósito puede plantearse de la forma mostrada en la Ec. (2.87).

$$\underbrace{\frac{dH}{dt}}_{\text{Derivada de la entalpía total del líquido}} = \underbrace{F_{H,in}}_{\text{Flujo entrante de entalpía}} - \underbrace{F_{H,out}}_{\text{Flujo saliente de entalpía}} + \underbrace{Q}_{\text{Flujo entrante de calor}} \quad (2.87)$$

La entalpía total del líquido almacenado en el depósito (H) es función de su masa (m), de su capacidad calorífica (C_p) y de su temperatura (T), tal como se muestra en la Ec. (2.88).

$$H = m \cdot C_p \cdot T \quad (2.88)$$

Supongamos además que la capacidad calorífica del líquido es función lineal de su temperatura:

$$C_p = C_{p,0} + C_{p,1} \cdot T \quad (2.89)$$

donde $C_{p,0}$ y $C_{p,1}$ son parámetros constantes de valor conocido. Sustituyendo en la Ec. (2.88) se obtiene:

$$H = m \cdot (C_{p,0} + C_{p,1} \cdot T) \cdot T \quad (2.90)$$

Los flujos entrante y saliente de entalpía pueden calcularse a partir de los flujos másicos de entrada y salida, de la temperatura a la que se encuentran los mismos y de las capacidades caloríficas, de la forma mostrada en las Ecs. (2.91) y (2.92).

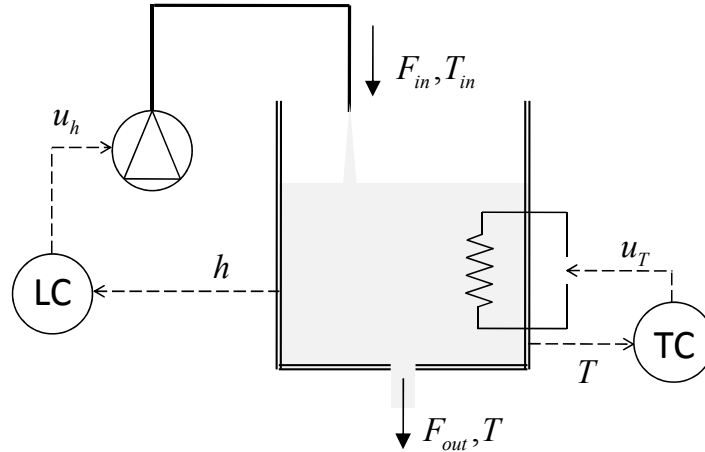


Figura 2.42: Control de nivel y temperatura en un depósito de líquido. Al sistema mostrado en la Figura 2.40 se le ha añadido un calefactor y un controlador de temperatura (etiquetado “TC”, del inglés *temperature controller*).

Tabla 2.17: Magnitudes físicas adicionales para describir el comportamiento energético.

Símbolo	Significado	Unidades	Valor
C_p	Capacidad calorífica del líquido	J/(kg·K)	
$C_{p,0}$	Termino de orden cero de C_p	J/(kg·K)	446
$C_{p,1}$	Termino de orden uno de C_p	J/(kg·K ²)	5.36
$F_{H,in}, F_{H,out}$	Flujos de entalpía	W	
H	Entalpía total del líquido del depósito	J	
e_T	Error calculado en controlador temperatura	K	
I_T	Integral del error e_T	K·s	
k_c	Parámetro del calentador	W/V	8E+6
$k_{T,I}$	Parám. integral controlador temperatura	K·s/V	50
$k_{T,P}$	Parám. proporcional controlador temperatura	V/K	0.3
Q	Flujo de calor del calefactor al líquido	W	
T	Temperatura del líquido en el depósito	K	
T_{in}	Temperatura del líquido de la fuente	K	300
T_{ref}	Valor de consigna para la temperatura	K	
u_T	Entrada de control al calefactor	V	

$$F_{H,in} = F_{in} \cdot (C_{p,0} + C_{p,1} \cdot T_{in}) \cdot T_{in} \quad (2.91)$$

$$F_{H,out} = F_{out} \cdot (C_{p,0} + C_{p,1} \cdot T) \cdot T \quad (2.92)$$

La relación constitutiva del calentador describe la relación existente entre el flujo de calor proporcionado por el calentador y el voltaje aplicado en su entrada de control. Si el voltaje es negativo, el calentador permanece apagado ($Q = 0$).

$$Q = \text{máx}(0, k_c \cdot u_T) \quad (2.93)$$

Las ecuaciones que describen el controlador y la consigna para la temperatura son las siguientes:

$$e_T = T_{ref} - T \quad (2.94)$$

$$\frac{dI_T}{dt} = e_T \quad (2.95)$$

$$u_T = k_{T,P} \cdot e_T + \frac{1}{k_{T,I}} \cdot I_T \quad (2.96)$$

$$T_{ref} = \begin{cases} 340 \text{ K} & \text{si } t < 500 \text{ s} \\ 320 \text{ K} & \text{si } t \geq 500 \text{ s} \end{cases} \quad (2.97)$$

Para inicializar el modelo es preciso especificar la temperatura inicial del líquido almacenado en el depósito y también indicar que el valor inicial de I_T es cero.

La descripción en Modelica del modelo se muestra en el Código 2.19 y 2.20. Simulándolo durante 1000 s, se obtiene que la temperatura del líquido almacenado en el depósito evoluciona como se muestra en la Figura 2.43.

Obsérvese que a medida que se van añadiendo más componentes al sistema y el modelo va haciéndose más complejo, su descripción como un modelo atómico va haciéndose más difícil de entender y de verificar. Una forma de proceder en este caso es realizar una descripción orientada a objetos del modelo. En este caso el sistema puede descomponerse de manera natural en los componentes siguientes (véase nuevamente la Figura 2.42): el depósito, los dos controladores, la fuente y el calefactor. Los dos controladores son básicamente el mismo, con lo cual bastará con definir una única clase de modelo e instanciarla dos veces.

model controlDeposito2

```

import SI = Modelica.SIunits;
import Modelica.Math.*;

constant SI.Acceleration g=9.81 "Aceleración gravitatoria";

// Líquido
parameter SI.Density rho=760 "Densidad del líquido";
parameter SI.SpecificHeatCapacity Cp0=446 "Término orden 0 de Cp";
parameter Real Cp1(unit="J/(kg.K2)") = 5.36 "Término orden 1 de Cp";

// Depósito
parameter SI.Area a=0.1 "Sección del agujero";
parameter SI.Area A=2 "Sección del deposito";

// Controlador nivel
parameter Real kI(unit="m.s/V") = 15 "Parám. integral controlador";
parameter Real kP(unit="V/m") = 2 "Parám. proporcional controlador";

// Fuente de líquido
parameter Real kf(unit="kg/(s.V)") = 100 "Parámetro de la fuente";

// Calefactor
parameter Real kc(unit="W/V") = 8E+6 "Parámetro del calentador";

// Controlador de temperatura
parameter Real kT_I(unit="K.s/V") = 50 "Parám. integral controlador temp";
parameter Real kT_P(unit="V/K") = 0.3 "Parám. proporcional controlador temp";

SI.Height e "href-h, calculado en el controlador";
SI.MassFlowRate Fin "Flujo másico de entrada al depósito";
SI.MassFlowRate Fout "Flujo másico de salida del depósito";
SI.Height h "Nivel en el depósito";
SI.Height href "Consigna para el nivel";
Real I(unit="m.s", start=0, fixed=true) "Integral del error e";
SI.Mass m(start=1e3, fixed=true) "Masa de líquido en el depósito";
SI.Voltage uh "Voltaje de salida del controlador";
SI.EnthalpyFlowRate FHin "Flujo de entrada de entalpía";
SI.EnthalpyFlowRate FHout "Flujo de salida de entalpía";
SI.Enthalpy H "Entalpía total del líquido";
SI.Temperature eT "Tref-T, calculado en controlador de temp";
Real IT(unit="K.s", start=0, fixed=true) "Integral del error eT";
SI.HeatFlowRate Q "Flujo de calor del calefactor";
SI.Temperature T(start=300, fixed=true) "Temp. del líquido en depósito";
parameter SI.Temperature Tin=300 "Temperatura del líquido de la fuente";
SI.Temperature Tref "Consigna para la temperatura";
SI.Voltage uT "Entrada de control del calefactor";

```

Código 2.19: Control de nivel y temperatura del depósito de la Figura 2.42 (1/2).

equation

```
// Depósito
der(m) = Fin - Fout;
der(H) = FHin - FHout + Q;
m = rho*A*h;
H = m*(Cp0 + Cp1*T)*T;
Fout = a*rho*sqrt(2*g*h);
FHout = Fout*(Cp0 + Cp1*T)*T;

// Fuente
Fin = max(0, kf*uh);
FHin = Fin*(Cp0 + Cp1*Tin)*Tin;

// Controlador de nivel
e = href - h;
der(I) = e;
uh = kP*e + I/kI;
href = if time < 300 then 5 else if time < 600 then 3 else 7;

// Controlador de temperatura
eT = Tref - T;
der(IT) = eT;
uT = kT_P*eT + IT/kT_I;
Tref = if time < 500 then 340 else 320;

// Calefactor
Q = max(0, kc*uT);

end controlDeposito2;
```

Código 2.20: Control de nivel y temperatura del depósito de la Figura 2.42 (2/2).

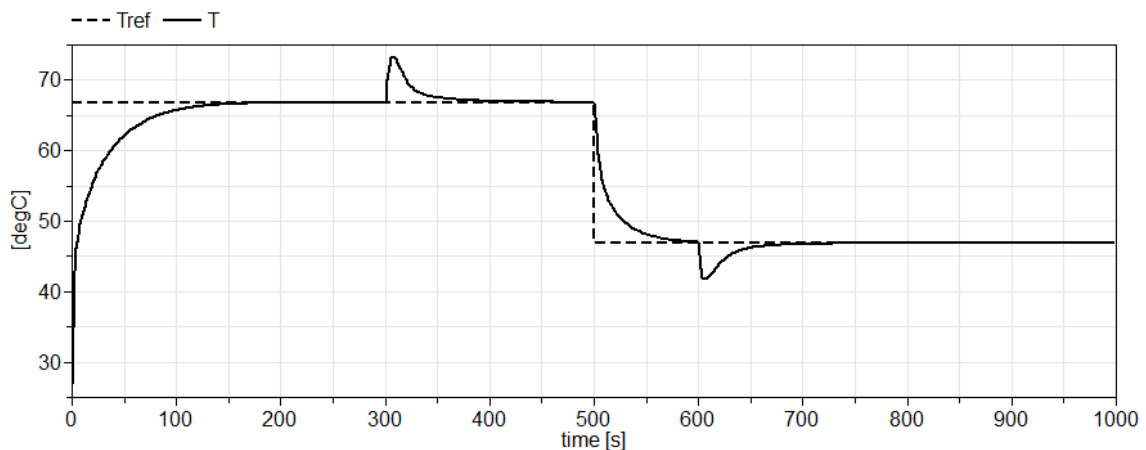


Figura 2.43: Simulación del Código 2.19 y 2.20: evolución de la consigna para la temperatura y de la temperatura del líquido almacenado.

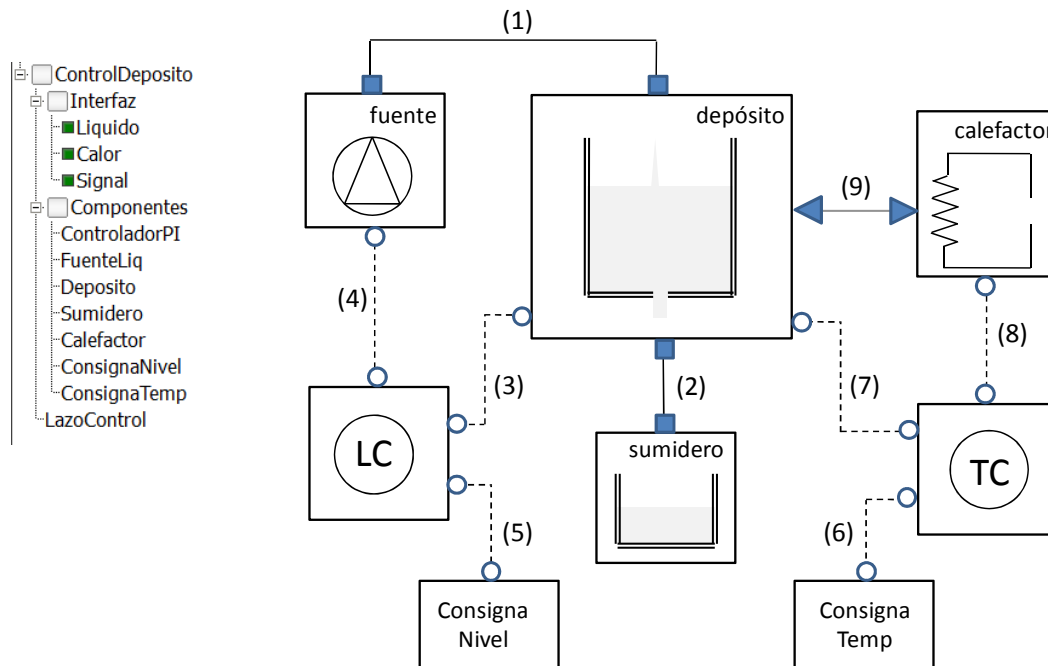


Figura 2.44: En la parte izquierda se muestra la estructura del *package* `ControlDeposito`. El modelo del sistema completo es `LazoControl`. En la parte derecha de la figura se muestra el diagrama de sus componentes y conexiones. Estas últimas se han numerado de la misma manera que se ha hecho mediante comentarios en el Código 2.23.

Desarrollemos una librería que contenga los componentes anteriores y el modelo del sistema completo, este último definido instanciando y conectando los componentes. Una posible forma de estructurar la librería es la mostrada en la parte izquierda de la Figura 2.44.

Obsérvese que se ha definido un *package* para agrupar los conectores y otro diferente para los componentes. El modelo del sistema completo, que se ha llamado `LazoControl`, se ha ubicado en el nivel jerárquico superior del *package*. La librería es un *encapsulated package* al que se ha llamado `ControlDeposito`.

En la parte de la derecha de la Figura 2.44 se muestran los componentes en que se ha dividido el sistema y las conexiones entre ellos. Se han definido tres clases de conectores: uno para describir el flujo de líquido, otro para el flujo de calor y otro para el flujo de información. Para poder distinguirlos en la figura, se representan mediante iconos diferentes. El conector de flujo de líquido se ha representado en la figura como un cuadrado, el de calor como un triángulo y el de información como una circunferencia.

Las variables de los conectores de flujo de líquido (flujo másico y de entalpía) y de flujo de calor se han declarado de tipo `through`. La variable del conector de flujo de información se ha declarado de tipo `across`. Como es habitual, las ecuaciones de

los componentes se plantearán suponiendo que el flujo tiene signo positivo cuando tiene sentido entrante al componente.

La descripción de la librería se muestra en el Código 2.21 – 2.23. Obsérvese que el modelo del controlador tiene tres conectores de flujo de información: el valor de la variable controlada, el valor de consigna para ésta y el valor de la variable manipulada. Se han definido dos componentes, llamados `ConsignaNivel` y `ConsignaTemp`, en los cuales se asigna valor a la consigna de los controladores.

Se ha definido un componente llamado `Sumidero`, que se ha conectado a la salida de líquido por la base del depósito. Es necesario hacer esto, ya que si el conector de la base del depósito se dejara sin conectar el entorno de modelado añadiría automáticamente al modelo las dos ecuaciones siguientes: el flujo másico a través de la base es cero y el flujo de entalpía a través de la base es cero.

En general, si un conector no está conectado a ningún otro conector de su mismo nivel jerárquico, entonces el entorno de modelado impone automáticamente, añadiendo las correspondientes ecuaciones al modelo, que el valor de cada una de las variables through del conector sea cero.

El entorno de modelado hace esto no sólo al traducir el modelo completo para su simulación, también lo hace cuando se hace *check* de una clase (véase nuevamente la Figura 2.5). Por ejemplo, si se hace *check* de la clase `FuenteLiq` (véase el Código 2.21), el entorno de modelado mostrará un mensaje indicando que la clase es estructuralmente singular: tiene 3 incógnitas y 4 ecuaciones. Las incógnitas son `portLiq.Fm`, `portLiq.FH` y `u.s`. Las ecuaciones son las dos escritas en la clase:

```
portLiq.Fm = -max(0, kf*u.s);
portLiq.FH = portLiq.Fm*(Cp0 + Cp1*T)*T;
```

mas las dos añadidas por el entorno de modelado igualando a cero las variables through de todos los conectores que están sin conectar con otros conectores de su mismo nivel jerárquico:

```
portLiq.Fm = 0;
portLiq.FH = 0;
```

Obtener un mensaje de este tipo al hacer *check* de una clase no es indicativo de que la clase esté incorrectamente definida, ya que la clase puede estar ideada para ser usada en un contexto computacional diferente al que el entorno de modelado supone al realizar el *check*. En el caso de la clase `FuenteLiq`, se ha diseñado suponiendo que ambos conectores estarán conectados, que la variable `u.s` podrá ser calculada en el

encapsulated package ControlDeposito

```
import SI = Modelica.SIunits;
import Modelica.Math.*;
```

package Interfaz

```
connector Liquido
  flow SI.MassFlowRate Fm "Flujo másico";
  flow SI.EnthalpyFlowRate FH "Flujo de entalpía";
end Liquido;

connector Calor
  flow SI.HeatFlowRate Q "Flujo de calor";
end Calor;

connector Signal
  Real s;
end Signal;
```

end Interfaz;

package Componentes

```
model ControladorPI
  Interfaz.Signal y;
  Interfaz.Signal ref;
  Interfaz.Signal u;
  parameter Real kI "Parámetro integral";
  parameter Real kP "Parámetro proporcional";
  protected
    Real e;
    Real I(start=0, fixed=true) "Integral de e";
  equation
    e = ref.s - y.s;
    der(I) = e;
    u.s = kP*e + I/kI;
  end ControladorPI;

model FuenteLiq
  Interfaz.Signal u;
  Interfaz.Liquido portLiq;
  parameter SI.Temperature T "Temperatura del líquido de la fuente";
  parameter Real kf(unit="kg/(s.V)") "Constante de proporcionalidad";
  parameter SI.SpecificHeatCapacity Cp0 "Término orden 0 de Cp";
  parameter Real Cp1(unit="J/(kg.K2)") "Término orden 1 de Cp";
  equation
    portLiq.Fm = -max(0, kf*u.s);
    portLiq.FH = portLiq.Fm*(Cp0 + Cp1*T)*T;
  end FuenteLiq;
```

Código 2.21: Librería para control de nivel y temperatura de un depósito (1/3).

```

model Deposito
  Interfaz.Liquido portLiqSup;
  Interfaz.Liquido portLiqBase;
  Interfaz.Calor portCalor;

  Interfaz.Signal signal_h;
  Interfaz.Signal signal_T;

  constant SI.Acceleration g=9.81 "Aceleracion gravitatoria";
  parameter SI.Area a "Seccion del agujero";
  parameter SI.Area A "Sección del deposito";
  parameter SI.Density rho "Densidad del liquido";
  parameter SI.SpecificHeatCapacity Cp0 "Término orden 0 de Cp";
  parameter Real Cp1(unit="J/(kg.K2)") "Término orden 1 de Cp";

  SI.Temperature T(start=300, fixed=true) "Temperatura del líquido";
  SI.Height h "Nivel del líquido";
  SI.Mass m(start=1e3, fixed=true) "Masa de líquido en el deposito";
  SI.Enthalpy H "Entalpía total del líquido";

equation
  // Depósito
  der(m) = portLiqSup.Fm + portLiqBase.Fm;
  der(H) = portLiqSup.FH + portLiqBase.FH + portCalor.Q;
  m = rho*A*h;
  H = m*(Cp0 + Cp1*T)*T;
  portLiqBase.Fm = -a*rho*sqrt(2*g*h);
  portLiqBase.FH = portLiqBase.Fm*(Cp0 + Cp1*T)*T;
  signal_h.s = h;
  signal_T.s = T;
end Deposito;

model Sumidero
  Interfaz.Liquido portLiq;
end Sumidero;

model Calefactor
  Interfaz.Signal cntrl;
  Interfaz.Calor portCalor;
  parameter Real kc(unit="W/V") "Constante de proporcionalidad";
equation
  portCalor.Q = -max(0, kc*cntrl.s);
end Calefactor;

model ConsignaNivel
  Interfaz.Signal ref;
equation
  ref.s = if time < 300 then 5 else if time < 600 then 3 else 7;
end ConsignaNivel;

```

Código 2.22: Librería para control de nivel y temperatura de un depósito (2/3).

```

model ConsignaTemp
    Interfaz.Signal ref;
equation
    ref.s = if time < 500 then 340 else 320;
end ConsignaTemp;

end Componentes;

model LazoControl
    import ControlDeposito.Componentes.*;
    // Líquido
    parameter SI.Density rho=760 "Densidad del liquido";
    parameter SI.SpecificHeatCapacity Cp0=446 "Término orden 0 de Cp";
    parameter Real Cp1(unit="J/(kg.K2)") = 5.36 "Término orden 1 de Cp";
    // Depósito
    parameter SI.Area a=0.1 "Seccion del agujero";
    parameter SI.Area A=2 "Sección del deposito";

    parameter SI.Temperature Tin=300 "Temperatura del líquido de la fuente";
    parameter Real kc(unit="W/V") = 8E+6 "Parámetro del calefactor";
    parameter Real kf(unit="kg/(s.V)") = 100 "Parámetro de la fuente";
    // Controlador nivel
    parameter Real kI(unit="m.s/V") = 15 "Param. integral controlador";
    parameter Real kP(unit="V/m") = 2 "Param. proporcional controlador";

    // Controlador de temperatura
    parameter Real kT_I(unit="K.s/V") = 50 "Parám. integral controlador temp";
    parameter Real kT_P(unit="V/K") = 0.3
        "Parám. proporcional controlador temp";

    FuenteLiq fuente(kf=kf,Cp0=Cp0,Cp1=Cp1,T=Tin);
    Calefactor calefactor(kc=kc);
    Deposito deposito(a=a,A=A,rho=rho,Cp0=Cp0,Cp1=Cp1);
    ControladorPI LC(kI=kI, kP=kP);
    ControladorPI TC(kI=kT_I, kP=kT_P);
    Sumidero sumidero;
    ConsignaNivel nivelSP;
    ConsignaTemp tempSP;

equation
    connect(deposito.portLiqSup, fuente.portLiq); // (1)
    connect(deposito.portLiqBase, sumidero.portLiq); // (2)
    connect(deposito.signal_h, LC.y); // (3)
    connect(LC.u, fuente.u); // (4)
    connect(nivelSP.ref, LC.ref); // (5)
    connect(tempSP.ref, TC.ref); // (6)
    connect(deposito.signal_T, TC.y); // (7)
    connect(calefactor.cntrl, TC.u); // (8)
    connect(calefactor.portCalor, deposito.portCalor); // (9)
end LazoControl;

end ControlDeposito;

```

Código 2.23: Librería para control de nivel y temperatura de un depósito (3/3).

componente conectado al conector `u`, y que las ecuaciones de la clase se emplearán para calcular `portLiq.Fm` y `portLiq.FH`. Si la clase se usa de esta forma, dará lugar a modelos bien definidos.

Al margen de este tema y como comentario final, indicar que en los modelos en Modelica habitualmente se incluyen sentencias **annotation** con la documentación de los modelos en formato HTML, el nombre y otros datos del autor, la versión de la librería estándar de Modelica con la que se ha desarrollado el modelo y los atributos gráficos de la clase. La representación gráfica de una clase consiste en dos niveles de abstracción: el icono y el diagrama.

- El **icono** representa la clase, sin mostrar los detalles de su estructura interna.
- El **diagrama** muestra la estructura interna de la clase: los iconos de los componentes de la clase, sus conectores y las conexiones entre ellos. Además, en el diagrama se pueden añadir elementos gráficos adicionales, que faciliten comprender la estructura interna de la clase, tales como letreros de texto, objetos gráficos (líneas, rectángulos, elipses, etc.), etc.

Por consiguiente, la información gráfica se da en tres contextos diferentes: asociada a una clase, para especificar la representación gráfica de su icono y su diagrama; asociada a un componente, para especificar la posición y el tamaño del componente; y asociada con las conexiones (ruta de la conexión, color, etc.). La información gráfica de los componentes y las conexiones forma parte de la definición del diagrama de la clase.

Aunque se observa una cierta estandarización, las anotaciones soportadas y la sintaxis de las anotaciones puede variar de un entorno de modelado a otro. En vista de ello y a fin de no inducir a error al lector, en este texto no entraremos en detalles acerca de cómo definir las anotaciones. Se recomienda consultar la documentación del entorno de modelado y observar cómo están definidas las anotaciones en las librerías que proporciona el entorno de modelado.

2.6.4. Ejemplo 4: disipación del calor de un circuito

Este ejemplo servirá para introducir el concepto de **campo físico** en Modelica, mediante el cual se puede especificar de forma cómoda que una variable, conector o función, definida en una clase, es alias de las definidas en sus componentes. Veamos en primer lugar el procedimiento para la declaración de **variables alias**.

Las formas que hemos visto hasta el momento de acceder a las variables de un modelo desde otro son los siguientes:

- *A través de la interfaz.* Si se pretende desde un modelo acceder a la variable de otro modelo que se encuentra en su mismo nivel jerárquico, se define la variable como parte de la interfaz de ambos modelos, incluyéndola dentro de un conector como variable across, y se realiza la conexión entre los dos modelos. De esta manera, la variable definida en uno de los modelos es igualada a la variable definida en el otro.
- *Mediante la notación punto.* Si desde un modelo se desea acceder a una variable de uno de sus componentes, puede hacerse mediante una ecuación, empleando la notación punto para referirse a la variable del componente. Por ejemplo, si la variable x de un modelo quiere igualarse a la variable y definida dentro de su componente A , puede incluirse en el modelo la ecuación:

$$x = A.y;.$$

La idea de los campos físicos es facilitar que determinadas magnitudes de un modelo puedan estar accesibles desde la definición de los componentes del modelo, y viceversa. Veamos un ejemplo.

Supongamos una clase, denominada **Entorno**, que está compuesta por tres objetos de la clase **Componente**, llamados $C1$, $C2$ y $C3$.

```
model Entorno
  Componente C1, C2, C3;
  ...
end Entorno;
```

Se desea que una variable declarada en **Entorno**, llamada $T0$, sea igual a una variable, también llamada $T0$, que ha sido declarada en la clase **Componente**. Supongamos que dicha variable representa la temperatura a la que se encuentra el entorno. Una manera de proceder es incluir en la clase **Entorno** las correspondientes ecuaciones, accediendo a la variable de los componentes mediante la notación punto. Es decir:

```
model Componente
  Real T0;
  ...
end Componente;

model Entorno
  Real T0;
```

```

    Componente C1, C2, C3;
    ...
equation
    C1.T0 = T0;
    C2.T0 = T0;
    C3.T0 = T0;
end Entorno;

```

La desventaja de este procedimiento para igualar las variables es que obliga a definir una ecuación por cada objeto de la clase **Componente** que se incluya en la clase **Entorno**.

Si siempre que se defina un objeto **Componente** dentro de la clase **Entorno**, deben igualarse las variables **T0** del objeto y de la clase **Entorno**, cabe plantearse si el lenguaje de modelado podría proporcionar un procedimiento para especificar esto al definir las clases, sin necesidad de tener que especificarlo cada vez que se incluye un objeto.

Con el fin de facilitar la definición de variables alias en modelos a distinto nivel jerárquico, se introdujeron en Modelica las palabras reservadas **inner** y **outer**.

Así pues, si cada vez que se defina un objeto de la clase **Componente** dentro de la clase **Entorno**, debe igualarse la variable **T0** de **Entorno** a la variable **T0** del objeto, esto puede especificarse declarando la variable **T0** en **Componente** como *outer* y la variable **T0** en **Entorno** como *inner*:

```

model Componente
    outer Real T0;
    ...
end Componente;

model Entorno
    inner Real T0;
    Componente C1, C2, C3;
    ...
end Entorno;

```

De esta manera, el entorno de modelado reconoce que se trata de variables alias e incluye de manera automática en el modelo las ecuaciones igualando las variables.

Esta misma idea puede aplicarse a la declaración de **conectores alias**, de modo que cada vez que se cree un objeto dentro de una clase contenedora, se establezca automáticamente la conexión entre un conector de la clase contenedora y un conector del objeto.

Supongamos que se define el conector **calorF**

```
connector calorF
...
end calorF;
```

tanto en la clase **Entorno** como en la clase **Componente**, como *inner* y *outer* respectivamente, de manera que ambos conectores se convierten en alias el uno del otro. Esto permite escribir la ecuación de conexión dentro de la clase **Componente**, en lugar de tener que escribirla en la clase **Entorno**.

```
model Componente
  outer calorF Qenv;
  calorF Qcomp;
  ...
equation
  connect(Qcomp, Qenv);
  ...
end Componente;

model Entorno
  inner calorF Qenv;
  Componente C1, C2, C3;
  ...
end Entorno;
```

Al estar escrita la ecuación de conexión en la clase **Componente**, cada vez que se cree un objeto de esa clase, se establece automáticamente la conexión.

Veamos a continuación otro ejemplo. En la Figura 2.45 se muestra el modelo térmico de un circuito, compuesto de tres componentes, y que se encuentra conectado a un modelo de su entorno. El entorno se encuentra a una temperatura constante de 300 K. Cada uno de los componentes eléctricos disipa una cantidad de calor que, sumada a la de los otros componentes, es entregada al entorno. En el Código 2.24 se muestra el modelo.

Es posible modificar el modelo, tal como se muestra en el Código 2.25, usando *inner* y *outer*, de modo que cada vez que se instancie un componente de la clase **Componente** dentro de **Circuito**, el calor disipado por este componente se suma al calor total que abandona el circuito y entra en el entorno. Para ello, al instanciar el componente se establece automáticamente la conexión entre el conector del componente y el conector del circuito.

Al igual que mediante *inner* y *outer* pueden definirse alias de variables y conectores, también pueden definirse **alias de funciones**. Veamos un ejemplo. Supongamos que la temperatura del entorno no es uniforme, sino que depende de la posición. El cálculo se realiza en una función, cuya entrada es la posición y cuya salida es la

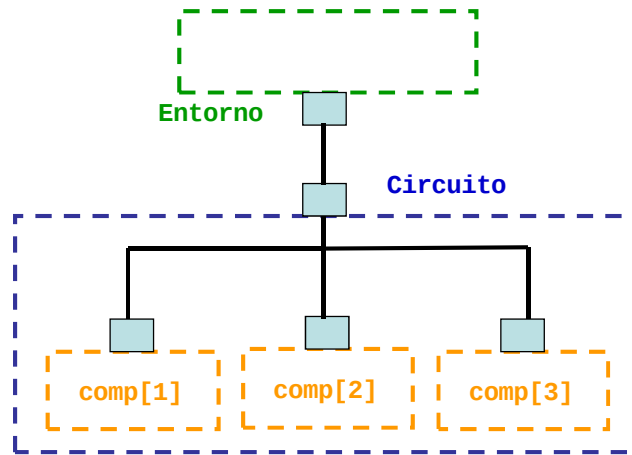


Figura 2.45: Diagrama de un circuito eléctrico con tres componentes y su entorno.

```

connector flujoCalor
  Modelica.SIunits.Temp_K T;
  flow Modelica.SIunits.HeatFlux q;
end flujoCalor;

model Componente
  flujoCalor heat;
  parameter Real qGenerado;
equation
  heat.q = qGenerado;
end Componente;

model Entorno
  flujoCalor heat;
equation
  heat.T = 300;
end Entorno;

model Circuito
  flujoCalor heat;
  Componente comp[3]
    (qGenerado={100,200,230});
equation
  for i in 1:3 loop
    connect(comp[i].heat, heat);
  end for;
end Circuito;

model Completo
  Circuito C;
  Entorno E;
equation
  connect(C.heat, E.heat);
end Completo;

```

Código 2.24: Modelo del flujo de calor entre un circuito eléctrico y su entorno.


```

connector flujoCalor
  Modelica.SIunits.Temp_K T;
  flow Modelica.SIunits.HeatFlux q;
end flujoCalor;

model Componente
  outer flujoCalor heatCircuito;
  flujoCalor heat;
  parameter Real qGenerado;
equation
  heat.q = qGenerado;
  connect (heat, heatCircuito);
end Componente;

model Entorno
  flujoCalor heat;
equation
  heat.T = 300;
end Entorno;

model Circuito
  inner flujoCalor heatCircuito;
  Componente comp[3]
    (qGenerado={100,200,230});
end Circuito;

model Completo
  Circuito C;
  Entorno E;
equation
  connect (C.heatCircuito, E.heat);
end Completo;

```

Código 2.25: Flujo de calor entre el circuito y su entorno usando *inner* y *outer*.

temperatura en esa posición. La interfaz de la función puede definirse como una función parcial,

```

partial function InterfazTempEntorno
  input Real r[3];
  output Real T;
end InterfazTempEntorno;

```

que será heredada por las N diferentes funciones que se definen para calcular, de N formas diferentes, la distribución espacial de la temperatura:

<pre>function TempEntorno1 extends InterfazTempEntorno; algorithm T := ... end TempEntorno1;</pre>	...	<pre>function TempEntornoN extends InterfazTempEntorno; algorithm T := ... end TempEntornoN;</pre>
--	-----	--

Varios objetos de la clase `ComponenteElectrico` se encuentran en el entorno descrito mediante la clase `Entorno`.

```
model Entorno
  ComponenteElectrico C1, C2, C3;
  ...
end Entorno;
```

Dado que se empleará la misma expresión de la temperatura del entorno para todos los componentes eléctricos, sería cómodo especificar en la clase `Entorno` qué función debe usarse. Por otra parte, la función que calcula la temperatura es invocada en el modelo del componente eléctrico.

Mediante *inner* y *outer* puede indicarse que la función `tempEntorno` de la clase `Entorno` es alias de la función `tempEntorno` definida en `ComponenteElectrico`, para aquellos objetos de la clase `ComponenteElectrico` instanciados dentro de `Entorno`.

```
model Entorno
  inner function tempEntorno = TempEntorno1;
  ComponenteElectrico C1, C2, C3;
  ...
end Entorno;

model ComponenteElectrico
  outer function tempEntorno = InterfazTempEntorno;
  Real r[3];
  ...
equation
  r = ...
  TempEnv = tempEntorno(r);
  ...
end ComponenteElectrico;
```

Obsérvese que al definir la función dentro de la clase `ComponenteElectrico`, se especifica la superclase que deben tener las funciones, `InterfazTempEntorno`, que es la interfaz de la función.

2.7. LECTURAS RECOMENDADAS

En el artículo (Åström et al. 1998) se explican las ventajas de los lenguajes de modelado basados en ecuaciones y orientados a objetos, demostrando que estos lenguajes resultan más adecuados para el modelado de sistemas físicos que los entornos de simulación basados en diagramas de bloques.

El sitio web de la Modelica Association es una de las fuentes principales de documentación sobre el lenguaje Modelica y sus librerías. Los artículos (Elmqvist 1993, Elmqvist et al. 1993, 1994, 1998, 1999, Fritzson et al. 2002) describen los primeros pasos en el desarrollo del lenguaje Modelica. El texto (Fritzson 2015) es una referencia muy completa sobre el estado actual del lenguaje.

El texto (Cellier 1991) es una excelente introducción al modelado de tiempo continuo. Los textos (Froment & Bischoff 1979, Bird et al. 1975, Himmelblau & Bischoff 1992, Incropera & DeWitt 1996, Ramirez 1989, Luyben 1990) son referencias fundamentales sobre modelado de fenómenos de transporte y procesos químicos. El texto (Cutlip & Shacham 1999) contiene una variedad de ejemplos sencillos de aplicaciones del modelado en el ámbito de la Ingeniería Química.

El documento (ModelicaTM 2000), que está disponible en Internet, es una introducción muy didáctica a las características básicas de Modelica.

Los grafos de ligadura (bondgraphs) es una metodología de modelado basada en la descripción del flujo, almacenamiento y transformación de la energía. Los siguientes textos son excelentes referencias sobre esta metodología (Rosenberg & Karnopp 1983, Karnopp et al. 1990, Thoma 1990, Damic & Montgomery 2003)

Se recomienda, al lector interesado en conocer más acerca de los controladores PI, consultar (Åström & Hagglund 1995).

2.8. EJERCICIOS DE AUTOCOMPROBACIÓN

Ejercicio 2.1

Entre en el sitio web de la *Modelica Association* y navegue por él. Preste especial atención a las páginas de publicaciones, a la de herramientas software y a las páginas de las International Modelica Conferences. Lea el índice de los proceedings de las últimas Int'l Modelica Conferences, con el fin de obtener una idea de qué líneas de trabajo están activas y qué grupos las desarrollan. Vea qué librerías están gratuitamente disponibles y qué librerías comerciales están anunciadas.

Descargue el documento que contiene la especificación más reciente del lenguaje Modelica. Vea qué partes tiene la especificación y adquiera una idea general de cuál es su contenido, de modo que pueda recurrir a ella cuando lo necesite.

Inspeccione el contenido de la Librería Estándar de Modelica (MSL). A continuación, emplee la librería estándar de Modelica para componer los circuitos mostrados en las Figura 2.15 y 2.20.

Ejercicio 2.2

Las Ecs. (2.98) y (2.99), denominadas modelo de Lotka-Volterra, describen la interacción entre depredadores y presas. La variable t representa el tiempo. El modelo tiene dos variables dependientes del tiempo: densidad de presas (h) y densidad de depredadores (p).

$$\frac{dh}{dt} = r \cdot h - a \cdot h \cdot p \quad (2.98)$$

$$\frac{dp}{dt} = b \cdot h \cdot p - m \cdot p \quad (2.99)$$

Los parámetros r , a , b y m no dependen del tiempo. Su significado es el siguiente:

- r Velocidad intrínseca de incremento de la población de presas.
- a Coeficiente de velocidad de la depredación.
- b Velocidad de reproducción de los depredadores por cada presa comida.
- m Tasa de mortandad de los depredadores.

Describa el modelo de Lotka-Volterra en Modelica. Asigne los siguientes valores a los parámetros:

$$r = 0.61 \quad a = 0.03 \quad b = 0.025 \quad m = 0.79 \quad (2.100)$$

y los siguientes valores a las condiciones iniciales:

$$h(0) = 2 \quad p(0) = 5 \quad (2.101)$$

Simule el modelo durante 80 unidades de tiempo. Ejecute tres réplicas independientes de la simulación. Emplee en la primera réplica el algoritmo DASSL, con tolerancia 0.0001. En las otras dos réplicas emplee el método de Euler explícito de primer orden, con tamaños del paso 0.01 y 0.0001 unidades de tiempo, respectivamente. Compare los resultados obtenidos en las tres réplicas.

Ejercicio 2.3

En la Figura 2.46 se muestra un sistema compuesto por dos muelles, un amortiguador, una polea y una palanca. Se supone que la masa de la polea y la palanca es despreciable, y que los muelles y el amortiguador se comportan linealmente. La velocidad v_1 es una función conocida del tiempo.

$$v_1 = V_{1,O} \cdot \sin(w \cdot t) \quad (2.102)$$

El criterio de signos para el sentido de la velocidad está señalado en la figura.

La distancia de los brazos de la palanca, desde el punto de apoyo de la misma hasta los puntos cuyas velocidades están señaladas como v_2 , v_3 y v_4 son L_2 , L_3 y L_4 respectivamente.

Plantee las ecuaciones del modelo, de modo que pueda calcularse la evolución de las velocidades v_2 , v_3 y v_4 , así como las fuerzas que ejercen los muelles y el amortiguador. Describa el modelo en lenguaje Modelica y simúlelo.

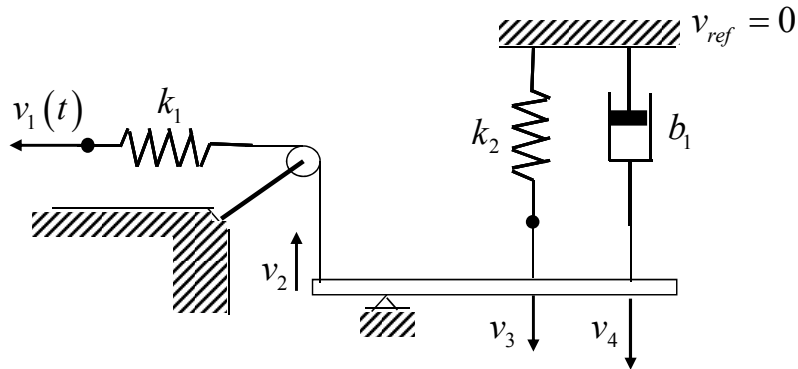


Figura 2.46: Mecanismo de polea y palanca.

Ejercicio 2.4

En la Figura 2.47 se muestra un sistema compuesto por dos muelles, una polea con masa m_{polea} e inercia rotacional J , y una carga de masa m_{carga} . Se supone que los muelles se comportan linealmente y que sus parámetros son k_1 y k_2 . La aceleración gravitatoria es g .

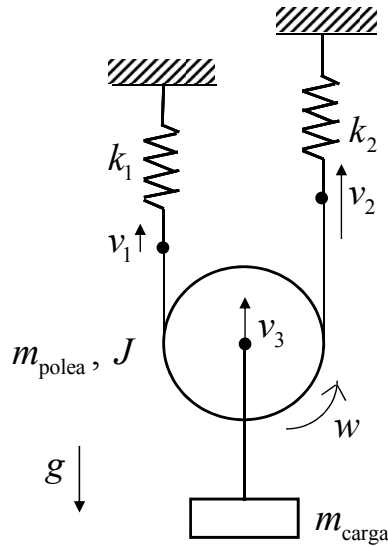


Figura 2.47: Polea con carga suspendida.

Plantee las ecuaciones del modelo, de modo que pueda calcularse la evolución en el tiempo de las velocidades (v_1 , v_2 , v_3) y la velocidad angular (w). Describa el modelo en Modelica.

Ejercicio 2.5

En la Figura 2.48 se muestra un sistema hidráulico y mecánico. Suponga que en la conducción que une las bombas hidráulicas con el depósito no se produce caída de la presión debido a la fricción. Así pues, la presión en las bombas hidráulicas y en la base del depósito es p .

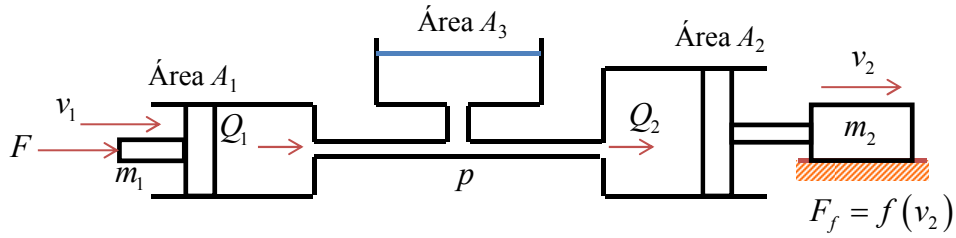


Figura 2.48: Sistema hidráulico y mecánico.

La fuerza externa aplicada F es una función conocida del tiempo. El émbolo de la bomba izquierda tiene una masa m_1 . El émbolo de la bomba derecha no tiene masa, pero se encuentra solidario a una masa m_2 que desliza con rozamiento sobre el suelo. La fuerza de fricción es una función conocida de la velocidad de la masa: $F_f = f(v_2)$.

Los parámetros A_1 y A_2 representan el área del pistón de la bomba situada a la izquierda y derecha, respectivamente. El área del orificio situado en la base del depósito se representa mediante el parámetro A_3 . La densidad del líquido es el parámetro ρ .

Explique cómo fluye la potencia a través del sistema. Plantee a continuación las ecuaciones del modelo suponiendo que la relación constitutiva de todos los componentes es lineal.

Ejercicio 2.6

Escriba una librería en Modelica que contenga modelos de los siguientes tres elementos mecánicos en 1 dimensión:

- *Masa puntual*. La relación constitutiva de este componente describe que la fuerza aplicada sobre él es igual a su masa multiplicada por la aceleración que adquiere.
- *Muelle sin masa*. La relación constitutiva describe que la fuerza que ejerce el muelle sobre cualquiera de sus extremos es igual a una constante de proporcionalidad K , expresada en N/m, multiplicada por la diferencia entre su elongación y su elongación natural. La elongación del muelle es la distancia que hay entre sus extremos. La elongación natural es aquella elongación para la cual la fuerza que ejerce el muelle sobre cualquiera de sus extremos es cero.
- *Referencia*. Se trata de un punto que se encuentra permanentemente en reposo.

A partir de estos elementos, componga diferentes sistemas de muelles y masas, incluyéndolas en un paquete de ejemplos incluido en la propia librería.

2.9. SOLUCIONES DE LOS EJERCICIOS

Solución al Ejercicio 2.1

En las Figuras 2.49 y 2.50 se muestran los diagramas de los dos circuitos. Han sido compuestos empleando Dymola 6.1 y la MSL versión 2.2.1. La descripción en Modelica de los modelos se muestra en el Código 2.26 y 2.27. Ambos modelos son compatibles con la versión 3.2.1 de la MSL, que es la actual en la fecha de escritura de este texto y pueden ser simulados con Dymola versión 2015.

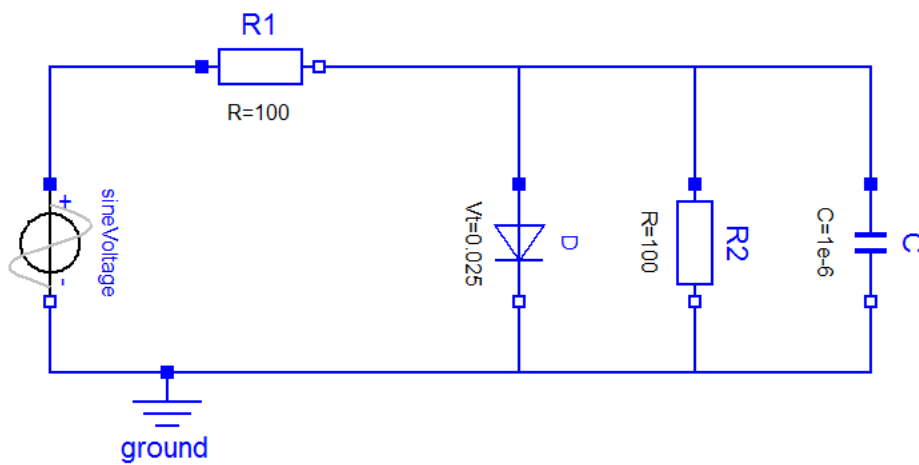


Figura 2.49: Diagrama del circuito eléctrico de la Figura 2.15, compuesto empleando Dymola 6.1 y la MSL versión 2.2.1.

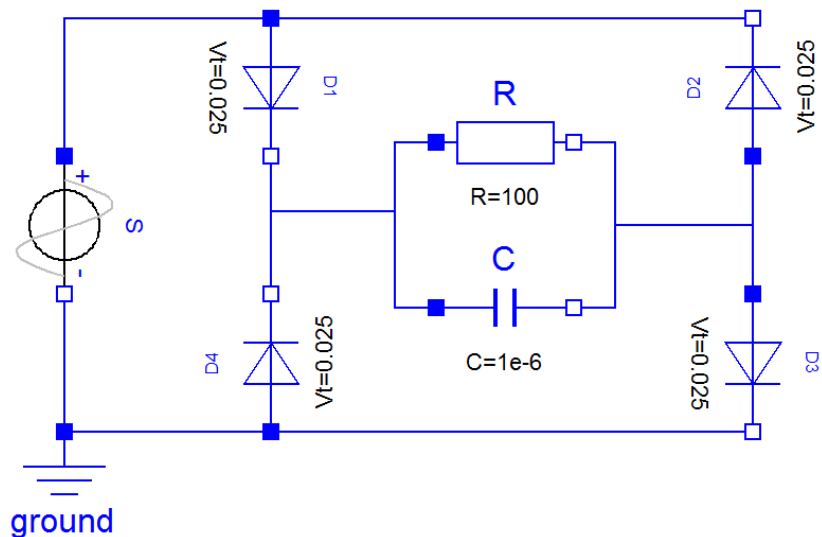


Figura 2.50: Diagrama del circuito eléctrico de la Figura 2.20, compuesto empleando Dymola 6.1 y la MSL versión 2.2.1.

```

model circuito1_MSL_2_2_1
  Modelica.Electrical.Analog.Basic.Ground ground
    annotation (extent=[-70,-12; -50,8]);
  Modelica.Electrical.Analog.Basic.Resistor R1(R=100)
    annotation (extent=[-54,50; -34,70]);
  Modelica.Electrical.Analog.Basic.Capacitor C(C=1e-6)
    annotation (extent=[50,20; 70,40], rotation=270);
  Modelica.Electrical.Analog.Semiconductors.Diode D(Ids=1e-9, Vt=0.025)
    annotation (extent=[-10,20; 10,40], rotation=270);
  Modelica.Electrical.Analog.Sources.SineVoltage sineVoltage(V=5, freqHz=100)
    annotation (extent=[-90,20; -70,40], rotation=270);
  annotation (Diagram);
  Modelica.Electrical.Analog.Basic.Resistor R2(R=100)
    annotation (extent=[20,20; 40,40], rotation=270);
equation
  connect(sineVoltage.p, R1.p)
    annotation (points=[-80,40; -80,60; -54,60],
      style(color=3, rgbcolor={0,0,255}));
  connect(R1.n, D.p)
    annotation (points=[-34,60; 0,60; 0,40; 1.83691e-015,40],
      style(color=3, rgbcolor={0,0,255}));
  connect(C.p, R1.n)
    annotation (points=[60,40; 60,60; -34,60],
      style(color=3, rgbcolor={0,0,255}));
  connect(sineVoltage.n, ground.p)
    annotation (points=[-80,20; -80,8; -60,8],
      style(color=3, rgbcolor={0,0,255}));
  connect(D.n, ground.p)
    annotation (points=[-1.83691e-015,20; 0,20; 0,8; -60,8],
      style(color=3, rgbcolor={0,0,255}));
  connect(C.n, ground.p)
    annotation (points=[60,20; 60,8; -60,8],
      style(color=3, rgbcolor={0,0,255}));
  connect(R2.p, R1.n)
    annotation (points=[30,40; 30,60; -34,60],
      style(color=3, rgbcolor={0,0,255}));
  connect(R2.n, ground.p)
    annotation (points=[30,20; 30,8; -60,8],
      style(color=3, rgbcolor={0,0,255}));
  annotation (uses(Modelica(version="2.2.1")));
end circuito1_MSL_2_2_1;

```

Código 2.26: Circuito mostrado en la Figura 2.49.

```

model circRectificador_MSL_2_2_1
  Modelica.Electrical.Analog.Basic.Ground ground
    annotation (extent=[-70,-20; -50,0]);
  Modelica.Electrical.Analog.Basic.Resistor R(R=100)
    annotation (extent=[-6,32; 14,52], rotation=0);
  Modelica.Electrical.Analog.Basic.Capacitor C(C=1e-6)
    annotation (extent=[-6,8; 14,28], rotation=0);
  Modelica.Electrical.Analog.Semiconductors.Diode D1(Ids=1e-9, Vt=0.025)
    annotation (extent=[-40,40; -20,60], rotation=270);
  Modelica.Electrical.Analog.Sources.SineVoltage S(V=5, freqHz=10)
    annotation (extent=[-70,20; -50,40], rotation=270);
  Modelica.Electrical.Analog.Semiconductors.Diode D2(Ids=1e-9, Vt=0.025)
    annotation (extent=[30,40; 50,60], rotation=90);
  Modelica.Electrical.Analog.Semiconductors.Diode D3(Ids=1e-9, Vt=0.025)
    annotation (extent=[30,0; 50,20], rotation=270);
  Modelica.Electrical.Analog.Semiconductors.Diode D4(Ids=1e-9, Vt=0.025)
    annotation (extent=[-40,0; -20,20], rotation=90);
equation
  connect(S.n, ground.p)
    annotation (points=[-60,20; -60,0],
      style(color=3, rgbcolor={0,0,255}));
  annotation (Diagram);
  connect(D2.p, D3.p)
    annotation (points=[40,40; 40,20],
      style(color=3, rgbcolor={0,0,255}));
  connect(C.p, R.p)
    annotation (points=[-6,18; -12,18; -12,42; -6,42],
      style(color=3, rgbcolor={0,0,255}));
  connect(R.n, C.n)
    annotation (points=[14,42; 20,42; 20,18; 14,18],
      style(color=3, rgbcolor={0,0,255}));
  connect(D4.n, D1.n)
    annotation (points=[-30,20; -30,40],
      style(color=3, rgbcolor={0,0,255}));
  connect(R.p, D1.n)
    annotation (points=[-6,42; -12,42; -12,32; -30,32; -30,40],
      style(color=3, rgbcolor={0,0,255}));
  connect(D2.p, R.n)
    annotation (points=[40,40; 40,30; 20,30; 20,42; 14,42],
      style(color=3, rgbcolor={0,0,255}));
  connect(D4.p, D3.n)
    annotation (points=[-30,0; 40,0],
      style(color=3, rgbcolor={0,0,255}));
  connect(D1.p, D2.n)
    annotation (points=[-30,60; 40,60],
      style(color=3, rgbcolor={0,0,255}));
  connect(D4.p, ground.p)
    annotation (points=[-30,0; -60,0],
      style(color=3, rgbcolor={0,0,255}));
  connect(D1.p, S.p)
    annotation (points=[-30,60; -60,60; -60,40],
      style(color=3, rgbcolor={0,0,255}));
  annotation (uses(Modelica(version="2.2.1")));
end circRectificador_MSL_2_2_1;

```

Código 2.27: Circuito mostrado en la Figura 2.50.

Solución al Ejercicio 2.2

La descripción en Modelica del modelo de Lotka-Volterra se muestra en el Código 2.28. Puesto que en el enunciado no se ha indicado en qué unidades están expresados los valores numéricos de los parámetros y las condiciones iniciales, esta información no se incorpora al modelo en Modelica, limitándonos a indicar que las variables y parámetros son de tipo *Real*.

```

model lotkaVolterra
  parameter Real r = 0.61;
  parameter Real a = 0.03;
  parameter Real b = 0.025;
  parameter Real m = 0.79;
  Real h(start=2, fixed=true);
  Real p(start=5, fixed=true);
equation
  der(h) = r*h - a*h*p;
  der(p) = b*h*p - m*p;
end lotkaVolterra;

```

Código 2.28: Modelo de Lotka-Volterra.

El algoritmo DASSL es un método de integración de orden y paso variable (se explicará en la Sección 3.8.1). El algoritmo explícito de Euler es un método de primer orden y paso fijo.

En la Figura 2.51 se muestra la evolución de la densidad de presas y depredadores obtenida en las tres réplicas de la simulación. En la parte superior de la figura se muestra la solución obtenida empleando el algoritmo DASSL.

Se observa a simple vista que la solución numérica obtenida empleando el método de Euler con tamaño del paso 0.01 es inestable: la amplitud de las oscilaciones crece con el tiempo. Reduciendo el tamaño del paso a 0.0001, se obtiene una solución que en el intervalo de tiempo analizado es visualmente similar a la obtenida con DASSL.

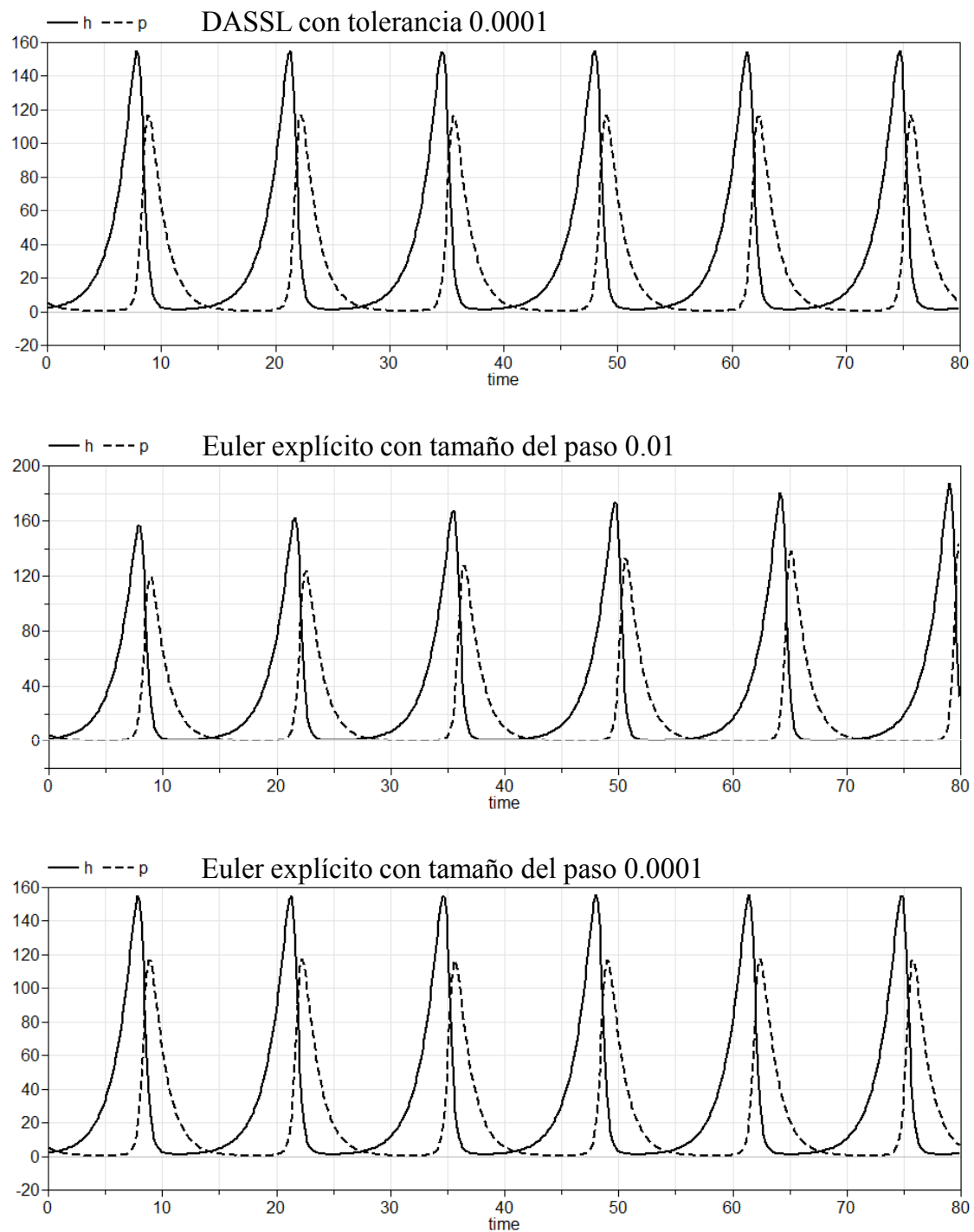


Figura 2.51: Simulación del Código 2.28.

Solución al Ejercicio 2.3

En el diagrama del sistema están señaladas las velocidades. Asignemos nombre a las fuerzas ejercidas por los muelles y el amortiguador: sean F_2 , F_3 y F_4 las fuerzas en los puntos en los cuales están señaladas las velocidades v_2 , v_3 y v_4 , respectivamente. El criterio de signos para el sentido de la fuerza es el mismo que hay dibujado en la Figura 2.46 para las respectivas velocidades.

Llamaremos e_1 y e_2 a la diferencia entre la elongación y la elongación natural de los muelles cuyas constantes son k_1 y k_2 , respectivamente. A continuación se muestran las ecuaciones del modelo, que tiene 9 variables desconocidas: las dos elongaciones efectivas de los muelles (e_1 , e_2), definidas como la diferencia entre la elongación del muelle y su elongación natural, las cuatro velocidades (v_1 , v_2 , v_3 , v_4) y las tres fuerzas ejercidas por los componentes (F_2 , F_3 , F_4).

$$v_1 = V_{10} \cdot \sin(w \cdot \text{time}) \quad (2.103)$$

$$F_2 = k_1 \cdot e_1 \quad (2.104)$$

$$\frac{de_1}{dt} = v_1 - v_2 \quad (2.105)$$

$$F_2 \cdot L_2 + F_3 \cdot L_3 + F_4 \cdot L_4 = 0 \quad (2.106)$$

$$\frac{v_2}{L_2} = \frac{v_3}{L_3} \quad (2.107)$$

$$\frac{v_2}{L_2} = \frac{v_4}{L_4} \quad (2.108)$$

$$F_3 = -k_2 \cdot e_2 \quad (2.109)$$

$$\frac{de_2}{dt} = v_3 \quad (2.110)$$

$$F_4 = -b_1 \cdot v_4 \quad (2.111)$$

La Ec. (2.103) describe la evolución de la velocidad del extremo del muelle, las Ecs. (2.104) y (2.105) al Muelle 1, las Ecs. (2.106) – (2.108) describen el comportamiento de la palanca, las Ecs. (2.109) y (2.110) describen al Muelle 2, y finalmente la Ec. (2.111) describe el amortiguador.

La descripción en Modelica del modelo se muestra en el Código 2.29.

```

model poleaPalanca

    import SI = Modelica.SIunits;

    // Velocidad del extremo del muelle
    parameter SI.Velocity V10 = 0.01;
    parameter SI.AngularFrequency w = 1;

    // Longitudes de los brazos de la palanca
    parameter SI.Length L2 = 1;
    parameter SI.Length L3 = 2;
    parameter SI.Length L4 = 3;

    // Muelles y amortiguador
    parameter SI.TranslationalSpringConstant k1 = 5;
    parameter SI.TranslationalSpringConstant k2 = 10;
    parameter SI.TranslationalDampingConstant b1 = 5;

    SI.Length e1(start=0, fixed=true); // elongación - elongación natural
    SI.Length e2(start=0, fixed=true);
    SI.Velocity v1, v2, v3, v4;
    SI.Force F2, F3, F4;
equation

    // Velocidad del extremo del muelle
    v1 = V10*sin(w*time);
    // Muelle 1
    F2 = k1*e1;
    der(e1) = v1 - v2;

    // Palanca
    F2*L2 + F3*L3 + F4*L4 = 0;
    v2/L2 = v3/L3;
    v2/L2 = v4/L4;

    // Muelle 2
    F3 = -k2*e2;
    der(e2) = v3;

    // Amortiguador
    F4 = -b1*v4;

    annotation (uses(Modelica(version="3.2.1")));
end poleaPalanca;

```

Código 2.29: Modelo del sistema mecánico mostrado en la Figura 2.46.

Solución al Ejercicio 2.4

Sean F_1 y F_2 las fuerzas ejercidas por los Muelles 1 y 2. Se adopta el mismo criterio de signos para la fuerza que para la velocidad: la fuerza es positiva cuando tiene sentido ascendente. El modelo está compuesto por las ecuaciones siguientes.

$$F_1 = k_1 \cdot e_1 \quad (2.112)$$

$$\frac{de_1}{dt} = -v_1 \quad (2.113)$$

$$F_2 = k_2 \cdot e_2 \quad (2.114)$$

$$\frac{de_2}{dt} = -v_2 \quad (2.115)$$

$$(m_{polea} + m_{carga}) \cdot \frac{dv_3}{dt} = F_1 + F_2 - g \cdot (m_{polea} + m_{carga}) \quad (2.116)$$

$$v_1 = v_3 - R_{polea} \cdot w \quad (2.117)$$

$$v_2 = v_3 + R_{polea} \cdot w \quad (2.118)$$

$$J \cdot \frac{dw}{dt} = R_{polea} \cdot (F_2 - F_1) \quad (2.119)$$

Las Ecs. (2.112) y (2.113) describen el Muelle 1, y las Ecs. (2.114) y (2.115) el Muelle 2. Las Ecs. (2.116) – (2.119) describen la polea y la carga.

El modelo tiene 8 variables desconocidas: las elongaciones efectivas de los muelles (e_1 , e_2), que son la diferencia entre la elongación del muelle y su elongación natural, las fuerzas (F_1 , F_2), las tres velocidades lineales (v_1 , v_2 , v_3) y la velocidad angular de la polea (w). El momento de inercia de la polea se calcula suponiendo que ésta es un disco delgado:

$$J = \frac{1}{2} \cdot m_{polea} \cdot R_{polea}^2 \quad (2.120)$$

donde m_{polea} es la masa de la polea y R_{polea} su radio.

La descripción del modelo en Modelica se muestra en el Código 2.30.

```

model poleaMuelles

import SI = Modelica.SIunits;

constant SI.Acceleration g = 9.81;
// Muelles
parameter SI.TranslationalSpringConstant k1 = 5;
parameter SI.TranslationalSpringConstant k2 = 10;
// Polea
parameter SI.Mass Mpolea = 0.5;
parameter SI.Radius Rpolea = 0.3;
parameter SI.MomentOfInertia J = 0.5*Mpolea*Rpolea^2;
// Carga
parameter SI.Mass Mcarga = 10;

SI.Length e1(start=0, fixed=true); // elongación - elongación natural
SI.Length e2(start=0, fixed=true);
SI.Velocity v1, v2, v3(start=0, fixed=true);
SI.Force F1, F2;
SI.AngularVelocity w(start=0, fixed=true);

equation
// Muelle 1
F1 = k1*e1;
der(e1) = -v1;

// Muelle 2
F2 = k2*e2;
der(e2) = -v2;

// Polea y carga
(Mpolea + Mcarga)*der(v3) = F1 + F2 - g*(Mpolea + Mcarga);
v1 = v3 - Rpolea*w;
v2 = v3 + Rpolea*w;
J*der(w) = Rpolea*(F2 - F1);

annotation (uses(Modelica(version="3.2.1")));
end poleaMuelles;

```

Código 2.30: Modelo del sistema mostrado en la Figura 2.47.

Solución al Ejercicio 2.5

Analicemos primeramente cómo fluye, se almacena, se transforma y se disipa la potencia en este sistema. Para mayor comodidad del lector, en la Figura 2.52 puede verse nuevamente el diagrama del sistema.

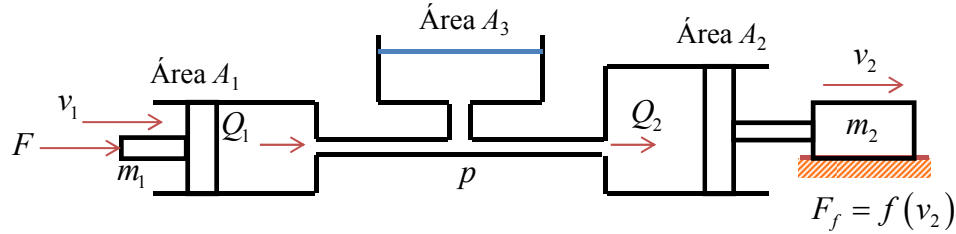


Figura 2.52: Sistema hidráulico y mecánico.

Se aplica una fuerza externa F sobre el émbolo de la bomba izquierda, el cual se desplaza con velocidad v_1 . El entorno fija la evolución de la fuerza F , por tanto el entorno actúa como una fuente de potencia que entrega una potencia $F \cdot v_1$ al sistema.

Parte de esta potencia es almacenada como energía cinética del émbolo, el cual tiene una masa m_1 y se desplaza a velocidad v_1 . El resto es transformada en potencia hidráulica, produciéndose un flujo de líquido Q_1 saliente de la bomba izquierda, encontrándose el líquido a una presión p . Esta potencia hidráulica es igual a $Q_1 \cdot p$.

Una parte de esta potencia hidráulica $Q_1 \cdot p$ saliente de la bomba izquierda se almacena en forma de energía potencial del líquido que se encuentra contenido en el depósito de área A_3 . En resto, $Q_2 \cdot p$, fluye hacia la bomba derecha.

Esta potencia hidráulica entrante a la bomba derecha es transformada en potencia mecánica, desplazando con una velocidad v_2 el émbolo de la bomba derecha y la masa m_2 que se encuentra solidaria a él. Una parte de esta potencia se almacena en forma de energía cinética de movimiento de la masa m_2 . El resto se disipa debido al rozamiento producido por el movimiento de la masa m_2 sobre el suelo.

Planteemos ahora las ecuaciones que describen el comportamiento del sistema. La fuerza externa aplicada es una función conocida del tiempo.

$$F = f(t) \quad (2.121)$$

Las ecuaciones que describen el comportamiento de la la bomba izquierda son las siguientes:

$$F = F_{m1} + F_1 \quad (2.122)$$

$$m_1 \cdot \frac{dv_1}{dt} = F_{m1} \quad (2.123)$$

$$F_1 = A_1 \cdot p \quad (2.124)$$

$$A_1 \cdot v_1 = Q_1 \quad (2.125)$$

La presión en la base del depósito se relaciona con el peso del líquido contenido en el depósito de la forma siguiente:

$$p = \frac{\rho \cdot V \cdot g}{A_3} \quad (2.126)$$

donde ρ es la densidad del líquido, V su volumen, g la aceleración gravitatoria y A_3 la sección del agujero situado en la base del tanque. Derivando esta ecuación se obtiene:

$$\frac{dp}{dt} = \frac{\rho \cdot g}{A_3} \cdot \frac{dV}{dt} \quad (2.127)$$

Teniendo en cuenta que la variación en el volumen del líquido almacenado en el depósito es igual al caudal entrante Q_3 , es decir, $\frac{dV}{dt} = Q_3$, la relación constitutiva del depósito es:

$$\frac{dp}{dt} = \frac{\rho \cdot g}{A_3} \cdot Q_3 \quad (2.128)$$

que corresponde a un condensador de capacidad $\frac{A_3}{\rho g}$.

En el punto de inserción del tanque en la conducción horizontal se plantea la siguiente ecuación de conservación

$$Q_1 = Q_2 + Q_3 \quad (2.129)$$

según la cual el caudal Q_1 saliente de la bomba izquierda es igual a la suma del caudal entrante a la bomba derecha (Q_2) y el caudal entrante al depósito (Q_3).

En la bomba situada en la parte derecha del diagrama se produce conversión de potencia del dominio hidráulico al mecánico:

$$F_2 = A_2 \cdot p \quad (2.130)$$

$$A_2 \cdot v_2 = Q_2 \quad (2.131)$$

La potencia mecánica se acumula en parte en la inercia y en parte se disipa debido a la fricción.

$$F_2 = F_{m2} + F_f \quad (2.132)$$

$$F_f = f(v_2) \quad (2.133)$$

$$m_2 \cdot \frac{dv_2}{dt} = F_{m2} \quad (2.134)$$

Recapitulando, el modelo está compuesto por 13 ecuaciones, (2.121) – (2.126) y (2.128) – (2.134), de las cuales deben evaluarse las siguientes 13 variables desconocidas: F , F_1 , F_2 , F_{m1} , F_{m2} , F_f , v_1 , v_2 , p , V , Q_1 , Q_2 y Q_3 .

La densidad del líquido (ρ), la aceleración gravitatoria (g), la sección del agujero situado en la base del tanque (A_3), las secciones de los émbolos (A_1 , A_2) y las masas (m_1 , m_2) son parámetros de valor conocido.

Solución al Ejercicio 2.6

En la Figura 2.53 se muestra la arquitectura de la librería. Se ha definido un conector llamado *Port* y dos clases parciales que definen las interfaces, y se han incluido en el paquete *Interfaces*. Los diagramas de los modelos incluidos en el paquete *Ejemplos* se muestran en las Figuras 2.54 – 2.56. La descripción en Modelica se muestra en el Código 2.31 – 2.35.

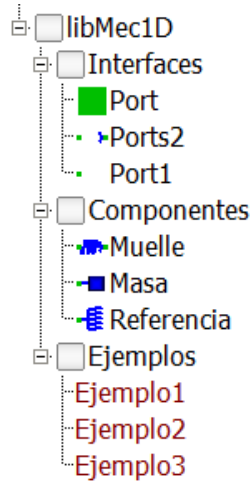


Figura 2.53: Arquitectura de la librería mecánica.

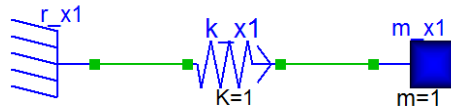


Figura 2.54: Diagrama del modelo `libMec1D.Ejemplos.Ejemplo1`.

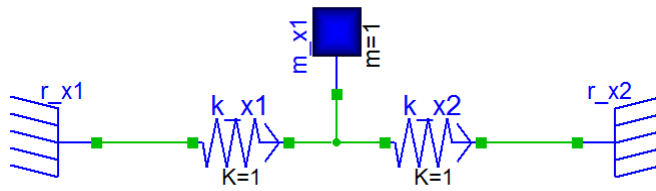


Figura 2.55: Diagrama del modelo `libMec1D.Ejemplos.Ejemplo2`.

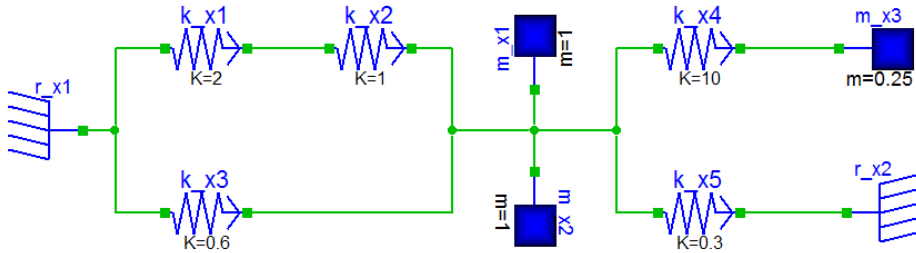


Figura 2.56: Diagrama del modelo `libMec1D.Ejemplos.Ejemplo3`.

```

package libMec1D "Libreria mecanica 1D"

package Interfaces "Interfaz de las clases de sistemas mecanicos"

connector Port "Posicion / Fuerza"
  Modelica.SIunits.Position x;
  flow Modelica.SIunits.Force f;
  annotation (
    Icon(Rectangle(extent=[-100, 100; 100, -100],
      style(pattern=0, fillColor=58, fillPattern=1)))
  );
end Port;

partial model Ports2
  Port p1 annotation (extent=[-110, -10; -90, 10]);
  Port p2 annotation (extent=[90, -10; 110, 10]);
  annotation (
    Icon(Coordsys(extent=[-100, -100; 100, 100], grid=[1, 1]),
      Line(points=[50, 40; 80, 0; 50, -40])),
    Documentation(info="
Dos conectores ( posicion / fuerza )
El modelo impone que se indique cual de los dos terminales
tiene mayor coordenada espacial. p2.x > p1.x
En el icono de la clase, la flecha esta en p2, apuntando
en la direccion de la coordenada espacial creciente.
Se supone que un extremo no puede 'pasar sobre el otro', con
lo cual, será siempre el mismo extremo el que posea coordenada
espacial mayor.
"));
end Ports2;

partial model Port1
  Port p annotation (extent=[-110, -10; -90, 10]);
  annotation (
    Icon(Coordsys(extent=[-100, -100; 100, 100], grid=[1, 1]),
      Documentation(info="Un conector ( posicion / fuerza )")
    );
end Port1;

end Interfaces;

```

Código 2.31: Librería mecánica en una dimensión (1/5).

```

package Componentes

model Muelle "Muelle ideal"
  extends Interfaces.Ports2;
  parameter Real K(unit="N/m") = 1 "Coef. restitution";
  parameter Modelica.SIunits.Length ElongNat=1 "Elongacion natural";
protected
  Modelica.SIunits.Length elong "Elongacion";
equation
  elong = p2.x - p1.x;
  p1.f = K*(elong - ElongNat);
  p2.f = -p1.f;
  annotation (
    Coordsys(extent=[-100, -100; 100, 100], grid=[1, 1]),
    Icon(
      Line(points=[-90, 0; -80, 0; -70, -50; -50, 50; -30, -50;
                    -10, 50; 10, -50; 30, 50; 40, 0; 90, 0]),
      Text(extent=[-98, 113; 98, 42], string="%name"),
      Text(
        extent=[-94, -46; 102, -102],
        string="%K",
        style(color=0))),
    Documentation(info="Muelle ideal.  $F = -K \cdot (\text{elong} - \text{elongNat})$ "));
end Muelle;

model Masa "Masa puntual"
  extends Interfaces.Port1;
  parameter Modelica.SIunits.Mass m=1 "Masa";
protected
  Modelica.SIunits.Velocity v "Velocidad";
equation
  der(p.x) = v;
  -p.f = m*der(v);
  annotation (
    Coordsys(extent=[-100, -100; 100, 100], grid=[1, 1]),
    Icon(
      Line(points=[-90, 0; -20, 0]),
      Rectangle(extent=[-20, -50; 80, 50], style(gradient=3)),
      Text(extent=[-98, 107; 99, 46], string="%name"),
      Text(
        extent=[-98, -45; 100, -105],
        string="%m",
        style(color=0))),
    Documentation(info="
Masa puntual.  $-F = m \cdot a$ 
La variable fuerza del conector representa la fuerza
aplicada por el elemento sobre el punto de union,
con lo cual va afectada de un signo menos en la
relacion constitutiva.
"));
end Masa;

```

Código 2.32: Librería mecánica en una dimensión (2/5).


```

model Referencia
    extends Interfaces.Port1;
equation
    der(p.x) = 0;
    annotation (
        Coordsys(extent=[-100, -100; 100, 100], grid=[1, 1]),
        Icon(
            Line(points=[-90, 0; -20, 0]),
            Line(points=[-20, 70; -20, -70]),
            Line(points=[-20, 70; 80, 95]),
            Line(points=[-20, 35; 80, 60]),
            Line(points=[-20, 0; 80, 25]),
            Line(points=[-20, -35; 80, -10]),
            Line(points=[-20, -70; 80, -45]),
            Text(extent=[-130, 135; 67, 74], string="%name")),
        Documentation(info="
Referencia. der(x) = 0
La referencia impone que el punto de conexión permanezca inmóvil.
La fuerza que ejerce la referencia sobre el punto de conexión es
la necesaria para que esto se satisfaga.

No puede conectarse más de una referencia a un determinado
punto de conexión.
"));
end Referencia;

end Componentes;

```

Código 2.33: Librería mecánica en una dimensión (3/5).

```

package Ejemplos

model Ejemplo1
  Componentes.Referencia r_x1 annotation (extent=[-40, 0; -60, 20]);
  annotation (Diagram);
  Componentes.Masa m_x1 annotation (extent=[20, 0; 40, 20]);
  Componentes.Muelle k_x1 annotation (extent=[-20, 0; 0, 20]);
equation
  connect(r_x1.p, k_x1.p1)
    annotation (points=[-40, 10; -20, 10], style(color=58));
  connect(k_x1.p2, m_x1.p)
    annotation (points=[0, 10; 20, 10], style(color=58));
end Ejemplo1;

model Ejemplo2
  Componentes.Referencia r_x1
    annotation (extent=[-40, 0; -60, 20]);
  annotation (Diagram);
  Componentes.Masa m_x1
    annotation (extent=[0, 20; 20, 40], rotation=90);
  Componentes.Muelle k_x1
    annotation (extent=[-20, 0; 0, 20]);
  Componentes.Muelle k_x2
    annotation (extent=[20, 0; 40, 20]);
  Componentes.Referencia r_x2
    annotation (extent=[60, 0; 80, 20]);
equation
  connect(r_x1.p, k_x1.p1)
    annotation (points=[-40, 10; -20, 10], style(color=58));
  connect(k_x2.p2, r_x2.p)
    annotation (points=[40, 10; 60, 10], style(color=58));
  connect(k_x1.p2, m_x1.p)
    annotation (points=[0, 10; 10, 10; 10, 20], style(color=58));
  connect(k_x2.p1, m_x1.p)
    annotation (points=[20, 10; 10, 10; 10, 20], style(color=58));
end Ejemplo2;

```

Código 2.34: Librería mecánica en una dimensión (4/5).

```

model Ejemplo3
  Componentes.Referencia r_x1
    annotation (extent=[-60, 0; -80, 20]);
  Componentes.Muelle k_x1(K=2)
    annotation (extent=[-40, 20; -20, 40]);
  Componentes.Muelle k_x3(K=0.6)
    annotation (extent=[-40, -20; -20, 0]);
  Componentes.Muelle k_x2
    annotation (extent=[0, 20; 20, 40]);
  Componentes.Masa m_x1
    annotation (extent=[40, 20; 60, 40], rotation=90);
  Componentes.Masa m_x2
    annotation (extent=[40, -20; 60, 0], rotation=-90);
  Componentes.Muelle k_x4(K=10)
    annotation (extent=[80, 20; 100, 40]);
  Componentes.Masa m_x3(m=0.25)
    annotation (extent=[124, 20; 144, 40]);
  Componentes.Muelle k_x5(K=0.3)
    annotation (extent=[80, -20; 100, 0]);
  Componentes.Referencia r_x2
    annotation (extent=[126, -20; 146, 0]);
  annotation (Diagram, Coordsys(extent=[-100, -100; 160, 160],
    grid=[2, 2]));
equation
  connect(r_x1.p, k_x1.p1)
    annotation (points=[-60, 10; -52, 10; -52, 30; -40, 30],
      style(color=58));
  connect(r_x1.p, k_x3.p1)
    annotation (points=[-60, 10; -52, 10; -52, -10; -40, -10],
      style(color=58));
  connect(k_x1.p2, k_x2.p1)
    annotation (points=[-20, 30; 0, 30], style(color=58));
  connect(k_x3.p2, k_x2.p2)
    annotation (points=[-20, -10; 30, -10; 30, 30; 20, 30],
      style(color=58));
  connect(m_x1.p, m_x2.p)
    annotation (points=[50, 20; 50, 0], style(color=58));
  connect(k_x2.p2, m_x2.p)
    annotation (points=[20, 30; 30, 30; 30, 10; 50, 10; 50, 0],
      style(color=58));
  connect(k_x4.p1, k_x5.p1)
    annotation (points=[80, 30; 70, 30; 70, -10; 80, -10],
      style(color=58));
  connect(k_x4.p2, m_x3.p)
    annotation (points=[100, 30; 124, 30], style(color=58));
  connect(k_x5.p2, r_x2.p)
    annotation (points=[100, -10; 126, -10], style(color=58));
  connect(m_x2.p, k_x4.p1)
    annotation (points=[50, 0; 50, 10; 70, 10; 70, 30; 80, 30],
      style(color=58));
end Ejemplo3;

end Ejemplos;

end libMec1D;

```

Código 2.35: Librería mecánica en una dimensión (5/5).

TEMA 3

SIMULACIÓN DE MODELOS DE TIEMPO CONTINUO

- 3.1. Introducción
- 3.2. Asignación de la causalidad computacional
- 3.3. Sistemas DAE estructuralmente singulares
- 3.4. Índice de los sistemas DAE
- 3.5. Inicialización de sistemas DAE
- 3.6. Lazos algebraicos
- 3.7. Selección de las variables de estado
- 3.8. Solución numérica de sistemas DAE
- 3.9. Lecturas recomendadas
- 3.10. Ejercicios de autocomprobación
- 3.11. Soluciones de los ejercicios

OBJETIVOS DOCENTES

Una vez estudiado el contenido del tema y realizados los ejercicios debería saber:

- Discutir la relación entre la descripción en Modelica de modelos de tiempo continuo y el código de simulación generado por el entorno de modelado.
- Discutir cómo se define el índice de un sistema DAE y calcularlo en casos sencillos. Discutir las dificultades que surgen al resolver numéricamente sistemas DAE con índice superior. Discutir varias alternativas para la reducción del índice, así como las ventajas y desventajas de cada una de ellas. Reducir el índice de sistemas DAE sencillos.
- Discutir la relación entre las hipótesis de modelado y los lazos algebraicos del modelo. Discutir la dificultad intrínseca asociada a la manipulación simbólica de los lazos algebraicos no lineales. Discutir en qué consiste el tearing de los lazos algebraicos.
- Discutir qué aplicación tiene la selección dinámica de las variables de estado por parte del entorno de modelado y la selección estática por parte del desarrollador del modelo. Seleccionar las variables de estado del modelo en Modelica. Dada una selección de las variables de estado de un modelo, reformular el modelo de manera que sean esas las variables que aparezcan derivadas en el modelo.
- Discutir conceptos básicos de los métodos de integración de ecuaciones diferenciales, incluyendo la estimación del error y la selección del tamaño del paso. Aplicar métodos de integración, tales como el método de Euler explícito y el método Runge-Kutta-Fehlberg. Discutir qué es un sistema stiff (rígido) y qué métodos de integración son adecuados para este tipo de sistemas.
- Discutir los fundamentos de los métodos para la resolución de sistemas de ecuaciones algebraico diferenciales (DAE), incluyendo los métodos de integración inline y mixed-mode.
- Discutir qué dificultades y errores pueden surgir durante la traducción y simulación del modelo, qué técnicas pueden aplicarse para su diagnosis y cómo puede abordarse la solución de dichos problemas.
- Plantear el algoritmo de la simulación de sistemas DAE sencillos.

3.1. INTRODUCCIÓN

En el Tema 2 se mostró el uso del lenguaje Modelica para la descripción de modelos y librerías de modelos de tiempo continuo. Sin embargo, para poder emplear eficientemente un lenguaje de modelado, no basta con conocer las bases conceptuales del modelado de sistemas dinámicos. Además, tiene gran importancia comprender el algoritmo de la simulación, a fin de plantear el modelo de manera que se evite que surjan errores en tiempo de simulación. Asimismo, deben conocerse los análisis y las manipulaciones que el entorno de modelado realiza de manera automática sobre el modelo, a fin de entender la información proporcionada por aquel durante la traducción y simulación del modelo. Algunas de estas manipulaciones requieren la toma de decisiones, no sólo fundamentadas en criterios numéricos, sino también en la lógica del modelado, con lo cual suele contemplarse la eventual intervención del usuario.

En este tema se explicarán algunos de los análisis y manipulaciones simbólicas que debe realizar el entorno de modelado para la simulación de los modelos orientados a objetos. También se introducirán algunos conceptos básicos de los métodos numéricos empleados en la simulación.

En la Figura 3.1 se muestran las etapas de que típicamente consta la conversión de un modelo en Modelica al código ejecutable de su simulación. En la **etapa de traducción** se deshace la herencia y la composición, transformándose el modelo descrito en Modelica en un modelo plano, que está compuesto por declaraciones de variables, y por ecuaciones y algoritmos. En el modelo plano se emplea la notación punto para nombrar las variables, acorde con la estructura jerárquica del modelo en Modelica.

En la fase de traducción, las *sentencias connect* del modelo en Modelica se sustituyen por ecuaciones de igualdad entre variables (variables across) y ecuaciones de suma igualada a cero (variables through). Las ecuaciones que establecen la igualdad entre dos variables se denominan **ecuaciones triviales** y típicamente son eliminadas del modelo en la **etapa de análisis y optimización**.

Por ejemplo, en la Figura 2.14 se muestra la conexión de cuatro conectores. Las tres ecuaciones de igualdad entre las cuatro variables across

$$u_1 = u_2 = u_3 = u_4 \quad (3.1)$$

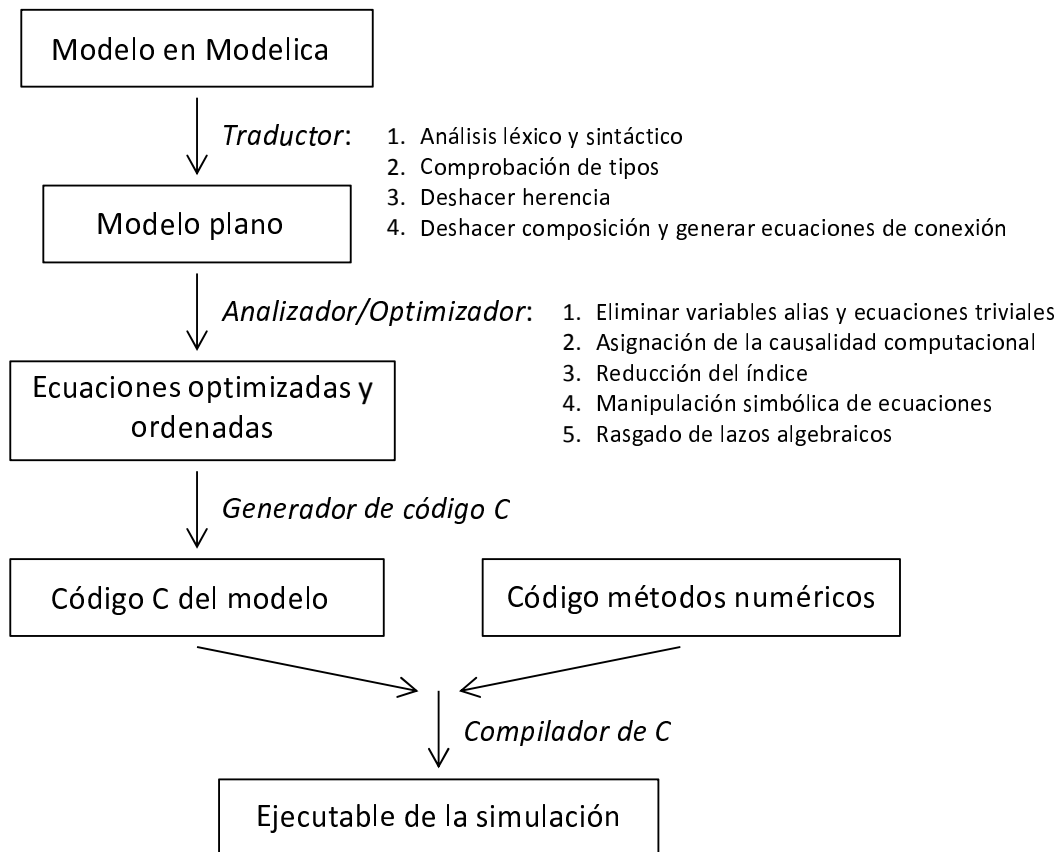


Figura 3.1: Análisis y manipulaciones realizadas automáticamente por el entorno de modelado sobre el modelo descrito en Modelica para generar el ejecutable de su simulación.

pueden ser eliminadas del modelo, sustituyéndose además las variables u_2 , u_3 y u_4 por u_1 en todas las ecuaciones en las que intervengan. Se dice en este caso que u_2 , u_3 y u_4 son **variables alias** de u_1 .

En la fase de análisis y optimización se realizan otras operaciones sobre el modelo, como son el análisis del índice y, si procede, su reducción; la asignación de la causalidad computacional; la manipulación simbólica de las ecuaciones lineales sencillas, a fin de despejar la variable a evaluar de la ecuación; la ordenación de las ecuaciones y algoritmos; y el rasgado (también denominado *tearing*) de los lazos algebraicos. En las siguientes secciones se explica de manera informal, sin entrar en los detalles algorítmicos, en qué consiste cada una de estas operaciones.

3.2. ASIGNACIÓN DE LA CAUSALIDAD COMPUTACIONAL

Dado un conjunto de relaciones matemáticas, la asignación de la causalidad computacional es *la decisión de qué relación debe emplearse para calcular cada incógnita*.

La causalidad computacional es una propiedad global del modelo completo, dado que la causalidad computacional de cada parte en que se ha dividido el sistema depende de las demás partes del mismo, de cómo se realice la conexión entre las partes y de las condiciones de contorno del sistema. Así pues, la causalidad computacional no puede decidirse localmente en un subsistema sin considerar el sistema completo en su conjunto.

Como se indicó en la Sección 1.4.7, existen herramientas de simulación de propósito general que imponen que el modelo contenga explícitamente información acerca de su causalidad computacional. Entre ellas están las herramientas que soportan el modelado gráfico basado en diagramas de bloques. Cuando se emplean estas herramientas, la tediosa tarea de establecer la causalidad computacional queda en manos de la persona que desarrolla el modelo.

Como se mostró en el Tema 2, el enfoque adoptado en los lenguajes de modelado orientado a objetos es diferente. Estos lenguajes permiten describir la parte de tiempo continuo del modelo mediante ecuaciones, en las cuales no hay información acerca de causalidad computacional. La tarea de asignar la causalidad computacional no está a cargo del desarrollador del modelo, sino del entorno de modelado, que dispone de los algoritmos necesarios para tal fin. En esta sección se describen estos algoritmos.

3.2.1. Clasificación de las variables

Como paso previo a la asignación de la causalidad computacional, se realizan las dos siguientes acciones:

1. *Sustituir las derivadas respecto al tiempo por variables mudas.* En este texto aplicaremos el criterio de llamar a la variable muda con el mismo nombre que la variable que aparece derivada, anteponiendo el prefijo “der”. Por ejemplo, $\frac{dx}{dt}$ se sustituiría por *derx*.
2. *Clasificar las variables en conocidas y desconocidas.* En este contexto, las variables desconocidas son aquellas que deben calcularse evaluando las ecua-

ciones del modelo. Por ello, el objetivo de la asignación de la causalidad computacional es decidir de qué ecuación se evalúa cada una de las variables desconocidas y en qué orden debe realizarse dicha evaluación.

Se consideran conocidas:

- La variable *tiempo*.
- Las *constantes y parámetros* del modelo, ya que sus valores no cambian durante el curso de la simulación y son conocidos al comienzo de ésta.
- Las *variables de estado*. Estas variables no son calculadas de las ecuaciones del modelo, sino integrando numéricamente sus derivadas.

Se consideran desconocidas:

- Las *variables mudas*, introducidas de la forma descrita anteriormente en sustitución de las derivadas respecto al tiempo.
- Las *restantes variables* del modelo. Es decir, aquellas variables que, no siendo variables de estado, dependen para su cálculo en el instante de evaluación del valor de otras variables.

En primera aproximación, supongamos (más adelante veremos que hay sistemas DAE en los cuales esta suposición no es correcta) que todas las variables que aparecen en el modelo derivadas respecto al tiempo pueden ser seleccionadas como variables de estado.

Atendiendo a esta clasificación de las variables del modelo en conocidas y desconocidas, puede representarse el conjunto de las ecuaciones del modelo en la forma

$$\mathbf{F}(\mathbf{x}, \mathbf{y}) = 0 \quad (3.2)$$

donde el vector $\mathbf{x}$ representa las variables conocidas y el vector $\mathbf{y}$ las variables desconocidas.

Sobre la base de dicha representación del modelo, se define la **matriz de incidencia** del modelo, también denominada matriz Jacobiana estructural, que es una matriz de elementos booleanos, que indica qué incógnitas aparecen en cada ecuación. Se define a partir de la matriz Jacobiana de la manera descrita a continuación.

El elemento (i, j) de la matriz de incidencia es cero si el mismo elemento de la matriz Jacobiana es cero, $\frac{\partial F_i}{\partial y_j} = 0$. Es decir, si en la ecuación i -ésima

del modelo no está presente la j -ésima variable desconocida. En caso contrario, el elemento de la matriz de incidencia vale 1 y se representa mediante un aspa.

En cada ecuación del modelo sólo están presentes unas pocas variables desconocidas, con lo cual la matriz Jacobiana de las ecuaciones respecto de las variables desconocidas tiene generalmente muchos elementos nulos. En general, las matrices que tienen gran parte de sus elementos nulos se denominan **matrices ralas** o **sparse**. Para almacenar este tipo de matrices se emplean estructuras de datos diferentes a las comúnmente empleadas para las **matrices densas**, que son aquellas con pocos elementos nulos. Por ejemplo, es más eficiente almacenar una matriz sparse en un grafo bipartito, que sólo contenga información sobre los elementos no nulos, que en un array bidimensional con información de todos los elementos. También se han desarrollado métodos específicos para operar y manipular eficientemente matrices sparse.

Para ilustrar la explicación anterior acerca de la sustitución de las derivadas por variables mudas, la clasificación de las variables y la construcción de la matriz de incidencia, supongamos un modelo compuesto por tres ecuaciones:

- En la primera ecuación intervienen tres variables: x , y y z .
- En la segunda ecuación intervienen las variables x e y , y la derivada respecto al tiempo de x .
- En la tercera ecuación intervienen las variables x y z .

Obsérvese en que no estamos interesados en la forma en concreto que tienen las ecuaciones del modelo. Únicamente estamos interesados en qué variables intervienen en cada ecuación. Por ello, podemos representar el modelo, indicando únicamente su estructura computacional, de la forma siguiente:

$$f_1(x, y, z) = 0 \quad (3.3)$$

$$f_2(x, \dot{x}, y) = 0 \quad (3.4)$$

$$f_3(x, z) = 0 \quad (3.5)$$

En este modelo hay una variable que aparece derivada respecto al tiempo: la variable x . Se introduce por tanto una variable muda, que llamaremos $derx$, para representar su derivada y sustituiremos en el modelo $\dot{x}$ por $derx$.

$$f_1(x, y, z) = 0 \quad (3.6)$$

$$f_2(x, derx, y) = 0 \quad (3.7)$$

$$f_3(x, z) = 0 \quad (3.8)$$

En la matriz de incidencia original sólo intervienen las variables desconocidas, que en este caso son $derx$, y y z .

$$\begin{matrix} & derx & y & z \\ \begin{matrix} f_1 \\ f_2 \\ f_3 \end{matrix} & \begin{pmatrix} 0 & X & X \\ X & X & 0 \\ 0 & 0 & X \end{pmatrix} \end{matrix} \quad (3.9)$$

La matriz de incidencia representando esta información estructural y que todavía no ha sufrido ninguna manipulación recibe el nombre de **matriz de incidencia original**.

3.2.2. Singularidad estructural

El siguiente paso consiste en comprobar que el modelo no sea estructuralmente singular. Para ello, se comprueba que:

1. El número de ecuaciones y de variables desconocidas (obtenido siguiendo el criterio anterior de clasificación de las variables en conocidas y desconocidas) es el mismo.
2. Cada variable desconocida puede emparejarse con una ecuación en que aparezca y con la cual no se haya emparejado ya otra variable desconocida.

Para ello, se buscan permutaciones de las variables (columnas) de la matriz de incidencia de modo que la matriz resultante, con las variables permutadas de orden, tenga todos sus elementos diagonales distintos de cero. Si no se consigue, se dice que es un **sistema DAE singular**.

Por ejemplo, la matriz de incidencia mostrada en (3.9) puede escribirse, intercambiando el orden de sus dos primeras columnas, de la forma mostrada a continuación:

$$\begin{array}{c} \text{\textit{derx}} \quad \text{\textit{y}} \quad \text{\textit{z}} \\ f_1 \left(\begin{array}{ccc} 0 & \text{X} & \text{X} \\ \text{X} & \text{X} & 0 \\ 0 & 0 & \text{X} \end{array} \right) \end{array} \rightarrow \begin{array}{c} \text{\textit{y}} \quad \text{\textit{derx}} \quad \text{\textit{z}} \\ f_1 \left(\begin{array}{ccc} \text{X} & 0 & \text{X} \\ \text{X} & \text{X} & 0 \\ 0 & 0 & \text{X} \end{array} \right) \end{array} \quad (3.10)$$

Puesto que se ha conseguido escribir la matriz con todos los elementos diagonales diferentes de cero, este modelo es un **sistema DAE no singular**. A continuación se muestra un ejemplo de sistema DAE singular.

$$\begin{array}{ccc} f_1(x, z) = 0 & f_1(x, z) = 0 & f_1 \left(\begin{array}{ccc} \text{\textit{derx}} & \text{\textit{y}} & \text{\textit{z}} \\ 0 & 0 & \text{X} \\ \text{X} & \text{X} & 0 \\ 0 & 0 & 0 \end{array} \right) \\ f_2(x, \dot{x}, y) = 0 & \rightarrow f_2(x, \text{\textit{derx}}, y) = 0 & \rightarrow \\ f_3(x) = 0 & f_3(x) = 0 & \end{array} \quad (3.11)$$

Obsérvese que sólo se dispone de una ecuación, la denominada f_2 , para calcular las variables $\text{\textit{derx}}$ e y , y que la ecuación denominada f_3 es redundante, ya que x se ha asumido que es conocida: no se calcula de las ecuaciones del modelo, sino integrando numéricamente $\text{\textit{derx}}$.

En general, si un modelo es estructuralmente singular es debido a uno de los dos motivos siguientes:

1. El *número de ecuaciones es igual al número de variables desconocidas*, pero al asociar ecuaciones con variables desconocidas se llega a un punto en que no se dispone de ecuación para calcular una variable desconocida y en el cual una ecuación es redundante, es decir, o bien no tiene ninguna variable desconocida, o bien todavía no ha sido asociada a ninguna variable desconocida y todas sus variables desconocidas ya han sido asociadas con otras ecuaciones. Obsérvese que éste es precisamente el caso del modelo representado por la matriz de incidencia (3.11). Esta situación puede deberse a que:
 - a) El modelo sea incorrecto matemáticamente.
 - b) El modelo sea correcto, pero que no todas las variables que aparecen derivadas puedan ser seleccionadas como variables de estado. Esto ocurre cuando el número de grados de libertad del modelo es menor que el número de variables que aparecen en él derivadas respecto al tiempo. Este caso se abordará detenidamente más adelante.

2. Si *el número de ecuaciones difiere del número de variables desconocidas*, pueden darse dos situaciones. Si el número de ecuaciones es mayor que el de variables desconocidas, se dice que el sistema está **sobredeterminado**. En caso contrario, se dice que está **infradeterminado**. En la Sección 3.2.4 se mostrará un procedimiento de diagnóstico de modelos infra y sobredeterminados.

Prosigamos por ahora la explicación sobre la asignación de la causalidad computacional suponiendo que el modelo es un sistema DAE no singular.

3.2.3. Algoritmo de partición

La partición consiste en encontrar las permutaciones de las filas (ecuaciones) y las permutaciones de las columnas (variables) de la matriz de incidencia que hagan que ésta quede convertida en una matriz triangular inferior por bloques (abreviadamente BLT, del inglés Block Lower Triangular). Esto es, en una matriz con elementos distintos de cero formando bloques cuadrados, del menor tamaño posible, en la diagonal; y con todos los elementos situados por encima de la diagonal y no pertenecientes a los bloques diagonales, iguales a cero.

La forma BLT significa que las ecuaciones se han ordenado de tal manera que las variables desconocidas de las ecuaciones de cada bloque son, o bien calculadas en bloques anteriores, o bien resolubles en el bloque. Se denomina a esta matriz BLT la **matriz de incidencia ordenada**.

Existen diferentes algoritmos para transformar una matriz de incidencia a su forma BLT. Por ejemplo, un algoritmo comúnmente empleado es el **algoritmo de Tarjan**, en el cual la matriz de incidencia es representada mediante un grafo bipartito: un conjunto de nodos representa las variables desconocidas y otro conjunto las ecuaciones. Una arista entre un nodo variable y un nodo ecuación indica que la variable interviene en la ecuación.

Un procedimiento que se emplea para realizar la partición manualmente en casos sencillos consiste en aplicar repetidamente las dos reglas siguientes.

Regla 1: si una ecuación contiene una única variable desconocida aún no evaluada, entonces debe emplearse esa ecuación para evaluar dicha variable.

Regla 2: si una variable desconocida que aún no ha sido evaluada aparece en una única ecuación del modelo, entonces la variable debe ser evaluada de esa ecuación.

Aplicando repetidamente estas reglas a sistemas no singulares, pueden darse dos situaciones:

1. *Se obtiene una solución que permite calcular todas las incógnitas usando para ello todas las ecuaciones.* Corresponde al caso en que todos los bloques diagonales de la matriz de incidencia ordenada BLT son escalares, lo cual significa que las variables pueden ser resueltas, una tras otra, en secuencia. En este caso, el algoritmo proporciona una ordenación de las ecuaciones tal que en cada ecuación hay una y sólo una incógnita que no haya sido previamente calculada. La forma en que se realice este cálculo (mediante manipulación simbólica o numéricamente) no es de la incumbencia del algoritmo de partición.
2. *Se llega a un punto en que todas las ecuaciones tienen al menos dos incógnitas y todas las incógnitas aparecen al menos en dos ecuaciones.* Corresponde al caso en que hay bloques diagonales no escalares en la matriz de incidencia ordenada BLT. Cada bloque diagonal no escalar representa un sistema de ecuaciones que deberá resolverse numérica o, en caso de que sea posible, simbólicamente.

Con el fin de ilustrar cómo se asigna la causalidad computacional aplicando repetidamente las dos reglas anteriores, consideremos nuevamente el modelo cuya matriz de incidencia es (3.9). Por comodidad para el lector se muestra nuevamente el modelo y su matriz de incidencia original.

$$\begin{array}{lll}
 f_1(x, y, z) = 0 & f_1(x, y, z) = 0 & \\
 f_2(x, \dot{x}, y) = 0 & \rightarrow f_2(x, \text{der}x, y) = 0 & \rightarrow \\
 f_3(x, z) = 0 & f_3(x, z) = 0 &
 \end{array}
 \rightarrow
 \begin{array}{c}
 f_1 \left(\begin{array}{ccc} \text{der}x & y & z \\ 0 & X & X \\ X & X & 0 \\ 0 & 0 & X \end{array} \right)
 \end{array}
 \quad (3.12)$$

1. La ecuación f_3 tiene una única variable desconocida: la variable z . Por tanto, deberá usarse la ecuación f_3 para evaluar z . Como z interviene en otras ecuaciones además de en f_3 , es necesario conocer su valor para poder calcular otras variables desconocidas. Por tanto, se pasa la ecuación f_3 a la primera fila de la matriz y la variable z a la primera columna.

La variable $\text{der}x$ interviene en una única ecuación: la ecuación f_2 . Deberá usarse esa ecuación para evaluarla. Como $\text{der}x$ no interviene en ninguna otra ecuación, no es necesario conocer su valor para evaluar otras variables desconocidas. Por tanto, se pasa la ecuación f_2 a la última fila y la variable $\text{der}x$ a la última columna.

$$\begin{array}{c}
f_3 \\
f_1 \\
f_2
\end{array}
\begin{pmatrix}
z & y & derx \\
\boxed{X} & 0 & 0 \\
X & X & 0 \\
0 & X & \boxed{X}
\end{pmatrix}
\quad (3.13)$$

2. Puesto que z va a ser evaluada de f_3 , la ecuación f_1 tiene una única variable desconocida aún no evaluada: la variable y . La variable y debe evaluarse de f_1 .

$$\begin{array}{c}
f_3 \\
f_1 \\
f_2
\end{array}
\begin{pmatrix}
z & y & derx \\
\boxed{X} & 0 & 0 \\
X & \boxed{X} & 0 \\
0 & X & \boxed{X}
\end{pmatrix}
\quad (3.14)$$

A continuación se muestra el modelo ordenado y con la causalidad computacional señalada. Obsérvese que se señala la variable a evaluar de cada ecuación escribiéndola entre corchetes.

$$f_3(x, [z]) = 0 \quad (3.15)$$

$$f_1(x, [y], z) = 0 \quad (3.16)$$

$$f_2(x, [derx], y) = 0 \quad (3.17)$$

3.2.4. Sistemas sobredeterminados e infradeterminados

Si el sistema está **infradeterminado**, es conveniente distinguir entre aquellas variables que pueden resolverse y aquellas que no. Para ello, se añade al conjunto de ecuaciones del sistema tantas ecuaciones ficticias, conteniendo cada una todas las variables del sistema, como exceso de variables. Esto equivale a añadir tantas filas de unos a la matriz de incidencia como sean necesarias para hacerla cuadrada.

A continuación se realiza la partición de todas las ecuaciones, las originales y las añadidas. Como todas las ecuaciones añadidas dependen de todas las variables, al realizar la partición todas estas ecuaciones ficticias aparecerán formando parte del último bloque de la matriz de incidencia, el cual tendrá la dimensión menor posible. Los restantes bloques definen el conjunto máximo de variables calculables y la secuencia de cálculo que produce sistemas de ecuaciones de menor dimensión.

A continuación se muestra un ejemplo. Consideremos un modelo compuesto por dos ecuaciones. En la primera intervienen las variables x , y , z y la derivada de x . En la segunda intervienen las variables x e y . El modelo tiene tres variables desconocidas ($derx$, y , z) y únicamente dos ecuaciones. Se añade por tanto a la matriz de incidencia una ecuación ficticia en la cual intervengan todas las variables desconocidas.

$$\begin{array}{l} f_1(x, \dot{x}, y, z) = 0 \\ f_2(x, y) = 0 \end{array} \quad \rightarrow \quad \begin{array}{l} f_1 \\ f_2 \\ \text{ec. ficticia} \end{array} \begin{array}{c} \begin{array}{ccc} derx & y & z \end{array} \\ \left(\begin{array}{ccc} X & X & X \\ 0 & X & 0 \\ X & X & X \end{array} \right) \end{array} \quad (3.18)$$

Obsérvese que la ecuación f_2 sólo tiene una variable desconocida: la variable y . Se pasa por tanto f_2 a la primera fila y la variable y a la primera columna. La matriz de incidencia BLT es (3.19). Las variables $derx$ y z se calculan en el último bloque de la diagonal, por tanto en el modelo falta una ecuación para calcular una de estas dos variables desconocidas.

$$\begin{array}{l} f_2 \\ f_1 \\ \text{ec. ficticia} \end{array} \begin{array}{c} \begin{array}{ccc} y & derx & z \end{array} \\ \left(\begin{array}{ccc} \boxed{X} & 0 & 0 \\ X & \boxed{X} & \boxed{X} \\ X & \boxed{X} & \boxed{X} \end{array} \right) \end{array} \quad (3.19)$$

El problema de la **sobredeterminación**, en que aparecen ecuaciones redundantes, es el dual del anterior: se añaden tantas variables ficticias a todas las ecuaciones como exceso de ecuaciones hay. Esto es equivalente a añadir tantas columnas de unos a la matriz de incidencia como sean necesarias para hacerla cuadrada.

Realizando la partición, se obtendrá que el primer bloque de la matriz de incidencia es el sistema de ecuaciones del cual se calculan, además de algunas variables desconocidas del modelo, todas las variables ficticias.

Todas las variables ficticias se calculan en el primer bloque, ya que por aparecer en todas las ecuaciones su conocimiento es necesario para poder resolver las demás variables. Las ecuaciones del primer bloque son necesarias para calcular las variables ficticias y las variables del modelo que aparecen en las ecuaciones redundantes, con lo cual las ecuaciones redundantes (tantas como variables ficticias) se encuentran entre las del primer bloque.

A continuación se muestra un ejemplo. Se trata de un modelo compuesto por cuatro ecuaciones en las que intervienen tres variables desconocidas. Las ecuaciones

y la matriz de incidencia original se muestra en (3.20). Obsérvese que se ha añadido una variable ficticia llamada α en todas las ecuaciones.

$$\begin{array}{l}
 f_1(y) = 0 \\
 f_2(y, z) = 0 \\
 f_3(x, y, z) = 0 \\
 f_4(\dot{x}, x, z) = 0
 \end{array}
 \rightarrow
 \begin{array}{l}
 f_1 \begin{pmatrix} 0 & X & 0 & X \end{pmatrix} \\
 f_2 \begin{pmatrix} 0 & X & X & X \end{pmatrix} \\
 f_3 \begin{pmatrix} 0 & X & X & X \end{pmatrix} \\
 f_4 \begin{pmatrix} X & 0 & X & X \end{pmatrix}
 \end{array}
 \quad (3.20)$$

La variable $derx$ sólo interviene en la ecuación f_4 , por tanto debe evaluarse de ella. Las otras tres variables desconocidas se evalúan del primer bloque de la diagonal. Por tanto, una de las tres ecuaciones f_1 , f_2 y f_3 es redundante.

$$\begin{array}{l}
 f_1 \begin{pmatrix} X & 0 & X & 0 \end{pmatrix} \\
 f_2 \begin{pmatrix} X & X & X & 0 \end{pmatrix} \\
 f_3 \begin{pmatrix} X & X & X & 0 \end{pmatrix} \\
 f_4 \begin{pmatrix} 0 & X & X & \boxed{X} \end{pmatrix}
 \end{array}
 \quad (3.21)$$

3.2.5. Ejemplo: simulación de un circuito eléctrico

A continuación se muestra un ejemplo que permite ilustrar el procedimiento para la asignación de la causalidad computacional. En la Figura 3.2 se muestra el diagrama de un circuito eléctrico compuesto por un generador sinusoidal de voltaje, dos resistencias y dos condensadores. Se ha asignado nombre a las corrientes y a los voltajes en los nodos. Se supone que las resistencias (R_1 , R_2), las capacidades (C_1 , C_2), y la amplitud (U) y frecuencia (w) del generador son parámetros del modelo, es decir, tienen valores constantes conocidos.

El modelo del circuito consta de las seis ecuaciones mostradas a continuación, donde la variable t representa el tiempo. Las Ecs. (3.22) – (3.26) son las relaciones constitutivas de los componentes. La Ec. (3.27) impone la conservación de la corriente en el nodo u_1 .

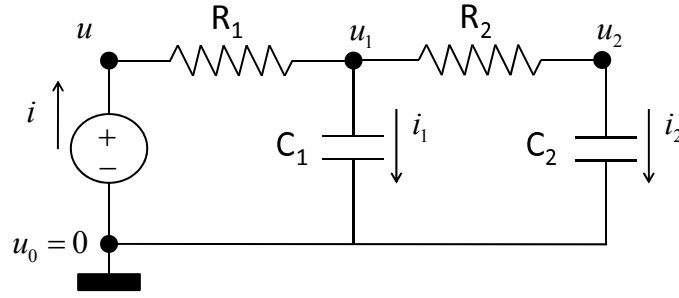


Figura 3.2: Diagrama del circuito eléctrico.

$$u = U \cdot \sin(w \cdot t) \quad (3.22)$$

$$u - u_1 = i \cdot R_1 \quad (3.23)$$

$$C_1 \cdot \frac{du_1}{dt} = i_1 \quad (3.24)$$

$$u_1 - u_2 = i_2 \cdot R_2 \quad (3.25)$$

$$C_2 \cdot \frac{du_2}{dt} = i_2 \quad (3.26)$$

$$i = i_1 + i_2 \quad (3.27)$$

Obsérvese que al nombrar las corrientes ya se ha impuesto que la corriente que sale del generador sea la misma que circula por R_1 , y que la corriente que circula por R_2 sea la misma que la corriente que circula por C_2 . Con ello se ha impuesto implícitamente la conservación de la corriente en los nodos u y u_2 . Por ello, en el modelo únicamente se plantea la ecuación de conservación de la corriente en el nodo u_1 .

Para asignar la causalidad computacional, se sustituyen las derivadas respecto al tiempo por variables mudas. Seguiremos el convenio de construir el nombre de la variable muda anteponiendo “der” al nombre de la variable que está siendo derivada.

$$\frac{du_1}{dt} \rightarrow \text{der}u_1 \quad (3.28)$$

$$\frac{du_2}{dt} \rightarrow \text{der}u_2 \quad (3.29)$$

Realizando las sustituciones, se obtiene el modelo siguiente:

$$u = U \cdot \sin(w \cdot t) \quad (3.30)$$

$$u - u_1 = i \cdot R_1 \quad (3.31)$$

$$C_1 \cdot deru_1 = i_1 \quad (3.32)$$

$$u_1 - u_2 = i_2 \cdot R_2 \quad (3.33)$$

$$C_2 \cdot deru_2 = i_2 \quad (3.34)$$

$$i = i_1 + i_2 \quad (3.35)$$

A efectos de la asignación de la causalidad computacional, las variables del modelo se clasifican en conocidas (el tiempo, los parámetros y las variables de estado) y desconocidas (las demás variables del modelo y las variables mudas introducidas sustituyendo a las derivadas). Supongamos que las dos variables que aparecen derivadas son variables de estado. De esta forma, las variables se clasifican de la manera siguiente:

- Conocidas: t
 U, w, R_1, C_1, R_2, C_2
 u_1, u_2
- Desconocidas: i, i_1, i_2, u
 $deru_1, deru_2$

Se muestra a continuación la matriz de incidencia original del modelo, construida de las Ecs. (3.30) – (3.35) y de la clasificación anterior de las variables.

$$\begin{array}{l}
 \begin{array}{l}
 u=U \cdot \sin(w \cdot t) \\
 u-u_1=i \cdot R_1 \\
 C_1 \cdot deru_1=i_1 \\
 u_1-u_2=i_2 \cdot R_2 \\
 C_2 \cdot deru_2=i_2 \\
 i=i_1+i_2
 \end{array}
 \begin{pmatrix}
 deru_1 & deru_2 & i & i_1 & i_2 & u \\
 0 & 0 & 0 & 0 & 0 & X \\
 0 & 0 & X & 0 & 0 & X \\
 X & 0 & 0 & X & 0 & 0 \\
 0 & 0 & 0 & 0 & X & 0 \\
 0 & X & 0 & 0 & X & 0 \\
 0 & 0 & X & X & X & 0
 \end{pmatrix}
 \end{array} \quad (3.36)$$

Con el fin de analizar si el modelo es estructuralmente singular, se comprueba que:

1. El número de ecuaciones y de variables desconocidas (incógnitas) es el mismo. Este modelo se compone de 6 ecuaciones y tiene 6 variables desconocidas ($deru_1, deru_2, i, i_1, i_2, u$).

2. Cada incógnita puede emparejarse con una ecuación en que aparezca y con la cual no se haya emparejado ya otra incógnita. Es equivalente a comprobar si es posible, reordenando las columnas de la matriz de incidencia, conseguir que todos sus elementos diagonales sean diferentes de cero. En este caso, tal como se muestra a continuación, sí es posible.

$$\begin{array}{l}
 u=U \cdot \sin(w \cdot t) \\
 u-u_1=i \cdot R_1 \\
 C_1 \cdot deru_1=i_1 \\
 u_1-u_2=i_2 \cdot R_2 \\
 C_2 \cdot deru_2=i_2 \\
 i=i_1+i_2
 \end{array}
 \begin{pmatrix}
 u & i & deru_1 & i_2 & deru_2 & i_1 \\
 \left(\begin{array}{cccccc}
 X & 0 & 0 & 0 & 0 & 0 \\
 X & X & 0 & 0 & 0 & 0 \\
 0 & 0 & X & 0 & 0 & X \\
 0 & 0 & 0 & X & 0 & 0 \\
 0 & 0 & 0 & X & X & 0 \\
 0 & X & 0 & X & 0 & X
 \end{array} \right)
 \end{pmatrix} \quad (3.37)$$

La matriz de incidencia BLT puede obtenerse razonando de la manera siguiente:

1. La ecuación del generador sólo tiene una variable desconocida: la variable u . La ecuación de la resistencia 2 sólo tiene una variable desconocida: la variable i_2 . Por tanto, cada una de estas variables deberá calcularse de la citada ecuación. Estas variables intervienen en otras ecuaciones, con lo cual pasamos las dos ecuaciones a las primeras filas y las dos variables a las primeras columnas.

La variable $deru_1$ interviene en una única ecuación: la del condensador 1. La variable $deru_2$ interviene en una única ecuación: la del condensador 2. Estas variables deben evaluarse de la única ecuación en la que intervienen. Como el valor de estas variables no se emplea en el cálculo de otras variables desconocidas, pasamos las dos ecuaciones a las últimas filas de la matriz y las dos variables a las últimas columnas.

$$\begin{array}{l}
 u=U \cdot \sin(w \cdot t) \\
 u_1-u_2=i_2 \cdot R_2 \\
 i=i_1+i_2 \\
 u-u_1=i \cdot R_1 \\
 C_1 \cdot deru_1=i_1 \\
 C_2 \cdot deru_2=i_2
 \end{array}
 \begin{pmatrix}
 u & i_2 & i & i_1 & deru_1 & deru_2 \\
 \left(\begin{array}{cccccc}
 \boxed{X} & 0 & 0 & 0 & 0 & 0 \\
 0 & \boxed{X} & 0 & 0 & 0 & 0 \\
 0 & X & X & X & 0 & 0 \\
 X & 0 & X & 0 & 0 & 0 \\
 0 & 0 & 0 & X & \boxed{X} & 0 \\
 0 & X & 0 & 0 & 0 & \boxed{X}
 \end{array} \right)
 \end{pmatrix} \quad (3.38)$$

2. Asumiendo que las variables u e i_2 ya han sido evaluadas, la ecuación de la resistencia 1 sólo tiene una variable desconocida no evaluada: la variable i .

Ubicando esta ecuación a tercera fila, y la variable i en la tercera columna, se obtiene la matriz mostrada a continuación.

$$\begin{array}{l}
 u=U \cdot \sin(w \cdot t) \\
 u_1 - u_2 = i_2 \cdot R_2 \\
 u - u_1 = i \cdot R_1 \\
 i = i_1 + i_2 \\
 C_1 \cdot \text{der}u_1 = i_1 \\
 C_2 \cdot \text{der}u_2 = i_2
 \end{array}
 \begin{pmatrix}
 u & i_2 & i & i_1 & \text{der}u_1 & \text{der}u_2 \\
 \boxed{X} & 0 & 0 & 0 & 0 & 0 \\
 0 & \boxed{X} & 0 & 0 & 0 & 0 \\
 X & 0 & \boxed{X} & 0 & 0 & 0 \\
 0 & X & X & X & 0 & 0 \\
 0 & 0 & 0 & X & \boxed{X} & 0 \\
 0 & X & 0 & 0 & 0 & \boxed{X}
 \end{pmatrix} \quad (3.39)$$

3. Asumiendo que las variables u , i_2 e i ya han sido evaluadas, la ecuación del nodo u_1 tiene una única variable desconocida sin evaluar. Dicha variable debe por tanto evaluarse de la ecuación del nodo. La matriz de incidencia ha quedado en forma BLT.

$$\begin{array}{l}
 u=U \cdot \sin(w \cdot t) \\
 u_1 - u_2 = i_2 \cdot R_2 \\
 u - u_1 = i \cdot R_1 \\
 i = i_1 + i_2 \\
 C_1 \cdot \text{der}u_1 = i_1 \\
 C_2 \cdot \text{der}u_2 = i_2
 \end{array}
 \begin{pmatrix}
 u & i_2 & i & i_1 & \text{der}u_1 & \text{der}u_2 \\
 \boxed{X} & 0 & 0 & 0 & 0 & 0 \\
 0 & \boxed{X} & 0 & 0 & 0 & 0 \\
 X & 0 & \boxed{X} & 0 & 0 & 0 \\
 0 & X & X & \boxed{X} & 0 & 0 \\
 0 & 0 & 0 & X & \boxed{X} & 0 \\
 0 & X & 0 & 0 & 0 & \boxed{X}
 \end{pmatrix} \quad (3.40)$$

Por tanto, las ecuaciones del modelo ordenadas y con la causalidad computacional señalada son las siguientes:

$$[u] = U \cdot \sin(w \cdot t) \quad (3.41)$$

$$u_1 - u_2 = [i_2] \cdot R_2 \quad (3.42)$$

$$u - u_1 = [i] \cdot R_1 \quad (3.43)$$

$$i = [i_1] + i_2 \quad (3.44)$$

$$C_1 \cdot [\text{der}u_1] = i_1 \quad (3.45)$$

$$C_2 \cdot [\text{der}u_2] = i_2 \quad (3.46)$$

Despejando en cada ecuación la variable a evaluar, se obtiene el modelo ordenado y resuelto:

$$[u] = U \cdot \sin(w \cdot t) \quad (3.47)$$

$$[i_2] = \frac{u_1 - u_2}{R_2} \quad (3.48)$$

$$[i] = \frac{u - u_1}{R_1} \quad (3.49)$$

$$[i_1] = i - i_2 \quad (3.50)$$

$$[deru_1] = \frac{\dot{i}_1}{C_1} \quad (3.51)$$

$$[deru_2] = \frac{\dot{i}_2}{C_2} \quad (3.52)$$

Obsérvese que la asignación de la causalidad computacional (la decisión de qué variable debe evaluarse de cada ecuación) en este modelo es única. Sin embargo, existen varias formas igualmente válidas de ordenar las ecuaciones del modelo. Por ejemplo, es indiferente evaluar primero la Ec. (3.47) y a continuación la Ec. (3.48), o hacerlo a la inversa. Lo mismo sucede con las Ecs. (3.51) y (3.52).

En la Figura 3.3 se muestra un algoritmo (probablemente el más sencillo posible) para la simulación de modelos de tiempo continuo y en la Figura 3.4 la particularización al caso del modelo del circuito. El algoritmo de integración empleado es el método de Euler explícito. Puede comprobarse que la clasificación de las variables del modelo en parámetros, variables de estado y variables algebraicas constituye la base para la simulación del modelo:

- Al comenzar la simulación, se asignan valores a los *parámetros*. Estos valores permanecen constantes durante toda la simulación.
- Las *variables de estado* son calculadas mediante la integración numérica de sus derivadas. Por ejemplo, la función de paso del método explícito de Euler para la ecuación diferencial ordinaria

$$\frac{dx}{dt} = f(x, t) \quad (3.53)$$

es la siguiente:

$$x_{i+1} = x_i + f(x_i, t_i) \cdot \Delta t \quad (3.54)$$

donde x_i y x_{i+1} representan el valor de la variable de estado x en los instantes t_i y $t_i + \Delta t$ respectivamente, y $f(x_i, t_i)$ representa el valor de la derivada de x (es decir, $\frac{dx}{dt}$) en el instante t_i .

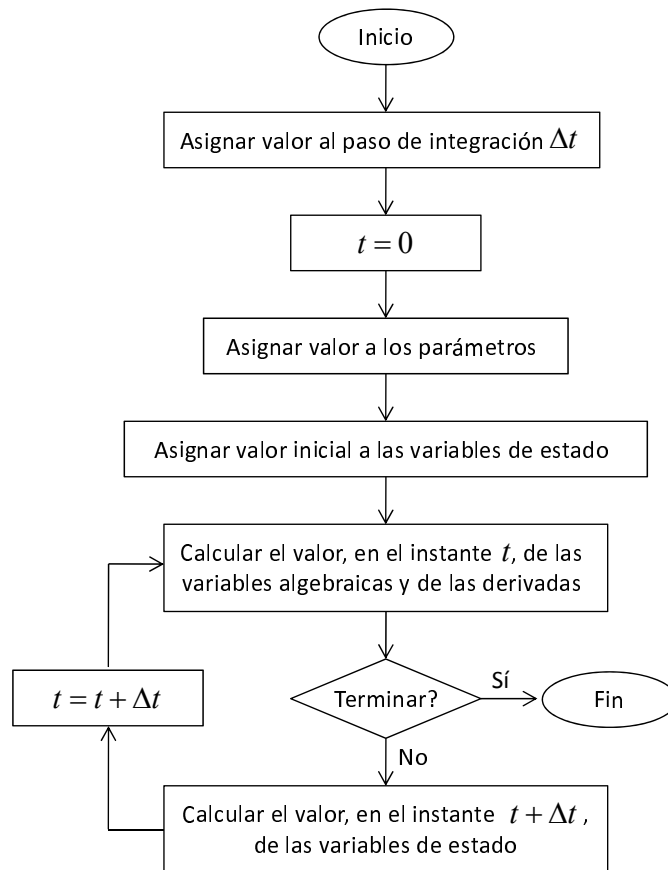


Figura 3.3: Un algoritmo para la simulación de modelos matemáticos de tiempo continuo.

- El valor de las *variables algebraicas* y las *derivadas de las variables de estado* se calcula en cada instante de tiempo, conocido el valor de los parámetros y el valor de las variables de estado en ese instante de tiempo, evaluando el modelo ordenado y resuelto resultante de aplicar el algoritmo de la partición.

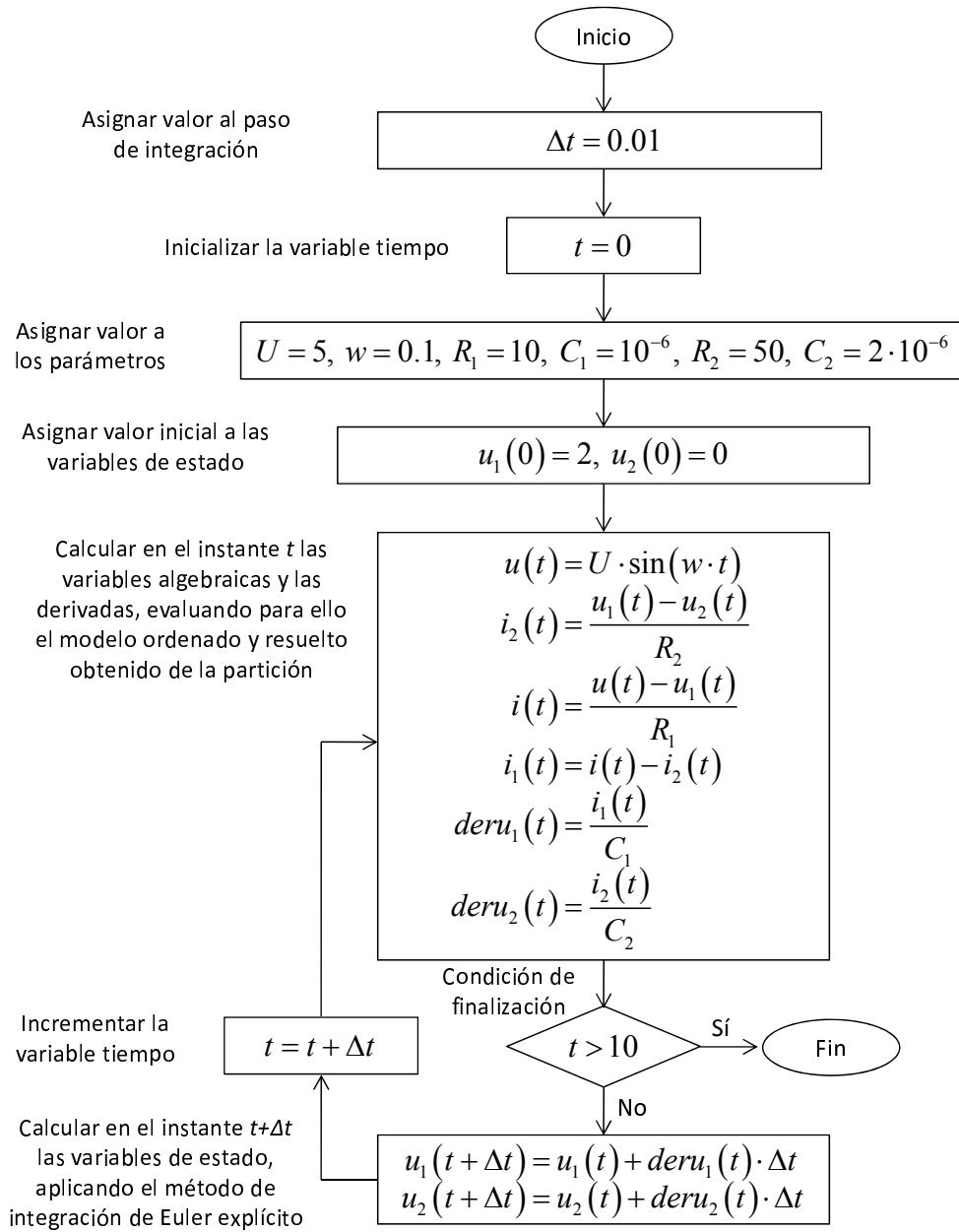


Figura 3.4: Algoritmo para la simulación del circuito de la Figura 3.2.

3.3. SISTEMAS DAE ESTRUCTURALMENTE SINGULARES

En la sección anterior se mostró un procedimiento para asignar la causalidad computacional. Tal como se explicó, la aplicación de dicho procedimiento puede conducir, dependiendo de la estructura del modelo analizado, a una de las tres situaciones siguientes:

1. El problema de la asignación de la causalidad computacional tiene una única solución. Esto es equivalente a que la matriz de incidencia BLT tenga todos sus bloques diagonales escalares, es decir, sean bloques de dimensión 1×1 .
2. El problema no puede ser resuelto de manera única, es decir, la matriz de incidencia ordenada BLT tiene bloques no escalares en la diagonal.
3. El problema no puede ser resuelto.

En este último caso, puede suceder que el problema esté infradeterminado o sobredeterminado, con lo cual el modelo está mal planteado y debe corregirse. Es también posible que el modelo matemático sea incorrecto y deba corregirse. Sin embargo, también puede suceder que el modelo esté bien planteado, pero que el número de grados de libertad del modelo sea menor que el número de variables que aparecen derivadas en el modelo. En este caso, no puede asumirse que todas las variables derivadas son variables de estado, ya que el número de variables de estado debe ser igual al número de grados de libertad del modelo. Así pues, en este tipo de sistemas DAE no es correcto clasificar como conocidas, a efectos de la asignación de la causalidad computacional, todas las variables que aparecen derivadas.

En este contexto, el **número de grados de libertad** del modelo es el número de variables dependientes del tiempo a las cuales se puede asignar valor inicial independientemente. Por ejemplo, en el modelo cuyo algoritmo de la simulación se muestra en la Figura 3.4, se asigna valor inicial a dos variables dependientes del tiempo: las variables u_1 y u_2 . El modelo tiene por tanto dos grados de libertad. Obsérvese que, conocido el valor en el instante inicial de la simulación de esas dos variables y el valor de los parámetros, es posible calcular de las ecuaciones del modelo el valor en el instante inicial de todas las demás variables dependientes del tiempo.

Para plantear el algoritmo de la simulación de los sistemas DAE estructuralmente singulares, que tienen más variables derivadas que grados de libertad, es preciso primeramente manipular simbólicamente sus ecuaciones, a continuación seleccionar las variables de estado de manera que su número sea igual al número de grados de

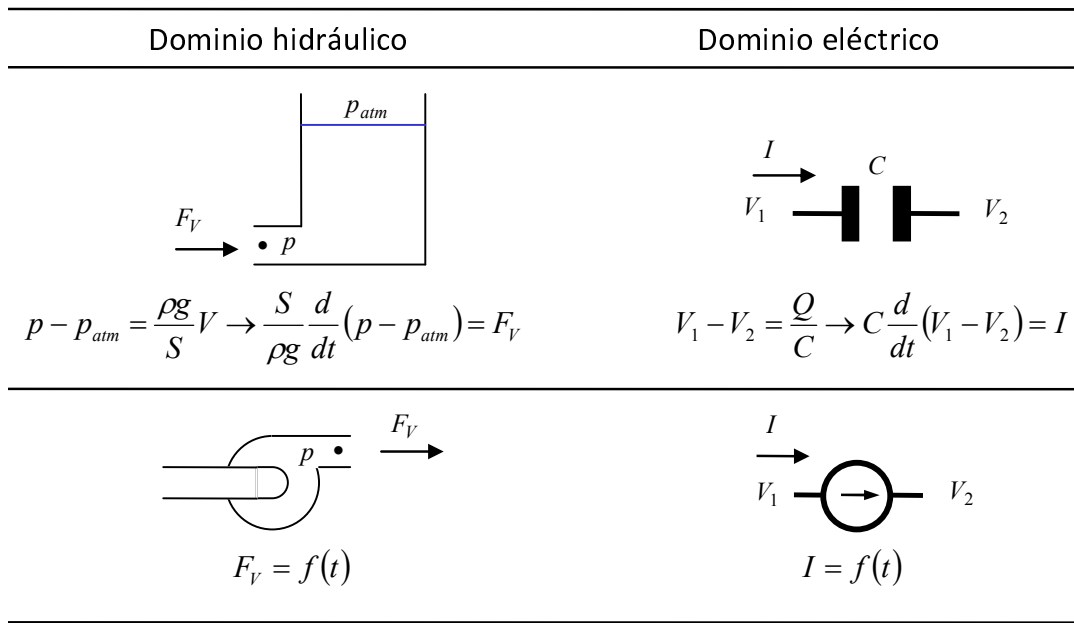


Figura 3.5: Ejemplo de analogía entre los dominios hidráulico y eléctrico. En la parte superior se muestra la ecuación constitutiva de un tanque atmosférico y de un condensador eléctrico. En la parte inferior, la ecuación constitutiva de una fuente de flujo volumétrico y de una fuente de corriente eléctrica.

libertad del sistema, y finalmente asignar la causalidad computacional. Veámoslo en el siguiente ejemplo, en el cual se proponer modelar un sistema que está formado por dos depósitos de líquido conectados en paralelo a una fuente de líquido.

Supongamos que el depósito almacena un volumen V de un líquido de densidad ρ constante. El líquido entra y sale del depósito a través de una tubería situada en su base. El modelo realizado del depósito es muy sencillo. Relaciona la variación en la presión del líquido en la base del depósito con el flujo volumétrico de líquido, F_V . La variable S representa la sección de la base del tanque, p_{atm} la presión atmosférica, p la presión en la base del tanque y g la aceleración gravitatoria.

Estableciendo una equivalencia entre el flujo volumétrico de líquido y la corriente eléctrica, y entre la presión del líquido y el voltaje eléctrico, puede definirse la capacidad del depósito, en analogía a la capacidad del condensador eléctrico, como se muestra en la Ec. (3.55) (véase la parte superior de la Figura 3.5). Asimismo, puede establecerse una analogía entre una fuente de flujo volumétrico y una fuente de corriente eléctrica (véase la parte inferior de la Figura 3.5).

$$C = \frac{S}{\rho \cdot g} \quad (3.55)$$

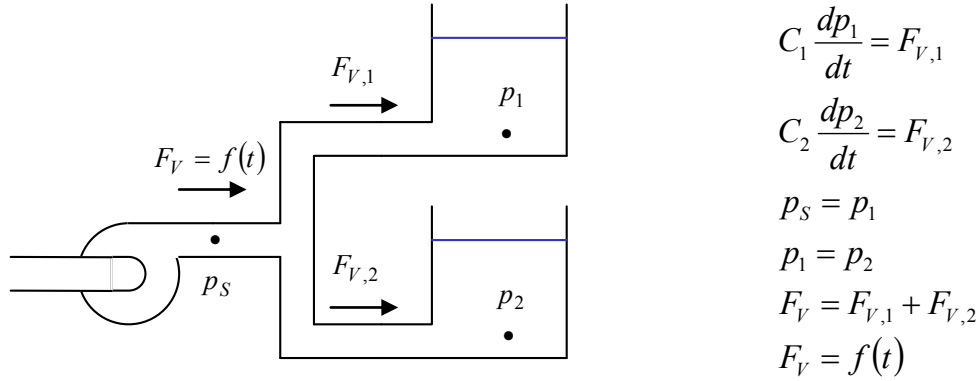


Figura 3.6: Diagrama del sistema formado por una fuente de líquido conectada a dos depósitos. En la parte derecha se muestran las ecuaciones del modelo.

El sistema completo a modelar, compuesto por la conexión de los dos depósitos y la fuente, se muestra en la Figura 3.6. Se supone que en las conexiones no se produce pérdida de carga y que la presión atmosférica es constante. Las ecuaciones del modelo se muestran en la parte derecha de la figura, donde $f(t)$ es una función conocida del tiempo.

Con el fin de asignar la causalidad computacional al modelo, se sustituyen sus derivadas por variables mudas:

$$\frac{dp_1}{dt} \rightarrow derp_1 \qquad \frac{dp_2}{dt} \rightarrow derp_2 \qquad (3.56)$$

El modelo mostrado en la Figura 3.6, con las derivadas sustituidas por las variables mudas, es:

$$C_1 \cdot derp_1 = F_{V,1} \qquad (3.57)$$

$$C_2 \cdot derp_2 = F_{V,2} \qquad (3.58)$$

$$p_s = p_1 \qquad (3.59)$$

$$p_1 = p_2 \qquad (3.60)$$

$$F_V = F_{V,1} + F_{V,2} \qquad (3.61)$$

$$F_V = f(t) \qquad (3.62)$$

Clasifiquemos las variables del modelo suponiendo que las dos variables que aparecen derivadas, es decir, p_1 y p_2 , son variables de estado.

- Conocidas: t
 C_1, C_2
 p_1, p_2
- Desconocidas: $F_{V,1}, F_{V,2}, F_V, p_S$
 $derp_1, derp_2$

La matriz de incidencia original se muestra a continuación:

$$\begin{array}{l}
 C_1 \cdot derp_1 = F_{V,1} \\
 C_2 \cdot derp_2 = F_{V,2} \\
 p_S = p_1 \\
 p_1 = p_2 \\
 F_V = F_{V,1} + F_{V,2} \\
 F_V = f(t)
 \end{array}
 \begin{pmatrix}
 & F_{V,1} & F_{V,2} & F_V & p_S & derp_1 & derp_2 \\
 \begin{pmatrix}
 X & 0 & 0 & 0 & X & 0 \\
 0 & X & 0 & 0 & 0 & X \\
 0 & 0 & 0 & X & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 \\
 X & X & X & 0 & 0 & 0 \\
 0 & 0 & X & 0 & 0 & 0
 \end{pmatrix}
 \end{pmatrix} \quad (3.63)$$

La cuarta fila de la matriz de incidencia, correspondiente a la ecuación $p_1 = p_2$, tiene todos los elementos cero, con lo cual no es posible conseguir mediante reordenación de las columnas de la matriz que todos los elementos de la diagonal sean distintos de cero. El sistema DAE es singular.

Para estudiar la estructura del sistema, aplicamos las dos reglas del algoritmo de la partición.

1. La Ec. (3.59) tiene cada una única variable desconocida: la variable p_S . La Ec. (3.62) también tiene una única variable desconocida: la variable F_V . Dichas variables deberán evaluarse de estas ecuaciones. Se pasan por tanto esas dos ecuaciones a las primeras filas de la matriz de incidencia y las dos variables a las primeras columnas.

La variable $derp_1$ sólo interviene en la Ec. (3.57) y la variable $derp_2$ sólo interviene en la Ec. (3.58). Ambas variables deben calcularse de dichas ecuaciones. Puesto que $derp_1$ y $derp_2$ no son necesarias para evaluar otras variables (cada una sólo interviene en una ecuación), las Ecs. (3.57) y (3.58) pueden ubicarse en las últimas filas de la matriz de incidencia y esas variables en las últimas columnas.

$$\begin{array}{l}
p_S=p_1 \\
F_V=f(t) \\
p_1=p_2 \\
F_V=F_{V,1}+F_{V,2} \\
C_1 \cdot \text{der}p_1=F_{V,1} \\
C_2 \cdot \text{der}p_2=F_{V,2}
\end{array}
\begin{pmatrix}
p_S & F_V & F_{V,1} & F_{V,2} & \text{der}p_1 & \text{der}p_2 \\
\boxed{X} & 0 & 0 & 0 & 0 & 0 \\
0 & \boxed{X} & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 \\
0 & X & X & X & 0 & 0 \\
0 & 0 & X & 0 & \boxed{X} & 0 \\
0 & 0 & 0 & X & 0 & \boxed{X}
\end{pmatrix} \quad (3.64)$$

2. Por tanto, quedan dos ecuaciones sin asignar causalidad, las Ecs. (3.60) y (3.61), y dos incógnitas: $F_{V,1}$, $F_{V,2}$. Sin embargo, la Ec. (3.60) no contiene ninguna incógnita, ya que se ha considerado que ambas presiones son variables de estado. Por tanto, se dispone de una única ecuación, la Ec. (3.61), para calcular $F_{V,1}$ y $F_{V,2}$.

Falta una ecuación y a la vez una ecuación es redundante, ya que relaciona variables clasificadas como conocidas. El problema es debido a que se ha supuesto que el sistema tiene dos variables de estado, es decir, dos grados de libertad, cuando en realidad sólo tiene uno. Si bien cada modelo del depósito por separado posee un grado de libertad, el sistema de los dos depósitos no posee dos grados de libertad, ya que al conectar los depósitos se impone que la presión en sus bases sea la misma, es decir, se introduce una ligadura que elimina un grado de libertad.

Escojamos, por tanto, sólo una de las presiones como variable de estado. Por ejemplo, escogemos p_1 como variable de estado (se supone por tanto conocida al hacer la partición, ya que se calculará integrando numéricamente $\text{der}p_1$) y consideramos que p_2 no es variable de estado, es decir, es una variable desconocida que debe ser evaluada de las ecuaciones del modelo. El resultado de asignar la causalidad computacional es el siguiente:

$$p_1 \quad \text{variable de estado} \quad (3.65)$$

$$[F_V] = f(t) \quad (3.66)$$

$$[p_S] = p_1 \quad (3.67)$$

$$p_1 = [p_2] \quad (3.68)$$

$$C_2 \cdot \frac{dp_2}{dt} = [F_{V,2}] \leftarrow \text{derivación numérica} \quad (3.69)$$

$$F_V = [F_{V,1}] + F_{V,2} \quad (3.70)$$

$$C_1 \cdot [\text{der}p_1] = F_{V,1} \quad (3.71)$$

Esta forma de resolver el modelo implica derivación numérica y por tanto no es un buen método. La derivación numérica debe evitarse siempre que sea posible, ya que la forma irregular de su error hace complicada la determinación del tamaño óptimo del paso en el tiempo.

Estudiemos otra posibilidad. La ecuación $p_1 = p_2$ es la ligadura que reduce el número de grados de libertad. Cabe plantearse la opción de **reemplazar** en el modelo esta ecuación que establece la igualdad entre las presiones, por su derivada respecto al tiempo:

$$p_1 = p_2 \quad \rightarrow \quad \dot{p}_1 = \dot{p}_2 \quad (3.72)$$

De esta forma, sería posible seleccionar ambas presiones como variables de estado, ya que el modelo tendría dos grados de libertad. Procediendo de esta manera, el modelo quedaría planteado de la forma siguiente:

$$C_1 \cdot \dot{p}_1 = F_{V,1} \quad (3.73)$$

$$C_2 \cdot \dot{p}_2 = F_{V,2} \quad (3.74)$$

$$p_S = p_1 \quad (3.75)$$

$$\dot{p}_1 = \dot{p}_2 \quad \leftarrow \text{derivada de la ligadura} \quad (3.76)$$

$$F_V = F_{V,1} + F_{V,2} \quad (3.77)$$

$$F_V = f(t) \quad (3.78)$$

Para que este sistema tenga la misma solución que el sistema original, deben asignarse valores iniciales a las variables de estado de manera que se satisfaga la ligadura original. Es decir, en el instante inicial de la simulación debe asignarse el mismo valor a ambas presiones.

La causalidad computacional del sistema compuesto por las Ecs. (3.73) – (3.78) puede asignarse, sustituyendo como siempre las derivadas por variables mudas y seleccionando las dos presiones como variables de estado, de la forma siguiente:

$$p_1, p_2 \quad \text{variables de estado} \quad (3.79)$$

$$[F_V] = f(t) \quad (3.80)$$

$$[p_S] = p_1 \quad (3.81)$$

$$C_1 \cdot derp_1 = F_{V,1} \quad (3.82)$$

$$C_2 \cdot derp_2 = F_{V,2} \quad (3.83)$$

$$F_V = F_{V,1} + F_{V,2} \quad (3.84)$$

$$derp_1 = derp_2 \quad (3.85)$$

donde las variables $derp_1$, $derp_2$, $F_{V,1}$ y $F_{V,2}$ pueden resolverse manipulando el lazo algebraico formado por las cuatro últimas ecuaciones.

Como vemos, este método se basa en considerar que imponer que las dos presiones sean iguales, $p_1(t) = p_2(t)$, es equivalente a imponer que inicialmente las presiones valgan lo mismo, $p_1(t_0) = p_2(t_0)$, y que además sus derivadas sean iguales, $\dot{p}_1(t) = \dot{p}_2(t)$. Sin embargo, este razonamiento no es correcto, ya que al integrar separadamente $derp_1$ y $derp_2$ pueden producirse pequeñas derivas numéricas que hagan que p_1 y p_2 difieran ligeramente, lo cual, dependiendo del problema en concreto, puede tener un efecto despreciable o no. Por este motivo, este método tampoco resulta adecuado.

Exploremos una tercera opción, que consiste en **añadir** al sistema (en vez de sustituir) la derivada respecto al tiempo de la ligadura que reduce el número de grados de libertad del sistema y seleccionar como variable de estado sólo una de las presiones, por ejemplo, p_1 .

Procediendo de esta manera, el modelo consta de una ecuación más que el modelo original: la ecuación $\dot{p}_1 = \dot{p}_2$. Es decir, consta de 7 ecuaciones. Dado que p_2 no es variable de estado, el modelo tiene 7 variables desconocidas: p_2 , $F_{V,1}$, $F_{V,2}$, F_V , p_S , $derp_1$, $derp_2$. Aplicando el algoritmo de la partición, se obtiene la asignación de la causalidad siguiente:

$$p_1 \quad \text{variable de estado} \quad (3.86)$$

$$[F_V] = f(t) \quad (3.87)$$

$$[p_S] = p_1 \quad (3.88)$$

$$p_1 = [p_2] \quad \leftarrow \text{ligadura} \quad (3.89)$$

$$C_1 \cdot derp_1 = F_{V,1} \quad (3.90)$$

$$C_2 \cdot derp_2 = F_{V,2} \quad (3.91)$$

$$derp_1 = derp_2 \quad \leftarrow \text{derivada de la ligadura} \quad (3.92)$$

$$F_V = F_{V,1} + F_{V,2} \quad (3.93)$$

donde las variables $derp_1$, $derp_2$, $F_{V,1}$ y $F_{V,2}$ se resuelven del lazo algebraico formado por las cuatro últimas ecuaciones. El resultado es por tanto plenamente satisfactorio.

El procedimiento anterior para sistemas DAE estructuralmente singulares, que es empleado por ejemplo por los entornos de modelado Dymola y OpenModelica, se basa en los principios siguientes:

- Las derivadas se sustituyen por variables mudas, las cuales son consideradas variables desconocidas y deben ser calculadas de las ecuaciones del modelo.
- Mediante manipulación simbólica de fórmulas, se deriva respecto al tiempo un cierto número de veces unas determinadas ecuaciones del modelo. Las ecuaciones derivadas se añaden al modelo. No se elimina del modelo ninguna de las ecuaciones originales.
- El número de variables seleccionadas como variables de estado es igual al número de grados de libertad del modelo.
- Las variables seleccionadas como variables de estado se calculan integrando numéricamente sus derivadas. Las variables de estado son clasificadas como variables conocidas para asignar la causalidad computacional al modelo.
- Las variables que aparecen derivadas, pero no han sido seleccionadas como variables de estado, se calculan de las ecuaciones del modelo y por tanto son clasificadas como desconocidas para asignar la causalidad computacional al modelo.

Cabe preguntarse si existe un criterio general para determinar qué ecuaciones del sistema deben ser derivadas y cuántas veces deben serlo. Esencialmente, la contestación a esa pregunta es: debe derivarse siempre que con ello se obtenga información

adicional útil. Cabe plantearse también si existe un procedimiento para determinar cuál es el número de variables de estado de un sistema DAE. Existen algoritmos que permiten responder de manera automática a estas preguntas, tales como el algoritmo de Pantelides, el cual simplemente mediante análisis de la estructura computacional del modelo permite seleccionar automáticamente qué ecuaciones hay que derivar y cuántas veces hay que hacerlo.

Los entornos de modelado para simulación de sistemas DAE emplean estos algoritmos en combinación con manipuladores simbólicos de fórmulas, mediante los cuales obtienen las ecuaciones derivadas que deben ser añadidas al modelo. También incorporan algoritmos para la selección de las variables de estado, de tal manera que todo el proceso puede ser realizado de manera automática. Veamos un ejemplo.

El Código 3.1 es la descripción en lenguaje Modelica del sistema de los dos depósitos mostrado en la Figura 3.6. Antes de hacer la traducción del modelo, es posible indicarle a Dymola que muestre, en caso de que el sistema DAE sea estructuralmente singular, las ecuaciones que automáticamente deriva y añade al modelo. Para ello, debe señalarse la opción *Output information when differentiating for index reduction*, en la ventana *Simulation Setup*. Dicha opción está señalada como (3) en la Figura 3.7.

Al realizar la traducción del modelo, Dymola muestra en su ventana de mensajes el texto siguiente:

```
Differentiated the equation
p2 = p1;

giving
der(p2) = der(p1);

Selected continuous time states
  Statically selected continuous time states
  p1
```

donde se indica que la ecuación $p_1 = p_2$ ha sido derivada simbólicamente y añadida al modelo, y que se ha seleccionado la variable p_1 como variable de estado. En la ventana *Variable Browser* será posible asignar valor inicial a la variable de estado y también modificar el valor asignado en el modelo a los parámetros (véase la Figura 3.8).

Obsérvese que en la ventana *Simulation Setup* puede también indicarse a Dymola que guarde en fichero tanto el modelo plano, como el modelo ordenado y resuelto (véanse las opciones señaladas como (1) y (2) en la Figura 3.7). El modelo plano quedará almacenado en el directorio de trabajo en un fichero llamado *DosDeposi-*

```

model DosDepositos
  import SI = Modelica.SIunits;
  SI.Pressure p1;
  SI.Pressure p2;
  SI.Pressure pS;
  SI.VolumeFlowRate Fv;
  SI.VolumeFlowRate Fv1;
  SI.VolumeFlowRate Fv2;
  parameter Real C1(unit="m3/Pa") = 1e-5;
  parameter Real C2(unit="m3/Pa") = 3e-5;
equation
  C1*der(p1) = Fv1;
  C2*der(p2) = Fv2;
  pS = p1;
  p1 = p2;
  Fv = Fv1 + Fv2;
  Fv = sin(time);
end DosDepositos;

```

Código 3.1: Fuente de líquido y dos depósitos de la Figura 3.6.

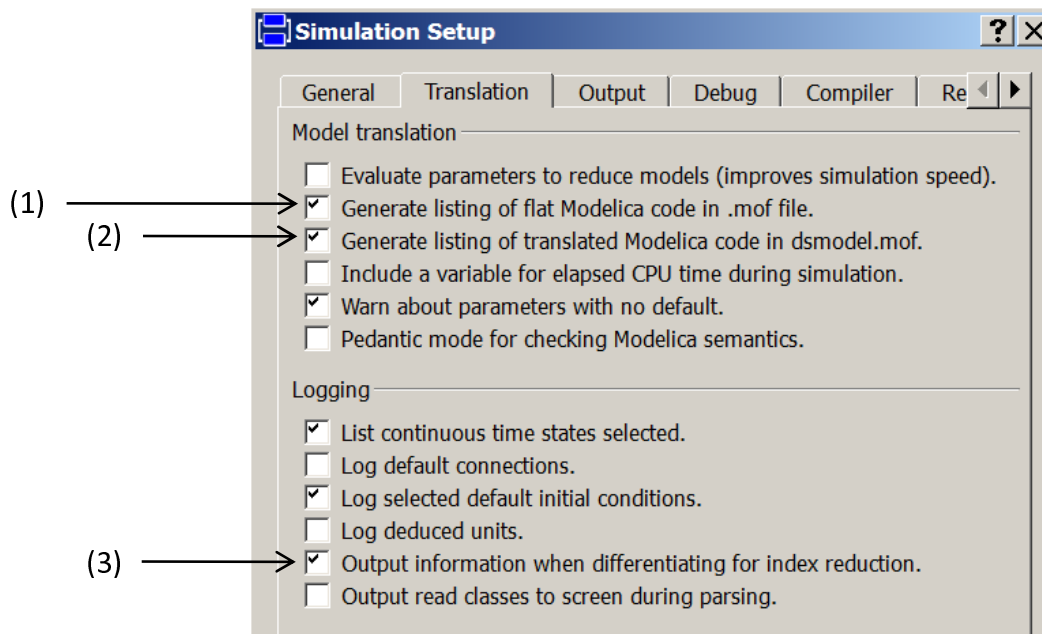


Figura 3.7: Ventana *Simulation Setup* de Dymola versión 2015. (1): guarda en fichero el modelo plano; (2): guarda en fichero el modelo ordenado y resuelto; y (3): muestra las ecuaciones que ha sido necesario derivar simbólicamente respecto al tiempo y añadir al modelo, en caso de que el modelo sea un sistema DAE estructuralmente singular.

Variable Browser		
Variables	Values	Unit
Initial Values		
☐ p1	0	Pa
☐ C1	1e-05	m3/Pa
☐ C2	3e-05	m3/Pa

Figura 3.8: Inicialización del modelo.

tos.mof. En el fichero *dsmodel.mof*, escrito igualmente en el directorio de trabajo, se muestra el modelo ordenado y resuelto.

Cabe preguntarse si existe algún procedimiento para indicar al entorno de modelado qué variables debe seleccionar como variables de estado. La respuesta a esta pregunta es sí. Para ello, en lenguaje Modelica se define un atributo de las variables de tipo Real llamado *stateSelect*. Si se asigna a dicho atributo el valor *StateSelect.always*, el entorno de modelado selecciona esa variable como variable de estado. Por ejemplo, si queremos que la variable de estado sea la variable p_2 , la sentencia de declaración de dicha variable debería ser:

```
SI.Pressure p2 ( stateSelect = StateSelect.always );
```

La selección de las variables de estado se explicará con más detalle en la Sección 3.7.

3.4. ÍNDICE DE LOS SISTEMAS DAE

El **índice** del sistema DAE es una propiedad relacionada con su causalidad computacional. Los sistemas DAE cuyo índice es mayor que uno se denominan **sistemas DAE de índice superior** y se caracterizan por el hecho de que el problema de asignación de la causalidad computacional no tiene ninguna solución que permita mantener todas las variables que aparecen derivadas en el modelo como variables de estado.

Como se vió en la sección anterior, estos sistemas DAE aparecen, por ejemplo, cuando mediante una conexión se reduce el número de grados de libertad del sistema. Es decir, cuando el número de grados de libertad del sistema completo es menor que la suma del número de grados de libertad de las partes por separado.

En este tema se definirá el concepto de índice de un modelo, se explicará qué dificultades entraña la integración numérica de los modelos de índice superior y por qué es necesario reducir el índice de los sistemas DAE antes de resolverlos numéricamente.

3.4.1. Definición de índice

El **índice** de un sistema DAE

$$\mathbf{F}(t, \mathbf{x}, \dot{\mathbf{x}}) = \mathbf{0} \quad (3.94)$$

se define como

el número mínimo de veces que deben ser derivadas respecto al tiempo todas o algunas de las ecuaciones del sistema DAE para obtener mediante manipulaciones algebraicas $\dot{\mathbf{x}}$ como una función continua de $\mathbf{x}$ y del tiempo (t).

Se muestran a continuación algunos ejemplos de cálculo del índice de sistemas DAE.

Ejemplo 1. El sistema de ecuaciones diferenciales ordinarias (ODE) explícito mostrado en la Ec. (3.95) es un DAE de **índice cero**, puesto que $\dot{\mathbf{x}}$ ya aparece expresado como una función continua de $\mathbf{x}$ y de t .

$$\dot{\mathbf{x}} = \mathbf{F}(t, \mathbf{x}) \quad (3.95)$$

Ejemplo 2. El DAE semi-implícito mostrado en las Ecs. (3.96) – (3.97) tiene **índice uno** si y sólo si la matriz Jacobiana de $\mathbf{F}_2$ respecto a $\mathbf{x}_2$ es no singular, es decir, su determinante es diferente de cero y tiene, por consiguiente, matriz inversa.

$$\dot{\mathbf{x}}_1 = \mathbf{F}_1(\mathbf{x}_1, \mathbf{x}_2, t) \quad (3.96)$$

$$0 = \mathbf{F}_2(\mathbf{x}_1, \mathbf{x}_2, t) \quad (3.97)$$

Obsérvese que derivando respecto al tiempo las ecuaciones algebraicas del sistema DAE, es decir, las Ecs. (3.97), se obtiene

$$0 = \frac{\partial \mathbf{F}_2(\mathbf{x}_1, \mathbf{x}_2, t)}{\partial \mathbf{x}_1} \cdot \dot{\mathbf{x}}_1 + \frac{\partial \mathbf{F}_2(\mathbf{x}_1, \mathbf{x}_2, t)}{\partial \mathbf{x}_2} \cdot \dot{\mathbf{x}}_2 + \frac{\partial \mathbf{F}_2(\mathbf{x}_1, \mathbf{x}_2, t)}{\partial t} \quad (3.98)$$

y que si $\frac{\partial \mathbf{F}_2}{\partial \mathbf{x}_2}$ es no singular, entonces es posible despejar $\dot{\mathbf{x}}_2$ de la Ec. (3.98). Sustituyendo entonces $\dot{\mathbf{x}}_1$ por $\mathbf{F}_1(\mathbf{x}_1, \mathbf{x}_2, t)$, se obtiene $\dot{\mathbf{x}}_2$ como una función continua de $\mathbf{x}_1$, $\mathbf{x}_2$ y t .

Ejemplo 3. Como un caso particular del ejemplo anterior, consideremos el siguiente sistema DAE

$$\dot{x} = x + y \quad (3.99)$$

$$0 = x + 2 \cdot y + a(t) \quad (3.100)$$

donde $a(t)$ es una función del tiempo, continua y diferenciable. Derivando la Ec. (3.100) y despejando $\dot{y}$ de la ecuación derivada, se obtiene

$$\dot{x} = x + y \quad (3.101)$$

$$\dot{y} = -\frac{\dot{x} + \dot{a}(t)}{2} \quad (3.102)$$

y sustituyendo la Ec. (3.101) en (3.102), se obtiene el siguiente ODE explícito:

$$\dot{x} = x + y \quad (3.103)$$

$$\dot{y} = -\frac{x + y + \dot{a}(t)}{2} \quad (3.104)$$

Por tanto, el sistema DAE tiene **índice uno**.

Ejemplo 4. El sistema DAE mostrado en las Ecs. (3.105) – (3.106) tiene **índice 2** si y sólo si la matriz Jacobiana de $\mathbf{F}_1$ respecto a $\mathbf{x}_2$ y la matriz Jacobiana de $\mathbf{F}_2$ respecto a $\mathbf{x}_1$ son no singulares. Obsérvese que para obtener el sistema en la forma ODE explícito hay que derivar una vez la Ec. (3.105) y dos veces la Ec. (3.106).

$$\dot{\mathbf{x}}_1 = \mathbf{F}_1(\mathbf{x}_1, \mathbf{x}_2, t) \quad (3.105)$$

$$0 = \mathbf{F}_2(\mathbf{x}_1, t) \quad (3.106)$$

Ejemplo 5. Consideremos el sistema DAE formado por las Ecs. (3.107) y (3.108).

$$\dot{x}_1 = x_2 \quad (3.107)$$

$$x_1 = t^2 + t + 2 \quad (3.108)$$

En la segunda ecuación sólo interviene x_1 , con lo cual la derivada de x_1 deberá calcularse de esa ecuación. En consecuencia, la derivada de x_2 deberá obtenerse de la primera ecuación. Derivando una vez el sistema se obtiene:

$$\ddot{x}_1 = \dot{x}_2 \quad (3.109)$$

$$\dot{x}_1 = 2 \cdot t + 1 \quad (3.110)$$

Derivando otra vez la segunda ecuación se obtiene

$$\ddot{x}_1 = 2 \quad (3.111)$$

Sustituyendo (3.111) en (3.109), se obtiene el sistema expresado en la forma ODE explícito:

$$\dot{x}_1 = 2 \cdot t + 1 \quad (3.112)$$

$$\dot{x}_2 = 2 \quad (3.113)$$

Puesto que ha sido necesario derivar 2 veces, sistema DAE compuesto por las Ecs. (3.107) y (3.108) tiene **índice 2**.

Ejemplo 6. Considérese el siguiente sistema DAE:

$$\dot{x}_1 = f_1(t) - x_3 \quad (3.114)$$

$$\dot{x}_2 = f_2(t) - x_1 \quad (3.115)$$

$$x_2 = f_3(t) \quad (3.116)$$

Puesto que x_3 sólo interviene en la Ec. (3.114), será necesario derivar esa ecuación para calcular $\dot{x}_3$. Asimismo, la única variable que aparece en la Ec. (3.116) es x_2 , con lo cual $\dot{x}_2$ deberá calcularse de la derivada de dicha ecuación. Finalmente, $\dot{x}_1$ se obtendrá de derivar la Ec. (3.115).

Derivando las Ecs. (3.114) – (3.116) se obtiene

$$\ddot{x}_1 = \dot{f}_1(t) - \dot{x}_3 \quad (3.117)$$

$$\ddot{x}_2 = \dot{f}_2(t) - \dot{x}_1 \quad (3.118)$$

$$\dot{x}_2 = \dot{f}_3(t) \quad (3.119)$$

Obsérvese que en las Ecs. (3.117) y (3.118) aparecen $\ddot{x}_1$ y $\ddot{x}_2$, respectivamente. Para expresar dichas derivadas segundas en función de las variables (x_1, x_2, x_3) , las derivadas primeras $(\dot{x}_1, \dot{x}_2, \dot{x}_3)$ y el tiempo (t) , derivamos una vez la Ec. (3.118)

$$\ddot{x}_2 = \ddot{f}_2(t) - \ddot{x}_1 \quad (3.120)$$

y calculamos la segunda y tercera derivada de x_2 derivando dos veces más la Ec. (3.119)

$$\ddot{x}_2 = \ddot{f}_3(t) \quad (3.121)$$

$$\ddot{x}_2 = \ddot{f}_3(t) \quad (3.122)$$

Sustituyendo, se obtiene

$$\ddot{f}_2(t) - \ddot{f}_3(t) = \dot{f}_1(t) - \dot{x}_3 \quad (3.123)$$

$$\ddot{f}_3(t) = \dot{f}_2(t) - \dot{x}_1 \quad (3.124)$$

$$\dot{x}_2 = \dot{f}_3(t) \quad (3.125)$$

es decir, se obtiene el ODE siguiente

$$\dot{x}_1 = \dot{f}_2(t) - \ddot{f}_3(t) \quad (3.126)$$

$$\dot{x}_2 = \dot{f}_3(t) \quad (3.127)$$

$$\dot{x}_3 = \dot{f}_1(t) - \dot{f}_2(t) + \ddot{f}_3(t) \quad (3.128)$$

Puesto que ha sido necesario derivar 3 veces, el sistema DAE tiene **índice 3**.

Ejemplo 7. La propiedad enunciada a continuación resulta en ocasiones útil para calcular el índice de sistemas DAE. Si el sistema DAE

$$\mathbf{F}(t, \mathbf{x}, \dot{\mathbf{x}}) = 0 \quad (3.129)$$

tiene **índice** $\mathbf{n}$, entonces el DAE semi-implícito

$$\dot{\mathbf{x}} = \mathbf{z} \quad (3.130)$$

$$\mathbf{F}(t, \mathbf{x}, \mathbf{z}) = 0 \quad (3.131)$$

tiene **índice** $\mathbf{n}+1$.

Obsérvese que si el DAE de la Ec. (3.129) tiene índice n , mediante n derivaciones se obtendrá $\dot{\mathbf{x}}$ en función de $\mathbf{x}$ y t . Ahora bien, por la Ec. (3.130) $\dot{\mathbf{x}}$ es igual a $\mathbf{z}$, así pues, mediante n derivaciones se obtendrá $\mathbf{z}$ en función de $\mathbf{x}$ y t . Entonces, mediante una derivación más se obtendrá $\dot{\mathbf{z}}$.

Ejemplo 8. Consideremos el modelo de los dos depósitos conectados a la fuente descrito en la Sección 3.3. Se muestran nuevamente las ecuaciones para mayor comodidad del lector.

$$C_1 \cdot \dot{p}_1 = F_{V,1} \quad (3.132)$$

$$C_2 \cdot \dot{p}_2 = F_{V,2} \quad (3.133)$$

$$p_S = p_1 \quad (3.134)$$

$$p_1 = p_2 \quad (3.135)$$

$$F_V = F_{V,1} + F_{V,2} \quad (3.136)$$

$$F_V = f(t) \quad (3.137)$$

Para calcular el índice del sistema DAE compuesto por las Ecs. (3.132) – (3.137), se analiza qué ecuaciones deben derivarse respecto al tiempo a fin de obtener un ODE explícito

$$\dot{\mathbf{x}} = \mathbf{g}(\mathbf{x}, t) \quad (3.138)$$

donde

$$\mathbf{x} = \{p_1, p_2, p_S, F_V, F_{V,1}, F_{V,2}\} \quad (3.139)$$

Derivando las ecuaciones algebraicas del sistema DAE se obtiene:

$$C_1 \cdot \dot{p}_1 = F_{V,1} \quad (3.140)$$

$$C_2 \cdot \dot{p}_2 = F_{V,2} \quad (3.141)$$

$$\dot{p}_S = \dot{p}_1 \quad (3.142)$$

$$\dot{p}_1 = \dot{p}_2 \quad (3.143)$$

$$\dot{F}_V = \dot{F}_{V,1} + \dot{F}_{V,2} \quad (3.144)$$

$$\dot{F}_V = \dot{f}(t) \quad (3.145)$$

Veamos si es posible calcular $\dot{\mathbf{x}}$ de las Ecs. (3.140) – (3.145), para lo cual se asigna la causalidad computacional suponiendo que las variables desconocidas son $\dot{\mathbf{x}}$. La matriz de incidencia original se muestra a continuación.

$$\begin{array}{l} C_1 \cdot \dot{p}_1 = F_{V,1} \\ C_2 \cdot \dot{p}_2 = F_{V,2} \\ \dot{p}_S = \dot{p}_1 \\ \dot{p}_1 = \dot{p}_2 \\ \dot{F}_V = \dot{F}_{V,1} + \dot{F}_{V,2} \\ \dot{F}_V = \dot{f}(t) \end{array} \begin{pmatrix} \dot{p}_1 & \dot{p}_2 & \dot{F}_V & \dot{p}_S & \dot{F}_{V,1} & \dot{F}_{V,2} \\ \text{X} & 0 & 0 & 0 & 0 & 0 \\ 0 & \text{X} & 0 & 0 & 0 & 0 \\ \text{X} & 0 & 0 & \text{X} & 0 & 0 \\ \text{X} & \text{X} & 0 & 0 & 0 & 0 \\ 0 & 0 & \text{X} & 0 & \text{X} & \text{X} \\ 0 & 0 & \text{X} & 0 & 0 & 0 \end{pmatrix} \quad (3.146)$$

1. Hay tres ecuaciones en las cuales interviene una única incógnita: $\dot{p}_1$ en la Ec. (3.140), $\dot{p}_2$ en la Ec. (3.141) y $\dot{F}_V$ en la Ec. (3.145). Por tanto, estas incógnitas deben calcularse de esas ecuaciones. Se ubican esas tres ecuaciones a las primeras filas de la matriz y las variables a las primeras columnas.

La variable $\dot{p}_S$ sólo interviene en la Ec. (3.142). Debe calcularse de ella. Como esta variable no interviene en ninguna otra ecuación, se pasa a la última columna de la matriz y la Ec. (3.142) a la última fila.

$$\begin{array}{l} C_1 \cdot \dot{p}_1 = F_{V,1} \\ C_2 \cdot \dot{p}_2 = F_{V,2} \\ \dot{F}_V = \dot{f}(t) \\ \dot{p}_1 = \dot{p}_2 \\ \dot{F}_V = \dot{F}_{V,1} + \dot{F}_{V,2} \\ \dot{p}_S = \dot{p}_1 \end{array} \begin{pmatrix} \dot{p}_1 & \dot{p}_2 & \dot{F}_V & \dot{F}_{V,1} & \dot{F}_{V,2} & \dot{p}_S \\ \boxed{\text{X}} & 0 & 0 & 0 & 0 & 0 \\ 0 & \boxed{\text{X}} & 0 & 0 & 0 & 0 \\ 0 & 0 & \boxed{\text{X}} & 0 & 0 & 0 \\ \text{X} & \text{X} & 0 & 0 & 0 & 0 \\ 0 & 0 & \text{X} & \text{X} & \text{X} & 0 \\ \text{X} & 0 & 0 & 0 & 0 & \boxed{\text{X}} \end{pmatrix} \quad (3.147)$$

2. La Ec. (3.143) es redundante, puesto que $\dot{p}_1$ y $\dot{p}_2$ ya han sido evaluadas de otras ecuaciones. Por otra parte, quedan dos variables desconocidas por evaluar, $\dot{F}_{V,1}$ y $\dot{F}_{V,2}$, las cuales intervienen en una única ecuación: la Ec. (3.144).

Puesto que el sistema obtenido no es un ODE bien definido, se deriva una vez más. Las ecuaciones a derivar se escogen atendiendo al razonamiento siguiente:

- La Ec. (3.142) es la única en la que interviene la variable $\dot{p}_S$ y, por ello, debe emplearse para evaluar esta variable. Si se deriva esta ecuación, se introduce una nueva variable, $\ddot{p}_S$, que debe evaluarse de la nueva ecuación. Dado que derivar esta ecuación no introduce ligaduras adicionales sobre las incógnitas del sistema, no tiene sentido hacerlo.
- Lo mismo sucede con las Ecs. (3.144) y (3.145), cuya derivación respecto al tiempo introduce las nuevas incógnitas $\ddot{F}_V$, $\ddot{F}_{V,1}$ y $\ddot{F}_{V,2}$.
- Así pues, se derivan las Ecs. (3.140), (3.141) y (3.143), obteniéndose

$$C_1 \cdot \ddot{p}_1 = \ddot{F}_{V,1} \quad (3.148)$$

$$C_2 \cdot \ddot{p}_2 = \ddot{F}_{V,2} \quad (3.149)$$

$$\ddot{p}_1 = \ddot{p}_2 \quad (3.150)$$

Añadiendo estas tres ecuaciones al sistema y eliminado del mismo la ecuación redundante (3.143), se obtiene:

$$C_1 \cdot \dot{p}_1 = \dot{F}_{V,1} \quad (3.151)$$

$$C_2 \cdot \dot{p}_2 = \dot{F}_{V,2} \quad (3.152)$$

$$\dot{p}_S = \dot{p}_1 \quad (3.153)$$

$$\dot{F}_V = \dot{F}_{V,1} + \dot{F}_{V,2} \quad (3.154)$$

$$\dot{F}_V = \dot{f}(t) \quad (3.155)$$

$$C_1 \cdot \ddot{p}_1 = \ddot{F}_{V,1} \quad (3.156)$$

$$C_2 \cdot \ddot{p}_2 = \ddot{F}_{V,2} \quad (3.157)$$

$$\ddot{p}_1 = \ddot{p}_2 \quad (3.158)$$

Puede asignarse la causalidad a este sistema, considerando que las incógnitas son $\{\dot{p}_1, \dot{p}_2, \dot{p}_S, \dot{F}_V, \dot{F}_{V,1}, \dot{F}_{V,2}, \ddot{p}_1, \ddot{p}_2\}$, de la forma descrita a continuación.

- La única incógnita que interviene en la Ec. (3.151) es $\dot{p}_1$, por ello esta incógnita debe evaluarse de esa ecuación. Por el mismo motivo, $\dot{p}_2$ debe evaluarse de la Ec. (3.152) y $\dot{F}_V$ de la Ec. (3.155).
- $\dot{p}_S$ debe evaluarse de la Ec. (3.153), en la cual puede sustituirse $\dot{p}_1$ por $\frac{F_{V,1}}{C_1}$.
- Finalmente, $\dot{F}_{V,1}$, $\dot{F}_{V,2}$, $\ddot{p}_1$ y $\ddot{p}_2$ pueden calcularse del lazo algebraico lineal

$$\dot{F}_V = \dot{F}_{V,1} + \dot{F}_{V,2} \quad (3.159)$$

$$C_1 \cdot \ddot{p}_1 = \dot{F}_{V,1} \quad (3.160)$$

$$C_2 \cdot \ddot{p}_2 = \dot{F}_{V,2} \quad (3.161)$$

$$\ddot{p}_1 = \ddot{p}_2 \quad (3.162)$$

La matriz de incidencia BLT es:

$$\begin{array}{l}
 C_1 \cdot \dot{p}_1 = F_{V,1} \\
 C_2 \cdot \dot{p}_2 = F_{V,2} \\
 \dot{F}_V = \dot{f}(t) \\
 \dot{p}_S = \dot{p}_1 \\
 \dot{F}_V = \dot{F}_{V,1} + \dot{F}_{V,2} \\
 C_1 \cdot \ddot{p}_1 = \dot{F}_{V,1} \\
 C_2 \cdot \ddot{p}_2 = \dot{F}_{V,2} \\
 \ddot{p}_1 = \ddot{p}_2
 \end{array}
 \begin{pmatrix}
 \boxed{X} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & \boxed{X} & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & \boxed{X} & 0 & 0 & 0 & 0 & 0 \\
 X & 0 & 0 & \boxed{X} & 0 & 0 & 0 & 0 \\
 0 & 0 & X & 0 & \boxed{X} & X & 0 & 0 \\
 0 & 0 & 0 & 0 & X & 0 & X & 0 \\
 0 & 0 & 0 & 0 & 0 & X & 0 & X \\
 0 & 0 & 0 & 0 & 0 & 0 & X & X
 \end{pmatrix} \quad (3.163)$$

Las expresiones para calcular $\dot{F}_{V,1}$ y $\dot{F}_{V,2}$ pueden obtenerse eliminando $\ddot{p}_1$ y $\ddot{p}_2$ del lazo algebraico formado por las Ecs. (3.159) – (3.162)

$$\dot{F}_{V,1} = C_1 \cdot \frac{\dot{F}_V}{C_1 + C_2} \quad (3.164)$$

$$\dot{F}_{V,2} = C_2 \cdot \frac{\dot{F}_V}{C_1 + C_2} \quad (3.165)$$

y sustituyendo $\dot{F}_V$ por $\dot{f}(t)$,

$$\dot{F}_{V,1} = C_1 \cdot \frac{\dot{f}(t)}{C_1 + C_2} \quad (3.166)$$

$$\dot{F}_{V,2} = C_2 \cdot \frac{\dot{f}(t)}{C_1 + C_2} \quad (3.167)$$

Así pues, se obtiene el ODE explícito mostrado a continuación, con lo cual, el sistema original tiene **índice dos**.

$$\dot{p}_1 = \frac{F_{V,1}}{C_1} \quad (3.168)$$

$$\dot{p}_2 = \frac{F_{V,2}}{C_2} \quad (3.169)$$

$$\dot{F}_V = \dot{f}(t) \quad (3.170)$$

$$\dot{p}_S = \frac{F_{V,1}}{C_1} \quad (3.171)$$

$$\dot{F}_{V,1} = C_1 \cdot \frac{\dot{f}(t)}{C_1 + C_2} \quad (3.172)$$

$$\dot{F}_{V,2} = C_2 \cdot \frac{\dot{f}(t)}{C_1 + C_2} \quad (3.173)$$

3.4.2. Dificultades asociadas al índice superior

El índice no es una característica intrínseca del modelo del sistema físico, sino de la forma en que se han expresado sus ecuaciones, ya que puede reducirse el índice de un sistema DAE derivando un determinado número de veces algunas de sus ecuaciones.

La razón por la cual debe realizarse la reducción de índice es que los algoritmos numéricos para la solución numérica de sistemas DAE, que son muy eficaces en la resolución de problemas de índice cero y uno, en ocasiones funcionan para problemas de índice dos, pero fallan para problemas de índice mayor que dos. Los sistemas DAE cuyo índice es mayor que uno se denominan de índice superior y son difíciles de resolver directamente porque, en principio, implican derivar además de integrar. En general, la dificultad numérica aumenta con el índice del problema.

Veamos un ejemplo. Considérese el siguiente sistema DAE de índice 2:

$$\dot{x}_1 = x_2 \quad (3.174)$$

$$x_1 = t^2 + t + 2 \quad (3.175)$$

Supongamos que para resolver numéricamente este sistema DAE se aplica el método de integración implícito del trapecio a la primera ecuación. El sistema discretizado temporalmente es

$$x_1(t_{k+1}) = x_1(t_k) + h \cdot \frac{x_2(t_{k+1}) + x_2(t_k)}{2} + O(h^2) \quad (3.176)$$

$$x_1(t_{k+1}) = t_{k+1}^2 + t_{k+1} + 2 \quad (3.177)$$

donde h es el paso de integración. Se puede emplear la segunda ecuación para obtener $x_1(t_{k+1})$ y entonces emplear la primera ecuación para calcular $x_2(t_{k+1})$. Despejando, se observa que $x_2(t_{k+1})$ se calcula mediante derivación numérica y que el término de error no es $O(h^2)$, sino $O(h)$:

$$x_2(t_{k+1}) = 2 \cdot \underbrace{\frac{x_1(t_{k+1}) - x_1(t_k)}{h}}_{\text{Derivada numérica}} - x_2(t_k) + O(h) \quad (3.178)$$

El cálculo de la derivada numérica resulta problemático, ya que tan malo es escoger valores de h demasiado pequeños como demasiado grandes.

- *Problema de escoger h demasiado pequeño.* Los números se almacenan en el ordenador en palabras binarias de longitud finita y los números reales en particular son representados empleando un número finito de dígitos decimales. A los efectos del cálculo de la derivada numérica, esto implica que si se escoge un valor de h demasiado pequeño, entonces el cálculo de la diferencia de los dos números del denominador, que son casi iguales, estará muy afectado por el error de redondeo y además se verá grandemente amplificado al ser dividido por el valor muy pequeño de h .
- *Problema de escoger h demasiado grande.* Si se escoge h demasiado grande se producen errores de aproximación, ya que idealmente $\dot{x}_1(t_{k+1})$ es igual a $\frac{x_1(t_{k+1}) - x_1(t_k)}{h}$ en el límite cuando $h \rightarrow 0$.

El problema que existe para la resolución numérica del DAE formado por las Ecs. (3.174) y (3.175) es aún más evidente si se usa el método de Euler explícito. Aplicando dicho método a la primera ecuación, el sistema DAE discretizado es:

$$x_1(t_{k+1}) = x_1(t_k) + h \cdot x_2(t_k) \quad (3.179)$$

$$x_1(t_{k+1}) = t_{k+1}^2 + t_{k+1} + 2 \quad (3.180)$$

Como se ve, ninguna de las dos ecuaciones contiene $x_2(t_{k+1})$. No es posible resolver el sistema usando el método de Euler explícito ni, por la misma razón, usando ningún otro método explícito de integración.

El sistema DAE formado por las Ecs. (3.174) y (3.175) tiene además un problema con la inicialización. Si se asume que x_1 es una variable de estado del sistema, es decir, que se calcula mediante la integración numérica de su derivada, entonces sería posible asignar un valor inicial $x_1(t_0)$ arbitrario a la variable x_1 en el instante inicial t_0 . Sin embargo, este valor inicial debe satisfacer la Ec. (3.175), y por tanto $x_1(t_0)$ debe valer $t_0^2 + t_0 + 2$. El número de valores iniciales que pueden asignarse arbitrariamente en este problema no es el mismo que el número de variables derivadas. Esta es una propiedad común a todos los sistemas DAE de índice superior.

3.5. INICIALIZACIÓN DE SISTEMAS DAE

Con el fin de estudiar el problema de la inicialización de sistemas DAE, representaremos el sistema DAE de la forma mostrada a continuación, en la cual se distingue explícitamente entre aquellas variables que aparecen derivadas y las que no

$$\mathbf{F}(\mathbf{x}, \dot{\mathbf{x}}, \mathbf{y}, t) = 0 \quad (3.181)$$

donde

- $\mathbf{x}, \dot{\mathbf{x}} \in \mathbb{R}^n$ son las incógnitas que aparecen derivadas y sus derivadas, respectivamente.
- $\mathbf{y} \in \mathbb{R}^m$ son las incógnitas algebraicas. Es decir, aquellas que no aparecen derivadas en el sistema.
- $\mathbf{F} : G \subseteq \mathbb{R}^n \times \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R} \rightarrow \mathbb{R}^{n+m}$ son $n + m$ funciones reales, que en general son no lineales respecto de $\dot{\mathbf{x}}$.

Para que el conjunto de condiciones iniciales $(\mathbf{x}_0, \dot{\mathbf{x}}_0, \mathbf{y}_0)$ sea consistente, es condición necesaria que satisfagan el **sistema DAE original** (3.181) en el instante inicial t_0 . Es decir, que se satisfaga:

$$\mathbf{F}(\mathbf{x}_0, \dot{\mathbf{x}}_0, \mathbf{y}_0, t_0) = 0 \quad (3.182)$$

Sin embargo, en ocasiones esta condición no es suficiente para asegurar la consistencia del sistema. Éste es el caso cuando derivando respecto al tiempo algunas de las ecuaciones originales un determinado número de veces, se obtienen ecuaciones que imponen nuevas ligaduras que deben ser satisfechas por las condiciones iniciales $(\mathbf{x}_0, \dot{\mathbf{x}}_0, \mathbf{y}_0)$. Estas ecuaciones, obtenidas derivando respecto al tiempo las ecuaciones del sistema DAE y que imponen ligaduras adicionales a las condiciones iniciales, se denominan **ligaduras ocultas**.

Hay sistemas DAE que no poseen ligaduras ocultas. Son aquellos en los cuales al derivar sus ecuaciones respecto al tiempo se introducen nuevas variables, de modo que las nuevas ecuaciones son satisfechas por todos los posibles valores de las condiciones iniciales $(\mathbf{x}_0, \dot{\mathbf{x}}_0, \mathbf{y}_0)$ y un conjunto apropiado de valores de las nuevas variables. En este caso, la derivación no genera información útil al no introducir ninguna ligadura adicional para los valores iniciales $(\mathbf{x}_0, \dot{\mathbf{x}}_0, \mathbf{y}_0)$.

La existencia de ligaduras ocultas está relacionada con la singularidad estructural del sistema DAE original. Supongamos que se asigna la causalidad computacional al sistema DAE original, para lo cual se asume que todas las variables que aparecen derivadas son variables de estado.

- Si el sistema DAE original tiene ligaduras ocultas, entonces al asignar su causalidad computacional será clasificado como estructuralmente singular. Vimos que los entornos de modelado manipulan simbólicamente los sistemas estructuralmente singulares, derivando algunas de sus ecuaciones y añadiéndolas al sistema. El objetivo de dicha manipulación es precisamente añadir al sistema DAE sus ligaduras ocultas. Al hacerlo, se está reduciendo el índice del sistema DAE.
- Aquellos sistemas que no poseen ligaduras ocultas serán clasificados como estructuralmente no singulares y será posible obtener su matriz de incidencia BLT.

3.5.1. Ligaduras ocultas y reducción del índice

En esta sección se muestran varios ejemplos que pretenden ilustrar la búsqueda de las ligaduras ocultas, la reducción del índice y la asignación de la causalidad computacional. El procedimiento seguido en todos los casos es el siguiente. Primeramente, se analiza si el sistema DAE original tiene ligaduras ocultas. A continuación, se procede de una de las dos maneras siguientes:

- Si el sistema no tiene ligaduras ocultas, entonces se asigna su causalidad computacional escogiendo tantas variables de estado como variables aparecen derivadas en el modelo.
- Si el sistema tiene ligaduras ocultas:
 1. Se añaden las ligaduras ocultas al sistema. El sistema formado por el sistema DAE original más sus ligaduras ocultas se denomina **sistema DAE ampliado**. El sistema DAE ampliado tiene la misma solución que el sistema DAE original y tiene los mismos grados de libertad, pero su índice es menor con lo cual su resolución numérica es menos problemática.
 2. Se calcula el número de grados de libertad del sistema DAE ampliado. Este número será menor que el número de variables que aparecen derivadas en el sistema DAE original.
 3. Se asigna la causalidad computacional del sistema DAE ampliado, escogiendo tantas variables de estado como grados de libertad tenga el sistema.

Ejemplo 1. Consideremos nuevamente el sistema DAE del ejemplo mostrado de la sección anterior.

$$\dot{x}_1 = x_2 \quad (3.183)$$

$$x_1 = t^2 + t + 2 \quad (3.184)$$

El problema consiste en determinar si las derivadas respecto al tiempo de estas ecuaciones imponen ligaduras adicionales a los valores iniciales de las variables $(x_1, \dot{x}_1, x_2)$, a los que llamaremos $(x_{1_0}, \dot{x}_{1_0}, x_{2_0})$. Derivando las dos ecuaciones respecto al tiempo se obtiene:

$$\ddot{x}_1 = \dot{x}_2 \quad (3.185)$$

$$\dot{x}_1 = 2 \cdot t + 1 \quad (3.186)$$

La Ec. (3.185) contiene únicamente las variables $\ddot{x}_1$ y $\dot{x}_2$, con lo que no impone ninguna ligadura sobre los valores iniciales de $(x_1, \dot{x}_1, x_2)$.

La Ec. (3.186) impone una ligadura sobre el valor inicial de la variable $\dot{x}_1$, el cual deberá ser igual a $2 \cdot t_0 + 1$.

Derivando respecto al tiempo las Ecs. (3.185) y (3.186) se obtienen ecuaciones en las que aparecen nuevas variables, no imponiendo ninguna ligadura adicional sobre los valores iniciales de $(x_1, \dot{x}_1, x_2)$.

Por tanto, los valores iniciales $(x_{1_0}, \dot{x}_{1_0}, x_{2_0})$ deben satisfacer el sistema DAE original y la Ec. (3.186). Es decir, deben satisfacer las tres ecuaciones mostradas a continuación, que constituyen el sistema DAE ampliado.

$$\dot{x}_1 = x_2 \quad (3.187)$$

$$x_1 = t^2 + t + 2 \quad (3.188)$$

$$\dot{x}_1 = 2 \cdot t + 1 \quad (3.189)$$

Llamando t_0 al instante inicial de la simulación, se obtiene que los valores iniciales son:

$$x_{1_0} = t_0^2 + t_0 + 2 \quad (3.190)$$

$$x_{2_0} = 2 \cdot t_0 + 1 \quad (3.191)$$

$$\dot{x}_{1_0} = 2 \cdot t_0 + 1 \quad (3.192)$$

Los valores iniciales quedan completamente determinados de las ecuaciones del sistema DAE ampliado, con lo cual el modelo tiene cero grados de libertad. Ninguna de las variables del modelo es variable de estado. Obsérvese que el sistema DAE compuesto por las Ecs. (3.187) – (3.189) es equivalente al sistema DAE original y su resolución numérica no entraña ninguna dificultad. A continuación se muestra el modelo ordenado y resuelto.

$$[x_1] = t^2 + t + 2 \quad (3.193)$$

$$[derx_1] = 2 \cdot t + 1 \quad (3.194)$$

$$[x_2] = derx_1 \quad (3.195)$$

Ejemplo 2. Este ejemplo muestra un sistema DAE en el cual la derivación de sus ecuaciones no supone ninguna ligadura adicional. Consideremos el siguiente sistema DAE

$$\dot{x} = x + y \quad (3.196)$$

$$0 = x + 2 \cdot y + a(t) \quad (3.197)$$

donde $a(t)$ es una función del tiempo, continua y diferenciable. Derivando estas dos ecuaciones respecto al tiempo, se obtiene

$$\ddot{x} = \dot{x} + \dot{y} \quad (3.198)$$

$$0 = \dot{x} + 2 \cdot \dot{y} + \dot{a}(t) \quad (3.199)$$

Estas ecuaciones contienen dos nuevas variables $(\dot{y}, \ddot{x})$ y son satisfechas por todos los valores iniciales $(x_0, \dot{x}_0, y_0)$ de las variables originales $(x, \dot{x}, y)$ si se seleccionan como valores iniciales para las nuevas variables:

$$\dot{y}_0 = -\frac{\dot{x}_0 + \dot{a}(t_0)}{2} \quad (3.200)$$

$$\ddot{x}_0 = \dot{x}_0 + \dot{y}_0 \quad (3.201)$$

Por tanto, las derivadas no representan nuevas ligaduras que deban ser satisfechas por las condiciones iniciales.

Las derivadas sucesivas respecto del tiempo de las ecuaciones no suponen ninguna ligadura adicional sobre las variables originales $(x, \dot{x}, y)$, ya que introducen nuevas variables: las derivadas sucesivas de $\dot{x}$ e y .

Es suficiente, por tanto, con que el vector de condiciones iniciales $(x_0, \dot{x}_0, y_0)$ satisfaga el sistema original. El vector de condiciones iniciales tiene 3 componentes, los cuales deben satisfacer 2 ecuaciones, por tanto el modelo tiene un

grado de libertad. Puede seleccionarse una variable como variable de estado. Escogiendo x como variable de estado y asignando la causalidad computacional, se obtiene el siguiente modelo ordenado y resuelto:

$$[y] = -\frac{x + a(t)}{2} \quad (3.202)$$

$$[derx] = x + y \quad (3.203)$$

Ejemplo 3. Sea el siguiente sistema DAE

$$\dot{x}_1 + \dot{x}_2 = a(t) \quad (3.204)$$

$$x_1 + x_2^2 = b(t) \quad (3.205)$$

donde $a(t)$ y $b(t)$ son funciones del tiempo, continuas y diferenciables.

Para examinar si las condiciones iniciales $(x_{1_0}, x_{2_0}, \dot{x}_{1_0}, \dot{x}_{2_0})$ deben satisfacer alguna ligadura adicional, se deriva el sistema respecto al tiempo, obteniéndose:

$$\ddot{x}_1 + \ddot{x}_2 = \dot{a}(t) \quad (3.206)$$

$$\dot{x}_1 + 2 \cdot x_2 \cdot \dot{x}_2 = \dot{b}(t) \quad (3.207)$$

La primera ecuación introduce dos nuevas variables $(\ddot{x}_1, \ddot{x}_2)$ y no contiene ninguna de las incógnitas originales $(x_1, x_2, \dot{x}_1, \dot{x}_2)$. La segunda ecuación sí supone una ligadura adicional. Por tanto, las condiciones iniciales $(x_{1_0}, x_{2_0}, \dot{x}_{1_0}, \dot{x}_{2_0})$ deben satisfacer:

$$\dot{x}_1 + \dot{x}_2 = a(t) \quad (3.208)$$

$$x_1 + x_2^2 = b(t) \quad (3.209)$$

$$\dot{x}_1 + 2 \cdot x_2 \cdot \dot{x}_2 = \dot{b}(t) \quad (3.210)$$

Derivando por segunda vez el sistema original respecto del tiempo se obtiene:

$$\frac{d^3 x_1}{dt^3} + \frac{d^3 x_2}{dt^3} = \ddot{a}(t) \quad (3.211)$$

$$\ddot{x}_1 + 2 \cdot \dot{x}_2^2 + 2 \cdot x_2 \ddot{x}_2 = \ddot{b}(t) \quad (3.212)$$

La primera ecuación introduce dos nuevas variables $(\frac{d^3 x_1}{dt^3}, \frac{d^3 x_2}{dt^3})$. La segunda ecuación contiene $\dot{x}_2$ y x_2 , pero no supone una ligadura adicional si para cualquier valor inicial de $\dot{x}_2$ y x_2 pueden encontrarse unos valores iniciales de $\ddot{x}_1$, $\ddot{x}_2$ que satisfagan el sistema

$$\ddot{x}_1 + \ddot{x}_2 = \dot{a}(t) \quad (3.213)$$

$$\ddot{x}_1 + 2 \cdot \dot{x}_2^2 + 2 \cdot x_2 \cdot \ddot{x}_2 = \ddot{b}(t) \quad (3.214)$$

o, equivalentemente

$$\ddot{x}_1 + \ddot{x}_2 = \dot{a}(t) \quad (3.215)$$

$$\ddot{x}_1 + 2 \cdot x_2 \cdot \ddot{x}_2 = \ddot{b}(t) - 2 \cdot \dot{x}_2^2 \quad (3.216)$$

Puesto que el determinante del sistema es $(2 \cdot x_2 - 1)$, es posible encontrar unos valores iniciales de $\ddot{x}_1$, $\ddot{x}_2$ que satisfagan el sistema siempre que el valor inicial de x_2 sea diferente de 0.5. Obsérvese que si $x_{2_0} = 0.5$, los términos a la izquierda de la igualdad de las ecuaciones primera y tercera de

$$\dot{x}_1 + \dot{x}_2 = a(t) \quad (3.217)$$

$$x_1 + x_2^2 = b(t) \quad (3.218)$$

$$\dot{x}_1 + 2 \cdot x_2 \cdot \dot{x}_2 = \dot{b}(t) \quad (3.219)$$

son idénticos.

Derivando por tercera vez el sistema original, se obtiene:

$$\frac{d^4 x_1}{dt^4} + \frac{d^4 x_2}{dt^4} = \frac{d^3 a(t)}{dt^3} \quad (3.220)$$

$$\frac{d^3 x_1}{dt^3} + 4 \cdot \dot{x}_2 \cdot \ddot{x}_2 + 2 \cdot \dot{x}_2 \cdot \ddot{x}_2 + 2 \cdot x_2 \cdot \frac{d^3 x_2}{dt^3} = \frac{d^3 b(t)}{dt^3} \quad (3.221)$$

La primera ecuación introduce dos nuevas variables $\left(\frac{d^4x_1}{dt^4}, \frac{d^4x_2}{dt^4}\right)$. La segunda ecuación contiene $\dot{x}_2$ y $\ddot{x}_2$, pero no supone una ligadura adicional, ya que para cualquier valor inicial de $\dot{x}_2$ y $\ddot{x}_2$ pueden encontrarse (siempre que el valor inicial de x_2 sea diferente de 0.5) unos valores iniciales de $\left(\frac{d^3x_1}{dt^3}, \frac{d^3x_2}{dt^3}\right)$ que hagan que se satisfaga el sistema

$$\frac{d^3x_1}{dt^3} + \frac{d^3x_2}{dt^3} = \ddot{a}(t) \quad (3.222)$$

$$\frac{d^3x_1}{dt^3} + 6 \cdot \dot{x}_2 \cdot \ddot{x}_2 + 2 \cdot x_2 \cdot \frac{d^3x_2}{dt^3} = \frac{d^3b(t)}{dt^3} \quad (3.223)$$

o equivalentemente,

$$\frac{d^3x_1}{dt^3} + \frac{d^3x_2}{dt^3} = \ddot{a}(t) \quad (3.224)$$

$$\frac{d^3x_1}{dt^3} + 2 \cdot x_2 \cdot \frac{d^3x_2}{dt^3} = \frac{d^3b(t)}{dt^3} - 6 \cdot \dot{x}_2 \cdot \ddot{x}_2 \quad (3.225)$$

Y así sucesivamente. Por tanto, las condiciones iniciales $(x_{1_0}, x_{2_0}, \dot{x}_{1_0}, \dot{x}_{2_0})$ deben satisfacer el siguiente sistema DAE, que es equivalente al sistema original:

$$\dot{x}_1 + \dot{x}_2 = a(t) \quad (3.226)$$

$$x_1 + x_2^2 = b(t) \quad (3.227)$$

$$\dot{x}_1 + 2 \cdot x_2 \cdot \dot{x}_2 = \dot{b}(t) \quad (3.228)$$

Dado que el vector de condiciones iniciales tiene 4 componentes y estos deben satisfacer 3 ecuaciones, el sistema DAE tiene un grado de libertad. Puede escogerse una única variable como variable de estado. Seleccionando x_2 como variable de estado y asignando la causalidad computacional, se obtiene el siguiente modelo ordenado y resuelto:

$$[x_1] = b(t) - x_2^2 \quad (3.229)$$

$$[derx_2] = \frac{\dot{b}(t) - a(t)}{2 \cdot x_2 - 1} \quad (3.230)$$

$$[derx_1] = a(t) - derx_2 \quad (3.231)$$

3.5.2. El algoritmo de Pantelides

El algoritmo de Pantelides proporciona un método para analizar la estructura del sistema de ecuaciones y determinar el subconjunto mínimo de ellas cuya derivación impone ligaduras adicionales al vector de condiciones iniciales. Estas ligaduras, junto con las ecuaciones originales, deben ser satisfechas por el vector de condiciones iniciales para que la inicialización sea consistente.

El algoritmo, que hace uso de la teoría de grafos para representar la estructura del sistema, no requiere la realización de ninguna operación aritmética ni la derivación simbólica de las ecuaciones.

El algoritmo converge siempre que el sistema DAE esté bien planteado. A continuación se muestran dos ejemplos de aplicación del algoritmo. En el primer ejemplo, el algoritmo se aplica a un sistema DAE bien planteado. En el segundo ejemplo se muestra qué sucede cuando el algoritmo se aplica a un sistema DAE mal planteado. Ambos ejemplos están basados en el siguiente modelo de un sistema dinámico:

$$0 = f_1(x, u_1, u_2) \quad (3.232)$$

$$0 = f_2(x, \dot{x}, y_1) \quad (3.233)$$

$$0 = f_3(x, y_2) \quad (3.234)$$

donde u_1 , u_2 son las variables manipuladas del sistema (entradas), y_1 , y_2 son las variables observadas (salidas) y x es una variable interna, que aparece derivada en el modelo.

Se desea analizar el sistema mediante simulación para dos causalidades computacionales diferentes (véase la Figura 3.9):

- En el denominado *problema directo*, conocida la variación en el tiempo de las entradas, se desea calcular la evolución en el tiempo de las salidas.
- En el *problema inverso* son conocidas las salidas y_1 , y_2 que se desea que tenga el sistema y debe calcularse qué entradas u_1 , u_2 son precisas para obtenerlas.

En la Figura 3.9 el sentido de las flechas señala la causalidad computacional de ambos problemas: la flecha entrante al elemento indica que la variable es una entrada computacional y la flecha saliente indica que se trata de una salida computacional.

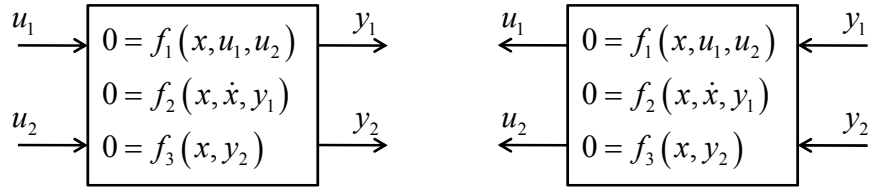


Figura 3.9: Causalidad computacional de los problemas directo (izquierda) e inverso (derecha).

Problema directo. La causalidad computacional del sistema, suponiendo que u_1 y u_2 son conocidas, y que x es una variable de estado, es la siguiente

$$x \quad \text{variable de estado} \quad (3.235)$$

$$0 = f_1(x, u_1, u_2) \quad \leftarrow \text{ecuación redundante} \quad (3.236)$$

$$0 = f_2(x, \dot{x}, y_1) \quad \leftarrow \text{incógnitas: } \dot{x}, y_1 \quad (3.237)$$

$$0 = f_3(x, [y_2]) \quad (3.238)$$

Hay una ecuación redundante y sólo se dispone de una ecuación para calcular dos incógnitas: $\dot{x}$, y_1 . Por tanto, hay dos posibilidades: el sistema DAE tiene índice superior o está mal planteado.

Analicemos si tiene índice superior, y de ser así, qué ecuaciones deben derivarse y añadirse al modelo a fin de reducir el índice. Para ello, veamos si derivando alguna de las ecuaciones se obtienen ligaduras adicionales que deban satisfacer los valores iniciales de $(x, \dot{x}, y_1, y_2)$. Derivando respecto del tiempo se obtiene:

$$0 = \frac{\partial f_1}{\partial x} \cdot \dot{x} + \frac{\partial f_1}{\partial u_1} \cdot \dot{u}_1 + \frac{\partial f_1}{\partial u_2} \cdot \dot{u}_2 \quad (3.239)$$

$$0 = \frac{\partial f_2}{\partial x} \cdot \dot{x} + \frac{\partial f_2}{\partial \dot{x}} \cdot \ddot{x} + \frac{\partial f_2}{\partial y_1} \cdot \dot{y}_1 \quad (3.240)$$

$$0 = \frac{\partial f_3}{\partial x} \cdot \dot{x} + \frac{\partial f_3}{\partial y_2} \cdot \dot{y}_2 \quad (3.241)$$

donde se observa que:

- La Ec. (3.239) constituye una nueva ligadura.
- La Ec. (3.240) introduce dos nuevas variables: $\ddot{x}$, $\dot{y}_1$.
- La Ec. (3.241) introduce una nueva variable: $\dot{y}_2$.

Derivando la Ec. (3.239) se introduce la nueva variable, $\ddot{x}$, cuyo valor inicial puede escogerse de modo que se satisfaga la ecuación para los valores iniciales de x , $\dot{x}$, y_1 , y_2 dados.

Determinados los valores iniciales de x , $\dot{x}$, $\ddot{x}$, y_1 , y_2 , puede escogerse el de $\dot{y}_1$ y de $\dot{y}_2$ de modo que se satisfagan las Ecs. (3.240) y (3.241), respectivamente.

Derivando de nuevo las ecuaciones no se obtienen más ligaduras, ya que se introducen las nuevas variables $\ddot{x}$, $\ddot{y}_1$, $\ddot{y}_2$.

La asignación de la causalidad computacional, añadiendo al sistema la derivada de la primera ecuación y considerando que x es una variable algebraica, proporciona el resultado siguiente:

$$0 = f_1([x], u_1, u_2) \quad (3.242)$$

$$0 = \frac{\partial f_1}{\partial x} \cdot [der x] + \frac{\partial f_1}{\partial u_1} \cdot \dot{u}_1 + \frac{\partial f_1}{\partial u_2} \cdot \dot{u}_2 \quad (3.243)$$

$$0 = f_2(x, der x, [y_1]) \quad (3.244)$$

$$0 = f_3(x, [y_2]) \quad (3.245)$$

Obsérvese que tanto x como su derivada son variables algebraicas, que se calculan de las ecuaciones del sistema. Es decir, el sistema DAE compuesto por las Ecs. (3.242)–(3.245) no tiene ninguna variable de estado.

Problema inverso. En este caso son conocidas las salidas y_1 , y_2 que se desea que tenga el sistema y debe calcularse qué entradas u_1 , u_2 son precisas para obtenerlas.

Sin embargo, el hecho de que las incógnitas u_1 , u_2 sólo aparezcan en la primera ecuación del sistema,

$$0 = f_1(x, u_1, u_2) \quad (3.246)$$

indica que no existe la suficiente información para el cálculo de estas incógnitas. Es decir, que el sistema no está bien planteado para el problema inverso. No es posible calcular de las ecuaciones del modelo las entradas necesarias para obtener la salida deseada.

Supóngase que se aplica el algoritmo de Pantelides a este sistema, buscando ligaduras adicionales para los valores iniciales de $(x, \dot{x}, u_1, u_2)$. Para ello, se analiza de qué variables depende cada una de las derivadas de las ecuaciones, a fin de identificar si alguna de estas ecuaciones supone una ligadura adicional.

Obsérvese que la primera ecuación del sistema,

$$0 = f_1(x, u_1, u_2) \quad (3.247)$$

depende de x, u_1, u_2 . Si se deriva una vez esta ecuación, la ecuación resultante

$$0 = \frac{\partial f_1}{\partial x} \cdot \dot{x} + \frac{\partial f_1}{\partial u_1} \cdot \dot{u}_1 + \frac{\partial f_1}{\partial u_2} \cdot \dot{u}_2 \quad (3.248)$$

en general depende de $x, u_1, u_2, \dot{x}, \dot{u}_1, \dot{u}_2$. Introduce dos incógnitas nuevas, que no aparecen en ninguna otra ecuación: $\dot{u}_1, \dot{u}_2$. Por tanto, no introduce ninguna ligadura adicional.

Cada una de las sucesivas derivadas de esta ecuación introduce dos nuevas variables que sólo aparecen en ella: las sucesivas derivadas de u_1, u_2 . Es posible, por tanto, descartar la primera ecuación a la hora de buscar nuevas ligaduras.

Analicemos ahora la segunda y tercera ecuación del sistema DAE

$$0 = f_2(x, \dot{x}, y_1) \quad (3.249)$$

$$0 = f_3(x, y_2) \quad (3.250)$$

La segunda ecuación depende de $x, \dot{x}$. Si se deriva una vez, la ecuación resultante dependerá en general de $x, \dot{x}, \ddot{x}$. Si se vuelve a derivar, la ecuación resultante dependerá de $x, \dot{x}, \ddot{x}, \dddot{x}$. Así sucesivamente.

Por otra parte, la tercera ecuación depende de x . Si se deriva, la ecuación resultante dependerá de $x, \dot{x}$. Volviendo a derivar, la ecuación resultante dependerá de $x, \dot{x}, \ddot{x}$. Así sucesivamente.

Obsérvese que la segunda ecuación, su derivada, la tercera ecuación y su derivada forman un sistema de 4 ecuaciones con 3 incógnitas: $x, \dot{x}, \ddot{x}$. Si la segunda y tercera ecuación del DAE son independientes, entonces no existe solución. Desde el punto de vista de la teoría del control, se diría que este sistema es incontrolable: no es posible calcular las entradas u_1, u_2 que hacen que el sistema tenga las salidas y_1, y_2 deseadas.

Puesto que las sucesivas derivadas de la segunda y tercera ecuación del DAE imponen ligaduras adicionales, el algoritmo de Pantelides no convergería. Es decir, continuaría indefinidamente derivando estas dos ecuaciones.

3.6. LAZOS ALGEBRAICOS

Lazo algebraico es la manera en que comúnmente se denominan las ecuaciones que dan lugar a un bloque diagonal no escalar en la matriz de incidencia BLT. Se distingue entre lazos algebraicos lineales y no lineales. En el primer caso, todas las variables desconocidas intervienen linealmente en las ecuaciones del sistema simultáneo de ecuaciones. En el segundo caso, al menos una de las variables desconocidas interviene de manera no lineal.

La aparición de lazos algebraicos en los modelos se debe típicamente a hipótesis de modelado consistentes en suponer que un determinado fenómeno, cuya dinámica es muy rápida, se produce de manera instantánea.

Supongamos, por ejemplo, un sistema con una parte hidráulica, una parte mecánica y una parte eléctrica, en el cual mediante un circuito eléctrico se controlan válvulas que limitan el flujo de líquido entre varios depósitos. El tiempo que necesita la parte eléctrica el responder a cambios en sus entradas está en el orden de los microsegundos, es decir, de la millonésima de segundo. Se dice por tanto que su constante de tiempo está en el orden de los microsegundos (desde el punto de vista matemático, ese sería el orden de magnitud de los autovalores del sistema de ecuaciones diferenciales que describe el sistema eléctrico). Por su parte, la parte mecánica tiene una constante de tiempo que está en el orden de las décimas de segundo y la parte hidráulica en el orden de las decenas de segundo. Obsérvese que las partes mecánica e hidráulica tienen constantes de tiempo que sólo difieren en dos órdenes de magnitud. Sin embargo, éstas son aproximadamente seis órdenes de magnitud más lentas que la constante de tiempo del sistema eléctrico.

En general, los sistemas que poseen dinámicas con constantes de tiempo que difieren en tres o más órdenes de magnitud se denominan **sistemas stiff**. Cuando las dinámicas del sistema difieren en seis o más órdenes de magnitud, se dice que el sistema es **fuertemente stiff**. El término inglés *stiff* se traduce en ocasiones al español como “rígido”, siendo ampliamente empleadas ambas denominaciones.

La dificultad que entraña integrar numéricamente sistemas stiff se debe a que, mientras la constante de tiempo más lenta define la escala de tiempo de la respuesta global del sistema, la constante de tiempo más rápida condiciona el tamaño máximo del paso de integración que puede usarse en los métodos explícitos de integración de modo que se mantenga la estabilidad. En el ejemplo anterior, si se emplea un método de integración explícito deberá escogerse un tamaño del paso por debajo de la millonésima de segundo. Por otra parte, si se desea observar cómo evoluciona la parte hidráulica, deberá simularse durante un tiempo del orden de las centenas

de segundo. El número total de pasos de integración de la simulación estaría por tanto en el orden de los mil millones. Por este motivo, los **métodos de integración explícitos** no son eficientes para sistemas stiff.

Los **métodos de integración implícitos** requieren que se resuelva un sistema simultáneo de ecuaciones en cada paso de integración, con lo cual su aplicación en cada instante de evaluación es más costosa computacionalmente que los métodos explícitos. Sin embargo, por ser incondicionalmente estables, los métodos implícitos permiten emplear tamaños mayores del paso de integración por lo cual resultan más adecuados que los explícitos para integrar sistemas stiff.

Dado que la integración numérica de sistemas stiff es costosa computacionalmente, en ocasiones es preferible despreciar las dinámicas muy rápidas del modelo para evitar que el sistema sea stiff, aun al coste del establecimiento de lazos algebraicos. Por el contrario, en otras ocasiones puede ser preferible modelar la dinámica en lugar de considerarla instantánea. Por ejemplo, cuando la existencia de discontinuidades haga que en determinados instantes de la simulación no se disponga de valores iniciales adecuados para la iteración del lazo. Obsérvese que puede obtenerse una solución a la ecuación algebraica escalar

$$0 = f(x) \quad (3.251)$$

resolviendo la ecuación diferencial

$$\varepsilon \cdot \frac{dx}{dt} = f(x) \quad (3.252)$$

donde ε toma un valor pequeño y tiene el signo que hace que la ecuación tenga una solución estable.

En esta sección se explicarán conceptos básicos importantes, relativos a la resolución numérica de los lazos algebraicos, que deben ser conocidos por los desarrolladores de modelos. No se entrará en la descripción detallada de los algoritmos simbólicos o numéricos para la resolución de los lazos.

3.6.1. Manipulación simbólica de los lazos algebraicos

Los lazos algebraicos en los que todas las incógnitas aparecen de forma lineal pueden resolverse mediante manipulación simbólica de forma automática. Sin embargo, cuando el número de ecuaciones del lazo es grande, puede resultar más eficiente

en términos de tiempo de ejecución resolverlo de forma numérica que mediante la evaluación de las expresiones obtenidas de la manipulación simbólica. La herramienta de modelado puede permitir que sea el usuario quien decida, para cada problema en concreto, qué método ha de usarse para resolver el lazo.

Los lazos algebraicos (bloques diagonales no escalares de la matriz de incidencia BLT) o las ecuaciones (bloques diagonales escalares) en las cuales las incógnitas no aparecen de forma lineal, deben resolverse empleando métodos numéricos. En algunos casos sencillos sería posible programar reglas de transformación para manipular ecuaciones donde la incógnita a despejar aparece de forma no lineal. Sin embargo, el problema es que la mayoría de las ecuaciones no lineales no proporcionan soluciones únicas. Por ejemplo, si debe despejarse la variable y de la ecuación:

$$x^2 + y^2 = 1 \quad (3.253)$$

se obtienen dos posibles relaciones matemáticas:

$$\begin{cases} y = \sqrt{1 - x^2} \\ y = -\sqrt{1 - x^2} \end{cases} \quad (3.254)$$

y el manipulador simbólico no sabría cuál de las dos ecuaciones emplear para evaluar y . Cuando se resuelve numéricamente la Ec. (3.253), dependiendo del algoritmo empleado, del valor inicial escogido para la iteración y del modo en que se formule el problema, el método puede **converger** a una solución, a la otra o puede no ser capaz de encontrar ninguna de las soluciones.

3.6.2. Solución de los lazos en la inicialización

Como hemos visto en la Sección 3.5, en el instante inicial de la simulación deben calcularse los valores iniciales de todas las variables desconocidas del modelo. Si para ello hay que resolver lazos algebraicos, es fundamental que se proporcionen valores iniciales para la iteración lo más próximos posible a la solución, para que el algoritmo converja rápidamente a la solución apropiada.

En los modelos en Modelica, sólo las variables reales de tiempo continuo pueden ser evaluadas resolviendo lazos algebraicos. Si una variable de tipo *Real* debe ser evaluada de un lazo algebraico y al declararla se le asigna el valor *false* a su atributo *fixed*, entonces el valor asignado a su atributo *start* se empleará como valor inicial

para la variable al iterar para la resolución del lazo algebraico en el instante inicial de la simulación.

Una vez resuelto el sistema en el instante inicial de la simulación, se toma la solución obtenida en cada instante de simulación como valor inicial de la iteración para el cálculo de la solución en el siguiente instante de simulación.

La solución del sistema en el instante inicial de la simulación resulta, en ocasiones, problemática. Es por ello que algunas herramientas de simulación de propósito específico (por ejemplo, SPICE) emplean técnicas especiales para intentar resolver este problema.

SPICE es una herramienta muy potente para la simulación de circuitos eléctricos. La determinación del punto inicial de operación es uno de los grandes problemas del análisis de circuitos, ya que no existen garantías de que los algoritmos numéricos, que resuelven los lazos, converjan si no se escogen adecuadamente los valores iniciales para la iteración.

El método empleado por SPICE consiste en simular primero un experimento en el cual todas las fuentes independientes de corriente y de tensión están inicialmente a cero y todos los dispositivos activos están en corte. Con esta configuración el punto de operación es trivial: la tensión es cero en todos los nodos. A continuación, se realiza el análisis transitorio, en el cual se aplica una rampa a todas las fuentes. El valor final de cada rampa es el valor inicial de la simulación real, el cual se mantiene constante durante algún tiempo para permitir que el circuito alcance el estado estacionario. El valor final de los voltajes de los nodos se guarda como valor inicial para resolver el problema real.

La filosofía del método es de carácter general. Si el cálculo de la solución del sistema en el estado inicial es problemático, pero hay algún estado del sistema donde sea conocido, entonces el problema puede resolverse mediante simulación. Se simula la evolución del sistema desde el estado conocido hasta el estado inicial del problema de interés y se deja que el sistema alcance aproximadamente el estacionario, obteniéndose así unas condiciones iniciales para la iteración del lazo próximas a la solución del sistema para el estado inicial. A continuación, se resuelve el problema de inicialización, empleando como valores iniciales para la iteración los valores obtenidos mediante simulación, y se realiza la simulación deseada.

3.6.3. Tearing de los lazos algebraicos no lineales

El *tearing* (traducido a veces por *rasgadura*) es una técnica aplicable a lazos algebraicos no lineales tales que alguna de las variables desconocidas aparezca en alguna ecuación de forma lineal. El objeto de la técnica es, sacando partido del hecho anterior, reducir el número de variables a iterar para la resolución numérica del lazo.

Considérese un sistema algebraico de ecuaciones

$$\mathbf{G}(\mathbf{y}) = 0 \quad (3.255)$$

Realizar el tearing de este sistema consiste en escribirlo de la forma

$$\begin{cases} \mathbf{G}_1(\mathbf{y}_1, \mathbf{y}_2) = 0 \\ \mathbf{G}_2(\mathbf{y}_1, \mathbf{y}_2) = 0 \end{cases} \quad (3.256)$$

de modo que se satisfaga que:

1. La matriz Jacobiana $\frac{\partial \mathbf{G}_1}{\partial \mathbf{y}_1}$ sea triangular inferior o triangular inferior por bloques, donde todos los bloques diagonales, escalares o no escalares, deben representar ecuaciones o sistemas de ecuaciones lineales respecto de las incógnitas que van a ser evaluadas de ellas.
2. La dimensión de $\mathbf{y}_2$ sea lo menor posible.

Una vez escrito el lazo de esta forma, las ecuaciones se resuelven iterando sobre $\mathbf{y}_2$. Conocido un valor inicial de $\mathbf{y}_2$, puede obtenerse $\mathbf{y}_1$ de resolver $\mathbf{G}_1(\mathbf{y}_1, \mathbf{y}_2) = 0$ empleando las técnicas de resolución de ecuaciones lineales. Con el valor de $\mathbf{y}_1$ calculado, se obtiene un nuevo valor de $\mathbf{y}_2$ de $\mathbf{G}_2(\mathbf{y}_1, \mathbf{y}_2) = 0$ y así sucesivamente.

Las incógnitas $\mathbf{y}_2$ reciben el nombre de **variables de tearing**, ya que su selección supone “rasgar” el lazo algebraico y comenzar la resolución del mismo a partir de los valores iniciales de aquellas. Las ecuaciones $\mathbf{G}_2(\mathbf{y}_1, \mathbf{y}_2) = 0$ que se emplean para evaluar las variables de tearing reciben el nombre de **ecuaciones residuo**.

Algunos entornos de modelado realizan una forma más restrictiva de tearing, en la cual la matriz Jacobiana $\frac{\partial \mathbf{G}_1}{\partial \mathbf{y}_1}$ debe ser triangular inferior y todos sus elementos diagonales deben representar ecuaciones lineales respecto a su incógnita. Es decir, conocido $\mathbf{y}_2$ pueden calcularse secuencialmente las incógnitas $\mathbf{y}_1$ de las ecuaciones $\mathbf{G}_1(\mathbf{y}_1, \mathbf{y}_2) = 0$, donde cada incógnita aparece linealmente en la ecuación que va a usarse para evaluarla.

Es común que en un lazo haya más de una posible elección de ecuaciones residuo y de variables de tearing. En este caso, conviene escoger como variables de tearing aquellas para las cuales se pueda especificar el valor inicial con mayor precisión. La elección de las ecuaciones residuo debe basarse en su buen comportamiento, a fin de facilitar la convergencia numérica.

Los algoritmos exhaustivos de búsqueda para determinar la estructura óptima de tearing son de complejidad exponencial. Por ello, los algoritmos para la selección de las variables de tearing y de las ecuaciones residuo que proporcionan los entornos de modelado suelen estar basados en reglas heurísticas.

3.7. SELECCIÓN DE LAS VARIABLES DE ESTADO

En general, la selección de las variables de estado de un sistema no es única. Es decir, es posible escoger diferentes conjuntos de variables como variables de estado. Esto es equivalente a decir que es posible expresar el sistema de manera que en él aparezcan derivados diferentes conjuntos de variables.

La adecuada selección de las variables de estado puede, en algunos modelos, incrementar la precisión y reducir la carga computacional de la simulación. Las variables de estado pueden ser seleccionadas por el desarrollador del modelo y por el entorno de modelado. En este último caso, el entorno de modelado puede variar en tiempo de simulación la selección de las variables de estado, lo cual se denomina **selección dinámica de los estados**.

La sección se ha estructurado de la forma siguiente. En la Sección 3.7.1, se explica una técnica de manipulación simbólica para la selección de las variables de estado.

En la Sección 3.7.2, se aborda la selección dinámica de los estados. Se muestra un ejemplo en el cual no existe una selección de las variables de estado válida para toda la trayectoria, con lo cual el modelo sólo puede ser simulado si el entorno de modelado varía la selección de las variables de estado en tiempo de simulación.

Finalmente, en la Sección 3.7.3 se describirá cómo el desarrollador del modelo puede seleccionar en Modelica qué variables del modelo deben ser usadas como variables de estado.

3.7.1. Manipulación del sistema DAE

En esta sección se explicará una técnica para manipular el sistema DAE,

$$\mathbf{F}(\mathbf{x}, \dot{\mathbf{x}}, \mathbf{y}, t) = 0 \quad (3.257)$$

de modo que en él aparezcan derivadas únicamente aquellas variables correspondientes a una determinada selección de las variables de estado, que representaremos $(\mathbf{x}_1, \mathbf{y}_1)$. Es decir, el sistema DAE obtenido deberá estar expresado de la forma

$$\mathbf{G}(\mathbf{x}_1, \mathbf{y}_1, \dot{\mathbf{x}}_1, \dot{\mathbf{y}}_1, \mathbf{x}_2, \mathbf{y}_2, t) = 0 \quad (3.258)$$

donde $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2)$, $\mathbf{y} = (\mathbf{y}_1, \mathbf{y}_2)$.

La técnica consiste en los pasos siguientes:

1. Se añaden al modelo original tantas ecuaciones mudas como variables algebraicas se desee que sean variables de estado. En cada una de estas ecuaciones mudas debe aparecer la derivada de la variable igualada a una variable muda. El conjunto de ecuaciones mudas puede ser de la forma

$$\dot{\mathbf{y}}_1 = \mathbf{a} \quad (3.259)$$

donde $\mathbf{a}$ es el vector de variables mudas. Los vectores de variables $\mathbf{y}_1$ y $\mathbf{a}$ tienen el mismo número de componentes.

Obsérvese que estas ecuaciones mudas no modifican la solución al sistema. Puesto que las variables mudas sólo intervienen en las ecuaciones mudas, estas ecuaciones deberán emplearse para evaluar las variables mudas.

2. El sistema aumentado con las ecuaciones mudas

$$\mathbf{F}(\mathbf{x}, \dot{\mathbf{x}}, \mathbf{y}, t) = 0 \quad (3.260)$$

$$\dot{\mathbf{y}}_1 = \mathbf{a} \quad (3.261)$$

tiene índice superior. Así pues, el siguiente paso es reducir el índice de este sistema DAE aumentado, seleccionando como variables de estado $(\mathbf{x}_1, \mathbf{y}_1)$.

Veamos un ejemplo de aplicación de esta técnica. Las ecuaciones que usaremos para ello son las que típicamente se emplean al plantear el balance de energía en un volumen de control. Esto nos permitirá introducir en este punto un concepto que

surge frecuentemente como resultado de las hipótesis de modelado: el concepto de volumen de control.

Un **volumen de control** es una región en la cual las propiedades son uniformes, esto es, independientes de la posición espacial, pero dependientes en general del tiempo. La interfaz entre el volumen de control y su entorno está compuesta por **planos de control**, que son aquellas partes de la superficie que engloba al volumen de control a través de las cuales se produce intercambio de materia, momento y energía. Normalmente se supone que la materia que sale del volumen de control tiene las mismas propiedades (composición, temperatura, etc.) que la materia contenida dentro del volumen de control. Esta hipótesis se conoce comúnmente como **aproximación de mezcla agitada**.

- La dimensión del volumen de control depende de las aproximaciones de modelado: mientras en el **modelado microscópico** se utilizan volúmenes de control de dimensiones infinitesimales, denominados **elementos de volumen**, en el **modelado macroscópico** se emplean volúmenes de control de dimensiones finitas.
- La forma del elemento de control se escoge dependiendo de la simetría del problema.
- El volumen del elemento de control puede considerarse constante o variable, dependiendo de las aproximaciones de modelado.
- Puede considerarse fijo en el espacio, obteniéndose entonces los **balances en forma conservativa**, o en movimiento junto con el fluido, de modo que siempre contenga las mismas partículas de éste (adaptando su forma y volumen para ello), con lo que se obtienen los **balances en forma no conservativa**.
- En aquellas aplicaciones en que debe modelarse la dependencia de las propiedades del medio con las coordenadas espaciales, puede dividirse el medio en volúmenes de control de dimensiones finitas, considerándose uniformes las propiedades del medio interior a cada uno de ellos. Asimismo, se describe el flujo entre elementos de control adyacentes, a través de los planos de control, en función de la diferencia en el valor de las propiedades en los planos de control.

Volviendo al ejemplo que nos ocupa, supongamos que se desea modelar un depósito que almacena un líquido o un gas. Para ello, se realiza la hipótesis de considerar que el líquido o el gas almacenado en su interior se encuentra perfectamente agitado.

Puede entonces definirse un volumen de control que coincida con la superficie interior del deposito y que contenga todo el líquido o gas que se encuentra dentro del mismo. Este volumen de control posee planos de control, a través de los cuales se produce flujo de materia (por ejemplo, el transportado por las tuberías que conectan el depósito con el resto de la planta) y flujo de calor (por ejemplo, con una camisa calefactada que rodee al depósito). Al plantear el balance de energía en el volumen de control, puede obtenerse un modelo que formalmente es similar al mostrado a continuación,

$$E = f(T) \quad (3.262)$$

$$\frac{dE}{dt} = g(T, Q) \quad (3.263)$$

$$Q = h(T) \quad (3.264)$$

donde E representa la energía (típicamente energía interna o entalpía) del gas o líquido contenido dentro del volumen de control, T es la temperatura a la que se encuentra el gas o líquido, y Q es el flujo de calor intercambiado con el exterior. La función f , que relaciona la temperatura del gas o líquido con su energía, es en general no lineal, ya que comúnmente se considera que la capacidad calorífica depende de manera polinómica de la temperatura.

Tal como está planteado el modelo, la energía (E) aparece derivada. Si se selecciona la energía como variable de estado, la causalidad del modelo será la siguiente:

$$E = f([T]) \quad (3.265)$$

$$\left[\frac{dE}{dt} \right] = g(T, Q) \quad (3.266)$$

$$[Q] = h(T) \quad (3.267)$$

Obsérvese que la temperatura debe calcularse resolviendo la primera ecuación, que es no lineal, lo cual puede ser costoso computacionalmente. Otra opción sería seleccionar como variable de estado la temperatura, en lugar de la energía. Si se procede de ese modo, la primera ecuación se empleará para evaluar la energía, lo cual es sencillo computacionalmente, ya que la energía aparece despejada.

Para seleccionar la temperatura como variable de estado, puede incluirse una ecuación muda en el modelo, en la cual intervenga la derivada de la temperatura, y

de la cual se evalúe una variable muda. A continuación se muestra el modelo con la ecuación muda añadida: la Ec. (3.271).

$$E = f(T) \quad (3.268)$$

$$\frac{dE}{dt} = g(T, Q) \quad (3.269)$$

$$Q = h(T) \quad (3.270)$$

$$\frac{dT}{dt} = a \quad (3.271)$$

Puesto que la variable muda a sólo interviene en la ecuación muda que acabamos de añadir al modelo, esta variable deberá evaluarse de la ecuación muda. La inclusión de esta ecuación, por tanto, no modifica el valor de las variables E , T , Q .

El modelo así planteado tiene índice superior: en él aparecen derivadas dos variables (E , T) y sólo tiene un grado de libertad. Sustituyamos las derivadas por variables mudas

$$\frac{dE}{dt} \rightarrow derE \quad \frac{dT}{dt} \rightarrow derT \quad (3.272)$$

y reduzcamos el índice del sistema DAE. Para ello, se deriva respecto al tiempo la Ec. (3.268) y se añade al sistema. Se obtiene:

$$E = f(T) \quad (3.273)$$

$$derE = g(T, Q) \quad (3.274)$$

$$Q = h(T) \quad (3.275)$$

$$derT = a \quad (3.276)$$

$$derE = \frac{df(T)}{dT} \cdot derT \quad (3.277)$$

Seleccionando la temperatura como variable de estado, asignando la causalidad computacional y ordenando las ecuaciones del modelo, se obtiene:

$$T \quad \text{variable de estado} \quad (3.278)$$

$$[E] = f(T) \quad (3.279)$$

$$[Q] = h(T) \quad (3.280)$$

$$[der E] = g(T, Q) \quad (3.281)$$

$$der E = \frac{df(T)}{dT} \cdot [der T] \quad (3.282)$$

$$der T = [a] \quad (3.283)$$

La ecuación muda ya ha cumplido con su propósito: forzar que el modelo tenga índice superior, permitiendo la aplicación del algoritmo para la reducción del índice. Así pues, la Ec. (3.283) ya puede ser eliminada del modelo. El modelo resultante está compuesto por las Ecs. (3.279) – (3.282), con T seleccionada como variable de estado.

3.7.2. Selección dinámica por el entorno de modelado

Para ilustrar cómo en determinados modelos no es posible escoger un conjunto de variables de estado que sea válido para toda la trayectoria simulada, consideremos el siguiente modelo de un péndulo plano expresado en coordenadas cartesianas (véase la Figura 3.10). Se han etiquetado las ecuaciones para facilitar hacer referencia a las mismas.

$$m \cdot \dot{v}_x = -\frac{x}{L} \cdot F \quad (a)$$

$$m \cdot \dot{v}_y = -\frac{y}{L} \cdot F - m \cdot g \quad (b)$$

$$x^2 + y^2 = L^2 \quad (c)$$

$$\dot{x} = v_x \quad (d)$$

$$\dot{y} = v_y \quad (e)$$

En este modelo aparecen cuatro variables derivadas (x , y , v_x , v_y) y tiene únicamente dos grados de libertad. Tiene índice 3. Para reducir el índice se deriva la Ec. (c) dos veces, y las Ecs. (d) y (e) una vez. Añadiendo las derivadas al modelo y eliminando las variables alias se obtiene:

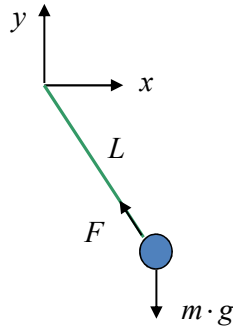


Figura 3.10: Diagrama del modelo de un péndulo plano. Se muestran los ejes cartesianos x - y , y las fuerzas que actúan sobre la masa del péndulo: la fuerza gravitatoria ($m \cdot g$) y la tensión ejercida por la cuerda del péndulo (F). La longitud de la cuerda es L .

$$m \cdot \dot{v}_x = -\frac{x}{L} \cdot F \quad (a)$$

$$m \cdot \dot{v}_y = -\frac{y}{L} \cdot F - m \cdot g \quad (b)$$

$$x^2 + y^2 = L^2 \quad (c)$$

$$\dot{x} = v_x \quad (d)$$

$$\dot{y} = v_y \quad (e)$$

$$x \cdot \dot{x} + y \cdot \dot{y} = 0 \quad (c')$$

$$x \cdot \dot{v}_x + \dot{x}^2 + y \cdot \dot{v}_y + \dot{y}^2 = 0 \quad (c'', d', e')$$

Consideremos las siguientes cuatro selecciones de las variables de estado:

$$\{x, v_x\}$$

$$\{x, v_y\}$$

$$\{y, v_x\}$$

$$\{y, v_y\}$$

La causalidad computacional es diferente para cada una de estas cuatro selecciones de las variables de estado. En los cuatro casos, las variables $\{F, \text{der}v_x, \text{der}v_y\}$ se evaluarían de las Ecs. (a), (b) y (c'', d', e'), una vez conocidas las restantes incógnitas. Puesto que esto es común a los cuatro casos, no se muestra. La causalidad computacional para evaluar las incógnitas $\{y, v_y, \text{der}x, \text{der}y\}$ es la siguiente:

1. Variables de estado: $\{x, v_x\}$

$$[\text{der}x] = v_x \quad (d)$$

$$x^2 + [y]^2 = L^2 \quad (c)$$

$$x \cdot \text{der}x + y \cdot [\text{der}y] = 0 \quad (c')$$

$$\text{der}y = [v_y] \quad (e)$$

Obsérvese que cuando y valga cero se producirá división por cero en la Ec. (c'), con lo cual la simulación abortará.

2. Variables de estado: $\{y, v_x\}$

$$[derx] = v_x \quad (d)$$

$$[x]^2 + y^2 = L^2 \quad (c)$$

$$x \cdot derx + y \cdot [der y] = 0 \quad (c')$$

$$der y = [v_y] \quad (e)$$

Con esta selección de las variables de estado también habrá división por cero en la Ec. (c') cuando y valga cero.

3. Variables de estado: $\{x, v_y\}$

$$[der y] = v_y \quad (e)$$

$$x^2 + [y]^2 = L^2 \quad (c)$$

$$x \cdot [der x] + y \cdot der y = 0 \quad (c')$$

$$der x = [v_x] \quad (d)$$

En este caso, se producirá error por división por cero en la Ec. (c') cuando x valga cero.

4. Variables de estado: $\{y, v_y\}$

$$[der y] = v_y \quad (e)$$

$$[x]^2 + y^2 = L^2 \quad (c)$$

$$x \cdot [der x] + y \cdot der y = 0 \quad (c')$$

$$der x = [v_x] \quad (d)$$

Nuevamente, cuando x valga cero se producirá división por cero en la Ec. (c').

Así pues, para cualquiera de las cuatro selecciones de las variables de estado puede producirse división por cero en la Ec. (c'). El significado físico de esta ecuación es que los vectores posición y velocidad son perpendiculares:

$$x \cdot derx + y \cdot der y = 0 \quad \Leftrightarrow \quad \mathbf{r} \cdot \mathbf{v} = 0 \quad \Leftrightarrow \quad \mathbf{r} \perp \mathbf{v}$$

En la Figura 3.11 se muestra la causalidad computacional que debe tener la Ec. (c') cuando el péndulo se encuentra en la posición horizontal y vertical.

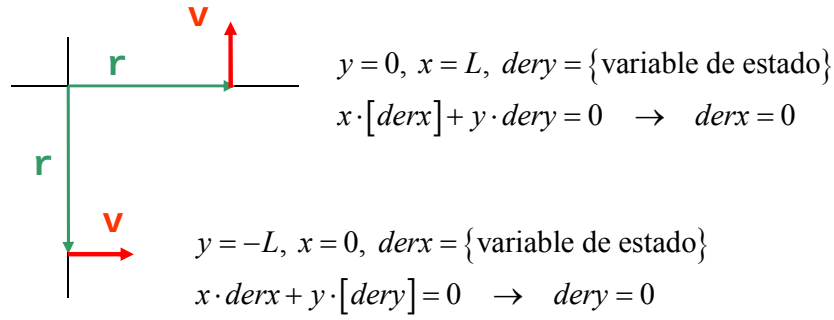


Figura 3.11: Péndulo en posición vertical y horizontal.

1. Cuando el péndulo está en la posición vertical, la condición de perpendicularidad entre los vectores posición y velocidad impone que la componente vertical de la velocidad sea cero. No impone ninguna condición sobre la componente horizontal de la velocidad, por ello no tiene sentido tratar de calcularla de la Ec. (c').
2. Cuando el péndulo está en la posición horizontal, la condición de perpendicularidad entre los vectores posición y velocidad impone que la componente horizontal de la velocidad sea cero. No impone ninguna condición sobre la componente vertical de la velocidad. Por ello, en esta posición no tiene sentido tratar de calcular la coordenada vertical de la velocidad de la Ec. (c').

Una solución evidente a este problema es emplear coordenadas polares, en lugar de rectangulares. Sin embargo, este ejemplo muestra la necesidad de realizar una asignación dinámica de los estados en tiempo de simulación.

El entorno de modelado Dymola realiza la selección dinámica de los estados. Para mostrarlo, traduzcamos y simulemos el modelo del péndulo descrito anteriormente. La descripción en Modelica se muestra en el Código 3.2. Obsérvese que se asigna valor inicial a las variables x y vy al declararlas:

```
Real x (unit="m", start=0.9, fixed=true) "Posición";
Real vy(unit="m/s", start=0, fixed=true) "Velocidad";
```

Los valores iniciales son: $x_0 = 0.9$ y $vy_0 = 0$. El valor inicial de y se calcula resolviendo numéricamente la Ec. (c). Hay dos posibles soluciones:

$$y_0 = \pm \sqrt{L^2 - x_0^2} = \pm \sqrt{1^2 - 0.9^2} = \pm 0.43589 \quad (3.284)$$

Dependiendo de la condición inicial que se de al método iterativo empleado para resolver la Ec. (c) en el instante inicial de la simulación, el método convergerá a la

```

model penduloSelecDinVE
  constant Real g(unit="m/s2") = 9.81 "Aceleración gravitatoria";
  parameter Real L(unit="m") = 1 "Longitud de la cuerda";
  parameter Real m(unit="kg") = 5 "Masa";
  Real x(unit="m", start=0.9, fixed=true) "Posición";
  Real y(unit="m", start=0.5, fixed=false) "Posición";
  Real vx(unit="m/s") "Velocidad";
  Real vy(unit="m/s", start=0, fixed=true) "Velocidad";
  Real F(unit="N") "Fuerza de la cuerda";
equation
  m*der(vx) = -x/L*F;
  m*der(vy) = -y/L*F - m*g;
  x^2 + y^2 = L^2;
  der(x) = vx;
  der(y) = vy;
end penduloSelecDinVE;

```

Código 3.2: Péndulo plano en coordenadas rectangulares.

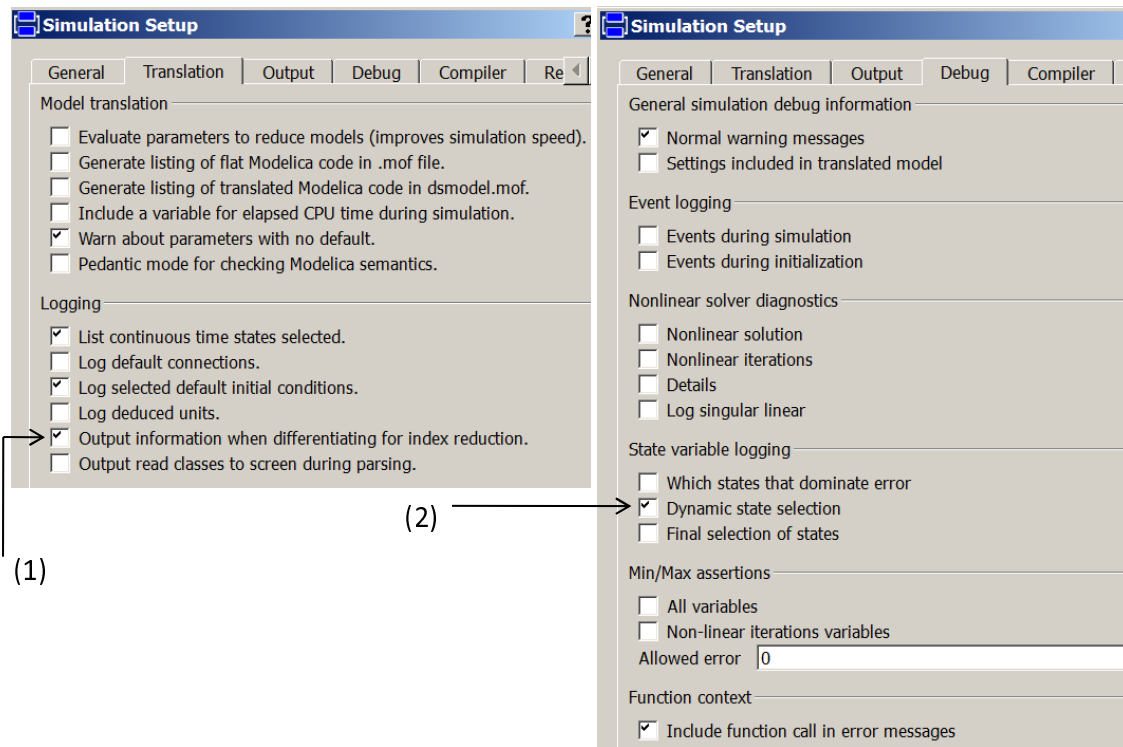


Figura 3.12: Ventana *Simulation Setup* de Dymola versión 2015. (1): muestra las ecuaciones derivadas simbólicamente respecto al tiempo y añadidas al modelo a fin de reducir su índice; y (2): muestra la selección dinámica de los estados realizada automáticamente en tiempo de simulación.

solución con signo positivo o a la solución con signo negativo. La declaración de y es:

```
Real y(unit="m", start=0.5, fixed=false) "Posición";
```

Al asignar a *fixed* el valor *false*, se está indicando que el valor asignado a *start* no debe ser impuesto como valor inicial de la variable, sino usado como valor inicial para el método iterativo empleado para evaluar la variable en el instante inicial. Empleando como valor inicial para la iteración el valor 0.5, el método iterativo converge a la solución con signo positivo. Es decir, a $y_0 = 0.43589$.

En la Figura 3.12 se indica cómo debe configurarse la ventana *Simulation Setup* antes de traducir el modelo con el fin de que Dymola muestre: (1) qué variables ha derivado simbólicamente y añadido al modelo, previamente a asignar la causalidad computacional, a fin de reducir el índice; y (2) cómo selecciona dinámicamente las variables de estado en tiempo de simulación.

Al traducir el modelo, Dymola muestra en la ventana de mensajes las ecuaciones que ha derivado respecto al tiempo y añadido al modelo. El texto mostrado es el siguiente:

```
Differentiated the equation
x^2+y^2 = L^2;

giving
2.0*(x*der(x)+y*der(y)) = 0.0;

Differentiated the equation
2.0*(x*vx+y*vy) = 0.0;

giving
2.0*(der(x)*vx+x*der(vx)+der(y)*vy+y*der(vy)) = 0.0;
```

Asimismo, Dymola muestra el mensaje mostrado a continuación, donde indica que seleccionará una de las variables de estado del conjunto $\{x, y\}$ y la otra del conjunto $\{vx, vy\}$.

```
Selected continuous time states
Dynamically selected continuous time states
There are 2 sets of dynamic state selection.
From set 1 there is 1 state to be selected from:
x
y

From set 2 there is 1 state to be selected from:
vx
vy
```

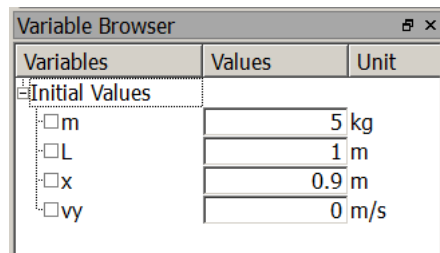


Figura 3.13: Ventana *Variable Browser* tras traducir el modelo del péndulo plano mostrado en el Código 3.2.

Una vez finalizada la traducción del modelo, en la ventana *Variable Browser* se muestran los parámetros y las variables a las que se ha asignado valor inicial, pudiéndose modificar en este punto sus valores (véase la Figura 3.13).

Simulando durante 3 s, se obtiene en la ventana de mensajes el siguiente informe acerca de la selección de las variables de estado.

```

Selected at 0:
  y.stateSelect=StateSelect.always
Selected at 0:
  vy.stateSelect=StateSelect.always

Integration started at T = 0 using integration method DASSL
(DAE multi-step solver (dassl/dasslrt of Petzold modified by Dynasim))

Selected at 0.534:
  x.stateSelect=StateSelect.always
Selected at 0.534:
  vx.stateSelect=StateSelect.always

Selected at 0.84:
  y.stateSelect=StateSelect.always
Selected at 0.84:
  vy.stateSelect=StateSelect.always

Selected at 1.872:
  x.stateSelect=StateSelect.always
Selected at 1.872:
  vx.stateSelect=StateSelect.always

Selected at 2.184:
  y.stateSelect=StateSelect.always
Selected at 2.184:
  vy.stateSelect=StateSelect.always

Integration terminated successfully at T = 3

```

Desde el instante inicial hasta el instante 0.534 s, las variables de estado han sido $\{y, vy\}$; desde el instante 0.534 s hasta el instante 0.84 s, las variables de estado han

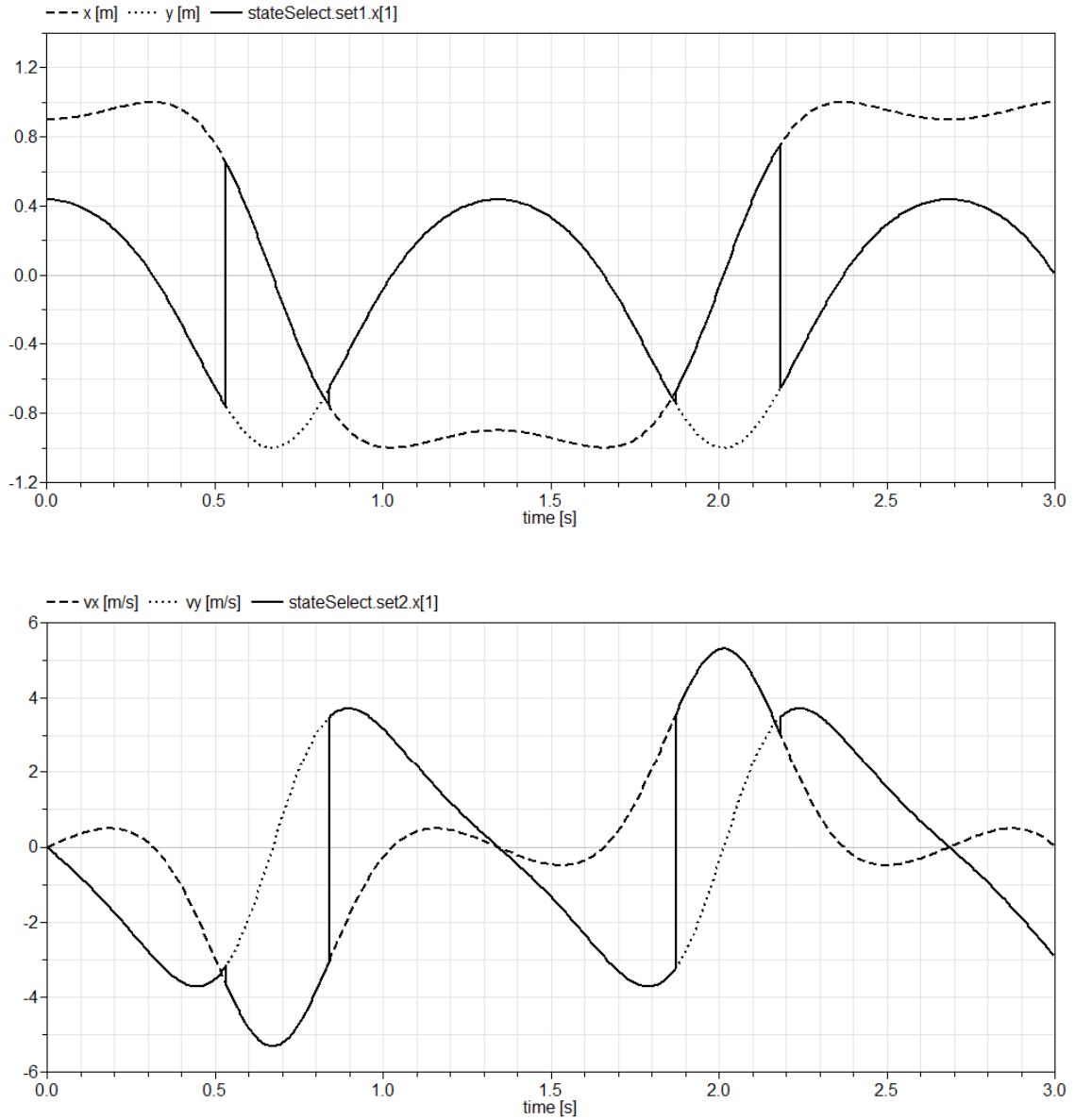


Figura 3.14: Simulación del péndulo plano en coordenadas cartesianas, con selección dinámica de las variables de estado. Dymola define dos conjuntos de variables: $set1 = \{x, y\}$, $set2 = \{v_x, v_y\}$. El modelo tiene dos variables de estado, a las que Dymola denomina $\{stateSelect.set1.x[1], stateSelect.set2.x[1]\}$. La variable $stateSelect.set1.x[1]$ se selecciona entre las variables del conjunto $set1$ y la variable $stateSelect.set2.x[1]$ entre las del conjunto $set2$. En parte de la trayectoria $\{stateSelect.set1.x[1], stateSelect.set2.x[1]\}$ es igual a $\{x, v_x\}$ y en el resto de la trayectoria es igual a $\{y, v_y\}$. La selección de las variables de estado es realizada durante la simulación de manera automática por Dymola.

sido $\{x, vx\}$; etc. Se observa que Dymola va alternando a lo largo de la simulación entre dos selecciones de las variables de estado: $\{y, vy\}$ y $\{x, vx\}$. Emplea la primera cuando la trayectoria se encuentra en las proximidades del eje horizontal y la segunda cuando se encuentra en las proximidades del eje vertical.

Dymola denomina *stateSelect.set1.x[1]* a la variable de estado que escoge del conjunto $\text{set1} = \{x, y\}$ y denomina *stateSelect.set2.x[1]* a la variable de estado que escoge del conjunto $\text{set2} = \{vx, vy\}$. En la Figura 3.14 se muestra la evolución de las dos variables de estado, *stateSelect.set1.x[1]* y *stateSelect.set2.x[1]*. Puede observarse como *stateSelect.set1.x[1]* es igual a x en parte de la trayectoria e igual a y en el resto, y como *stateSelect.set2.x[1]* es igual a vx en parte de la trayectoria e igual a vy en el resto.

3.7.3. Selección por el desarrollador del modelo

Existen varias razones por las cuales el desarrollador de un modelo puede querer especificar qué variables del modelo deben ser usadas como variables de estado en la simulación. Algunas de estas razones son:

- Mejorar la precisión de la solución numérica del modelo.
- Evitar la inversión de funciones.
- Reducir el número de ecuaciones no lineales.
- Cuando en un evento se modifica el valor de una variable de tiempo continuo, esta variable debe ser una variable de estado (esto se explicará con detalle en el siguiente tema).

El recurso que proporciona Modelica para seleccionar las variables de estado es el **atributo stateSelect** de las variables de tipo *Real*. En la Tabla 3.1 se indican los posibles valores del atributo. Por ejemplo:

```
Real w ( stateSelect = StateSelect.prefer );
```

Para distinguirla de la **selección dinámica** explicada en la Sección 3.7.2, que es realizada automáticamente por el entorno de modelado en tiempo de simulación, a la selección realizada por el desarrollador del modelo empleando el atributo *stateSelect* se la denomina **selección estática** de los estados. El motivo por el cual se denomina

Tabla 3.1: Posibles valores del atributo *stateSelect* de las variables de tipo *Real*.

Valor	Significado
<code>StateSelect.never</code>	Nunca será seleccionada como estado.
<code>StateSelect.avoid</code>	Se selecciona como estado si aparece derivada respecto al tiempo y si además no existe ninguna otra selección posible de variables con <code>default</code> , <code>prefer</code> o <code>always</code> .
<code>StateSelect.default</code>	Los estados son seleccionados automáticamente entre las variables que aparecen derivadas respecto al tiempo. Significa nunca para las variables que no aparecen derivadas.
<code>StateSelect.prefer</code>	No necesita aparecer derivada respecto al tiempo para ser escogida como variable de estado, y tiene preferencia frente a las variables cuyo atributo tiene el valor <code>default</code> .
<code>StateSelect.always</code>	La variable siempre será seleccionada como estado.

estática es que el usuario realiza la selección al definir el modelo y esta selección no puede ser modificada en tiempo de simulación.

Un punto importante es que la selección de los estados es independiente de la especificación de las condiciones iniciales. El atributo *fixed* no tiene influencia en la selección de las variables de estado. Por ejemplo, el modelo del péndulo plano descrito en coordenadas cartesianas se ha inicializado asignando valor a las variables $\{x, vy\}$ (véase el Código 3.2 y la Figura 3.13) y la selección dinámica de las variables de estado en el instante inicial de la simulación ha sido $\{y, vy\}$.

Recíprocamente, Modelica no impone que deba asignarse valor inicial a las variables de estado. Pueden asignarse valores iniciales a un conjunto cualquiera de variables algebraicas, variables de estado y parámetros, siempre que con eso el problema inicial quede bien definido. Volveremos sobre esto más adelante.

3.8. SOLUCIÓN NUMÉRICA DE SISTEMAS DAE

En esta sección se introducen algunos conceptos básicos del algoritmo de integración para sistemas DAE llamado **DASSL**, que es reconocido como uno de los más robustos existentes y es ampliamente usado. También se introducen dos técnicas de integración denominadas **integración inline** e **integración mixed-mode**, que pueden mejorar la eficiencia de la simulación sacando partido del conocimiento de

la estructura computacional del modelo y del orden de magnitud de las constantes de tiempo de los estados.

3.8.1. DASSL

El algoritmo para la solución numérica de DAE denominado DASSL emplea el método BDF (Backward Difference Formulae) de discretización. El orden de la aproximación y el tamaño del paso de integración son escogidos automáticamente en cada paso, en función del comportamiento de la solución. DASSL emplea el método de iteración de Newton para resolver el sistema no lineal de ecuaciones algebraicas obtenido.

Considérese la siguiente representación general de un sistema DAE, donde $\mathbf{y}(t)$ es un vector de variables desconocidas,

$$\mathbf{f}(\dot{\mathbf{y}}, \mathbf{y}, t) = 0 \quad (3.285)$$

y considérese el siguiente esquema de discretización, que es usado por gran cantidad de algoritmos de integración implícitos (por ejemplo, BDF),

$$\mathbf{y} = h \cdot \dot{\mathbf{y}} + \text{old}(\mathbf{y}) \quad (3.286)$$

donde los vectores $\mathbf{y}$ e $\dot{\mathbf{y}}$ representan los valores en el nuevo instante de tiempo. El escalar h es conocido. La forma en que h depende del tamaño del paso de integración varía de un algoritmo de integración a otro. $\text{old}(\mathbf{y})$ es una función de los valores conocidos de $\mathbf{y}$ en los instantes anteriores de tiempo.

Por ejemplo, la discretización BDF de tercer orden cae dentro de esta categoría:

$$\mathbf{y}_{n+1} = \underbrace{\frac{6}{11} \cdot \bar{h}}_h \cdot \dot{\mathbf{y}}_{n+1} + \underbrace{\left(\frac{18}{11} \cdot \mathbf{y}_n - \frac{9}{11} \cdot \mathbf{y}_{n-1} + \frac{2}{11} \cdot \mathbf{y}_{n-2} \right)}_{\text{old}(\mathbf{y})} \quad (3.287)$$

donde $\bar{h}$ es el tamaño del paso de integración.

Aplicando la discretización (3.286) al sistema DAE (3.285), se obtiene:

$$\mathbf{f}\left(\frac{\mathbf{y} - \text{old}(\mathbf{y})}{h}, \mathbf{y}, t\right) = 0 \quad (3.288)$$

El método estándar de iteración de Newton que utiliza DASSL para resolver $\mathbf{y}$ del sistema de ecuaciones algebraicas no lineales anterior está dado por las siguientes ecuaciones (el método de Newton itera sobre el índice k):

$$\mathbf{y}^{k+1} = \mathbf{y}^k - \mathbf{J}^{-1} \cdot \mathbf{f} \left(\frac{\mathbf{y}^k - \text{old}(\mathbf{y})}{h}, \mathbf{y}^k, t \right) \quad (3.289)$$

$\mathbf{J}^{-1}$ representa la inversa de la matriz Jacobiana. La matriz Jacobiana de $\mathbf{f}$ se define como:

$$\mathbf{J} = \frac{d\mathbf{f}}{d\mathbf{y}} = \frac{\partial \mathbf{f}}{\partial \mathbf{y}} + \frac{1}{h} \frac{\partial \mathbf{f}}{\partial \dot{\mathbf{y}}} \quad (3.290)$$

La inversa de la matriz Jacobiana es evaluada en un paso en el tiempo y es mantenida constante tanto tiempo como sea posible. El valor de la matriz Jacobiana puede calcularse empleando técnicas de manipulación simbólica o numéricamente, por ejemplo, usando diferencias finitas.

3.8.2. Integración inline

La integración inline es una técnica cuya finalidad es incrementar la eficiencia de la simulación. Consiste en utilizar la información estructural acerca del sistema para, en lugar de aplicar el algoritmo de Newton al sistema completo (como se hace en (3.289)), aplicarlo únicamente a los lazos algebraicos no lineales identificados de realizar la partición y el tearing al modelo ampliado con las ecuaciones de discretización de las variables de estado, resolviendo secuencialmente las demás ecuaciones (en las que las incógnitas aparecen de forma lineal).

El entorno de modelado establece el sistema no lineal de ecuaciones discretizadas y lo resuelve utilizando la estructura conocida de las ecuaciones, llamando a una función para realizar la iteración de Newton sobre un conjunto mínimo de ecuaciones algebraicas no lineales. La función o bien devuelve los valores de las variables de estado en el nuevo instante o bien informa de que la iteración no converge una vez superado el número de iteraciones definido por el integrador. Sólo la discretización de las ecuaciones de estado y la solución del sistema algebraico de ecuaciones discretizadas (normalmente no lineal) se trasladan al dominio del modelo. La estimación del error, el control del tamaño del paso y los detalles de la discretización de la fórmula (cálculo de h a partir del tamaño del paso, cálculo de $\text{old}(\mathbf{x})$ a partir de los valores de $\mathbf{x}$ en los instantes previos, etc.) están en el dominio del integrador.

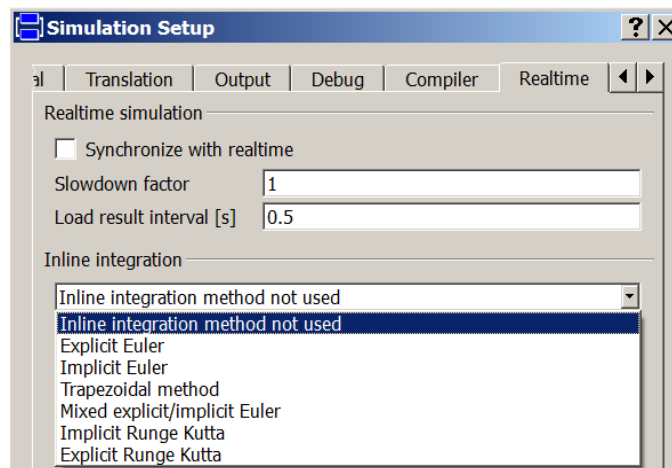


Figura 3.15: Métodos de integración inline en Dymola 2015.

El integrador también proporciona una estimación para el valor de las variables de estado en el nuevo instante de tiempo.

Si se desea emplear la integración inline en Dymola, antes de traducir el modelo es necesario indicarlo en la lengüeta *Realtime* de la ventana *Simulation Setup*. En la Figura 3.15 se muestran los métodos de integración inline soportados por Dymola 2015.

3.8.3. Integración mixed-mode

Escoger entre un método de integración explícito y uno implícito significa escoger entre usar un paso de integración pequeño o resolver un sistema de ecuaciones simultáneas en cada paso de integración. Esta situación motiva la idea de encontrar un término medio entre el método implícito y el explícito. La idea es dividir el sistema en dos partes:

1. Un sistema rápido, a ser posible de pequeñas dimensiones, que pueda ser tratado mediante métodos implícitos.
2. Un sistema lento, al cual puedan aplicarse métodos explícitos.

La técnica mixed-mode puede aplicarse en combinación con la integración inline. La idea entonces sería aplicar un método de integración explícito a todos los estados del modelo salvo a aquellos que el desarrollador del modelo señale como estados rápidos, a los cuales se aplicaría un método de integración implícito.

3.9. LECTURAS RECOMENDADAS

Los artículos (Mattsson et al. 2000, Otter & Olsson 2002) describen la selección de las variables de estado en Modelica.

En la tesis doctoral (Elmqvist 1978) y en el libro (Steward 1981) se aborda el análisis de la estructura de los modelos matemáticos.

El algoritmo de Pantelides es propuesto en el artículo (Pantelides 1988) y el método para la reducción del índice basado en la introducción de derivadas mudas en (Mattsson & Söderlind 1992).

El tearing de los lazos algebraicos en Dymola es descrito en (Elmqvist & Otter 1994).

El algoritmo DASSL está descrito en (Brenan et al. 1996). La integración inline en Modelica es descrita en (Elmqvist et al. 1995, 2002). La integración mixed-mode es descrita en (Schiela & Olsson 2000).

En (Cellier & Kofman 2006) se describen de manera clara y sencilla procedimientos para realizar la asignación de la causalidad computacional, el tearing de los lazos algebraicos y la reducción del índice.

3.10. EJERCICIOS DE AUTOCOMPROBACIÓN

Ejercicio 3.1

En la Figura 3.16 se muestra un circuito eléctrico compuesto por dos generadores de tensión, tres resistencias y un condensador. Se ha señalado el nodo escogido como referencia para el voltaje, se ha asignado nombre a la tensión en los nodos y a la corriente que circula por los componentes.

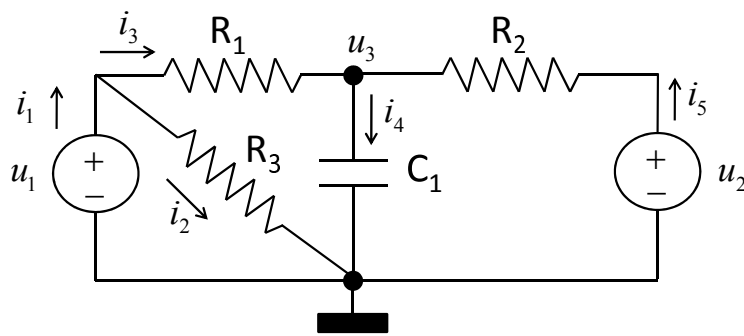


Figura 3.16: Diagrama de un circuito eléctrico.

Los valores de las resistencias (R_1 , R_2 , R_3) y la capacidad (C_1) son conocidos y constantes. Las relaciones constitutivas de los generadores de tensión son:

$$u_1 = U_1 \cdot \sin(w_1 \cdot t) \quad (3.291)$$

$$u_2 = U_2 \cdot \sin(w_2 \cdot t + \phi) \quad (3.292)$$

donde las amplitudes (U_1 , U_2), las frecuencias (w_1 , w_2) y el desfase (ϕ) tienen valores constantes conocidos. La variable t representa el tiempo.

Escriba las ecuaciones del modelo del circuito y clasifique las variables del modelo en parámetros, variables de estado y variables algebraicas. A continuación, realice la asignación de la causalidad computacional. Finalmente, escriba el diagrama de flujo del algoritmo para la simulación de este modelo. Emplee el método de integración de Euler explícito. La condición de finalización de la simulación es que el tiempo simulado alcance el valor 10 s.

Ejercicio 3.2

El modelo mostrado a continuación describe, de manera aproximada, el movimiento vertical de un cohete en una de las fases de acercamiento a la superficie de la luna, donde todas las variables están expresadas en unidades del SI.

$$\frac{dh}{dt} = v \quad (3.293)$$

$$\frac{dv}{dt} = a \quad (3.294)$$

$$m \cdot a = (\text{empuje} - m \cdot g) \quad (3.295)$$

$$\frac{dm}{dt} = -c_1 \cdot \text{empuje} \quad (3.296)$$

$$g = \frac{c_2}{(h + r)^2} \quad (3.297)$$

$$r = 1.738 \cdot 10^6 \quad (3.298)$$

$$c_1 = 2.77 \cdot 10^{-4} \quad (3.299)$$

$$c_2 = 4.925 \cdot 10^{12} \quad (3.300)$$

$$\text{empuje} = 3.635 \cdot 10^4 \quad (3.301)$$

La variable h es la altura a la que se encuentra el cohete, medida desde la superficie de la luna. La componente vertical de la velocidad del cohete es v y la componente vertical de su aceleración es a .

Los motores del cohete consumen combustible. Debido a ello, la masa del cohete (m) disminuye con el tiempo, tal como se expresa en la Ec. (3.296), donde el parámetro c_1 describe la eficiencia del combustible.

En esta fase del alunizaje, la fuerza que ejercen los motores del cohete sobre el cohete es constante y se representa mediante el parámetro *empuje*. También son parámetros del modelo la constante gravitacional (c_2) y el radio de la luna (r). La aceleración gravitatoria que actúa sobre el cohete (g) es función de la altura (h) a la que éste se encuentre, tal y como está descrito en la Ec. (3.297).

Clasifique las variables del modelo en parámetros, variables de estado y variables algebraicas. A continuación, realice la asignación de la causalidad computacional. Finalmente, escriba el diagrama de flujo del algoritmo para la simulación de este modelo. Emplee el método de integración de Euler explícito. Asigne el valor inicial que desee a las variables de estado. La condición de finalización de la simulación es que el tiempo simulado alcance el valor 10 s.

Ejercicio 3.3

A continuación, se describe el modelo de un depósito destinado al almacenamiento de líquido.

El volumen de líquido que en cada instante se encuentra contenido en el depósito está representado por la variable V . El depósito es rectangular, siendo la sección de su base constante e igual a $A = 2 \text{ m}^2$. La relación entre la altura (h) y el volumen (V) del líquido contenido en el depósito es: $V = A \cdot h$.

Por la parte superior del depósito entra un flujo de líquido, F_{in} , cuyo valor es una función conocida del tiempo, $F_{in} = 2 + \sin(\text{time})$, expresado en unidades de $\text{m}^3 \cdot \text{s}^{-1}$.

En la base del depósito hay un orificio de sección $a = 0.1 \text{ m}^2$, por el cual sale un flujo de líquido F_{out} que depende de la altura de líquido (h), de la sección del orificio (a) y de la aceleración gravitatoria ($g = 9.8 \text{ m} \cdot \text{s}^{-2}$) de la forma siguiente: $F_{out} = a \cdot \sqrt{2 \cdot g \cdot h}$.

La variación en el volumen del líquido contenido en el depósito es igual al flujo entrante menos el flujo saliente. Es decir: $\frac{dV}{dt} = F_{in} - F_{out}$.

Resumiendo las explicaciones anteriores, las ecuaciones del modelo son las siguientes:

$$\frac{dV}{dt} = F_{in} - F_{out} \quad (3.302)$$

$$F_{in} = 2 + \sin(\text{time}) \quad (3.303)$$

$$F_{out} = a \cdot \sqrt{2 \cdot g \cdot h} \quad (3.304)$$

$$V = A \cdot h \quad (3.305)$$

$$A = 2 \quad (3.306)$$

$$a = 0.1 \quad (3.307)$$

$$g = 9.8 \quad (3.308)$$

Realice la asignación de la causalidad computacional y escriba el diagrama de flujo del algoritmo para la simulación de este modelo. Emplee el método de integración de Euler explícito. Asigne el valor inicial que desee a las variables de estado. La condición de finalización de la simulación es que el tiempo simulado alcance el valor 10 s.

Ejercicio 3.4

Escriba las ecuaciones que describen el comportamiento del circuito eléctrico mostrado en la Figura 3.17, que está compuesto por un generador sinusoidal de tensión de amplitud U_0 y frecuencia angular ω , dos resistencias, una inducción y un condensador.

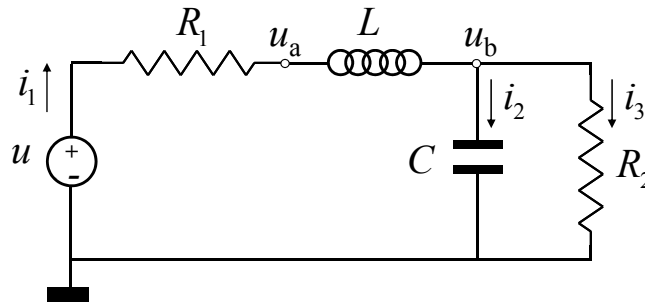


Figura 3.17: Esquema de un circuito eléctrico.

Se supone que el valor de la resistencia R_1 depende linealmente de la temperatura (T) y que ésta aumenta linealmente con el tiempo (t):

$$R_1 = R_{10} \cdot (1 + T_c \cdot (T - T_{nom})) \quad (3.309)$$

$$T = 300 + 0.5 \cdot t \quad (3.310)$$

Los valores de los parámetros del modelo se muestran a continuación, estando todos expresados en el SI de unidades.

$U_0 = 5$	$\omega = 0.1$	$T_{nom} = 300$
$R_{10} = 2 \cdot 10^3$	$T_c = 0.05$	$R_2 = 10^3$
$C = 10^{-4}$	$L = 0.01$	

Realice la asignación de la causalidad computacional. A continuación, escriba el diagrama de flujo del algoritmo para la simulación de este modelo. Emplee el método de integración de Euler explícito. Asigne el valor inicial que desee a las variables de estado. La condición de finalización de la simulación es que el tiempo simulado alcance el valor 10 s.

Ejercicio 3.5

En la Figura 3.18 se muestra el esquema de una casa que dispone de un sistema de calefacción, que introduce un flujo de energía Φ_{in} . Asimismo, hay un flujo de energía que sale de la casa, Φ_{out} , debido a la diferencia entre la temperatura en el interior de la casa T y la temperatura en el exterior T_{ext} .

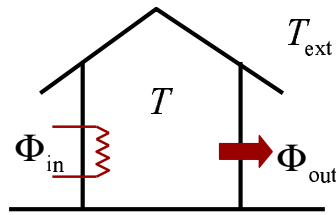


Figura 3.18: Esquema de una casa, con sistema de calefacción y disipación de calor al exterior.

La evolución de la temperatura en el interior de la casa puede calcularse aplicando tres leyes básicas.

1. *Balance de energía.* La diferencia entre los flujos de energía Φ_{in} y Φ_{out} incrementa la energía almacenada en la casa, Q (J).

$$\frac{dQ}{dt} = \Phi_{\text{in}} - \Phi_{\text{out}} \quad (3.311)$$

2. La relación entre Q y la temperatura T depende de la capacidad calorífica de la casa C .

$$Q = C \cdot T \quad (3.312)$$

3. El flujo de energía que sale al exterior a través de las paredes y ventanas depende de la diferencia entre la temperatura interior de la casa y la temperatura del exterior.

$$\Phi_{\text{out}} = k \cdot (T - T_{\text{ext}}) \quad (3.313)$$

donde la constante de proporcionalidad k es la conductividad térmica de las paredes y ventanas.

Se supone que la temperatura exterior, T_{ext} , y el flujo de energía proporcionado por la calefacción, Φ_{in} , son una función conocida del tiempo y un parámetro, respectivamente:

$$T_{\text{ext}} = 275 + 10 \cdot \sin\left(\frac{2 \cdot \pi \cdot t}{3600 \cdot 24}\right) \quad (3.314)$$

$$\Phi_{\text{in}} = 1000 \quad (3.315)$$

donde T_{ext} está expresado en K y Φ_{in} en W. Se conoce el valor de los parámetros C y k :

$$C = 1E5 \text{ J/K} \quad k = 55 \text{ W/K} \quad (3.316)$$

El valor inicial de Q es $3E7$ J.

Asigne la causalidad computacional a las ecuaciones del modelo seleccionando Q como variable de estado.

A continuación, formule el modelo de manera que sólo la temperatura T aparezca derivada y asigne la causalidad computacional seleccionando T como variable de estado.

Ejercicio 3.6

Se desea calcular la fuerza F necesaria para que una masa m siga una determinada trayectoria conocida $x_R(t)$. El modelo está descrito por las ecuaciones siguientes, donde v es la velocidad y a la aceleración.

$$x = x_R(t) \quad (3.317)$$

$$F = m \cdot a \quad (3.318)$$

$$\dot{v} = a \quad (3.319)$$

$$\dot{x} = v \quad (3.320)$$

Indique cuál es el índice del modelo. Determine cuántos grados de libertad posee, reduzca el índice y asigne la causalidad computacional.

Ejercicio 3.7

A continuación se muestra el modelo de un péndulo plano expresado en coordenadas cartesianas. El modelo tiene 3 variables (x, y, λ) y dos parámetros (L, g) .

$$x^2 + y^2 - L^2 = 0 \quad (3.321)$$

$$m \cdot \ddot{x} + \frac{\lambda}{L} \cdot x = 0 \quad (3.322)$$

$$m \cdot \ddot{y} + \frac{\lambda}{L} \cdot y + m \cdot g = 0 \quad (3.323)$$

Indique cuántos grados de libertad tiene el modelo. Reduzca el índice y asigne la causalidad computacional.

Ejercicio 3.8

Considérese el modelo siguiente, en el cual $e(t)$ es una función conocida del tiempo, $\{v_1, v_2, i\}$ son variables y $\{R, C_1, C_2\}$ son parámetros.

$$e(t) = R \cdot i + v_1 \quad (3.324)$$

$$i = C_1 \cdot \dot{v}_1 + C_2 \cdot \dot{v}_2 \quad (3.325)$$

$$v_1 = v_2 \quad (3.326)$$

Indique cuántos grados de libertad tiene el modelo. Reduzca el índice y asigne la causalidad computacional seleccionando v_1 como variable de estado.

Ejercicio 3.9

Considérese el modelo siguiente, en el cual $u(t)$ es una función conocida del tiempo.

$$x = u(t) \quad (3.327)$$

$$z = \dot{x} \quad (3.328)$$

$$y = \frac{\dot{z} + \sin(x)}{\cos(x)} \quad (3.329)$$

Indique cuántos grados de libertad tiene el modelo. Reduzca el índice y asigne la causalidad computacional.

Ejercicio 3.10

Considérese el modelo siguiente, en el cual $e_1(t)$ y $e_2(t)$ son funciones conocidas del tiempo; p es un parámetro de valor conocido; $g(v_2)$ representa una función conocida de v_2 ; y $\{v_1, v_2$ y $v_3\}$ son las variables a calcular mediante la simulación del modelo.

$$\dot{v}_1 = e_1(t) - e_2(t) \cdot v_2 \quad (3.330)$$

$$v_1 = p \cdot v_2 \cdot v_3 \quad (3.331)$$

$$v_3 = g(v_2) \quad (3.332)$$

Seleccione v_1 como variable de estado y asigne la causalidad computacional.

A continuación, formule el modelo de manera que v_2 aparezca derivada y asigne la causalidad computacional seleccionando v_2 como variable de estado.

Finalmente, formule el modelo de manera que v_3 aparezca derivada y asigne la causalidad computacional seleccionando v_3 como variable de estado.

Ejercicio 3.11

A continuación se muestra un algoritmo para resolver el sistema lineal de N ecuaciones con N incógnitas

$$\mathbf{A} \cdot \mathbf{y} = \mathbf{z} \quad (3.333)$$

donde $\mathbf{A}$ es una matriz cuadrada tridiagonal,

$$\mathbf{A} = \begin{pmatrix} a_1 & c_1 & 0 & \dots & \dots & 0 \\ b_2 & a_2 & c_2 & \ddots & & \vdots \\ 0 & b_3 & a_3 & c_3 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & 0 \\ \vdots & & \ddots & b_{N-1} & a_{N-1} & c_{N-1} \\ 0 & \dots & \dots & 0 & b_N & a_N \end{pmatrix} \quad (3.334)$$

e $\mathbf{y}$ y $\mathbf{z}$ son vectores columna de N componentes:

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} \quad \mathbf{z} = \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_N \end{pmatrix} \quad (3.335)$$

Algoritmo: Resolución de $\mathbf{A}\mathbf{y} = \mathbf{z}$ cuando $\mathbf{A}$ es la matriz tridiagonal (3.334).

```

Set:   $w = a_1, y_1 = \frac{z_1}{w}$ 
For    $i = 2, 3, \dots, N$ 
       $v_i = \frac{c_{i-1}}{w}$ 
       $w = a_i - b_i \cdot v_i$ 
       $y_i = \frac{z_i - b_i \cdot y_{i-1}}{w}$ 
End
For    $j = N - 1, N - 2, \dots, 1$ 
       $y_j = y_j - v_{j+1} \cdot y_{j+1}$ 
End

```

Escriba un programa en lenguaje R que aplique el algoritmo anterior, comprobando previamente si $\mathbf{A}$ es una matriz cuadrada tridiagonal. Durante la aplicación del algoritmo debe vigilarse que la variable w no se haga cero (en cuyo caso A no es invertible), ya que eso produciría error de división por cero.

Una vez calculada la solución, el programa debe calcular el residuo, definido como se muestra en la Ec. (3.336). El residuo **res** debe ser un vector de N componentes, cuyos componentes tengan un valor muy próximo a cero.

$$\mathbf{res} = \mathbf{A} \cdot \mathbf{y} - \mathbf{z} \quad (3.336)$$

Ejercicio 3.12

El *método de la secante* es un algoritmo para resolver $f(x) = 0$. Partiendo de dos valores iniciales x_0 y x_1 , el método de la secante obtiene $x_2, x_3, \dots$ aplicando la ecuación siguiente:

$$x_{k+1} = x_k - \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} \cdot f(x_k) \quad \text{para } k = 1, 2, \dots \quad (3.337)$$

Escriba un programa en lenguaje R que aplique el método de la secante para resolver:

$$f(x) = x - \cos(x) = 0 \quad (3.338)$$

La condición de finalización del programa es que se satisfaga cualquiera de las tres condiciones siguientes:

1. Que el número de iteraciones alcance un determinado valor máximo N .
2. Que se satisfaga $f(x_{k+1}) = f(x_k)$.
3. Que se satisfaga $\left| \frac{x_{k+1} - x_k}{x_{k+1}} \right| < \varepsilon$, donde ε es un valor real especificado por el usuario.

Ejercicio 3.13

El *método de Newton* es un algoritmo muy eficiente para resolver $f(x) = 0$. Partiendo de un valor inicial x_0 , el método de Newton obtiene $x_1, x_2, \dots$, recursivamente mediante la fórmula:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \quad (3.339)$$

Escriba un programa en R que aplique el método de Newton para resolver:

$$f(x) = x^2 - 2 = 0 \quad (3.340)$$

Puesto que $f'(x) = 2 \cdot x$, obtenemos:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = x_k - \frac{x_k^2 - 2}{2 \cdot x_k} \quad (3.341)$$

Ejercicio 3.14

El *método del punto fijo* es un método para resolver una ecuación de la forma

$$f(x) = x \quad (3.342)$$

El método consiste en elegir un valor inicial x_0 y realizar la iteración

$$x_{i+1} = f(x_i) \quad (3.343)$$

hasta que la diferencia $|x_{i+1} - x_i|$ sea inferior a una determinada tolerancia.

Escriba un programa en lenguaje R que aplique el método del punto fijo a la resolución de la ecuación

$$\cos(x) = x \quad (3.344)$$

El programa debe finalizar bien cuando el residuo sea inferior a un determinado valor, o bien cuando el número de iteraciones alcance un determinado límite.

3.11. SOLUCIONES DE LOS EJERCICIOS

Solución al Ejercicio 3.1

A fin de plantear el modelo del circuito, primeramente se escribe la relación constitutiva de cada uno de los componentes del circuito:

$$u_1 = U_1 \cdot \sin(w_1 \cdot t) \quad (3.345)$$

$$u_2 = U_2 \cdot \sin(w_2 \cdot t + \phi) \quad (3.346)$$

$$u_1 - u_3 = i_3 \cdot R_1 \quad (3.347)$$

$$u_1 = i_2 \cdot R_3 \quad (3.348)$$

$$u_2 - u_3 = i_5 \cdot R_2 \quad (3.349)$$

$$C_1 \cdot \frac{du_3}{dt} = i_4 \quad (3.350)$$

A continuación, se escriben las ecuaciones que imponen que la corriente total entrante a un nodo es igual a la corriente total saliente de dicho nodo. Se plantea una ecuación de conservación de la corriente en cada nodo, salvo en el nodo tierra. En este caso, las ecuaciones son las siguientes:

$$i_1 = i_2 + i_3 \quad (3.351)$$

$$i_3 + i_5 = i_4 \quad (3.352)$$

Las Ecs. (3.345) – (3.352) son un posible modelo matemático del circuito. Las variables del modelo pueden clasificarse de la forma siguiente:

- Parámetros: $U_1, w_1, U_2, w_2, \phi, R_1, R_2, R_3, C_1$
- Variable de estado: u_3
- Variables algebraicas: $i_1, i_2, i_3, i_4, i_5, u_1, u_2$

Para asignar la causalidad computacional al modelo, se sustituyen las derivadas de las variables de estado por variables mudas ($\frac{du_3}{dt} \rightarrow deru_3$), obteniéndose el modelo siguiente:

$$u_1 = U_1 \cdot \sin(w_1 \cdot t) \quad (3.353)$$

$$u_2 = U_2 \cdot \sin(w_2 \cdot t + \phi) \quad (3.354)$$

$$u_1 - u_3 = i_3 \cdot R_1 \quad (3.355)$$

$$u_1 = i_2 \cdot R_3 \quad (3.356)$$

$$u_2 - u_3 = i_5 \cdot R_2 \quad (3.357)$$

$$C_1 \cdot \text{der}u_3 = i_4 \quad (3.358)$$

$$i_1 = i_2 + i_3 \quad (3.359)$$

$$i_3 + i_5 = i_4 \quad (3.360)$$

Las variables del modelo pueden clasificarse en conocidas (el tiempo, los parámetros y las variables de estado) y desconocidas (las variables algebraicas y las derivadas de las variables de estado):

- Conocidas: t
 $U_1, w_1, U_2, w_2, \phi, R_1, R_2, R_3, C_1$
 u_3
- Desconocidas: $i_1, i_2, i_3, i_4, i_5, u_1, u_2$
 $\text{der}u_3$

Con el fin de analizar si el modelo es *estructuralmente singular*, se comprueba que:

1. El número de ecuaciones y de variables desconocidas (incógnitas) es el mismo. Este modelo se compone de 8 ecuaciones y hay 8 incógnitas ($i_1, i_2, i_3, i_4, i_5, u_1, u_2, \text{der}u_3$).
2. Cada incógnita puede emparejarse con una ecuación en que aparezca y con la cual no se haya emparejado ya otra incógnita.

$$u_1 \rightarrow u_1 = U_1 \cdot \sin(w_1 \cdot t) \quad \text{Ec. (3.353)}$$

$$u_2 \rightarrow u_2 = U_2 \cdot \sin(w_2 \cdot t + \phi) \quad \text{Ec. (3.354)}$$

$$i_3 \rightarrow u_1 - u_3 = i_3 \cdot R_1 \quad \text{Ec. (3.355)}$$

$$i_2 \rightarrow u_1 = i_2 \cdot R_3 \quad \text{Ec. (3.356)}$$

$$i_5 \rightarrow u_2 - u_3 = i_5 \cdot R_2 \quad \text{Ec. (3.357)}$$

$$i_4 \rightarrow i_3 + i_5 = i_4 \quad \text{Ec. (3.360)}$$

$$\text{der}u_3 \rightarrow C_1 \cdot \text{der}u_3 = i_4 \quad \text{Ec. (3.358)}$$

$$i_1 \rightarrow i_1 = i_2 + i_3 \quad \text{Ec. (3.359)}$$

Finalmente, el modelo ordenado y resuelto, con la causalidad computacional señalada, es el siguiente:

$$[u_1] = U_1 \cdot \sin(w_1 \cdot t) \quad (3.361)$$

$$[u_2] = U_2 \cdot \sin(w_2 \cdot t + \phi) \quad (3.362)$$

$$[i_3] = \frac{u_1 - u_3}{R_1} \quad (3.363)$$

$$[i_2] = \frac{u_1}{R_3} \quad (3.364)$$

$$[i_5] = \frac{u_2 - u_3}{R_2} \quad (3.365)$$

$$[i_4] = i_3 + i_5 \quad (3.366)$$

$$[deru_3] = \frac{i_4}{C_1} \quad (3.367)$$

$$[i_1] = i_2 + i_3 \quad (3.368)$$

En la Figura 3.19 se muestra el diagrama de flujo para la simulación del modelo, empleando el método de integración de Euler explícito y considerando como condición de finalización que el tiempo alcance el valor 10 s. Se han asignado valores arbitrarios a los parámetros y al valor inicial de la variable de estado.

La asignación de la causalidad computacional determina el orden en el cual deben evaluarse las ecuaciones del modelo y qué variable desconocida debe evaluarse de cada ecuación, siendo las variables a evaluar las variables algebraicas y las derivadas de las variables de estado. En concreto, obsérvese que en el paso del algoritmo en el cual se evalúan las variables algebraicas y las derivadas (véase nuevamente la Figura 3.19), se ejecutan por orden las asignaciones mostradas en las Ecs. (3.361) – (3.368).

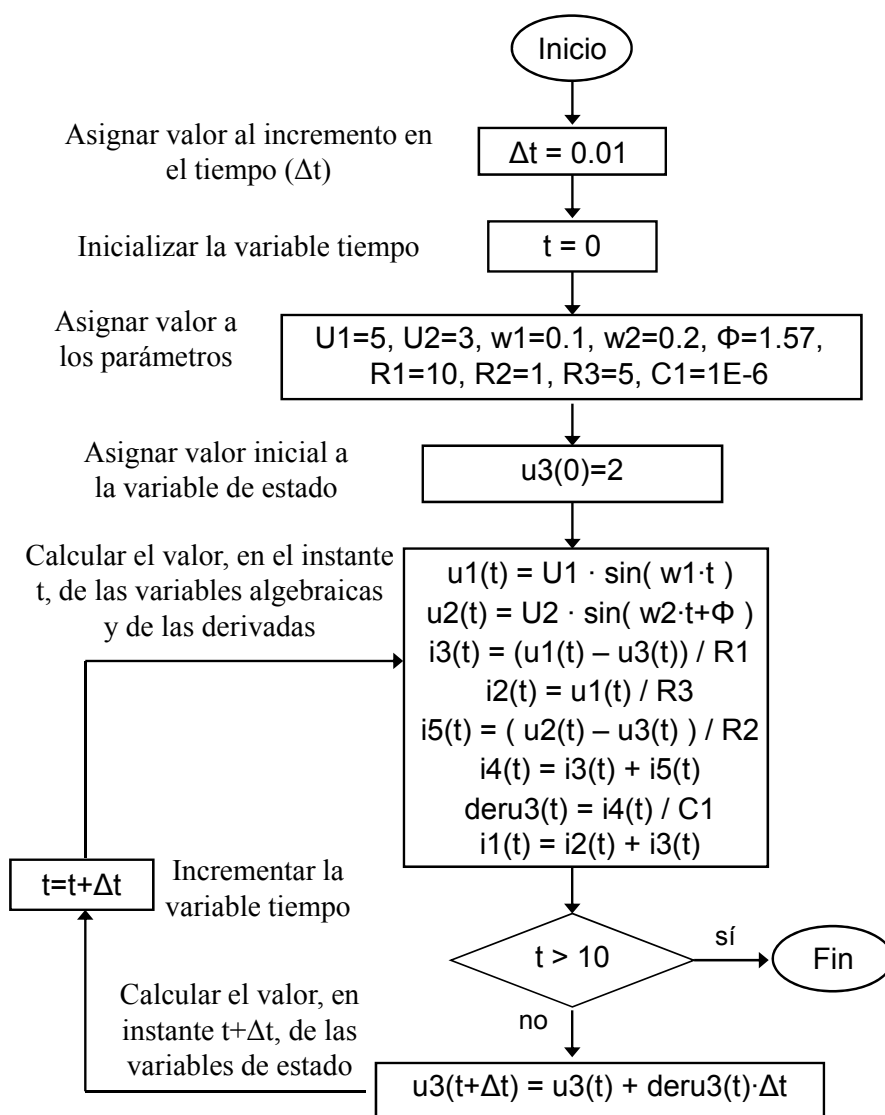


Figura 3.19: Algoritmo de la simulación del circuito mostrado en la Figura 3.16.

Solución al Ejercicio 3.2

Las variables del modelo pueden clasificarse de la forma siguiente:

- Parámetros: $r, c_1, c_2, \text{empuje}$
- Variables de estado: h, v, m
- Variables algebraicas: a, g

Para asignar la causalidad computacional al modelo, se sustituyen las derivadas de las variables de estado por variables mudas ($\frac{dh}{dt} \rightarrow \text{der}h$, $\frac{dv}{dt} \rightarrow \text{der}v$, $\frac{dm}{dt} \rightarrow \text{der}m$). Omitiendo las ecuaciones en las que se asigna valor a los parámetros, se obtiene el modelo siguiente:

$$\text{der}h = v \quad (3.369)$$

$$\text{der}v = a \quad (3.370)$$

$$m \cdot a = (\text{empuje} - m \cdot g) \quad (3.371)$$

$$\text{der}m = -c_1 \cdot \text{empuje} \quad (3.372)$$

$$g = \frac{c_2}{(h + r)^2} \quad (3.373)$$

Las variables del modelo pueden clasificarse en conocidas (el tiempo, los parámetros y las variables de estado) y desconocidas (las variables algebraicas y las derivadas de las variables de estado):

- Conocidas: t
 $r, c_1, c_2, \text{empuje}$
 h, v, m
- Desconocidas: a, g
 $\text{der}h, \text{der}v, \text{der}m$

Con el fin de analizar si el modelo es *estructuralmente singular*, se comprueba que:

1. El número de ecuaciones y de variables desconocidas (incógnitas) es el mismo. Este modelo tiene 5 ecuaciones, Ecs. (3.369) – (3.373), y 5 incógnitas: $a, g, \text{der}h, \text{der}v, \text{der}m$.
2. Cada incógnita puede emparejarse con una ecuación en que aparezca y con la cual no se haya emparejado ya otra incógnita.

$$\text{der}h \rightarrow \text{der}h = v \quad \text{Ec. (3.369)}$$

$$\text{der}v \rightarrow \text{der}v = a \quad \text{Ec. (3.370)}$$

$$a \rightarrow m \cdot a = (\text{empuje} - m \cdot g) \quad \text{Ec. (3.371)}$$

$$\text{der}m \rightarrow \text{der}m = -c_1 \cdot \text{empuje} \quad \text{Ec. (3.372)}$$

$$g \rightarrow g = \frac{c_2}{(h+r)^2} \quad \text{Ec. (3.373)}$$

Finalmente, las ecuaciones del modelo, con la asignación de la causalidad computacional señalada, son las siguientes:

$$[\text{der}h] = v \quad (3.374)$$

$$[\text{der}v] = a \quad (3.375)$$

$$m \cdot [a] = (\text{empuje} - m \cdot g) \quad (3.376)$$

$$[\text{der}m] = -c_1 \cdot \text{empuje} \quad (3.377)$$

$$[g] = \frac{c_2}{(h+r)^2} \quad (3.378)$$

El modelo ordenado y resuelto es el siguiente (existen otras posibles ordenaciones de las ecuaciones que son igualmente válidas):

$$[\text{der}h] = v \quad (3.379)$$

$$[\text{der}m] = -c_1 \cdot \text{empuje} \quad (3.380)$$

$$[g] = \frac{c_2}{(h+r)^2} \quad (3.381)$$

$$[a] = \frac{\text{empuje} - m \cdot g}{m} \quad (3.382)$$

$$[\text{der}v] = a \quad (3.383)$$

En la Figura 3.20 se muestra el diagrama de flujo para la simulación del modelo, empleando el método de integración de Euler explícito y considerando como condición de finalización que el tiempo alcance el valor 100 s. Se han asignado valores arbitrarios a los parámetros y a los valores iniciales de las variables de estado.

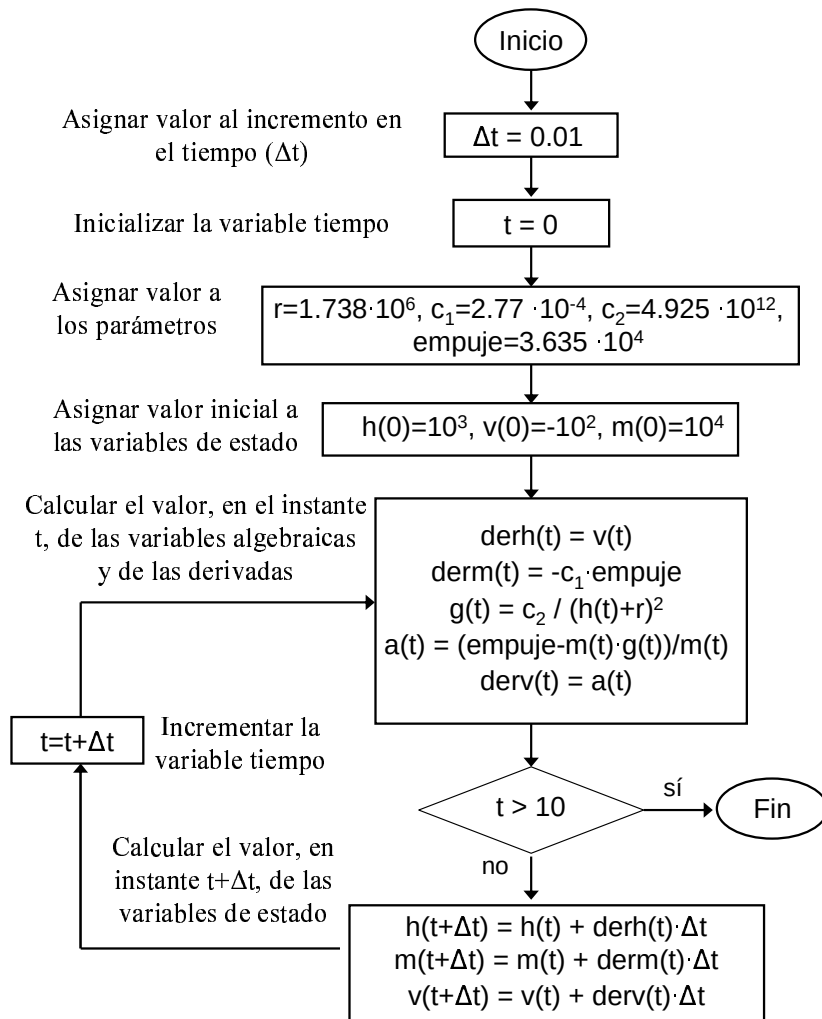


Figura 3.20: Simulación del modelo de una fase del alunizaje.

Solución al Ejercicio 3.3

Las variables del modelo pueden clasificarse de la forma siguiente:

- Parámetros: A, a, g
- Variables de estado: V
- Variables algebraicas: F_{in}, F_{out}, h

Para asignar la causalidad computacional al modelo, se sustituyen las derivadas de las variables de estado por variables mudas ($\frac{dV}{dt} \rightarrow derV$). Omitiendo las ecuaciones en las que se asigna valor a los parámetros, se obtiene el modelo siguiente:

$$derV = F_{in} - F_{out} \quad (3.384)$$

$$F_{in} = 2 + \sin(t) \quad (3.385)$$

$$F_{out} = a \cdot \sqrt{2 \cdot g \cdot h} \quad (3.386)$$

$$V = A \cdot h \quad (3.387)$$

Las variables del modelo pueden clasificarse en conocidas (el tiempo, los parámetros y las variables de estado) y desconocidas (las variables algebraicas y las derivadas de las variables de estado):

- Conocidas: t
 A, a, g
 V
- Desconocidas: F_{in}, F_{out}, h
 $derV$

Con el fin de analizar si el modelo es *estructuralmente singular*, se comprueba que:

1. El número de ecuaciones y de variables desconocidas (incógnitas) es el mismo. Este modelo tiene 4 ecuaciones, Ecs. (3.384) – (3.387), y 4 incógnitas: $F_{in}, F_{out}, h, derV$.
2. Cada incógnita puede emparejarse con una ecuación en que aparezca y con la cual no se haya emparejado ya otra incógnita.

$$derV \rightarrow derV = F_{in} - F_{out} \quad \text{Ec. (3.384)}$$

$$F_{in} \rightarrow F_{in} = 2 + \sin(t) \quad \text{Ec. (3.385)}$$

$$F_{out} \rightarrow F_{out} = a \cdot \sqrt{2 \cdot g \cdot h} \quad \text{Ec. (3.386)}$$

$$h \rightarrow V = A \cdot h \quad \text{Ec. (3.387)}$$

Finalmente, las ecuaciones del modelo, con la asignación de la causalidad computacional señalada, son las siguientes:

$$[derV] = F_{in} - F_{out} \quad (3.388)$$

$$[F_{in}] = 2 + \sin(t) \quad (3.389)$$

$$[F_{out}] = a \cdot \sqrt{2 \cdot g \cdot h} \quad (3.390)$$

$$V = A \cdot [h] \quad (3.391)$$

El modelo ordenado y resuelto es el siguiente (hay otras posibles ordenaciones, por ejemplo, puede intercambiarse el orden de las dos primeras ecuaciones, obteniéndose una secuencia de cálculo igualmente válida):

$$[F_{in}] = 2 + \sin(t) \quad (3.392)$$

$$[h] = \frac{V}{A} \quad (3.393)$$

$$[F_{out}] = a \cdot \sqrt{2 \cdot g \cdot h} \quad (3.394)$$

$$[derV] = F_{in} - F_{out} \quad (3.395)$$

En la Figura 3.21 se muestra el diagrama de flujo para la simulación del modelo, empleando el método de integración de Euler explícito y considerando como condición de finalización que el tiempo alcance el valor 10 s. Se ha considerado que el valor inicial del volumen de líquido es 2 m^3 .

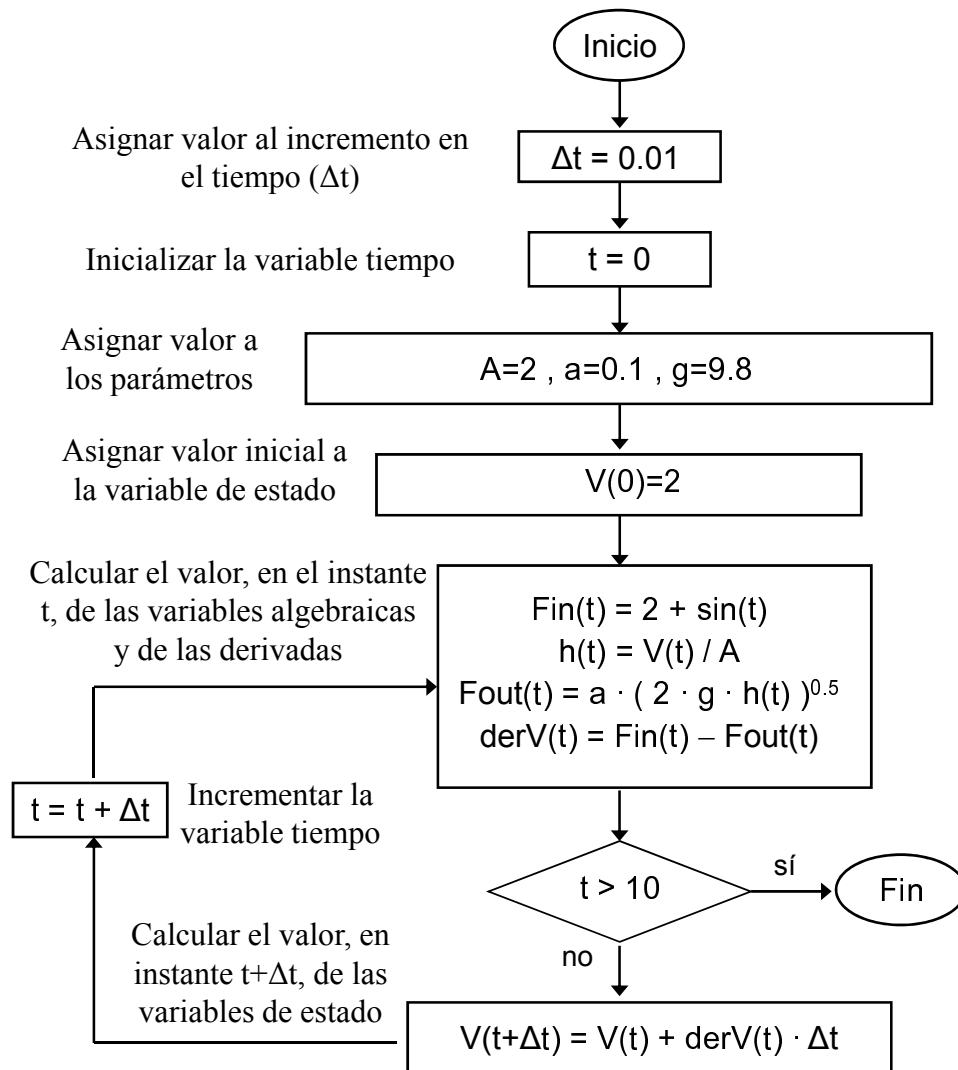


Figura 3.21: Simulación del modelo de un depósito de líquido.

Solución al Ejercicio 3.4

El modelo del circuito eléctrico mostrado en la Figura 3.17 está descrito por las ecuaciones siguientes:

$$u = U_0 \cdot \text{sen}(\omega \cdot t) \quad (3.396)$$

$$u - u_a = i_1 \cdot R_1 \quad (3.397)$$

$$R_1 = R_{10} \cdot (1 + T_c \cdot (T - T_{nom})) \quad (3.398)$$

$$T = 300 + 0.5 \cdot t \quad (3.399)$$

$$L \cdot \frac{di_1}{dt} = u_a - u_b \quad (3.400)$$

$$C \cdot \frac{du_b}{dt} = i_2 \quad (3.401)$$

$$u_b = R_2 \cdot i_3 \quad (3.402)$$

$$i_1 = i_2 + i_3 \quad (3.403)$$

Las variables del modelo pueden clasificarse de la forma siguiente:

- Parámetros: $U_0, \omega, R_{10}, T_c, T_{nom}, C, L, R_2$
- Variables de estado: u_b, i_1
- Variables algebraicas: u, u_a, i_2, i_3, R_1, T

Para asignar la causalidad computacional al modelo, se sustituyen las derivadas de las variables de estado por variables mudas ($\frac{du_b}{dt} \rightarrow \text{der}u_b, \frac{di_1}{dt} \rightarrow \text{der}i_1$). Omitiendo las ecuaciones en las que se asigna valor a los parámetros, se obtiene el modelo siguiente:

$$u = U_0 \cdot \text{sen}(\omega \cdot t) \quad (3.404)$$

$$u - u_a = i_1 \cdot R_1 \quad (3.405)$$

$$R_1 = R_{10} \cdot (1 + T_c \cdot (T - T_{nom})) \quad (3.406)$$

$$T = 300 + 0.5 \cdot t \quad (3.407)$$

$$L \cdot \text{der}i_1 = u_a - u_b \quad (3.408)$$

$$C \cdot \text{der}u_b = i_2 \quad (3.409)$$

$$u_b = R_2 \cdot i_3 \quad (3.410)$$

$$i_1 = i_2 + i_3 \quad (3.411)$$

Las variables del modelo pueden clasificarse en conocidas (el tiempo, los parámetros y las variables de estado) y desconocidas (las variables algebraicas y las derivadas de las variables de estado):

- Conocidas: t
 $U_0, \omega, R_{10}, T_c, T_{nom}, C, L, R_2$
 u_b, i_1
- Desconocidas: u, u_a, i_2, i_3, R_1, T
 $deri_1, deru_b$

Con el fin de analizar si el modelo es *estructuralmente singular*, se comprueba que:

1. El número de ecuaciones y de variables desconocidas (incógnitas) es el mismo. Este modelo tiene 8 ecuaciones, Ecs. (3.404) – (3.411), y 8 incógnitas: $u, u_a, i_2, i_3, R_1, T, deri_1, deru_b$.
2. Cada incógnita puede emparejarse con una ecuación en que aparezca y con la cual no se haya emparejado ya otra incógnita.

$$\begin{array}{lll}
 u & \rightarrow & u = U_0 \cdot \text{sen}(\omega \cdot t) & \text{Ec. (3.404)} \\
 u_a & \rightarrow & u - u_a = i_1 \cdot R_1 & \text{Ec. (3.405)} \\
 R_1 & \rightarrow & R_1 = R_{10} \cdot (1 + T_c \cdot (T - T_{nom})) & \text{Ec. (3.406)} \\
 T & \rightarrow & T = 300 + 0.5 \cdot t & \text{Ec. (3.407)} \\
 deri_1 & \rightarrow & L \cdot deri_1 = u_a - u_b & \text{Ec. (3.408)} \\
 deru_b & \rightarrow & C \cdot deru_b = i_2 & \text{Ec. (3.409)} \\
 i_3 & \rightarrow & u_b = R_2 \cdot i_3 & \text{Ec. (3.410)} \\
 i_2 & \rightarrow & i_1 = i_2 + i_3 & \text{Ec. (3.411)}
 \end{array}$$

Finalmente, las ecuaciones del modelo, con la asignación de la causalidad computacional señalada, son las siguientes:

$$[u] = U_0 \cdot \text{sen}(\omega \cdot t) \quad (3.412)$$

$$u - [u_a] = i_1 \cdot R_1 \quad (3.413)$$

$$[R_1] = R_{10} \cdot (1 + T_c \cdot (T - T_{nom})) \quad (3.414)$$

$$[T] = 300 + 0.5 \cdot t \quad (3.415)$$

$$L \cdot [deri_1] = u_a - u_b \quad (3.416)$$

$$C \cdot [deru_b] = i_2 \quad (3.417)$$

$$u_b = R_2 \cdot [i_3] \quad (3.418)$$

$$i_1 = [i_2] + i_3 \quad (3.419)$$

El modelo ordenado y resuelto es el siguiente (existen otras posibles ordenaciones de las ecuaciones que son igualmente válidas):

$$[u] = U_0 \cdot \text{sen}(\omega \cdot t) \quad (3.420)$$

$$[T] = 300 + 0.5 \cdot t \quad (3.421)$$

$$[R_1] = R_{10} \cdot (1 + T_c \cdot (T - T_{nom})) \quad (3.422)$$

$$[u_a] = u - i_1 \cdot R_1 \quad (3.423)$$

$$[deri_1] = \frac{u_a - u_b}{L} \quad (3.424)$$

$$[i_3] = \frac{u_b}{R_2} \quad (3.425)$$

$$[i_2] = i_1 - i_3 \quad (3.426)$$

$$[deru_b] = \frac{i_2}{C} \quad (3.427)$$

En la Figura 3.22 se muestra el diagrama de flujo para la simulación del modelo, empleando el método de integración de Euler explícito y considerando como condición de finalización que el tiempo alcance el valor 10 s. Se han asignado valores arbitrarios a los valores iniciales de las variables de estado.

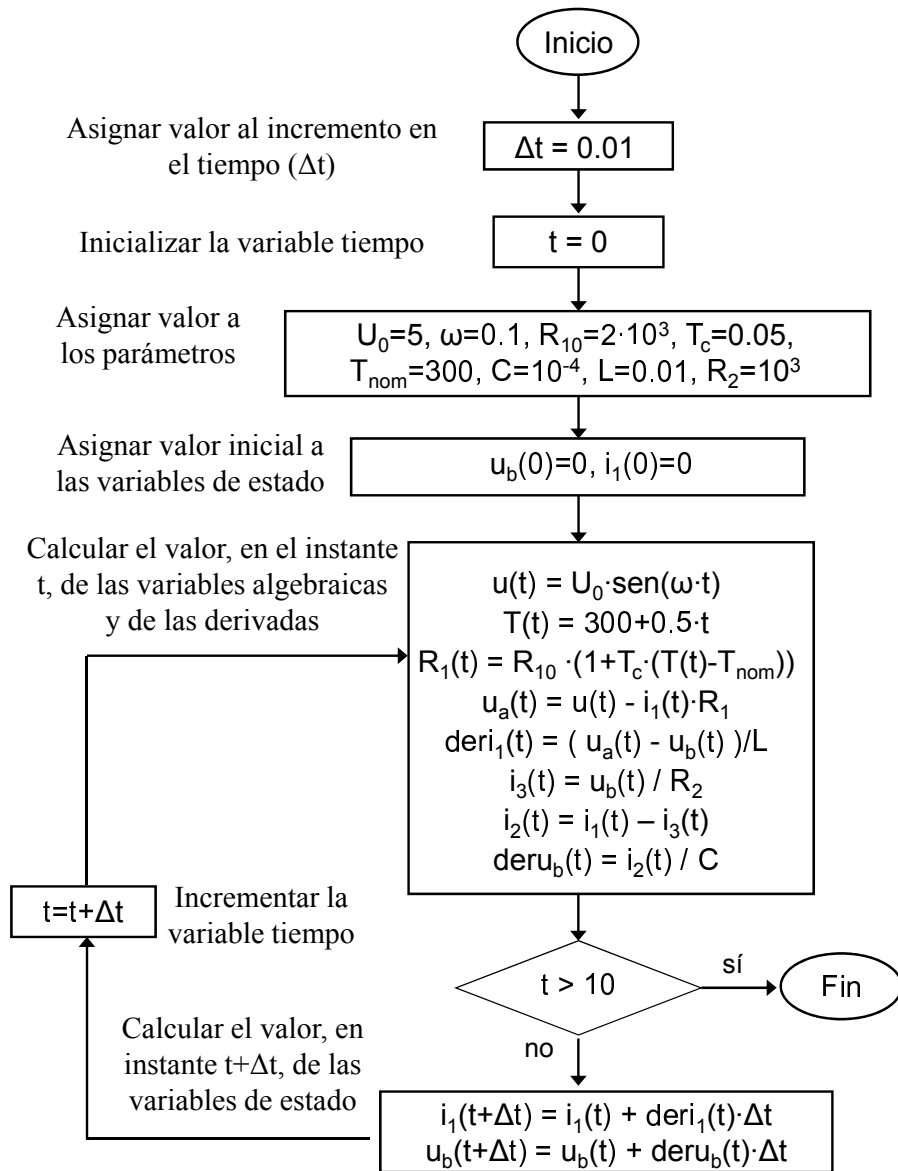


Figura 3.22: Algoritmo de la simulación del circuito mostrado en la Figura 3.17.

Solución al Ejercicio 3.5

El modelo completo está compuesto por las ecuaciones mostradas a continuación, donde C , k y Φ_{in} son parámetros de valor conocido.

$$\frac{dQ}{dt} = \Phi_{\text{in}} - \Phi_{\text{out}} \quad (3.428)$$

$$Q = C \cdot T \quad (3.429)$$

$$\Phi_{\text{out}} = k \cdot (T - T_{\text{ext}}) \quad (3.430)$$

$$T_{\text{ext}} = 275 + 10 \cdot \sin\left(\frac{2 \cdot \pi \cdot t}{3600 \cdot 24}\right) \quad (3.431)$$

La variable Q aparece derivada. La seleccionamos como variable de estado. Sustituyendo la derivada de Q por la variable muda $\text{der}Q$ y realizando la asignación de la causalidad computacional, se obtiene el modelo ordenado y con la causalidad computacional señalada:

$$Q \quad \text{Variable de estado} \quad (3.432)$$

$$[T_{\text{ext}}] = 275 + 10 \cdot \sin\left(\frac{2 \cdot \pi \cdot t}{3600 \cdot 24}\right) \quad (3.433)$$

$$Q = C \cdot [T] \quad (3.434)$$

$$[\Phi_{\text{out}}] = k \cdot (T - T_{\text{ext}}) \quad (3.435)$$

$$[\text{der}Q] = \Phi_{\text{in}} - \Phi_{\text{out}} \quad (3.436)$$

La Ec. (3.429) relaciona T y Q . Derivamos respecto al tiempo dicha ecuación y la añadimos al modelo. Puesto que C es un parámetro (su derivada es cero), en esta nueva ecuación sólo intervienen $\frac{dQ}{dt}$ y $\frac{dT}{dt}$. Como esta es la única ecuación en la que aparece $\frac{dT}{dt}$, esta ecuación deberá emplearse para calcular dicha derivada. Sustituimos las derivadas por variables mudas y asignamos la causalidad computacional, seleccionando T como variable de estado. El modelo ordenado (existen otras posibles ordenaciones igualmente válidas) y con la causalidad computacional señalada es:

$$T \quad \text{Variable de estado} \quad (3.437)$$

$$[T_{\text{ext}}] = 275 + 10 \cdot \sin\left(\frac{2 \cdot \pi \cdot t}{3600 \cdot 24}\right) \quad (3.438)$$

$$[Q] = C \cdot T \quad (3.439)$$

$$[\Phi_{\text{out}}] = k \cdot (T - T_{\text{ext}}) \quad (3.440)$$

$$[\text{der}Q] = \Phi_{\text{in}} - \Phi_{\text{out}} \quad (3.441)$$

$$\text{der}Q = C \cdot [\text{der}T] \quad (3.442)$$

Solución al Ejercicio 3.6

Se desea calcular la fuerza F necesaria para que una masa m siga una determinada trayectoria conocida $x_R(t)$. El modelo está descrito por las ecuaciones siguientes, donde v es la velocidad y a la aceleración.

$$x = x_R(t) \quad (3.443)$$

$$F = m \cdot a \quad (3.444)$$

$$\dot{v} = a \quad (3.445)$$

$$\dot{x} = v \quad (3.446)$$

Para calcular el índice, analicemos cuantas veces es necesario derivar todo o parte del sistema DAE para obtener un ODE explícito $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, t)$, donde $\mathbf{x}=(x, v, a, F)$.

Obsérvese que la única variable que interviene en la Ec. (3.443) es x . Por otra parte, F sólo aparece en la Ec. (3.444). En consecuencia, $\dot{x}$ deberá calcularse derivando la Ec. (3.443) y $\dot{F}$ de derivar la Ec. (3.444). Puesto que $\dot{x}$ se calculará derivando la Ec. (3.443), $\dot{v}$ deberá calcularse derivando la Ec. (3.446). Finalmente, $\dot{a}$ deberá calcularse derivando la Ec. (3.445).

Derivando las cuatro ecuaciones del sistema se obtiene:

$$\dot{x} = \dot{x}_R(t) \quad (3.447)$$

$$\dot{F} = m \cdot \dot{a} \quad (3.448)$$

$$\ddot{v} = \dot{a} \quad (3.449)$$

$$\ddot{x} = \dot{v} \quad (3.450)$$

En la Ec. (3.449) interviene $\ddot{v}$ y en la Ec. (3.450) $\ddot{x}$. Para eliminar estas dos nuevas variables, se derivan las Ecs. (3.447) y (3.450), obteniéndose:

$$\ddot{x} = \ddot{x}_R(t) \quad (3.451)$$

$$\ddot{x} = \ddot{v} \quad (3.452)$$

Aparece una nueva variable, $\ddot{x}$. Derivando nuevamente la Ec. (3.451), se obtiene una ligadura para esta variable:

$$\ddot{x} = \ddot{x}_R(t) \quad (3.453)$$

Recopilando las ecuaciones anteriores, se obtiene:

$$\dot{x} = \dot{x}_R(t) \quad (3.454)$$

$$\dot{F} = m \cdot \dot{a} \quad (3.455)$$

$$\ddot{v} = \dot{a} \quad (3.456)$$

$$\ddot{x} = \dot{v} \quad (3.457)$$

$$\ddot{x} = \ddot{x}_R(t) \quad (3.458)$$

$$\ddot{x} = \ddot{v} \quad (3.459)$$

$$\ddot{x} = \ddot{x}_R(t) \quad (3.460)$$

Eliminando de este sistema las variables $\ddot{v}$, $\ddot{x}$, $\ddot{x}$, se obtiene el siguiente ODE implícito:

$$\dot{x} = \dot{x}_R(t) \quad (3.461)$$

$$\dot{F} = m \cdot \dot{a} \quad (3.462)$$

$$\ddot{x}_R(t) = \dot{a} \quad (3.463)$$

$$\ddot{x}_R(t) = \dot{v} \quad (3.464)$$

que puede manipularse a fin de obtener un ODE explícito:

$$\dot{x} = \dot{x}_R(t) \quad (3.465)$$

$$\dot{v} = \ddot{x}_R(t) \quad (3.466)$$

$$\dot{a} = \ddot{x}_R(t) \quad (3.467)$$

$$\dot{F} = m \cdot \ddot{x}_R(t) \quad (3.468)$$

Ha sido preciso derivar tres veces la Ec. (3.443), dos veces la Ec. (3.446), y una vez las Ecs. (3.444) y (3.445). Por tanto, el sistema DAE compuesto por las Ecs. (3.443) – (3.446) tiene índice 3.

Para reducir el índice debe examinarse si derivando alguna de las ecuaciones del sistema se obtienen ligaduras adicionales para las variables que aparecen derivadas (x , v), las derivadas de dichas variables ($\dot{x}$, $\dot{v}$) o las variables algebraicas (a , F).

Obsérvese que la derivada de la Ec. (3.443) supone una ligadura para $\dot{x}$. Asimismo, la segunda derivada de la Ec. (3.443) y la derivada de la Ec. (3.446) suponen una ligadura para $\dot{v}$. Añadiendo estas ecuaciones derivadas al sistema, se obtiene:

$$x = x_R(t) \quad (3.469)$$

$$F = m \cdot a \quad (3.470)$$

$$\dot{v} = a \quad (3.471)$$

$$\dot{x} = v \quad (3.472)$$

$$\dot{x} = \dot{x}_R(t) \quad (3.473)$$

$$\ddot{x} = \dot{v} \quad (3.474)$$

$$\ddot{x} = \ddot{x}_R(t) \quad (3.475)$$

El sistema no posee ningún grado de libertad: las variables que aparecen derivadas son variables algebraicas, que se calculan de las ecuaciones del modelo, no de integrar sus derivadas.

Reemplazando las derivadas por variables mudas y asignando la causalidad computacional a este sistema de 7 ecuaciones con 7 incógnitas, se obtiene:

$$[x] = x_R(t) \quad (3.476)$$

$$[derx] = \dot{x}_R(t) \quad (3.477)$$

$$[derderx] = \ddot{x}_R(t) \quad (3.478)$$

$$derx = [v] \quad (3.479)$$

$$derderx = [derv] \quad (3.480)$$

$$derv = [a] \quad (3.481)$$

$$[F] = m \cdot a \quad (3.482)$$

Solución al Ejercicio 3.7

En primer lugar, introduzcamos las variables v_x , v_y en el modelo, que representan la componente horizontal y vertical de la velocidad.

$$x^2 + y^2 - L^2 = 0 \quad (3.483)$$

$$v_x = \dot{x} \quad (3.484)$$

$$v_y = \dot{y} \quad (3.485)$$

$$m \cdot \dot{v}_x + \frac{\lambda}{L} \cdot x = 0 \quad (3.486)$$

$$m \cdot \dot{v}_y + \frac{\lambda}{L} \cdot y + m \cdot g = 0 \quad (3.487)$$

De esta manera, el modelo queda expresado como un sistema DAE. Es decir, de la forma:

$$\mathbf{F}(\dot{\mathbf{x}}, \mathbf{x}, \mathbf{y}, t) = 0 \quad (3.488)$$

donde:

$$\mathbf{x} = (x, y, v_x, v_y) \quad \mathbf{y} = (\lambda) \quad (3.489)$$

Asignemos la causalidad computacional al sistema formado por las Ecs. (3.483) – (3.487), a fin de averiguar si su índice es cero, uno o tiene índice superior.

En el sistema aparecen 4 variables derivadas: x , y , v_x , v_y . Asumiendo que son variables de estado y sustituyendo sus derivadas por variables auxiliares ($\dot{x} \rightarrow \text{der}x$, $\dot{y} \rightarrow \text{der}y$, $\dot{v}_x \rightarrow \text{der}v_x$, $\dot{v}_y \rightarrow \text{der}v_y$),

$$x, y, v_x, v_y \quad \text{Variables de estado} \quad (3.490)$$

$$x^2 + y^2 - L^2 = 0 \quad (3.491)$$

$$v_x = \text{der}x \quad (3.492)$$

$$v_y = \text{der}y \quad (3.493)$$

$$m \cdot \text{der}v_x + \frac{\lambda}{L} \cdot x = 0 \quad (3.494)$$

$$m \cdot \text{der}v_y + \frac{\lambda}{L} \cdot y + m \cdot g = 0 \quad (3.495)$$

se obtiene que:

- Las variables $derx$, $dery$ deben calcularse de las Ecs. (3.492) y (3.493), respectivamente, ya que son las únicas incógnitas en esas ecuaciones.
- La Ec. (3.491) es redundante, ya que x , y son variables de estado.
- Se dispone únicamente de dos ecuaciones, Ecs. (3.494) y (3.495), para calcular tres incógnitas: $derv_x$, $derv_y$, λ .

El problema de asignación de la causalidad computacional no tiene solución. Así pues, el DAE o bien tiene índice superior o bien está mal planteado.

Obsérvese que derivando dos veces la Ec. (3.483), una vez las Ecs. (3.484) y (3.485), y añadiendo estas ecuaciones derivadas al sistema se obtiene:

$$x^2 + y^2 - L^2 = 0 \quad (3.496)$$

$$v_x = \dot{x} \quad (3.497)$$

$$v_y = \dot{y} \quad (3.498)$$

$$m \cdot \dot{v}_x + \frac{\lambda}{L} \cdot x = 0 \quad (3.499)$$

$$m \cdot \dot{v}_y + \frac{\lambda}{L} \cdot y + m \cdot g = 0 \quad (3.500)$$

$$x \cdot \dot{x} + y \cdot \dot{y} = 0 \quad (3.501)$$

$$\dot{x}^2 + x \cdot \ddot{x} + \dot{y}^2 + y \cdot \ddot{y} = 0 \quad (3.502)$$

$$\dot{v}_x = \ddot{x} \quad (3.503)$$

$$\dot{v}_y = \ddot{y} \quad (3.504)$$

Las cuatro ecuaciones añadidas al sistema introducen dos nuevas variables: $\ddot{x}$, $\ddot{y}$. Estas dos variables son evaluadas de las cuatro ecuaciones introducidas, de modo que las otras dos ecuaciones introducidas representan ligaduras que deben ser satisfechas por las variables del sistema DAE original. Estas dos ligaduras reducen el número de grados de libertad de 4 a 2.

Para analizar la causalidad computacional, debe escogerse qué 2 variables son variables de estado. Hay varias posibles selecciones de las variables de estado, por ejemplo: $\{x, v_x\}$, $\{x, v_y\}$, $\{y, v_x\}$, $\{y, v_y\}$.

La causalidad computacional, si se escogen $\{x, v_x\}$ como variables de estado, es:

$$x, v_x \quad \text{Variables de estado} \quad (3.505)$$

$$x^2 + ([y])^2 - L^2 = 0 \quad (3.506)$$

$$v_x = [derx] \quad (3.507)$$

$$x \cdot derx + y \cdot [dery] = 0 \quad (3.508)$$

$$[v_y] = dery \quad (3.509)$$

$$m \cdot deriv_x + \frac{\lambda}{L} \cdot x = 0 \quad (3.510)$$

$$m \cdot deriv_y + \frac{\lambda}{L} \cdot y + m \cdot g = 0 \quad (3.511)$$

$$derx^2 + x \cdot derderx + dery^2 + y \cdot derdery = 0 \quad (3.512)$$

$$deriv_x = derderx \quad (3.513)$$

$$deriv_y = derdery \quad (3.514)$$

Las últimas cinco ecuaciones forman un lazo algebraico del cual pueden calcularse las siguientes cinco incógnitas: $\{deriv_x, deriv_y, derderx, derdery, \lambda\}$. Manipulando el lazo, y eliminando del sistema las variables $\{derderx, derdery\}$, se obtiene:

$$x, v_x \quad \text{Variables de estado} \quad (3.515)$$

$$([y])^2 = L^2 - x^2 \quad (3.516)$$

$$[derx] = v_x \quad (3.517)$$

$$[dery] = -\frac{x}{y} \cdot derx \quad (3.518)$$

$$[v_y] = dery \quad (3.519)$$

$$[\lambda] = \frac{m}{L} \cdot (derx^2 + dery^2 - y \cdot g) \quad (3.520)$$

$$[deriv_x] = -\frac{\lambda}{m \cdot L} \cdot x \quad (3.521)$$

$$[deriv_y] = -\frac{\lambda}{m \cdot L} \cdot y - g \quad (3.522)$$

Solución al Ejercicio 3.8

Asignemos primeramente la causalidad computacional suponiendo que las dos variables que aparecen derivadas (v_1, v_2) son variables de estado. Para ello, se sustituyen las derivadas por variables auxiliares ($\dot{v}_1 \rightarrow \text{deriv}_1, \dot{v}_2 \rightarrow \text{deriv}_2$). Se obtiene el sistema siguiente:

$$v_1, v_2 \quad \text{Variables de estado} \quad (3.523)$$

$$e(t) = R \cdot i + v_1 \quad (3.524)$$

$$i = C_1 \cdot \text{deriv}_1 + C_2 \cdot \text{deriv}_2 \quad (3.525)$$

$$v_1 = v_2 \quad (3.526)$$

que está compuesto por 3 ecuaciones y donde hay 3 variables desconocidas: $\{\text{deriv}_1, \text{deriv}_2, i\}$. Obsérvese que:

- La incógnita i debe calcularse de la primera ecuación, ya que es la única incógnita que interviene en ella.
- La tercera ecuación es redundante, ya que no contiene ninguna incógnita.
- Existe una única ecuación para calcular las incógnitas $\{\text{deriv}_1, \text{deriv}_2\}$.

Dado que el problema de asignación de la causalidad computacional no tiene solución, el sistema o bien tiene índice superior o está mal planteado.

Derivando la Ec. (3.526), se obtiene una ligadura adicional sobre las incógnitas $\{\text{deriv}_1, \text{deriv}_2\}$. Añadiendo esta ligadura al sistema y asignando la causalidad computacional seleccionando v_1 como variable de estado, se obtiene

$$v_1 \quad \text{Variable de estado} \quad (3.527)$$

$$e(t) = R \cdot [i] + v_1 \quad (3.528)$$

$$v_1 = [v_2] \quad (3.529)$$

$$i = C_1 \cdot \text{deriv}_1 + C_2 \cdot \text{deriv}_2 \quad (3.530)$$

$$\text{deriv}_1 = \text{deriv}_2 \quad (3.531)$$

donde las dos últimas ecuaciones forman un lazo algebraico del cual deben evaluarse las incógnitas $\{deriv_1, deriv_2\}$. El sistema ordenado y resuelto es:

$$v_1 \quad \text{Variable de estado} \quad (3.532)$$

$$[i] = \frac{e(t) - v_1}{R} \quad (3.533)$$

$$[v_2] = v_1 \quad (3.534)$$

$$[deriv_1] = \frac{i}{C_1 + C_2} \quad (3.535)$$

$$[deriv_2] = deriv_1 \quad (3.536)$$

Solución al Ejercicio 3.9

Supongamos primeramente que las dos variables que aparecen derivadas son variables de estado. Sustituyendo sus derivadas por variables auxiliares, se obtiene:

$$x, z \quad \text{Variables de estado} \quad (3.537)$$

$$x = u(t) \quad (3.538)$$

$$z = \text{der}x \quad (3.539)$$

$$y = \frac{\text{der}z + \sin(x)}{\cos(x)} \quad (3.540)$$

La Ec. (3.538) es redundante y sólo se dispone de la Ec. (3.540) para calcular dos incógnitas: $\text{der}z$, y . Así pues, el problema de asignación de la causalidad computacional no tiene solución.

La derivada de la Ec. (3.538) es una ligadura adicional para $\dot{x}$. Por otra parte, la segunda derivada de la Ec. (3.538) y la primera derivada de la Ec. (3.539) suponen una ligadura para $\dot{z}$. Añadiendo estas ecuaciones al sistema, se obtiene

$$x = u(t) \quad (3.541)$$

$$\dot{x} = \dot{u}(t) \quad (3.542)$$

$$\ddot{x} = \ddot{u}(t) \quad (3.543)$$

$$z = \dot{x} \quad (3.544)$$

$$\dot{z} = \ddot{x} \quad (3.545)$$

$$y = \frac{\dot{z} + \sin(x)}{\cos(x)} \quad (3.546)$$

Estas tres ecuaciones introducen una variable nueva: $\ddot{x}$. Por tanto, reducen en dos los grados de libertad del sistema. Considerando que no hay variables de estado, el resultado de la asignación de la causalidad computacional es:

$$[x] = u(t) \quad (3.547)$$

$$[\text{der}x] = \dot{u}(t) \quad (3.548)$$

$$[\text{derder}x] = \ddot{u}(t) \quad (3.549)$$

$$[z] = \text{der}x \quad (3.550)$$

$$[\text{der}z] = \text{derder}x \quad (3.551)$$

$$[y] = \frac{\text{der}z + \sin(x)}{\cos(x)} \quad (3.552)$$

Solución al Ejercicio 3.10

Si se escoge v_1 como variable de estado y se sustituye su derivada por una variable auxiliar, se obtiene:

$$v_1 \quad \text{Variable de estado} \quad (3.553)$$

$$deriv_1 = e_1(t) - e_2(t) \cdot v_2 \quad (3.554)$$

$$v_1 = p \cdot v_2 \cdot v_3 \quad (3.555)$$

$$v_3 = g(v_2) \quad (3.556)$$

Las Ecs. (3.555) y (3.556) forman un lazo algebraico del cual pueden evaluarse $\{v_2, v_3\}$. Una vez calculadas estas dos variables, puede evaluarse $deriv_1$ de la Ec. (3.554). El problema de asignación de la causalidad computacional tiene solución, con lo cual el modelo tiene un grado de libertad. El sistema, con la causalidad computacional señalada y ordenado, es el siguiente:

$$v_1 \quad \text{Variable de estado} \quad (3.557)$$

$$\left. \begin{array}{l} v_1 = p \cdot v_2 \cdot v_3 \\ v_3 = g(v_2) \end{array} \right| \quad \text{Lazo algebraico: } v_2, v_3 \quad (3.558)$$

$$[deriv_1] = e_1(t) - e_2(t) \cdot v_2 \quad (3.559)$$

Con el fin de conseguir que la variable v_2 aparezca derivada, añadimos una ecuación muda al sistema, en la cual interviene $\dot{v}_2$ y una variable muda, a .

$$\dot{v}_1 = e_1(t) - e_2(t) \cdot v_2 \quad (3.560)$$

$$v_1 = p \cdot v_2 \cdot v_3 \quad (3.561)$$

$$v_3 = g(v_2) \quad (3.562)$$

$$\dot{v}_2 = a \quad (3.563)$$

Ahora aparecen dos variables derivadas en el sistema. Dado que éste tiene sólo un grado de libertad, el sistema formulado de esta forma tiene índice superior. Para reducir el índice, añadimos la derivada de las Ecs. (3.561) y (3.562) al sistema.

Para asignar la causalidad computacional, se selecciona v_2 como variable de estado y se sustituyen las derivadas por variables auxiliares. Resulta el siguiente sistema de 6 ecuaciones con 6 incógnitas: $\{v_1, deriv_1, deriv_2, v_3, deriv_3, a\}$.

$$v_2 \quad \text{Variable de estado} \quad (3.564)$$

$$deriv_1 = e_1(t) - e_2(t) \cdot v_2 \quad (3.565)$$

$$v_1 = p \cdot v_2 \cdot v_3 \quad (3.566)$$

$$v_3 = g(v_2) \quad (3.567)$$

$$deriv_2 = a \quad (3.568)$$

$$deriv_1 = p \cdot (deriv_2 \cdot v_3 + v_2 \cdot deriv_3) \quad (3.569)$$

$$deriv_3 = \frac{\partial g(v_2)}{\partial v_2} \cdot deriv_2 \quad (3.570)$$

El sistema, con la causalidad computacional señalada y ordenado, se muestra a continuación. La Ec. (3.576) puede eliminarse del modelo.

$$v_2 \quad \text{Variable de estado} \quad (3.571)$$

$$[deriv_1] = e_1(t) - e_2(t) \cdot v_2 \quad (3.572)$$

$$[v_3] = g(v_2) \quad (3.573)$$

$$[v_1] = p \cdot v_2 \cdot v_3 \quad (3.574)$$

$$\left. \begin{array}{l} deriv_1 = p \cdot (deriv_2 \cdot v_3 + v_2 \cdot deriv_3) \\ deriv_3 = \frac{\partial g(v_2)}{\partial v_2} \cdot deriv_2 \end{array} \right| \quad \text{Lazo: } deriv_2, deriv_3 \quad (3.575)$$

$$deriv_2 = [a] \quad (3.576)$$

Puede procederse de manera análoga para conseguir que v_3 sea la variable que aparece derivada en el sistema. Se añade al sistema una ecuación muda, que contenga $\dot{v}_3$ y una variable muda. Con ello, se fuerza a que el sistema tenga índice superior. Al reducir el índice, se escoge v_3 como variable de estado.

El sistema, al cual se ha añadido la ecuación muda es:

$$\dot{v}_1 = e_1(t) - e_2(t) \cdot v_2 \quad (3.577)$$

$$v_1 = p \cdot v_2 \cdot v_3 \quad (3.578)$$

$$v_3 = g(v_2) \quad (3.579)$$

$$\dot{v}_3 = a \quad (3.580)$$

Para reducir el índice, se añade al sistema la derivada de las Ecs. (3.578) y (3.579).

$$\dot{v}_1 = e_1(t) - e_2(t) \cdot v_2 \quad (3.581)$$

$$v_1 = p \cdot v_2 \cdot v_3 \quad (3.582)$$

$$v_3 = g(v_2) \quad (3.583)$$

$$\dot{v}_3 = a \quad (3.584)$$

$$\dot{v}_1 = p \cdot (\dot{v}_2 \cdot v_3 + v_2 \cdot \dot{v}_3) \quad (3.585)$$

$$\dot{v}_3 = \frac{\partial g(v_2)}{\partial v_2} \cdot \dot{v}_2 \quad (3.586)$$

Sustituyendo las derivadas por variables auxiliares y considerando que v_3 es la variable de estado, asignamos la causalidad computacional. Se muestra a continuación el modelo con la causalidad señalada y ordenado. La Ec. (3.592) podría ya en este punto ser eliminada del modelo.

$$v_3 \quad \text{Variable de estado} \quad (3.587)$$

$$v_3 = g([v_2]) \quad (3.588)$$

$$[deriv_1] = e_1(t) - e_2(t) \cdot v_2 \quad (3.589)$$

$$[v_1] = p \cdot v_2 \cdot v_3 \quad (3.590)$$

$$\left. \begin{array}{l} deriv_1 = p \cdot (deriv_2 \cdot v_3 + v_2 \cdot deriv_3) \\ deriv_3 = \frac{\partial g(v_2)}{\partial v_2} \cdot deriv_2 \end{array} \right| \quad \text{Lazo: } deriv_2, deriv_3 \quad (3.591)$$

$$deriv_3 = [a] \quad (3.592)$$

Solución al Ejercicio 3.11

A continuación se muestra la definición de dos funciones. La primera comprueba si la matriz pasada por argumento es tridiagonal. La segunda aplica el algoritmo.

```
esTridiagonal <- function(A) {
  if ( length(dim(A)) != 2      |
      dim(A)[1] != dim(A)[2]   |
      dim(A)[1] < 3 ) {
    return (FALSE)
  }
  N <- dim(A)[1]
  # Elementos por encima de la tridiagonal
  for ( i in 1:(N-2) )
    for ( j in (i+2):N )
      if ( A[i,j] != 0 ) return (FALSE)
  # Elementos por debajo de la tridiagonal
  for ( i in 3:N )
    for ( j in 1:(i-2) )
      if ( A[i,j] != 0 ) return (FALSE)
  return (TRUE)
}

algoritmoSistTridiag <- function( A, z ) {
  # Vectores a, b y c definidos en el algoritmo
  a <- diag(A)
  b <- c(NA, diag( A[2:N,1:(N-1)] ))
  c <- diag( A[1:(N-1),2:N] )
  # Algoritmo
  w <- a[1]
  if ( w == 0 ) return( rep(NA,length(a)) )
  y <- numeric(N) # Vector de N componentes cero
  y[1] <- z[1] / w
  v <- numeric(N) # Vector de N componentes cero
  for ( i in 2:N ) {
    v[i] <- c[i-1] / w
    w <- a[i] - b[i]*v[i]
    if ( w == 0 ) return( rep(NA,length(a)) )
    y[i] <- ( z[i] - b[i]*y[i-1] ) / w
  }
  for ( j in (N-1):1 )
    y[j] <- y[j] - v[j+1]*y[j+1]
  return ( y )
}
```

El siguiente código define el sistema lineal de ecuaciones, comprueba si la matriz **A** es tridiagonal y resuelve el sistema de ecuaciones. Finalmente calcula y muestra el residuo.


```

# -----
# Planteamiento del problema
# -----
z <- c(2, 5, -1, 3, 2.3)
N <- length(z) # Número de incógnitas
A <- matrix( c ( 5, 6, 0, 0, 0,
                 1, 43, 4, 0, 0,
                 0, 2, 3, 1, 0,
                 0, 0, 4, 6, 3,
                 0, 0, 0, 2, -5.2 ),
             nrow = N, ncol = N, byrow = TRUE )
# -----
# Solución y salida por consola
# -----
if ( esTridiagonal(A) ) {
  y <- algoritmoSistTridiag( A, z )
  print(y) # Solución
  res <- A %*% y - z # %*% realiza multiplicación matricial
  print(res) # Residuo
} else {
  print("A no es tridiagonal")
}

```

La ejecución del código anterior proporciona el resultado mostrado a continuación. El primer vector es la solución y el segundo el residuo.

```

[1] 0.1747822 0.1876815 -0.8112721 1.0584532 -0.0352103

      [,1]
[1,] 0.000000e+00
[2,] 8.881784e-16
[3,] 2.220446e-16
[4,] 4.440892e-16
[5,] 0.000000e+00

```

Solución al Ejercicio 3.12

```

f <- function( x ) {
  return ( x - cos(x) )
}

x0 <- 3      # Valores iniciales
x1 <- 4
eps <- 1E-6  # Tolerancia
N <- 100     # Número máximo de iteraciones
# -----
# Método de la secante
# -----
f0 <- f(x0)
f1 <- f(x1)
codigoSalida <- 1
resultIter <- data.frame( x = c(x0,x1),
                          f = c(f0,f1) )

for ( k in 1:N ) {
  if ( f1 == f0 ) {
    codigoSalida <- 2
    break # Sale del bucle for
  }
  dif <- (x1-x0)*f1/(f1-f0)
  x0 <- x1
  f0 <- f1
  x1 <- x1 - dif
  f1 <- f(x1)
  resultIter <- rbind(resultIter , c(x1,f1))
  if ( x1 != 0 & abs(dif/x1) < eps ) {
    codigoSalida <- 3
    break # Sale del bucle for
  }
}
print (
  switch ( codigoSalida,
    "Alcanzado limite de iteraciones",
    "Se satisface f1 = f0",
    "Se satisface abs(dif/x1)<eps"
  )
)
options(digits=16) # Número de dígitos decimales salida
resultIter

```

Ejecutando el código anterior se obtiene el resultado siguiente.

```
[1] "Se satisface abs(dif/x1)<eps"
```

```
> resultIter
```

	x	f
1	3.0000000000000000	3.989992496600445e+00
2	4.0000000000000000	4.653643620863612e+00
3	-3.0121837373972999	-2.020545392260347e+00
4	-0.8893137508834936	-1.519258894008570e+00
5	5.5445102581067909	4.805148949433702e+00
6	0.6562292612231042	-1.360692520500150e-01
7	0.7908407501179965	8.759291275330805e-02
8	0.7381227930476462	-1.610241747635688e-03
9	0.7390744259332729	-1.791979340048933e-05
10	0.7390851354915116	3.809728310599780e-09
11	0.7390851332151552	-9.103828801926284e-15

Solución al Ejercicio 3.13

```

f <- function( x ) {
  return ( x*x - 2 )
}

df <- function( x ) {
  return ( 2*x )
}

xk <- 2      # Valor inicial
N <- 6       # Número de iteraciones
fk <- f(xk)
resultIter <- data.frame( x = xk,
                          f = fk )

# -----
# Método de Newton
# -----
for ( k in 1:N ) {
  xk <- xk - fk / df(xk)
  fk <- f(xk)
  resultIter <- rbind(resultIter , c(xk,fk))
}
options(digits=16) # Número de dígitos decimales salida
resultIter

```

Ejecutando el código anterior se obtiene el resultado siguiente.

```

> resultIter
      x              f
1 2.0000000000000000 2.0000000000000000e+00
2 1.5000000000000000 2.5000000000000000e-01
3 1.4166666666666667 6.944444444444444e-03
4 1.414215686274510 6.007304882871267e-06
5 1.414213562374690 4.510614104447086e-12
6 1.414213562373095 4.440892098500626e-16
7 1.414213562373095 -4.440892098500626e-16

```

Solución al Ejercicio 3.14

```

f <- function( x ) {
  return ( cos(x) )
}

x  <- 3      # Valor inicial
eps <- 1E-6   # Tolerancia
N  <- 100    # Número máximo de iteraciones
# -----
# Método del punto fijo
# -----
fx <- f(x)
resultIter <- data.frame( x = x,
                          f = fx,
                          res = abs(x-fx) )

for ( k in 1:N ) {
  x  <- fx
  fx <- f(x)
  res <- abs(x-fx)
  resultIter <- rbind(resultIter , c(x,fx,res))
  if ( res < eps ) break # Sale del bucle for
}
ifelse ( k == N,
        print("Alcanzado limite de iteraciones"),
        print("Residuo menor que la tolerancia") )
options(digits=8) # Número de dígitos decimales salida
resultIter

```

A continuación se muestran las últimas cinco líneas del data frame `resultIter` obtenido ejecutando el código anterior.

```

> resultIter[(length(resultIter$x)-5):length(resultIter$x),]
      x      f      res
30 0.73908918 0.73908240 6.7804564e-06
31 0.73908240 0.73908697 4.5674003e-06
32 0.73908697 0.73908389 3.0766543e-06
33 0.73908389 0.73908597 2.0724720e-06
34 0.73908597 0.73908457 1.3960418e-06
35 0.73908457 0.73908551 9.4039067e-07

```

TEMA 4

MODELADO Y SIMULACIÓN DE SISTEMAS HÍBRIDOS

- 4.1. Introducción
- 4.2. Especificación de los modelos híbridos
- 4.3. Detección y ejecución de los eventos
- 4.4. Modelos con estructura variable
- 4.5. Modelado de sistemas híbridos en Modelica
- 4.6. Inicialización del modelo en Modelica
- 4.7. Experimentación con modelos en Modelica
- 4.8. Rand Model Designer
- 4.9. Lecturas recomendadas
- 4.10. Ejercicios de autocomprobación
- 4.11. Soluciones de los ejercicios

OBJETIVOS DOCENTES

Una vez estudiado el contenido del tema y realizados los ejercicios debería saber:

- Discutir y aplicar el formalismo OHM para la descripción de modelos híbridos. Discutir la relación entre esta especificación formal y el algoritmo de la simulación, y entre esta especificación formal y la descripción en lenguaje Modelica. Escribir la descripción formal y plantear el algoritmo de la simulación de modelos DAE híbrido de pequeña dimensión.
- Discutir los procedimientos empleados por los entornos de modelado de Modelica para la detección de los eventos y para la ejecución de eventos simultáneos y cadenas de eventos.
- Discutir qué es el chattering.
- Discutir qué son modelos con estructura variable, cómo son descritos en Modelica y cómo son simulados por los entornos de modelado de Modelica.
- Discutir el planteamiento del problema de inicialización de un modelo DAE híbrido. Aplicar los recursos que proporciona Modelica para definir la inicialización del modelo.
- Desarrollar modelos y librerías de modelos híbridos en lenguaje Modelica, de manera que se favorezca su reutilización.
- Experimentar con modelos escritos en Modelica empleando el lenguaje de comandos de Modelica.
- Discutir las capacidades del entorno de modelado Rand Model Designer para la descripción y simulación de modelos híbridos y para el desarrollo de laboratorios virtuales. Discutir las diferencias con Modelica en lo que respecta a la descripción y simulación de modelos híbridos.

4.1. INTRODUCCIÓN

Los modelos híbridos, que contienen variables continuas y eventos discretos, son muy difíciles de analizar si no es por medio de simulaciones. Un sistema híbrido general consiste en un proceso continuo interactuando con un autómata de estado finito.

El estado de un sistema híbrido evoluciona mediante el cambio continuo de sus estados continuos o mediante cambios instantáneos en su estado total, continuo y discreto, llamados eventos. El algoritmo de simulación de sistemas híbridos debe estar especialmente diseñado para conmutar entre la resolución del problema continuo y la ejecución de los eventos.

En esta sección se explica un formalismo para la descripción de modelos híbridos, que ha sido especialmente concebido para facilitar su simulación. También se abordan diferentes aspectos de la resolución numérica de modelos híbridos como son la detección y tratamiento de los eventos, la resolución del problema de reinicio y la simulación de los modelos con estructura variable. Asimismo, se muestra un problema que surge específicamente en la simulación de modelos híbridos: el chattering.

4.2. ESPECIFICACIÓN DE LOS MODELOS HÍBRIDOS

Existen varios formalismos para la descripción de modelos híbridos. El expuesto en esta sección está basado en el formalismo denominado OHM (Omola Hybrid Model), que fue propuesto como parte del desarrollo del lenguaje de modelado Omola a principios de la década de 1990. El lenguaje Omola se encuentra hoy en día en desuso. Algunos de sus desarrolladores participaron en la definición de Modelica, incorporándose en Modelica las ideas más notables que introdujo Omola.

4.2.1. Formalismo OHM

Un modelo DAE híbrido M puede representarse mediante la tupla

$$M = \langle \mathbf{q}, \mathbf{x}, \mathbf{y}, E, G, H, \Phi, \Delta \rangle \quad (4.1)$$

donde los componentes de la tupla se definen de la forma descrita a continuación.

$\mathbf{q} = \{q_1, \dots, q_{n_q}\}$ es el vector de **variables de tiempo discreto** del modelo. Las variables de tiempo discreto pueden ser de tipo real, entero, booleano o string.

$\mathbf{x} = \{x_1, \dots, x_{n_x}\}$ es el vector de **variables de estado de tiempo continuo** del modelo. Las variables de tiempo continuo sólo pueden ser de tipo real.

$\mathbf{y} = \{y_1, \dots, y_{n_y}\}$ es el vector de **variables algebraicas de tiempo continuo** del modelo. Como se ha indicado anteriormente, por ser variables de tiempo continuo, deben ser de tipo real.

$E = \{e_1, \dots, e_{n_e}\}$ es el conjunto de los **posibles tipos de eventos**.

$G = \{g_1, \dots, g_{n_g}\}$ es el conjunto de expresiones que define el **comportamiento de tiempo continuo del modelo**. Cada una de estas expresiones define una ecuación de tiempo continuo del modelo, de manera que las ecuaciones del modelo son:

$$g_i(\mathbf{x}, \dot{\mathbf{x}}, \mathbf{y}, \mathbf{q}_t, t) = 0 \quad \text{con } i = 1, \dots, n_g \quad (4.2)$$

Las ecuaciones de tiempo continuo deben satisfacerse en todos los instantes de la simulación, incluidas la inicialización y la ejecución de los eventos.

La notación $\mathbf{q}_t$ indica que el valor de las variables de tiempo discreto $\mathbf{q}$ es actualizado en los instantes de los eventos, siendo constante entre ellos. A efectos del integrador de la parte de tiempo continuo del modelo, las variables de tiempo discreto tienen un valor constante conocido. El sistema DAE que describe la parte de tiempo continuo del modelo está compuesto por el conjunto de ecuaciones (4.2). Puede representarse de la forma:

$$\mathbf{g}(\mathbf{x}, \dot{\mathbf{x}}, \mathbf{y}, \mathbf{q}_t, t) = 0 \quad (4.3)$$

Obsérvese que se asume que el sistema DAE ha sido manipulado simbólicamente, de manera que las variables que aparecen derivadas son las variables de estado.

$H = \{h_1, \dots, h_{n_h}\}$ es un conjunto de expresiones booleanas del tipo:

$$h_i(\mathbf{x}, \dot{\mathbf{x}}, \mathbf{y}, \mathbf{q}_t, t) \quad (4.4)$$

denominadas **invariantes** del modelo. Los invariantes dividen el espacio de estados en dos partes:

1. El conjunto de estados admisibles, donde todas las expresiones booleanas h_i valen *true*.
2. Los restantes estados, donde al menos una de ellas vale *false*.

Cuando el sistema cruza la frontera definida por los invariantes, saliendo del conjunto de estados admisibles, se **habilita** un evento. Así pues, los invariantes son expresiones booleanas chivatas que indican, al pasar de *true* a *false*, que las condiciones de disparo de su evento asociado se satisfacen.

Los complementos booleanos de los invariantes se llaman **condiciones de evento**. Una condición de evento que depende de al menos una variable de tiempo continuo se llama **condición continua**. Si sólo depende de variables de tiempo discreto, se llama **condición discreta**. Los eventos en el tiempo tienen invariantes del tipo:

$$h_i = t < t_i \quad (4.5)$$

donde t es la variable tiempo y t_i es la variable de tiempo discreto que representa el instante futuro en que el evento debe ser disparado.

$\Phi : H \rightarrow E$ es la aplicación que **asocia cada función invariante con un tipo de evento**: cuando h_i pasa de *true* a *false*, se habilita el evento asociado a h_i mediante la aplicación Φ .

$\Delta = \{\delta_1, \dots, \delta_{n_e}\}$ son un conjunto de expresiones vectoriales que describen el cambio en las variables del modelo producido por la **ejecución** de cada tipo de evento. Las ecuaciones vectoriales son de la forma

$$\delta_i(\mathbf{x}_a, \dot{\mathbf{x}}_a, \mathbf{y}_a, \mathbf{q}_a, \mathbf{x}_b, \dot{\mathbf{x}}_b, \mathbf{y}_b, \mathbf{q}_b, t_e) = 0 \quad (4.6)$$

donde $\{\mathbf{x}_a, \dot{\mathbf{x}}_a, \mathbf{y}_a, \mathbf{q}_a\}$ representa el valor de las variables después de la ejecución del evento, $\{\mathbf{x}_b, \dot{\mathbf{x}}_b, \mathbf{y}_b, \mathbf{q}_b\}$ representa el valor previo a la ejecución del evento y t_e es el valor de la variable tiempo en el instante de ejecución del evento. Los subíndices “a” y “b” provienen del inglés *after* y *before*, respectivamente. Obsérvese que en el instante t_e en que se dispara el evento se producirá en general un cambio discontinuo en el valor de las variables del modelo, las cuales pasan de valer $\{\mathbf{x}_b, \dot{\mathbf{x}}_b, \mathbf{y}_b, \mathbf{q}_b\}$ a valer $\{\mathbf{x}_a, \dot{\mathbf{x}}_a, \mathbf{y}_a, \mathbf{q}_a\}$.

La expresión vectorial δ_i está asociada el evento e_i . En el instante en que se dispara el evento e_i , conocido el valor de las variables antes del evento, se emplean la ecuación vectorial asociada al evento y el sistema DAE de tiempo continuo para calcular el valor de las variables tras el evento.

4.2.2. Especificación formal y algoritmo de la simulación

La especificación formal descrita anteriormente está muy próxima al algoritmo de la simulación empleado por los entornos de modelado de Modelica. Dicho algoritmo, que fue explicado a grandes rasgos en la Sección 1.4.7, consiste esencialmente en (véase nuevamente la Figura 1.12):

1. La **solución del problema de tiempo continuo**, que viene descrito por el sistema DAE (4.3). Como se mostró en el tema anterior, la resolución numérica del sistema DAE conlleva en general resolver sistemas de ecuaciones no lineales e integrar numéricamente. Las variables de tiempo discreto tienen un valor constante conocido durante la resolución del problema de tiempo continuo ya que, por definición, las variables de tiempo discreto sólo cambian su valor en los instantes en que se producen los eventos.
2. La **detección de los eventos**, para lo cual se observa el valor de los invariantes durante la resolución del problema de tiempo continuo. Cuando un invariante pasa de *true* a *false*, se suspende la solución del problema de tiempo continuo, y se localiza y ejecuta el evento correspondiente.
3. La **determinación del instante de disparo del evento**. Dado que la integración del sistema DAE avanza a saltos (determinados por el tamaño del paso de integración), en general se detectará el evento en un instante posterior a su instante de disparo. Se intenta por tanto buscar, dentro del último paso de integración, una acotación precisa del instante de disparo. Una vez determinado con la precisión deseada el intervalo dentro del cual está el instante de disparo, se asume que dicho instante coincide con el límite derecho del intervalo. Sea t_e dicho instante.
4. La **ejecución del evento**. Los nuevos valores de las variables deben ser consistentes con las ecuaciones que describen el comportamiento de tiempo continuo del sistema, ya que actúan como nuevos valores iniciales para el problema de tiempo continuo. La ejecución de un evento suele llamarse por ello **el problema del reinicio**. La ejecución del evento e_i consiste en calcular $\{\mathbf{x}_a, \dot{\mathbf{x}}_a, \mathbf{y}_a, \mathbf{q}_a\}$ resolviendo el siguiente sistema de ecuaciones:

$$\delta_i(\mathbf{x}_a, \dot{\mathbf{x}}_a, \mathbf{y}_a, \mathbf{q}_a, \mathbf{x}_b, \dot{\mathbf{x}}_b, \mathbf{y}_b, \mathbf{q}_b, t_e) = 0 \quad (4.7)$$

$$\mathbf{g}(\mathbf{x}_a, \dot{\mathbf{x}}_a, \mathbf{y}_a, \mathbf{q}_a, t_e) = 0 \quad (4.8)$$

El problema de reinicio expresado de esta manera es difícil de resolver, ya que contiene no sólo variables desconocidas de tiempo continuo, sino también variables desconocidas de tiempo discreto, entre las cuales puede haber variables de tipo entero y booleano.

El problema de reinicio es más sencillo de resolver si puede ser expresado de la forma siguiente:

$$\mathbf{q}_a = \delta_{i,1}(\mathbf{x}_b, \dot{\mathbf{x}}_b, \mathbf{y}_b, \mathbf{q}_b, t_e) \quad (4.9)$$

$$\mathbf{x}_a = \delta_{i,2}(\mathbf{q}_a, \mathbf{x}_b, \dot{\mathbf{x}}_b, \mathbf{y}_b, \mathbf{q}_b, t_e) \quad (4.10)$$

$$\mathbf{g}(\mathbf{x}_a, \dot{\mathbf{x}}_a, \mathbf{y}_a, \mathbf{q}_a, t_e) = 0 \quad (4.11)$$

Expresado de esta forma, el cálculo del nuevo estado discreto puede realizarse trivialmente de la Ec. (4.9), a partir del valor de las variables antes de la ejecución del evento. Conocido el nuevo estado discreto, puede calcularse el nuevo valor de las variables de estado de tiempo continuo empleando la Ec. (4.10). Finalmente, el sistema DAE (4.11) se emplea para evaluar el nuevo valor de las variables algebraicas y las derivadas.

4.2.3. Especificación formal y descripción en Modelica

El lenguaje de modelado que emplearemos en este texto para describir y simular los modelos híbridos es Modelica. Con el fin de aproximar aún más la especificación formal del modelo a las capacidades del lenguaje, facilitando que la descripción en Modelica pueda hacerse de manera más directa, veremos primeramente qué tipo de acciones permite Modelica realizar en los eventos. A continuación sugeriremos una manera de presentar la especificación formal que resulta útil en la práctica.

Los eventos en Modelica son descritos esencialmente mediante cláusulas *when* y sentencias *if*. A continuación se describen ambas brevemente (se volverá sobre ello en la Sección 4.5).

Cláusulas *when*, permiten describir cambios en el valor de las variables de tiempo discreto y de las variables de estado de tiempo continuo. Para ello, la cláusula *when* está compuesta de una **condición booleana** y unas ecuaciones que especifican los cambios en las variables. Las ecuaciones de la cláusula *when* se denominan **ecuaciones instantáneas** y se impone que se verifiquen cuando

la condición de la cláusula pasa de valer *false* a valer *true*. La sintaxis de la cláusula *when* es básicamente la siguiente:

$$\begin{array}{l} \mathbf{when} \text{ condición booleana } \mathbf{then} \\ \text{ecuaciones instantáneas} \\ \mathbf{end when}; \end{array} \quad (4.12)$$

Las ecuaciones instantáneas deben escribirse de manera que el problema de reinicio quede básicamente formulado como se muestra en las Ecs. (4.9) – (4.11):

- Las ecuaciones para evaluar las variables de tiempo discreto deben escribirse con la variable a evaluar despejada en el lado izquierdo de la igualdad. Los entornos de modelado de Modelica no permiten el establecimiento en el problema de reinicio de lazos algebraicos de los cuales deban evaluarse variables de tiempo discreto.
- El nuevo valor de las variables de estado de tiempo continuo debe expresarse mediante una función denominada **reinit**, en la cual el nuevo valor de la variable de estado debe aparecer despejado.

Sentencias if, permiten describir cambios en la estructura del modelo. El entorno de modelado traduce automáticamente las sentencias *if* a ecuaciones. Por ejemplo, la sentencia *if*

$$0 = \mathbf{if} \text{ cond } \mathbf{then} f1 \mathbf{else} f2; \quad (4.13)$$

indica que mientras *cond* valga *true* debe satisfacerse $0 = f1$ y mientras *cond* valga *false* debe satisfacerse $0 = f2$. La sentencia *if* anterior es traducida automáticamente por el entorno de modelado a la ecuación

$$0 = \alpha \cdot f_1 + (1 - \alpha) \cdot f_2 \quad (4.14)$$

donde α es una variable muda de tiempo discreto que vale uno cuando *cond* vale *true* y vale cero cuando *cond* vale *false*. Los cambios en el valor de α pueden describirse mediante eventos. La Ec. (4.14), que representa la sentencia *if*, sería parte del elemento *G* de la tupla y la variable muda α sería parte de **q**.

Con el fin de aproximar la especificación formal del modelo híbrido a su descripción en Modelica, en la práctica realizaremos la especificación de la manera siguiente:

1. Indicaremos las variables de tiempo discreto ($\mathbf{q}$), las variables de estado de tiempo continuo ($\mathbf{x}$) y las variables algebraicas de tiempo continuo ($\mathbf{y}$). Las variables mudas introducidas al traducir las sentencias if a ecuaciones se incluyen en $\mathbf{q}$.
2. Escribiremos las ecuaciones (en lugar de las expresiones) que describe el comportamiento de tiempo continuo, incluyendo las ecuaciones equivalentes a las sentencias if.
3. Describiremos los elementos de la tupla E , H , Φ y Δ , empleando para ello una tabla con las características siguientes.

- La tabla tiene n_e filas: una por cada tipo de evento del conjunto E . En la primera columna de la tabla se muestra la numeración consecutiva de sus filas. La fila i -ésima corresponde con el tipo de evento e_i .
- En la segunda columna de la tabla se muestra la condición de disparo del tipo de evento (en lugar de invariante), es decir, los complementos booleanos de H .

La aplicación Φ asocia el invariante con el tipo de evento. Dado que la condición de disparo del evento i -ésimo se muestra en la i -ésima fila de la tabla, esta información queda recogida en la tabla.

- En la tercera columna se muestran las ecuaciones (en lugar de la expresión vectorial) que describen el cambio para ese tipo de evento en las variables de tiempo discreto y en las variables de estado de tiempo continuo.

Cuando no se indica explícitamente que una variable de estado de tiempo continuo cambia, se supone que dicha variable no cambia de valor al ejecutarse el evento. Lo mismo rige para las variables de tiempo discreto.

Veamos a continuación varios ejemplos que ilustran esta forma de presentar la especificación formal del modelo híbrido.

4.2.4. Ejemplo 1: rebote de una pelota

Supongamos que se desea modelar el movimiento vertical de una pelota que cae bajo la acción de la gravedad y que rebota al llegar al suelo. La aceleración gravitatoria se considera constante e igual a $g = 9.8 \text{ m/s}^2$. Sea $h(t)$ la altura de la pelota en el instante t y sea $v(t)$ su velocidad en dicho instante.

Las referencias para la posición y la velocidad son las siguientes. Se supone que el suelo se encuentra en reposo a una altura cero. Las posiciones por encima del suelo corresponden a alturas positivas y las posiciones por debajo del suelo a alturas negativas. La velocidad tiene sentido positivo cuando la pelota asciende y negativo cuando cae.

Las ecuaciones que describen la caída de la pelota son:

$$\frac{dh}{dt} = v \quad (4.15)$$

$$\frac{dv}{dt} = -g \quad (4.16)$$

En el instante inicial la altura y velocidad de la pelota son h_0 y v_0 , respectivamente.

Cuando la pelota llega al suelo, rebota elásticamente. La velocidad de la pelota cambia instantáneamente al contactar con el suelo: se invierte el sentido de la velocidad y su módulo se reduce en un 20 %. Llamando v_{antes} a la velocidad antes del rebote y v_{despues} a la velocidad después del rebote, en el instante del rebote se verifica:

$$v_{\text{despues}} = -0.8 \cdot v_{\text{antes}} \quad (4.17)$$

Por ejemplo, si la velocidad justo antes de que se produzca el rebote vale -10 m/s, la velocidad justo después del rebote será 8 m/s. El signo negativo de la velocidad indica que la pelota está descendiendo y el signo positivo indica que la pelota está ascendiendo.

Como veremos más adelante, en lenguaje Modelica la Ec. (4.17) puede describirse mediante una **cláusula when**. Empleando una sintaxis similar a la de Modelica para describir el evento, la descripción de éste sería:

$$\begin{aligned} &\mathbf{when} \ h \leq 0 \ \mathbf{then} \\ &\quad \mathbf{reinit}(v, -0.8 \cdot v); \\ &\mathbf{end \ when;} \end{aligned} \quad (4.18)$$

La función **reinit** admite dos argumentos. El primero es una variable de estado de tiempo continuo. El segundo es una expresión de tipo real. Al ejecutar la *sentencia*

Tabla 4.1: Tabla de eventos del modelo de rebote de una pelota.

	Condición de evento	Ecuaciones del evento
1	$h \leq 0$	$v_a = -0.8 \cdot v_b$ $h_a = 0$

reinit, se evalúa la expresión de su segundo argumento y se asigna dicho valor a la variable de estado pasada como primer argumento.

La altura de la pelota en el instante de ejecución del evento estará próxima a cero, pero probablemente no sea exactamente igual a cero. Si deseamos asignar el valor cero a la altura al ejecutar el evento, deberemos añadir la correspondiente reinicialización de la variable de estado:

$$\begin{aligned}
 &\mathbf{when} \ h \leq 0 \ \mathbf{then} \\
 &\quad \mathbf{reinit}(v, -0.8 \cdot v); \\
 &\quad \mathbf{reinit}(h, 0); \\
 &\mathbf{end} \ \mathbf{when};
 \end{aligned} \tag{4.19}$$

La expresión booleana ($h \leq 0$) es la condición de la cláusula *when* y las dos sentencias *reinit* son las ecuaciones instantáneas de la cláusula.

La especificación formal de este modelo puede expresarse de la manera siguiente. Los vectores de variables del modelo son:

$$\mathbf{q} = \{ \} \tag{4.20}$$

$$\mathbf{x} = \{h, v\} \tag{4.21}$$

$$\mathbf{y} = \{ \} \tag{4.22}$$

Dado que el modelo no contiene variables de tiempo discreto, ni tampoco variables algebraicas de tiempo continuo, dichos vectores de variables no tienen componentes. Las ecuaciones de tiempo continuo son las Ecs. (4.15) y (4.16). La tabla de eventos se muestra en la Tabla 4.1.

Obsérvese que el problema de reinicio para el evento consiste en calcular $\{h_a, v_a, derh_a, deriv_a\}$ del sistema de cuatro ecuaciones formado por las dos ecuaciones del evento (véase la segunda columna de la Tabla 4.1), y las dos ecuaciones que describen del comportamiento de tiempo continuo, esto es, las Ecs. (4.15) y (4.16).

Las ecuaciones del problema de reinicio para el único tipo de evento que hay en este modelo son las siguientes:

$$v_a = -0.8 \cdot v_b \quad (4.23)$$

$$h_a = 0 \quad (4.24)$$

$$\text{der}h_a = v_a \quad (4.25)$$

$$\text{der}v_a = -g \quad (4.26)$$

4.2.5. Ejemplo 2: depósito con válvula de desagüe

Consideremos un sistema como el representado en la Figura 4.1, que está compuesto por un depósito, una tubería de entrada de líquido al depósito y una tubería de desagüe con una válvula. La tubería de entrada está situada encima del depósito y la tubería de salida está situada a media altura.

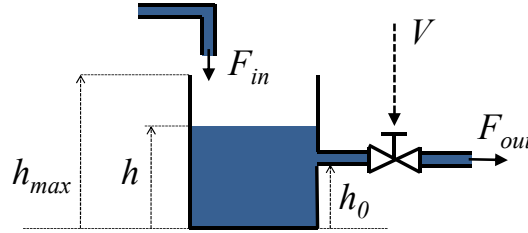


Figura 4.1: Esquema del depósito con válvula de desagüe.

Se supone que el caudal de entrada de líquido (F_{in}) y la apertura de la válvula (V) son funciones conocidas del tiempo:

$$F_{in} = f_1(t) \quad (4.27)$$

$$V = f_2(t) \quad (4.28)$$

La apertura de la válvula puede tomar valores comprendidos entre 0 (no circula líquido) y 1 (está completamente abierta).

La tubería de desagüe está situada a una altura h_0 , con lo cual sólo circula líquido por esta tubería si la altura del líquido en el depósito (h) es mayor que h_0 y si la válvula se encuentra abierta. El caudal de salida es:

$$F_{out} = \begin{cases} 0 & \text{si } h \leq h_0 \\ K \cdot V \cdot \sqrt{h - h_0} & \text{si } h > h_0 \end{cases} \quad (4.29)$$

donde K y h_0 son parámetros de valor conocido. Esta ecuación con dos ramas, que puede expresarse en Modelica mediante una *sentencia if*, puede ser equivalentemente descrita mediante la ecuación siguiente:

$$F_{out} = \alpha \cdot K \cdot V \cdot \sqrt{h - h_0} \quad (4.30)$$

donde α es una variable muda de tiempo discreto, que vale cero cuando $h \leq h_0$ y vale uno cuando $h > h_0$.

Asumiendo que la densidad del líquido es constante, el balance de masa en el depósito puede escribirse de la forma siguiente:

$$A \cdot \frac{dh}{dt} = F_{in} - F_{out} \quad (4.31)$$

donde el área A del depósito es un parámetro de valor conocido.

Se define una altura máxima de seguridad del líquido en el depósito, superada la cual el sistema genera una señal de alarma descrita mediante una variable booleana. Esta altura máxima tiene un valor conocido y constante: h_{max} . Mientras el nivel de líquido h sea superior al valor h_{max} la variable *Alarma* vale *true*, en caso contrario vale *false*.

A continuación se muestra la especificación formal del modelo. Las variables del modelo son:

$$\mathbf{q} = \{\alpha, Alarma\} \quad (4.32)$$

$$\mathbf{x} = \{h\} \quad (4.33)$$

$$\mathbf{y} = \{F_{in}, F_{out}, V\} \quad (4.34)$$

Las ecuaciones que describen el comportamiento de tiempo continuo son las Ecs. (4.27), (4.28), (4.30) y (4.31). La tabla de eventos del modelo se muestra en la Tabla 4.2. Obsérvese que se han descrito cuatro tipos diferentes de evento.

Si no se indica explícitamente que una determinada variable de estado de tiempo continuo cambia al ejecutarse un evento, entonces se asume que no cambia, añadién-

Tabla 4.2: Tabla de eventos del modelo del depósito con válvula de desagüe.

	Condición de evento	Ecuaciones del evento
1	$\alpha == 0$ and $h > h_0$	$\alpha_a = 1$
2	$\alpha == 1$ and $h \leq h_0$	$\alpha_a = 0$
3	$Alarma == 0$ and $h > h_{max}$	$Alarma_a = 1$
4	$Alarma == 1$ and $h \leq h_{max}$	$Alarma_a = 0$

dose la correspondiente ecuación (valor *after* = valor *before*) al plantear el problema de reinicio. Lo mismo aplica a las variables de tiempo discreto.

Así, por ejemplo, el problema de reinicio cuando se dispara el primer tipo de evento, asumiendo que su instante de disparo es t_e , es:

$$\alpha_a = 1 \quad (4.35)$$

$$Alarma_a = Alarma_b \quad (4.36)$$

$$h_a = h_b \quad (4.37)$$

$$F_{in_a} = f_1(t_e) \quad (4.38)$$

$$V_a = f_2(t_e) \quad (4.39)$$

$$F_{out_a} = \alpha_a \cdot K \cdot V_a \cdot \sqrt{h_a - h_0} \quad (4.40)$$

$$A \cdot derh_a = F_{in_a} - F_{out_a} \quad (4.41)$$

La Ec. (4.35) es la especificada en la tabla de eventos. Las Ecs. (4.36) y (4.37) son añadidas al sistema para indicar que las variables *Alarma* y *h* no cambian de valor al ejecutar el evento. Las Ecs. (4.38) – (4.41) son las que describen el comportamiento de tiempo continuo.

De este sistema de siete ecuaciones debe evaluarse el valor tras el disparo del evento de las variables de tiempo discreto, las variables de estado, sus derivadas y las variables algebraicas. Es decir, deben calcularse las siguientes siete variables: $\{\alpha_a, Alarma_a, h_a, derh_a, F_{in_a}, V_a, F_{out_a}\}$.

4.2.6. Ejemplo 3: dos depósitos conectados mediante una válvula

Consideremos un modelo muy sencillo de un sistema compuesto por dos depósitos de líquido conectados mediante una válvula. En la Figura 4.2 se muestra un diagrama del sistema, así como las ecuaciones que describen cada uno de sus tres componentes.

En el primer depósito hay una masa m_1 de líquido, que se encuentra a temperatura T_1 . En el segundo depósito hay una masa m_2 de líquido, que se encuentra a una temperatura T_2 . La evolución en la masa de líquido almacenado en cada tanque se calcula aplicando el balance de masa. La aceleración gravitatoria, g , la sección de los depósitos, S_1 y S_2 , y el coeficiente de la válvula, K_v , son parámetros del modelo.

La masa de líquido que fluye entre los dos depósitos por unidad de tiempo, F^m , depende de un parámetro característico K_v , de la apertura θ de la válvula y de la diferencia presión entre los dos extremos de la válvula. El sentido del flujo está determinado por cuál de las dos presiones sea mayor. Si la presión en la base del primer depósito es mayor, el líquido circulará del primer depósito al segundo. Si la presión en la base del segundo depósito es mayor, el líquido circulará del segundo depósito al primero. Si ambas presiones son iguales, no se producirá flujo. Se adopta el criterio de considerar que el flujo másico es positivo cuando lleva sentido saliente del primer depósito y entrante en el segundo (véase la Figura 4.2).

La evolución en la temperatura del líquido almacenado en cada depósito se calcula aplicando el balance de energía. La temperatura del líquido circulante es igual a la temperatura del líquido almacenado en el depósito situado aguas arriba. Se supone que la capacidad calorífica del líquido, C_p , es constante.

La apertura θ de la válvula es una variable de tiempo discreto. Debe asignársele un valor inicial, que se mantiene constante hasta el instante t_0 , que es parámetro del modelo. En ese instante, la apertura de la válvula cambia instantáneamente de su valor inicial al valor θ_0 , el cual es también un parámetro del modelo. Este cambio instantáneo en el valor de la variable θ se describe en la Figura 4.2 mediante una *cláusula when*, empleando una sintaxis similar a la de Modelica.

Las ecuaciones con dos ramas para el cálculo de la temperatura (T_f) y el flujo másico (F^m) en el modelo de la válvula pueden ser descritas de la forma

$$T_f = \alpha \cdot T_1 + (1 - \alpha) \cdot T_2 \quad (4.42)$$

$$F^m = \alpha \cdot K_v \cdot \theta \cdot \sqrt{p_1 - p_2} + (1 - \alpha) \cdot (-K_v \cdot \theta \cdot \sqrt{p_2 - p_1}) \quad (4.43)$$

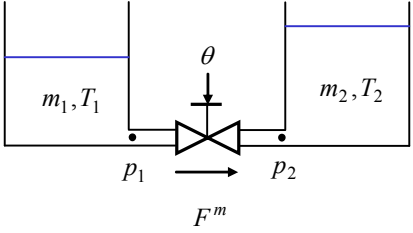
	Modelo de la válvula $T_f = \begin{cases} T_1 & \text{si } p_1 > p_2 \\ T_2 & \text{en caso contrario} \end{cases}$ $F^m = \begin{cases} K_v \theta \sqrt{p_1 - p_2} & \text{si } p_1 > p_2 \\ -K_v \theta \sqrt{p_2 - p_1} & \text{en caso contrario} \end{cases}$ when $t > t_0$ then $\theta = \theta_0$ end when;
Modelo del primer depósito $\frac{dm_1}{dt} = -F^m$ $m_1 C_p \frac{dT_1}{dt} = -F^m C_p (T_f - T_1)$ $p_1 = \frac{m_1 g}{S_1}$	Modelo del segundo depósito $\frac{dm_2}{dt} = F^m$ $m_2 C_p \frac{dT_2}{dt} = F^m C_p (T_f - T_2)$ $p_2 = \frac{m_2 g}{S_2}$

Figura 4.2: Dos depósitos conectados mediante una válvula.

introduciendo una variable muda α que valga uno cuando $p_1 > p_2$ y cero en caso contrario.

A continuación se muestra la especificación formal del modelo. Las variables del modelo son:

$$\mathbf{q} = \{\theta, \alpha\} \quad (4.44)$$

$$\mathbf{x} = \{m_1, m_2, T_1, T_2\} \quad (4.45)$$

$$\mathbf{y} = \{F^m, T_f, p_1, p_2\} \quad (4.46)$$

Las ecuaciones que describen el comportamiento de tiempo continuo son las siguientes:

Tabla 4.3: Tabla de eventos del modelo de dos depósitos y una válvula.

	Condición de evento	Ecuaciones del evento
1	$\alpha == 0$ and $p_1 > p_2$	$\alpha_a = 1$
2	$\alpha == 1$ and $p_1 \leq p_2$	$\alpha_a = 0$
3	$t > t_0$	$\theta_a = \theta_0$

$$T_f = \alpha \cdot T_1 + (1 - \alpha) \cdot T_2 \quad (4.47)$$

$$F^m = \alpha \cdot K_v \cdot \theta \cdot \sqrt{p_1 - p_2} + (1 - \alpha) \cdot (-K_v \cdot \theta \cdot \sqrt{p_2 - p_1}) \quad (4.48)$$

$$\frac{dm_1}{dt} = -F^m \quad (4.49)$$

$$m_1 \cdot C_p \cdot \frac{dT_1}{dt} = -F^m \cdot C_p \cdot (T_f - T_1) \quad (4.50)$$

$$p_1 = \frac{m_1 \cdot g}{S_1} \quad (4.51)$$

$$\frac{dm_2}{dt} = F^m \quad (4.52)$$

$$m_2 \cdot C_p \cdot \frac{dT_2}{dt} = F^m \cdot C_p \cdot (T_f - T_2) \quad (4.53)$$

$$p_2 = \frac{m_2 \cdot g}{S_2} \quad (4.54)$$

La tabla de eventos del modelo se muestra en la Tabla 4.3. Obsérvese que se han descrito tres tipos diferentes de eventos. Los dos primeros señalan el cambio en el sentido del flujo a través de la válvula. El tercero señala el instante en que cambia el valor de la apertura de la válvula.

Complicuemos un poco el modelo de la válvula. En el modelo mostrado en la Figura 4.2 se ha asumido que el flujo másico de líquido es una función algebraica de la diferencia de presiones. Esto es equivalente a considerar que la inercia del fluido que circula a través de la válvula es despreciable. Supongamos que modificamos el modelo de la válvula, de manera que se tenga en cuenta la inercia del líquido que circula por ella. Empleando *cláusulas when* para describir la asignación de valor a la apertura de la válvula y la reinicialización del momento lineal, la descripción del modelo sería la siguiente:

$$T_f = \begin{cases} T_1 & \text{si } P > 0 \\ T_2 & \text{en caso contrario} \end{cases} \quad (4.55)$$

$$\frac{dP}{dt} = K_v^* \cdot \theta^2 \cdot (p_1 - p_2) \quad (4.56)$$

$$P = F^m \cdot L \quad (4.57)$$

$$\begin{aligned} &\mathbf{when } t > t_0 \mathbf{ then} \\ &\quad \theta = \theta_0 \\ &\mathbf{end when;} \end{aligned} \quad (4.58)$$

$$\begin{aligned} &\mathbf{when } \theta \leq 0 \mathbf{ then} \\ &\quad \mathbf{reinit } (P, 0); \\ &\mathbf{end when;} \end{aligned} \quad (4.59)$$

donde la variable P representa el momento lineal del agua contenida en la conducción situada entre los dos depósitos y el parámetro L la longitud efectiva de la misma. En este caso, el parámetro característico de la válvula es K_v^* . Cuando la apertura de la válvula se hace cero, es preciso asignar al momento lineal P el valor cero, forzándose un cambio instantáneo en el estado continuo del sistema. Esto es descrito mediante la *cláusula when* de la Ec. (4.59).

Se muestra a continuación la especificación formal del modelo de los dos depósitos y la válvula, considerando la inercia del líquido que circula a través de esta última. Las variables del modelo son:

$$\mathbf{q} = \{\theta, \alpha\} \quad (4.60)$$

$$\mathbf{x} = \{m_1, m_2, T_1, T_2, P\} \quad (4.61)$$

$$\mathbf{y} = \{F^m, T_f, p_1, p_2\} \quad (4.62)$$

donde α es una variable muda introducida al traducir la ecuación con dos ramas (4.55), que describe la temperatura T_f del líquido que circula a través de la válvula. Las ecuaciones que representan el comportamiento de tiempo continuo son las siguientes:

Tabla 4.4: Tabla de eventos del modelo de dos depósitos y una válvula, considerando la inercia del líquido que circula a través de la válvula.

	Condición de evento	Ecuaciones del evento
1	$\alpha == 0$ and $P > 0$	$\alpha_a = 1$
2	$\alpha == 1$ and $P \leq 0$	$\alpha_a = 0$
3	$t > t_0$	$\theta_a = \theta_0$
4	$\theta \leq 0$	$P_a = 0$

$$T_f = \alpha \cdot T_1 + (1 - \alpha) \cdot T_2 \quad (4.63)$$

$$\frac{dP}{dt} = K_v^* \cdot \theta^2 \cdot (p_1 - p_2) \quad (4.64)$$

$$P = F^m \cdot L \quad (4.65)$$

$$\frac{dm_1}{dt} = -F^m \quad (4.66)$$

$$m_1 \cdot C_p \cdot \frac{dT_1}{dt} = -F^m \cdot C_p \cdot (T_f - T_1) \quad (4.67)$$

$$p_1 = \frac{m_1 \cdot g}{S_1} \quad (4.68)$$

$$\frac{dm_2}{dt} = F^m \quad (4.69)$$

$$m_2 \cdot C_p \cdot \frac{dT_2}{dt} = F^m \cdot C_p \cdot (T_f - T_2) \quad (4.70)$$

$$p_2 = \frac{m_2 \cdot g}{S_2} \quad (4.71)$$

La tabla de eventos del modelo se muestra en la Tabla 4.4. En este caso se han definido cuatro tipos diferentes de eventos. Los dos primeros señalan el cambio en el sentido del flujo a través de la válvula. El tercero señala el instante en que cambia el valor de la apertura de la válvula. El cuarto reinicializa a cero el momento lineal del líquido que fluye a través de la válvula cuando ésta se cierra.

4.3. DETECCIÓN Y EJECUCIÓN DE LOS EVENTOS

En esta sección se analiza un poco más en profundidad la detección y ejecución de los eventos. También se explica el concepto de función de cruce y se muestra cómo

los entornos de modelado de Modelica emplean funciones de cruce para detectar los eventos. Finalmente, se explica el fenómeno del chattering.

4.3.1. Eventos simultáneos

La ejecución de un evento puede motivar el disparo, en ese mismo instante, de otro evento. Esto ocurre cuando la solución del problema de reinicio no satisface uno de los invariantes, lo que hace que inmediatamente se ejecute el evento correspondiente a dicho invariante. De esta manera, pueden irse disparando eventos en secuencia hasta que todos los invariantes son satisfechos, reiniciándose entonces la solución del problema continuo. La ejecución de una secuencia de eventos se denomina una **cadena de eventos**

Cuando no se satisface un único invariante, no existe duda acerca de la forma de proceder. Dado que está habilitado un único evento (el asociado al invariante que no se satisface), simplemente hay que ejecutarlo, para lo cual hay que plantear y resolver un nuevo problema de reinicio. Si la solución a este problema satisface todos los invariantes, se reanuda la solución del problema continuo. Si no satisface un invariante, se ejecuta el evento habilitado y así sucesivamente, hasta que todos los invariantes sean satisfechos. El problema de reinicio para el cual todos los invariantes son satisfechos se emplea entonces como condición inicial para la reanudación de la solución del problema de tiempo continuo.

Sin embargo, puede suceder que al ir resolviendo el problema de tiempo continuo se detecte en un cierto instante que se han habilitado simultáneamente varios eventos. También puede suceder que, en uno o varios pasos de una cadena de eventos, el problema de reinicio no satisfaga varios invariantes. En ambas situaciones es necesario establecer un criterio que permita decidir en qué orden se ejecutan estos eventos habilitados simultáneamente.

El orden de ejecución de los eventos simultáneos es en algunos casos irrelevante. Esto ocurre típicamente cuando los eventos afectan a diferentes partes del modelo, sin ninguna interacción entre ellas. Sin embargo, en ocasiones el orden de disparo sí es importante, ya que afecta a la solución del problema de reinicio final de la cadena de eventos. Por ejemplo, consideremos el modelo descrito mediante las Ecs. (4.72) – (4.76).

$$\frac{dx_1}{dt} = 1 \quad (4.72)$$

$$\frac{dx_2}{dt} = -1 \quad (4.73)$$

$$x_1 + x_2 = y \quad (4.74)$$

$$\begin{aligned} &\mathbf{when} \ x_1 \geq 0.5 \cdot y \ \mathbf{then} \\ &\quad \mathbf{reinit} \ (x_1, 0) \\ &\mathbf{end \ when;} \end{aligned} \quad (4.75)$$

$$\begin{aligned} &\mathbf{when} \ x_2 \leq 0.5 \cdot y \ \mathbf{then} \\ &\quad \mathbf{reinit} \ (x_2, 10) \\ &\mathbf{end \ when;} \end{aligned} \quad (4.76)$$

Si se asignan a las variables de estado los valores iniciales

$$x_1(0) = 0 \qquad x_2(0) = 10 \quad (4.77)$$

entonces en el instante $t = 5$ se habilitan los dos eventos simultáneamente, ya que

$$x_1(5) = 5 \qquad x_2(5) = 5 \qquad y(5) = 10 \quad (4.78)$$

Si se dispara en primer lugar el evento cuya condición es $(x_1 \geq 0.5 \cdot y)$, se obtienen los valores mostrados en la Tabla 4.5 para las variables x_1 , x_2 e y , y para las condiciones de evento. Obsérvese que en la solución del problema de reinicio la condición de evento $(x_2 \leq 0.5 \cdot y)$ vale *false*, con lo cual su evento asociado no está habilitado y, por consiguiente, no se dispara.

Si por el contrario se ejecuta en primer lugar el evento definido por la condición $(x_2 \leq 0.5 \cdot y)$, se obtienen los valores mostrados en la Tabla 4.6. En este caso, en la solución del problema de reinicio la condición $(x_1 \geq 0.5 \cdot y)$ vale *false*, con lo cual su evento asociado no está habilitado. Como se ve, el estado del sistema tras el evento depende de la elección de cuál de los eventos habilitados se dispara en primer lugar.

Existen varios **métodos de decisión del orden de ejecución de los eventos**, algunos deterministas y otros no deterministas. Un método consiste en dispararlos de acuerdo al orden en que han sido definidos en el código del modelo. Los eventos planificados en el tiempo pueden ser ordenados desde un principio, estableciéndose una cola.

Tabla 4.5: Resultado del evento cuya condición es $(x_1 \geq 0.5 \cdot y)$.

Antes de la ejecución del evento	Después de la ejecución del evento
$x_1 = 5$	$x_1 = 0$
$x_2 = 5$	$x_2 = 5$
$y = 10$	$y = 5$
$(x_1 \geq 0.5 \cdot y) = \text{true}$	$(x_1 \geq 0.5 \cdot y) = \text{false}$
$(x_2 \leq 0.5 \cdot y) = \text{true}$	$(x_2 \leq 0.5 \cdot y) = \text{false}$

Tabla 4.6: Resultado del evento cuya condición es $(x_2 \leq 0.5 \cdot y)$.

Antes de la ejecución del evento	Después de la ejecución del evento
$x_1 = 5$	$x_1 = 5$
$x_2 = 5$	$x_2 = 10$
$y = 10$	$y = 15$
$(x_1 \geq 0.5 \cdot y) = \text{true}$	$(x_1 \geq 0.5 \cdot y) = \text{false}$
$(x_2 \leq 0.5 \cdot y) = \text{true}$	$(x_2 \leq 0.5 \cdot y) = \text{false}$

Tabla 4.7: Resultado de la ejecución simultánea de los dos eventos habilitados.

Antes de la ejecución del evento	Después de la ejecución del evento
$x_1 = 5$	$x_1 = 0$
$x_2 = 5$	$x_2 = 10$
$y = 10$	$y = 10$
$(x_1 \geq 0.5 \cdot y) = \text{true}$	$(x_1 \geq 0.5 \cdot y) = \text{false}$
$(x_2 \leq 0.5 \cdot y) = \text{true}$	$(x_2 \leq 0.5 \cdot y) = \text{false}$

```

model dispEvent
  Real x1(start=0, fixed=true);
  Real x2(start=10, fixed=true);
  Real y;
equation
  der(x1) = 1;
  der(x2) = -1;
  x1 + x2 = y;
  when x1 >= 0.5*y then
    reinit(x1,0);
  end when;
  when x2 <= 0.5*y then
    reinit(x2,10);
  end when;
end dispEvent;

```

Código 4.1: Modelo con disparo simultáneo de dos eventos.

Supóngase, por ejemplo, que se han habilitado simultáneamente los eventos e_1 y e_2 . Se ejecuta primero el evento e_1 , ya que ha sido definido antes que el e_2 . El problema de reinicio satisface una de las tres condiciones siguientes:

1. Si todos los invariantes son satisfechos, entonces reiniciarse la solución del problema continuo.
2. Si únicamente hay un invariante no satisfecho, entonces se ejecuta el evento asociado a este invariante.
3. Si varios invariantes no son satisfechos, entonces se ejecuta, entre los eventos asociados a esos invariantes, el de menor orden de definición.

Por tanto, el algoritmo empleado sería:

Algoritmo:

- Paso 1. Disparar el evento de menor orden de definición de entre los habilitados. Se dice que un evento está habilitado cuando su invariante vale *false*.
- Paso 2. Comprobar si hay eventos habilitados. En caso afirmativo, ir al Paso 1. En caso contrario, reanudar la solución del problema de tiempo continuo.

Otro algoritmo diferente sería:

Algoritmo:

- Paso 1. Determinar y ordenar, de acuerdo con el orden de definición, el conjunto de eventos habilitados, E' .
- Paso 2. Si el conjunto E' es vacío, reanudar la solución del problema continuo, finalizando el presente algoritmo.
- Paso 3. Disparar el primer evento del conjunto E' .
- Paso 4. Examinar el siguiente evento del conjunto ordenado E' . Si aún no ha sido ejecutado y continúa habilitado, se ejecuta.
- Paso 5. Si han sido examinados todos los eventos de E' , ir al Paso 1. En caso contrario, ir al Paso 4.

Es posible formular modelos en los cuales el algoritmo de disparo de eventos nunca finalice. No hay modo de detectar este problema analíticamente, sin ejecutar la simulación, excepto en caso muy sencillos.

El esquema anterior de ejecución de secuencias de eventos supone que es posible encontrar un estado consistente después del disparo de cada evento. Es posible relajar esta condición y permitir el disparo de una secuencia de eventos donde sólo el último de ellos debe resultar en un estado de reinicio consistente.

Otra posibilidad consiste en ejecutar a la vez todos los eventos habilitados. En este caso, para evitar que la ejecución de varios eventos afecte al valor de una misma variable, se impone la denominada **regla de asignación única**, según la cual todos los cambios instantáneos que debe sufrir un estado continuo o discreto deben estar recogidos en una única ecuación instantánea. Con ello se garantiza que un mismo estado (continuo o discreto) no puede ser modificado en dos ecuaciones instantáneas correspondientes a dos condiciones de evento diferentes. Con ello se elimina el riesgo potencial de que puedan activarse simultáneamente varios eventos diferentes que asignen valores diferentes a un mismo estado.

Ésta es la forma en que proceden los entornos de modelado de Modelica. Aplicando este criterio de ejecución de los eventos al ejemplo anterior, se obtiene el resultado mostrado en la Tabla 4.7. Si se describe el modelo en Modelica y se simula usando Dymola, puede comprobarse que efectivamente ese es el resultado obtenido. Hagámoslo. La descripción en lenguaje Modelica del modelo se encuentra en el Código 4.1. Es posible pedir a Dymola que muestre en la ventana de mensajes los eventos ejecutados. Para ello, antes de traducir el modelo, deben señalarse las opciones *Event logging*, en la lengüeta *Debug* de la ventana *Simulation Setup* (véase la Figura 4.3). En la Figura 4.4 se muestra el resultado de la simulación de este modelo durante 16 s, empleando Dymola. Dymola escribe en la ventana de mensajes el siguiente informe sobre el disparo de los eventos.

```
Expression x2 <= 0.5*y became false ( (x2)-(0.5*y) = 5 )
Expression x1 >= 0.5*y became false ( (x1)-(0.5*y) = -5 )
Iterating to find consistent restart conditions.
    during event at Time : 0

Integration started at T = 0 using integration method DASSL
(DAE multi-step solver (dassl/dasslrt of Petzold modified by Dynasim))

Expression x2 <= 0.5*y became true ( (x2)-(0.5*y) = -9.99991e-011 )
Expression x1 >= 0.5*y became true ( (x1)-(0.5*y) = 1e-010 )
Iterating to find consistent restart conditions.
Expression x2 <= 0.5*y became false ( (x2)-(0.5*y) = 5 )
Expression x1 >= 0.5*y became false ( (x1)-(0.5*y) = -5 )
Iterating to find consistent restart conditions.
    during event at Time : 5.0000000001

Expression x2 <= 0.5*y became true ( (x2)-(0.5*y) = -5e-010 )
Expression x1 >= 0.5*y became true ( (x1)-(0.5*y) = 5e-010 )
Iterating to find consistent restart conditions.
Expression x2 <= 0.5*y became false ( (x2)-(0.5*y) = 5 )
Expression x1 >= 0.5*y became false ( (x1)-(0.5*y) = -5 )
Iterating to find consistent restart conditions.
    during event at Time : 10.0000000006
```

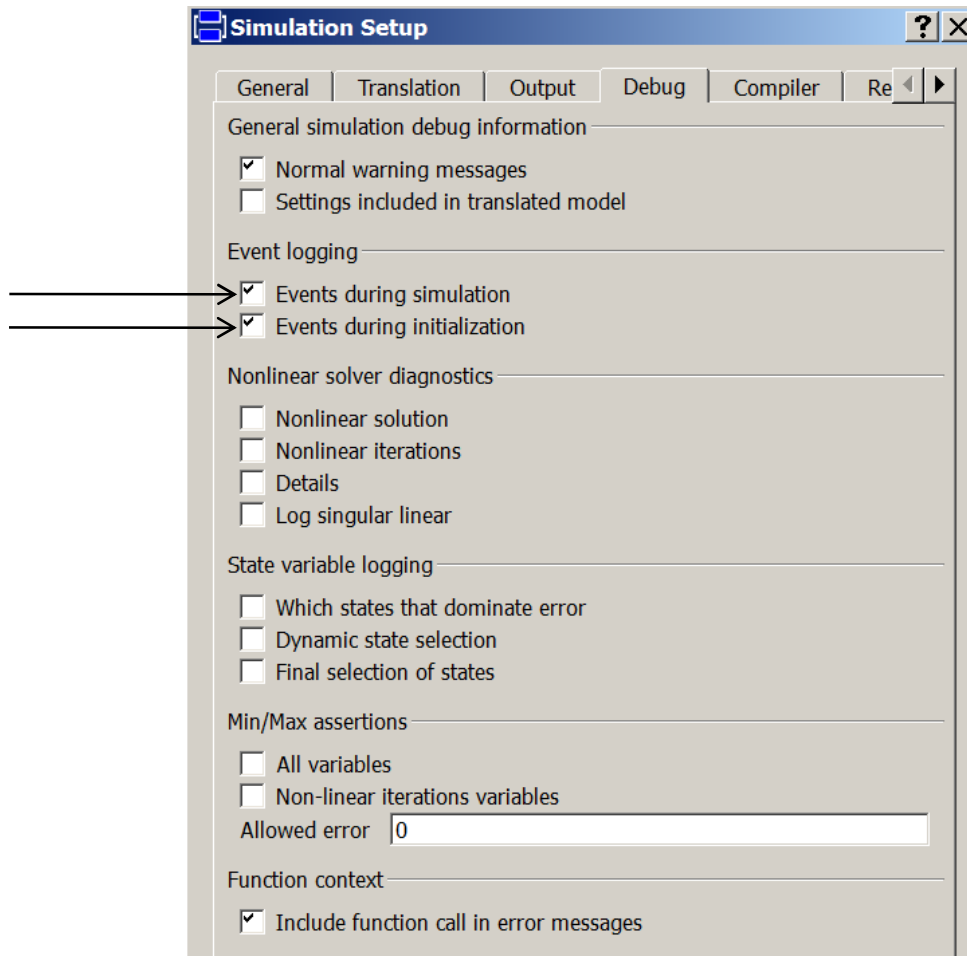


Figura 4.3: Configuración de Dymola para que muestre en la ventana de mensajes los eventos disparados durante la inicialización y la simulación.

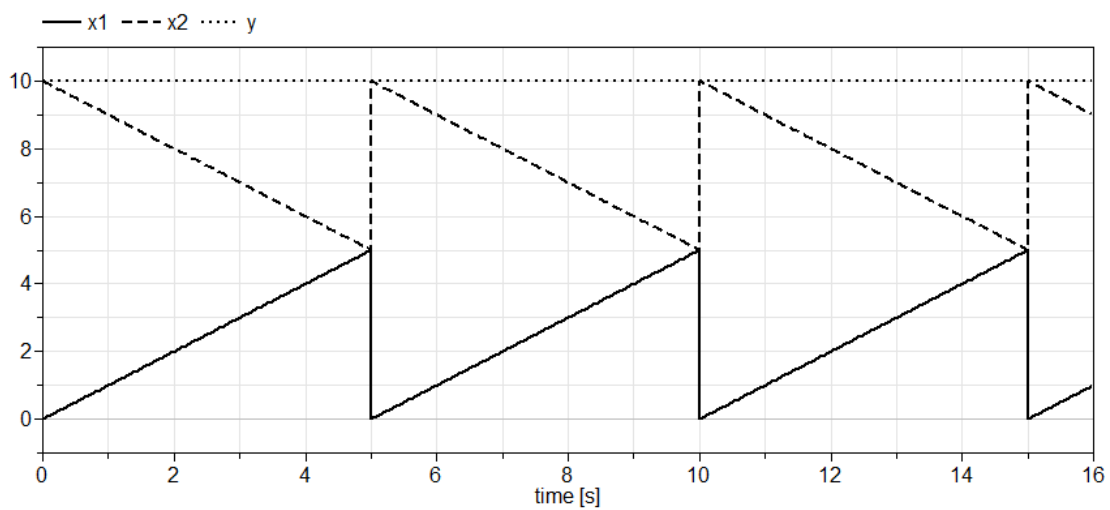


Figura 4.4: Simulación del modelo con eventos simultáneos del Código 4.1.

```

Expression x2 <= 0.5*y became true ( (x2)-(0.5*y) = -5.00001e-010 )
Expression x1 >= 0.5*y became true ( (x1)-(0.5*y) = 5e-010 )
Iterating to find consistent restart conditions.
Expression x2 <= 0.5*y became false ( (x2)-(0.5*y) = 5 )
Expression x1 >= 0.5*y became false ( (x1)-(0.5*y) = -5 )
Iterating to find consistent restart conditions.
    during event at Time : 15.0000000011

```

Integration terminated successfully at T = 16

Obsérvese que en el instante inicial de la simulación las dos condiciones de evento valen *false*. Comienza la resolución del problema de tiempo continuo y ésta progresa hasta que el algoritmo de integración supera el instante $t = 5$ s, momento en el cual se detecta que ambas condiciones de evento valen *true*. Se suspende la resolución del problema de tiempo continuo y se retorcede en el tiempo a fin de acotar con precisión, dentro del último paso de integración, el instante en que se habilitan los eventos. Dicho instante de disparo se acota dentro de un pequeño intervalo y se asume que el instante de disparo es el extremo derecho del intervalo, que en este caso vale 5.0000000001 s. Se resuelve el problema de reinicio en dicho instante. Las condiciones de evento valen *false*, con lo cual se reinicia la solución del problema de tiempo continuo. Ésta progresa hasta que el algoritmo de integración supera el instante $t = 10$ s, en el cual se repite la misma operación. Se procede análogamente para el evento del instante $t = 15$ s. Alcanzado el instante $t = 16$ s, finaliza la simulación.

4.3.2. Función de cruce

Los entornos de modelado de sistemas híbridos típicamente emplean funciones de cruce para detectar los eventos. A tal fin, expresan las condiciones de evento como funciones de cruce y vigilan estas funciones de cruce durante la resolución del problema de tiempo continuo. La **función de cruce** es una expresión cuyo resultado es positivo mientras la condición de evento vale *true* y negativo mientras vale *false*, de manera que en el instante en que la condición de evento cambia de valor (de *true* a *false*, o viceversa) la función de cruce corta el valor cero. Se habilita el evento cuando la función de cruce corta el cero, o bien cuando pasa de ser diferente de cero a valer exactamente cero.

Veamos un ejemplo. Supongamos que la variable y viene descrita mediante la siguiente sentencia if,

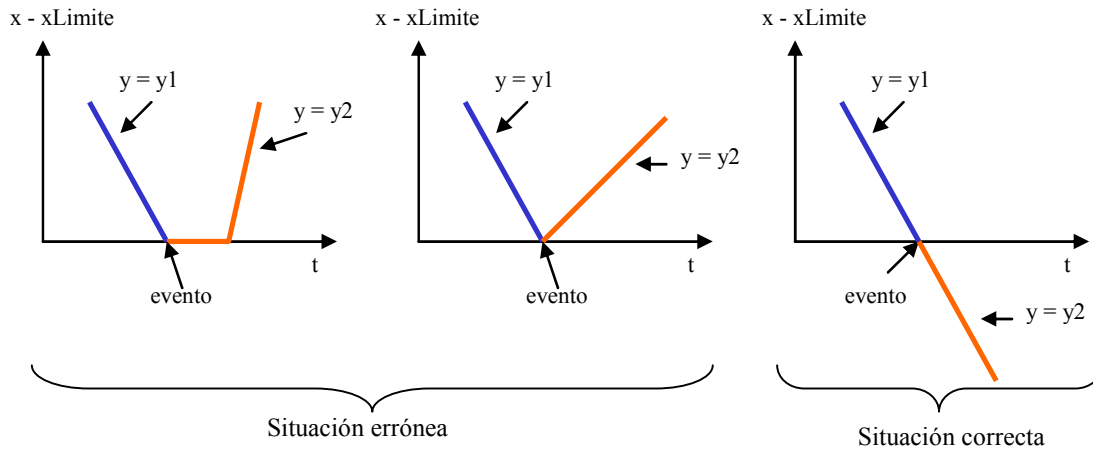


Figura 4.5: Fallo en la detección de eventos cuando se usa una única función de cruce.

$$y = \text{if } x > xLimite \text{ then } y1 \text{ else } y2; \quad (4.79)$$

Esta sentencia indica que mientras $x > xLimite$, la variable y es igual a la variable $y1$. En caso contrario, la variable y es igual a la variable $y2$. La detección del evento puede realizarse asociando a la condición de evento $x > xLimite$ la siguiente función de cruce, que llamaremos z :

$$z = x - xLimite \quad (4.80)$$

Sin embargo, el empleo de una única función de cruce puede hacer que en determinadas circunstancias no se detecten los eventos. Por ese motivo, los entornos de modelado de Modelica no suele emplearse una única función de cruce, sino que emplean dos.

Para ilustrar el problema a que puede dar lugar el uso de una única función de cruce, supóngase que x decrece hasta que vale exactamente $xLimite$. La función de cruce pasa de valer diferente de cero a valer exactamente cero. Entonces se ejecuta el evento y se escoge la rama $y = y2$ para reiniciar la integración. Supóngase que la función de cruce evaluada en el instante del evento, tras la ejecución del evento, sigue valiendo cero (si no vale cero, no se presenta ninguna dificultad). Dependiendo del modelo pueden darse tres posibilidades (véase la Figura 4.5): (1) que la función de cruce permanezca con valor cero durante cierto tiempo; (2) que se haga positiva; o (3) que se haga negativa.

En cualquiera de los tres casos, no se dispara un segundo evento cuando la función de cruce deja de valer cero, ya que no se produce cruce por el cero, ni tampoco pasa

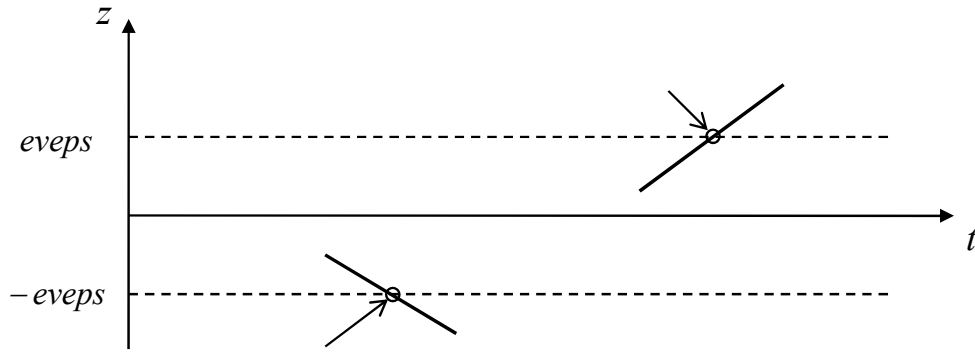


Figura 4.6: Los entornos de modelado de Modelica asocian dos funciones de cruce a la condición de evento: $zp = z + eveps$, $zn = z - eveps$. Cuando z cruza $-eveps$ con pendiente negativa, se considera que la condición de evento pasa de *true* a *false*. Cuando z cruza $+eveps$ con pendiente positiva, se considera que la condición de evento pasa de *false* a *true*.

de un valor diferente de cero a valer exactamente cero. Esto conduce a un resultado erróneo en cualquiera de los dos casos siguientes:

1. Si la función de cruce permanece valiendo cero cierto tiempo y luego se hace positiva.
2. Si, después de valer cero en el instante del evento (antes y después de la ejecución de la acción), en el siguiente instante de evaluación vuelve a hacerse positiva.

El resultado es erróneo puesto que, al no detectarse el evento, se emplea la rama $y = y2$ siendo la función de cruce positiva, cuando en realidad debería emplearse la rama $y = y1$.

La forma en que los entornos de modelado de Modelica evitan este problema es mediante la definición de un pequeño intervalo $(-eveps, eveps)$ entorno al cero, de modo que el cambio en el valor de la condición de evento no se detecta cuando z cruza por cero o cuando pasa a valer exactamente cero. El cambio de la condición de evento del valor *false* al valor *true* se detecta cuando z cruza $eveps$ con pendiente positiva (véase la Figura 4.6). El cambio de la condición de evento del valor *true* al valor *false* se detecta cuando z cruza $-eveps$ con pendiente negativa. Esto es equivalente a asociar a la condición de evento las dos siguientes funciones de cruce:

$$zp = z + eveps \quad (4.81)$$

$$zn = z - eveps \quad (4.82)$$

Si debido a las condiciones iniciales la función de cruce permanece en un principio dentro del intervalo $(-eveps, eveps)$, durante ese tiempo se considera que el valor de la función de cruce es igual a cero y se usa la rama correspondiente de la expresión `if`.

El valor de *eveps* es muy pequeño. En el caso de Dymola, vale por defecto 1E-10. Dymola permite modificar este valor. Para ello es preciso editar manualmente el fichero de experimento generado por Dymola, *dsin.txt*, una vez traducido el modelo y antes de realizar la simulación. Para evitar que Dymola sobrescriba el fichero *dsin.txt*, la simulación no debe ejecutarse desde la interfaz de usuario de Dymola, sino ejecutando el siguiente comando desde la ventana del sistema operativo, en el directorio de trabajo de Dymola, donde Dymola ha generado los ficheros *dymosim.exe* y *dsin.txt*:

```
dymosim dsin.txt
```

Al finalizar la simulación, Dymola almacena los resultados en un fichero llamado *dsres.mat*. Este fichero se puede abrir desde la interfaz de usuario de Dymola (*Plot / Open Result*), a fin de representar gráficamente las variables del modelo.

Para ilustrar cómo realiza Dymola la detección de los eventos, vamos a simular el modelo mostrado en el Código 4.2 empleando para ello dos valores diferentes de *eveps*. La primera simulación del modelo se ha realizado empleado el valor de *eveps* por defecto, que es 1E-10. En la segunda simulación se ha asignado manualmente a *eveps* el valor 0.6. Para ello, en el fichero *dsin.txt* se ha reemplazado la línea

```
1.0000000000000000E-010 # eveps Hysteresis epsilon at event points
```

por:

```
0.6000000000000000      # eveps Hysteresis epsilon at event points
```

y a continuación se ha ejecutado *dymosim.exe* desde una ventana de MS-DOS, cargándose posteriormente los resultados desde Dymola para su representación gráfica.

El resultado de las dos simulaciones se muestra en la Figura 4.7. La condición de evento ($x > 0$) es traducida a dos funciones de cruce: $zp = x + eveps$, $zn = x - eveps$. Se detecta que la condición de evento cambia de *false* a *true* cuando x cruza $+eveps$ con pendiente positiva, y se detecta el cambio en el valor de la condición de evento de *true* a *false* cuando x cruza $-eveps$ con pendiente negativa. Esto es claramente visible en la gráfica situada en la parte inferior de la Figura 4.7.

El mecanismo empleado por el entorno de modelado para la detección de los eventos tiene una gran relevancia desde el punto de vista práctico, debiendo ser

```

model ejemEveps
  Real x;
  Real y;
equation
  y = if x > 0 then 1 else -1;
  x = sin(time);
end ejemEveps;

```

Código 4.2: Modelo para ilustrar la detección de los eventos en Dymola.

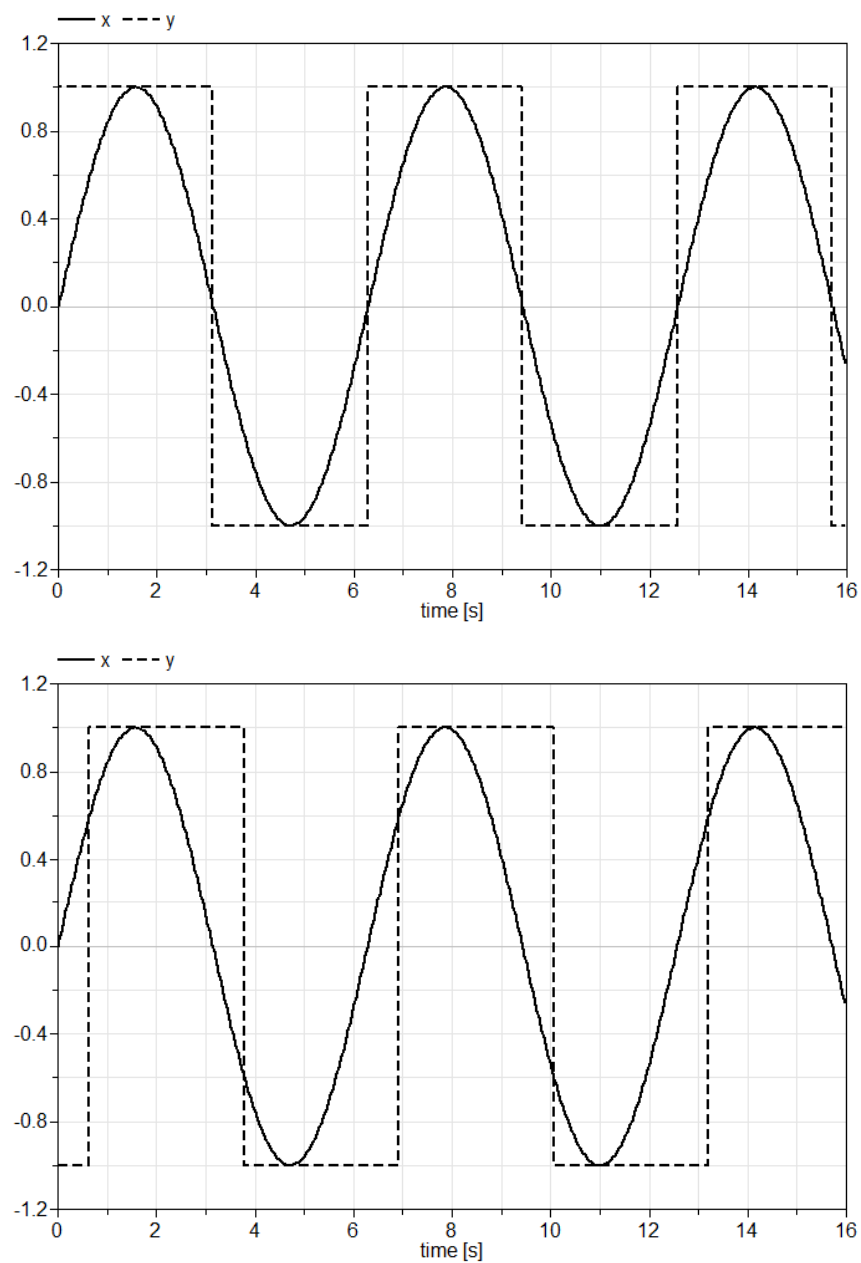


Figura 4.7: Resultado de la simulación para dos valores diferentes de *eveps*: 1E-10 (figura superior) y 0.6 (figura inferior).

conocido por el desarrollador del modelo, ya que en ocasiones dicho mecanismo puede dar lugar a artificios numéricos que condicionen el resultado de la simulación. Se muestra a continuación un ejemplo.

Consideremos nuevamente el modelo de la pelota que cae verticalmente bajo la acción de la gravedad y rebota al tocar el suelo. La descripción en lenguaje Modelica se muestra en el Código 4.3. Ejecutamos la simulación durante 14 s. Obsérvese que inicialmente la pelota está en reposo, situada a una altura de 10 m sobre el suelo. Al comenzar la simulación la pelota cae verticalmente acelerada por la gravedad. Al tocar el suelo cambia el sentido de la velocidad, ascendiendo y posteriormente cayendo nuevamente. Dado que la pelota pierde velocidad (y consiguientemente, energía) con cada rebote, a medida que progresa la simulación la altura máxima que alcanza la pelota entre sucesivos rebotes va siendo cada vez menor. Sin embargo, puede verse que entorno a los 13 s de simulación la pelota cae por debajo del nivel del suelo. Este comportamiento no está descrito en el modelo: se trata de un artificio numérico producto del mecanismo empleado para la detección de los eventos.

Para ver de manera sencilla por qué la pelota cae por debajo del nivel del suelo, repitamos la simulación del modelo, pero esta vez dándole a la pelota una altura inicial de $5E - 9$ m. Como en la simulación anterior, la pelota se encuentra inicialmente en reposo. Establecemos además que el instante final de la simulación sea 0.0003 s. Ejecutando la simulación se obtiene el resultado mostrado en la gráfica superior de la Figura 4.9. Haciendo zoom en la gráfica al intervalo $2.16E - 4 < t < 2.52E - 4$, se observa (véase la gráfica inferior de la figura) que llega un momento en que cuando la pelota rebota a la altura *-eveps* lo hace con tan poca energía que en su ascenso no es capaz de superar la altura *+eveps*, con lo cual Dymola no detecta que la condición $x \leq 0$ se hace *false*. Por este motivo, a partir del rebote producido aproximadamente en el instante $2.36E - 4$ s la condición de evento mantiene el valor *true*. Como las ecuaciones instantáneas de la cláusula *when* se activan cuando la condición pasa de *false* a *true*, la cláusula no vuelve a activarse, con lo cual la velocidad no vuelve a reiniciarse y la pelota cae por debajo del nivel del suelo.

4.3.3. Determinación del instante de disparo de los eventos

Dependiendo de su condición de disparo, los eventos pueden clasificarse en eventos en el tiempo y eventos en el estado.

A su vez, los **eventos en el tiempo** pueden clasificarse en exógenos o endógenos: si el instante en que se habilita el evento se especifica en el modelo, se dice que el

```

model pelota1
  Modelica.SIunits.Distance x(start=10, fixed=true);
  Modelica.SIunits.Velocity v(start=0, fixed=true);
  parameter Real c = 0.8 "Coef. rebote elástico";
  parameter Modelica.SIunits.Acceleration g = 9.8;
equation
  der(v) = -g;
  der(x) = v;
  when x <= 0 then
    reinit(v, -c*v);
  end when;
end pelota1;

```

Código 4.3: Caída vertical y rebote contra el suelo de una pelota.

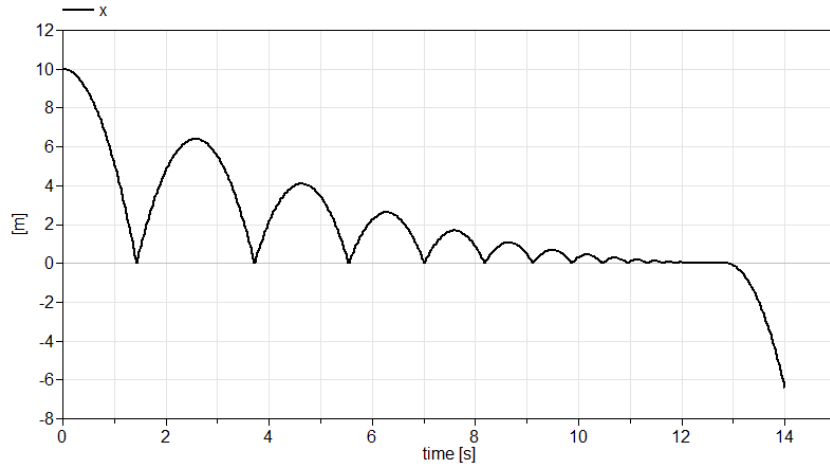
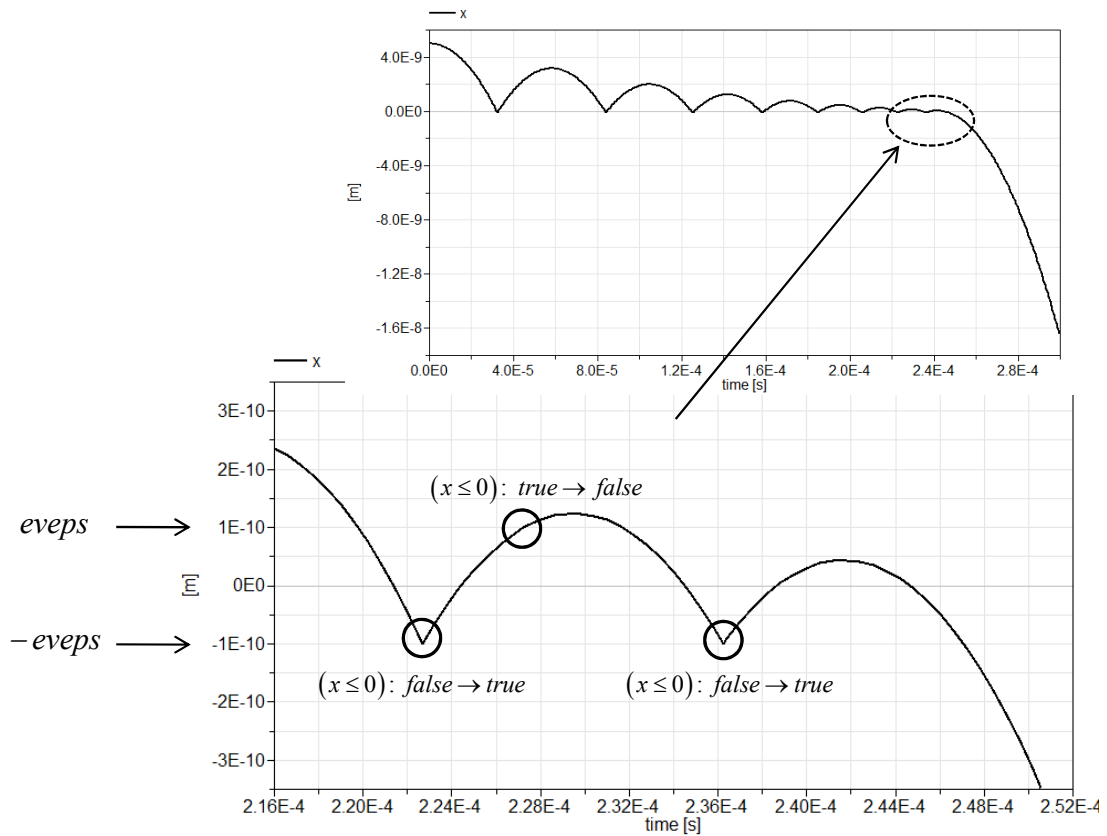


Figura 4.8: Resultado de simular el Código 4.3.

Figura 4.9: Resultado de simular el Código 4.3, pero con una altura inicial de $5E-9$ m.

evento es **exógeno**, mientras que si dicho instante se calcula durante la ejecución de la simulación, como resultado de la ejecución de un evento en el estado o de un evento planificado en el tiempo previos, se dice que el evento en el tiempo es **endógeno**.

Una vez detectado que el evento en el tiempo más próximo se produce dentro del siguiente paso de integración, el tamaño del paso se modifica de modo que el instante de evaluación coincida con el evento en el tiempo. Una vez ejecutado el evento, el algoritmo de integración es reiniciado, empleando su método propio para establecer el tamaño del paso de integración.

Los **eventos en el estado** se disparan cuando el estado del sistema satisface ciertas condiciones. Su instante de disparo no es conocido con antelación y debe calcularse mediante iteración.

A fin de ilustrar el método de cálculo por iteración del instante en que se produce el evento, considérese una ecuación definida mediante diferentes ramas, cada una de las cuales es válida en un determinado dominio del espacio de estados. Supóngase que eventos en el estado señalan el fin del dominio de validez de una rama y el comienzo del dominio de validez de otra. Se definen funciones de cruce para la detección de los eventos, que señalan el cruce de la trayectoria en el espacio de estados de un dominio a otro. Cuando se detecta que se ha producido un evento en el estado, se suspende la integración y se activa un proceso iterativo, cuya finalidad es determinar el instante temporal en que se ha producido el evento. Este proceso iterativo conlleva la evaluación de la ecuación y para ello se emplea la rama “antigua”, extendiéndola más allá de su dominio de validez. Una vez determinado el instante en que se ha producido el evento en el estado, se conmuta, para la evaluación de la ecuación, de la rama “antigua” a la “nueva” y una vez resuelto el problema de reinicio, se reinicia el algoritmo de integración comenzando en el instante del evento.

A la hora de realizar el modelo debe tenerse en cuenta que las ramas de las ecuaciones deben poder ser extendidas más allá de su dominio de definición, ya que en caso contrario se producirá un error numérico en tiempo de simulación cuando la trayectoria en el espacio de estados salga de un dominio para entrar en otro.

Se muestra a continuación un ejemplo que pretende ilustrar este problema. Supongamos que el flujo másico de líquido (F) que se produce a través de una válvula está relacionado con la caída en la presión (Δp) mediante la siguiente relación constitutiva de la válvula

$$F = \text{sgn}(\Delta p) \cdot K \cdot \sqrt{\Delta p} \quad (4.83)$$

donde la función signo vale uno cuando su argumento es positivo o cero, y menos uno cuando es negativo. K es una constante dependiente de la geometría de la válvula, del factor de fricción y de la densidad del líquido.

Si la relación constitutiva de la válvula se describe en el modelo de la manera siguiente (se usa una sintaxis similar a la de Modelica),

$$\begin{aligned}
 F = & \text{ if } \Delta p > 0 \\
 & \text{ then } K \cdot (\Delta p)^{0.5} \\
 & \text{ else } -K \cdot (-\Delta p)^{0.5}
 \end{aligned} \tag{4.84}$$

se producirá un error numérico al cambiar Δp de signo, ya que el algoritmo de la simulación comprueba si se han habilitado eventos (calculando e inspeccionando las funciones de cruce) una vez que se han evaluado todas las variables algebraicas en el instante de evaluación. Es decir, una vez que ya se ha intentado calcular la raíz cuadrada de un número real negativo, produciéndose el consiguiente error.

Para que expresiones de este tipo funcionaran, sería preciso que el algoritmo de simulación comprobara, antes de evaluar la ecuación con estructura variable, cuál de las dos ramas de la ecuación debe usar, en función del valor que tenga la condición en ese instante. El lenguaje Modelica proporciona un operador, denominado **noEvent()**, para indicar que debe procederse de esa manera con una determinada ecuación.

Por ejemplo, si se escribe en Modelica

```
F = if noEvent(difPresion > 0)
    then K*difPresion^0.5 else -K*(-difPresion)^0.5;
```

se está indicando al entorno de modelado que, antes de evaluar la función, debe comprobar cuál de las dos ramas debe emplear. Asimismo, el operador **noEvent()** indica al entorno de modelado que, en caso de producirse cambio de rama, no debe iterarse para localizar de manera precisa el instante en que se produjo el evento. Esta forma de tratar las ecuaciones de estructura variable se denomina realizar un **tratamiento textual** de la ecuación, en oposición al **tratamiento basado en eventos** explicado anteriormente.

Obsérvese que al realizarse un tratamiento textual de una ecuación con estructura variable, se está integrando a través de la conmutación entre ramas. Esto implica que las discontinuidades en la función, en caso de haberlas, pueden hacer que el algoritmo de integración falle. Esto es debido a que los algoritmos de integración

están ideados partiendo de la premisa de que la función a integrar es continua y sus derivadas son continuas.

El lenguaje Modelica proporciona funciones con sus ramas extendidas más allá de su dominio de validez, con el fin de evitar que se produzcan errores numéricos cuando se realiza un tratamiento basado en eventos de las ecuaciones con estructura variable. Este es el caso de la función raíz cuadrada, `sqrt()`. Así, por ejemplo, no se produce error numérico si la relación constitutiva de la válvula se expresa de la manera siguiente:

```
F = if difPresion > 0 then
    K*sqrt(difPresion) else -K*sqrt(-difPresion);
```

puesto que la función `sqrt()` ha sido extendida de modo que cuando se le pase como argumento un número negativo no produzca error numérico. Se volverá sobre todo ello en la Sección 4.5.

4.3.4. Chattering

La simulación de modelos híbridos plantea problemas teóricos y computacionales que no se presentan en los modelos continuos. Uno de ellos es el chattering. Se dice que en la simulación se produce chattering cuando el número de eventos en el estado es grande comparado con el número de pasos de integración.

El chattering produce gran lentitud en la simulación, debido a que cada vez que se produce un evento es preciso determinar su instante de disparo, ejecutar las acciones asociadas y solucionar el problema de reinicio. La única solución al chattering es modificar el modelo.

Veamos un ejemplo. Para ello, consideremos nuevamente el modelo del rebote de la pelota que se mostró en el Código 4.3. Supongamos que modificamos el modelo a fin de evitar que la pelota caiga por debajo del nivel del suelo, tal como se muestra en el Código 4.4. El cambio está en la ecuación que describe la aceleración de la pelota: cuando la pelota se encuentra en el suelo o por debajo del nivel del suelo, su aceleración es cero.

Ejecutamos la simulación durante 14 s. Como puede verse en la Figura 4.10, la pelota no cae por debajo del nivel de suelo. Sin embargo, a partir de aproximadamente el segundo 12 la velocidad de ejecución se ralentiza. Una vez finalizada la simulación, Dymola indica en la ventana de mensajes que ha necesitado 3.22 segundos de tiempo de CPU para ejecutar la simulación y que el número de eventos en el estado es


```

model pelota2
  Modelica.SIunits.Distance x(start=10, fixed=true);
  Modelica.SIunits.Velocity v(start=0, fixed=true);
  parameter Real c = 0.8 "Coef. rebote elástico";
  parameter Modelica.SIunits.Acceleration g = 9.8;
equation
  der(v) = if (x <= 0) then 0 else -g;
  der(x) = v;
  when x <= 0 then
    reinit(v, -c*v);
  end when;
end pelota2;

```

Código 4.4: Modelo con chattering de la caída vertical y rebote de una pelota.

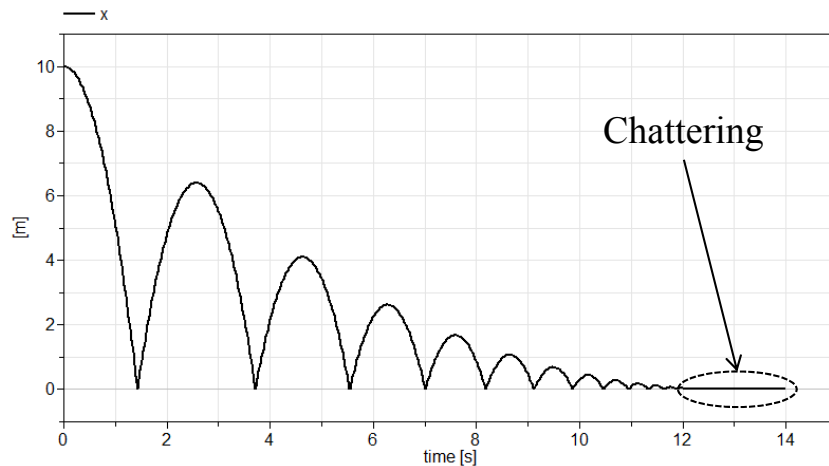


Figura 4.10: Resultado de simular el Código 4.4.

141128 eventos, un número ciertamente elevado. Debido a ello, Dymola avisa de que posiblemente el modelo presenta chattering. Se reproduce a continuación parte del mensaje generado por Dymola:

```

Integration started at T = 0 using integration method DASSL
(DAE multi-step solver (dassl/dasslrt of Petzold modified by Dynasim))
Integration terminated successfully at T = 14
WARNING: You have many state events. It might be due to chattering.
Enable logging of event in Simulation/Setup/Debug/Events during simulation
CPU-time for integration      : 3.22 seconds
CPU-time for one GRID interval: 6.44 milli-seconds
Number of result points      : 282757
Number of GRID points        : 501
Number of (successful) steps : 209012
Number of F-evaluations       : 370896
Number of H-evaluations       : 1089681
Number of Jacobian-evaluations: 161884
Number of (model) time events : 0
Number of (U) time events     : 0
Number of state events        : 141128
Number of step events         : 0
Minimum integration stepsize  : 1.75e-011
Maximum integration stepsize  : 1.65

```

```
Maximum integration order      : 3
Calling terminal section
... "dsfinal.txt" creating (final states)
```

Una contribución importante al tiempo de ejecución de la simulación es el necesario para salvar en fichero los resultados de la simulación. Esto tiene una especial relevancia cuando se produce chattering, en cuyo caso también el tamaño de los ficheros de resultados puede llegar a ser problemático cuando los modelos son complejos.

Recuérdese que Dymola permite al usuario especificar qué tipo de variables deben salvarse a fichero y en qué instantes. Esto se indica en la lengüeta *Output* de la ventana *Simulation / Setup*. La selección más habitual es almacenar los resultados en instantes de tiempo equiespaciados y además almacenarlos cuando se producen eventos. El intervalo de tiempo entre dichos instantes equiespaciados se denomina **intervalo de comunicación** (*Output interval*). El tamaño del intervalo de comunicación o el número de intervalos de comunicación se especifica en la lengüeta *General* de la ventana *Simulation / Setup*.

Para analizar por qué se produce chattering en este modelo, repitamos la simulación con unas condiciones iniciales y un instante de finalización diferentes. Asumiremos que inicialmente la pelota se encuentra en reposo, a una altura de $5E - 9$ m, y ejecutaremos la simulación durante 0.0006 s. El resultado se muestra en la Figura 4.11. Obsérvese que a partir de aproximadamente el instante $4E - 4$ s la pelota queda rebotando de manera periódica, produciéndose un rebote (es decir, un evento en el estado) cada aproximadamente $2E - 5$ s.

El hecho de que la pelota muestre este comportamiento periódico en el estacionario, rebotando indefinidamente, es un artificio numérico consecuencia del mecanismo para la detección de los eventos que emplea Dymola. Obsérvese que mientras la pelota asciende recorriendo el rango de alturas $(-eveps, +eveps)$ la condición de evento $x \leq 0$ vale *true*, con lo cual durante ese tiempo la gravedad no actúa sobre la pelota y por consiguiente se le está “regalando” energía a la pelota. Cuando esa energía regalada es igual a la energía perdida en el rebote, el modelo queda oscilando indefinidamente.

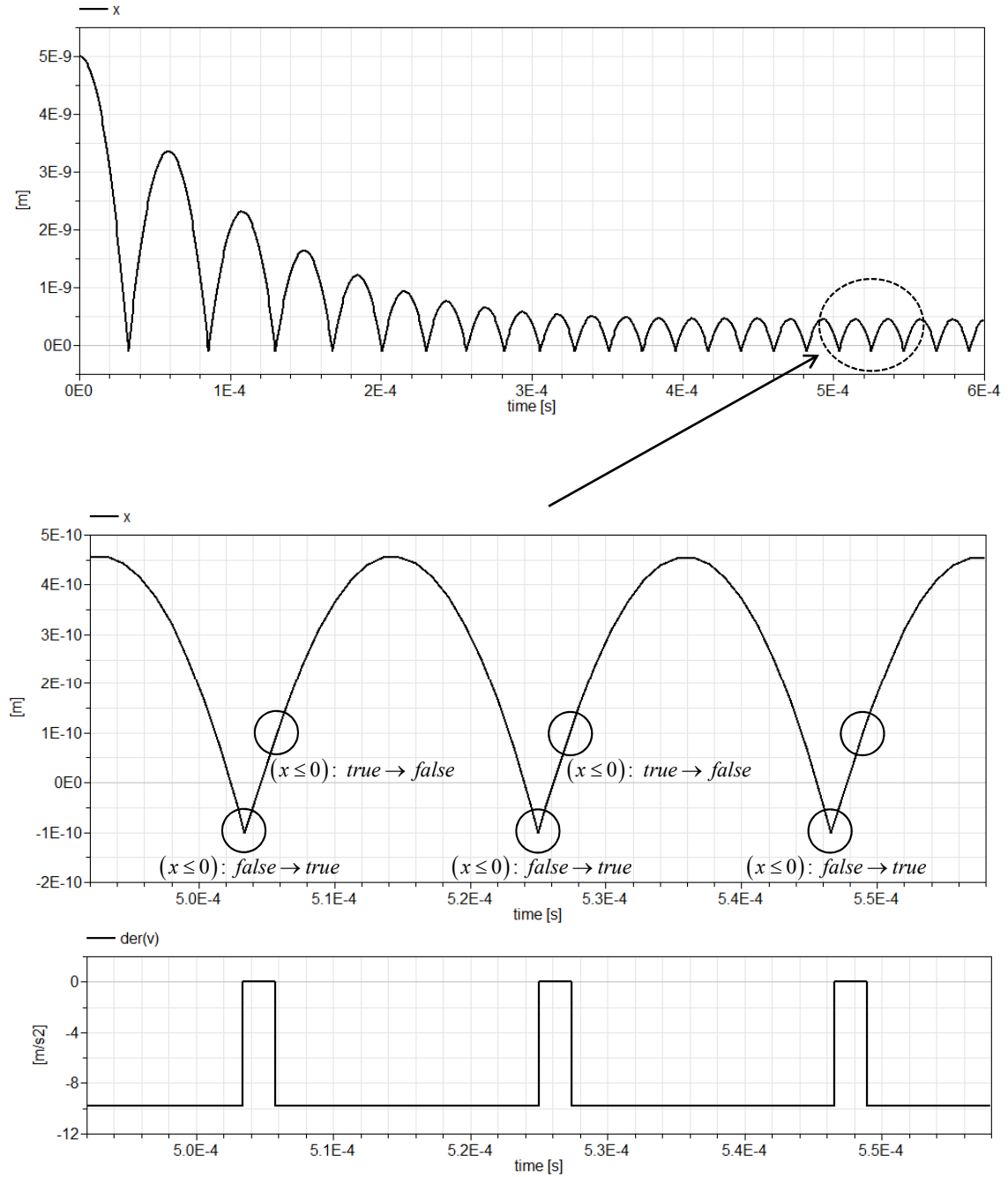


Figura 4.11: La pelota alcanza un comportamiento periódico estacionario debido al procedimiento empleado por el entorno de modelado para la detección de los eventos.

4.4. MODELOS CON ESTRUCTURA VARIABLE

Se denominan **modelos con estructura variable** a aquellos cuya *descripción matemática varía en tiempo de simulación*. Son sistemas en los que se definen varias **fases**, también denominadas **modos**, cada una de las cuales está descrita por un determinado conjunto de ecuaciones. En tiempo de simulación, de acuerdo a unas condiciones bien establecidas, se produce la conmutación entre sus fases, y, en consecuencia, entre los distintos modelos matemáticos que describen el sistema en sus distintas fases.

4.4.1. Ejemplo 1: interruptor ideal de flujo

Un ejemplo típico de sistema con dos fases es un interruptor ideal de flujo. En la Figura 4.12 se muestra el modelo en cada una de sus fases: abierto y cerrado. La fase viene determinada por el valor de una variable lógica llamada *OpenSw*. Además de ésta, en el modelo intervienen otras tres variables: las presiones en los extremos del interruptor (p_A , p_B) y el caudal volumétrico (F_V) que circula a través del interruptor. El modelo está descrito mediante una relación constitutiva que depende de la fase.

- Mientras *OpenSw* vale *true*, el interruptor está **abierto**: no circula flujo a su través y la caída de presión entre sus extremos viene impuesta por el resto del circuito. La relación constitutiva es:

$$F_V = 0 \quad (4.85)$$

- Mientras *OpenSw* vale *false*, el interruptor está **cerrado**, la caída de presión entre sus extremos es cero y el flujo que circula a su través depende del resto del circuito de conducción. La relación constitutiva es:

$$p_A = p_B \quad (4.86)$$

La causalidad computacional del interruptor no viene impuesta por la forma en que se realiza su conexión con el entorno, sino que es función del valor de su variable *OpenSw*. Si *OpenSw* vale *true*, de la relación constitutiva del interruptor debe evaluarse el flujo: $[F_V] = 0$. Si vale *false*, debe evaluarse la presión en uno de sus terminales: $[p_A] = p_B$ o bien $p_A = [p_B]$.

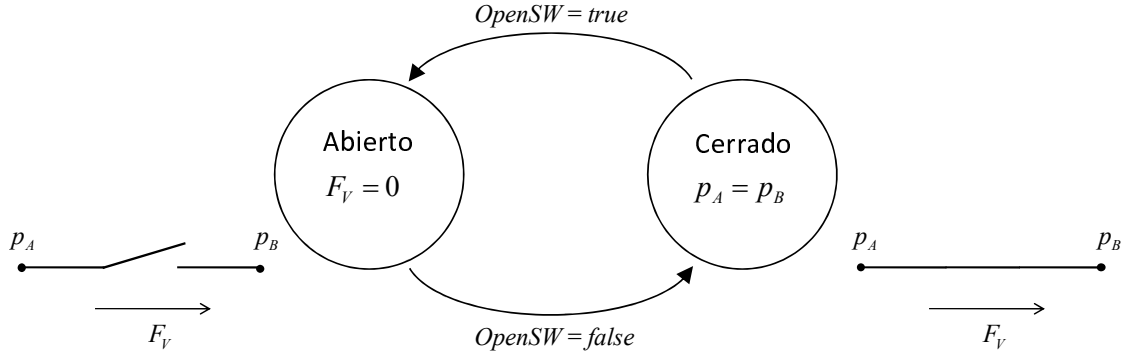


Figura 4.12: Modelo con dos fases de un interruptor de flujo. Interruptor de flujo abierto (izquierda) y cerrado (derecha).

Dado que la causalidad computacional no queda determinada al conectar el interruptor al entorno, ya que depende de $OpenSw$, al asignar la causalidad computacional al modelo completo se encontrará que la relación constitutiva del interruptor formará parte de un lazo algebraico.

Modelica permite describir modelos con estructura variable mediante sentencias y cláusulas if, las cuales son traducidas automáticamente a ecuaciones por el entorno de modelado, permitiendo así que el usuario realice una descripción de alto nivel del sistema de estructura variable. Por ejemplo, el interruptor de flujo puede describirse en lenguaje Modelica mediante la siguiente sentencia if:

$$0 = \text{if } OpenSw \text{ then } F_V \text{ else } p_A - p_B; \quad (4.87)$$

que, como se explicó en la Sección 4.2.3, es traducida por el entorno de modelado a la ecuación:

$$0 = \alpha \cdot F_V + (1 - \alpha) \cdot (p_A - p_B) \quad (4.88)$$

donde la variable muda de tiempo discreto α vale 1 mientras $OpenSw$ vale *true* y vale 0 mientras $OpenSw$ vale *false*.

4.4.2. Ejemplo 2: tubería con sistema de desagüe

Para ilustrar cómo la relación constitutiva del interruptor pasa a formar parte de un lazo algebraico, consideremos el modelo de un sistema de desagüe compuesto por la tubería principal y su sistema de seguridad, el cual consta de un interruptor

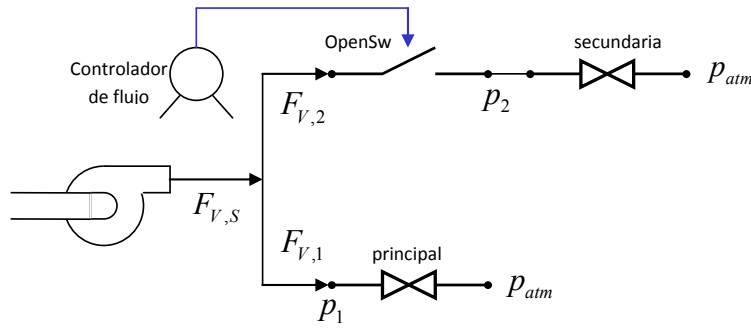


Figura 4.13: Tubería con sistema de desagüe de seguridad.

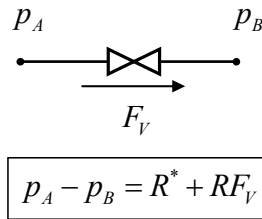


Figura 4.14: Modelo lineal de la tubería.

de flujo y una segunda tubería, dispuesta en paralelo tal como se muestra en la Figura 4.13.

Cuando el caudal de la tubería principal excede determinado límite de seguridad, se cierra el interruptor de flujo, derivándose parte del caudal por la tubería secundaria de seguridad. En condiciones normales de funcionamiento, el interruptor se encuentra abierto y todo el fluido circula por la tubería principal.

Si el fluido puede considerarse incompresible, la caída de presión (Δp) debida a la fricción en una tubería comercial viene dada por

$$F_V = S \cdot c_{desc} \cdot \sqrt{\Delta p} \quad (4.89)$$

donde S es la sección de la tubería y c_{desc} es el coeficiente de descarga, el cual viene descrito por la Ec. (4.90).

$$c_{desc} = \sqrt{\frac{2 \cdot D}{\kappa_{Fanning} \cdot L \cdot \rho}} \quad (4.90)$$

Se considera que, en un entorno del punto normal de operación, puede suponerse que el flujo que circula por una tubería es proporcional a la caída de presión entre sus extremos. El modelo linealizado es

$$F_V = F_{V,0} + \frac{S \cdot c_{desc}}{2 \cdot \sqrt{\Delta p_0}} \cdot (\Delta p - \Delta p_0) \quad (4.91)$$

donde $(F_{V,0}, \Delta p_0)$ representa el punto normal de trabajo que, en general, es diferente para cada tubería. El modelo puede reescribirse de la forma siguiente (véase la Figura 4.14):

$$p_A - p_B = R^* + R \cdot F_V \quad (4.92)$$

Las ecuaciones del circuito de flujo son:

$$p_1 - p_{atm} = R_1^* + R_1 \cdot F_{V,1} \quad (4.93)$$

$$p_2 - p_{atm} = R_2^* + R_2 \cdot F_{V,2} \quad (4.94)$$

$$0 = \alpha \cdot F_{V,2} + (1 - \alpha) \cdot (p_1 - p_2) \quad (4.95)$$

$$F_{V,S} = F_{V,1} + F_{V,2} \quad (4.96)$$

$$F_{V,S} = f(t) \quad (4.97)$$

La presión atmosférica, p_{atm} , se considera constante. Para simplificar la explicación, no se ha incluido en el modelo la ecuación para calcular la variable de tiempo discreto α a partir de la variable booleana *OpenSw*, ni tampoco la ecuación para el cálculo de *OpenSw*. Al realizar la partición supondremos que α es conocida. El sistema de ecuaciones obtenido es lineal. Resolviéndolo se obtiene:

$$[F_{V,S}] = f(t) \quad (4.98)$$

$$[F_{V,2}] = \frac{(R_1^* - R_2^* + R_1 \cdot F_{V,S})(1 - \alpha)}{(R_1 + R_2)(1 - \alpha) - \alpha} \quad (4.99)$$

$$[F_{V,1}] = F_{V,S} - F_{V,2} \quad (4.100)$$

$$[p_1] = R_1^* + R_1 \cdot F_{V,1} + p_{atm} \quad (4.101)$$

$$[p_2] = R_2^* + R_2 \cdot F_{V,2} + p_{atm} \quad (4.102)$$

El denominador del término de la derecha de la Ec. (4.99) no es cero en ningún caso, con lo cual el modelo es válido para cualquiera de las dos fases del interruptor de flujo, es decir, tanto mientras $\alpha = 0$, como mientras $\alpha = 1$. Si el circuito de flujo contuviera N interruptores, se establecerían los lazos algebraicos de modo que se obtendría un único modelo válido para sus 2^N posibles fases.

En este ejemplo el índice del sistema es independiente de la fase del interruptor. Sin embargo, puede ocurrir que al producirse la conmutación se introduzcan ligaduras adicionales que hagan variar los grados de libertad del sistema. Dicho de otra manera, al producirse la conmutación puede variarse la causalidad computacional de uno o más elementos almacenadores, de integral a derivativa o viceversa, modificándose el índice del sistema. A continuación se muestran algunos ejemplos sencillos de sistemas de índice variable.

4.4.3. Ejemplo 3: desagüe del líquido de un depósito

Supongamos que la característica de la tubería que desea modelarse no es la pérdida de carga del fluido por fricción con las paredes de la conducción, como en el ejemplo anterior, sino la dinámica del fluido inducida por la diferencia de presión entre los extremos de la conducción.

Consideremos un líquido de densidad ρ constante, que circula con velocidad uniforme v por el interior de una tubería de sección S y longitud L . El momento lineal del líquido (P) en la tubería puede calcularse teniendo en cuenta que la masa del líquido contenido en la tubería es igual al producto de su densidad por el volumen interno de la tubería: $\rho \cdot L \cdot S$.

$$P = \underbrace{\rho \cdot L \cdot S}_{\text{Masa}} \cdot v = \rho \cdot L \cdot F_V \quad (4.103)$$

Obsérvese que el caudal volumétrico del líquido (metros cúbicos que circulan por segundo) se ha calculado multiplicando la velocidad del líquido por la sección interna de la tubería:

$$F_V = S \cdot v \quad (4.104)$$

Se supone que la única fuerza que se ejerce sobre el fluido contenido en la tubería es igual al producto de la sección de ésta por la diferencia de presiones entre sus extremos.

Dado que la derivada del momento lineal del fluido es igual a la fuerza que se ejerce sobre él, se obtiene el modelo de la inercia del fluido representado en la Figura 4.15, donde, por analogía con el caso eléctrico, se denomina inductancia al coeficiente I de la Ec. (4.105). En la Figura 4.15 también se muestra el modelo no lineal de la caída de presión en la tubería debida a la fricción.

$$I = \frac{\rho \cdot L}{S} \quad (4.105)$$

Consideremos el circuito de flujo de la Figura 4.16, en el cual el desagüe del líquido contenido en un depósito se produce a través de dos tramos de conducción que están separados por un interruptor de flujo. En uno de los tramos de conducción la característica modelada es la pérdida de presión debida a la fricción. El comportamiento de interés en el otro tramo es la dinámica asociada a la inercia del líquido.

Como en el ejemplo anterior, cuando el interruptor de flujo está cerrado, la caída de presión entre sus puntos de conexión es cero, mientras que cuando está abierto no se produce flujo de desagüe. Véanse las Figuras 4.17 y 4.18, donde se muestra el modelo en cada una de las fases del interruptor.

Por simplicidad, se ha supuesto que en las condiciones normales de funcionamiento las variaciones en la presión en la base del depósito debido al desagüe son tan pequeñas que pueden ser despreciadas, con lo cual se considera p_1 una función del tiempo. La presión atmosférica se considera constante.

Obsérvese que cuando el interruptor está abierto, el problema tiene índice dos y cero grados de libertad. Cuando está cerrado, el índice es uno y el modelo posee un grado de libertad.

Para reducir el índice del modelo cuando el interruptor está abierto basta con derivar la ligadura sobre la variable que aparece derivada, es decir, la ecuación $F_V = 0$, y añadir la ecuación derivada, $\frac{dF_V}{dt} = 0$, al sistema.

A la fecha de escritura de este texto, los entornos de modelado de Modelica no tratan adecuadamente los sistemas cuyo número de grados de libertad varía en tiempo de simulación, por ello el desarrollador del modelo debe modificar el modelo.

Un modo de hacerlo es cambiar las hipótesis de modelado del interruptor, de modo que su ecuación constitutiva contenga, en cualquiera de sus dos fases (abierto y cerrado), tanto la presión en sus puntos de conexión como el flujo de volumen. De esta forma se evita el problema que se plantea cuando en una determinada fase del interruptor todas las variables de su ecuación constitutiva aparecen derivadas en otros componentes del sistema.

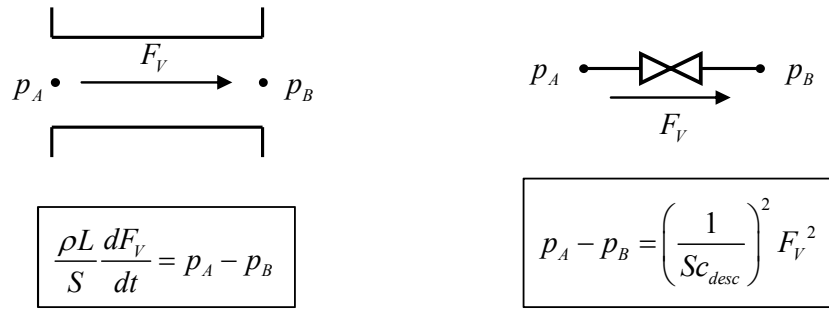


Figura 4.15: Modelado del flujo: inercia (izquierda) y resistencia (derecha).

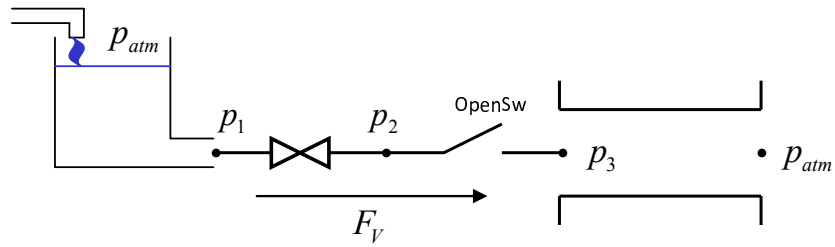


Figura 4.16: Esquema del modelo del circuito de desagüe.

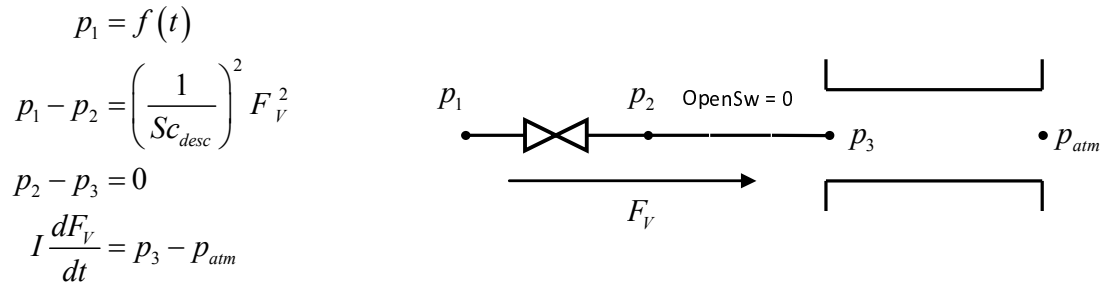


Figura 4.17: Interruptor de flujo cerrado.

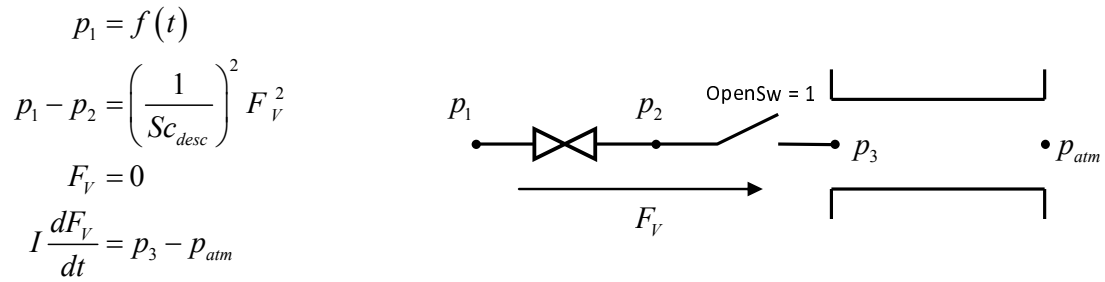


Figura 4.18: Interruptor de flujo abierto.

4.4.4. Ejemplo 4: llenado de dos depósitos en paralelo

En el ejemplo anterior, el problema de índice superior se plantea debido a que la ecuación constitutiva del interruptor abierto ($F_V = 0$) sólo contiene el flujo de volumen (siendo, por tanto, obligado su cálculo de esta ecuación) y el flujo de volumen aparece derivado en la ecuación constitutiva de la inercia del fluido.

El problema dual se plantea, por ejemplo, si se conectan en paralelo dos depósitos a una bomba, de tal manera que entre uno de ellos y la bomba se disponga un interruptor de flujo. En las Figuras 4.19 y 4.20 se muestran las dos fases del modelo. La relación constitutiva del interruptor cerrado sólo contiene las presiones de sus puntos de conexión, con lo cual debe emplearse para evaluar una de ellas. Puesto que ambas variables aparecen derivadas en las ecuaciones constitutivas de los tanques, el problema tiene índice superior.

4.4.5. Ejemplo 5: interruptor no ideal de flujo

Debido a las dificultades asociadas con los interruptores ideales, muchos desarrolladores de modelos prefieren reemplazar los interruptores ideales por elementos no ideales, en los cuales el interruptor tiene una resistencia muy pequeña mientras está cerrado y una resistencia muy grande mientras está abierto.

En la Figura 4.21 se muestra una modificación del modelo del interruptor de flujo, de modo que en las relaciones constitutivas de sus dos fases aparecen tanto la presión en sus dos puntos de conexión como el flujo de volumen. En la figura, $\varepsilon()$ representa cualquier función monovaluada cuyo valor sea lo suficientemente pequeño como para ser despreciable. En general deberá escogerse una función diferente para cada fase. Una posible elección es:

$$p_A - p_B = \varepsilon \cdot F_V \quad \text{si } OpenSw = 0 \quad (4.106)$$

$$p_A - p_B = \frac{1}{\varepsilon} \cdot F_V \quad \text{si } OpenSw = 1 \quad (4.107)$$

donde en este caso ε es un parámetro lo suficientemente pequeño como para que el término al cual multiplica pueda considerarse despreciable. Como se muestra en las Figuras 4.22 – 4.25, este modelo para el interruptor de flujo no da lugar, en los ejemplos anteriores, a modelos cuyas fases tienen diferente número de grados de libertad.

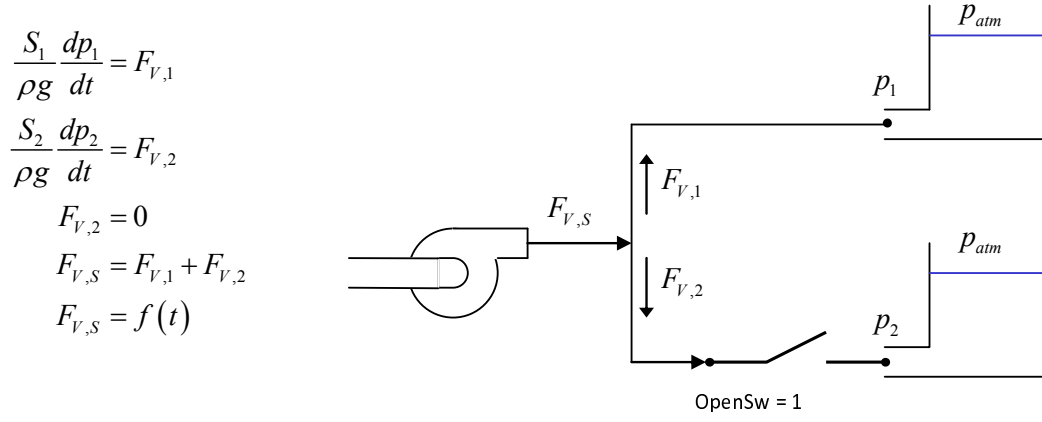


Figura 4.19: Interruptor abierto.

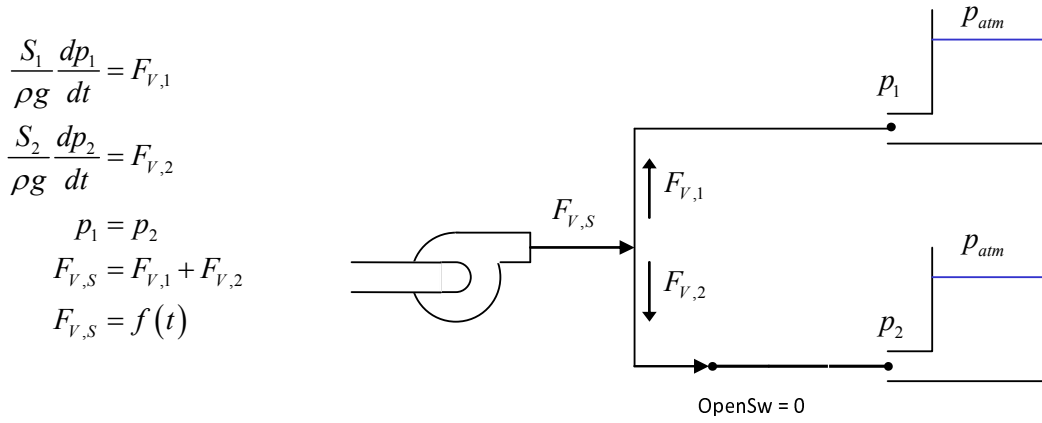


Figura 4.20: Interruptor cerrado.

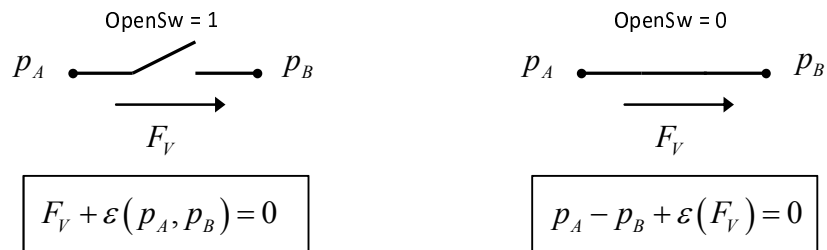


Figura 4.21: Interruptor no ideal de flujo.

F_V variable de estado

$$[p_1] = f(t)$$

$$p_1 - [p_2] = \left(\frac{1}{Sc_{desc}} \right)^2 F_V^2$$

$$p_2 - [p_3] = \frac{1}{\varepsilon} F_V$$

$$I \left[\frac{dF_V}{dt} \right] = p_3 - p_{atm}$$

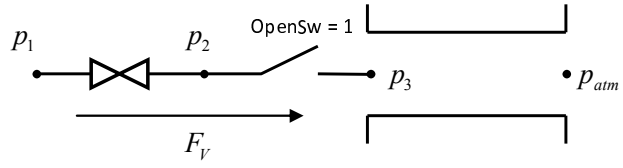


Figura 4.22: Interruptor de flujo abierto.

F_V variable de estado

$$[p_1] = f(t)$$

$$p_1 - [p_2] = \left(\frac{1}{Sc_{desc}} \right)^2 F_V^2$$

$$p_2 - [p_3] = \varepsilon F_V$$

$$I \left[\frac{dF_V}{dt} \right] = p_3 - p_{atm}$$

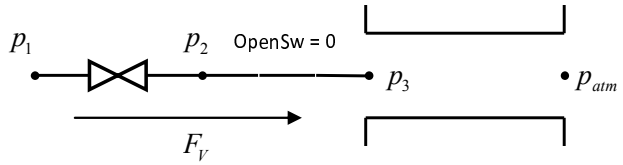


Figura 4.23: Interruptor de flujo cerrado.

p_1, p_2 variables de estado

$$[F_{V,S}] = f(t)$$

$$p_1 - p_2 = \frac{1}{\varepsilon} [F_{V,2}]$$

$$F_{V,S} = [F_{V,1}] + F_{V,2}$$

$$\frac{S_1}{\rho g} \left[\frac{dp_1}{dt} \right] = F_{V,1}$$

$$\frac{S_2}{\rho g} \left[\frac{dp_2}{dt} \right] = F_{V,2}$$

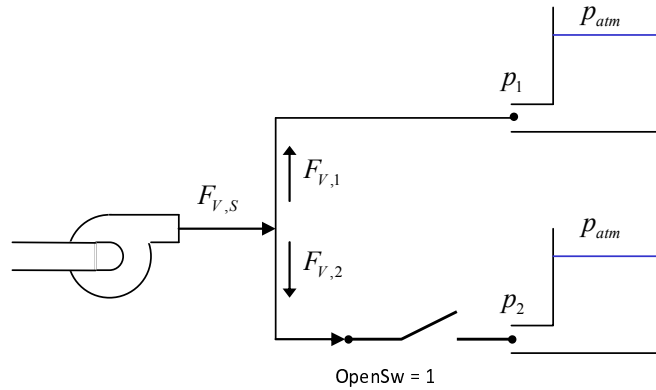


Figura 4.24: Interruptor de flujo abierto.

p_1, p_2 variables de estado

$$[F_{V,S}] = f(t)$$

$$p_1 - p_2 = \varepsilon [F_{V,2}]$$

$$F_{V,S} = [F_{V,1}] + F_{V,2}$$

$$\frac{S_1}{\rho g} \left[\frac{dp_1}{dt} \right] = F_{V,1}$$

$$\frac{S_2}{\rho g} \left[\frac{dp_2}{dt} \right] = F_{V,2}$$

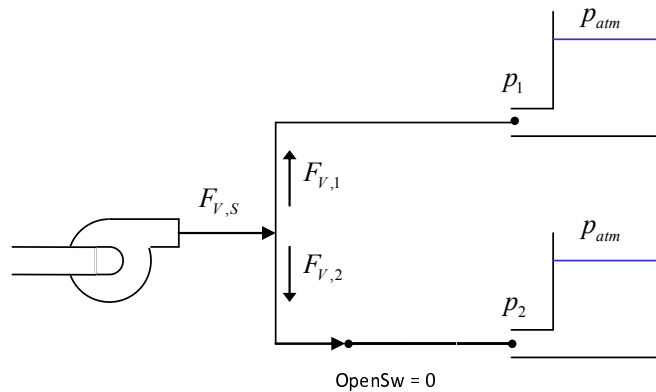


Figura 4.25: Interruptor de flujo cerrado.

4.5. MODELADO DE SISTEMAS HÍBRIDOS EN MODELICA

Un **modelo híbrido** es aquel en el cual se combina el comportamiento de tiempo continuo y los eventos. Un **evento** es un conjunto de acciones que suceden en el instante en que se verifica una condición. Así pues, la definición del evento implica especificar la condición lógica que lo habilita y las acciones que deben realizarse.

La ejecución de las acciones del evento no consume tiempo simulado. Por ello, en el mismo instante de tiempo en el cual se dispara el evento, las variables modificadas en el evento tienen dos valores: el valor antes de la ejecución del evento y el valor posterior a la ejecución del evento.

Modelica proporciona la función **pre()** para distinguir entre el valor de la variable previo al evento, **pre(variable)**, y el nuevo valor de la variable tras la ejecución del evento, que se define empleando el nombre de la variable. Así, por ejemplo, si la acción asociada a un evento es

$$x = 2 \cdot \text{pre}(x) \quad (4.108)$$

entonces cada vez que se ejecute ese evento el valor de la variable x se duplicará.

Las acciones que Modelica permite realizar en un evento son básicamente las siguientes:

- **Cambio en la estructura del modelo.** Puede emplearse un evento para señalar el cambio en la estructura matemática del modelo. Es decir, el cambio en las ecuaciones que describen el comportamiento del modelo.
- **Evaluación de variables de tiempo discreto.** La acción asociada a un evento puede ser evaluar una o varias variables de tiempo discreto del modelo. Los cambios en el valor se producen en los instantes de los eventos, manteniéndose este constante entre eventos sucesivos.
- **Reinicialización de variables de estado de tiempo continuo.** Otra acción asociada a un evento puede ser modificar el valor de una variable de tiempo continuo. Para que este cambio tenga efecto, la variable cuyo valor se modifica debe ser una variable de estado del modelo.

Para la primera se emplean la sentencia y la cláusula **if**. Para las dos últimas de emplea la cláusula **when**. A continuación se explica todo esto con detalle.

4.5.1. Sentencia y cláusula if

La sentencia y la cláusula if permiten describir modelos con estructura variable. Ambas pueden incluirse tanto en las secciones *equation*, como en las secciones *algorithm*.

Como se ha mostrado en los ejemplos anteriores, la **sentencia if** permite describir funciones con varias ramas. Su sintaxis, cuando está escrita dentro de una sección *equation* y *algorithm* respectivamente, es básicamente la siguiente:

$$\begin{array}{l} \textbf{equation} \\ \quad \textit{expr}_1 = \textbf{if } \textit{cond} \textbf{ then } \textit{expr}_2 \textbf{ else } \textit{expr}_3; \end{array} \quad (4.109)$$

$$\begin{array}{l} \textbf{algorithm} \\ \quad \textit{var} := \textbf{if } \textit{cond} \textbf{ then } \textit{expr}_1 \textbf{ else } \textit{expr}_2; \end{array}$$

donde *var* representa una variable, *cond* una expresión booleana y *expr* una expresión. Las ramas *else* pueden sustituirse por *elseif-then-else*. Por ejemplo:

$$\begin{array}{l} \textbf{equation} \\ \quad \textit{expr}_1 = \textbf{if } \textit{cond}_1 \quad \quad \textbf{then } \textit{expr}_2 \\ \quad \quad \textbf{elseif } \textit{cond}_2 \textbf{ then } \textit{expr}_3 \textbf{ else } \textit{expr}_4; \end{array} \quad (4.110)$$

$$\begin{array}{l} \textbf{algorithm} \\ \quad \textit{var} := \textbf{if } \textit{cond}_1 \quad \quad \textbf{then } \textit{expr}_1 \\ \quad \quad \textbf{elseif } \textit{cond}_2 \textbf{ then } \textit{expr}_2 \textbf{ else } \textit{expr}_3; \end{array}$$

La sintaxis de la **cláusula if** es básicamente la siguiente:

$$\begin{array}{l} \textbf{if } \textit{cond} \textbf{ then} \\ \quad \text{ecuaciones o asignaciones} \\ \textbf{else} \\ \quad \text{ecuaciones o asignaciones} \\ \textbf{end if}; \end{array} \quad (4.111)$$

Dentro de las secciones *then* y *else* de la cláusula if se escriben ecuaciones o asignaciones dependiendo de si la cláusula se encuentra dentro de una sección *equation* o *algorithm*, respectivamente. Como en el caso de las sentencias if, las ramas *else* pueden sustituirse por *elseif-then-else*. Por ejemplo,

```

if  $cond_1$  then
    ecuaciones o asignaciones
elseif  $cond_2$  then
    ecuaciones o asignaciones
else
    ecuaciones o asignaciones
end if;
    
```

(4.112)

4.5.2. Ejemplo 1: interruptor ideal eléctrico

Supongamos que se desea modelar el circuito mostrado en la Figura 4.26, que está compuesto por un generador de tensión, una resistencia y un interruptor ideal. La variable booleana *open*, cuya evolución puede verse en la figura, es una función conocida del tiempo. La descripción en Modelica del circuito se muestra en el Código 4.5. Se ha empleado una sentencia *if* para describir la relación constitutiva del interruptor ideal eléctrico.

$$0 = \text{if } open \text{ then } i \text{ else } u_D; \quad (4.113)$$

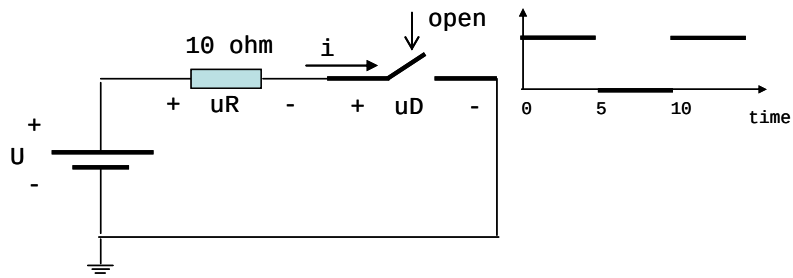
Comparando esta relación constitutiva con la mostrada en la Ec. (4.87), puede observarse una vez más la analogía existente entre los dominios hidráulico y eléctrico.

Al traducir el modelo, el entorno de modelado mostrará un aviso indicando que las ramas de la relación constitutiva del interruptor tienen diferentes unidades: la rama *i* está expresada en amperios y la rama u_D en voltios. En este modelo esto no supone ningún error, con lo cual ignoramos el aviso.

Ejecutando la simulación durante 16 s se obtienen los resultados mostrados en la Figura 4.27. Mientras el interruptor está abierto no circula corriente, con lo cual toda la tensión del generador cae en el interruptor. Mientras el interruptor está cerrado, circula corriente y toda la tensión cae en la resistencia.

4.5.3. Ejemplo 2: diodo ideal

Puede modificarse el modelo del interruptor, de manera que describa un diodo ideal. Para ello, la condición de apertura debe estar controlada por las variables internas del dispositivo, en lugar de venir impuesta por una función dada del tiempo.

Figura 4.26: Circuito eléctrico y variable booleana *open*.

```

model circuitoResistConmut
  Modelica.SIunits.Voltage U, uR, uD;
  Modelica.SIunits.Current i;
  Boolean open;
  parameter Modelica.SIunits.Resistance R = 10;
  parameter Modelica.SIunits.Voltage U0 = 5;
  parameter Modelica.SIunits.AngularFrequency w = 2;
equation
  U = U0*sin(w*time); // Generador
  uR = i*R; // Resistencia
  0 = if open then i else uD; // Interruptor
  open = time < 5 or time > 10; // Control del interruptor
  U = uR + uD; // Ecuación de malla
end circuitoResistConmut;

```

Código 4.5: Circuito de la Figura 4.26.

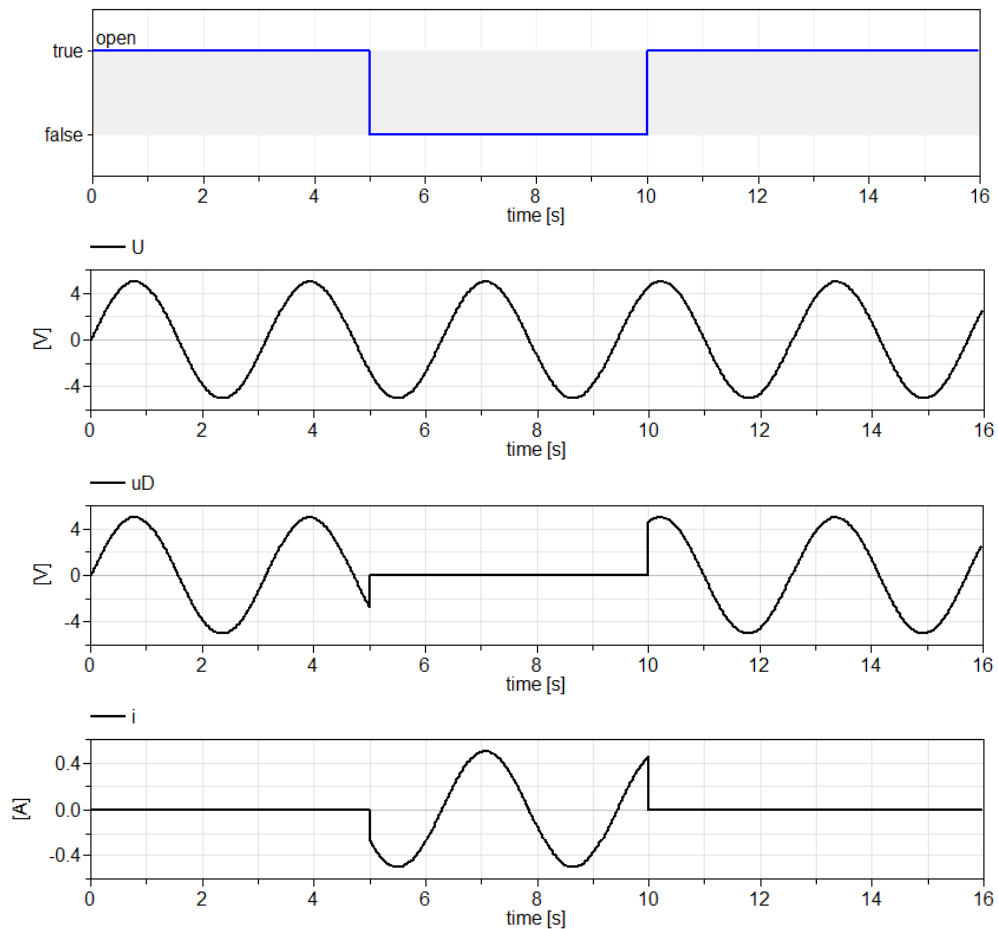


Figura 4.27: Simulación del Código 4.5.

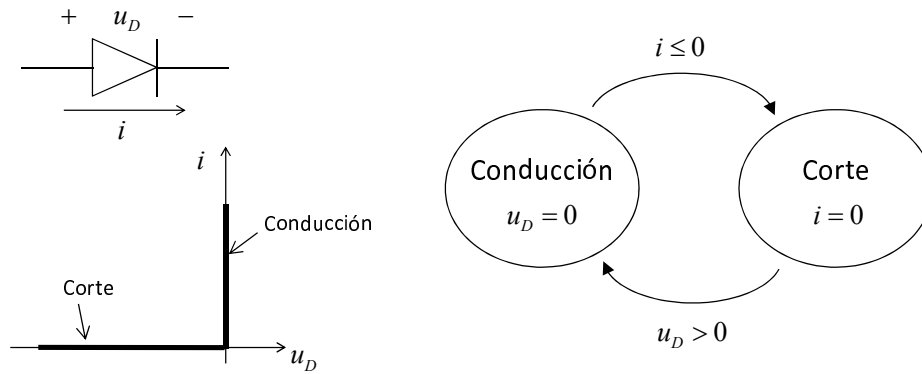


Figura 4.28: Curva I-V característica de un diodo ideal y su modelo con dos fases.

El diodo ideal puede encontrarse en dos fases: conducción y corte, que son análogas a las fases cerrado y abierto de un interruptor. El comportamiento del diodo se modela de la forma siguiente (véase la Figura 4.28):

- Mientras está en la **fase conducción**, la caída de tensión entre sus bornes es cero y debe satisfacerse que la corriente que circula a través suyo tenga signo positivo. Si esta última condición no se satisface, pasa a la *fase corte*.
- Mientras está en la **fase corte**, la corriente que circula a su través es cero y debe satisfacerse que la caída de tensión entre sus bornes debe ser menor o igual que cero. Si esta última condición no se satisface, pasa a la fase conducción.

La relación constitutiva del diodo ideal puede describirse en Modelica mediante las dos ecuaciones siguientes:

$$\begin{aligned} 0 &= \text{if } \text{corte} \text{ then } i \text{ else } u_D; \\ \text{corte} &= i \leq 0 \text{ and not } u_D > 0; \end{aligned} \quad (4.114)$$

Obsérvese que la variable booleana *corte* es evaluada de la segunda ecuación, a partir de una expresión lógica en la que intervienen dos variables de tiempo continuo.

Con el fin de ilustrar la aplicación del modelo anterior, consideremos el circuito rectificador mostrado en la Figura 4.29. En el diagrama del circuito se ha asignado nombre a los nodos y a las corrientes que circulan a través de los componentes. El modelo del circuito se muestra a continuación.

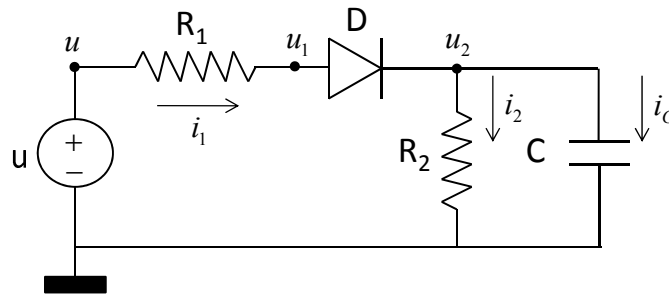


Figura 4.29: Circuito rectificador.

```

model circuitoDiodoIdeal
  import SI = Modelica.SIunits;
  SI.Current i_1, i_2, i_C;
  SI.Voltage u, u_1, u_2(start=0, fixed=true);
  Boolean corte "Fase del diodo";
  parameter SI.Voltage U0=5;
  parameter SI.Frequency frec=50;
  parameter SI.AngularFrequency w=2*Modelica.Constants.pi*frec;
  parameter SI.Resistance R1=10, R2=50;
  parameter SI.Capacitance C=1e-3;
equation
  u = U0*sin(w*time);
  u - u_1 = i_1*R1;
  0 = if corte then i_1 else u_1 - u_2;
  corte = i_1 <= 0 and not u_1 - u_2 > 0;
  u_2 = i_2*R2;
  C*der(u_2) = i_C;
  i_1 = i_2 + i_C;
end circuitoDiodoIdeal;

```

Código 4.6: Circuito rectificador de la Figura 4.29 con diodo ideal.

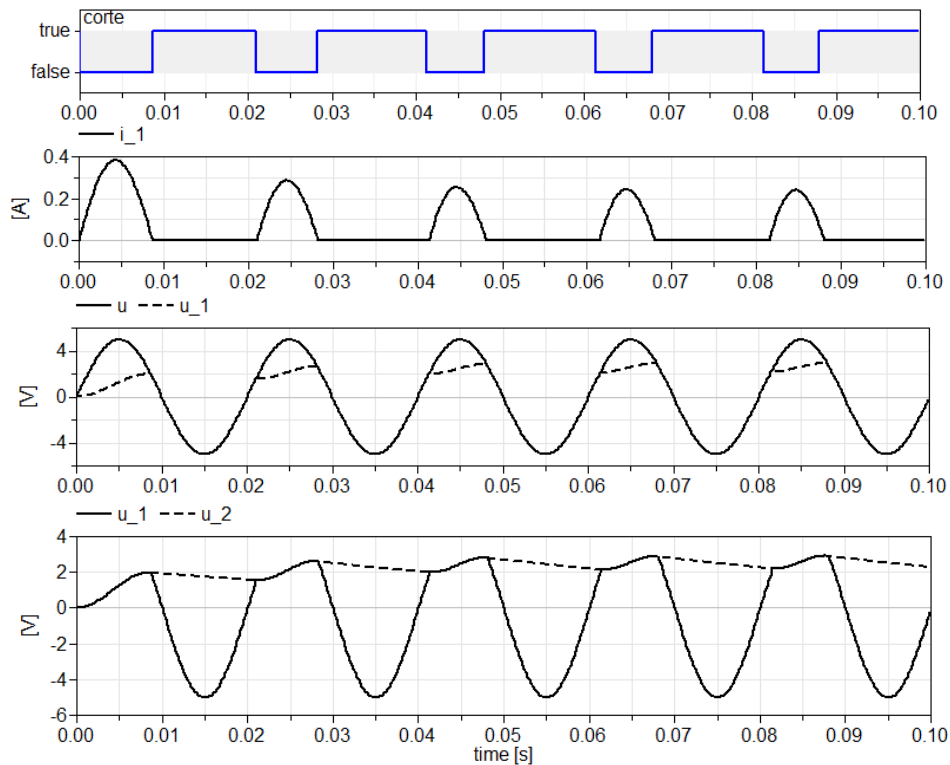


Figura 4.30: Resultado de simular el Código 4.6.

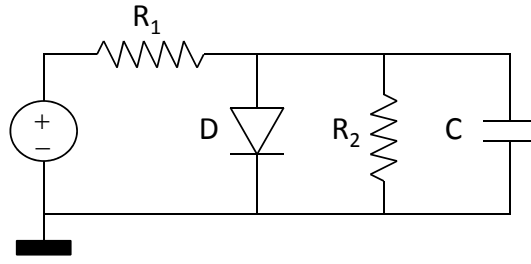


Figura 4.31: Circuito eléctrico en el cual no debe emplearse la relación constitutiva (4.114) para describir el diodo.

$$u = U_0 \cdot \sin(w \cdot t) \quad (4.115)$$

$$u - u_1 = i_1 \cdot R_1 \quad (4.116)$$

$$0 = \text{if } corte \text{ then } i_1 \text{ else } u_1 - u_2 \quad (4.117)$$

$$corte = i_1 \leq 0 \text{ and not } u_1 - u_2 > 0 \quad (4.118)$$

$$u_2 = i_2 \cdot R_2 \quad (4.119)$$

$$C \cdot \frac{du_2}{dt} = i_C \quad (4.120)$$

$$i_1 = i_2 + i_C \quad (4.121)$$

La descripción en Modelica se muestra en el Código 4.6. Simulando el modelo durante 0.1 s se obtienen los resultados de la Figura 4.30. Obsérvese que mientras el diodo está en la fase corte, la corriente a través del diodo vale cero, con lo cual no hay caída de tensión en la resistencia R_1 , lo que implica que u_1 sea igual a u . Mientras el diodo no está en la fase corte, la caída de tensión entre sus bornes es cero, es decir, se verifica que u_1 es igual a u_2 . Parte de la tensión del generador cae en la resistencia R_1 , y el resto en el paralelo de R_2 y C .

El modelo del diodo ideal no debe emplearse en modelos en los cuales el cambio de fase del diodo implique un cambio en el número de grados de libertad del modelo completo.

Por ejemplo, no debería emplearse el modelo del diodo ideal para modelar el circuito mostrado en la Figura 2.15, que se repite en la Figura 4.31 para mayor comodidad del lector. Si se emplea el modelo del diodo ideal en este circuito, el entorno de modelado mostrará un mensaje de error en tiempo de simulación. Se produce la situación siguiente.

- El modelo del circuito tiene un grado de libertad mientras el diodo está en corte. La relación constitutiva del diodo impone que no circule corriente a su

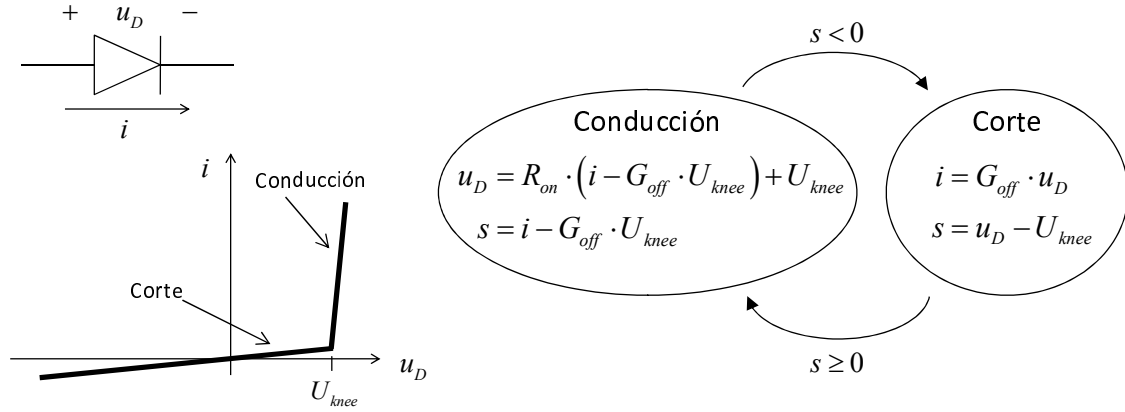


Figura 4.32: Modelo del diodo ideal incluido en la MSL, donde U_{knee} representa la tensión umbral del diodo y s es una variable auxiliar.

través, con lo cual la corriente proporcionada por el generador, que circula íntegramente a través de R_1 , se distribuye entre R_2 y C . La caída de tensión entre los bornes del condensador podría seleccionarse como variable de estado.

- Sin embargo, mientras el diodo está en conducción, la relación constitutiva del diodo impone que la caída de tensión entre sus bornes sea cero. Esto implica que la caída de tensión entre los bornes del condensador se calcule de la relación constitutiva del diodo, con lo cual dicha variable no es una variable de estado. Mientras el modelo está en esta fase, tiene cero grados de libertad.

Para evitar que el cambio en la fase del diodo produzca un cambio en el número de grados de libertad del modelo, el modelo del diodo ideal incluido en la Librería Estándar de Modelica (MSL) incorpora una pequeña resistencia (R_{on}) cuando está en conducción y una pequeña conductancia (G_{off}) cuando está en corte. La relación constitutiva del diodo ideal, en la versión 3.2.1 de la MSL que es distribuida con Dymola 2015, es la siguiente (véase la Figura 4.32):

$$\begin{aligned}
 corte &= s < 0 \\
 u_D &= (s \cdot unitCurrent) \cdot (\text{if } corte \text{ then } 1 \text{ else } R_{on}) + U_{knee} \\
 i &= (s \cdot unitVoltage) \cdot (\text{if } corte \text{ then } G_{off} \text{ else } 1) + G_{off} \cdot U_{knee}
 \end{aligned} \tag{4.122}$$

donde el parámetro U_{knee} representa la tensión umbral del diodo, es decir, el valor de u_D para el cual se produce el cambio en la pendiente de la característica I-V. Por defecto, U_{knee} vale cero. El valor por defecto de la resistencia R_{on} es 10^{-5} ohm y el de la conductancia G_{off} es 10^{-5} ohm $^{-1}$. El valor de estos tres parámetros puede ser modificado al instanciar el componente y al definir el experimento.

La variable auxiliar s , que determina el punto de operación sobre la característica I-V, es una variable adimensional (`unit="1"`) de tipo real. Mientras el diodo está en corte, s es igual a $u_D - U_{knee}$. Mientras está en conducción, s es igual a $i - G_{off} \cdot U_{knee}$. Si $s = 0$, el diodo opera en la tensión umbral U_{knee} . Mientras $s < 0$, el punto de operación está por debajo de la tensión umbral, con lo cual el diodo está en corte. Mientras $s > 0$, el punto de operación está por encima de la tensión umbral, con lo cual el diodo está en conducción.

Los constantes `unitCurrent` y `unitVoltage` tienen la finalidad de evitar que el entorno de modelado detecte inconsistencia en las unidades de la segunda y tercera ecuaciones de la relación constitutiva (4.122). Se declaran de la forma siguiente:

```
constant Modelica.SIunits.Voltage unitVoltage = 1;
constant Modelica.SIunits.Current unitCurrent = 1;
```

4.5.4. Cláusula when

La sintaxis de la cláusula `when` de Modelica es básicamente la siguiente:

```
when cond then
    ecuaciones instantáneas
end when;          (4.123)
```

donde *cond* es una expresión booleana. En este caso, las ecuaciones instantáneas escritas dentro del cuerpo de la cláusula `when` se ejecutan sólo en el instante en el cual la expresión booleana pasa de valer *false* a valer *true*.

También es posible especificar la condición de disparo de la cláusula mediante un vector de expresiones booleanas:

```
when {cond1, ..., condn} then
    ecuaciones instantáneas
end when;          (4.124)
```

En este último caso, la cláusula `when` se dispara cuando cualquiera de las componentes del vector pasa de valer *false* a valer *true*.

A fin de ilustrar la diferencia entre estas dos formas de especificar la condición de disparo, obsérvense las dos cláusulas `when` mostradas a continuación:

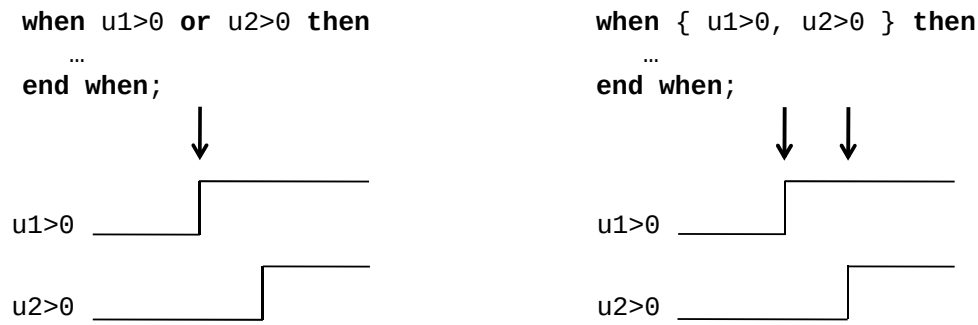


Figura 4.33: Ejemplo que muestra la diferencia entre especificar la condición de disparo mediante una expresión booleana (izquierda) y mediante un vector de expresiones booleanas (derecha). Las flechas verticales indican el disparo de la cláusula when.

```
when u1>0 or u2>0 then
    ...
    b1 = not pre(b1);
end when;
```

```
when { u1>0 , u2>0 } then
    ...
    b2 = not pre(b2);
end when;
```

La cláusula de la parte izquierda se dispara en el instante en que la expresión booleana ($u1>0$ or $u2>0$) pasa de valer *false* a valer *true*. La cláusula de la parte derecha se dispara cuando $u1>0$ pasa de *false* a *true* y también cuando $u2>0$ pasa de *false* a *true*. Véase la Figura 4.33.

Las **ecuaciones instantáneas** pueden ser de dos tipos:

- **Ecuaciones en diferencias.** Asignaciones en las cuales se evalúe el nuevo valor de variables de tiempo discreto. Dependiendo de si la cláusula when está incluida dentro de una sección equation o algorithm, estas asignaciones son de la forma:

$$var = expr; \quad (4.125)$$

o bien de la forma:

$$var := expr; \quad (4.126)$$

Como ya se ha indicado, la función **pre()** permite distinguir entre el valor de la variable previo al evento, **pre(variable)**, y el nuevo valor de la variable tras la ejecución del evento, que se define empleando el nombre de la variable.

- **Sentencias reinit**, en las cuales se modifica el valor de variables de estado de tiempo continuo. Como ya se explicó, la función reinit se invoca de la forma siguiente:

reinit(*var*, *expr*); (4.127)

Tiene dos argumentos. El primero es la variable de estado cuyo valor va a modificarse. El segundo es la expresión que debe emplearse para evaluar el nuevo valor de la variable de estado.

Obsérvese que aunque la cláusula *when* esté dentro de una sección *equation*, las ecuaciones para evaluar las variables de tiempo discreto deben escribirse como asignaciones usando, no obstante, el signo de igualdad (=) en lugar del signo de asignación (:=). Es decir, deben escribirse con la variable a evaluar despejada en el lado izquierdo de la igualdad.

Dependiendo de que la cláusula *when* esté dentro de una sección *equation* o *algorithm*, rige o no la **regla de asignación única** dentro del cuerpo de la cláusula *when*.

- **Sección *equation*.** Cuando la cláusula *when* está dentro de una sección *equation*, el orden en que se escriben las ecuaciones del cuerpo de la cláusula es indiferente, puesto que el entorno de modelado se encarga de ordenarlas. Para que esto sea posible, se impone la **regla de asignación única**: *cada variable sólo puede ser evaluada de una única ecuación*.
- **Sección *algorithm*.** Las asignaciones de una sección *algorithm* son consideradas un conjunto indivisible, y se ejecutan en el orden en que se han declarado. Por ello, dentro de una sección *algorithm* **no aplica la regla de asignación única**. Esto significa que pueden realizarse varias asignaciones a una misma variable dentro de una misma sección algoritmo. Por ejemplo, es posible escribir:

```
algorithm
  when h1>3 then
    cerrado := true;
  end when;
  when h2>1 then
    cerrado := false;
  end when;
```

puesto que si simultáneamente se activan los dos eventos, al ser ejecutados en orden, la variable *cerrado* valdría *false*.

No obstante, puesto que los bloques *algorithm* deben ser ordenados junto con el resto del modelo, no puede realizarse en un bloque *algorithm* una asignación

a una variable que sea evaluada en otro bloque `algorithm` o en otra ecuación del modelo.

Resumiendo lo anterior, dado que las secciones `algorithm` deben ser ordenadas junto con el resto del modelo, una variable sólo puede ser evaluada en una única sección `algorithm`, empleando para ello una o varias asignaciones dentro de dicha sección `algorithm`.

Las funciones `sample`, `initial` y `terminal` pueden emplearse en la **condición booleana** de la cláusula `when`.

- La función **sample** (que podría traducirse por “muestreo”) dispara la cláusula `when` periódicamente, cada cierto periodo de tiempo, comenzando por un determinado instante inicial. Por ejemplo, la cláusula

```
when sample(t0, T) then
  x = a*pre(x) + b*u;
  y = c*pre(x) + d*u;
end when;
```

se dispara en los instantes de tiempo $t_0 + n \cdot T$, con $n = 0, 1, 2, \dots$

- La función **initial** dispara la cláusula `when` en la inicialización del modelo. Con ello se consigue que las ecuaciones incluidas en el cuerpo de la cláusula intervengan en el problema inicial. Explicaremos esto detalladamente en la Sección 4.6. La sintaxis es:

```
when initial() then
  ...
end when;
```

(4.128)

- La función **terminal** dispara la cláusula en el instante en el cual se verifica la condición de finalización de la simulación. La sintaxis es:

```
when terminal() then
  ...
end when;
```

(4.129)

Otra aplicación de la cláusula `when` es forzar la **finalización de la simulación**. En los ejemplos mostrados hasta ahora la condición de finalización era que el tiempo simulado alcance un cierto valor establecido al definir el experimento. Sin embargo,

en algunos estudios de simulación la condición de terminación no depende sólo del tiempo, sino de variables del modelo. La descripción de estas condiciones de finalización puede hacerse mediante una llamada a la función **terminate** en el cuerpo de una cláusula **when**.

La función **terminate** tiene un único argumento de tipo **String**: el mensaje que debe mostrarse al terminar la simulación. Por ejemplo:

```
Modelica.SIunits.Temperature T          "Temperatura de la mezcla";
parameter Real eps (unit="K/s") = 1e-3 "Valor pequeño";
equation
  when abs(der(T)) < eps then
    terminate("Alcanzado estado estacionario");
  end when;
```

Entre los tipos predefinidos de variable de Modelica se encuentra **String**. Las variables de tipo **String** almacenan cadenas de caracteres. Para trabajar con **Strings**, resulta útil saber que:

- El operador **+** concatena **Strings**.
- Dentro de la cadena de caracteres pueden incluirse saltos de línea **\n** y tabuladores **\t**.
- Las funciones **realString** e **integerString** permiten realizar la conversión de los tipos **Real** e **Integer** a **String**, respectivamente. Ambas funciones tienen tres argumentos de entrada y una salida:

```
function realString "Convert a real to a string."
  input Real number          "The number to convert to a string";
  input Integer minWidth := 1 "Minimum width of result";
  input Integer precision := 6 "Number of significant digits";
  output String result;
```

```
function integerString "Convert an integer to a string."
  input Integer number       "The number to convert to a string";
  input Integer minWidth := 1 "Minimum width of result";
  input Integer precision := 1 "Minimum number of digits";
  output String result;
```

Otra aplicación de la cláusula **when** es la **escritura en la ventana de mensajes**, lo cual puede ser útil para la depuración de los modelos o para obtener información sobre el valor de determinadas variables en instantes específicos de la

simulación. La función **LogVariable** permite escribir el valor de una variable en la ventana de mensajes del entorno de modelado. Por ejemplo, el siguiente código escribe el valor de la variable x cada 0.1 s.

```
equation
  when sample(0,0.1) then
    LogVariable(x);
  end when;
```

4.5.5. Ejemplo 3: dos depósitos conectados mediante una válvula

Consideremos nuevamente el sistema descrito en la Sección 4.2.6, que está compuesto por dos depósitos de líquido conectados mediante una válvula. En la Figura 4.34 se muestra nuevamente el diagrama y las ecuaciones del modelo, indicándose además el valor de los parámetros del modelo y de las condiciones iniciales. El objetivo es describir este modelo en Modelica y simularlo durante 200 s.

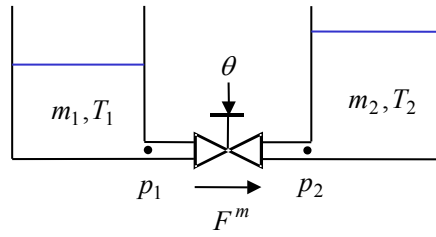
El modelo se muestra en el Código 4.7. Se emplean sentencias `if` para describir el caudal másico y la temperatura del líquido que circula a través de la válvula, y una sentencia `when` para representar el cambio en la apertura de la válvula, que se produce cuando el tiempo simulado supera el valor $t_0 = 50$ s.

En la Figura 4.35 se muestra el resultado de la simulación. Puede observarse que mientras la variable `theta` vale cero el flujo entre los depósitos es cero. Se mantienen por tanto constantes la masa y temperatura en cada uno de los depósitos.

Cuando en el instante 50 s la variable `theta` pasa a valer 0.5, el flujo másico entre los depósitos se hace instantáneamente distinto de cero. Puesto que en ese instante la altura de líquido es mayor en el segundo depósito, el líquido fluye desde el segundo depósito hacia el primero.

El líquido que circula a través de la válvula se encuentra a la misma temperatura a la que está el líquido en el depósito situado aguas arriba. Así pues, el líquido circulante se encuentra a la temperatura del segundo depósito, la cual es mayor que la temperatura del primer depósito. El líquido del primer depósito se va calentando a medida que va entrando el líquido proveniente del segundo depósito.

A medida que la altura de líquido en los depósitos va igualándose, el flujo a través de la válvula va tendiendo a cero. Cuando las alturas (y consiguientemente las presiones) se igualan, el flujo se hace cero. Al no entrar más líquido en el



Modelo del primer depósito	Modelo de la válvula	Modelo del segundo depósito
$\frac{dm_1}{dt} = -F^m$ $m_1 C_p \frac{dT_1}{dt} = -F^m C_p (T_f - T_1)$ $p_1 = \frac{m_1 g}{S_1}$ $m_1 = \rho S_1 h_1$	$T_f = \begin{cases} T_1 & \text{si } p_1 > p_2 \\ T_2 & \text{en caso contrario} \end{cases}$ $F^m = \begin{cases} K_v \theta \sqrt{p_1 - p_2} & \text{si } p_1 > p_2 \\ -K_v \theta \sqrt{p_2 - p_1} & \text{en caso contrario} \end{cases}$ when time > t₀ then $\theta = \theta_0$ end when	$\frac{dm_2}{dt} = F^m$ $m_2 C_p \frac{dT_2}{dt} = F^m C_p (T_f - T_2)$ $p_2 = \frac{m_2 g}{S_2}$ $m_2 = \rho S_2 h_2$

Condiciones iniciales: $m_1 = 1000 \text{ Kg}$, $m_2 = 4000 \text{ Kg}$, $T_1 = 300 \text{ K}$, $T_2 = 350 \text{ K}$, $\theta = 0 \text{ rad}$

Parámetros: $\rho = 1000 \text{ Kg/m}^3$, $S_1 = 1 \text{ m}^2$, $S_2 = 2 \text{ m}^2$, $K_v = 0.2$, $\theta_0 = 0.5 \text{ rad}$, $t_0 = 50 \text{ s}$

Figura 4.34: Dos depósitos conectados mediante una válvula.

```

model depositosValvula
import SI = Modelica.SIunits;
constant SI.Acceleration g=9.81 "Aceleración gravitatoria";
parameter SI.Area S1 = 1, S2 = 2;
parameter Real Kv = 0.2;
parameter Real theta0(unit="1") = 0.5;
parameter SI.Time t0 = 50;
parameter SI.Density rho = 1000;
SI.Mass m1(start=1000,fixed=true), m2(start=4000,fixed=true);
SI.Temperature T1(start=300,fixed=true), T2(start=350,fixed=true), Tf;
SI.Height h1, h2;
SI.Pressure p1, p2;
SI.MassFlowRate Fm;
Real theta( start=0, fixed=true, unit="1");
equation
// Depósito 1
der(m1) = -Fm;
m1 * der(T1) = -Fm * (Tf-T1);
p1 = m1 * g / S1;
m1 = rho * h1 * S1;
// Depósito 2
der(m2) = Fm;
m2 * der(T2) = Fm * (Tf-T2);
p2 = m2 * g / S2;
m2 = rho * h2 * S2;
// Válvula
Tf = if p1 > p2 then T1 else T2;
Fm = if p1 > p2 then Kv*theta*sqrt(abs(p1-p2))
      else -Kv*theta*sqrt(abs(p2-p1));
when time > t0 then
    theta = theta0;
end when;
end depositosValvula;
    
```

Código 4.7: Dos depósitos conectados mediante una válvula.

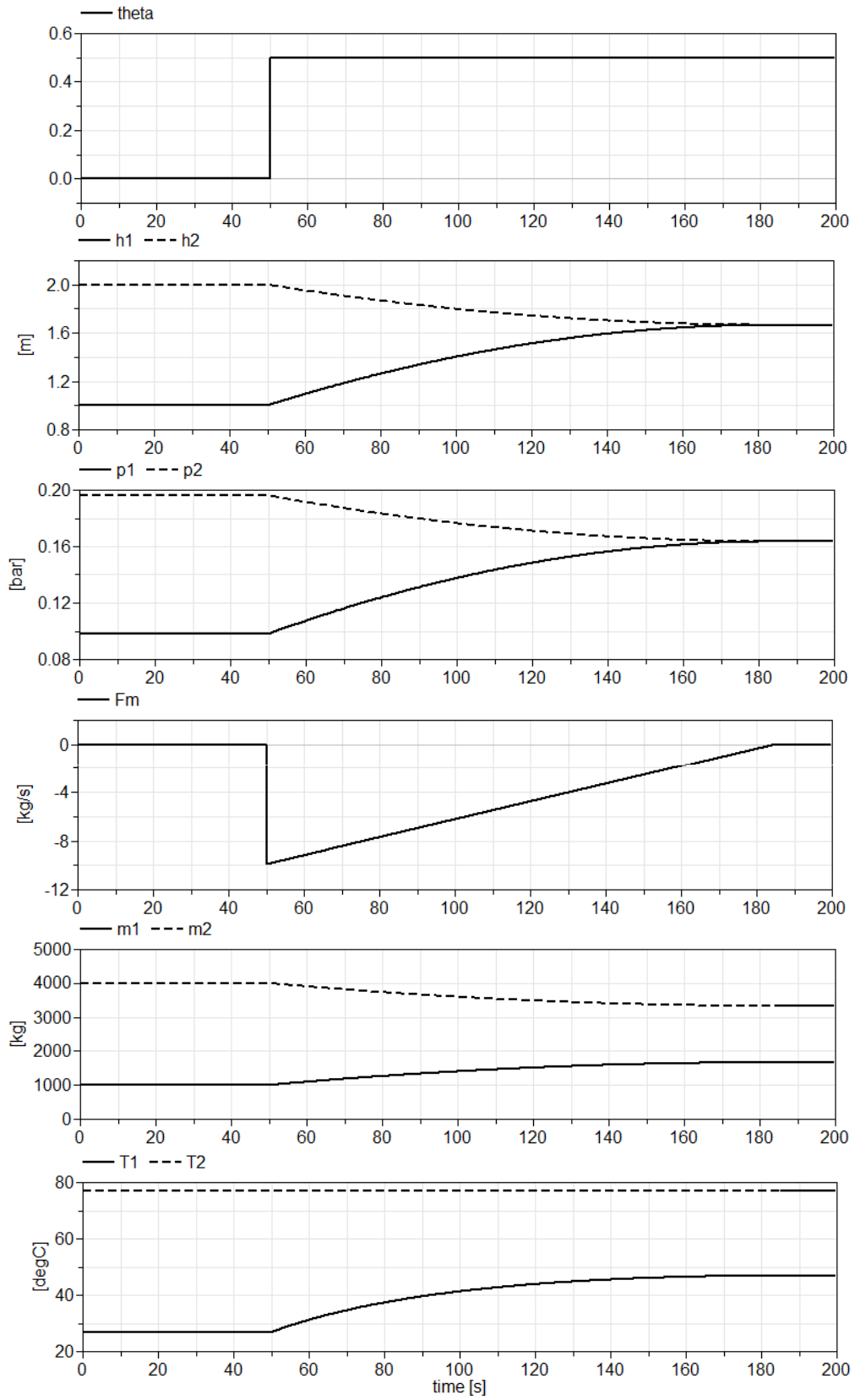


Figura 4.35: Simulación del modelo mostrado en el Código 4.7.

primer depósito, su temperatura no varía. La temperatura del segundo depósito ha permanecido constante durante toda la simulación, ya que en ningún momento se ha producido entrada de líquido en ese depósito.

4.5.6. Ejemplo 4: rebote de una pelota

Consideremos nuevamente el movimiento vertical de una pelota que cae bajo la acción de la gravedad y rebota contra el suelo. El sistema fue descrito en la Sección 4.2.4 y se han propuesto dos modelos diferentes que, como hemos visto, no han resultado satisfactorios: el modelo planteado en el Código 4.3 permite la caída de la pelota por debajo del nivel del suelo y el modelo planteado en el Código 4.4 produce chattering.

Modifiquemos el comportamiento del modelo. Supongamos ahora que se define la energía específica (energía por unidad de masa) de la pelota de la manera siguiente:

$$e = \frac{1}{2} \cdot v^2 + g \cdot x \quad (4.130)$$

donde el primer término corresponde con su energía cinética y el segundo con su energía potencial.

La energía específica se calcula al inicio de la simulación, $e_{inicial}$, y tras cada rebote. Cuando la energía específica tras un rebote se hace menor que una cierta proporción de la inicial, la pelota entra en una fase, que denominaremos *Parada*, en la cual se impone que su posición y velocidad sean igual a cero. Obsérvese que la carga computacional del modelo cuando está en la fase *Parada* es muy baja y se evita el problema del chattering.

Para evitar que la pelota caiga por debajo del nivel del suelo, tras cada rebote se comprueba que la energía cinética de la pelota sea suficiente para ascender una altura superior a $(2 \cdot e_{veps})$. En caso contrario termina la simulación, mostrando el mensaje de que el modelo ha salido fuera de su marco experimental. Con ello se quiere indicar al usuario que está usando el modelo en un contexto experimental para el cual no es válido.

El comportamiento del modelo está representado en la Figura 4.36. El punto negro relleno representa la entrada al diagrama. Desde él se produce una transición incondicional a la fase *Movimiento*, en la cual el modelo describe la caída y rebote de la pelota. La energía de la pelota cambia tras cada rebote. Si la pelota sale de su marco experimental, finaliza la simulación. El final de la simulación se representa

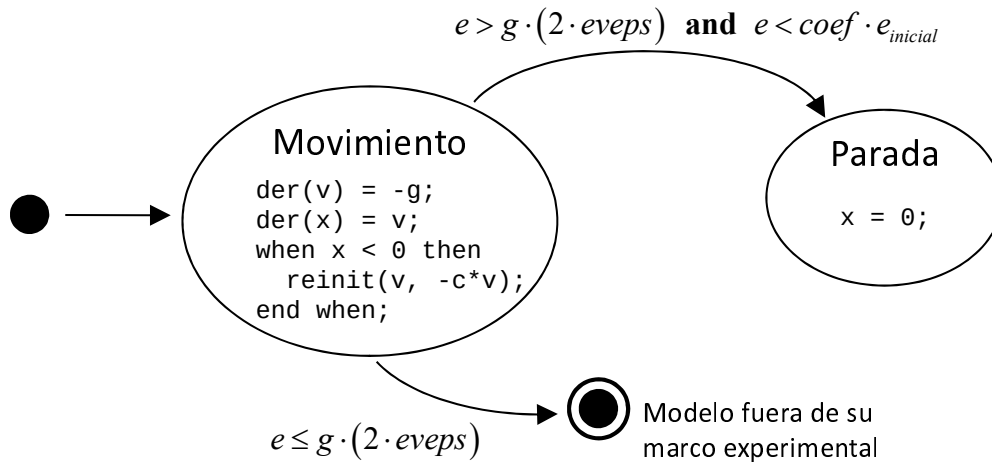


Figura 4.36: Modelo del rebote de la pelota con dos fases: movimiento y parada.

model pelota3

```

import SI = Modelica.SIunits;

constant Real eveps(unit="1") = 1E-10;
constant SI.Acceleration g=9.8 "Aceleración gravitatoria";

parameter Real c(unit="1")=0.8 "Coef. rebote elastico";
parameter Real coef(unit="1")=1e-3 "Coef. transición a fase Parada";

SI.Distance x(start=10, fixed=true) "Altura";
SI.Velocity v(start=0, fixed=true) "Velocidad";
Real eInicial(unit="J/kg") "Energía específica inicial";
Boolean parada(start=false, fixed=true);

equation

der(v) = if parada then 0 else -g;
der(x) = v;

when initial() then
  eInicial = 0.5*v^2 + g*x;
end when;

when x < 0 then
  assert(0.5*(c*v)^2 > g*2*eveps, "Modelo fuera de su marco experimental");
  parada = (0.5*(c*v)^2 < coef*eInicial);
  reinit(v, if parada then 0 else -c*v);
end when;

when parada then
  reinit(x, 0);
end when;

end pelota3;
  
```

Código 4.8: Rebote de la pelota con dos fases.

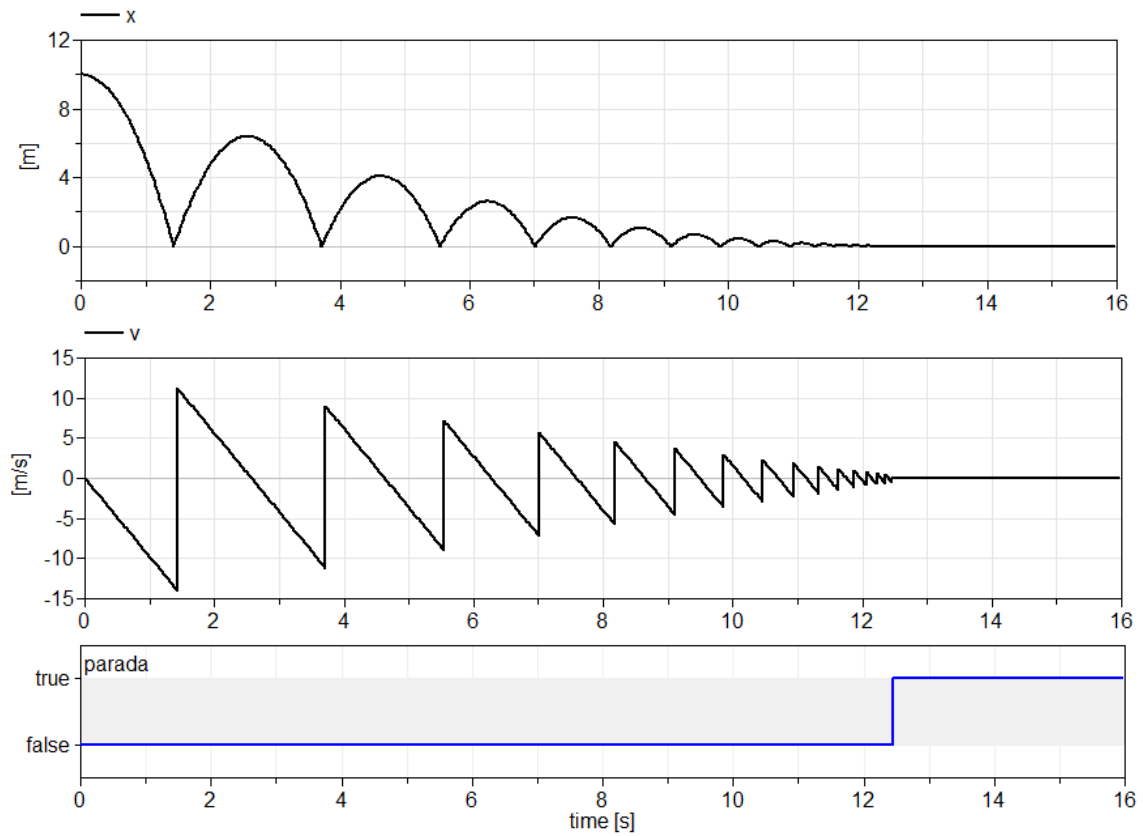


Figura 4.37: Resultado de simular el Código 4.8.

mediante dos círculos concéntricos, de manera que el círculo exterior está hueco y el interior relleno de color negro. Si, dentro del marco experimental, la energía específica es menor que $coef \cdot e_{inicial}$, se produce la transición a la fase *Parada*, en la cual el modelo permanece indefinidamente.

La descripción en Modelica del modelo se muestra en el Código 4.8. Obsérvese que:

- La energía específica inicial se calcula empleando una cláusula when cuya condición es la función `initial()`. Se asigna un valor a la variable `eInicial` en el instante inicial y este valor permanece constante durante toda la simulación.
- Se emplea una sentencia `assert` para comprobar si el modelo ha salido de su marco experimental.
- El modelo tiene dos variables de estado en ambas fases. Al producirse el cambio a la fase *Parada* se reinician las variables de estado x y v al valor cero, y mientras el modelo se encuentra en esa fase las derivadas respecto al tiempo de ambas variables valen cero. De esta forma se consigue que tanto la

posición como la velocidad valgan cero mientras el modelo se encuentra en la fase *Parada*.

4.5.7. Ejemplo 5: fricción seca

Supongamos que se desea modelar la fuerza de fricción entre dos objetos sólidos que se encuentran en contacto y que pueden deslizar el uno sobre el otro. El modelo planteado viene descrito por la curva característica de la Figura 4.38, la cual depende de tres parámetros: R_0 , R_m y R_v . La curva debe interpretarse de la manera siguiente.

- Mientras la velocidad relativa (v) entre los objetos sea diferente de cero, la fuerza de fricción (f_r) depende linealmente de dicha velocidad relativa.

$$f_r = \begin{cases} R_v \cdot v - R_m & \text{si } v < 0 \\ R_v \cdot v + R_m & \text{si } v > 0 \end{cases} \quad (4.131)$$

- Si la velocidad relativa entre los objetos es cero y se aplica una fuerza externa, la fuerza de fricción se opone a ella, de manera que:
 - Si el módulo de la fuerza externa aplicada no es mayor que un cierto valor R_0 , la fuerza de fricción contrarresta exactamente la fuerza aplicada, de manera que los objetos permanezcan en reposo relativo.
 - Si el módulo de la fuerza externa aplicada es mayor que R_0 , entonces la fuerza de fricción no es capaz de impedir el movimiento relativo de los objetos, calculándose entonces la fuerza de fricción mediante la Ec. (4.131).

De la descripción anterior se deduce que la fuerza de fricción podría describirse mediante el siguiente modelo con dos fases.

1. En esta fase la velocidad entre los dos cuerpos es diferente de cero. En esta fase, la fuerza de fricción se relaciona con la velocidad relativa mediante la Ec. (4.131). La condición para abandonar esta fase es que la velocidad relativa se haga cero.
2. En esta fase, que llamaremos *Stuck*, la velocidad relativa es cero. El valor de la fuerza de fricción es aquel que hace que la velocidad relativa continúe siendo cero. Si llamamos f_c al valor de la fuerza de fricción en esta situación, éste se calcula imponiendo que se satisfaga la condición $v = 0$. La condición para abandonar esta fase es $|f_c| > R_0$.

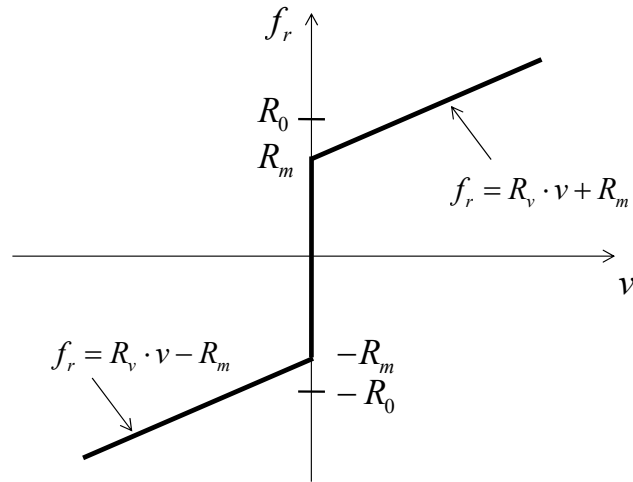


Figura 4.38: Curva característica de la fuerza de fricción seca.

El modelo en la fase *Stuck* tiene una ecuación ($v = 0$) más y una variable (f_c) más que en la otra fase. Dado que los entornos de modelado de Modelica no permiten declarar variables válidas únicamente mientras el modelo está en determinada fase, es preciso asignar un valor a la variable f_c mientras el modelo no se encuentra en la fase *Stuck*. Dicho valor es arbitrario, ya que f_c sólo tiene sentido físico mientras el modelo está en la fase *Stuck*. Por simplicidad, se asigna el valor cero. Procediendo de esta manera, la fuerza de fricción vendría descrita por la relación constitutiva siguiente.

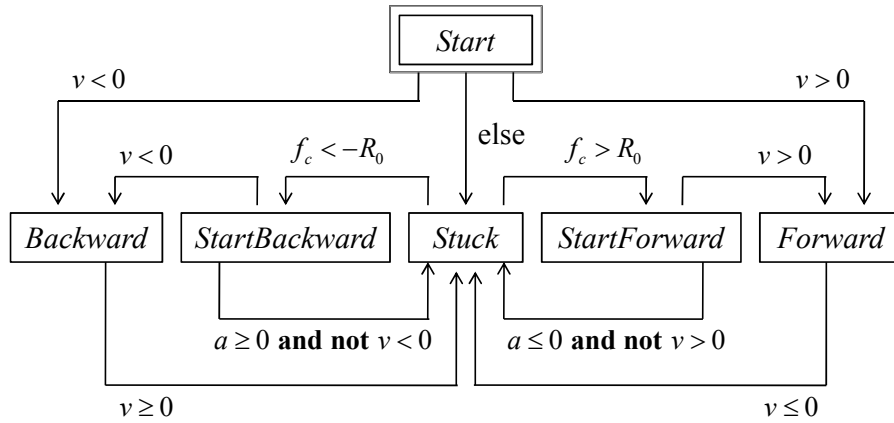
$$\begin{aligned}
 f_r &= \text{if } v > 0 \text{ then } R_v \cdot v + R_m \\
 &\quad \text{elseif } v < 0 \text{ then } R_v \cdot v - R_m \\
 &\quad \text{else } f_c \\
 0 &= \text{if } Stuck \text{ then } v \\
 &\quad \text{else } f_c
 \end{aligned} \tag{4.132}$$

Sin embargo, el modelo no debe formularse de esta manera. El motivo es que en los modelos de sistemas mecánicos las velocidades o bien son variables de estado, o bien son calculadas a partir de variables de estado. Por ello, el modelo de la fricción en la fase *Stuck*, al imponer $v = 0$, reduce el número de grados de libertad del modelo. Veamos a continuación una posible alternativa.

Dado que la aceleración no es variable de estado en los modelos de sistemas mecánicos, puede reformularse el modelo sustituyendo la ecuación $v = 0$ por $\frac{dv}{dt} = 0$. Es decir, en lugar de imponer que la velocidad relativa sea cero mientras el modelo está en la fase *Stuck*, se impone que la aceleración relativa sea cero en dicha fase. Además, cuando el modelo pasa a la fase *Stuck*, se reinicializa la velocidad a cero.

Tabla 4.8: Fases del modelo de la fricción seca.

Fase	Condición de la fase
<i>Forward</i>	$v > 0$ and $f_r = R_v \cdot v + R_m$
<i>StartForward</i>	$v = 0$ and $a > 0$ and $f_r = R_m$
<i>Stuck</i>	$v = 0$ and $a = 0$ and $-R_0 \leq f_r \leq R_0$
<i>StartBackward</i>	$v = 0$ and $a < 0$ and $f_r = -R_m$
<i>Backward</i>	$v < 0$ and $f_r = R_v \cdot v - R_m$

**Figura 4.39:** Transición entre fases del modelo de la fricción seca.

Con ello se evita la deriva numérica de la velocidad relativa respecto al valor cero, que podría ser relevante si el modelo permanece suficiente tiempo en la fase *Stuck*.

El siguiente paso es definir cómo se produce la transición entre las fases del modelo. Supondremos que el modelo puede encontrarse en las cinco fases siguientes:

- *Stuck*. En esta fase la velocidad relativa entre los cuerpos es cero y la fuerza de rozamiento satisface $-R_0 \leq f_r \leq R_0$.
- *Forward* y *Backward*. En estas fases los cuerpos deslizan entre sí, siendo por tanto la velocidad relativa diferente de cero. En la fase *Forward*, la velocidad relativa satisface $v > 0$ y la fuerza de fricción es $f_r = R_v \cdot v + R_m$. En la fase *Backward*, $v < 0$ y $f_r = R_v \cdot v - R_m$.
- *StartForward* y *StartBackward*. Son las fases intermedias entre el reposo relativo y el deslizamiento. En estas fases la velocidad es cero, pero la aceleración es diferente de cero. La fuerza de fricción en la fase *StartForward* vale $f_r = R_m$ y en la fase *StartBackward* vale $f_r = -R_m$.

En la Tabla 4.8 se resumen las condiciones de cada fase. Las transiciones entre las fases pueden describirse mediante la máquina de estado finito que se muestra en la Figura 4.39. El diagrama consta de seis fases: las cinco descritas anteriormente y la fase *Start*, en la que se encuentra el modelo inicialmente.

Es posible describir una máquina de estado finito en Modelica declarando una variable booleana por cada fase y describiendo la condición de permanencia en cada fase de la forma siguiente:

$$\begin{aligned}
 fase = & \text{pre}(fasePre1) \text{ and } condicionIn1 & \text{or} \\
 & \text{pre}(fasePre2) \text{ and } condicionIn2 & \text{or} \\
 & \dots & \text{or} \\
 & \text{pre}(fase) \text{ and not} & (condicionOut1 \text{ or} \\
 & & condicionOut2 \dots)
 \end{aligned} \tag{4.133}$$

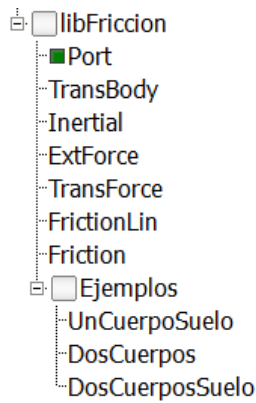
donde *fasePre1*, *fasePre2*, ... son las fases desde las cuales puede haber una transición a *fase* si se cumplen las condiciones *condicionIn1*, *condicionIn2*, ..., respectivamente. Las expresiones booleanas *condicionOut1*, *condicionOut2*, ... son las condiciones de las transiciones de salida de *fase*.

La descripción en Modelica de la máquina de estados de la Figura 4.39 es la siguiente.

```

Forward = pre(Start)      and v>0 or
          pre(StartForward) and v>0 or
          pre(Forward)    and not v<=0;
Backward = pre(Start)      and v<0 or
          pre(StartBackward) and v<0 or
          pre(Backward)    and not v>=0;
StartForward = pre(Stuck)    and fc > R0 or
               pre(StartForward) and not (v>0 or a<=0 and not v>0);
StartBackward = pre(Stuck)    and fc < -R0 or
               pre(StartBackward) and not (v<0 or a>=0 and not v<0);
Start = if initial() then true else false;
Stuck = not ( Forward      or Backward or StartForward or
             StartBackward or Start );
    
```

Para ilustrar la aplicación del modelo de la fricción, se ha programado una librería en Modelica llamada *libFriccion*, cuya arquitectura se muestra en la Figura 4.40. El código de la librería es Código 4.9 – 4.11.

Figura 4.40: Arquitectura de la librería *libFriccion*.

```

package libFriccion

import SI = Modelica.SIunits;

connector Port
  SI.Position p;
  flow SI.Force f;
end Port;

model TransBody
  Port port;
  parameter SI.Mass m = 1;
  parameter SI.Position p_Inicial = 0;
  parameter SI.Velocity v_Inicial = 0;
  SI.Position p( start=p_Inicial, fixed=true);
  SI.Velocity v( start=v_Inicial, fixed=true);
  SI.Acceleration a;
equation
  port.f = m*a;
  port.p = p;
  der(p) = v;
  der(v) = a;
end TransBody;

model Inertial
  Port port;
equation
  port.p = 0;
end Inertial;

model ExtForce
  Port port;
  SI.Force f;
  parameter Real Tstart = 50;
  parameter Real Tend = 75;
  parameter Real Kf = 1;
equation
  f = if time > Tstart and time < Tend
    then Kf*(time-Tstart)
    else 0;
  port.f = -f;
end ExtForce;

```

Código 4.9: Librería *libFriccion* (1/3).

```

model TransForce
  Port port1;
  Port port2;
  SI.Position p;
  SI.Velocity v(stateSelect = StateSelect.always);
  SI.Acceleration a;
equation
  p = port2.p - port1.p;
  der(p) = v;
  der(v) = a;
end TransForce;

model FrictionLin
  extends TransForce;
  SI.Force fr "Fuerza de rozamiento";
  parameter Real Kf=1;
equation
  port2.f = -fr;
  port1.f = fr;
  fr = -Kf*v;
end FrictionLin;

model Friction
  extends TransForce;
  SI.Force fr "Fuerza de rozamiento";
  SI.Force fc "Variable dummy";
  parameter Real R0 = 1 "Valor umbral";
  parameter Real Rm = 0.5;
  parameter Real Rv = 1;
  Boolean Start(start=true, fixed=true);
  Boolean Stuck(start=false, fixed=true);
  Boolean StartBackward(start=false, fixed=true);
  Boolean StartForward(start=false, fixed=true);
  Boolean Backward(start=false, fixed=true);
  Boolean Forward(start=false, fixed=true);
  constant SI.Mass unitMass = 1;
equation
  port2.f = -fr;
  port1.f = fr;
  fr = if Forward then -(Rv*v+Rm) else
        if Backward then -(Rv*v-Rm) else
        if StartForward then -Rm else
        if StartBackward then Rm else fc;
  0 = if Stuck then unitMass*a else fc;

  Stuck = not (Forward or Backward or StartForward or
               StartBackward or Start);
  Forward = pre(Start) and v>0 or
            pre(StartForward) and v>0 or
            pre(Forward) and not v<=0;
  Backward = pre(Start) and v<0 or
             pre(StartBackward) and v<0 or
             pre(Backward) and not v>=0;
  StartForward = pre(Stuck) and fc < -R0 or
                 pre(StartForward) and not (v>0 or a<=0 and not v>0);
  StartBackward = pre(Stuck) and fc > R0 or
                  pre(StartBackward) and not (v<0 or a>=0 and not v<0);
  Start = if initial() then true else false;
  // Este when sólo tiene efecto si v es variable de estado, lo cual
  // se consigue mediante el atributo stateSelect en TransForce
  when Stuck then
    reinit(v,0);
  end when;
end Friction;

```

Código 4.10: Librería *libFriccion* (2/3).

```

package Ejemplos

model UnCuerpoSuelo
  TransBody cuerpo(p_Inicial=0, v_Inicial=0, m=10);
  Friction friccion;
  Inertial suelo;
  ExtForce extForce1(Tstart=10, Tend=50, Kf=0.1);
  ExtForce extForce2(Tstart=100, Tend=150, Kf=-0.1);
equation
  connect( cuerpo.port, friccion.port2);
  connect( friccion.port1, suelo.port);
  connect(extForce1.port, cuerpo.port);
  connect(extForce2.port, cuerpo.port);
  annotation (experiment(StopTime=250));
end UnCuerpoSuelo;

model DosCuerpos
  TransBody cuerpo1(p_Inicial=0, v_Inicial=2, m=10);
  TransBody cuerpo2(p_Inicial=0, v_Inicial=0, m=10);
  Friction friccion;
  ExtForce extForce_c2(Tstart=100, Tend=150, Kf=-0.1);
equation
  connect( cuerpo1.port, friccion.port2);
  connect( friccion.port1, cuerpo2.port);
  connect( extForce_c2.port, cuerpo2.port);
  annotation (experiment(StopTime=250));
end DosCuerpos;

model DosCuerposSuelo
  // cuerpo2 - cuerpo1 - suelo
  TransBody cuerpo1(p_Inicial=0, v_Inicial=2, m=10);
  TransBody cuerpo2(p_Inicial=0, v_Inicial=-2, m=10);
  Friction friccion_c2_c1;
  Friction friccion_c1_suelo;
  Inertial suelo;
  ExtForce extForce_c1(Tstart=50, Tend=100, Kf=0.1);
  ExtForce extForce_c2(Tstart=200, Tend=250, Kf=-0.1);
equation
  connect( cuerpo2.port, friccion_c2_c1.port2);
  connect( friccion_c2_c1.port1, cuerpo1.port);
  connect( cuerpo1.port, friccion_c1_suelo.port2);
  connect( friccion_c1_suelo.port1, suelo.port);
  // Fuerzas externas
  connect( extForce_c1.port, cuerpo1.port);
  connect( extForce_c2.port, cuerpo2.port);
  annotation (experiment(StopTime=350));
end DosCuerposSuelo;

end Ejemplos;

end libFriccion;

```

El connector *Port* describe el puerto mecánico, que en esta ocasión está compuesto de dos variables: la posición y la fuerza. Equivalentemente podría haberse escogido como variable across la velocidad, en lugar de la posición.

El modelo *Transbody* describe un objeto que puede moverse en una dimensión. El objeto tiene un único conector. La fuerza del conector es la fuerza aplicada sobre el objeto, la cual hace que el objeto adquiera una aceleración. La posición del conector representa la posición en la que se encuentra el objeto.

El modelo *Inertial* describe un objeto fijo en el origen de coordenadas. Este modelo se empleará para describir el suelo.

El modelo *ExtForce* describe una fuerza externa que puede aplicarse sobre los objetos. El modelo tiene tres parámetros: T_{start} , T_{end} y K_f . La fuerza vale cero excepto en el intervalo de tiempo $(T_{start}, T_{end}]$, durante el cual crece linealmente con pendiente K_f N/s, partiendo en el instante T_{start} del valor cero.

El modelo *TransForce* posee dos puertos mecánicos, y define la posición y velocidades relativas de los dos puertos. Este modelo se empleará como superclase de los modelos de la fricción.

La librería contiene dos modelos diferentes de la fricción. En el modelo *Friction-Lin*, la fuerza de fricción depende linealmente de la velocidad relativa para cualquier valor de ésta. El modelo *Friction* describe el modelo de la fricción seca con seis fases de la Figura 4.39, cuya curva característica se muestra en la Figura 4.38.

El package *Ejemplos* contiene tres modelos. El modelo *UnCuerpoSuelo* describe el deslizamiento con fricción seca de un cuerpo sobre el suelo. Sobre el cuerpo se aplica una fuerza externa en dos intervalos diferentes de tiempo. El modelo *DosCuerpos* describe el deslizamiento con fricción seca de dos cuerpos. En el modelo *DosCuerposSuelo* se incluye además el modelo del suelo, con lo cual se produce deslizamiento con fricción de un cuerpo sobre otro, y de éste sobre el suelo. Se invita al lector a que simule por sí mismo cada uno de estos tres modelos e interprete los resultados obtenidos.

4.5.8. Tratamiento literal de las expresiones if

Como se ha explicado en las secciones anteriores, el entorno de modelado realiza un tratamiento basado en eventos de las expresiones y cláusulas if. Es decir, cuando se detecta el evento:

1. Se suspende la solución del problema de tiempo continuo.
2. Se itera para calcular el instante de disparo del evento.
3. Se resuelve el problema de reinicio, usando la rama if correspondiente.

Sin embargo, en ocasiones la conmutación de rama no introduce discontinuidad ni un punto no diferenciable, o bien el efecto de estos es pequeño. En estos casos, se puede integrar a través de estos puntos, lo cual a veces permite reducir el tiempo de simulación, ya que evita localizar de manera precisa el instante del evento y la reinicialización del modelo.

Modelica proporciona la función **noEvent**, que permite al desarrollador del modelo indicar que desea que se realice un tratamiento literal, no basado en eventos, de una condición booleana.

Por ejemplo, si se escribe

$$expr_1 = \text{if noEvent}(cond) \text{ then } expr_2 \text{ else } expr_3; \quad (4.134)$$

dentro de una sección *equation*, o bien

$$var := \text{if noEvent}(cond) \text{ then } expr_1 \text{ else } expr_2; \quad (4.135)$$

dentro de una sección *algorithm*, entonces el entorno de modelado evalúa la condición booleana en primer lugar, para decidir cuál de las dos ramas de la sentencia debe usar, y a continuación calcula la correspondiente variable empleando dicha rama.

Otra aplicación de la función **noEvent** es el caso en el cual las ramas no pueden extenderse más allá de su dominio de validez (véase la Sección 4.3.3). En este caso, debe realizarse un tratamiento literal de la expresión. Por ejemplo, la sentencia *if*

$$y = \text{if } u \geq 0 \text{ then } u^{0.5} \text{ else } (-u)^{0.5}; \quad (4.136)$$

produciría error de ejecución al intentar calcular la raíz cuadrada de un número negativo. Se evitaría este error empleando **noEvent**:

$$y = \text{if noEvent}(u \geq 0) \text{ then } u^{0.5} \text{ else } (-u)^{0.5}; \quad (4.137)$$

4.5.9. Ejemplo 6: conducción de calor en una pared

En este ejemplo se va a analizar mediante simulación la conducción de calor en la pared de una cámara de refrigeración que está compuesta de tres capas de diferente material, que llamaremos A, B y C, de espesores $L_A = 15$ mm, $L_B = 100$ mm y $L_C = 75$ mm. En la parte superior de la Figura 4.41 se muestra un corte transversal de la pared.

La superficie interior de la pared se encuentra a una temperatura constante de 0°C , y la temperatura exterior oscila entre -20°C y 20°C a lo largo de 24 horas. Expresando el tiempo t en segundos (24 horas equivalen a $24 \cdot 60 \cdot 60 = 86400$ segundos) y la temperatura en Kelvin, la temperatura de la superficie interior (T_1) y exterior (T_5) de pared de la cámara están descritas por las Ecs. (4.138) y (4.139).

$$T_1 = 273.15 \quad (4.138)$$

$$T_5 = 273.15 + 20 \cdot \sin\left(\frac{2 \cdot \pi \cdot t}{86400}\right) \quad (4.139)$$

Se desea calcular el flujo de calor por unidad de superficie (Q), así como la temperatura en las interfaces entre los diferentes materiales (T_2, T_3) y en la superficie exterior (T_4) de la pared. Para ello, se modela el flujo de calor mediante el circuito térmico equivalente mostrado en la parte inferior de la Figura 4.41. El parámetro S que interviene en el denominador de las resistencias térmicas es el área de la pared, al cual asignaremos el valor uno ($S = 1 \text{ m}^2$). De esta forma, Q es igual al flujo de calor por unidad de área de la pared.

Las conductividades térmicas de los materiales, κ_A , κ_B y κ_C , expresadas con unidades $\text{W} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$, están descritas por las Ecs. (4.140) – (4.142).

$$\kappa_A = 0.151 \quad (4.140)$$

$$\kappa_B = 2.5 \cdot \exp\left(\frac{-1225}{T}\right) \quad (4.141)$$

$$\kappa_C = 0.762 \quad (4.142)$$

La conductividad térmica del material B es función de la temperatura T expresada en Kelvin, como puede observarse en la Ec. (4.141). En primera aproximación,

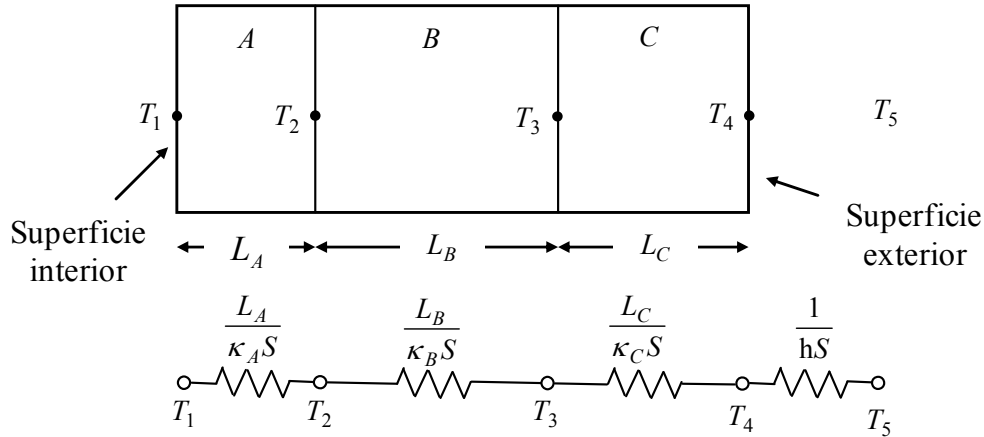


Figura 4.41: Estructura de la pared de la cámara y circuito térmico equivalente.

```

model paredMulticapa
  import SI = Modelica.SIunits;
  // Espesores de las capas
  parameter SI.Length La = 15E-3;
  parameter SI.Length Lb = 100E-3;
  parameter SI.Length Lc = 75E-3;
  // Área de la pared
  parameter SI.Area S = 1;
  // Conductividades térmicas de las capas
  parameter SI.ThermalConductivity Ka = 0.151;
  SI.ThermalConductivity Kb;
  parameter SI.ThermalConductivity Kc = 0.762;
  // Coeficiente de transferencia de calor por convección
  SI.CoefficientOfHeatTransfer h;
  // Temperaturas
  parameter SI.Temperature T1 = 273.15;
  SI.Temperature T2(start=273,fixed=false);
  SI.Temperature T3(start=273,fixed=false);
  SI.Temperature T4(start=273,fixed=false);
  SI.Temperature T5;
  // Flujo de calor
  SI.HeatFlowRate Q;
equation
  // Temperatura exterior
  T5=273.15+20*sin(time*2*Modelica.Constants.pi/86400);
  // Conductividad térmica del material B
  Kb= 2.5*exp(-1225*2/(T2+T3));
  // Coef. transferencia de calor por convección para la pared exterior
  h= if noEvent(T4>T5) then 1.37*((T4-T5)/6)^0.25 else
      if noEvent(T5>T4) then 1.37*((T5-T4)/6)^0.25 else
      1;
  // Relaciones constitutivas de las resistencias térmicas
  T1 - T2 = Q * La/(Ka*S);
  T2 - T3 = Q * Lb/(Kb*S);
  T3 - T4 = Q * Lc/(Kc*S);
  T4 - T5 = Q * 1/(h*S);
end paredMulticapa;
  
```

Código 4.12: Pared con tres capas de la Figura 4.41.

se considera que la temperatura en el material B es igual al promedio de las temperaturas en las interfaces A-B y B-C. Es decir,

$$\kappa_B = 2.5 \cdot \exp \left(\frac{-1225}{0.5 \cdot (T_2 + T_3)} \right) \quad (4.143)$$

El coeficiente de transferencia de calor por convección natural para la pared exterior, expresado en unidades $\text{W} \cdot \text{m}^{-2} \cdot \text{K}^{-1}$, vale:

$$h = 1.37 \cdot \left| \frac{T_5 - T_4}{6} \right|^{1/4} \quad (4.144)$$

Llamando Q al flujo de calor expresado en vatios que circula desde la superficie interior a la exterior de la pared, las relaciones constitutivas de las cuatro resistencias térmicas mostradas en la Figura 4.41 son las siguientes:

$$T_1 - T_2 = Q \cdot \frac{L_A}{\kappa_A \cdot S} \quad (4.145)$$

$$T_2 - T_3 = Q \cdot \frac{L_B}{\kappa_B \cdot S} \quad (4.146)$$

$$T_3 - T_4 = Q \cdot \frac{L_C}{\kappa_C \cdot S} \quad (4.147)$$

$$T_4 - T_5 = Q \cdot \frac{1}{h \cdot S} \quad (4.148)$$

Recapitulando, el modelo tiene 8 variables ($T_1, T_2, T_3, T_4, T_5, Q, \kappa_B, h$) y está compuesto por 8 ecuaciones. Éstas son las Ecs. (4.138), (4.139), (4.143), (4.144) y (4.145) – (4.148). El valor de los parámetros $L_A, L_B, L_C, S, \kappa_A$ y κ_C es conocido.

Observando la Ec. (4.148), se aprecia que el coeficiente de transferencia de calor por convección natural para la pared exterior (h) aparece en el denominador de la resistencia térmica. Cuando las temperaturas T_4 y T_5 se hagan iguales, el valor de h calculado de la Ec. (4.144) será cero. El valor $h = 0$ implica valor infinito de la resistencia térmica $1/(h \cdot S)$, lo cual producirá un error numérico.

Conviene por tanto redefinir el valor de h cuando T_4 y T_5 son iguales. Dado que en esa situación el flujo de calor vale $Q = 0$, es indiferente el valor que se asigne a h cuando $T_4 = T_5$, siempre y cuando éste sea diferente de cero. Definiremos h de la forma siguiente:

$$\begin{aligned}
h = & \text{ if noEvent}(T4 > T5) \text{ then } 1.37 \cdot ((T4 - T5)/6)^{0.25} \\
& \text{ else if noEvent}(T5 > T4) \text{ then } 1.37 \cdot ((T5 - T4)/6)^{0.25} \\
& \text{ else } 1;
\end{aligned} \tag{4.149}$$

Ejecutando la simulación del Código 4.12 durante 86400 s, se obtienen los resultados mostrados en la Figura 4.42. Puede observarse en la gráfica de la parte superior de la figura que el mayor escalón térmico se produce en el material B. El flujo de calor Q es positivo si tiene sentido saliente de la cámara de refrigeración, es decir, mientras $T_1 > T_5$.

Recuérdese que la conductividad térmica del material B es función de la temperatura y que se ha realizado la aproximación de considerar que la temperatura en el material B es igual al promedio de las temperaturas en las interfaces A-B y B-C. Para analizar el error asociado a esta hipótesis de modelado, puede dividirse el material B en N_{elemB} capas iguales, cada una de ellas de espesor L_B/N_{elemB} .

Definiremos un vector de temperaturas, cuyos componentes son

$$T_B[1], \dots, T_B[N_{elemB} + 1] \tag{4.150}$$

de manera que

$$T_B[1] = T_2 \tag{4.151}$$

$$T_B[N_{elemB} + 1] = T_3 \tag{4.152}$$

y donde

$$T_B[i] \quad \text{con } i = 2, \dots, N_{elemB} \tag{4.153}$$

representa la temperatura de la interfaz entre la capa $i - 1$ y la capa i en que se ha dividido el material B. La conductividad térmica en la capa i -ésima del material B vale:

$$\kappa_B[i] = 2.5 \cdot \exp \left(\frac{-1225}{0.5 \cdot (T_B[i] + T_B[i + 1])} \right) \tag{4.154}$$

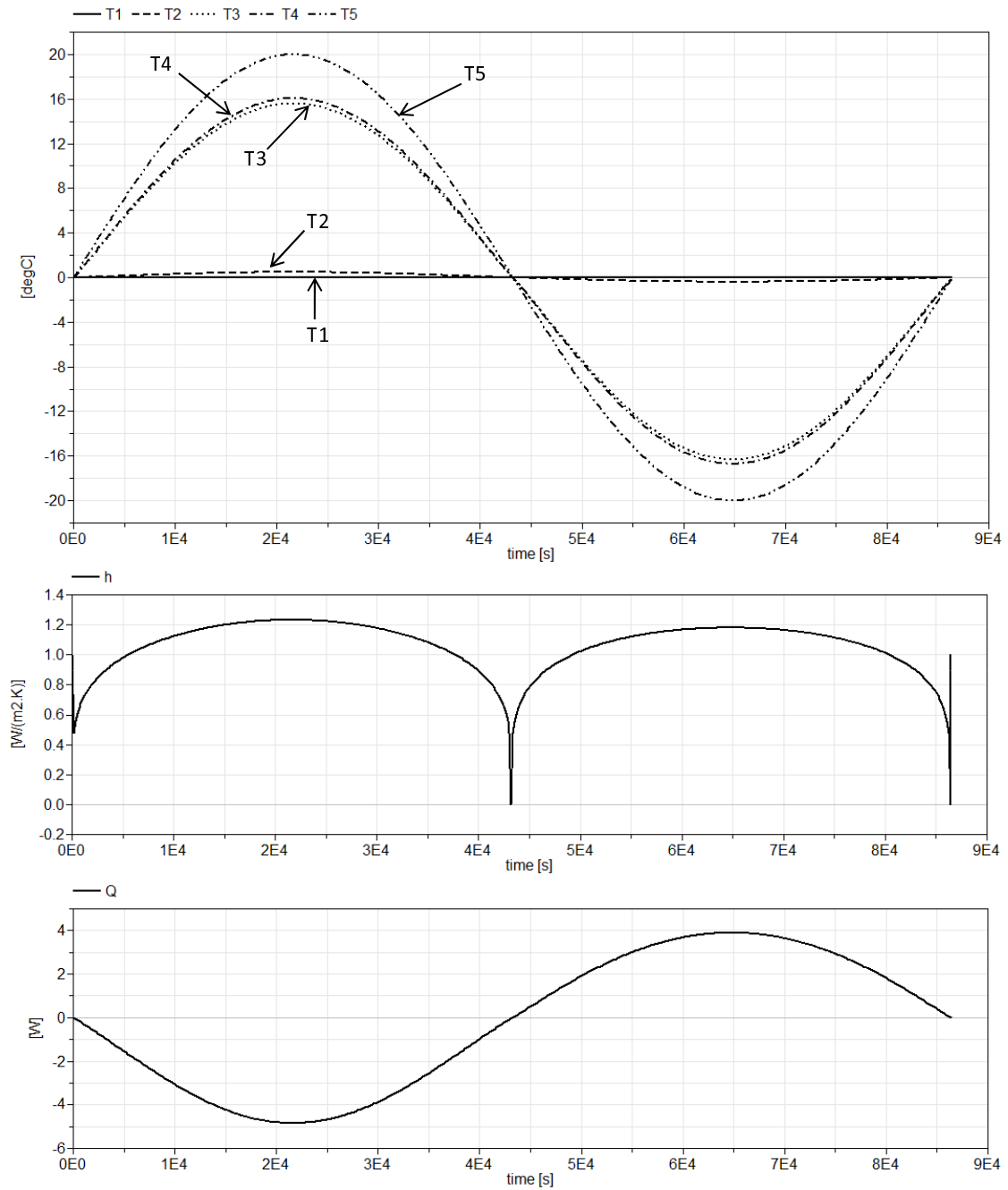


Figura 4.42: Resultado de la simulación del Código 4.12.

La relación constitutiva de la resistencia térmica que describe la capa i -ésima del material B es:

$$T_B[i] - T_B[i + 1] = Q \cdot \frac{L_B/N_{elemB}}{\kappa_B[i] \cdot S} \quad (4.155)$$

La descripción en Modelica del modelo se muestra en el Código 4.13. Se ha realizado la simulación dividiendo el material B en $N_{elemB} = 50$ capas y los resultados obtenidos se han comparado con los que se obtuvieron al simular el Código 4.12. El resultado de dicha comparación se muestra en la Figura 4.43, donde puede apreciarse el orden de magnitud del error en el flujo de calor, y en las temperaturas T_2 y T_3 .

```

model paredMulticapaN
  import SI = Modelica.SIunits;
  // Número de capas del material B
  constant Integer NelemB = 50;
  // Espesores de las capas
  parameter SI.Length La = 15E-3;
  parameter SI.Length Lb = 100E-3;
  parameter SI.Length Lc = 75E-3;
  // Área de la pared
  parameter SI.Area S = 1;
  // Conductividades térmicas de las capas
  parameter SI.ThermalConductivity Ka = 0.151;
  SI.ThermalConductivity Kb[NelemB];
  parameter SI.ThermalConductivity Kc = 0.762;
  // Coeficiente de transferencia de calor por convección
  SI.CoefficientOfHeatTransfer h;
  // Temperaturas
  parameter SI.Temperature T1 = 273.15;
  SI.Temperature T2(start=273,fixed=false);
  SI.Temperature T3(start=273,fixed=false);
  SI.Temperature T4(start=273,fixed=false);
  SI.Temperature T5;
  SI.Temperature TB[NelemB+1]( start=273*ones(NelemB+1),fixed=false);
  // Flujo de calor
  SI.HeatFlowRate Q;
equation
  // Temperatura exterior
  T5=273.15+20*sin(time*2*Modelica.Constants.pi/86400);
  // Coef. transferencia de calor por convección para la pared exterior
  h= if noEvent(T4>T5) then 1.37*((T4-T5)/6)^0.25 else
    if noEvent(T5>T4) then 1.37*((T5-T4)/6)^0.25 else
    1;
  // Relaciones constitutivas de las resistencias térmicas
  T1 - T2 = Q * La/(Ka*S);
  T3 - T4 = Q * Lc/(Kc*S);
  T4 - T5 = Q * 1 /(h*S);
  // Capa B
  TB[1] = T2;
  for i in 1:NelemB loop
    TB[i] - TB[i+1] = Q * (Lb/NelemB)/(Kb[i]*S);
    Kb[i] = 2.5*exp(-1225*2/(TB[i] + TB[i+1]));
  end for;
  TB[NelemB+1] = T3;
end paredMulticapaN;

```

Código 4.13: Flujo de calor en una pared (material B dividido en N_{elemB} capas).

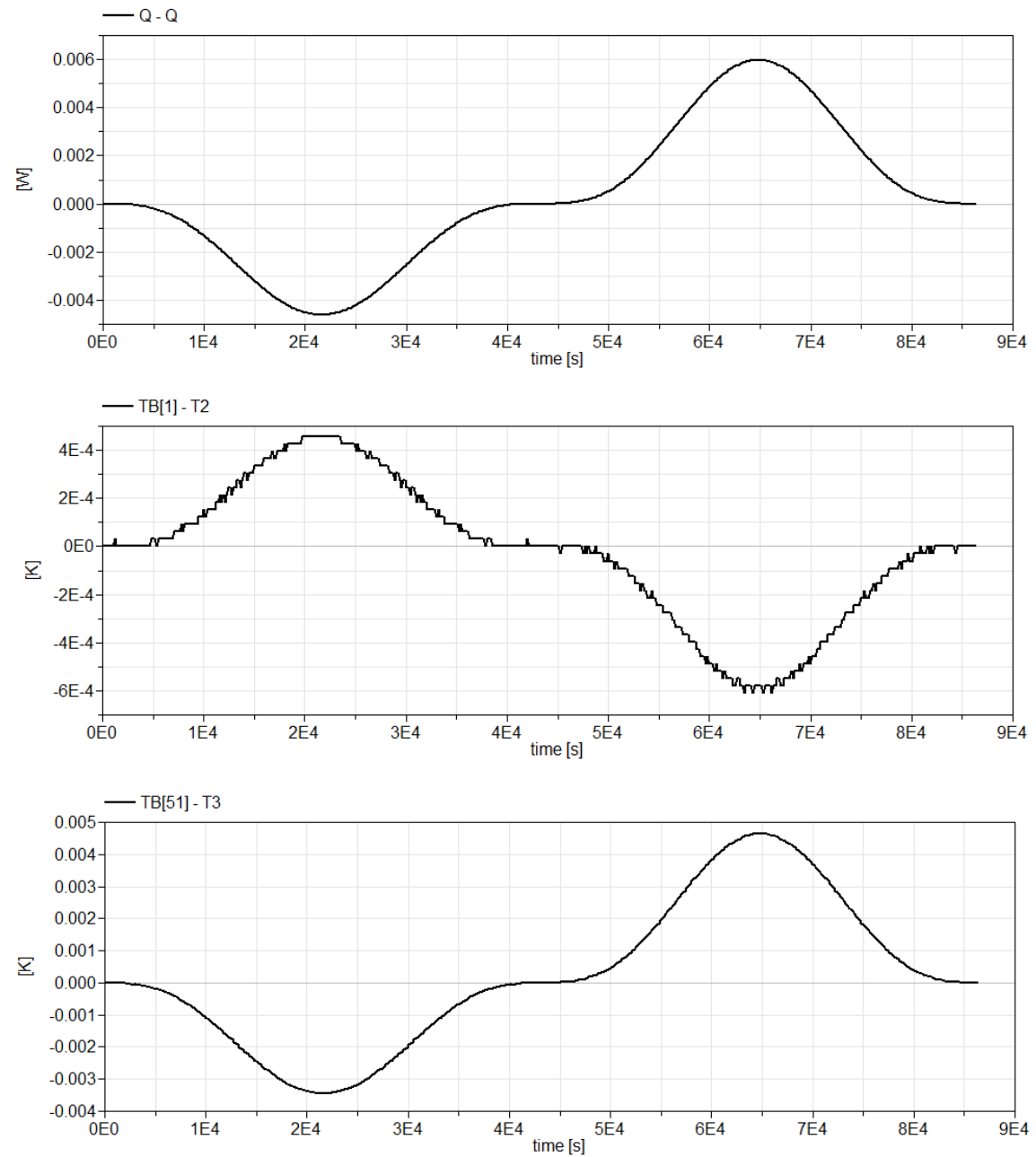


Figura 4.43: (Resultados con $N_{elemB} = 50$) – (Resultados con $N_{elemB} = 1$).

4.6. INICIALIZACIÓN DEL MODELO EN MODELICA

La inicialización del modelo, es decir, el cálculo del valor de las variables en el instante inicial de la simulación, es en ocasiones problemático. Si el modelo contiene lazos algebraicos no lineales, estos deben ser resueltos mediante métodos iterativos, partiendo de las condiciones iniciales para la iteración proporcionadas por el desarrollador del modelo. Si estas condiciones iniciales no están “lo suficientemente cerca” de la solución, el método iterativo puede diverger.

Una vez resuelto el modelo en el instante inicial, la solución de los lazos durante la integración del problema continuo es mucho menos problemática. En cada instante se emplea como valor inicial para la iteración de los lazos algebraicos el valor calculado de las variables en el anterior instante de evaluación. Si el método no converge, puede reducirse el tamaño del paso de integración, de modo que la solución que se busca se acerque a los valores iniciales que se están empleando para iterar el lazo.

Obsérvese que cuando los eventos producen cambios instantáneos en el estado del modelo, se puede producir también una situación problemática, análoga a la del problema de inicialización, puesto que la discontinuidad en los estados puede producir que la solución de los lazos algebraicos se encuentre “demasiado alejada” de los valores iniciales que están siendo usados para la iteración. Estos valores iniciales son el valor que tenían las variables en el instante del evento, pero previamente a la ejecución del mismo.

Dado que la inicialización de modelos complejos puede ser ciertamente problemática, el lenguaje Modelica proporciona recursos para simplificar este problema. En concreto, separa completamente el problema de la inicialización de la simulación dinámica del modelo. Vamos esto con detalle.

4.6.1. Planteamiento de la inicialización

El objetivo de la inicialización del modelo es calcular valores consistentes para todas las variables del modelo. Con este fin, deben proporcionarse tantas ecuaciones, que describan el comportamiento del modelo en el instante inicial, como variables tenga el modelo. El vector de incógnitas, es decir, de variables a evaluar, es

$$\{\text{der}(\mathbf{x}), \mathbf{x}, \mathbf{y}, \mathbf{q}, \text{pre}(\mathbf{q}), \mathbf{p}, \mathbf{c}\} \quad (4.156)$$

donde los términos del vector representan respectivamente las derivadas, las variables que aparecen derivadas, las variables de tiempo continuo que no aparecen derivadas, los parámetros, las variables de tiempo discreto, el valor “pre” de las variables de tiempo discreto, y las condiciones booleanas de las cláusulas if y when.

Para evaluar estas incógnitas se dispone de todos los algoritmos y ecuaciones que describen el comportamiento del modelo y que son usados en la simulación, y además se dispone de cierto número de ligaduras adicionales, denominadas condiciones iniciales. El número de estas condiciones iniciales debe ser igual al número de estados continuos ($\leq \dim(\mathbf{x})$), más el número de parámetros ($= \dim(\mathbf{p})$), más el número de variables de tiempo discreto ($= \dim(\mathbf{q})$).

4.6.2. Variables de tiempo continuo

El valor inicial de las variables de tiempo continuo puede especificarse mediante los atributos **start** y **fixed**. Por ejemplo, si se escribe

```
Real x (start = x0, fixed = true);
```

estos atributos **start** y **fixed** se traducen en la ligadura

```
x = x0
```

que estará activa durante la inicialización.

Si no se asigna valor al atributo **fixed**, los valores por defecto son:

```
fixed = true   para constantes y parámetros
fixed = false  para las demás variables
```

Otro procedimiento para definir ligaduras activas sólo en la inicialización consiste en incluir estas ligaduras dentro de una sección **initial equation**. Por ejemplo, consideremos el modelo mostrado en el Código 4.14.

El valor de los parámetros **estacionario** y **fixed** condiciona el planteamiento de la inicialización. Debe asignarse valor a estos parámetros antes de realizar la traducción del modelo y el entorno de modelado no permitirá modificar los valores al definir el experimento.

Si se asigna al parámetro booleano **estacionario** el valor *true*, la ligadura activa en la inicialización es:

```
der(x) = 0
```

```

model ejem1
  parameter Real x0=1;
  parameter Real a=2;
  parameter Real b=3;
  parameter Boolean estacionario=true;
  parameter Boolean fixed=false;
  Real x(start=x0, fixed=fixed and not estacionario);
equation
  der(x) = a*x + b;
initial equation
  if estacionario then
    der(x) = 0;
  end if;
end ejem1;

```

Código 4.14: Ejemplo de inicialización de un modelo de tiempo continuo.

Por el contrario, si `estacionario` vale *false* y el parámetro `fixed` vale *true*, entonces la ligadura activa sería:

$$x = x0$$

Otras ligaduras son los valores asignados a los parámetros `a` y `b`, es decir, `a = 2`, `b = 3`.

Además, para resolver el problema en el instante inicial se consideran, las ecuaciones del modelo:

$$\text{der}(x) = a \cdot x + b$$

Así pues, en el primer caso el problema de inicialización es:

```

estacionario = true, fixed = false
a = 2, b = 3
der(x) = a*x + b
der(x) = 0

```

y en el segundo caso es:

```

estacionario = false, fixed = true
a = 2, b = 3
der(x) = a*x + b
x = x0

```

Obsérvese que en ambos casos el problema está bien definido, ya que se plantean seis ecuaciones para calcular seis incógnitas.

4.6.3. Ejemplo 1: péndulo plano

En la Figura 4.44 se muestra un péndulo de longitud L constante. Sea $m = 1$ kg la masa del péndulo y $g = 9.81 \text{ m}\cdot\text{s}^{-2}$ la aceleración gravitatoria.

El ángulo respecto a la vertical (φ) y la velocidad angular (w) del péndulo se relacionan con la fuerza ejercida por la gravedad sobre la masa del péndulo mediante las ecuaciones siguientes:

$$\dot{\varphi} = w \quad (4.157)$$

$$J \cdot \dot{w} = -m \cdot g \cdot L \cdot \sin(\varphi) \quad (4.158)$$

Las coordenadas rectangulares de la masa (x , y) se relacionan mediante las ecuaciones siguientes con el ángulo (φ) y la longitud del péndulo (L):

$$x = L \cdot \sin(\varphi) \quad (4.159)$$

$$y = L \cdot \cos(\varphi) \quad (4.160)$$

En el Código 4.15 se muestran tres formas alternativas de inicializar este modelo del péndulo plano.

1. En el modelo de la parte superior, se asigna valor a los parámetros y a las variables que aparecen derivadas: el ángulo, $\varphi(0) = 1 \text{ rad}$, y la velocidad angular, $w(0) = 0 \text{ rad}\cdot\text{s}^{-1}$.
2. En el modelo mostrado en la parte central, se asigna valor inicial a la altura de la masa $y(0) = 0.9 \text{ m}$ y a la velocidad angular $w(0) = 0 \text{ rad}\cdot\text{s}^{-1}$. Como hay dos valores del ángulo que corresponden a un determinado valor de la altura, se proporciona un valor inicial para el cálculo del ángulo, $\varphi \simeq 0.1 \text{ rad}$.
3. En el modelo mostrado en la parte inferior, se inicializa el modelo asignando valor a la posición inicial de la masa, $x(0) = 0.5 \text{ m}$, $y(0) = 0.9 \text{ m}$, y su velocidad angular inicial, $w(0) = 0 \text{ rad}\cdot\text{s}^{-1}$. En este caso no se especifica ni la longitud del péndulo (L) ni el ángulo φ , para los cuales sólo se proporcionan valores aproximados: $L \simeq 1 \text{ m}$, $\varphi \simeq 0.1 \text{ rad}$.

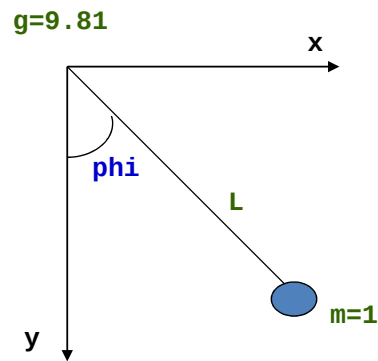


Figura 4.44: Péndulo plano.

```

model pendulo
    parameter Real m=1, g=9.81;
    parameter Real L=1;
    parameter Real J=m*L^2;
    Real phi (start=1, fixed=true);
    Real w (start=0, fixed=true);
    Real x;
    Real y;
equation
    der(phi) = w;
    J*der(w) = -m*g*L*sin(phi);
    x = sin(phi)*L;
    y = cos(phi)*L;
end pendulo;

model pendulo
    parameter Real m=1, g=9.81;
    parameter Real L=1;
    parameter Real J=m*L^2;
    Real phi (start=0.1, fixed=false);
    Real w (start=0, fixed=true);
    Real x;
    Real y (start=0.9, fixed=true);
equation
    der(phi) = w;
    J*der(w) = -m*g*L*sin(phi);
    x = sin(phi)*L;
    y = cos(phi)*L;
end pendulo;

model pendulo
    parameter Real m=1, g=9.81;
    parameter Real L (fixed=false) = 1;
    parameter Real J=m*L^2;
    Real phi (start=0.1, fixed=false);
    Real w (start=0, fixed=true);
    Real x (start=0.5, fixed=true);
    Real y (start=0.9, fixed=true);
equation
    der(phi) = w;
    J*der(w) = -m*g*L*sin(phi);
    x = sin(phi)*L;
    y = cos(phi)*L;
end pendulo;
    
```

Código 4.15: Tres formas alternativas de inicializar el modelo del péndulo.

4.6.4. Variables de tiempo discreto

Modelica proporciona dos mecanismos para asignar valor a los valores pre de las variables de tiempo discreto:

1. Mediante los atributos **start** y **fixed**. Por ejemplo, si se escribe

```
Boolean b (start = false, fixed = true);
Integer i (start = 1      , fixed = true);
```

las correspondientes ecuaciones de inicialización son:

```
pre(b) = false;
pre(i) = 1;
```

2. También puede asignarse valor a los valores pre de las variables de tiempo discreto incluyendo las correspondientes ecuaciones en una sección **initial equation**. Por ejemplo,

```
initial equation
  pre(xd) = 0;
  pre(u)  = 0;
```

Las ligaduras sobre el valor de las variables de tiempo discreto deben incluirse dentro de cláusulas **when** cuya condición de disparo sea la función **initial()**. Por ejemplo:

```
equation
  when { initial(), condición1, ... } then
    v = ...
  end when;
```

Es importante tener en cuenta que la cláusula **when** sólo se activa durante la inicialización si la condición es **initial()**. No es posible definir ligaduras para el problema de inicialización empleando otras condiciones, como por ejemplo **not time < 0**, o por ejemplo **time >= 0**.

Si durante la inicialización la cláusula **when** en la que se evalúa una variable no está activa, entonces la variable es igual a su valor pre. Por ejemplo, si la cláusula **when** de la cual se evalúa la variable **v** no contiene la condición **initial()**, entonces en la inicialización **v = pre(v)**.

4.6.5. Ejemplo 2: lazo de control

Consideremos el lazo de control mostrado en la Figura 4.45. El símbolo circular representa la operación de resta de la variable que representa el valor de consigna (x_{ref}) y la salida de la planta (x). El resultado de la resta, $x_{ref} - x$, denominado *señal de error* puesto que es la diferencia entre el valor que se desea que tenga la salida de la planta y el valor actual de la salida de la planta, es la entrada al controlador PI. La salida del controlador, u , es la entrada a la planta.

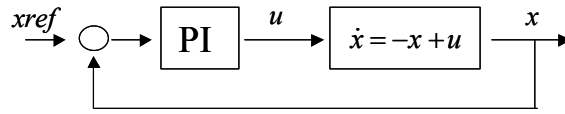


Figura 4.45: Lazo de control.

El controlador PI es discreto, con periodo de muestreo T_S . Está descrito mediante las ecuaciones en diferencias:

$$x_d = \text{pre}(x_d) + \frac{T_S}{T} \cdot (x_{ref} - x) \quad (4.161)$$

$$u = k \cdot (x_{ref} - x) + x_d \quad (4.162)$$

donde $x_{ref} - x$ es el error, k y T los parámetros proporcional e integral del controlador, x_d representa el termino integral y $k \cdot (x_{ref} - x)$ el término proporcional del controlador.

El modelo de la planta es el mostrado en la figura:

$$\dot{x} = -x + u \quad (4.163)$$

Se describen a continuación cuatro formas alternativas de inicializar este modelo. En cada caso se muestra el fragmento de código Modelica y el problema de inicialización asociado.

Caso 1

Supongamos primeramente que el modelo se define de la forma indicada en el siguiente fragmento de código:

```

parameter Real k=10, T=1; // Parámetros PI
parameter Real Ts=0.01 "Tiempo de muestreo";
input      Real xref      "Referencia";
           Real x  (fixed=true, start=2);
discrete   Real xd (fixed=true, start=0);
discrete   Real u   (fixed=true, start=0);
equation
// Modelo de la planta
der(x) = -x + u;
// Controlador PI discreto
when sample(0,Ts) then
  xd = pre(xd) + Ts/T*(xref-x);
  u  = k*(xref-x)+xd;
end when;

```

El problema de inicialización tiene las incógnitas siguientes:

```

x, der(x)
xd, u, pre(xd), pre(u)

```

La cláusula when no está activa durante la inicialización. Por ello, se ha sustituido por:

```

xd := pre(xd)
u  := pre(u)

```

Las ecuaciones ordenadas y resueltas del problema de la inicialización son:

```

x      := x.start // =2
pre(xd) := xd.start // =0
pre(u)  := u.start // =0
xd      := pre(xd) // =0
u       := pre(u)  // =0
der(x)  := -x+u    // =-2

```

Caso 2

Consideremos ahora la forma siguiente de inicializar el modelo:

```

parameter Real k=10, T=1; // Parámetros PI
parameter Real Ts=0.01 "Tiempo de muestreo";
input      Real xref      "Referencia";
           Real x  (fixed=true, start=2);
discrete   Real xd (fixed=true, start=0);
discrete   Real u   (fixed=true, start=0);
equation
// Modelo de la planta
der(x) = -x + u;
// Controlador PI discreto
when {initial(),sample(0,Ts)} then
    xd = pre(xd) + Ts/T*(xref-x);
    u  = k*(xref-x)+xd;
end when;
```

Puesto que `initial()` es una de las condiciones de disparo de la cláusula `when`, ésta está activa en la inicialización. La cláusula se ha sustituido por:

```

xd := pre(xd) + Ts/T*(xref-x)
u  := k*(xref-x) + xd
```

Las ecuaciones ordenadas y resueltas del problema de la inicialización son:

```

x      := x.start    // =2
pre(xd) := xd.start   // =0
pre(u)  := u.start    // =0
xd      := pre(xd)+Ts/T*(xref-x)
u       := k*(xref-x)+xd
der(x)  := -x+u       // =-2
```

Caso 3

Supongamos ahora que el modelo se inicializa de la manera siguiente

```

parameter Real k=10, T=1; // Parámetros PI
parameter Real Ts=0.01 "Tiempo de muestreo";
input      Real xref      "Referencia";
           Real x  (fixed=true, start=2);
discrete   Real xd;
discrete   Real u;
equation
  // Modelo de la planta
  der(x) = -x + u;
  // Controlador PI discreto
  when {initial(), sample(0, Ts)} then
    xd = pre(xd) + Ts/T*(xref-x);
    u  = k*(xref-x)+xd;
  end when;
initial equation
  pre(xd) = 0;
  pre(u)  = 0;

```

En este caso, la cláusula when está activa en la inicialización y además se especifican los valores pre de las dos variables de tiempo discreto en la sección initial equation.

Las ecuaciones ordenadas y resueltas del problema de la inicialización son:

```

x      := x.start    // =2
pre(xd) := 0
pre(u)  := 0
xd      := pre(xd)+Ts/T*(xref-x)
u       := k*(xref-x)+xd
der(x)  := -x+u      // =-2

```

Caso 4

Finalmente, supongamos que el modelo se inicializa imponiendo que se encuentre en el estacionario:

```

parameter Real k=10, T=1; // Parámetros PI
parameter Real Ts=0.01 "Tiempo de muestreo";
input      Real xref      "Referencia";
           Real x (fixed=false, start=2);
discrete   Real xd;
discrete   Real u;
equation
// Modelo de la planta
der(x) = -x + u;
// Controlador PI discreto
when {initial(),sample(0,Ts)} then
    xd = pre(xd) + Ts/T*(xref-x);
    u  = k*(xref-x)+xd;
end when;
initial equation
pre(xd) = xd;
pre(u)  = u;
der(x)  = 0;

```

Las ecuaciones ordenadas del problema de la inicialización son:

```

der(x)  = 0
// Sistema lineal de 4 ecuaciones
// Incóg: xd, pre(xd), u, x
| pre(xd) = xd
| xd      = pre(xd)+Ts/T*(xref-x)
| u       = k*(xref-x)+xd
| der(x)  = -x+u
pre(u)   = u

```

Las ecuaciones ordenadas y resueltas son:

```

der(x)  := 0
xd      := xref
u       := xref
x       := xref
pre(xd) := xd
pre(u)  := u

```

4.6.6. Modelo para la inicialización

Hasta ahora hemos visto cómo Modelica permite especificar las condiciones iniciales. Estas condiciones iniciales, junto con las ecuaciones del modelo, constituyen el problema a resolver en el instante inicial.

Ahora veremos cómo las ecuaciones del modelo pueden ser sustituidas por otras ecuaciones, que sólo se emplean para resolver la inicialización. Para ello, pueden emplearse **sentencias if** cuya condición sea **initial()**. Es decir:

```
equation
  y = if initial() then
        // formulación adecuada para resolver la inicialización
      else
        // formulación adecuada para la simulación
```

Esta capacidad permite definir un modelo sencillo para la inicialización, por ejemplo, un modelo linealizado entorno al punto de operación inicial, con el fin de facilitar la convergencia numérica en el instante inicial.

4.7. EXPERIMENTACIÓN CON MODELOS EN MODELICA

Algunos entornos de modelado de Modelica, como es el caso de Dymola y OpenModelica, permiten definir el experimento desde la interfaz gráfica de usuario. En la definición del experimento, es posible especificar:

- El nombre de los ficheros en los cuales se encuentran los modelos.
- El nombre del modelo que va a simularse.
- El atributo start de aquellas variables cuyo atributo fixed vale *true*, asignando nuevos valores o modificando los valores dados en el modelo, de manera que se complete la definición del problema de inicialización.
- El intervalo de simulación, es decir, el valor de los instantes inicial y final.
- El número de instantes de comunicación o la longitud del intervalo de tiempo entre dos instantes consecutivos de comunicación.

Los instantes de comunicación son los instantes en los cuales el simulador salva en disco el valor calculado de las variables. Además de en los instantes

de comunicación, el valor de las variables se almacena también en aquellos instantes en los cuales se dispara algún evento.

El intervalo de comunicación es conceptualmente diferente del paso de integración, si bien éste debe ser menor o igual que aquel.

- El algoritmo de integración.
- Que se ejecute la simulación.

En muchos casos, lo más cómodo es definir el experimento empleando la interfaz de usuario del entorno de modelado. Sin duda, éste es el caso cuando se está desarrollando el modelo y se realizan las primeras pruebas.

Sin embargo, en la fase de estimación (ajuste de los parámetros del modelo a datos experimentales) y de explotación del modelo, frecuentemente es más conveniente describir el experimento en un fichero, de modo que se vayan ejecutando automáticamente las acciones experimentales, sin necesidad de intervención del usuario.

Este es el caso, por ejemplo, cuando se emplean técnicas basadas en algoritmos genéticos para estimar los parámetros del modelo a partir de datos experimentales. Este tipo de experimentos requiere de la realización de un elevado número de réplicas de la simulación, en cada una de las cuales se emplean unos valores de los parámetros del modelo que han sido calculados analizando la información obtenida en las anteriores réplicas, por ejemplo, analizando la desviación de los resultados simulados respecto a los datos experimentales.

Modelica proporciona comandos para poder definir experimentos complejos sobre los modelos. La descripción de estas facilidades para la definición de los experimentos es el objeto de este capítulo.

4.7.1. Operaciones básicas

Los comandos para la simulación del modelo se almacenan en un fichero de texto, cuya extensión es por convenio *.mos* (MModelica Scripting language). Como veremos, estos comandos son en realidad llamadas a funciones: funciones definidas en el estándar de Modelica, funciones proporcionadas por el entorno de modelado, y también funciones en lenguaje Modelica que el desarrollador del modelo puede programar e invocar no sólo desde otros modelos, sino también desde el experimento.

Por ejemplo, las siguientes funciones cargan todos los modelos contenidos en los ficheros *nombreFich1.mo*, *nombreFich2.mo* y *nombreFich3.mo*, hacen la com-

probación de uno de estos modelos, llamado *nombreModelo*, generan su código de simulación, lo simulan durante 100 segundos empleando el algoritmo DASSL, almacenan los resultados en el fichero *resultSimu* y representan en una misma gráfica las variables x e y frente al tiempo.

```
openModel("nombreFich1.mo");
openModel("nombreFich2.mo");
openModel("nombreFich3.mo");
checkModel(problem="nombreModelo");
translateModel(problem="nombreModelo");
simulateModel(problem="nombreModelo",
               stopTime=100, method="dassl", resultFile="resultSimu");
plot({"y","x"});
```

Cuando se realiza la traducción del modelo (`translateModel`), Dymola genera el código de la simulación, *dsmode.c*, el ejecutable de la simulación, *dymosim.exe*, y el fichero donde se define el experimento, *dsin.txt*. Cuando desde la GUI de Dymola se cambia la definición del experimento, por ejemplo, el valor de un parámetro o el instante final de la simulación, Dymola actualiza automáticamente el fichero *dsin.txt*.

El fichero del experimento, *dsin.txt*, puede obtenerse ejecutando en una ventana de MS-DOS

```
dymosim -i
```

Al ejecutar la simulación (`simulateModel`), se modifica el fichero del experimento *dsin.txt* y se ejecuta el fichero *dymosim.exe*, que lee el fichero *dsin.txt*. Los resultados de la simulación se almacenan por defecto en el fichero *dsres.mat*. Puede especificarse en la GUI de Dymola que los resultados se almacenen en formato ascii (Simulation/Setup/Output/Format/Textual data format). Alternativamente, puede convertirse este fichero a formato ascii ejecutando en la ventana de MS-DOS:

```
alist -as dsres.mat dsres.txt
```

El estado final del modelo se almacena en el fichero *dsfinal.txt*, el cual tiene el mismo formato que *dsin.txt*.

Puede obtenerse una lista de las funciones soportadas por Dymola ejecutando

```
listfunctions()
```

en la línea de comandos de la ventana Simulation. Puede emplearse la función `document` para obtener información acerca de una función, por ejemplo, si se escribe

```
document("plot")
```

se obtiene la documentación de la función `plot`.

El entorno Dymola tiene predefinidas ciertas variables que determinan la manera en que se realiza la traducción del modelo y su simulación. Puede obtenerse la lista de estas variables y sus valores escribiendo:

```
list()
```

El valor de estas variables puede modificarse en el experimento. En la Tabla 4.9 se indican algunas de estas variables y su significado.

4.7.2. Experimentos complejos

Los comandos para la definición del experimento pueden organizarse en diferentes ficheros *.mos*, que pueden ser invocados desde un fichero *.mos* principal, que coordine la ejecución del experimento.

El siguiente fragmento de código, extraído de la librería GAPILib, muestra cómo desde un fichero *.mos* pueden ejecutarse de manera automática otros ficheros *.mos*.

```
// Borra modelos cargados, declaración y valores de variables, etc
// Es igual que si de ejecuta File/Clear en la GUI de Dymola
clear

// Se asigna valor a la variable N_Cycle
// Al asignarsele valor automáticamente se declara
N_Cycle = 4

// Ejecuta el fichero GAPILib_INI.mos
@GAPILib_INI.mos

// Bucle for. No es preciso declarar i_cycle
for i_cycle in 1:N_Cycle loop

    // Muestra una ventana con el número de ciclo
    messageBox("CYCLE = " + integerString(i_cycle), 20);

    // Ejecuta el fichero GAPILib_CYCLE.mos
    @GAPILib_CYCLE.mos

end for;
```

En los ficheros *.mos* comúnmente se emplean variables específicas del experimento, que pueden ser de tipo string, real, entero y booleano. Estas variables se denominan variables interactivas. No es preciso declararlas, ya que se declaran automáticamente al asignárseles valor por primera vez. En el siguiente fragmento de código de un fichero *.mos*, se asigna valor a variables de diferente tipo:

Tabla 4.9: Algunas de las variables empleadas por Dymola para configurar la ejecución del experimento. Se indica en cada caso el valor por defecto.

Variable	Significado
<code>Evaluate:=true;</code>	Sustituye en el código de la simulación los parámetros por sus valores, exceptuando los parámetros que están en el nivel jerárquico superior del modelo. Esto permite generar un código de simulación más eficiente.
<code>OutputFlatModelica:=true;</code>	Genera el modelo plano al hacer la traducción. El modelo plano se guarda en un fichero con el mismo nombre que el modelo y extensión <i>.mof</i> .
<code>OutputCPUtime:=true;</code>	Incluye en el modelo una variable que representa el tiempo de CPU invertido en la simulación. La pendiente de esta señal es una medida indirecta de la complejidad computacional de la simulación.
<code>LogDefaultConnect:=true;</code>	Al hacer la traducción, muestra los conectores que no están conectados. Esto puede ayudar a detectar errores en el modelo.
<code>LogStateSelection:=true;</code>	Al hacer la traducción, muestra qué variables han sido seleccionadas como variables de estado.
<code>LogDifferentiatedEquations:=true;</code>	Al hacer la traducción, muestra qué ecuaciones han sido diferenciadas simbólicamente para reducir el índice.
<code>LogReadClasses:=true;</code>	Muestra las clases leídas al parsear los ficheros.
<code>WarnAboutParametersWithNoDefault:=true;</code>	Muestra aquellos parámetros a los cuales no se ha asignado valor.

```
GAPILibRoot = "C:\Archivos de programa\Dymola6\Work\GAPILib8(Polinimial)";
N_Cross_Point = 3;
CPS = zeros(N_Cross_Point);
Para = ["a","b","c","d"];
Ram_Seed = [0.31,0.25,0.72,0.93,0.13,0.23];
```

Ejecutando

```
list()
```

también se muestran las variables interactivas definidas por el usuario, así como su valor.

Para la inicialización, así como para controlar el flujo del programa, pueden emplearse bucles for y bloques if.

```
RANGE_control = "Auto";
if RANGE_control=="Interact" then
    Para_Range=[    0.031, 0.035;
                  5,    5.2;
                  10,   10.2;
                  0.01, 0.012];
end if;
if RANGE_control=="Auto" then
    @calculaPara_Range.mos
end if;
```

Por ejemplo, en el siguiente fragmento de código se genera el fichero *dsin.mat*, el cual contiene la descripción del experimento, y se obtiene la posición de determinados parámetros (en este caso a, b, c, d) en el fichero *dsin.mat*.

```
N_Parameters = 4;
Para = ["a","b","c","d"];
cd(SimuPath);
// Crea fichero dsin.mat con los parámetros que hay en dymosim
system("dymosim -ib dsin.mat");
Size_dsin = readMatrixSize("dsin.mat","initialName");
Para_Names = readStringMatrix("dsin.mat","initialName",Size_dsin[1]);
Pos_dsin = zeros(N_Parameters);
for i in 1:N_Parameters loop
    // Busca la posición del parámetro en dsin.mat/initialNames
    for j in 1:Size_dsin[1] loop
        if Para[1,i]==Para_Names[j] then
            // Posición del parámetro en dsin
            Pos_dsin[i] = j;
        end if;
    end for;
end for;
```

En el fragmento de código anterior se leyó de fichero. La función `writeMatrix` sirve para escribir una variable a fichero. Puede obtenerse información sobre el uso de estas funciones usando la función `document`. Por ejemplo, escribiendo en la ventana de comandos de Dymola

```
document("writeMatrix")
```

Desde el fichero `.mos` pueden invocarse funciones predefinidas de Modelica, así como funciones que hayan sido definidas en alguna librería, empleando en este caso la notación punto. A continuación, se muestra un fragmento de código de un fichero `.mos` en el cual se invocan dos funciones definidas dentro de la librería `GAPILib`:

```
RAM_cross = GAPILib.Basics.Ram_Gen_ARENA(Ram_Seed,N_Parameters-1);
(ExpData,Xexp,Yexp,ExpSize)=
  GAPILib.Basics.Exp_Adq_SISO(ExpFileName,XExpMatrixName,YExpMatrixName);
```

4.8. RAND MODEL DESIGNER

Además de los entornos de modelado basados en el lenguaje Modelica, existen otros entornos de simulación para sistemas DAE híbridos basados en otros formalismos de modelado diferentes al de Modelica. Un ejemplo interesante es el entorno de modelado Rand Model Designer (RMD), que soporta un lenguaje de modelado acausal y orientado a objetos denominado Model Vision Language, el cual está basado en Unified Modeling Language (UML).

Uno de los puntos fuertes de Model Vision Language es la flexibilidad que ofrece para la descripción de los modelos híbridos con estructura variable, que se realiza empleando gráficos de comportamiento (B-charts). Como ayuda para la descripción del modelo, RMD incorpora un editor de ecuaciones, un editor gráfico de B-charts y un editor de modelos.

Asimismo, RMD proporciona una colección de elementos visuales animados 2D y 3D, y controles (deslizadores, botones, campos numéricos, etc.) que facilitan la composición de interfaces de usuario para la interacción con la simulación del modelo. Estas capacidades posibilitan desarrollar de manera sencilla laboratorios virtuales, que permiten experimentar con el modelo y observar cuál es su respuesta dinámica. De hecho, la principal aplicación de RMD es el desarrollo de laboratorios virtuales para la enseñanza de las Ciencias y la Ingeniería, y de simuladores de entrenamiento para adiestramiento de operarios en el ámbito industrial. RMD genera los laboratorios virtuales como aplicaciones Windows independientes y también genera la

simulación como una librería dinámica de Windows (dll), que se puede usar como una aplicación embebida interactiva.

Si bien RMD es una herramienta comercial, existe una versión para uso educativo no comercial que ha sido la empleada en el desarrollo y simulación del caso de estudio mostrado en la Sección 4.8.5. Existe asimismo una versión no comercial de RMD, denominada Model Vision Studium (MVS).

4.8.1. Modelado de sistemas físicos

El lenguaje de modelado **Model Vision Language** está basado en Unified Modeling Language (UML), proporcionando las capacidades para la descripción del modelo más comúnmente ofrecidas por los lenguajes de modelado orientado a objetos. Entre ellas, cabe destacar las siguientes:

- **Modelado modular y jerárquico.** Pueden definirse nuevos modelos mediante la conexión de modelos previamente definidos. Los modelos compuestos pueden ser, a su vez, conectados a otros modelos.
- La conexión de modelos se basa en el **principio de balance de energía**. La clase *connector* facilita la descripción de las interfaces y conexiones del modelo. Al igual que en Modelica, las variables del conector son o bien across, o bien through. Las variables across tienen el mismo valor en el punto de conexión, mientras que las variables through se suman y ésta suma se iguala a cero.
- **Herencia de clases.** Model Vision Language soporta herencia simple. Una clase derivada o subclase hereda el comportamiento y estructura de la clase base (superclase), y la clase base se puede extender añadiendo nuevo comportamiento y estructura (ecuaciones, componentes conexiones, etc.). Es posible suprimir en la clase derivada una parte del comportamiento heredado.
- **Parametrización de clases.** En sentido amplio, un parámetro es cualquier propiedad que se puede modificar para adaptar el modelo a sus diferentes aplicaciones. Model Vision Language permite que las variables independientes del tiempo sean declaradas como parámetros formales de la clase, pudiendo así recibir sus valores actuales en cada instancia de la clase. A diferencia de Modelica, Model Vision Language no soporta la parametrización de la clase de los objetos.
- Definición de **clases parciales**, también conocidas como clases abstractas, que no se pueden instanciar.

- Facilita la aplicación de principios de modelado orientado a objetos tales como la **abstracción** (capacidad de usar componentes sin conocer sus detalles internos) y el **encapsulado de la información** (sólo las interfaces del modelo son accesibles por otros modelos). La abstracción es soportada permitiendo distinguir entre la interfaz del modelo y su descripción interna. La primera describe completamente la interacción entre el modelo y su entorno. La última contiene la información sobre la estructura y el comportamiento del modelo.
- **Descripción acausal** del modelo, usando ecuaciones diferenciales y algebraicas. El lenguaje permite el uso de ecuaciones vectoriales y matriciales.
- Permite el **uso combinado de ecuaciones y algoritmos** (secuencias de sentencias de asignación) y la reutilización de los algoritmos definiéndolos como *funciones*.
- Pueden incluirse en los modelos llamadas a código escrito en los **lenguajes de programación C y FORTRAN**.
- Pueden incluirse **anotaciones** e información adicional en el modelo, por ejemplo la descripción de la representación gráfica de los iconos del modelo, diagramas, etc.
- Declaración de **tipos de datos** definidos por el usuario.
- Desarrollo y reutilización de **librerías de modelos**.

Model Vision Language permiten describir el comportamiento de tiempo continuo del modelo mediante ecuaciones y algoritmos. Para facilitar al desarrollador del modelo dicha tarea, RMD tiene un **editor de ecuaciones** donde pueden escribirse las fórmulas empleando notación matemática. En la Figura 4.46 se muestra un ejemplo, donde se emplea el editor de ecuaciones de RMD para describir el flujo de líquido a través de una tubería.

El comportamiento híbrido es descrito en Model Vision Language empleando gráficos de comportamiento, que reciben el nombre de B-charts. Un **B-chart** es un diagrama compuesto de modos (también llamadas fases), transiciones internas a los modos y transiciones entre modos. RMD también incorpora un editor gráfico de B-charts (véase la Figura 4.47). Los **modos** se representan como rectángulos con esquinas redondeadas y las **transiciones entre modos** como flechas conectando el modo origen de la transición con el modo destino.

En la parte superior de la Figura 4.48 se muestra un modo y las etiquetas que indican las acciones de entrada y salida, y la actividad del modo. En la parte inferior

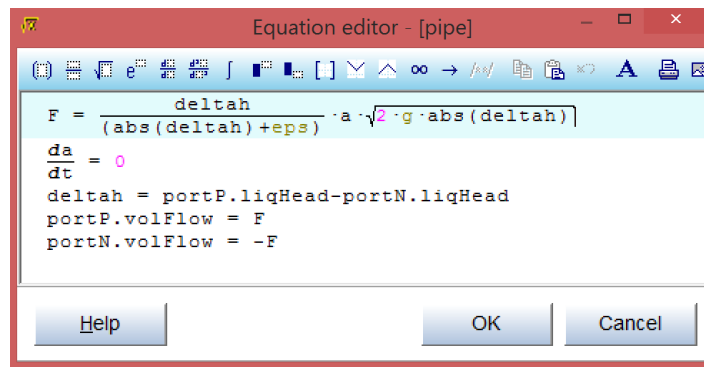


Figura 4.46: Ejemplo de uso del editor de ecuaciones de RMD.

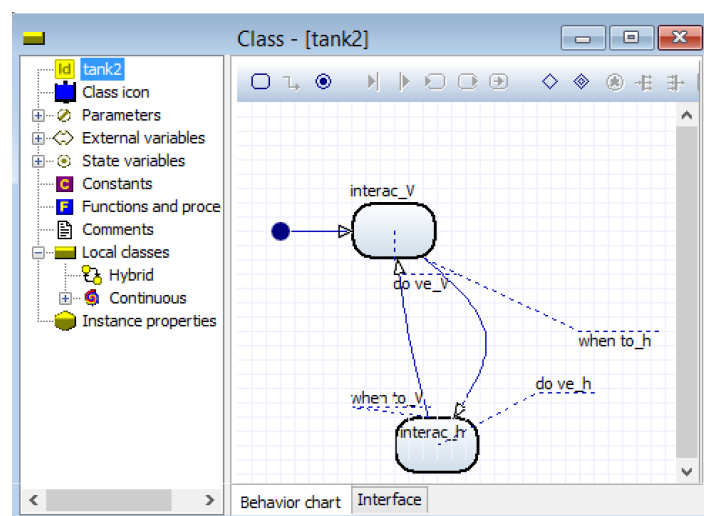


Figura 4.47: Ejemplo de uso del editor de B-charts de RMD.

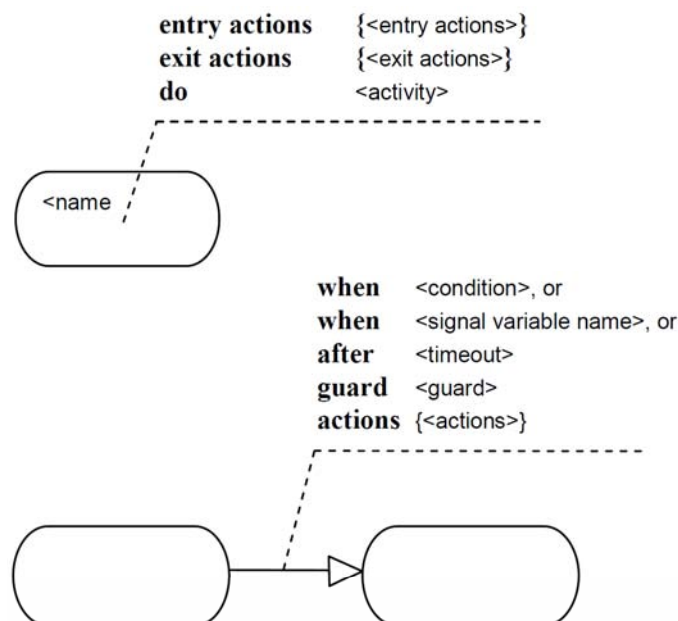


Figura 4.48: Modo de un *B-chart* (arriba) y transición entre modos (abajo).

de la Figura 4.48 puede verse una transición entre dos modos y sus etiquetas, las cuales indican las condiciones de disparo y guarda, y las acciones asociadas.

En cada instante de la simulación, uno y sólo uno de los modos del B-chart es el modo activo. Es posible asociar a cada modo acciones de entrada y salida, y una actividad (véase nuevamente la parte superior de la Figura 4.48).

- Las **acciones de entrada y salida** son secuencias de sentencias de asignación que se ejecutan cada vez que se entra y sale del modo, respectivamente.
- La **actividad** del modo se especifica mediante una clase local, llamada **clase actividad del modo**, que describe la estructura y comportamiento del modelo mientras ese modo es visitado. Las clases de actividad pueden ser tan complejas como se precise, incluyendo modelos compuestos y modelos híbridos descritos mediante otros B-charts.

Pueden declararse variables internas a la clase actividad de un modo, que sólo existen mientras dicho modo está activo, y también variables locales del B-chart, que existen mientras exista el B-chart, con independencia del modo del B-chart que se encuentre activo.

Como pudo verse en la Figura 4.48, es posible asociar a las transiciones entre modos **condiciones de disparo y guarda**, así como **acciones** (secuencias de sentencias de asignación) que se ejecutarán cada vez que se produzca la transición. La lógica seguida en la ejecución de las transiciones es la siguiente. Si el modo origen de una transición es el modo activo del B-chart, y además se satisface tanto la condición de disparo como la condición de guarda, entonces se ejecuta la acción asociada a la transición y el modo destino de la transición pasa a ser el modo activo del B-chart. Si se satisface la condición de disparo, pero no la condición de guarda, entonces no se ejecuta la transición, perdiéndose el disparo de la misma.

Model Vision Language permite describir modelos compuestos definidos mediante instanciación y conexión de otras clases de modelos, los cuales a su vez pueden ser tanto atómicos (sin estructura interna) como compuestos. RMD contiene un **editor gráfico de modelos**, que facilita la definición de los modelos compuestos (véase la Figura 4.49). Los componentes son instanciados pinchando y arrastrando, desde la ventana donde se encuentra el árbol de clases disponibles, hasta la ventana de edición del modelo. La conexión entre componentes se realiza con el ratón, señalando los conectores que deben ser conectados. Puede accederse a los parámetros de un componente haciendo doble clic sobre su icono en la ventana de edición del modelo.

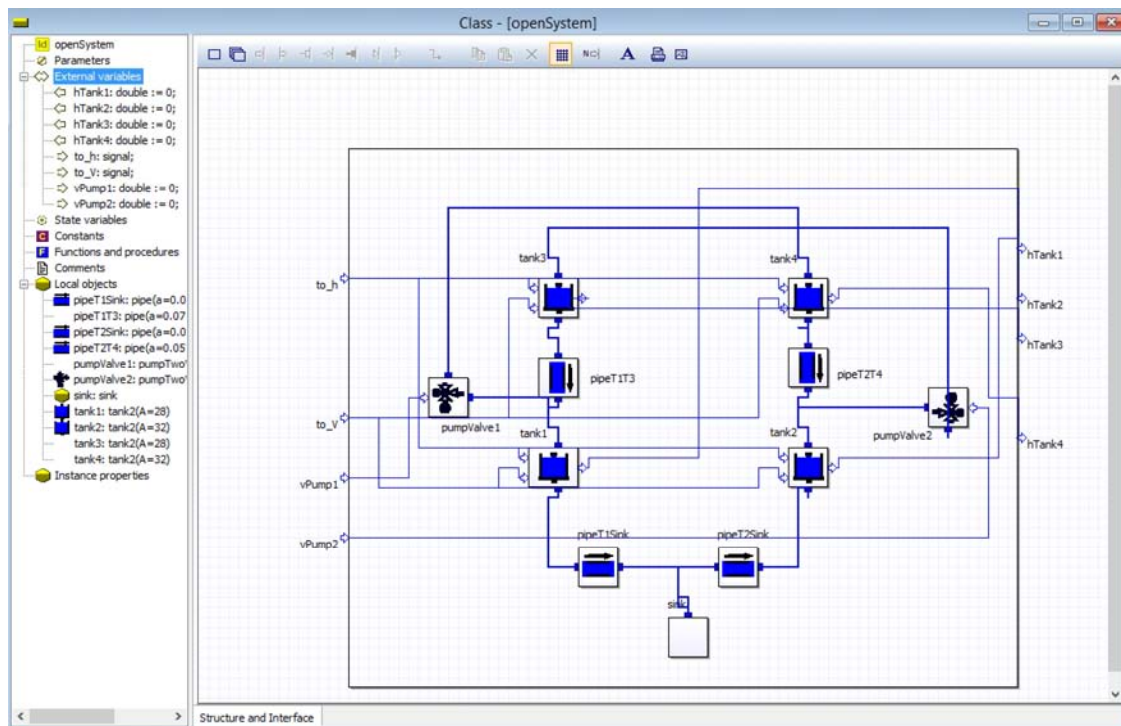


Figura 4.49: Ejemplo de uso del editor de modelos de RMD. El modelo mostrado es el del sistema de cuatro tanques que será descrito en la Sección 4.8.5.

Los componentes de un modelo compuesto pueden tener comportamiento híbrido. Cada componente con comportamiento híbrido está descrito mediante su propio B-chart. En cada instante de la simulación, cada B-chart tendrá uno de sus modos activo. El sistema de ecuaciones del modelo completo actual es construido automáticamente por RMD en tiempo de simulación a partir de las actividades de los modos que se encuentran activos en el instante actual (uno por cada B-chart). Las transiciones entre modos pueden modificar las ecuaciones del modelo y el número de las mismas, así como el número y significado de las variables del modelo.

La flexibilidad ofrecida por Model Vision Language para la descripción de modelos con estructura variable es uno de sus puntos fuertes y también es una de las diferencias conceptuales sustanciales respecto al lenguaje Modelica. Sin embargo, ofrecer esta flexibilidad conlleva que RMD debe construir, analizar y manipular el modelo completo actual en tiempo de simulación cada vez que se produce una transición.

Durante la ejecución de la simulación, RMD sólo mantiene en memoria el código de la simulación del modelo actual. Cuando se produce una transición, RMD elimina el código de la simulación antiguo y genera automáticamente el nuevo código de simulación. Para ello, construye la descripción matemática del modelo completo

actual, elimina las variables redundantes y ecuaciones triviales resultantes de las conexiones entre componentes, analiza la estructura computacional del modelo, selecciona los métodos numéricos y genera el código de la simulación.

El análisis de la estructura del modelo y la generación del código de la simulación se realiza repetidamente durante la ejecución de la simulación. Esto implica que, a fin de mantener la eficiencia de la simulación, RMD sólo puede emplear algoritmos rápidos, entre los que se encuentra un algoritmo muy eficiente para seleccionar cuál de los métodos numéricos soportados es más adecuado al modelo actual en cuestión. RMD emplea métodos numéricos específicos para resolución de sistemas de ecuaciones algebraicas, ecuaciones diferenciales y ecuaciones algebraico diferenciales. Entre estos últimos se encuentran RADAU, DDASSL y DDASPK. Sin embargo, en la fecha de escritura de este texto RMD no soporta ni derivación simbólica, ni reducción del índice de sistemas de ecuaciones algebraico diferenciales.

4.8.2. Ejemplo 1: diodo ideal

Supongamos que se desea realizar en RMD una librería de componentes eléctricos que contenga los modelos de un generador de voltaje, resistencia, condensador, diodo, etc. En particular, supongamos que se desea describir el diodo ideal mediante un modelo con dos modos, tal como se muestra en la Figura 4.32. El B-chart que describe dicho comportamiento, compuesto con RMD, es mostrado en la Figura 4.50. Obsérvese que el modelo puede encontrarse en dos modos o fases, llamadas *off* y *on*, correspondientes a las condiciones de funcionamiento de corte y conducción, respectivamente.

De manera análoga a como se hizo al desarrollar la librería eléctrica en Modelica en la Sección 2.4.2, las interfaces de los componentes de la librería eléctrica descrita en RMD están compuestos de conectores en los cuales la variable across es el voltaje y la variable through es la corriente.

Pueden definirse en RMD modelos compuestos, instanciando y conectando componentes previamente definidos. El circuito rectificador mostrado en la Figura 4.51 es un ejemplo de ello. Los diodos empleados en el modelo del circuito son instancias de la clase diodo definida mediante el B-chart de la Figura 4.50.

RMD realiza la simulación del circuito rectificador de la forma siguiente. En cada instante de la simulación, cada uno de los cuatro diodos se encuentra en uno de sus modos (*off* u *on*). Dado que el circuito rectificador contiene cuatro diodos, existen $2^4 = 16$ posibles combinaciones de modos. Cada vez que se produce una transición

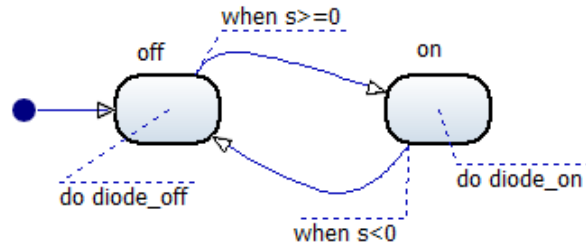


Figura 4.50: B-chart describiendo el comportamiento de un diodo ideal.

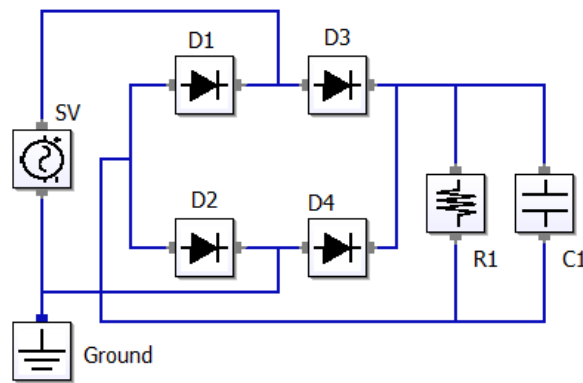


Figura 4.51: Diagrama del modelo de un circuito rectificador descrito con RMD.

en uno de los modelos de los cuatro diodos, RMD automáticamente construye el sistema de ecuaciones que representa la nueva estructura del circuito completo y reanuda la simulación usando este nuevo sistema de ecuaciones.

Dado que es posible declarar variables locales en las clases actividad de los modos, en RMD el número y significado de las variables puede cambiar en tiempo de simulación. Asimismo, una variable del B-chart puede ser variable de estado en algunos de los modos, variable algebraica en otros y estar indefinida en los restantes. En este último caso, el valor de la variable es NaN (Not a Number).

Obsérvese la diferencia con el lenguaje Modelica, en el cual aplica la *regla de asignación única*. Esta regla impone que el número de variables desconocidas y de ecuaciones del modelo debe ser igual y constante a lo largo de la simulación, y además el número de ecuaciones en cada rama de una cláusula if debe ser igual y constante.

Las ecuaciones en Modelica describen relaciones entre variables que deben satisfacerse de manera concurrente. El conjunto de ecuaciones activas está compuesto únicamente de ecuaciones de tiempo continuo durante la resolución del problema de tiempo continuo, y está compuesto de ecuaciones de tiempo continuo y discreto

durante la resolución de los problemas de reinicio. Al traducir el modelo, el entorno de modelado de Modelica asigna la causalidad computacional considerando tanto las ecuaciones de tiempo continuo como las ecuaciones discretas, obteniendo un orden de evaluación de las ecuaciones que es correcto para todos los posibles conjuntos de ecuaciones activas. La ventaja es que el análisis estructural y las manipulaciones simbólicas del modelo son realizadas una única vez, en un paso previo a la generación del código ejecutable de la simulación.

4.8.3. Desarrollo de simulaciones interactivas

Se dice que una simulación es **interactiva** cuando objetos externos pueden cambiar el valor de determinadas variables del modelo, a las que se denomina **variables interactivas**, durante la simulación del modelo. Estos cambios se denominan **eventos interactivos**. Los instantes de tiempo en que se producen los eventos interactivos están determinados por los objetos externos. En general, durante la ejecución de una simulación puede producirse un número finito arbitrario de eventos interactivos.

Dependiendo de la aplicación, los objetos externos que producen los eventos interactivos son personas (como sucede en los laboratorios virtuales), hardware (como ocurre en las simulaciones con hardware en el lazo) u otras simulaciones (como en la simulación distribuida en tiempo real).

Durante la ejecución de la simulación interactiva, se establece un flujo bidireccional de información entre dicha simulación y los objetos externos. La simulación del modelo envía a los objetos externos el valor actual de determinadas variables del modelo. Los objetos externos envían a la simulación del modelo la información precisa para ejecutar los eventos interactivos.

Para que puedan producirse los cambios interactivos en el modelo, es preciso formular éste adecuadamente. En particular, para que una variable sea interactiva, es preciso que sea una variable de estado del modelo. Esto tiene las dos implicaciones siguientes.

1. Una variable dependiente del tiempo puede ser interactiva si y sólo si existe al menos una selección de variables de estado que incluya a dicha variable.
2. Un conjunto de variables pueden ser magnitudes interactivas modificadas en la misma acción interactiva si y sólo si existe al menos una selección de variables de estado que incluya todas las variables del conjunto.

Con el fin de permitir cambios interactivos en el valor de los parámetros del modelo físico, es preciso definir estos como variables dinámicas cuya derivada sea en todo momento igual a cero y seleccionarlas como variables de estado del modelo. Así, el valor de estas variables de estado sólo cambia en los instantes de los eventos interactivos, manteniéndose constante el resto del tiempo.

Otro hecho a tener en cuenta es que el resultado de una acción interactiva depende de la selección de las variables de estado del modelo. Esto es debido a que en una acción interactiva se mantienen constantes todas aquellas variables de estado que no hayan sido modificadas por los objetos externos.

El hecho de que las variables interactivas deban ser seleccionadas como variables de estado implica que algunas simulaciones interactivas requieran modificar en tiempo de simulación la selección de las variables de estado del modelo. Para ilustrar el significado de esta afirmación nos valdremos del ejemplo mostrado en la Figura 4.52. Se trata de un modelo que describe el comportamiento dinámico de un sistema compuesto por una bomba de líquido y un depósito. La bomba introduce líquido por la parte superior del depósito y éste pierde líquido a través de un orificio situado en su base.

Supongamos que el voltaje aplicado a la bomba (v) es una variable de entrada, ya que su valor no se calcula a partir de las ecuaciones del modelo. La sección transversal del tanque (A) y del orificio de salida (a), el parámetro de la bomba (k) y la aceleración gravitacional (g) son parámetros. El volumen de líquido (V), los flujos de entrada y salida (F_{in} , F), y el nivel de líquido (h) son variables dependientes del tiempo.

Este modelo admite diferentes selecciones de las variables de estado. Por ejemplo, puede seleccionarse como variable de estado el volumen de líquido (V), el nivel de líquido (h) o el flujo de salida (F). La selección de las variables de estado debe hacerse de modo que incluya todas las magnitudes interactivas. Por ejemplo, si se quiere cambiar interactivamente el valor del nivel del líquido, la selección apropiada para la variable de estado es h . Si se quiere cambiar el volumen de líquido o el flujo de salida, entonces la selección adecuada es V o F , respectivamente.

Ahora bien, supongamos que la simulación interactiva deba soportar las tres formas anteriores de especificar los cambios interactivos en la cantidad de líquido contenido en el depósito. Cada vez que el objeto externo necesite cambiar la cantidad de líquido, puede elegir entre describir el cambio en términos del volumen, del nivel o del flujo de salida. La simulación interactiva debería ejecutar los eventos interactivos usando en cada caso la selección adecuada de las variables de estado.

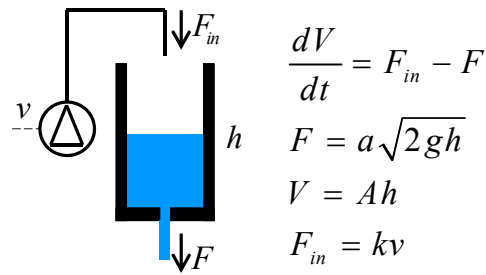


Figura 4.52: Bomba de líquido conectada a un depósito.

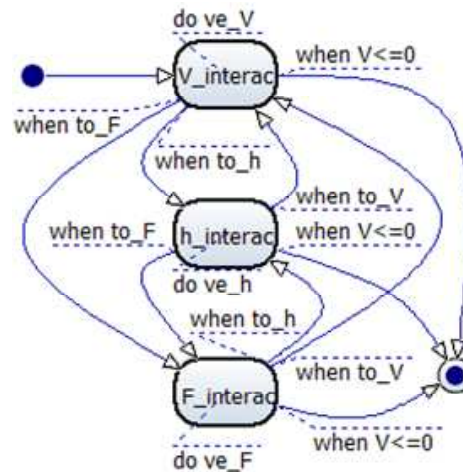


Figura 4.53: B-chart del modelo interactivo del sistema de la Figura 4.52.

Obsérvese que los cambios en el valor de las magnitudes interactivas tienen diferente efecto dependiendo de la selección de las variables de estado. Supongamos que se cambia interactivamente la sección transversal del tanque (A). Si el volumen (V) es una variable de estado, entonces el cambio en A produce un cambio abrupto en el valor del nivel de líquido (h) y el flujo (F), mientras que el volumen de líquido permanece constante. Por el contrario, si la variable de estado es el nivel (h) o el flujo (F), los valores de estas magnitudes no cambiarán con un cambio instantáneo en la sección transversal (A), pero sí cambiará el volumen (V).

Los B-charts permiten definir de manera sencilla modelos interactivos compuestos por varias instancias del modelo físico, cada una de ellas con una selección diferente de las variables de estado. Por ejemplo, en la Figura 4.53 se muestra el B-chart del modelo interactivo del sistema bomba-tanque de la Figura 4.52. El B-chart consta de tantos modos como diferentes selecciones de las variables de estado se precisan: modos $V_interac$, $h_interac$ y $F_interac$.

La clase de actividad ve_V , asociada al modo $V_interac$, contiene el modelo con el volumen de líquido como variable de estado. Análogamente, las clases de actividad

ve_h y ve_F , asociadas a los modos $h_{interac}$ y $F_{interac}$, contienen el modelo con el nivel de líquido y el flujo de salida como variables de estado, respectivamente. Las áreas del tanque (A) y del orificio de salida (a), así como voltaje de entrada de la bomba (v), son variables de estado en las tres clases de actividad.

Se ha definido el B-chart de modo que cada acción interactiva se ejecute sobre la instancia del modelo que tiene la selección adecuada de las variables de estado. Para ello, siempre que sea necesario se producirá la correspondiente transición de modo previamente al cambio en el valor de las variables interactivas. Como puede apreciarse en el B-chart, las condiciones de disparo de las transiciones (véanse las cláusulas *when* asociadas a las transiciones) se definen mediante las señales to_V , to_h y to_F . Estas señales pueden ser enviadas, por ejemplo, por botones situados en la vista del laboratorio virtual. La simulación finaliza en este ejemplo cuando el volumen de líquido se hace menor o igual que cero.

4.8.4. Descripción de la vista del laboratorio virtual

En un laboratorio virtual, se denomina **vista** a la interfaz gráfica interactiva entre la simulación del modelo y el usuario del laboratorio virtual. El propósito de la vista es doble. Por una parte, proporcionar al usuario información visual del comportamiento dinámico del modelo. Por otra, permitir al usuario modificar el valor de ciertas variables del modelo.

Para ello, las propiedades visuales de los elementos de la vista deben enlazarse a las variables del modelo, produciéndose un flujo bidireccional de información en tiempo de ejecución entre la vista y el modelo: los cambios en el valor de las variables del modelo son mostrados en la vista y las acciones del usuario sobre la vista modifican el valor de las variables del modelo.

RMD proporciona una colección de componentes predefinidos de animación 2D y 3D, y de control (deslizadores, botones, campos numéricos, etc.), que facilitan la creación de interfaces interactivas compuestas de diagramas animados, gráficos y controles. Los controles y elementos visuales 2D se sitúan en la ventana de la vista pinchando y arrastrándolos con el ratón desde la ventana “Standard 2D components”. En la parte superior de la Figura 4.54 se muestra dicha ventana. En la parte inferior de la figura puede verse la ventana en la cual se configuran las propiedades de los elementos 3D proporcionados por RMD, entre los que se encuentran línea, esfera, cilindro, cono, toro, muelle, haz, cuadrado, flecha, superficie y texto.

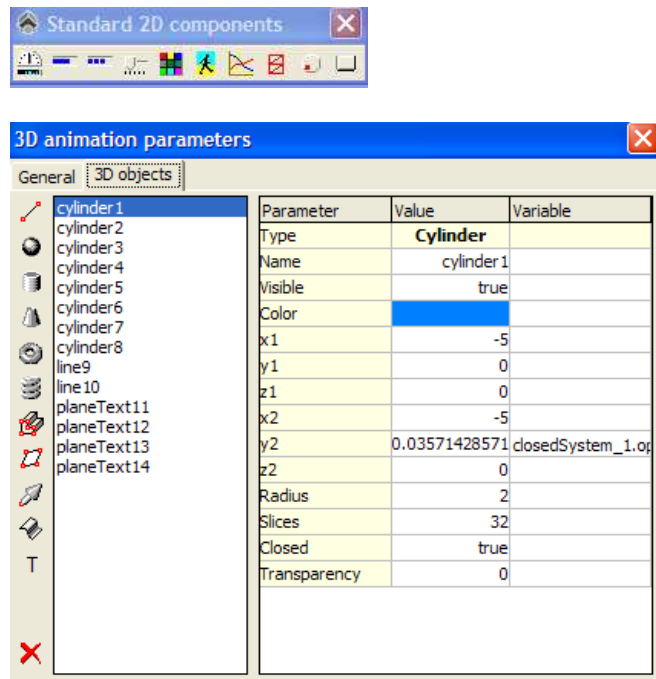


Figura 4.54: Componentes gráficos proporcionados por RMD.

Las propiedades de los elementos visuales y los controles pueden ser enlazadas de manera sencilla a las variables del modelo. Para facilitar la selección de las variables a enlazar, RMD permite navegar por la estructura jerárquica del modelo, muestra las variables de los componentes y simplemente haciendo clic sobre una variable ésta queda enlazada a la propiedad seleccionada del elemento gráfico o control. El ejemplo mostrado a continuación servirá para ilustrar estas capacidades.

4.8.5. Ejemplo 2: sistema de cuatro tanques

El sistema de cuatro tanques mostrado en la Figura 4.55 se emplea para ilustrar conceptos de la teoría de control automático multivariable. El objetivo es controlar el nivel de líquido en los dos tanques inferiores (h_1 y h_2), manipulando para ello los voltajes de las bombas (v_1 y v_2). Los balances de líquido en los tanques vienen descritos por las Ecs. (4.164) – (4.167), donde a_i y A_i son respectivamente el área del orificio y de la base del tanque i . La constante de proporcionalidad entre el voltaje de entrada y el flujo de salida de la bomba j es k_j . La proporción del flujo de la bomba j que va al tanque superior es γ_j .

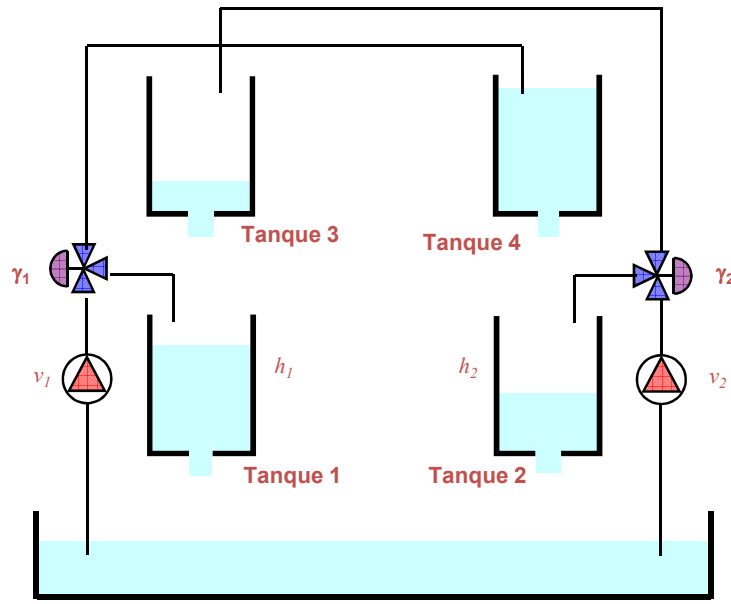


Figura 4.55: Representación esquemática del sistema de los cuatro tanques.

$$\frac{dh_1}{dt} = -\frac{a_1}{A_1} \sqrt{2 \cdot g \cdot h_1} + \frac{a_3}{A_1} \sqrt{2 \cdot g \cdot h_3} + \frac{\gamma_1 \cdot k_1}{A_1} \cdot v_1 \quad (4.164)$$

$$\frac{dh_2}{dt} = -\frac{a_2}{A_2} \sqrt{2 \cdot g \cdot h_2} + \frac{a_4}{A_2} \sqrt{2 \cdot g \cdot h_4} + \frac{\gamma_2 \cdot k_2}{A_2} \cdot v_2 \quad (4.165)$$

$$\frac{dh_3}{dt} = -\frac{a_3}{A_3} \sqrt{2 \cdot g \cdot h_3} + \frac{(1 - \gamma_2) \cdot k_2}{A_3} \cdot v_2 \quad (4.166)$$

$$\frac{dh_4}{dt} = -\frac{a_4}{A_4} \sqrt{2 \cdot g \cdot h_4} + \frac{(1 - \gamma_1) \cdot k_1}{A_4} \cdot v_1 \quad (4.167)$$

El modelo linealizado de este sistema tiene un cero multivariable, que se puede localizar en el semiplano izquierdo o derecho manipulando las válvulas de doble vía, es decir, cambiando los valores de γ_1 y γ_2 . El sistema tiene fase no mínima si $0 < \gamma_1 + \gamma_2 < 1$ y fase mínima si $1 < \gamma_1 + \gamma_2 \leq 2$.

El laboratorio virtual deberá permitir al usuario realizar cambios interactivos en los parámetros físicos de los tanques (sección transversal de los tanques y de los orificios de salida) y en la cantidad de líquido almacenado en los mismos. Los cambios en la cantidad de líquido almacenado podrán especificarse indicando el volumen y el nivel de líquido. Para ello, el modelo interactivo del depósito se describirá mediante un B-chart con dos modos, correspondientes a dos descripciones del modelo físico del depósito: una en la cual el volumen de líquido es variable de estado y otra en la cual lo es el nivel del líquido.

El laboratorio virtual deberá soportar dos estrategias de control: manual y PI descentralizado. El usuario podrá cambiar la estrategia de control en tiempo de simulación. En modo automático, podrá cambiar interactivamente el valor de los parámetros de los controladores PI. En el modo manual, el usuario podrá cambiar el voltaje aplicado a las bombas (v_1 y v_2). Asimismo, el usuario podrá cambiar en tiempo de simulación el valor de las variables γ_1 y γ_2 de las válvulas de doble vía. Por simplicidad, en este modelo se asigna valor uno a k_1 y k_2 .

Se ha empleado RMD para desarrollar una librería de componentes hidráulicos (tanque, tubería, válvula y bomba) y el controlador PI. Como se ha indicado anteriormente, el modelo del tanque es similar al B-chart de la Figura 4.53, pero con sólo dos modos: *interac_V* e *interac_h*. El sistema de control se describe empleando un B-chart con dos modos, cada uno de los cuales describe una de las dos posibles estrategias de control: manual y automático. Los botones situados en la vista del laboratorio virtual emiten señales que disparan las transiciones entre los modos.

En las Figuras 4.56 y 4.57 se muestran los modelos del sistema de cuatro tanques en lazo abierto y en lazo cerrado. Puede observarse en el modelo en lazo cerrado que se ha conectado un saturador a la salida de cada controlador PI. La finalidad es evitar que el voltaje de entrada a las bombas adquiera valores negativos.

La interfaz del modelo en lazo abierto, que es mostrado en la Figura 4.56, está compuesta por dos variables de entrada (v_{Pump1} y v_{Pump2}) que describen los voltajes aplicados a las bombas, dos señales de entrada (to_h y to_V) que disparan las transiciones a los modos *interac_V* e *interac_h* en los B-charts que describen los tanques, y cuatro variables de salida que son los niveles de líquido en los cuatro tanques.

La interfaz del modelo en lazo cerrado (véase la Figura 4.57) está compuesta por cuatro señales y cuatro variables de entrada. Las señales *auto* y *manual* permiten cambiar el control del modo manual al modo automático. Las señales to_h y to_V disparan el cambio de modo en los modelos de los tanques. Cuando el sistema opera en modo manual, las variables de entrada ($SP1$, $SP2$, $uManual1$, $uManual2$) permiten fijar los valores de las referencias y de la señal de control. El valor de estas variables y señales de entrada al modelo es asignado en la vista del laboratorio virtual.

En las Figuras 4.58 y 4.59 se muestran dos momentos en la definición de la vista del laboratorio virtual. En la Figura 4.58 se muestra el descenso por la jerarquía del modelo para visualizar las variables de uno de sus componentes (en este ejemplo, el componente llamado *pumpValve1*) y asociar una de esas variables a una propiedad de un elemento de la vista. En la Figura 4.59 puede apreciarse cómo va construyéndose

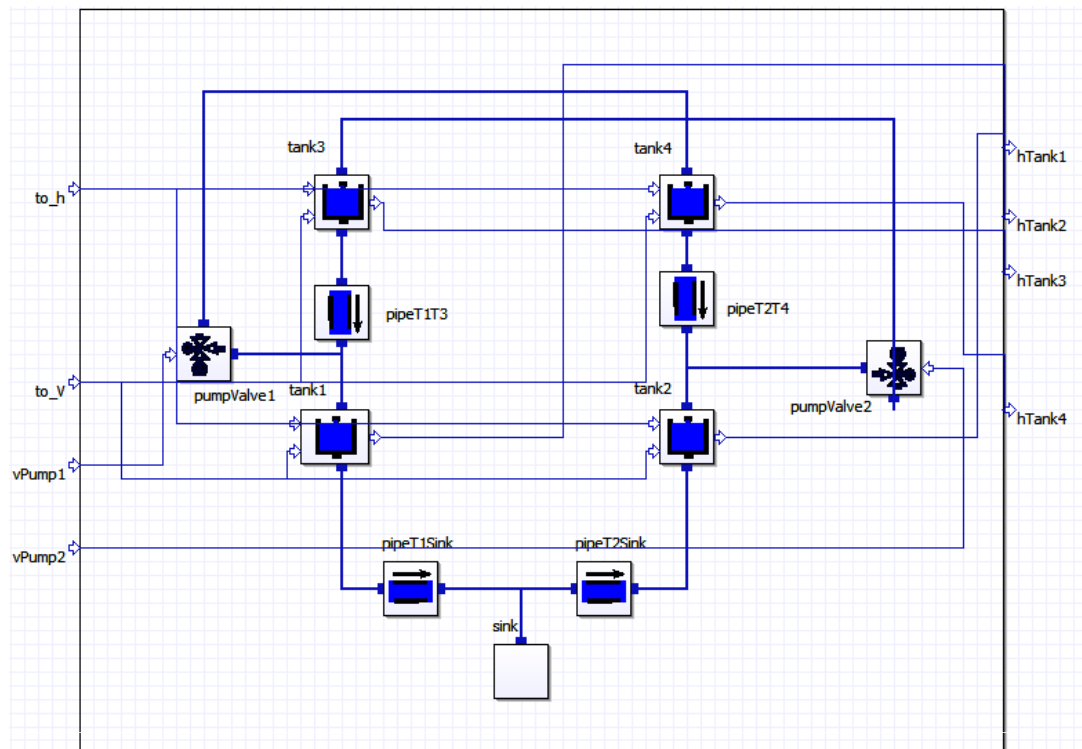


Figura 4.56: Diagrama del modelo del sistema de cuatro tanques en RMD.

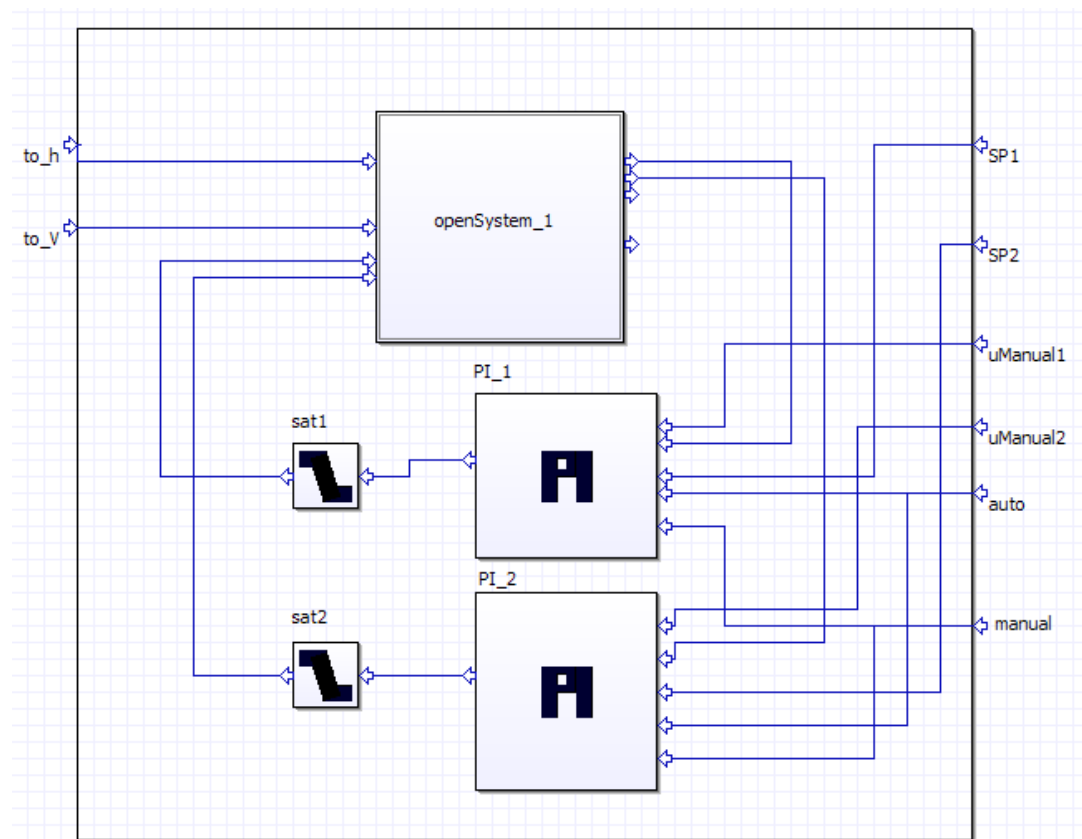


Figura 4.57: Diagrama del modelo en lazo cerrado del sistema de cuatro tanques en RMD.

la vista del laboratorio virtual, pinchando y arrastrando elementos desde las ventanas “Standard 2D components” y “3D animation parameters”.

Finalmente, en la Figura 4.60 se muestra la vista del laboratorio virtual desarrollado. Se compone de dos ventanas: “2D animation” y “3D animation”. La ventana “2D animation” contiene botones, deslizadores y gráficos que muestran la evolución temporal de los niveles de líquido y de sus referencias. Los botones permiten al usuario cambiar la selección de las variables de estado y la estrategia de control. Los deslizadores permiten cambiar las secciones transversales de los tanques y de los orificios de salida, las referencias para los niveles, los flujos de entrada y los parámetros de los controladores PI. La ventana “3D animation” contiene una representación animada del sistema que muestra la evolución de los niveles de líquido.

Este laboratorio virtual sirve para ilustrar conceptos de la teoría de control automático multivariable, permitiendo a los estudiantes visualizar el comportamiento dinámico del proceso y el efecto de las perturbaciones, evaluar la interacción entre los dos lazos de control para diferentes configuraciones de la planta, estudiar las diferencias entre los comportamientos de fase mínima y no mínima y las dificultades de control que entraña este último caso, y sintetizar controladores PI. A continuación se enumeran algunas de las actividades que pueden realizarse con el laboratorio virtual.

1. Modificar el valor de uno de los parámetros de diseño del sistema de cuatro tanques (secciones transversales de los tanques, secciones de los orificios de salida, apertura de las válvulas, voltaje de las bombas, γ_1 y γ_2) y observar cómo evoluciona el nivel de líquido de los tanques.
2. Dadas unas condiciones iniciales, unos valores de γ_1 y γ_2 para los que el sistema está en fase mínima (es fácil de controlar), y unos valores de referencia para h_1 y h_2 , realizar el control manual de la planta.
3. Dadas unas condiciones iniciales, unos valores de γ_1 y γ_2 para los que el sistema está en fase no mínima (es difícil de controlar) y unos valores de referencia para h_1 y h_2 , realizar el control manual de la planta.
4. Sintetizar los dos controladores PI y usarlos para controlar la planta usando el laboratorio virtual.
5. Introducir perturbaciones en la planta manipulando el laboratorio virtual (p. ej., produciendo cambios en los niveles de líquido de los tanques) y discutir el comportamiento en lazo cerrado de la planta.

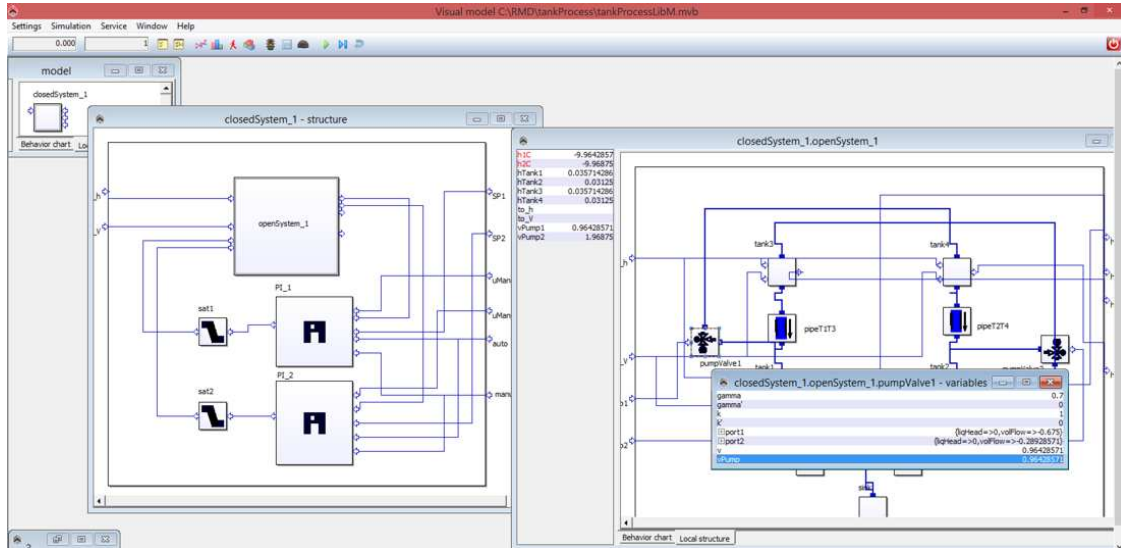


Figura 4.58: Navegación por la jerarquía del modelo para asociar las variables del modelo a las propiedades de los elementos de la vista.

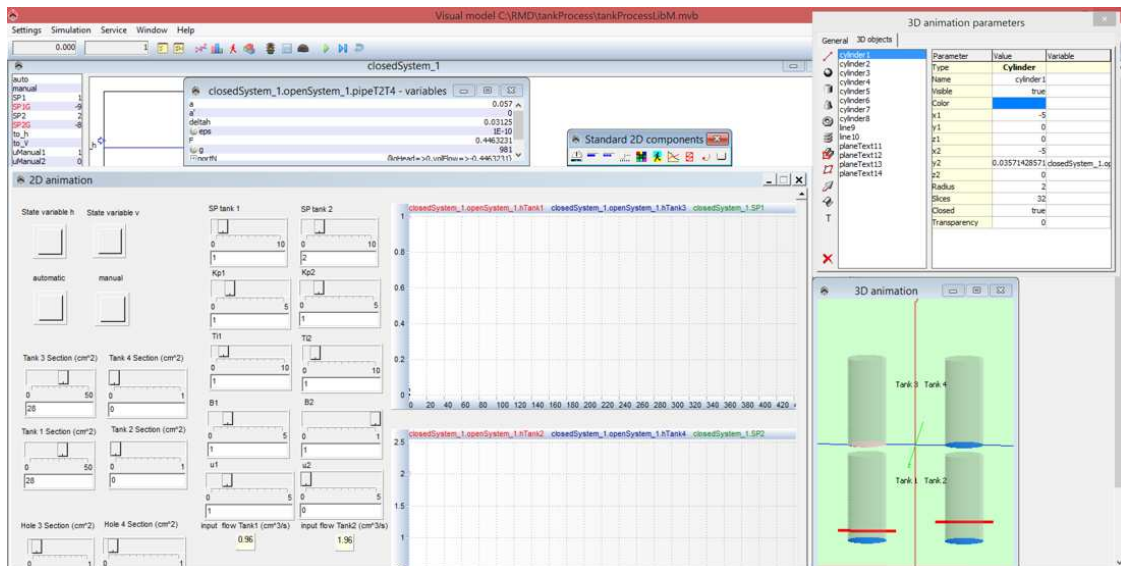


Figura 4.59: Composición de la vista del laboratorio virtual a partir de los componentes visuales y controles predefinidos de RMD.

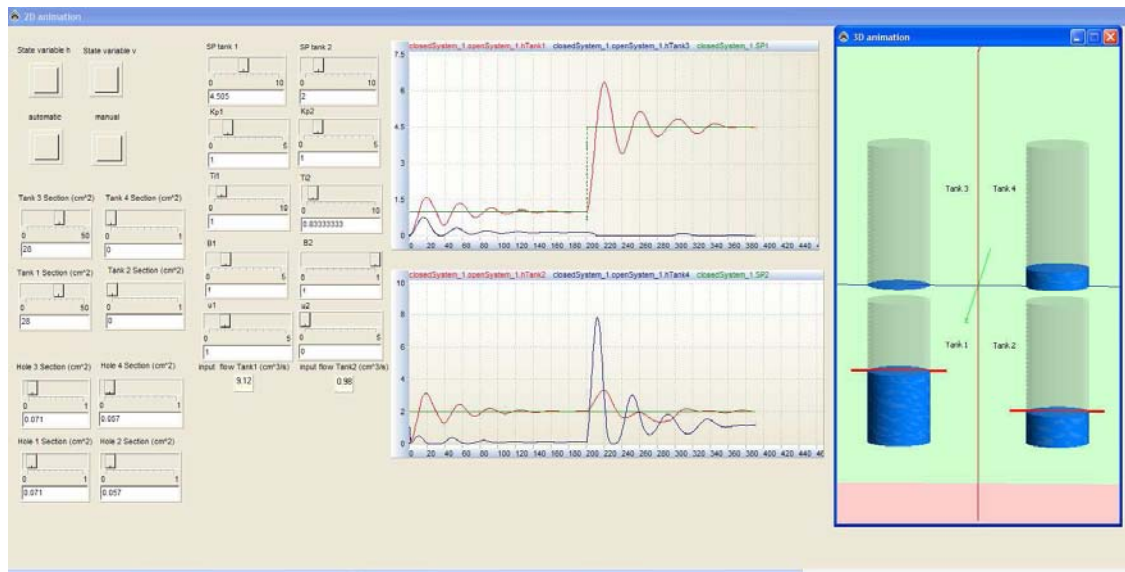


Figura 4.60: Vista del laboratorio virtual de los cuatro tanques en RMD.

La descripción anterior demuestra que RMD permite desarrollar laboratorios virtuales aplicando la metodología del modelado orientado a objetos, mediante un lenguaje de modelado propio llamado Model Vision Language que está basado en UML. Para facilitar la descripción del modelo, RMD incorpora un editor gráfico de modelos, un editor de ecuaciones y un editor gráfico de B-charts. Un punto fuerte de RMD es la flexibilidad ofrecida en la descripción de modelos con estructura variable. La vista del laboratorio virtual se compone instanciando los elementos visuales animados 2D y 3D, y los controles proporcionados por RMD. La conexión entre el modelo y la vista se realiza a través de una interfaz gráfica de usuario, que facilita la navegación a través de la estructura jerárquica del modelo y la visualización de las variables de cada uno de sus componentes.

La metodología para el desarrollo de los laboratorios virtuales y las herramientas de ayuda ofrecidas por RMD facilitan desarrollar laboratorios virtuales de manera rápida, simple y con poco esfuerzo. El código de la simulación generado por RMD es eficiente computacionalmente y la calidad gráfica de la vista del laboratorio es buena. Actualmente RMD presenta la limitación de que no soporta diferenciación simbólica, ni reducción del índice de los sistemas de ecuaciones algebraico diferenciales.

4.9. LECTURAS RECOMENDADAS

Puede encontrarse más información acerca del formalismo OHM y el lenguaje de modelado Omola (actualmente en desuso) en (Andersson 1990, 1993, 1994).

En las tesis doctorales (Cellier 1979, Barton 1992, Urquía 2000) se abordan diferentes aspectos de la simulación de modelos dinámicos híbridos. El texto (Cellier & Kofman 2006) constituye una revisión muy didáctica sobre el tema.

El modelo de la fricción seca descrito en la Sección 4.5.7 está extraído del artículo (Elmqvist et al. 1993), en el cual se describen las capacidades para el modelado de sistemas híbridos de un lenguaje de modelado llamado Dymola, que era soportado a principios de los años 1990 por el entorno de modelado del mismo nombre y que actualmente se encuentra en desuso, ya que ha sido reemplazado por Modelica. No obstante, puede comprobarse que el tratamiento de los eventos descrito en ese artículo es esencialmente el mismo que tiene Dymola hoy en día y que las cláusulas de Dymola para describir modelos de estructura variable y ecuaciones instantáneas son esencialmente las mismas que tiene Modelica.

En el artículo (Mattsson et al. 2002) se describe la inicialización de modelos en Modelica.

En la Sección 4.7.2 se mencionó la librería en Modelica GAPILib. Se trata de una librería que facilita la estimación del valor de los parámetros de los modelos en Modelica, empleando para ello algoritmos genéticos. La librería GAPILib está descrita en (Rubio 2008).

El lector interesado en conocer más acerca de RMD puede consultar (Inihov et al. 2012). En (Senichenkov et al. 2013, Martín-Villalba et al. 2014) puede encontrarse una comparación entre las capacidades de RMD, y las de Modelica y sus entornos de modelado. El sistema de los cuatro tanques fue propuesto en (Johansson 2000). El lector interesado en casos de uso más complejos puede consultar (Tarasov et al. 2012), donde se describe la aplicación de RMD a la simulación del proceso de descarga de buques. El lenguaje de modelado Model Vision Language está basado en Unified Modeling Language (UML), que es descrito en (Rumbaugh et al. 2005).

La descripción del sistema de los cuatro tanques usado en la Sección 4.8.5 para ilustrar el uso de RMD se encuentra en el artículo (Johansson 2000).

4.10. EJERCICIOS DE AUTOCOMPROBACIÓN

Ejercicio 4.1

Describa en Modelica la curva característica de una bomba de líquido mostrada en la Figura 4.61, donde F^{SP} representa la consigna para el flujo de líquido y F el flujo real que proporciona la bomba. El criterio de signos para el flujo es: positivo para flujo entrante a la bomba y negativo para saliente de la bomba. La presión de la carga, p , es la presión del líquido en el punto de conexión de la bomba con el recipiente del cual la bomba extrae o en el cual introduce líquido.

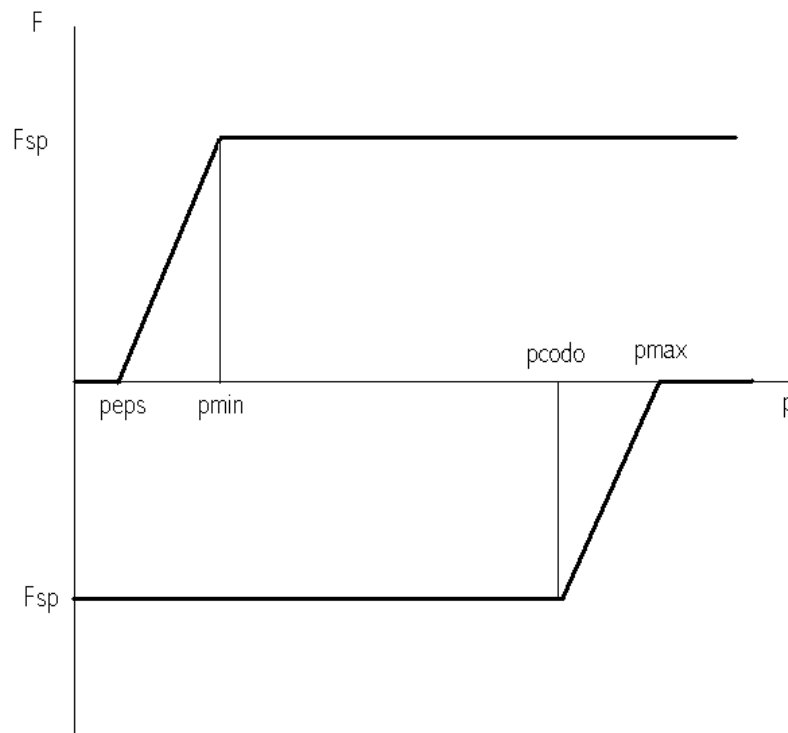


Figura 4.61: Curva característica de la bomba de líquido.

La curva característica está definida por los parámetros siguientes:

- p_{eps} Presión de la carga por debajo de la cual la bomba no puede extraer más líquido de la carga.
- p_{min} Presión de la carga por debajo de la cual el flujo que extrae la bomba de la carga cae linealmente.
- p_{codo} Presión de la carga por encima de la cual el flujo que introduce la bomba en la carga comienza a caer linealmente.
- p_{max} Presión de la carga por encima de la cual la bomba no tiene fuerza para introducir líquido en la carga.

Ejercicio 4.2

En la Sección 4.8.5 se modeló con RMD el *sistema de los cuatro tanques*, que es descrito en detalle en (Johansson 2000). El sistema está compuesto por cuatro tanques con un orificio en su base, una cubeta, dos bombas y dos válvulas de doble vía. El modelo, cuyo diagrama se muestra en Figura 4.55, está descrito por las Ecs. (4.164) – (4.167).

Se propone programar en Modelica una librería de modelos que permita componer el modelo del sistema de los cuatro tanques. A continuación, programe una clase que describa un controlador PI con una entrada y una salida. Empleando las clases de la planta y del controlador PI, modele la planta en lazo cerrado. Emplee una estrategia de control PI descentralizado.

Ejercicio 4.3

Describe en Modelica el modelo de un gas ideal contenido en un recipiente calefactado, que se muestra en la Figura 4.62. El recipiente tiene un orificio, a través del cual puede entrar y salir un flujo F de gas. La temperatura del gas entrante es T_{in} . El gas sale a la temperatura (T) a la que se encuentra en el recipiente. Las variables F , T_{in} y el flujo de calor Q son funciones conocidas del tiempo. El volumen del gas (V) y sus capacidades caloríficas (C_P, C_V) son parámetros del modelo. Obsérvese que el modelo tiene dos modos: *empty* (recipiente vacío) y *not empty* (recipiente con gas). En la Tabla 4.10 se indica el significado de las variables del modelo.

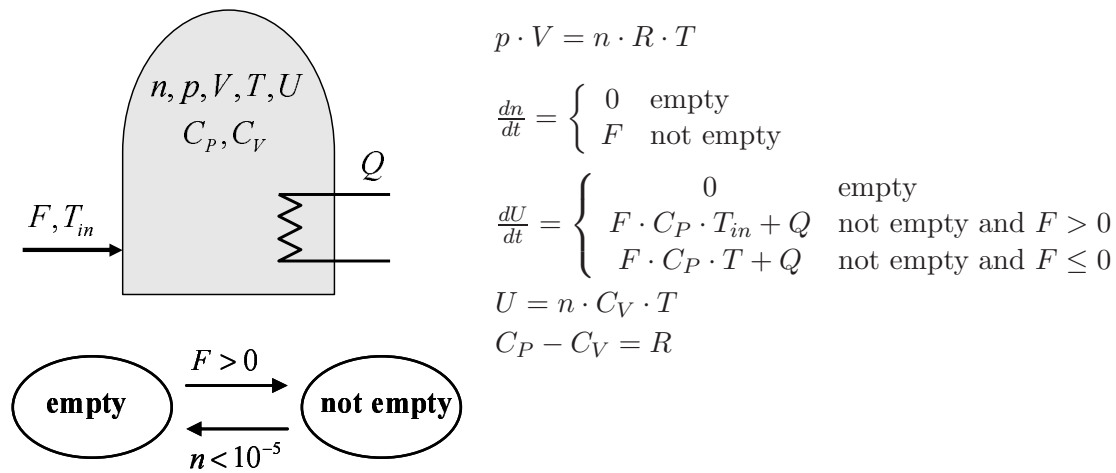


Figura 4.62: Modelo de un gas ideal en un recipiente calefactado.

Tabla 4.10: Variables del modelo mostrado en la Figura 4.62.

Variable	Significado	Unidades
n	Número de moles	mol
p	Presión del gas	Pa
V	Volumen del gas	m ³
T	Temperatura del gas	K
U	Energía interna	J
C_P	Capacidad calorífica a presión constante	J/(mol·K)
C_V	Capacidad calorífica a volumen constante	J/(mol·K)
F	Flujo molar de gas	mol/s
T_{in}	Temperatura del gas entrante	K
Q	Flujo de calor	W
R	Constante de los gases ideales	8.3145 J/(mol·K)

Ejercicio 4.4

En la Figura 4.63 se muestra un lazo de control compuesto por una planta SISO (*Single-Input and Single-Output*) y un controlador con histéresis. La señal r es la referencia. La señal de error e se calcula restando la salida de la planta, y , a la señal de referencia. La señal de error es la entrada al controlador. La salida del controlador, u , es la entrada a la planta.

La relación constitutiva del controlador, mostrada en la Figura 4.64, determina el valor de la salida del controlador, que es calculada a partir de la entrada al controlador y de la fase $\{s_1, s_2, s_3, s_4\}$ en la que encuentre el controlador. Obsérvese que la relación constitutiva del controlador depende de la posición de los puntos $\{a, b, c, d, e, f\}$. En la figura se muestra sólo la curva en el primer cuadrante, si bien la curva es simétrica en el tercer cuadrante: $u(e) = -u(-e)$.

El modelo de la planta se muestra en la Figura 4.63, donde $\mathbf{A}$ es una matriz $N \times N$ de componentes constantes; $\mathbf{B}$ y $\mathbf{C}$ son vectores de N componentes constantes; y el vector de estados $\mathbf{x}$ tiene N componentes. Dado que la planta es SISO, tanto la entrada (u) como la salida (y) son variables escalares. Programe el lazo de control en Modelica.

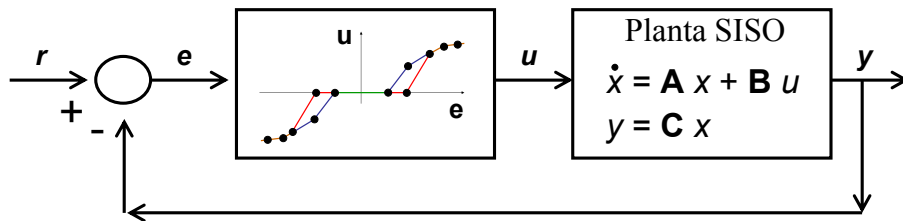


Figura 4.63: Lazo de control.

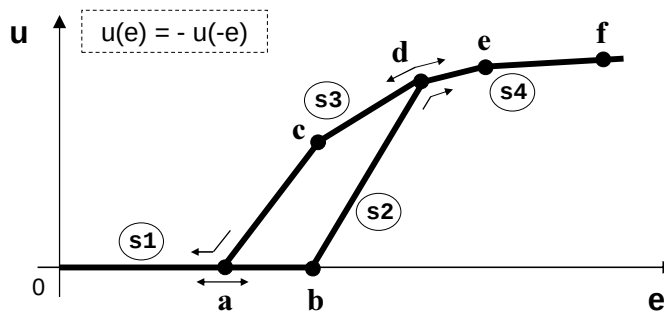


Figura 4.64: Relación constitutiva del controlador en el primer cuadrante.

Ejercicio 4.5

Se pretende simular la dinámica de vaciado de un tanque que contiene benceno, a través de una tubería de dos pulgadas de diámetro interno ($D = 5.08$ cm) y $L = 2$ m de longitud, que desemboca a un sumidero a presión atmosférica. La tubería está conectada horizontalmente, a la altura de la base del tanque.

El tanque tiene un volumen de 1 m^3 y su base tiene una superficie de 1 m^2 . El benceno se encuentra entorno a los 27°C . A esa temperatura su densidad vale aproximadamente $\rho = 875 \text{ kg/m}^3$ y su viscosidad dinámica $\mu = 56.5E-5 \text{ N}\cdot\text{s/m}^2$.

La tubería debe modelarse planteando el balance dinámico del momento lineal del líquido que circula por ella. Se trata de una tubería larga (su longitud es mucho mayor que su diámetro), recta, sin inclinación y de sección S uniforme. Se supone que mientras hay líquido en el depósito, la tubería está completamente llena.

Sobre el líquido contenido en la tubería actúan dos fuerzas en la dirección del flujo: la fuerza debida a la diferencia de presiones entre los extremos de la tubería (f_p) y la debida a la fricción del fluido en movimiento con las paredes internas de la tubería (f_F).

$$\frac{dP}{dt} = f_F + f_p \quad (4.168)$$

$$f_p = S \cdot \Delta p \quad (4.169)$$

$$f_F = \begin{cases} -S_w \cdot \frac{1}{2} \cdot \rho \cdot v^2 \cdot \kappa_{Fanning} & \text{si } P \geq 0 \\ S_w \cdot \frac{1}{2} \cdot \rho \cdot v^2 \cdot \kappa_{Fanning} & \text{si } P < 0 \end{cases} \quad (4.170)$$

donde S_w es la superficie interna de la conducción mojada por el líquido. El momento lineal del líquido contenido en la tubería (P) se relaciona con el flujo másico de líquido (F) y la longitud de la tubería (L) de la forma siguiente:

$$P = F \cdot L \quad (4.171)$$

La relación entre la velocidad (v), la densidad (ρ), el flujo másico del líquido (F) y la sección de la tubería (S) es:

$$F = S \cdot \rho \cdot v \quad (4.172)$$

El factor adimensional de Fanning, $\kappa_{Fanning}$, depende de la geometría de la sección de la tubería, de las características de su pared interna, de la velocidad, densidad y viscosidad del líquido, etc. El factor de fricción de Fanning para flujo laminar y turbulento, en tubos largos, lisos y circulares, suponiendo que el fluido tiene una temperatura homogénea, puede aproximarse de la forma siguiente (fórmula de Blasius):

$$\kappa_{Fanning} = \begin{cases} \frac{16}{Re} & \text{si } Re < 2100 & (\text{flujo laminar}) \\ \frac{0.0791}{Re^{1/4}} & \text{si } 2100 < Re < 10^5 & (\text{flujo turbulento}) \end{cases} \quad (4.173)$$

donde el número de Reynolds puede calcularse de la forma siguiente:

$$Re = \frac{D \cdot v \cdot \rho}{\mu} \quad (4.174)$$

Se ha conectado una fuente de líquido a la parte superior del tanque. Inicialmente el tanque se encuentra vacío. La fuente llena el tanque con un flujo constante de $0.01 \text{ m}^3/\text{s}$ durante diez minutos, transcurridos los cuales cesa su flujo. La simulación finaliza una vez el contenido del tanque se ha vaciado totalmente a través de la tubería.

4.11. SOLUCIONES DE LOS EJERCICIOS

Solución al Ejercicio 4.1

La curva característica puede representarse mediante las siguientes ecuaciones. Para flujo saliente de la fuente, $F^{SP} < 0$

$$F = \begin{cases} F^{SP} & \text{si } p < p_{codo} \\ \frac{F^{SP}}{p_{max} - p_{codo}} (p_{max} - p) & \text{si } p_{codo} \leq p < p_{max} \\ 0 & \text{si } p \geq p_{max} \end{cases} \quad (4.175)$$

y para flujo entrante $F^{SP} > 0$

$$F = \begin{cases} F^{SP} & \text{si } p > p_{min} \\ \frac{F^{SP}}{p_{min} - p_{eps}} (p - p_{eps}) & \text{si } p_{eps} < p \leq p_{min} \\ 0 & \text{si } p \leq p_{eps} \end{cases} \quad (4.176)$$

En el Código 4.16 – 4.18 pueden verse tres modelos diferentes de la curva característica de la bomba y un modelo, denominado *TestBomba*, que permite observar el comportamiento de los anteriores. En la Figura 4.65 se muestra la evolución de la presión de la carga y el flujo de la bomba para el primero de los tres modelos equivalentes de bomba.

```

model BombaLiq
  import SI = Modelica.SIunits;
  parameter SI.Pressure peps
    "Presión de la carga por debajo de la cual" +
    " la bomba no puede extraer más líquido de la carga";
  parameter SI.Pressure pmin
    "Presión de la carga por debajo de la cual el flujo que" +
    " extrae la bomba de la carga cae linealmente";
  parameter SI.Pressure pcodo
    "Presión de la carga por encima de la cual el flujo que" +
    " introduce la bomba en la carga comienza a caer linealmente";
  parameter SI.Pressure pmax
    "Presión de la carga por encima de la cual la bomba no" +
    " tiene fuerza para introducir líquido en la carga";
  SI.MassFlowRate Fsp "Consigna para el flujo entrante a la bomba";
  SI.MassFlowRate F "Flujo real entrante a la bomba";
  SI.Pressure p "Presión de la carga";
end BombaLiq;

```

Código 4.16: Curva característica de la bomba de líquido (1/3).

```

model BombaLiq1
  extends BombaLiq;
equation
  //Curva característica de la bomba
  if Fsp < 0 then
    F = if p < pcodo then
      Fsp
    elseif p >= pcodo and p < pmax then
      Fsp*(pmax-p)/(pmax-pcodo)
    else
      0;
  else
    F = if p > pmin then
      Fsp
    elseif p > peps and p <= pmin then
      Fsp*(p-peps)/(pmin-peps)
    else
      0;
  end if;
end BombaLiq1;

model BombaLiq2
  extends BombaLiq;
equation
  F = if (Fsp < 0 and p < pcodo) or (Fsp >= 0 and p > pmin) then
    Fsp
  elseif Fsp < 0 and p >= pcodo and p < pmax then
    Fsp*(pmax-p)/(pmax-pcodo)
  elseif Fsp >= 0 and p > peps and p <= pmin then
    Fsp*(p-peps)/(pmin-peps)
  else
    0;
end BombaLiq2;

model BombaLiq3
  extends BombaLiq;
equation
  F = if Fsp < 0 then
    if p < pcodo then
      Fsp
    elseif p >= pcodo and p < pmax then
      Fsp*(pmax-p)/(pmax-pcodo)
    else
      0
  else
    if p > pmin then
      Fsp
    elseif p > peps and p <= pmin then
      Fsp*(p-peps)/(pmin-peps)
    else
      0;
  end if;
end BombaLiq3;

```

Código 4.17: Curva característica de la bomba de líquido (2/3).

```

model TestBomba
  import SI = Modelica.SIunits;

  BombaLiq1 bombaL1(peps=1E5, pmin=3E5, pcodo=8E5, pmax=10E5);
  BombaLiq2 bombaL2(peps=1E5, pmin=3E5, pcodo=8E5, pmax=10E5);
  BombaLiq3 bombaL3(peps=1E5, pmin=3E5, pcodo=8E5, pmax=10E5);

  Boolean FlujoSaleBomba;
  SI.Pressure p(start=0, fixed=true) "Presión de la carga";

  parameter SI.MassFlowRate Fsp = 10 "Magnitud consigna para flujo";

equation

  FlujoSaleBomba = sin(time) > 0;

  der(p) = 4E5;
  when { FlujoSaleBomba, not FlujoSaleBomba } then
    reinit(p,0);
  end when;

  bombaL1.p = p;
  bombaL1.Fsp = if FlujoSaleBomba then -Fsp else Fsp;
  bombaL2.p = p;
  bombaL2.Fsp = if FlujoSaleBomba then -Fsp else Fsp;
  bombaL3.p = p;
  bombaL3.Fsp = if FlujoSaleBomba then -Fsp else Fsp;
end TestBomba;

```

Código 4.18: Curva característica de la bomba de líquido (3/3).

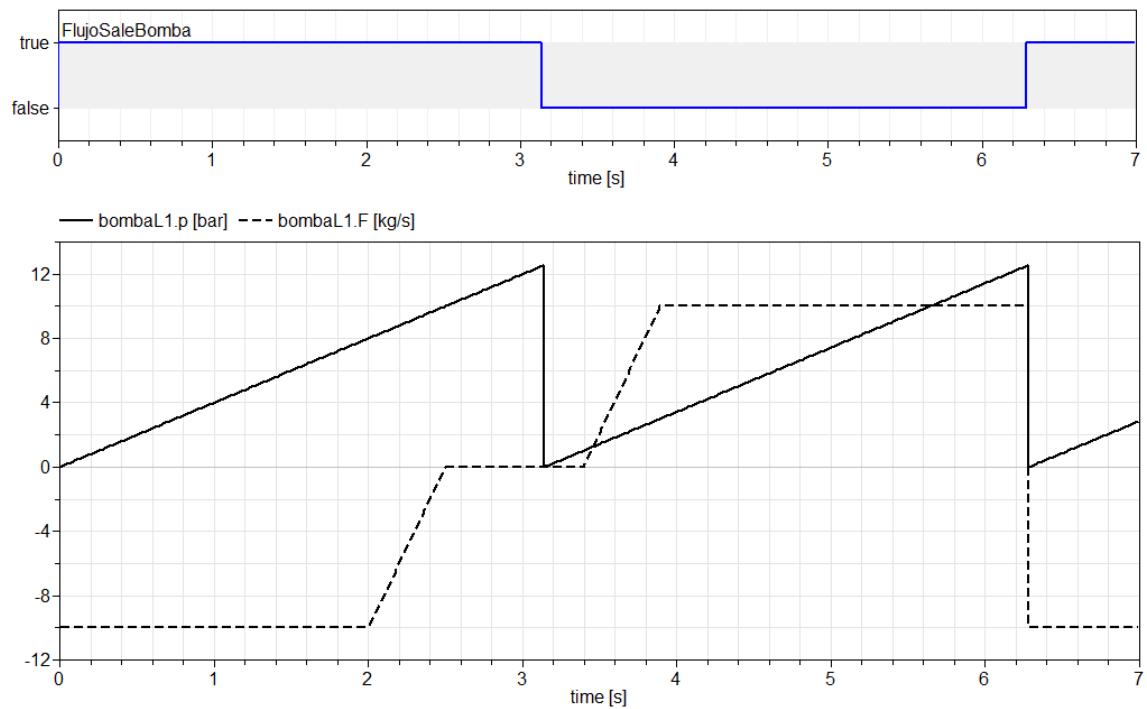


Figura 4.65: Simulación de la curva característica de la bomba de líquido.

Solución al Ejercicio 4.2

En la Figura 4.66 se muestra la arquitectura de la librería. Obsérvese que se han englobado la bomba y la válvula de doble vía en un solo modelo, al cual se ha llamado *BombaValvula2V*.

Es necesario modelar la cubeta, ya que de no hacerlo los conectores de la base de los tanques 1 y 2 quedarían sin conectar y, puesto que el caudal es una variable flow de esos conectores, el entorno de modelado impondría que el caudal en esos conectores fuese cero.

Los componentes de han modelado suponiendo que el caudal de la interfaz tiene signo positivo cuando entra al componente. Al describir el flujo proporcionado por la bomba se ha impuesto que éste sólo puede tener sentido saliente, es decir, que la bomba no puede extraer líquido del tanque al cual está conectada. Igualmente, se ha impuesto que el líquido sólo puede salir a través del orificio situado en la base del tanque, nunca podrá tener sentido entrante al tanque.

Con el fin de evitar que el listado de la librería sea excesivamente extenso, no se incluyen las anotaciones. El modelo en Modelica se muestra en el Código 4.19 y 4.20.

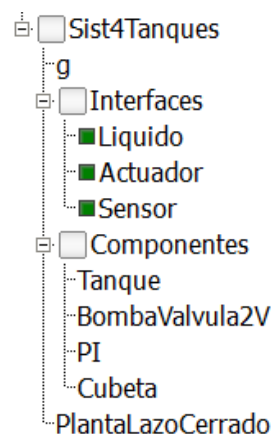


Figura 4.66: Estructura de la librería del sistema de los cuatro tanques.


```

package Sist4Tanques

import SI = Modelica.SIunits;
constant SI.Acceleration g= 9.81 "Aceleración gravitatoria";

package Interfaces

connector Liquido
  flow SI.VolumeFlowRate q "Caudal";
end Liquido;

connector Actuador
  SI.Voltage v "Voltaje aplicado a la bomba";
end Actuador;

connector Sensor
  SI.Height h "Altura de líquido medida en el tanque";
end Sensor;

end Interfaces;

package Componentes

model Tanque
  Interfaces.Liquido liqArriba, liqBase;
  Interfaces.Sensor sensorAltura;

  parameter SI.Height h0 "Altura inicial";
  parameter SI.Area A "Sección transversal";
  parameter SI.Area a "Sección orificio";

  SI.Volume V "Volumen de líquido";
  SI.Height h(start=h0, fixed=true) "Altura del líquido";

equation
  V = h*A;
  sensorAltura.h = h;
  der(V) = liqArriba.q + liqBase.q;
  liqBase.q = if noEvent(V>0) then -a*sqrt(2*g*h) else 0;
end Tanque;

model BombaValvula2V
  Interfaces.Liquido salidaLiq1;
  Interfaces.Liquido salidaLiq2;
  Interfaces.Actuador entradaV;
  parameter Real k(unit="m3/(s.V)");
  parameter Real gamma;
equation
  salidaLiq1.q = if noEvent(entradaV.v > 0)
    then -gamma*k*entradaV.v
    else 0;
  salidaLiq2.q = if noEvent(entradaV.v > 0)
    then -(1-gamma)*k*entradaV.v
    else 0;
end BombaValvula2V;

```

Código 4.19: Sistema de los cuatro tanques (1/2).

```

model PI
  Interfaces.Sensor  entrada;
  Interfaces.Actuador salida;

  Real error(unit="V");
  Real errorint(start=0, fixed=true, unit="V.s");

  parameter Real kc(unit="V/m") = 0.5E2 "Conversión a voltios";

  parameter SI.Height hSP "Consigna";
  parameter Real K (unit="") "Constante proporcional";
  parameter SI.Time Ti "Constante integral";

equation
  error = kc * ( hSP - entrada.h );
  der(errorint) = error;
  salida.v = K*error + errorint/Ti;
end PI;

model Cubeta
  Interfaces.Liquido liq;
end Cubeta;

end Componentes;

model PlantaLazoCerrado
  Componentes.Tanque tanque1(A=28E-4, a=0.071E-4, h0=12.4E-2);
  Componentes.Tanque tanque2(A=32E-4, a=0.057E-4, h0=12.7E-2);
  Componentes.Tanque tanque3(A=28E-4, a=0.071E-4, h0=1.80E-2);
  Componentes.Tanque tanque4(A=32E-4, a=0.057E-4, h0=1.40E-2);
  Componentes.BombaValvula2V bomba14(k=3.33E-6, gamma=0.7);
  Componentes.BombaValvula2V bomba23(k=3.35E-6, gamma=0.6);
  Componentes.PI pi1(Ti=30,K=3.0,hSP=12E-2);
  Componentes.PI pi2(Ti=40,K=2.7,hSP=12E-2);
  Componentes.Cubeta cubeta;
equation
  connect(bomba14.salidaLiq1,tanque1.liqArriba);
  connect(bomba14.salidaLiq2,tanque4.liqArriba);
  connect(bomba23.salidaLiq1,tanque2.liqArriba);
  connect(bomba23.salidaLiq2,tanque3.liqArriba);
  connect(tanque1.liqBase,cubeta.liq);
  connect(tanque2.liqBase,cubeta.liq);
  connect(tanque3.liqBase,tanque1.liqArriba);
  connect(tanque4.liqBase,tanque2.liqArriba);
  connect(bomba14.entradaV, pi1.salida);
  connect(bomba23.entradaV, pi2.salida);
  connect(pi1.entrada,tanque1.sensorAltura);
  connect(pi2.entrada,tanque2.sensorAltura);
end PlantaLazoCerrado;

end Sist4Tanques;

```

Código 4.20: Sistema de los cuatro tanques (2/2).

Solución al Ejercicio 4.3

El modelo del gas ideal se muestra en el Código 4.21, así como una subclase en la cual asigna valor a las variables F , T_{in} y Q . En la Figura 4.67 se muestra el resultado de la simulación.

```

model GasIdeal
  parameter Real V( unit="m3") = 1 "Volumen del gas";
  parameter Real R( unit="J/(mol.K)") = 8.3145 "Constante gases ideales";
  parameter Real Cp( unit="J/(mol.K)") = 5*R/2 "Gas ideal monoatómico";
  parameter Real Cv( unit="J/(mol.K)") = Cp - R "Ley de Mayer";
  parameter Real molEps = 1E-5 "Moles para recipiente vacío";
  Real n( unit="mol", start=20, fixed=true) "Número de moles";
  Real p( unit="N.m-2") "Presión";
  Real T( unit="K", start=300, fixed=true) "Temperatura";
  Real F( unit="mol.s-1") "Input flow";
  Real Tin( unit="K") "Input temperature";
  Real Q( unit="J.s-1") "Heat flow";
  Real U( unit="J") "Energía interna";
  Boolean empty "Fase del modelo";
equation
  p * V = n * R * T; // Ecuación de estado
  der(n) = if empty then 0 else F; // Balance de moles
  // Balance de energía
  der(U) = if empty then 0 else if F>0 then F*Cp*Tin+Q else F*Cp*T+Q;
  U = n * Cv * T; // Energía interna
  when F > 0 and pre(empty) or n < molEps and not pre(empty) then
    empty = not pre(empty);
  end when;
initial equation
  empty = ( n < molEps );
end GasIdeal;

model Test
  extends GasIdeal;
equation
  Tin = if time < 10 then 275 else 350;
  F = if time < 10 then 2
    else if time < 30 then -4
    else if time < 50 then 3 else 0;
  Q = if time > 60 and time < 80 then 1E3 else 0;
end Test;

```

Código 4.21: Modelo de un gas perfecto en un recipiente calefactado.

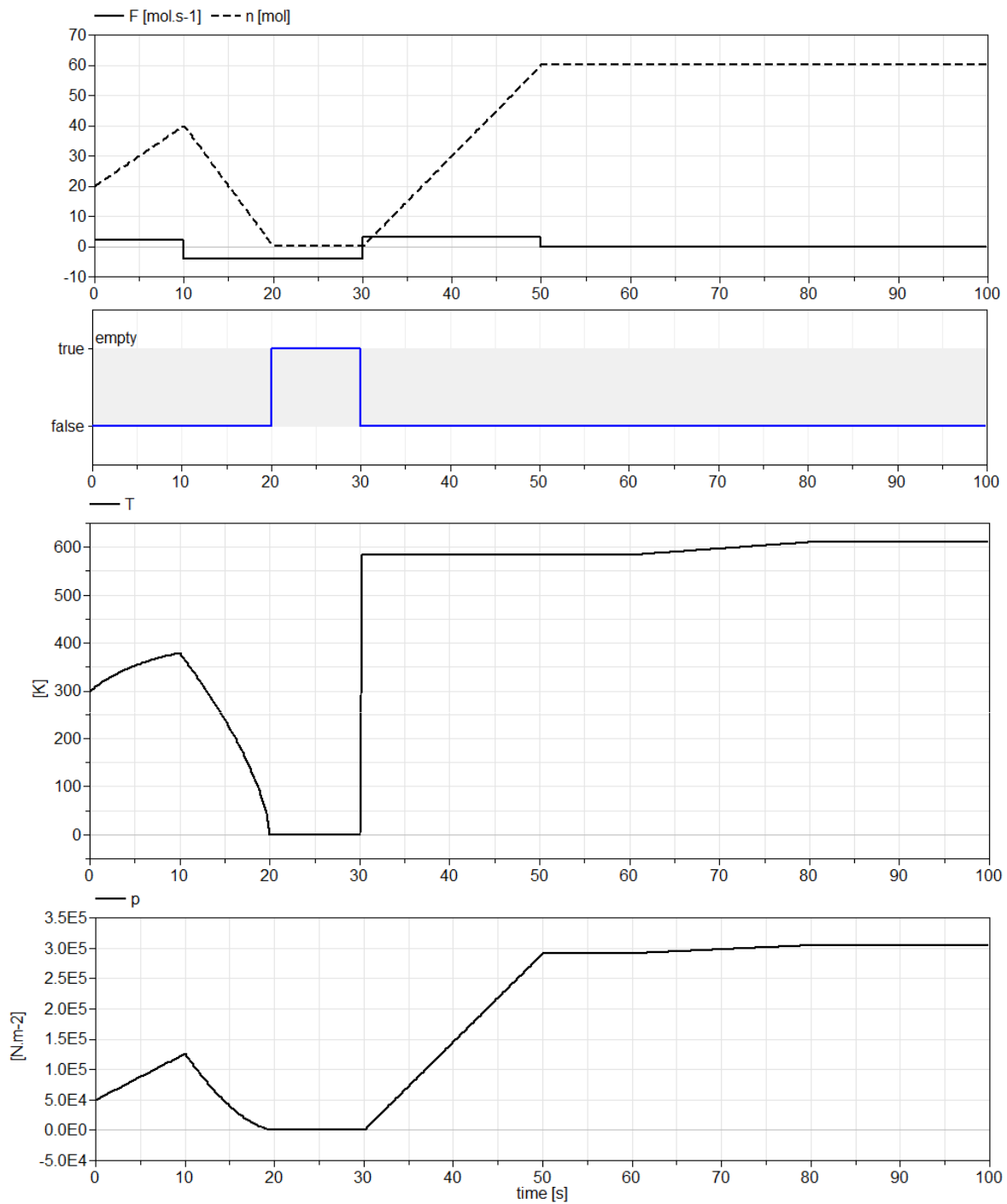


Figura 4.67: Resultado de simular el Código 4.21.

Solución al Ejercicio 4.4

Los modelos del controlador, la planta y el lazo de control se muestran en el Código 4.22 y 4.23. Se han incluido dentro de un paquete llamado *ControlHisteresis*, como puede verse en la Figura 4.68.

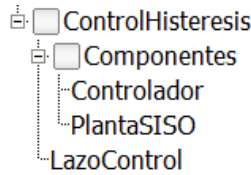


Figura 4.68: Estructura del paquete *ControlHisteresis*.

El controlador y la planta han sido descritos como clases block. La clase block es similar a la clase model, con la diferencia de que en la clase block todas las variables de la interfaz deben tener la causalidad computacional explícitamente declarada. Es decir, cada variable de la interfaz debe ser declarada como entrada (input) o como salida (output).

El modelo de la planta está compuesto por las Ecs. (4.177) y (4.178).

$$\dot{\mathbf{x}} = \mathbf{A} \cdot \mathbf{x} + \mathbf{B} \cdot u \quad (4.177)$$

$$y = \mathbf{C} \cdot \mathbf{x} \quad (4.178)$$

El modelo tiene una entrada (u), una salida (y) y un vector de variables internas ($\mathbf{x}$). Como puede verse en el código de la clase *PlantaSISO*, las variables de la interfaz son declaradas como input o output. El vector $\mathbf{x}$ se declara dentro de la sección `protected`. Recuerdese que las variables declaradas dentro de la sección `protected` no pueden ser accedidas desde fuera del modelo. De esta manera Modelica soporta la ocultación de la información.

La clase *PlantaSISO* está descrita de tal manera que hasta que no se asigna valor a la matriz $\mathbf{A}$ no queda definido el número de componentes de la matriz y vectores de la clase, así como el número de ecuaciones a los que da lugar la ecuación vectorial $\dot{\mathbf{x}} = \mathbf{A} \cdot \mathbf{x} + \mathbf{B} \cdot u$.

En la Figura 4.69 se muestra el resultado de la simulación. En la gráfica superior puede verse cómo la salida de la planta va siguiendo la referencia. En la gráfica inferior se ha representado la salida del controlador frente a su entrada.

```

package ControlHisteresis

package Componentes

block Controlador
    input    Real    e;
    output   Real    u;
    // Parámetros de la trayectoria
    parameter Real    xA,
                      xB,
                      xC, yC,
                      xD, yD,
                      xE, yE,
                      xF, yF;
    constant Real      eps = 1e-10;
protected
    Real    traject1 "0 - a",
            traject2 "a - b - d",
            traject3 "a - c - d",
            traject4 "d - e - f";
    Boolean  s1 (start=true) "traject1",
            s2 (start=false) "traject2",
            s3 (start=false) "traject3",
            s4 (start=false) "traject4";
    Real     absE,
            absU;
equation

    // -----
    // Definición de las trayectorias
    // -----
    absE = abs(e);
    traject1 = 0;
    traject2 = if absE < xB
                then 0
                else ( yD * absE - xB * yD ) / ( abs( xD - xB ) + eps );
    traject3 = if absE < xC
                then ( yC * absE - xA * yC ) / ( abs( xC - xA ) + eps )
                else ( ( yD - yC ) * absE + yC * xD - xC * yD ) / ( abs(xD - xC) + eps );
    traject4 = if absE < xE
                then ( ( yE - yD ) * absE + yD * xE - xD * yE ) / ( abs(xE - xD) + eps )
                else ( ( yF - yE ) * absE + yE * xF - xE * yF ) / ( abs(xF - xE) + eps );

    // -----
    // Ecuación característica
    // -----
    absU = if s1
            then traject1
            else if s2
                then traject2
                else if s3
                    then traject3
                    else traject4;
    u = if e > 0 then absU else -absU;

```

Código 4.22: Planta SISO y controlador con histéresis (1/2).

```

// -----
// Transición entre los modos
// -----
s1 = pre(s3) and absE < xA or
    pre(s2) and absE < xA or
    pre(s1) and not absE > xA;

s2 = pre(s1) and absE > xA or
    pre(s2) and not ( absE < xA or absE > xD );

s3 = pre(s4) and absE < xD or
    pre(s3) and not ( absE < xA or absE > xD );

s4 = pre(s2) and absE > xD or
    pre(s3) and absE > xD or
    pre(s4) and not absE < xD;
end Controlador;

block PlantaSISO
    input Real u;
    output Real y;
    parameter Real A[:,:],
                  B[size(A,1)],
                  C[size(A,2)];
protected
    Real x[size(A,2)];
equation
    assert( size(A,1) == size(A,2), "La matriz A debe ser cuadrada");
    der(x) = A * x + B * u;
    y = C * x;
end PlantaSISO;

end Componentes;

model LazoControl
    Componentes.Controlador NL(xA = 1,
                              xB = 2,
                              xC = 1.5, yC = 1,
                              xD = 2.5, yD = 2.1,
                              xE = 3, yE = 2.5,
                              xF = 4, yF = 5);
    Componentes.PlantaSISO Planta ( A = [ 0, 1; 0, -1 ],
                                    B = { 0, 1 },
                                    C = { 1, 0 } );

    Real referencia;
equation
    referencia = 15*sin(time/10) + sin(time);
    NL.u = Planta.u;
    NL.e = referencia - Planta.y;
end LazoControl;

end ControlHisteresis;

```

Código 4.23: Planta SISO y controlador con histéresis (2/2).

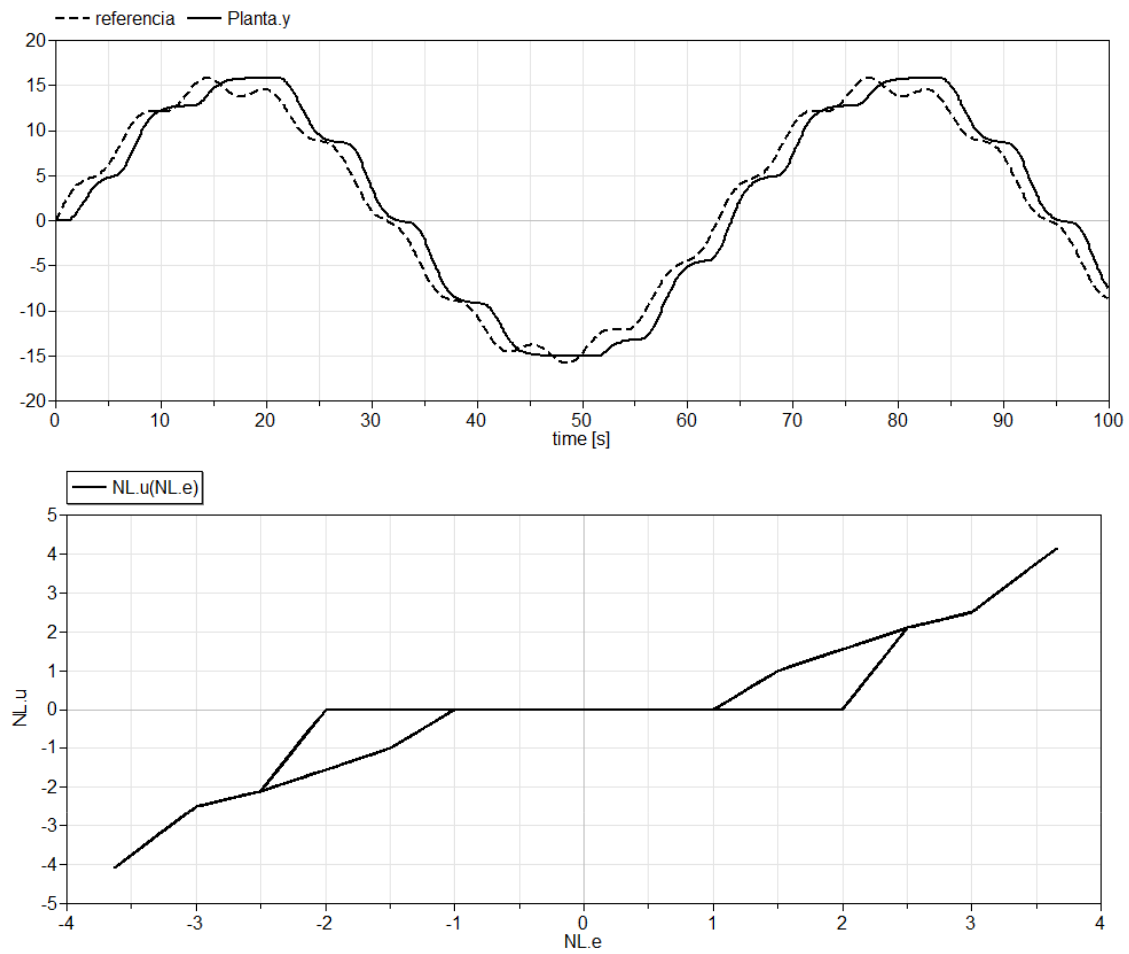


Figura 4.69: Simulación del Código 4.22 y 4.23.

Solución al Ejercicio 4.5

En la Figura 4.70 se muestra la estructura de la librería desarrollada. La descripción en Modelica se muestra en el Código 4.24 – 4.27. En la Figura 4.71 puede verse la evolución de la masa de líquido almacenada en el tanque, y del momento lineal, el número de Reynolds y el régimen del líquido que circula por la tubería.

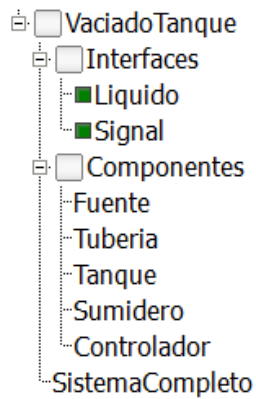


Figura 4.70: Estructura de la librería *VaciadoTanque*.

```

package VaciadoTanque
import SI = Modelica.SIunits;

package Interfaces
connector Liquido
  SI.Pressure p "Presión";
  flow SI.MassFlowRate Fm "Flujo másico";
end Liquido;

connector Signal
  Real s "Consigna";
end Signal;
end Interfaces;

package Componentes

model Fuente
  Interfaces.Liquido liq;
  Interfaces.Signal sig;
equation
  liq.Fm = -sig.s;
end Fuente;
  
```

Código 4.24: Vaciado de un tanque a través de una tubería (1/4).

```

model Tuberia
  constant Real PI = 2*Modelica.Math.asin(1.0);

  Interfaces.Liquido liqI, liq0;

  parameter SI.Length L "Longitud";
  parameter SI.Diameter D "Diametro";
  parameter SI.Area S = PI*D^2/4 "Seccion tuberia";
  parameter SI.Area Sw = PI*D*L "Superficie mojada";

  parameter SI.Density rho "Densidad del líquido";
  parameter SI.DynamicViscosity mu "Viscosidad del líquido";

  parameter Real ReC(unit="") = 2100 "Numero de Reynolds critico";
  SI.Force fP "Fuerza debida a diferencia de presiones";
  SI.Force fF "Fuerza de friccion";
  SI.Momentum P(start=0, fixed=true) "Momento lineal del líquido";
  SI.Velocity v "Velocidad del líquido";

  Real Re(unit="") "Numero de Reynolds";
  Real Kfanning "Factor de Fanning";
  Boolean laminar "Flujo laminar";

equation
  der(P) = fF + fP;
  fP = S * ( liqI.p - liq0.p );
  fF = if noEvent(P<0) then 0.5*Sw*rho*v^2*Kfanning
      else -0.5*Sw*rho*v^2*Kfanning;

  P = liqI.Fm * L;
  liqI.Fm = -liq0.Fm;
  liqI.Fm = S*rho*v;
  laminar = Re < ReC;
  Kfanning = if noEvent(Re<ReC) then 16/Re else
      0.0791/Re^0.25;

  Re = if noEvent( v>0 )
      then D*v*rho/mu
      else if noEvent( v<0 )
      then -D*v*rho/mu
      else 1;

  when fP <= 0 then
    reinit(P,0);
  end when;
end Tuberia;

```

Código 4.25: Vaciado de un tanque a través de una tubería (2/4).

```

model Tanque
  Interfaces.Liquido liqBase, liqSuperior;

  constant SI.Acceleration g= 9.81 "Aceleración gravitatoria";

  SI.Mass m(start=0, fixed=true);
  SI.Volume V "Volumen de líquido";

  parameter SI.Density rho "Densidad";
  parameter SI.Area S "Sección transversal";
  parameter SI.Volume Vmax "Volumen máximo";

equation
  der(m) = liqBase.Fm + liqSuperior.Fm;
  V      = m/rho;
  liqSuperior.p = 0;
  liqBase.p = if m>0 then m*g/S else 0;
  when m<0 then
    reinit(m,0);
  end when;
  assert(V<=Vmax, "Excedido volumen maximo");
end Tanque;

model Sumidero
  Interfaces.Liquido liq;
equation
  liq.p = 0;
end Sumidero;

model Controlador
  Interfaces.Signal signal;
  parameter SI.Time t1;
  parameter SI.MassFlowRate FmSP1, FmSP2;
equation
  signal.s = if time<t1 then FmSP1 else FmSP2;
end Controlador;

end Componentes;

```

Código 4.26: Vaciado de un tanque a través de una tubería (3/4).

```

model SistemaCompleto
  // Propiedades del benceno
  parameter SI.Density rho = 875 "Densidad";
  parameter SI.DynamicViscosity mu = 56.5e-5 "Viscosidad";
  // Tubería
  parameter SI.Length L = 2 "Longitud";
  parameter SI.Diameter D = 5.08E-2 "Diametro";
  // Tanque
  parameter SI.Area S = 1 "Sección transversal";
  parameter SI.Volume Vmax = 1 "Volumen máximo";

  Componentes.Tanque tanque(S=S, Vmax=Vmax, rho=rho);
  Componentes.Fuente fuente;
  Componentes.Tuberia tuberia(L=L, D=D, rho=rho, mu=mu);
  Componentes.Sumidero sumidero;
  Componentes.Controlador control(t1=600, FmSP1=0.01*rho, FmSP2=0);
equation
  connect(control.signal,fuente.sig);
  connect(tanque.liqSuperior,fuente.liq);
  connect(tanque.liqBase,tuberia.liqI);
  connect(sumidero.liq,tuberia.liqO);
end SistemaCompleto;

end VaciadoTanque;

```

Código 4.27: Vaciado de un tanque a través de una tubería (4/4).

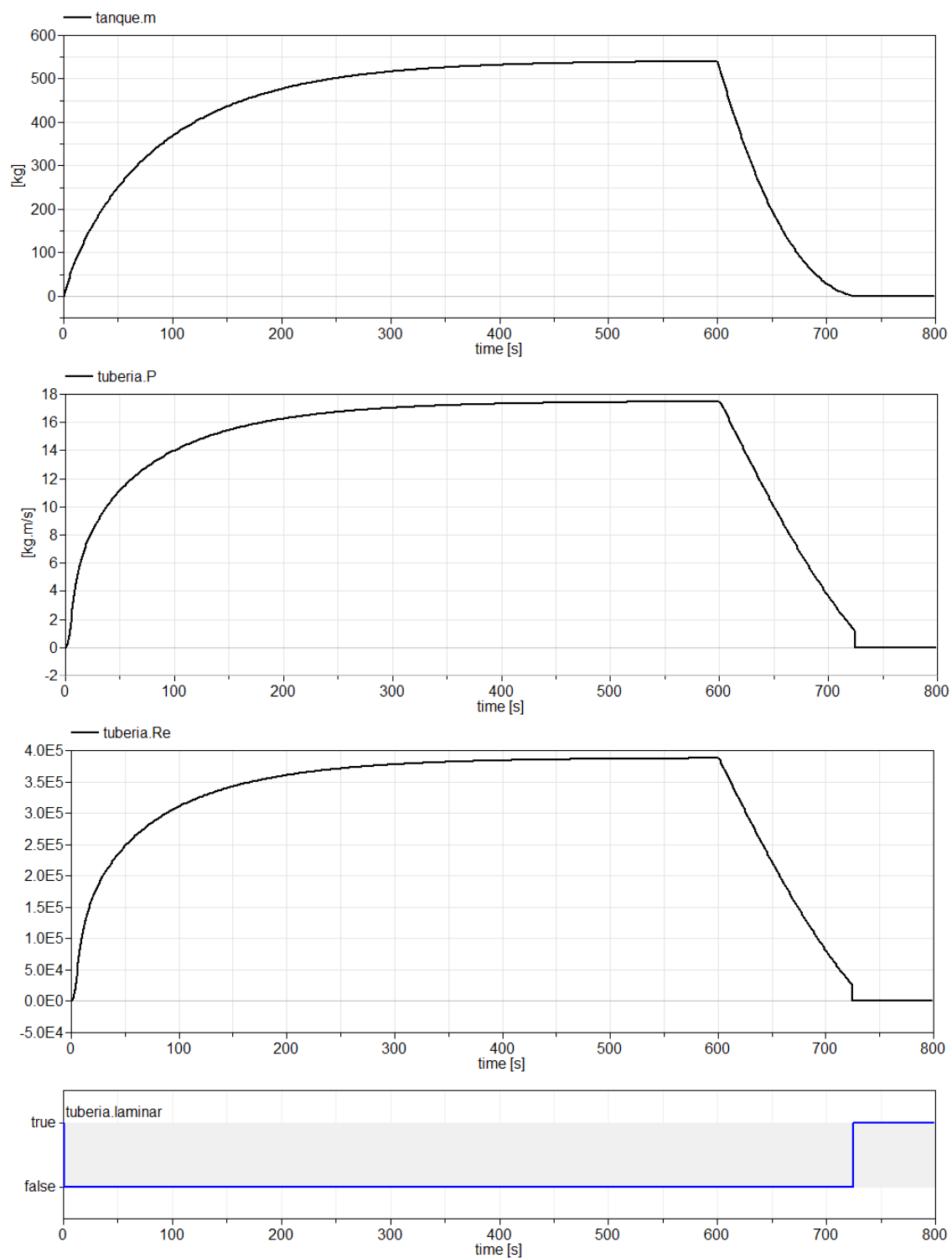


Figura 4.71: Resultado de la simulación del Código 4.24 – 4.27.

TEMA 5

MODELOS EN DERIVADAS PARCIALES

- 5.1. Introducción
- 5.2. Operadores diferenciales
- 5.3. Tipos de PDE lineales de segundo orden
- 5.4. Condiciones iniciales y de frontera
- 5.5. Métodos de resolución
- 5.6. Entornos de simulación de PDE
- 5.7. FlexPDE
- 5.8. Ecuación de transferencia de calor
- 5.9. Ecuación de ondas
- 5.10. Ecuación de Laplace
- 5.11. Lecturas recomendadas
- 5.12. Ejercicios de autocomprobación
- 5.13. Soluciones de los ejercicios

OBJETIVOS DOCENTES

Una vez estudiado el contenido del tema y realizados los ejercicios prácticos, debería saber:

- Discutir las características principales de los campos escalares y vectoriales.
- Discutir las características básicas de los operadores diferenciales gradiente, divergencia, rotacional y laplaciano. Ser capaz de aplicar estos operadores en coordenadas cartesianas.
- Saber clasificar las PDE lineales de segundo orden en hiperbólicas, parabólicas y elípticas.
- Discutir qué son las condiciones iniciales y de frontera.
- Discutir las características básicas del método de diferencias finitas, y del método de elementos finitos.
- Emplear el entorno de simulación FlexPDE para obtener la solución de sistemas de PDE de una y de dos dimensiones, con condiciones iniciales y condiciones de contorno. En particular, aplicar el entorno FlexPDE para resolver ecuaciones de transferencia de calor, de ondas y de Poisson.
- Saber derivar la ecuación de transferencia de calor en una dimensión, la ecuación de ondas homogéneas en una dimensión y la ecuación de Poisson a partir de principios físicos.

5.1. INTRODUCCIÓN

Las ecuaciones diferenciales parciales (PDE) juegan un papel importante en muchas disciplinas, incluyendo la ingeniería, física, economía y biología. En la Sección 1.4.8 ya se introdujeron algunos conceptos sobre las ecuaciones diferenciales, como algunas de sus características básicas, las diferencias entre ecuaciones diferenciales ordinarias y parciales, y el método de las líneas (MOL) para resolver PDE.

Este capítulo supone una continuación de lo ya explicado en dicha sección. No se pretende que el alumno sea un experto en PDE, ya que es un tema que abarcaría una asignatura en sí misma por su complejidad y extensión, pero sí que alcance dos objetivos básicos: entender los fundamentos de los algoritmos numéricos que emplean los entornos de simulación de PDE y aprender el manejo de estos entornos para editar, simular y realizar representaciones gráficas de modelos descritos mediante PDE.

El capítulo comienza con una descripción de los operadores diferenciales, que posteriormente se emplearán para describir PDE, se describen algunas de las ecuaciones diferenciales de la física y se derivan algunas ecuaciones sencillas partiendo de principios físicos. Finalmente, se explica los principios del manejo del entorno FlexPDE y se emplea este entorno para simular PDE y obtener representaciones gráficas.

5.2. OPERADORES DIFERENCIALES

Un operador diferencial es un operador lineal definido como una función del operador de diferenciación. Cuando se consideran derivadas respecto a variables diferentes, las derivadas parciales pueden escribirse como:

$$\frac{\partial}{\partial x_i} = \partial_{x_i} \quad (5.1)$$

$$\frac{\partial^n}{\partial x_1 \cdots \partial x_n} = \partial_{x_1, \dots, x_n}^n \quad (5.2)$$

Antes de describir algunos de los operadores diferenciales más frecuentemente usados, vamos a definir los conceptos de campo escalar y campo vectorial.

Un **campo escalar** representa la distribución espacial de una magnitud escalar, asociando un valor a cada punto del espacio. Esto es, asocia a cada punto de

un espacio vectorial un número escalar $f(x_1, x_2, \dots, x_n)$. Matemáticamente esto se expresa como: $f : R^n \rightarrow R$. Por ejemplo, se emplean campos escalares para indicar la distribución de temperaturas o de presiones en el espacio. Los campos escalares pueden depender además del tiempo, en cuyo caso el valor del campo escalar depende del punto del espacio y del tiempo, es decir, $f(x_1, \dots, x_n, t)$.

Un **campo vectorial** representa la distribución espacial de una magnitud vectorial. Es decir, asocia a cada punto de un espacio vectorial un vector $f : R^n \rightarrow R^m$. Por ejemplo, el campo vectorial se emplea para representar la velocidad de un fluido en el espacio, o la intensidad y dirección de fuerzas como la gravitatoria o la electromagnética. Como en el caso de los campos escalares, los campos vectoriales dependen a menudo del punto del espacio y, además, del tiempo.

A continuación se describen los operadores diferenciales gradiente, divergencia, rotacional y laplaciano.

5.2.1. Gradiente

El gradiente de un campo escalar $f(\vec{r})$ se define como el único campo vectorial cuyo producto escalar con cualquier vector $\vec{v}$ en cada punto $\vec{r}$ es la derivada direccional de f a lo largo de $\vec{v}$. Esta última se representa $D_{\vec{v}}(f)$.

$$D_{\vec{v}}(f) \equiv \vec{\nabla} f(\vec{r}) \cdot \vec{v} \quad (5.3)$$

Son comunes las siguientes notaciones para expresar el gradiente de f : ∇f , $\text{grad}(f)$ o $\vec{\nabla} f$.

Geométricamente el gradiente se interpreta como un vector que se encuentra normal (perpendicular) a la curva de nivel que se está estudiando. Por ejemplo, en el caso de considerar la altura H en todos los puntos (x, y) de una montaña, el gradiente de H en un punto estará en la dirección para la que hay mayor grado de inclinación. La magnitud del gradiente en un punto nos medirá el grado de inclinación del terreno en dicho punto.

El gradiente verifica las siguientes propiedades:

- Es ortogonal a las superficies equiescalares (definidas por $f = \text{constante}$).
- Apunta en la dirección en que la derivada direccional es máxima.
- Su norma es igual a la derivada direccional máxima.

- Se anula en los puntos estacionarios (máximos, mínimos y puntos de silla).
- El campo formado por el gradiente es irrotacional. Esto es: $\nabla \times (\nabla f) \equiv 0$

El gradiente en coordenadas cartesianas se puede calcular empleando la siguiente expresión:

$$\nabla f(\vec{r}) = \frac{\partial f}{\partial x} \cdot \vec{i} + \frac{\partial f}{\partial y} \cdot \vec{j} + \frac{\partial f}{\partial z} \cdot \vec{k} \quad (5.4)$$

donde $\vec{i}$, $\vec{j}$, $\vec{k}$ son los vectores unitarios (es decir, de módulo unidad) en la dirección de los ejes coordenados cartesianos X, Y y Z, respectivamente.

5.2.2. Divergencia

La divergencia de una campo vectorial en un punto es un campo escalar, y se define como el flujo del campo vectorial por unidad de volumen conforme el volumen alrededor del punto tiende a cero.

$$\text{div}(\vec{F}) = \nabla \cdot \vec{F} \equiv \lim_{\Delta V \rightarrow 0} \frac{1}{\Delta V} \oint_S \vec{F} \cdot d\vec{S} \quad (5.5)$$

donde S es una superficie cerrada que encierra a ΔV y que se reduce a un punto en el límite, y el símbolo ∇ representa el operador nabla.

La divergencia de un campo vectorial mide la diferencia entre el flujo entrante y el flujo saliente de un campo vectorial sobre la superficie que rodea un volumen. La divergencia de un campo es un valor escalar con signo.

La divergencia de un campo en un punto es distinta de cero cuando existen fuentes puntuales en dicho punto. Por ejemplo, en el caso del campo eléctrico las fuentes puntuales son las cargas eléctricas y la divergencia es distinta de cero en un punto cuando existe en él una carga eléctrica. La divergencia es positiva en un punto cuando la carga eléctrica en ese punto es positiva. En ese caso, existen líneas de campo que salen del punto. Por otro lado, la divergencia es negativa cuando en ese punto la carga eléctrica es negativa y entonces las líneas de campo entran en el punto. Cuando no existen cargas eléctricas en un punto, la divergencia del campo eléctrico es cero en ese punto.

La divergencia del campo $\vec{F} = F_x \cdot \vec{i} + F_y \cdot \vec{j} + F_z \cdot \vec{k}$, en el sistema de coordenadas cartesiano, tiene la expresión siguiente:

$$\nabla \cdot \vec{F} = \frac{\partial F_x}{\partial x} + \frac{\partial F_y}{\partial y} + \frac{\partial F_z}{\partial z} \quad (5.6)$$

5.2.3. Rotacional

El rotacional de un campo vectorial $\vec{F}$ es un vector que se define en términos de su proyección en varias curvas situadas en planos perpendiculares. Si $\vec{n}$ es cualquier vector unitario, la proyección del rotacional de F en $\vec{n}$ se define del modo siguiente:

$$(\nabla \times \vec{F}) \cdot \vec{n} \equiv \lim_{\Delta S_c \rightarrow 0} \frac{\oint_C \vec{F} \cdot d\vec{l}}{\Delta S_c} \quad (5.7)$$

donde $\vec{n}$ es la normal al área S_C encerrada por la línea elemental C . El resultado de este límite no es el rotacional completo, sino sólo su componente según la dirección normal a S_C y orientada según la regla de la mano derecha. Para obtener el rotacional completo deberán calcularse tres límites, considerando tres curvas situadas en planos perpendiculares.

En coordenadas cartesianas, podemos calcular el rotacional empleando la expresión siguiente:

$$\begin{aligned} \text{rot}(\vec{F}) = \nabla \times \vec{F} &= \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ \frac{\partial}{\partial x} & \frac{\partial}{\partial y} & \frac{\partial}{\partial z} \\ F_x & F_y & F_z \end{vmatrix} \\ &= \left(\frac{\partial F_z}{\partial y} - \frac{\partial F_y}{\partial z} \right) \cdot \vec{i} + \left(\frac{\partial F_x}{\partial z} - \frac{\partial F_z}{\partial x} \right) \cdot \vec{j} + \left(\frac{\partial F_y}{\partial x} - \frac{\partial F_x}{\partial y} \right) \cdot \vec{k} \end{aligned} \quad (5.8)$$

El rotacional muestra la tendencia de un campo vectorial a inducir rotación alrededor de un punto.

Supongamos que $\vec{F}$ representa la velocidad de un campo rotando, entonces $\text{rot}(\vec{F})$ tiene la dirección del eje de rotación y su módulo es el doble de la velocidad angular de la rotación.

Si el rotacional de un campo vectorial es cero, decimos que el campo vectorial es **irrotacional**.

5.2.4. Laplaciano

El laplaciano es la divergencia del gradiente de una función escalar. Para designar el laplaciano se emplean los símbolos $\nabla \cdot \nabla$, ∇^2 o Δ .

En coordenadas cartesianas tiene la expresión siguiente:

$$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2} \quad (5.9)$$

El laplaciano aparece en ecuaciones diferenciales que describen muchos fenómenos físicos, tales como los potenciales eléctrico y gravitatorio, la ecuación de difusión para el flujo de calor y de líquido, la propagación de ondas y la mecánica cuántica. El laplaciano es la divergencia de un campo de gradientes y su sentido físico es el de una divergencia de un tipo especial de campo vectorial: un campo cuyo rotacional es nulo.

5.3. TIPOS DE PDE LINEALES DE SEGUNDO ORDEN

Consideremos la forma genérica de una PDE lineal de segundo orden, de 2 variables, con coeficientes constantes:

$$a \cdot \frac{\partial^2 u}{\partial x^2} + b \cdot \frac{\partial^2 u}{\partial x \partial y} + c \cdot \frac{\partial^2 u}{\partial y^2} + d \cdot \frac{\partial u}{\partial x} + e \cdot \frac{\partial u}{\partial y} + f \cdot u = g(x, y) \quad (5.10)$$

Para que dicha ecuación sea de segundo orden, los tres coeficientes a , b y c no pueden ser todos iguales a cero.

Las soluciones de las ecuaciones de la forma (5.10) tienen diferentes propiedades, según el valor de los coeficientes de mayor orden a , b y c . Clasificamos estas ecuaciones en las siguientes categorías, según el valor del discriminante $b^2 - 4 \cdot a \cdot c$.

- **Ecuaciones hiperbólicas.** Para estas ecuaciones el discriminante satisface: $b^2 - 4 \cdot a \cdot c > 0$. Entre las ecuaciones de este tipo destaca la ecuación de ondas.

A continuación se muestra la ecuación de ondas en una dimensión, que modela las vibraciones transversales de una cuerda elástica o las vibraciones longitudinales en una barra de metal.

$$\frac{\partial^2 u}{\partial t^2} - c^2 \cdot \frac{\partial^2 u}{\partial x^2} = f(x, t) \quad (5.11)$$

En este caso, $a = 1$, $b = 0$, $c = -c^2$. Por tanto, el discriminante satisface: $4 \cdot c^2 > 0$.

- **Ecuaciones parabólicas.** Para estas ecuaciones el discriminante satisface: $b^2 - 4 \cdot a \cdot c = 0$. Ejemplo de ecuaciones de este tipo son las obtenidas en problemas de difusión del calor y de reacciones químicas.

La ecuación del calor en una dimensión, para una conductividad constante κ , es mostrada a continuación.

$$\frac{\partial u}{\partial t} = \kappa \cdot \frac{\partial^2 u}{\partial x^2} \quad (5.12)$$

En este caso, $a = -\kappa$, $b = 0$, $c = 0$. Por tanto, el discriminante vale cero.

- **Ecuaciones elípticas.** Para estas ecuaciones el discriminante satisface: $b^2 - 4 \cdot a \cdot c < 0$. Entre las ecuaciones de este tipo destacan las ecuaciones de Laplace y Poisson.

La ecuación de Laplace es la siguiente:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0; \quad (5.13)$$

En este caso, $a = 1$, $b = 0$, $c = 1$. Por tanto, el discriminante es -4.

En general, las ecuaciones elípticas describen procesos en equilibrio. Por otro lado, las ecuaciones hiperbólicas y parabólicas modelan procesos que evolucionan en el tiempo.

5.4. CONDICIONES INICIALES Y DE FRONTERA

Supongamos que deseamos encontrar la solución u a una PDE que tiene una dependencia temporal y que está definida dentro de una región cerrada como la mostrada en la Figura 5.1. Para encontrar la solución es necesario especificar las condiciones iniciales y las condiciones de frontera, llamadas también condiciones de contorno.

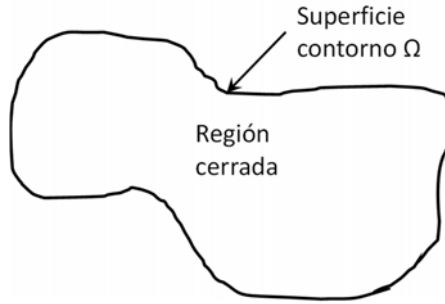


Figura 5.1: Región cerrada donde se desea obtener la solución de la PDE.

- Es necesario especificar las **condiciones iniciales** cuando la variable tiene dependencia temporal. Las condiciones iniciales indican la expresión de dicha variable en el instante inicial.
- Las **condiciones de frontera** especifican las condiciones que ha de verificar la solución en la frontera del dominio, que en este caso es el contorno Ω . Existen fundamentalmente los siguientes tipos de condiciones de frontera:

1. Condición de frontera de **Dirichlet** o de **primer tipo**. En este caso, se especifican los valores de la solución en la frontera del dominio Ω (véase la Figura 5.1),

$$u(\vec{r}) = p(\vec{r}) \quad \vec{r} \text{ sobre } \Omega \quad (5.14)$$

donde $p(\vec{r})$ es una función conocida sobre el contorno Ω .

2. Condición de frontera de **Neumann** o de **segundo tipo**. En este caso, se especifican los valores de la derivada de una solución tomada sobre el contorno del dominio Ω ,

$$\frac{\partial u(\vec{r})}{\partial n} = q(\vec{r}) \quad \vec{r} \text{ sobre } \Omega \quad (5.15)$$

donde $\vec{n}$ es la normal hacia fuera del dominio, $\frac{\partial u}{\partial n}$ es la derivada direccional de u a lo largo de la normal hacia fuera del contorno y $q(\vec{r})$ es una función conocida sobre el contorno Ω .

3. Condición de frontera **mixta**. Son combinación de los dos anteriores tipos de condiciones de contorno.

Las PDE donde sólo se especifican condiciones de contorno se denominan **ecuaciones de estado estacionario** y las PDE donde se especifican sólo valores iniciales se llaman **ecuaciones transitorias**.

5.5. MÉTODOS DE RESOLUCIÓN

En muchos casos no es posible obtener soluciones analíticas de las PDE y es preciso obtener aproximaciones a dichas soluciones, aplicando para ello métodos numéricos. Podemos dividir los métodos de resolución en dos grupos:

1. Los que se basan en discretizar la forma *diferencial* de las PDE.
2. Los que se basan en discretizar la forma *integral* de las PDE.

En la Sección 1.4.8 ya se describió un método basado en discretizar la forma diferencial de las PDE: el método de las líneas (MOL). Otro de estos métodos es el **método de diferencias finitas**, que fue uno de los primeros empleados en resolver PDE. Es fácil de implementar en regiones rectangulares de 2 y 3 dimensiones, pero difícil de implementar en geometrías más complicadas.

Los métodos de resolución que usan la forma integral de las PDE permiten un tratamiento más natural de las condiciones de contorno de Neumann, necesitan menos requisitos de regularidad o suavidad en la solución, y permiten tratar mejor geometrías complejas en problemas multidimensionales. Entre los métodos basados en discretizar la forma integral cabe destacar el **método de elementos finitos**, muy comúnmente empleado en entornos de simulación de PDE, el **método de los volúmenes finitos**, empleado en muchos paquetes de computación de dinámica de fluidos, y el **método de elementos de contorno**.

De todos estos métodos de resolución, únicamente se van a explicar en este capítulo el método de diferencias finitas y el método de elementos finitos. Para describir de forma sencilla estos métodos, vamos a considerar ecuaciones diferenciales cuya solución es una función de una variable, $u(x)$. Dado un conjunto de puntos x_i ($i = 0, \dots, N$) en el dominio de definición de $u(x)$, como se muestra en la Figura 5.2, la solución numérica que buscamos se representa por un conjunto discreto de valores de la función $\{u_0, \dots, u_N\}$ que aproxima a u en esos puntos. Es decir, $u_i \simeq u(x_i)$, con $i = 0, \dots, N$

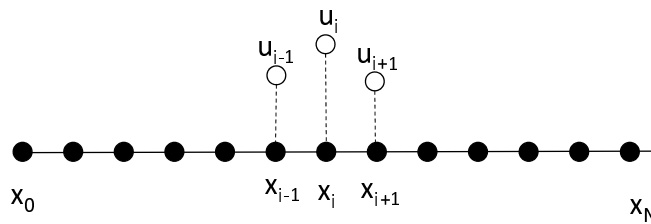


Figura 5.2: Discretización del dominio de la función $u(x)$.

En lo que sigue, consideraremos que los puntos están equiespaciados a lo largo del dominio, con una distancia constante $\Delta x = x_{i+1} - x_i$, $i = 0, \dots, N - 1$. De este modo, escribimos $u_{i+1} \simeq u(x_{i+1}) = u(x_i + \Delta x)$. Esta partición del dominio en menores subdominios se denomina comúnmente malla, y *mesh* o *grid* en lengua inglesa.

5.5.1. Método de diferencias finitas

En este método las ecuaciones diferenciales se aproximan a ecuaciones en diferencias, al reemplazar las derivadas por aproximaciones a las mismas en diferencias finitas. Por ejemplo, la primera y segunda derivada de la función $u(x)$ se pueden aproximar por las expresiones siguientes:

$$\frac{du(x_i)}{dx} = \frac{u_{i+1} - u_{i-1}}{2 \cdot \Delta x} + O(\Delta x^2) \quad (5.16)$$

$$\frac{d^2u(x_i)}{dx^2} = \frac{u_{i+1} - 2 \cdot u_i + u_{i-1}}{\Delta x^2} + O(\Delta x^2) \quad (5.17)$$

Ambas aproximaciones son de $O(\Delta x^2)$ y se obtienen empleando series de Taylor (para más información, véase (Sherwin 2004)).

Vamos a explicar el procedimiento general de aplicación del método de diferencias finitas, empleando como ejemplo la ecuación diferencial siguiente:

$$\frac{d^2u(x)}{dx^2} = f(x) \quad \text{en la región} \quad \Omega = \{x : 0 \leq x \leq 1\} \quad (5.18)$$

Vamos a emplear dos conjuntos de condiciones de contorno:

Tipo A: Condiciones de contorno de Dirichlet

$$u(0) = \alpha_1 \quad u(1) = \alpha_2$$

Tipo B: Condiciones de contorno mixtas (Dirichlet y Neumann)

$$u(0) = \alpha_1 \quad \frac{du}{dx}(1) = g$$

Para obtener una solución aproximada, hay que seguir los pasos siguientes:

Paso 1 Generar una malla:

$$x_i = i \cdot \Delta x \quad \text{con} \quad \begin{cases} i &= 0, 1, \dots, N \\ \Delta x &= \frac{1}{N} \end{cases} \quad (5.19)$$

Paso 2 Reemplazar las derivadas por una aproximación en cada punto de la malla donde desconocemos la solución. Sea u_i el valor de u en el punto x_i . Reemplazamos $\frac{d^2 u(x)}{dx^2}$ por la Ec. (5.17), obteniendo las siguientes $N - 1$ ecuaciones con $N + 1$ incógnitas $(u_0, \dots, u_N)$:

$$\frac{u_{i+1} - 2 \cdot u_i + u_{i-1}}{\Delta x^2} = f_i \quad \text{donde } i = 1, \dots, N - 1 \quad (5.20)$$

donde $f(x_i)$ se ha escrito abreviadamente f_i .

Las otras 2 ecuaciones necesarias las obtenemos a partir de las condiciones de contorno.

Para las condiciones de contorno de Tipo A, obtenemos el sistema de ecuaciones siguiente:

$$u_0 = \alpha_1 \quad (5.21)$$

$$\frac{u_2 - 2 \cdot u_1 + u_0}{\Delta x^2} = f_1 \quad (5.22)$$

$$\frac{u_3 - 2 \cdot u_2 + u_1}{\Delta x^2} = f_2 \quad (5.23)$$

$$\dots \quad (5.24)$$

$$\frac{u_N - 2 \cdot u_{N-1} + u_{N-2}}{\Delta x^2} = f_{N-1} \quad (5.25)$$

$$u_N = \alpha_2 \quad (5.26)$$

El sistema de ecuaciones anterior se puede escribir en forma matricial, $\mathbf{A} \cdot \mathbf{u} = \mathbf{s}$, de la manera siguiente:

$$\begin{pmatrix} 1 & 0 & \dots & & 0 \\ 1 & -2 & 1 & 0 & \dots & 0 \\ 0 & 1 & -2 & 1 & 0 & \dots & 0 \\ & & \ddots & \ddots & \ddots & & \\ 0 & \dots & 0 & 1 & -2 & 1 & 0 \\ 0 & & \dots & 0 & 1 & -2 & 1 \\ 0 & & & \dots & 0 & 1 \end{pmatrix} \begin{pmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{N-2} \\ u_{N-1} \\ u_N \end{pmatrix} = \begin{pmatrix} \alpha_1 \\ \Delta x^2 \cdot f_1 \\ \Delta x^2 \cdot f_2 \\ \vdots \\ \Delta x^2 \cdot f_{N-2} \\ \Delta x^2 \cdot f_{N-1} \\ \alpha_2 \end{pmatrix} \quad (5.27)$$

La matriz $\mathbf{A}$ es no singular, con lo cual el sistema admite una única solución $\mathbf{u}$.

Para las condiciones de contorno de Tipo B, podemos añadir sin más la primera condición de contorno, pero para añadir la condición de Neumann

$$\frac{du}{dx}(1) = g \quad (5.28)$$

tenemos que usar una aproximación de la primera derivada. Empleando la aproximación Ec. (5.16) obtenemos la siguiente ecuación:

$$\frac{u_{N+1} - u_{N-1}}{2 \cdot \Delta x} = g \quad (5.29)$$

Para obtener u_{N+1} , empleamos la ecuación diferencial (5.18) y la aproximación mostrada en la Ec. (5.17). Tenemos así la ecuación:

$$\frac{u_{N+1} - 2 \cdot u_N + u_{N-1}}{\Delta x^2} = f_N \quad (5.30)$$

Combinando las dos ecuaciones anteriores, es decir, la Ec. (5.29) y la Ec. (5.30), obtenemos:

$$u_N - u_{N-1} = g \cdot \Delta x - \frac{1}{2} \cdot f_N \cdot \Delta x^2 \quad (5.31)$$

Añadiendo esta ecuación y la condición de Dirichlet, el sistema de ecuaciones queda de la forma siguiente:

$$\begin{pmatrix} 1 & 0 & & & & 0 \\ 1 & -2 & 1 & 0 & \cdots & 0 \\ 0 & 1 & -2 & 1 & 0 & \cdots & 0 \\ & & \ddots & \ddots & \ddots & & \\ 0 & \cdots & 0 & 1 & -2 & 1 & 0 \\ 0 & & \cdots & 0 & 1 & -2 & 1 \\ 0 & & & \cdots & & -1 & 1 \end{pmatrix} \begin{pmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{N-2} \\ u_{N-1} \\ u_N \end{pmatrix} = \begin{pmatrix} \alpha_1 \\ \Delta x^2 \cdot f_1 \\ \Delta x^2 \cdot f_2 \\ \vdots \\ \Delta x^2 \cdot f_{N-2} \\ \Delta x^2 \cdot f_{N-1} \\ g \cdot \Delta x - \frac{1}{2} \cdot f_N \cdot \Delta x^2 \end{pmatrix} \quad (5.32)$$

Se observa que al introducir la condición de contorno de Neumann se obtiene una matriz menos simétrica.

Paso 3 Resolvemos el sistema de ecuaciones para obtener la solución aproximada en cada punto de la malla (u_i).

El método de diferencias finitas se puede aplicar también a PDE dependientes del tiempo. Para ello, hay que discretizar la solución tanto en el espacio como en el tiempo.

Por ejemplo, consideremos la PDE siguiente:

$$\frac{\partial u(x, t)}{\partial t} - b \cdot \frac{\partial^2 u(x, t)}{\partial x^2} = f(x) \quad \text{en } \Omega = \left\{ x, t : \begin{array}{l} 0 \leq x \leq 1 \\ 0 \leq t \leq T \end{array} \right\} \quad (5.33)$$

con las siguientes condiciones inicial y de contorno:

$$u(x, 0) = u_0(x) \quad \left\{ \begin{array}{l} u(0, t) = \alpha_1(t) \\ u(1, t) = \alpha_2(t) \end{array} \right. \quad (5.34)$$

donde α_1 y α_2 son funciones conocidas dependientes de t .

Además de la discretización espacial $\{x_0 = 0, x_1, \dots, x_{N-1}, x_N = 1\}$, tenemos una discretización temporal representada por la secuencia $\{t_0 = 0, t_1, \dots, t_j, \dots, T\}$. Por simplicidad, consideramos intervalos constantes tanto en el espacio, como en el tiempo. La solución discreta en un punto se representa por $u_{i,j} = u(x_i, t_j)$. Reemplazaremos las derivadas temporales y espaciales por sus respectivas aproximaciones en diferencias.

5.5.2. Método de elementos finitos

A continuación se describen los pasos que deben seguirse para aplicar el método de elementos finitos. En las explicaciones se ha intentado reducir al mínimo indispensable el aparataje matemático.

Paso 1 Se reescribe la PDE en una forma llamada *débil* o *variacional*.

En una formulación débil, no se precisa que una ecuación se mantenga de forma absoluta. La ecuación tiene soluciones débiles que son válidas respecto a ciertos “vectores de test” o “funciones de test”. El teorema más importante en este proceso de reformulación en tres dimensiones es el *Teorema de la Divergencia*, que en dos dimensiones se conoce como el *Teorema de Green* y en una dimensión como *Integración por Partes*.

El **teorema de la integración por partes** relaciona la integral de un producto de funciones con la integral de su derivada y su antiderivada. Consideremos las funciones $u = u(x)$ y $v = v(x)$, siendo:

$$du = \frac{du(x)}{dx} \cdot dx \quad dv = \frac{dv(x)}{dx} \cdot dx \quad (5.35)$$

La fórmula de integración por partes para estas funciones es la siguiente:

$$\int u(x) \cdot \frac{dv(x)}{dx} \cdot dx = u(x) \cdot v(x) - \int v(x) \cdot \frac{du(x)}{dx} \cdot dx \quad (5.36)$$

que se puede escribir en notación más compacta del siguiente modo

$$\int u \cdot dv = u \cdot v - \int v \cdot du \quad (5.37)$$

La integral definida en el intervalo $[a, b]$ es la siguiente:

$$\int_a^b u(x) \cdot \frac{dv(x)}{dx} \cdot dx = u(b) \cdot v(b) - u(a) \cdot v(a) - \int_a^b v(x) \cdot \frac{du(x)}{dx} \cdot dx \quad (5.38)$$

Un método para obtener la formulación débil de una ecuación diferencial en una dimensión consiste en multiplicar la ecuación diferencial por una función de test e integrar por partes el término diferencial de la ecuación.

Paso 2 Se realiza el mallado.

El dominio de variables independientes se divide mediante una partición en subdominios, llamados elementos finitos. Asociada a la partición anterior se construye un espacio vectorial de dimensión finita, llamado espacio de elementos finitos, siendo la solución numérica aproximada obtenida por elementos finitos una combinación lineal en dicho espacio vectorial.

Los elementos finitos pueden tener diferentes formas. En espacios de una dimensión son rectas, mientras en dos dimensiones los elementos son triangulares o cuadrangulares, y en tres dimensiones tetraedros, pentaedros, hexaedros, etc.

Paso 3 Se realiza la proyección del problema original sobre el espacio de elementos finitos.

Se obtiene la proyección del problema variacional original sobre el espacio de elementos finitos obtenido al realizar el mallado. Esto da lugar a un sistema

con un número de ecuaciones finito, aunque en general con un número elevado de ecuaciones e incógnitas. El número de incógnitas será igual a la dimensión del espacio vectorial de elementos finitos obtenido y, en general, cuanto mayor sea dicha dimensión, tanto mejor será la aproximación numérica obtenida.

Paso 4 Cálculo numérico de la solución del sistema de ecuaciones.

El cálculo de la solución nos permitirá conocer los valores de las variables en los nodos. Conocidos estos valores podremos obtener los valores de la función en cualquier sitio y no únicamente en los nodos.

Al contrario que en el método de diferencias finitas, el método de elementos finitos no da como resultado simplemente un conjunto de valores, sino una función definida en el dominio completo. La reconstrucción de la función a partir de las magnitudes calculadas constituye la esencia de este método.

Cabe resaltar que este método permite resolver PDE en dominios cerrados y no en dominios abiertos. No obstante, a efectos prácticos podemos considerar que el dominio es abierto si alejamos mucho las fronteras del dominio de la zona donde nos interesa obtener la solución a la PDE.

A continuación, vamos a aplicar del método de elementos finitos para resolver la siguiente ecuación diferencial:

$$\frac{d^2 u(x)}{dx^2} = f(x) \quad \text{en la región} \quad \Omega = \{x : 0 \leq x \leq 1\} \quad (5.39)$$

con las condiciones de contorno mixtas (Dirichlet y Neumann):

$$u(0) = \alpha \quad \frac{du(1)}{dx} = g \quad (5.40)$$

Discretizamos la región de interés Ω en N subdominios o elementos

$$\Omega_i = \{x : x_i \leq x \leq x_{i+1}\} \quad \text{con } i = 0, \dots, N-1. \quad (5.41)$$

Consideramos que la solución aproximada se representa por:

$$u^\delta(x) = \sum_{i=0}^N u_i \cdot N_i(x) \quad (5.42)$$

donde el conjunto de funciones $N_i(x)$ se conoce como **la base de expansión**, y también como funciones de peso o de forma. Su soporte se define como el conjunto de puntos donde $N_i(x) \neq 0$. Obsérvese que conocidos los términos u_i , la solución de la ecuación u^δ es conocida en cualquier punto x de la región Ω .

Los elementos finitos $N_i(x)$ tienen la peculiaridad de que su valor y el de sus derivadas son diferentes de cero sólo en los elementos que contienen el nodo i . Es decir, para un nodo intermedio i , el término N_i es distinto de cero sólo en los intervalos Ω_i y Ω_{i-1} . El término N_0 es sólo distinto de cero en el intervalo Ω_0 y el término N_N es sólo distinto de cero en el intervalo Ω_{N-1} . En dichos elementos, son rectas que vienen definidas por las siguientes ecuaciones.

$$N_i(x) = \begin{cases} \frac{x-x_{i-1}}{x_i-x_{i-1}} & \text{si } x_{i-1} \leq x \leq x_i \\ \frac{x_{i+1}-x}{x_{i+1}-x_i} & \text{si } x_i \leq x \leq x_{i+1} \\ 0 & \text{si } x \notin [x_{i-1}, x_{i+1}] \end{cases} \quad (5.43)$$

Las derivadas de $N_i(x)$ vienen dadas por las expresiones siguientes:

$$\frac{dN_i(x)}{dx} = \begin{cases} \frac{1}{x_i-x_{i-1}} & \text{si } x_{i-1} \leq x \leq x_i \\ \frac{-1}{x_{i+1}-x_i} & \text{si } x_i \leq x \leq x_{i+1} \\ 0 & \text{si } x \notin [x_{i-1}, x_{i+1}] \end{cases} \quad (5.44)$$

La solución $u^\delta(x)$ es una aproximación lineal a trozos de la solución, como se observa en la Figura 5.3. El valor de $u^\delta(x)$ en un punto x perteneciente al tramo Ω_i , donde $i = 0, \dots, N-1$ tiene sólo dos contribuciones: $u_i \cdot N_i(x)$ y $u_{i+1} \cdot N_{i+1}(x)$.

La forma débil correspondiente a la ecuación diferencial que queremos resolver la podemos obtener multiplicado la ecuación por la función $w(x)$ e integrando. Tenemos así:

$$\int_0^1 w(x) \cdot \frac{d^2 u(x)}{dx^2} \cdot dx = \int_0^1 w(x) \cdot f(x) \cdot dx \quad (5.45)$$

A continuación, integramos por partes el lado izquierdo de la ecuación anterior, para lo que aplicamos la Ec. (5.38) y obtenemos:

$$-\int_0^1 \frac{dw(x)}{dx} \cdot \frac{du(x)}{dx} \cdot dx + w(1) \cdot \frac{du(1)}{dx} - w(0) \cdot \frac{du(0)}{dx} = \int_0^1 w(x) \cdot f(x) \cdot dx \quad (5.46)$$

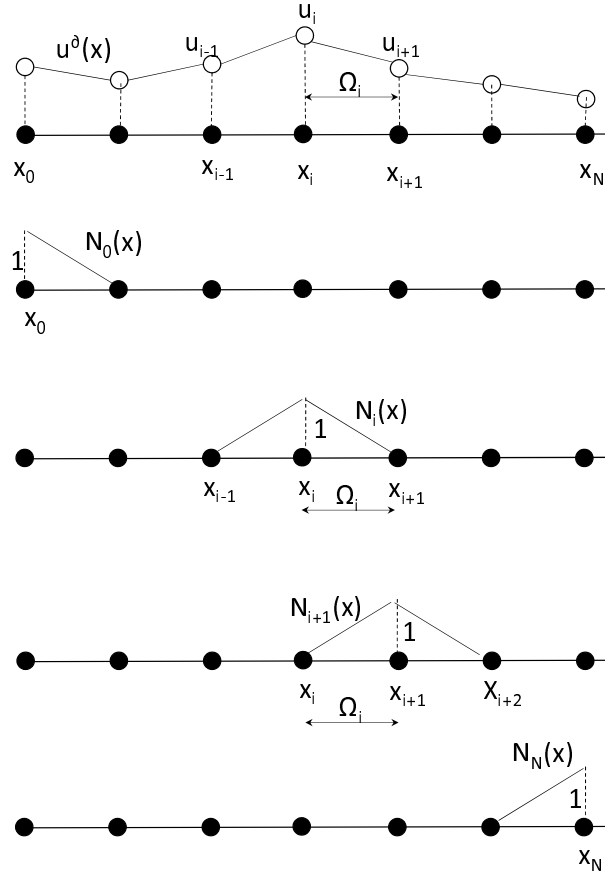


Figura 5.3: Aproximación lineal a trozos de $u^\delta(x)$ y funciones de forma.

donde hemos considerado que $a = 0$, $b = 1$, y que las funciones u y v de la Ec. (5.38) son:

$$u = w(x) \quad (5.47)$$

$$dv = \frac{d^2 u}{dx^2} \quad \Rightarrow \quad v = \frac{du}{dx} \quad (5.48)$$

La imposición de la condición de contorno de Neumann se hace simplemente sustituyendo $\frac{du(1)}{dx} = g$ en la anterior ecuación. La condición de contorno de Dirichlet se aplica imponiendo $u(0) = \alpha$ y haciendo que en el punto $x = 0$ la función de peso $w(x)$ sea cero: $w(0) = 0$. En general, siempre se impone que las funciones $w(x)$ sean cero en los puntos donde se aplican las condiciones de contorno de Dirichlet.

$$-\int_0^1 \frac{dw(x)}{dx} \cdot \frac{du(x)}{dx} \cdot dx + w(1) \cdot g = \int_0^1 w(x) \cdot f(x) \cdot dx \quad (5.49)$$

Reemplazando

$$u(x) \approx u^\delta(x) = \sum_{j=0}^N u_j \cdot N_j(x) \quad (5.50)$$

en la ecuación anterior, obtenemos:

$$- \int_0^1 \frac{dw(x)}{dx} \cdot \sum_{j=0}^N u_j \cdot \frac{dN_j(x)}{dx} \cdot dx + w(1) \cdot g = \int_0^1 w(x) \cdot f(x) \cdot dx \quad (5.51)$$

De la condición de contorno de Dirichlet, ya conocemos u_0 . Necesitamos conocer $u_1, \dots, u_N$, que vamos a obtener de las siguientes N ecuaciones, donde hemos considerando que $w(x) = N_i(x)$, para $i = 1, \dots, N$.

$$- \int_0^1 \frac{dN_i(x)}{dx} \cdot \sum_{j=0}^N u_j \cdot \frac{dN_j(x)}{dx} \cdot dx + N_i(1) \cdot g = \int_0^1 N_i(x) \cdot f(x) \cdot dx \quad (5.52)$$

donde $i = 1, \dots, N$

Vamos a deducir la ecuación para un determinado elemento i , donde el valor de i puede ser $1, \dots, N-1$. Para ello vamos a tener en cuenta los siguientes puntos:

1. $N_i(1) \cdot g = 0$ para cualquier $i \neq N$ y $N_N(1) \cdot g = g$.
2. $\frac{dN_i(x)}{dx}$ sólo es diferente de cero en los intervalos Ω_i y Ω_{i-1} . En consecuencia, la integral del lado izquierdo de la Ec. (5.52) tiene sólo las contribuciones de estos dos intervalos. Es decir, la contribución de $x \in [x_i, x_{i+1}]$ y $x \in [x_{i-1}, x_i]$.
3. Para $x \in [x_{i-1}, x_i]$, el valor de $\sum_{j=0}^N u_j \cdot \frac{dN_j(x)}{dx}$ es:

$$\sum_{j=0}^N u_j \cdot \frac{dN_j(x)}{dx} = u_{i-1} \cdot \frac{dN_{i-1}}{dx} + u_i \cdot \frac{dN_i}{dx} \quad (5.53)$$

4. Para $x \in [x_i, x_{i+1}]$, el valor de $\sum_{j=0}^N u_j \cdot \frac{dN_j(x)}{dx}$ es:

$$\sum_{j=0}^N u_j \cdot \frac{dN_j(x)}{dx} = u_i \cdot \frac{dN_i}{dx} + u_{i+1} \cdot \frac{dN_{i+1}}{dx} \quad (5.54)$$

Teniendo en cuenta estos puntos, obtenemos la ecuación siguiente:

$$\begin{aligned}
& - \int_{x_{i-1}}^{x_i} \frac{dN_i(x)}{dx} \cdot \left(u_{i-1} \cdot \frac{dN_{i-1}}{dx} + u_i \cdot \frac{dN_i}{dx} \right) \cdot dx \\
& - \int_{x_i}^{x_{i+1}} \frac{dN_i(x)}{dx} \cdot \left(u_i \cdot \frac{dN_i}{dx} + u_{i+1} \cdot \frac{dN_{i+1}}{dx} \right) \cdot dx \\
& = \int_{x_{i-1}}^{x_i} N_i(x) \cdot f(x) \cdot dx + \int_{x_i}^{x_{i+1}} N_i(x) \cdot f(x) \cdot dx
\end{aligned} \tag{5.55}$$

La parte izquierda de la ecuación, tras sustituir $\frac{dN_{i-1}(x)}{dx}$, $\frac{dN_i(x)}{dx}$, $\frac{dN_{i+1}(x)}{dx}$ por sus respectivas expresiones e integrar, es:

$$\begin{aligned}
& - \int_{x_{i-1}}^{x_i} \frac{1}{x_i - x_{i-1}} \cdot \left(u_{i-1} \cdot \frac{-1}{x_i - x_{i-1}} + u_i \cdot \frac{1}{x_i - x_{i-1}} \right) \cdot dx \\
& - \int_{x_i}^{x_{i+1}} \frac{-1}{x_{i+1} - x_i} \cdot \left(u_i \cdot \frac{-1}{x_{i+1} - x_i} + u_{i+1} \cdot \frac{1}{x_{i+1} - x_i} \right) \cdot dx \\
& = \frac{u_{i+1} - u_i}{x_{i+1} - x_i} - \frac{(u_i - u_{i-1})}{x_i - x_{i-1}}
\end{aligned} \tag{5.56}$$

La parte derecha de la ecuación, tras sustituir $N_i(x)$ por su expresión, es:

$$\int_{x_{i-1}}^{x_i} \frac{x - x_{i-1}}{x_i - x_{i-1}} \cdot f(x) \cdot dx + \int_{x_i}^{x_{i+1}} \frac{x_{i+1} - x}{x_{i+1} - x_i} \cdot f(x) \cdot dx \tag{5.57}$$

Podemos evaluar la anterior expresión empleando la regla del trapecio, que dice:

$$\int_{x_i}^{x_{i+1}} g(x) \cdot dx \approx \frac{g(x_i) + g(x_{i+1})}{2} \cdot (x_{i+1} - x_i) \tag{5.58}$$

Empleando esta regla, obtenemos:

$$\left(\frac{x_i - x_{i-1}}{2} + \frac{x_{i+1} - x_i}{2} \right) f_i \tag{5.59}$$

Igualando la parte izquierda, Ec. (5.56), y la parte derecha, Ec. (5.59), de la ecuación se obtiene:

$$-\frac{u_i - u_{i-1}}{x_i - x_{i-1}} + \frac{u_{i+1} - u_i}{x_{i+1} - x_i} = \frac{(x_i - x_{i-1}) + (x_{i+1} - x_i)}{2} \cdot f_i \quad (5.60)$$

Si consideramos que los puntos están distribuidos equidistantemente, entonces $x_{i+1} - x_i = x_i - x_{i-1} = \Delta x$. Manipulando la Ec. (5.60), considerando que los puntos están distribuidos equidistantes, llegamos a la siguiente ecuación, que es idéntica a la fórmula obtenida aplicando el método de diferencias finitas:

$$\frac{u_{i+1} - 2 \cdot u_i + u_{i-1}}{\Delta x} = \Delta x \cdot f_i \quad (5.61)$$

Vamos a deducir la ecuación para el elemento $i = N$. Para ello, vamos a tener en cuenta los siguientes puntos:

1. $N_N(1) \cdot g = g$.
2. $\frac{dN_N(x)}{dx}$ sólo es diferente de cero en el intervalo Ω_{N-1} , En consecuencia, la integral del lado izquierdo de la Ec. (5.52) tiene sólo la contribución de $x \in [x_{N-1}, x_N]$.
3. Para $x \in [x_{N-1}, x_N]$, el valor de $\sum_{j=0}^N u_j \cdot \frac{dN_j(x)}{dx}$ es:

$$\sum_{j=0}^N u_j \cdot \frac{dN_j(x)}{dx} = u_{N-1} \cdot \frac{dN_{N-1}}{dx} + u_N \cdot \frac{dN_N}{dx} \quad (5.62)$$

Teniendo en cuenta estos puntos, obtenemos la ecuación siguiente:

$$\begin{aligned} - \int_{x_{N-1}}^{x_N} \frac{dN_N(x)}{dx} \cdot \left(u_{N-1} \cdot \frac{dN_{N-1}}{dx} + u_N \cdot \frac{dN_N}{dx} \right) \cdot dx + g \\ = \int_{x_{N-1}}^{x_N} N_N(x) \cdot f(x) \cdot dx \end{aligned} \quad (5.63)$$

Sustituyendo $N_N(x)$, $N_{N-1}(x)$ y sus derivadas por sus respectivas expresiones, obtenemos:

$$\begin{aligned} - \int_{x_{N-1}}^{x_N} \frac{1}{x_N - x_{N-1}} \cdot \left(u_{N-1} \cdot \frac{-1}{x_N - x_{N-1}} + u_N \cdot \frac{1}{x_N - x_{N-1}} \cdot dx \right) + g \\ = \int_{x_{N-1}}^{x_N} \frac{x - x_{N-1}}{x_N - x_{N-1}} \cdot f(x) \cdot dx \end{aligned} \quad (5.64)$$

Realizando las integrales y aplicando la regla del trapecio al término a la derecha de la ecuación, obtenemos:

$$u_N - u_{N-1} = g \cdot (x_N - x_{N-1}) - \frac{1}{2} \cdot f_N \cdot (x_N - x_{N-1})^2 \quad (5.65)$$

Si consideramos que los puntos están distribuidos equidistantemente, entonces $(x_N - x_{N-1}) = \Delta x$.

$$u_N - u_{N-1} = g \cdot \Delta x - \frac{1}{2} \cdot f_N \cdot \Delta x^2 \quad (5.66)$$

Tenemos así el siguiente conjunto de ecuaciones que nos permite calcular los valores de $u_0, \dots, u_N$:

$$u_0 = \alpha \quad (5.67)$$

$$\frac{u_{i+1} - 2 \cdot u_i + u_{i-1}}{\Delta x} = \Delta x \cdot f_i \quad \text{para } i = 1, \dots, N-1 \quad (5.68)$$

$$u_N - u_{N-1} = g \cdot \Delta x - \frac{1}{2} \cdot f_N \cdot \Delta x^2 \quad (5.69)$$

Estas ecuaciones se pueden escribir en forma matricial como se muestra a continuación.

$$\begin{pmatrix} 1 & 0 & \dots & & & 0 \\ 1 & -2 & 1 & 0 & \dots & 0 \\ 0 & 1 & -2 & 1 & 0 & \dots & 0 \\ & & \ddots & \ddots & \ddots & & \\ 0 & \dots & 0 & 1 & -2 & 1 & 0 \\ 0 & & \dots & 0 & 1 & -2 & 1 \\ 0 & & & \dots & & -1 & 1 \end{pmatrix} \begin{pmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{N-2} \\ u_{N-1} \\ u_N \end{pmatrix} = \begin{pmatrix} \alpha_1 \\ \Delta x^2 \cdot f_1 \\ \Delta x^2 \cdot f_2 \\ \vdots \\ \Delta x^2 \cdot f_{N-2} \\ \Delta x^2 \cdot f_{N-1} \\ g \cdot \Delta x - \frac{1}{2} \cdot f_N \cdot \Delta x^2 \end{pmatrix} \quad (5.70)$$

La matriz obtenida es idéntica a la obtenida por el método de diferencias finitas, Como puede comprobarse comparando esta ecuación con la Ec. (5.32). Tenemos que resolver este sistema de ecuaciones para obtener los valores de $u_0, \dots, u_N$, que nos van a permitir reconstruir la solución aproximada u^δ .

5.6. ENTORNOS DE SIMULACIÓN DE PDE

Existen multitud de entornos, tanto comerciales como gratuitos, que facilitan la simulación de PDE. A continuación se van a describir las principales características de algunos de ellos, aunque igualmente se podrían haber escogido otros:

- COMSOL Multiphysics.

Software comercial inicialmente desarrollado en la Universidad Tecnológica Real de Estocolmo, Suecia (COMSOL 2016). Emplea el método de elementos finitos como método de resolución de PDE. Permite introducir sistemas acoplados de PDE empleando tanto la formulación diferencial como la formulación débil. Además, tiene multitud de módulos de aplicación específicos como AC/DC, Acústica, CDF, corrosión, etc. que facilitan la definición sencilla de problemas en diferentes dominios.

- FlexPDE.

Software comercial que permite resolver problemas estacionarios o dependientes de tiempo, análisis de autovalores o problemas de contorno libre empleando el método de elementos finitos (Inc. 2016). La versión de estudiantes está restringida a 5 ecuaciones simultáneas y a 100, 800 y 1600 nodos en los mallado de una, dos y tres dimensiones, respectivamente.

- Partial Differential Equation Toolbox.

Toolbox de Matlab que proporciona funciones para resolver PDE de dos y tres dimensiones usando el método de elementos finitos (MathWorks 2016). Existe la opción de realizar automáticamente el mallado especificando un parámetro de la malla. Para problemas de dos dimensiones emplea elementos triangulares con base lineal a tramos y para problemas de tres dimensiones tetraedros lineales o de base cuadrática, según especifique el usuario.

- FreeFem++.

Software libre con licencia GPL escrito en C++ enfocado a resolver PDE usando el método de los elementos finitos (Hecht 2012). Está desarrollado y mantenido por la Universidad Pierre y Marie Curie y el Laboratorio Jacques-Louis Lion.

- QuickField.

Software comercial que tiene una versión gratuita de estudiantes (Analysis 2016). Emplea el método de elementos finitos, se pueden especificar problemas 2D y 3D con simetría axial tanto estáticos como cuasi-estáticos, y está

orientado a la simulación visual de procesos electromagnéticos, térmicos y de elasticidad de materiales. No se introducen ecuaciones, sino que se especifica a través de una serie de ventanas desplegables el tipo de problema, el sistema de coordenadas empleado, las propiedades de cada material, etc. La geometría del problema se puede definir también de forma sencilla, empleando el ratón.

- Finite element method magnetostatics (FEMM).

Software gratuito desarrollado por David Meeker, que desarrolló su tesis doctoral en la *School of Engineering and Applied Science* de la universidad de Virginia (Meeker 2016). Emplea el método de elementos finitos. Permite describir problemas magnetostáticos, electrostáticos, problemas magnéticos armónicos en el tiempo, de flujo de corriente y de flujo de calor en estado estacionario en dominios de 2 dimensiones y 3 dimensiones con simetría axial. Al igual que en QuickField, el problema se especifica a través de una serie de ventanas desplegables y no se escribe ninguna ecuación diferencial. Para dibujar la geometría se pueden usar botones en modo dibujo (punto, segmento de línea, segmento de arco, etc.) o importar un fichero AutoCAD de formato Dxf.

- High Frequency Electromagnetic Field Simulation (HFSS).

Software comercial de la compañía Ansys, que emplea el método de los elementos finitos para resolver problemas electromagnéticos (Ansys 2016). Existe también una versión de estudiante de este software. Se usa habitualmente para el diseño de antenas y elementos circuitales electrónicos RF complejos, como filtros y líneas de transmisión. Fue originariamente desarrollado por el profesor Zoltas Cendes y sus estudiantes, en la Universidad Carnegie Mellon.

5.7. FLEXPDE

Partiendo de un fichero de texto con extensión *.pde* escrito por el usuario, que contiene una descripción del sistema de ecuaciones en derivadas parciales, FlexPDE realiza las operaciones necesarias para transformar dicha descripción en un modelo de elementos finitos, resolver el modelo y representar los resultados en forma tabular y/o gráfica. El entorno FlexPDE tiene un editor que permite generar los ficheros de entrada, un generador de malla para construir la malla de elementos finitos, un resolutor de elementos finitos para encontrar la solución y un sistema gráfico para representar los resultados. La ventana que se muestra al arrancar FlexPDE se muestra en la Figura 5.4.

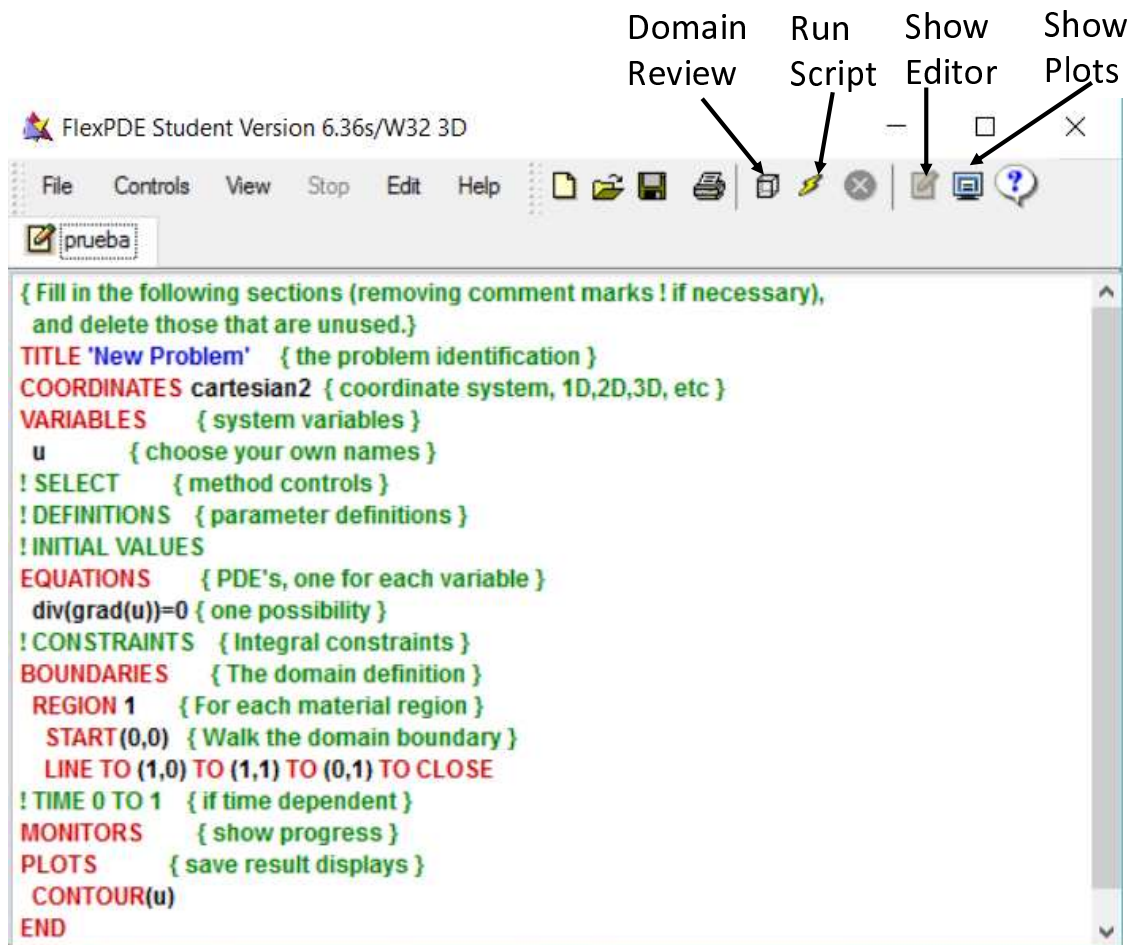


Figura 5.4: Ventana para la edición de ficheros de comandos (*script*) de FlexPDE versión 6.36s. En la parte superior hay una regleta con botones para abrir un script (fichero de texto con extensión *.pde*), guardarlo en fichero, comprobar el dominio de definición del modelo y su mallado (botón *Domain Review*), ejecutar un script (botón *Run Script*), pasar a la vista de edición del script (botón *Show Editor*) y pasar a la vista donde se muestran los gráficos (botón *Show Plots*).

En esta sección se va a mostrar cómo usar FlexPDE para describir un PDE de una o de dos dimensiones, realizar su simulación y representar los datos de salida. Se recomienda leer el manual de usuario de FlexPDE (Inc. 2016) para entender en profundidad FlexPDE y aprender a simular problemas en tres dimensiones.

5.7.1. Ficheros de trabajo

FlexPDE trabaja con un conjunto de ficheros cuyos nombres tienen la estructura `<nombre_modelo>.<extensión>`, siendo `<nombre_modelo>` el identificador único del modelo a ejecutar. En la Tabla 5.1 se indica el significado de algunas de las extensiones más comunes.

Tabla 5.1: Extensiones de ficheros usadas por FlexPDE.

Extensión	Significado
.PDE	FlexPDE lee la descripción del modelo de un script con esta extensión. Se trata de un fichero de texto, creado por el usuario para describir el modelo a ejecutar. Este fichero establece el <code><nombre_modelo></code> usado por los otros ficheros.
.PG6	Fichero de salida donde FlexPDE escribe información gráfica con un formato específico de FlexPDE, por lo que no puede ser leído por otros programas.
.LOG	Fichero de texto donde FlexPDE escribe un resumen del progreso de cada ejecución. Contiene información sobre el paso de la simulación, estimación de los errores, uso de memoria y otros datos.

5.7.2. Fichero con extensión .PDE

Este fichero consiste en un conjunto de secciones, cada una identificada por una cabecera. En las Tablas 5.2 y 5.3 se muestra el significado de algunas de las secciones.

En cualquier punto del fichero se pueden incluir comentarios. Los comentarios que ocupen varias líneas deben escribirse delimitados entre llaves (`{}`). Los comentarios pueden también empezar por una exclamación (`!`). En este último caso, el comentario termina al final de la línea.

Al crear un nuevo *script* con el editor de FlexPDE, se muestra un fichero con muchos campos ya rellenos para facilitar la tarea de edición (véase nuevamente la Figura 5.4).

A continuación se describe cómo especificar el problema, creando el fichero de comandos con extensión *.pde*.

5.7.3. Geometría

La sección opcional `COORDINATES` permite definir la geometría del problema. En caso de no existir esta sección, se asume que se emplea el sistema coordenado `CARTESIAN2` (véase la Tabla 5.4).

En geometrías 2D y 1D, se supone que la solución del sistema de PDE no tiene variación en una dirección (en la caso 2D) o en dos direcciones (en el caso 1D) de

Tabla 5.2: Algunas secciones de un script de FlexPDE.

Sección	Significado
TITLE	Etiqueta descriptiva para la salida.
SELECT	Controles de usuario que modifican el comportamiento por defecto de FlexPDE.
COORDINATES	Especificación de las coordenadas en que se define el problema.
VARIABLES	Declaración de las variables dependientes. Cada variable tiene que ir asociada a una ecuación y puede tener también asociada una condición de contorno. Al asignar nombres a las variables debe tenerse en cuenta que: – FlexPDE no distingue entre minúsculas y mayúsculas. – El nombre puede tener cualquier longitud, debe empezar por una letra y puede contener cualquier combinación de caracteres (excluyendo el carácter ‘-’, que está reservado para el signo menos), números y/o símbolos que no sean palabras reservadas – El nombre de la variable no puede ser “t”, ya que este nombre está reservado a la variable tiempo.
GLOBAL VARIABLES	Sección opcional donde se declaran las variables globales, que tienen las mismas reglas para declararse que las variables dependientes definidas en la sección VARIABLES , pero no tienen asociadas condiciones de contorno. Pueden ser estacionarias o dependientes del tiempo, y pueden estar definidas en términos de integrales sobre el dominio, o por valores puntuales de otras funciones.
DEFINITIONS	En esta sección se definen parámetros, relaciones o funciones. Normalmente, cuando se declara una definición se le asigna un valor, que puede ser una constante o una expresión. Las definiciones no se representan por una aproximación de elementos finitos sobre la malla, sino que se calculan cuando son necesarias en varios instantes y posiciones. Ejemplo: $\text{Viscosity} = 3.02\text{e-}4 * \exp(-5 * \text{Temp})$ Se puede cambiar el valor de los parámetros definidos en las regiones (REGION) definidas en la sección BOUNDARIES. De este modo, los parámetros del sistema de ecuaciones diferenciales pueden cambiar de valor en las diferentes zonas del dominio donde se desea encontrar la solución de dicho sistema de ecuaciones.

Tabla 5.3: Algunas secciones de un script de FlexPDE (continuación).

Sección	Significado
INITIAL VALUES	Sección donde se dan valores iniciales a las variables dependientes. Para problemas de estado estacionario, esta sección es opcional y para problemas dependientes del tiempo, esta sección debería incluir una sentencia por cada variable. Las sentencias de valor inicial se forman siguiendo el nombre de la variable por un operador de asignación '=' seguido de una constante, función, expresión o una definición previamente definida. Cuando no se especifica el valor inicial de una variable dependiente, dicha variable se inicializa a cero. Ejemplo: <pre>INITIAL VALUES U = 1.0-x</pre>
EQUATIONS	En esta sección, cada variable se asocia con una ecuación en derivadas parciales.
BOUNDARIES	Esta sección se usa para describir el dominio del problema sobre el que se ha de resolver el sistema de ecuaciones, y para especificar las condiciones de contorno sobre las superficies externas de este dominio. Para describir la geometría se especifica el perímetro del dominio mediante segmentos rectilíneos o curvos. Los tipos principales de condiciones de contorno son de tipo Dirichlet (VALUE en la terminología de FlexPDE) o de tipo natural (NATURAL en la terminología de FlexPDE). En las condiciones de contorno de tipo Dirichlet se especifica el valor que ha de tomar una variable en el contorno del dominio, mientras que en las condiciones de tipo natural se especifica un flujo en el contorno del dominio.
TIME	Define el dominio temporal.
MONITORS, PLOTS	Se lista la salida gráfica deseada. Los gráficos incluyen gráficos de contorno, de superficie, de elevación y vectoriales.
END	Fin del <i>script</i> .

Tabla 5.4: Sistemas de ejes coordenados de FlexPDE.

Nombre FlexPDE	Ejes Coordenados	Coordenadas del Modelo
CARTESIAN1	Cartesiano (X, Y, Z)	X
CYLINDER1	Cilíndrico (R, Phi, Z)	R
SPHERE1	Esférico (R, Theta, Phi)	R
CARTESIAN2	Cartesiano (X, Y, Z)	X, Y
XCYLINDER	Cilíndrico (R, Phi, Z)	Z, R
YCYLINDER	Cilíndrico (R, Phi, Z)	R, Z
CARTESIAN3	Cartesiano (X, Y, Z)	X, Y, Z

los ejes coordenados. Se construye la malla en el espacio restante y se considera que las derivadas en la coordenada que falta tienen valor 0.

En la Tabla 5.4 se muestran los ejes coordenados que se pueden especificar en FlexPDE. La primera coordenada mostrada en la tercera columna de la tabla se usa como eje horizontal en la salida gráfica, mientras que la segunda coordenada se usa como eje vertical.

5.7.4. Variables y ecuaciones

La sección `EQUATIONS` se usa para incluir las PDE que definen las variables dependientes del problema. Las PDE se han de definir usando derivadas espaciales de primer y segundo orden. Para definir las se pueden emplear los operadores diferenciales mostrados en la Tabla 5.5. Para definir el sistema de PDE se pueden emplear los parámetros previamente definidos en la sección `DEFINITIONS`. En la sección `DEFINITIONS`, además de declarar los parámetros, se les puede dar un valor por defecto.

Tiene que existir una ecuación por cada variable dependiente listada en las secciones `VARIABLES` y `GLOBAL VARIABLES`. Cada ecuación, salvo que exista una única ecuación, debe etiquetarse con el nombre de la variable a la cual está asociada. Las condiciones de contorno se definen en la sección `BOUNDARIES` y se asocian con ecuaciones por el uso del nombre de la variable, que selecciona una ecuación a través de su etiqueta.

Tabla 5.5: Algunos de los operadores diferenciales existentes en FlexPDE.

Operador en FlexPDE	Descripción
$D\langle n \rangle(\arg)$	Derivada parcial de primer orden de $\arg$ con respecto a la coordenada $\langle n \rangle$. Por ejemplo, en coordenadas cartesianas de dos dimensiones tenemos los siguientes operadores: $DX(\arg)$, $DY(\arg)$. En las geometrías cilíndricas predefinidas en FlexPDE ($XCYLINDER$ e $YCYLINDER$) están definidos los operadores $DR(\arg)$ y $DZ(\arg)$.
$D\langle n \rangle \langle m \rangle (\arg)$	Derivada parcial de segundo orden de $\arg$ con respecto a las coordenadas $\langle n \rangle$ y $\langle m \rangle$. Por ejemplo, en coordenadas cartesianas de dos dimensiones tenemos: $DXX(\arg)$, $DYY(\arg)$, $DXY(\arg)$ y $DYX(\arg)$. En las geometrías cilíndricas predefinidas en FlexPDE ($XCYLINDER$ e $YCYLINDER$) están definidos los operadores $DRR(\arg)$, $DRZ(\arg)$, $DZR(\arg)$ y $DZZ(\arg)$.
$DIV(\text{vector_arg})$	Divergencia del argumento vectorial que produce un resultado escalar.
$DIV(\text{argx}, \text{argy} \{, \text{argz}\})$	Divergencia del vector cuyas componentes son argx y argy (y posiblemente argz en 3D).
$GRAD(\text{scalar_arg})$	Gradiente del argumento escalar que produce un resultado vectorial.
$GRAD(\text{vector_arg})$	Gradiente del argumento vectorial que da como resultado un tensor. En geometría curvilínea genera términos adicionales debido a la diferenciación de los vectores unitarios. Sólo es equivalente al gradiente de los componentes del vector en geometría cartesiana. FlexPDE se encarga de realizar estas expansiones correctamente en todas las geometrías soportadas.
$CURL(\text{vector_arg})$	Devuelve el vector resultante de aplicar el operador rotacional a vector_arg .
$DEL2(\text{scalar_arg})$	Laplaciano de scalar_arg , que es equivalente a $DIV(GRAD(\text{scalar_arg}))$.
$DEL2(\text{vector_arg})$	Laplaciano de vector_arg , que es equivalente a $DIV(GRAD(\text{vector_arg}))$.

5.7.5. Dominio del problema: regiones y parámetros

El espacio se compone de regiones, que se especifican del modo siguiente dentro de la sección `BOUNDARIES`:

```
REGION "nombre" <numero>
```

o bien

```
REGION "nombre" <nombre>
```

donde el nombre entre comillas es opcional.

Las regiones especifican el dominio del problema bajo estudio, y pueden tener lazos cerrados o islas. Puede existir una o más regiones en el dominio y en cada región se pueden especificar expresiones para el cálculo del valor de los parámetros, pero estas expresiones no pueden variar en una misma región.

Por convenio se suele escribir una primera región que contiene el dominio completo del problema bajo estudio. Las regiones definidas con posterioridad en el *script* cubren partes de regiones definidas anteriormente.

Para describir una región se especifica un perímetro, pasando de una unión a otra mediante segmentos lineales (`LINE`), splines (`SPLINE`) o arcos (`ARC`). Se supone que cada segmento continúa desde el final del anterior segmento y que el comienzo de la descripción del perímetro ha de comenzar con la cláusula `START`. Se puede hacer que un segmento vuelva al inicio con la palabra `CLOSE` o `TO CLOSE`.

Se muestra a continuación la descripción de diferentes regiones, así como los dominios a que dan lugar.

- Región rectangular.

```
START (0, 0)
  LINE TO (1, 0)
  TO (1, 1)
  TO (0, 1)
  TO CLOSE
```

Cualquier otra figura poligonal puede ser construida de forma análoga. La definición de esta región rectangular se corresponde con la región mostrada en la Figura 5.5.

A continuación se muestra un ejemplo donde existe un rectángulo externo definido en la `REGION 1` y dos rectángulos internos definidos en la `REGION 2`. Esta definición se corresponde con la Figura 5.6.

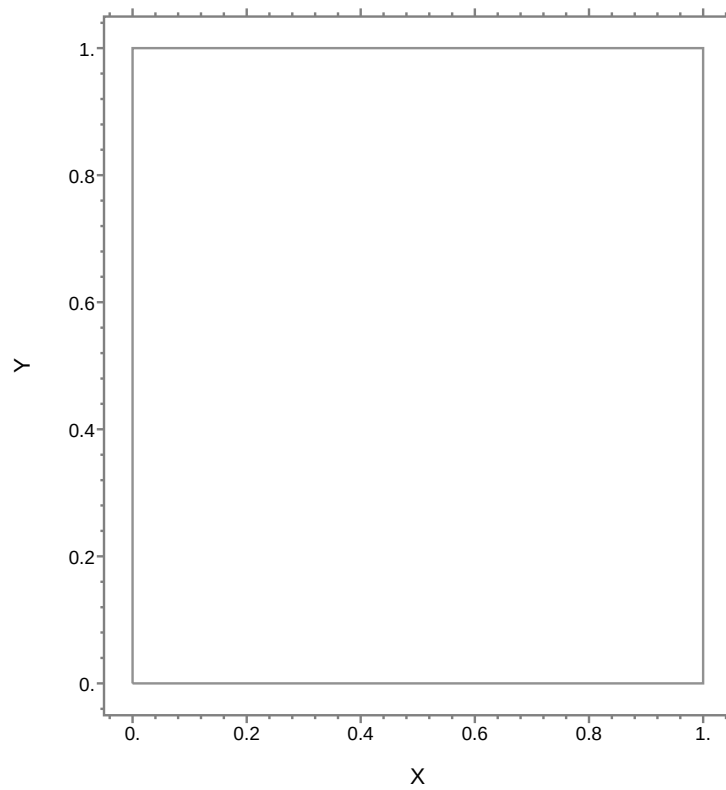


Figura 5.5: Dominio rectangular.

```

REGION 1      { un rectangulo externo }
START(0,0)
LINE TO (10,0) TO (10,10) TO (0,10) TO CLOSE

REGION 2      { dos rectangulos internos }
START(1,1)
LINE TO (2,1) TO (2,2) TO (1,2) TO CLOSE

START(5,5)
LINE TO (6,5) TO (6,6) TO (5,6) TO CLOSE

```

– Región circular.

Una región curva se puede especificar dando el centro y el ángulo que abarca un arco. Para especificar la región circular mostrada en la Figura 5.7 se ha especificado el punto inicial del arco, su centro y el ángulo que abarca.

```

START (1, 0)
  ARC (CENTER = 0, 0) ANGLE = 360 TO CLOSE

```

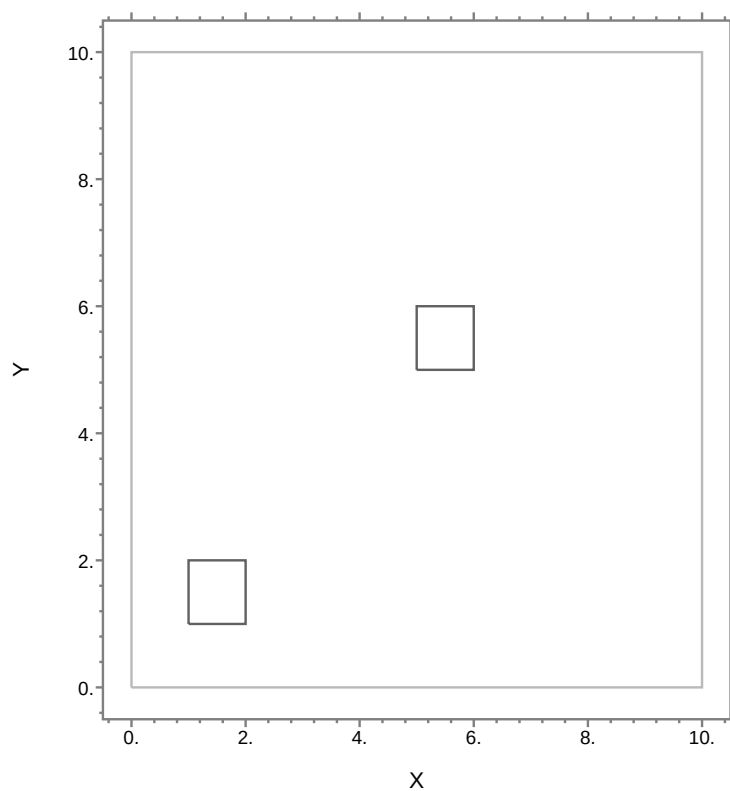


Figura 5.6: Dominio rectangular con dos regiones.

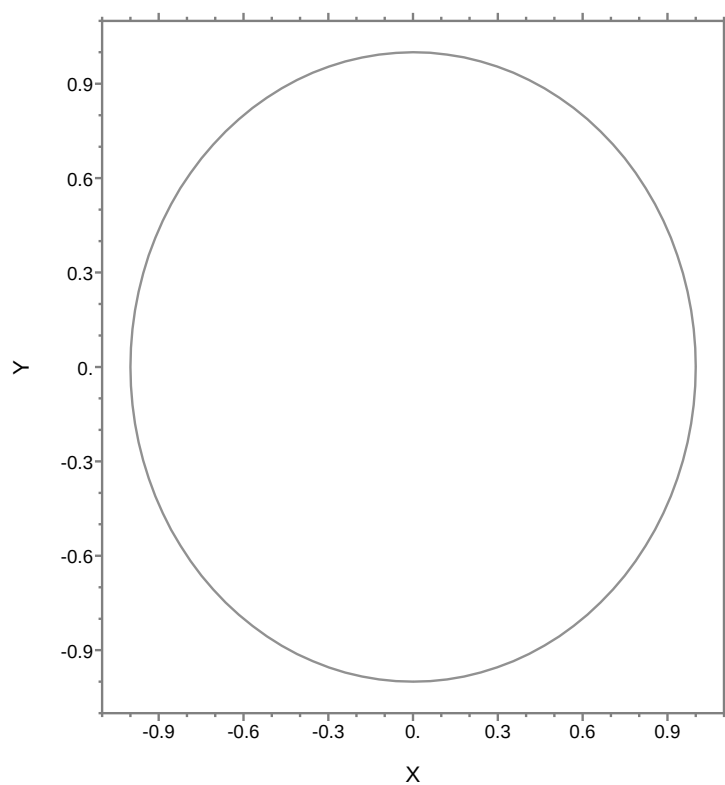


Figura 5.7: Dominio circular.

5.7.6. Especificación de las condiciones de contorno

Cada sentencia de condiciones de contorno toma como argumento el nombre de una variable dependiente. Este nombre asocia la condición de contorno con una de las ecuaciones del listado de ecuaciones. Dicha ecuación se ve modificada por la condición de contorno especificada.

Se pueden especificar dos tipos de condiciones de contorno: **VALUE** y **NATURAL**.

La condición de contorno **VALUE** especifica el valor que una variable debe tomar sobre el contorno del dominio.

La condición de contorno por defecto de FlexPDE es:

$$\text{NATURAL}(\text{VARIABLE}) = 0$$

El significado preciso de la condición de contorno **NATURAL** depende de la PDE para la que se especifique. En el caso de las ecuaciones de Laplace y Poisson, es equivalente a la condición de contorno de Neumann o a la derivada normal. Consideremos la ecuación de Laplace:

$$\nabla \cdot \nabla u = 0 \quad (5.71)$$

El teorema de la divergencia establece que la integral de esta ecuación sobre todo el espacio es igual a la integral sobre la superficie del contorno de la componente normal del flujo. La ecuación en dos dimensiones es la siguiente:

$$\iint_A \text{div}(\text{grad}(u)) \cdot dA = \oint_S n \cdot \text{grad}(u) \cdot dl \quad (5.72)$$

El valor sobre la superficie del dominio del término $n \cdot \text{grad}(u)$ es la condición de contorno natural para las ecuaciones de Laplace y Poisson. Es decir, se especifica el flujo en el contorno del dominio.

El teorema de la divergencia es una manifestación particular del proceso más general de integración por partes. FlexPDE aplica la integración por partes a todos los términos de la PDE que contienen derivadas de segundo orden. La condición de contorno natural en FlexPDE proporciona el valor del flujo superficial, definido dicho flujo por la integración por partes de los términos de segundo orden de la PDE.

5.7.7. Control de la precisión de la solución

FlexPDE aplica un chequeo de consistencia (*consistency accuracy*) a las integrales de las PDE sobre las celdas de la malla. A partir de ello, estima la incertidumbre relativa en las variables solución y compara ésta con la tolerancia de precisión. Si cualquier celda de la malla excede esta tolerancia, la celda se divide y se recalcula la solución.

Este error de tolerancia se llama `ERRLIM` y tiene un valor por defecto de 0.002. Esto significa que FlexPDE refina la malla hasta que el error estimado en cualquier variable (referido al rango de dicha variable) es menor que el 0.2 % sobre cualquier celda de la malla. Esto no significa que FlexPDE garantice que la solución tiene una precisión del 0.2 % sobre el dominio, ya que los errores en celdas individuales se pueden cancelar o acumular en modos difíciles de predecir. El usuario puede establecer un nuevo valor de `ERRLIM` en la sección `SELECT` del *script*.

5.7.8. Salida gráfica

La salida gráfica se puede especificar tanto en la sección `MONITORS`, como en la sección `PLOTS`.

- La salida especificada en la sección `MONITORS` se muestra durante el proceso de ejecución de la simulación y no se almacena en disco.
- Por otro lado, la salida especificada en `PLOTS` se produce sólo cuando FlexPDE ha terminado el proceso de mallado y resolución, y se cumple que la solución en todas las celdas del mallado está dentro de una determinada tolerancia. Esta salida se guarda en disco en un fichero con extensión *.pg6*. En la Tabla 5.6 se muestran algunas de las salidas gráficas que permite especificar FlexPDE.

5.7.9. Problemas en una dimensión

FlexPDE trata los problemas en una dimensión como un caso degenerado de problema de dos dimensiones. Para describir un problema en una dimensión se siguen las mismas pautas que en el caso de dos dimensiones, con las siguientes particularidades:

Tabla 5.6: Algunas de las salidas gráficas existentes en FlexPDE.

Nombre en FlexPDE	Descripción
CONTOUR(arg)	Mapa de contorno bidimensional del argumento arg , con niveles en intervalos uniformes del argumento.
CONTOUR (arg1, arg2)	Mapa de contorno bidimensional de los argumentos arg1 y arg2 , con niveles en intervalos uniformes independientes. Se muestra una tabla de nivel para arg1 y otra para arg2 .
ELEVATION (arg1 [, arg2, ...]) path	Gráfico bidimensional que muestra el valor de sus argumentos verticalmente y el valor del camino (path) horizontalmente. Cada ELEVATION debe tener al menos un argumento y puede tener varios argumentos separados por comas. El path puede ser cualquier segmento lineal, el cual es especificado de la forma FROM (x1, y1) TO (x2, y2) o de la forma ON name donde name se corresponde con un path definido en la sección BOUNDARIES y etiquetado con name. Ejemplos: ELEVATION(...) FROM (x1,y1) TO (x2,y2) ELEVATION(...) ON "outer_boundary"
TABLE (arg1 [, arg2, ...])	Exporta la lista de valores de los argumentos en formato tabular ASCII. La salida será de dos o tres dimensiones, siguiendo el actual sistema de coordenadas o los modificadores ON. Salvo que se emplee el modificador FILE, la salida se escribe al fichero <problema>_<secuencia>.tbl
VECTOR(vector)	Muestra flechas en un gráfico bidimensional, siendo la dirección y magnitud de las flechas las del argumento vector . El origen de cada flecha se coloca en su punto de referencia.
VECTOR(arg1, arg2)	Muestra un gráfico bidimensional con flechas, siendo su componente horizontal y vertical las dadas por arg1 y arg2 . El origen de cada flecha se coloca en su punto de referencia.

- La especificación de las coordenadas en la sección COORDINATES sólo puede ser CARTESIAN1, CYLINDER1 o SPHERE1.
- Las posiciones se especifican dando valor a un punto que tiene una única coordenada. Por ejemplo:

START(0) LINE TO (5)

- El camino de contorno es el dominio, así que no debería cerrarse en sí mismo.
- Las condiciones de contorno se especifican en los extremos del dominio, en lugar de especificarse a lo largo de camino. Se emplean las cláusulas POINT VALUE y POINT LOAD para especificar las condiciones de contorno en los extremos.

Por ejemplo:

START(0) POINT VALUE(u) = 0
LINE TO (5) POINT LOAD(u) = 1

- Las únicas salidas gráficas permitidas son ELEVATION e HISTORY.

5.8. ECUACIÓN DE TRANSFERENCIA DE CALOR

La ecuación del calor es una PDE parabólica que describe la distribución de calor (o variación de temperatura) en una región a lo largo del tiempo. La ecuación de calor en un medio homogéneo, sin fuentes de calor, en tres dimensiones, es la siguiente:

$$\frac{\partial temp}{\partial t} = \alpha \cdot \nabla^2 temp \quad (5.73)$$

donde $temp(\vec{r}, t)$ es la temperatura y α es la difusividad térmica.

En el estacionario, el término $\frac{\partial temp}{\partial t}$ es cero y se obtiene la ecuación siguiente:

$$\alpha \cdot \nabla^2 temp = 0 \quad (5.74)$$

La ecuación de calor, cuando existen fuentes de calor, es una PDE no homogénea que tiende a la expresión siguiente:

$$\frac{\partial temp}{\partial t} - \alpha \cdot \nabla^2 temp = f(\vec{r}, t) \quad (5.75)$$

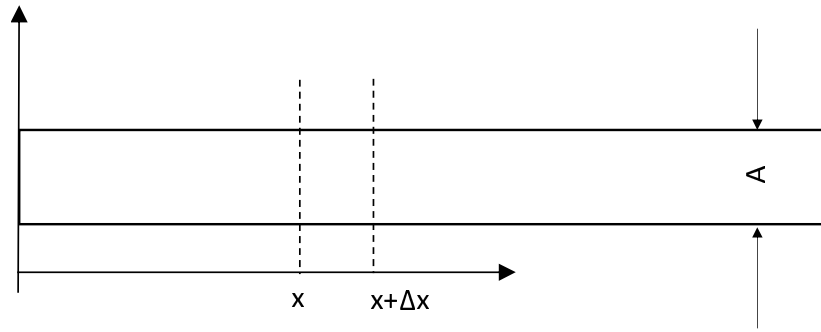


Figura 5.8: Conducción de calor a lo largo de una varilla.

donde $f(\vec{r}, t)$ representa la tasa a la cual se añade energía calorífica al material y sus unidades son de energía por unidad de tiempo y por unidad de volumen.

A continuación vamos a derivar la ecuación del calor en una dimensión partiendo de principios físicos. Para ello consideramos una varilla delgada, homogénea (densidad ρ constante), de longitud L , sección transversal A , con calor específico c constante y conductividad térmica κ constante. En la Figura 5.8 se muestra la varilla bajo estudio, colocada a lo largo del eje x .

Para derivar la ecuación, vamos a emplear los tres principios siguientes:

1. La energía calorífica (Q) de un cuerpo con propiedades uniformes es:

$$Q = m \cdot c \cdot temp \quad (5.76)$$

donde m es la masa del cuerpo y c es su calor específico.

2. Ley de transferencia de calor de Fourier. Esta ley empírica establece que el flujo de calor resultante de la conducción térmica es proporcional a la magnitud del gradiente de temperatura y opuesto a su signo.

$$\frac{1}{A} \cdot \frac{\partial Q}{\partial t} = -\kappa \cdot \frac{\partial temp}{\partial x} \quad (5.77)$$

3. Conservación de la energía.

Consideramos que la temperatura de la varilla depende únicamente de su posición x , siendo la misma en todos los puntos de su sección transversal. Denominamos $temp(x, t)$ a la función de la temperatura, que depende del tiempo t y la posición x .

Vamos a calcular la transferencia de calor en un elemento de la varilla comprendido entre las secciones ubicadas en x y $x + \Delta x$. Teniendo en cuenta el principio de conservación de la energía, la transferencia de calor será el flujo térmico que entra en dicho elemento, menos el flujo que sale del mismo.

El flujo calorífico que entra al elemento se calcula aplicando la Ec. (5.77), obteniéndose:

$$\frac{\partial Q}{\partial t} = -\kappa \cdot A \cdot \frac{\partial temp(x, t)}{\partial x} \quad (5.78)$$

y análogamente se calcula el flujo calorífico saliente del elemento:

$$\frac{\partial Q}{\partial t} = -\kappa \cdot A \cdot \frac{\partial temp(x + \Delta x, t)}{\partial x} \quad (5.79)$$

Por tanto, el flujo neto en el elemento de la varilla, flujo entrante menos saliente, será el siguiente:

$$\frac{\partial Q}{\partial t} = \kappa \cdot A \cdot \left[\frac{\partial temp(x + \Delta x, t)}{\partial x} - \frac{\partial temp(x, t)}{\partial x} \right] \quad (5.80)$$

Por otro lado, calculando la derivada temporal de la Ec. (5.76) se obtiene:

$$\frac{\partial Q}{\partial t} = m \cdot c \cdot \frac{\partial temp}{\partial t} \quad (5.81)$$

Combinando la Ec. (5.80) y la Ec. (5.81), se obtiene:

$$\kappa \cdot A \cdot \left[\frac{\partial temp(x + \Delta x, t)}{\partial x} - \frac{\partial temp(x, t)}{\partial x} \right] = m \cdot c \cdot \frac{\partial temp}{\partial t} \quad (5.82)$$

Sustituyendo en la Ec. (5.82) la expresión siguiente y reordenando los términos

$$m = \rho \cdot \Delta x \cdot A \quad (5.83)$$

obtenemos la ecuación siguiente:

$$\frac{\kappa}{\rho \cdot c} \cdot \frac{\frac{\partial temp(x + \Delta x, t)}{\partial x} - \frac{\partial temp(x, t)}{\partial x}}{\Delta x} = \frac{\partial temp}{\partial t} \quad (5.84)$$

Para obtener la ecuación del calor unidimensional, se calcula el límite cuando $\Delta x \rightarrow 0$ de la Ec. (5.84). Se obtiene la ecuación siguiente:

$$\frac{\partial temp}{\partial t} = \alpha \cdot \frac{\partial^2 temp}{\partial x^2} \quad (5.85)$$

donde el coeficiente $\alpha = \frac{\kappa}{\rho \cdot c}$ se denomina difusividad térmica.

A continuación se muestra la ecuación del calor, generalizada al caso de dos y de tres dimensiones, en coordenadas cartesianas:

$$\frac{\partial temp}{\partial t} = \alpha \cdot \left(\frac{\partial^2 temp}{\partial x^2} + \frac{\partial^2 temp}{\partial y^2} \right) \quad (5.86)$$

$$\frac{\partial temp}{\partial t} = \alpha \cdot \left(\frac{\partial^2 temp}{\partial x^2} + \frac{\partial^2 temp}{\partial y^2} + \frac{\partial^2 temp}{\partial z^2} \right) \quad (5.87)$$

Obsérvese que la ecuación del calor puede expresarse empleando el operador laplaciano (Δ o ∇^2), que en coordenadas rectangulares es: $\Delta = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2}$.

Para encontrar una solución a la Ec. (5.85), es necesario especificar las condiciones inicial y de frontera.

- La condición inicial es la distribución de temperatura en el instante inicial. En el caso unidimensional es:

$$temp(x, 0) = f(x)$$

- Las condiciones de frontera indican las restricciones que han de satisfacer la función de la temperatura en los bordes o en las fronteras de la región. Estas condiciones de frontera podrían ser en el caso unidimensional las siguientes:

$$\begin{cases} u(0, t) = T_1 \\ u(L, t) = T_2 \end{cases} \quad \text{o bien} \quad \begin{cases} \left(\frac{\partial temp}{\partial x} \right)_{x=0} = flujoQ_{x=0} \\ \left(\frac{\partial u}{\partial x} \right)_{x=L} = flujoQ_{x=L} \end{cases}$$

donde $flujoQ_x$ representa la energía calorífica que abandona la varilla en el punto x . Si los extremos de la varilla están aislados, entonces en ambos extremos dicho flujo es 0.

5.8.1. Ejemplo 1: transmisión de calor en una varilla rectangular

Una varilla rectangular, de longitud 1 m y anchura 0.1 m, con conductividad térmica $\kappa = 1 \text{ W}/(\text{K}\cdot\text{m})$, está aislada y tiene una temperatura en sus extremos de 400 K y de 300 K, respectivamente. Se pretende mostrar un gráfico vectorial del flujo de calor a lo largo de la varilla y un mapa de contorno de la distribución de temperatura en la varilla. La especificación del problema se muestra en el Código 5.1, en la Figura 5.9 la malla generada por FlexPDE, y la salida gráfica en la Figura 5.10.

```

TITLE 'Flujo de calor'           {Ejemplo 1 de flujo de calor }
COORDINATES cartesian2        !Sistema de coordenadas
VARIABLES
    temp           !temperatura
DEFINITIONS           !parametros
    k = 1          !Conductividad termica
    L = 1          !Longitud del material
    W = 0.1        !Anchura del material
EQUATIONS
    temp:div(-k*grad(temp))=0    !Ec de calor sin fuentes de calor
BOUNDARIES           { Definicion del dominio }
    REGION 1
        START(0,0)
        NATURAL(temp)=0 LINE TO (L,0)    !Flujo de Q 0
        VALUE(temp) = 300 LINE TO (L,W)
        NATURAL (temp) = 0 LINE TO (0,W) !Flujo de Q 0
        VALUE(temp) =400 LINE TO CLOSE
PLOTS                 !Salida grafica
    CONTOUR(temp)
    VECTOR(-k*GRAD(temp))
END
    
```

Código 5.1: Transmisión de calor en una varilla rectangular.

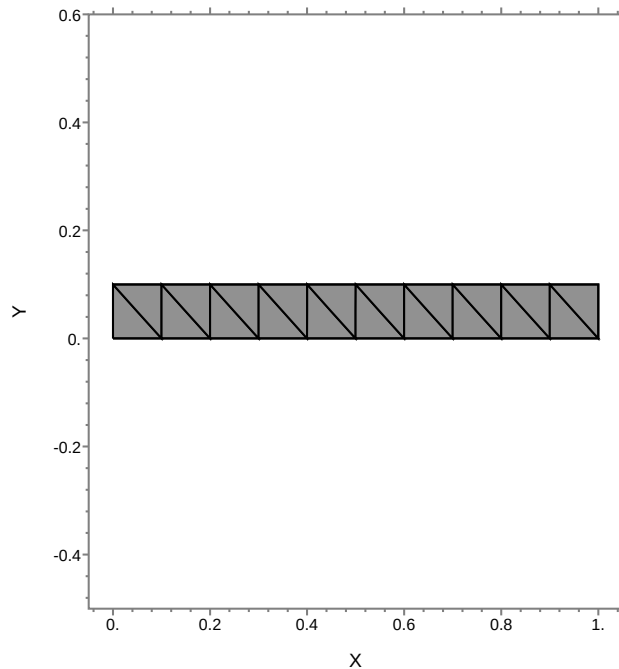
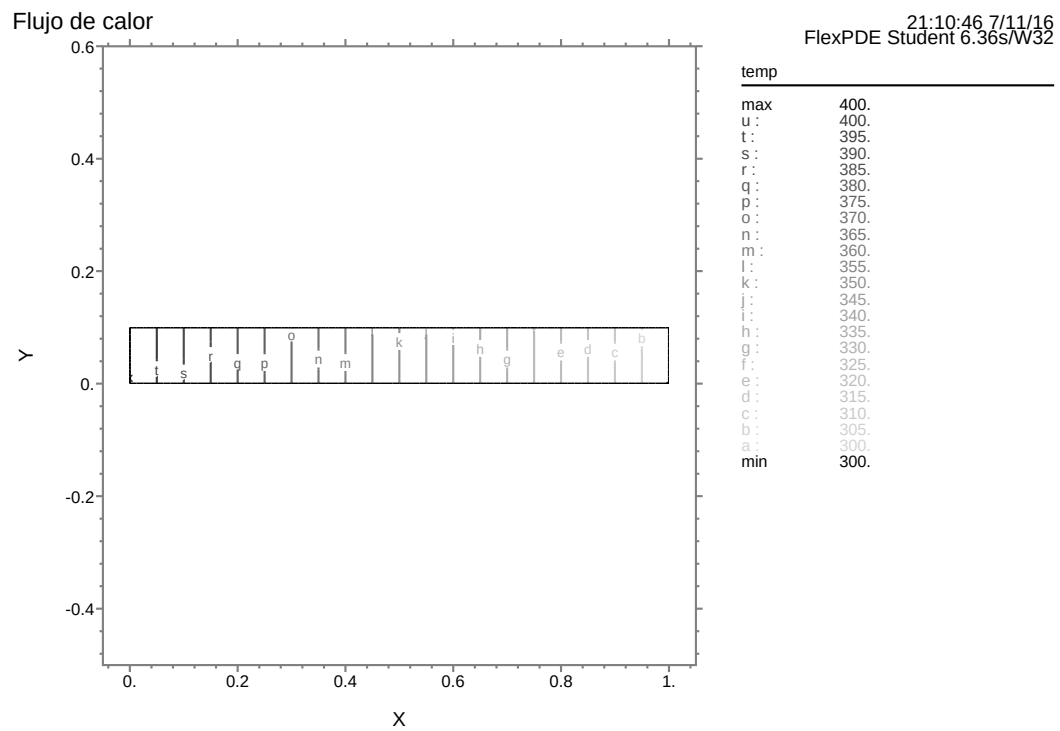
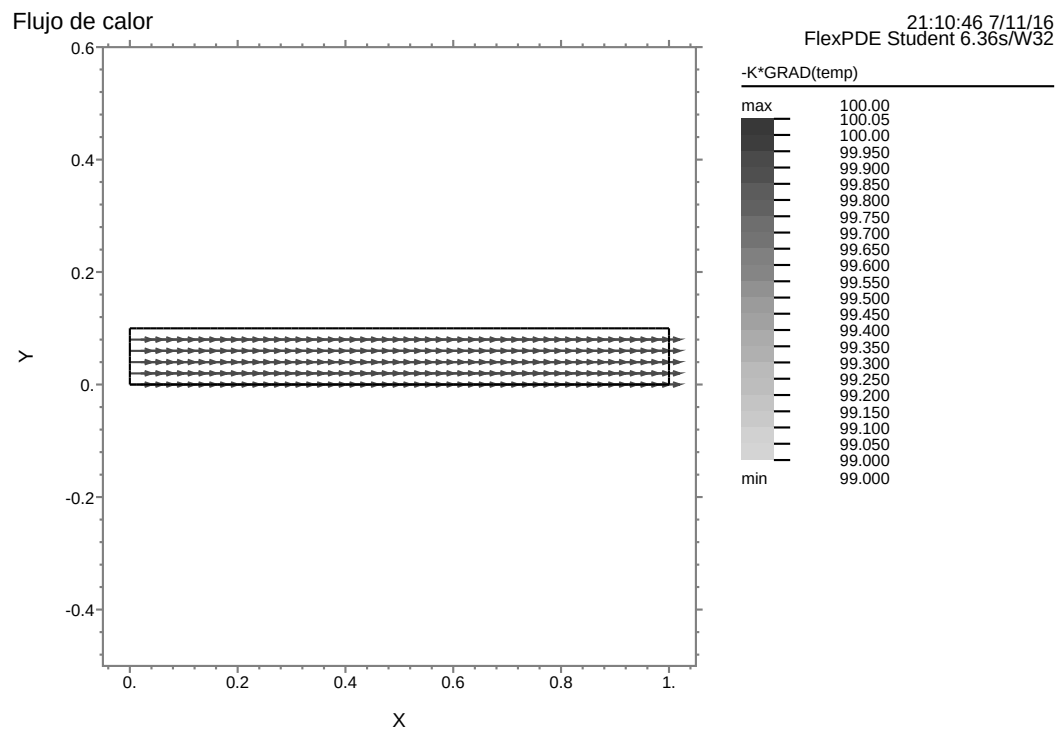


Figura 5.9: Malla generada por FlexPDE para la transmisión de calor en una varilla rectangular.



HeatFlow1: Grid#1 P2 Nodes=63 Cells=20 RMS Err= 2.e-15
Integral= 35.00000



HeatFlow1: Grid#1 P2 Nodes=63 Cells=20 RMS Err= 2.e-15

Figura 5.10: Gráfico de distribución de temperatura (arriba) y flujo de calor (abajo) el ejemplo sobre el estudio de la transmisión de calor en una varilla rectangular.

5.8.2. Ejemplo 2: transmisión de calor en una superficie cuadrada

Un material cuadrado aislado, de lado 1 m, conductividad térmica $\kappa = 1 \text{ W}/(\text{K}\cdot\text{m})$, tiene una temperatura de 400 K y 300 K en sus extremos. En su centro hay un círculo de diámetro 0.5 m hecho de un material aislante, de conductividad térmica $\kappa = 0.001 \text{ W}/(\text{K}\cdot\text{m})$. Se pretende mostrar un gráfico vectorial del flujo de calor y un mapa de contorno de la distribución de temperatura. La descripción se muestra en el Código 5.2. En las Figuras 5.11 y 5.12 se muestra el mallado y la salida gráfica.

```

TITLE 'Flujo de calor'      {Ejemplo21 de flujo de calor }
COORDINATES cartesian2    !Sistema de coordenadas
VARIABLES
    temp      !temperatura
DEFINITIONS      !parametros
    k = 1      !Conductividad termica
    R = 1/4    !Radio del circulo interno
    L = 1      !Lado del cuadrado
EQUATIONS
    temp:div(-K*grad(temp))=0 !Ec de calor sin fuentes de calor
BOUNDARIES      { Definicion del dominio }
    REGION 1      'cuadrado'
        START(0,0)
        NATURAL(temp)=0 LINE TO (L,0) !Flujo de Q 0
        VALUE(temp) = 300 LINE TO (L,L)
        NATURAL (temp) = 0 LINE TO (0,L) !Flujo de Q 0
        VALUE(temp) =400 LINE TO CLOSE
    REGION 2 'circulo'
        k = 0.001
        START 'ring' (L/2-R,L/2)
        NATURAL(temp) = 0 ARC(CENTER=L/2,L/2) ANGLE=360 TO CLOSE
PLOTS      !Salida grafica
    CONTOUR(temp)
    VECTOR(-K*GRAD(temp))
END
    
```

Código 5.2: Transmisión de calor en una superficie cuadrada.

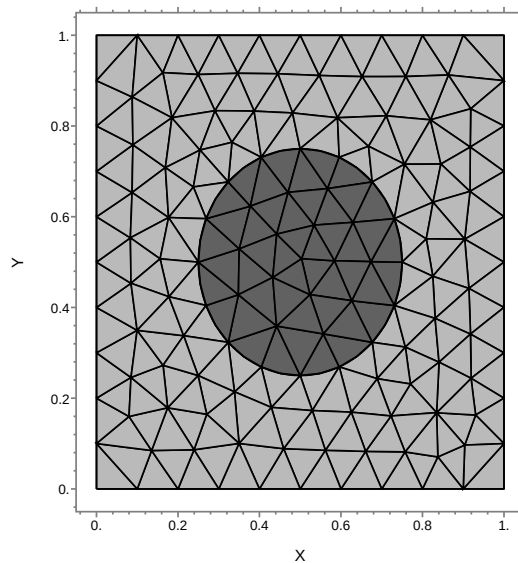
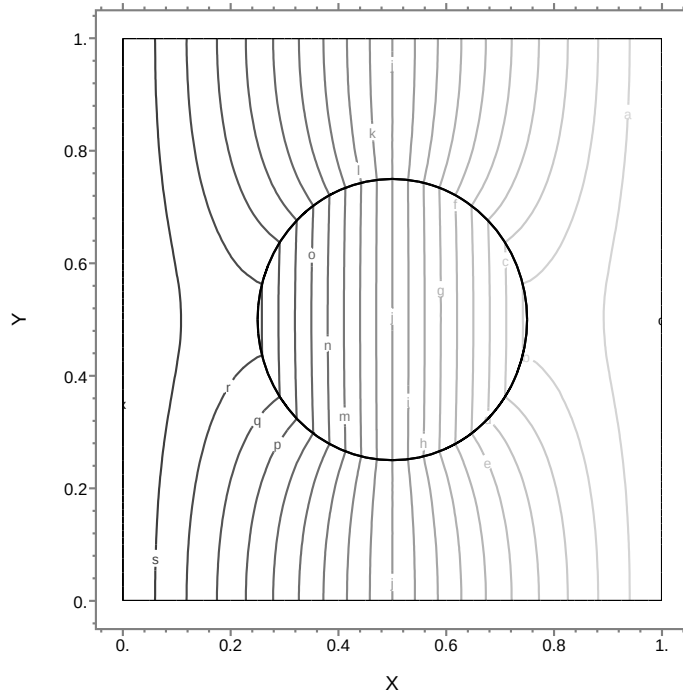


Figura 5.11: Malla generada por FlexPDE.

Flujo de calor



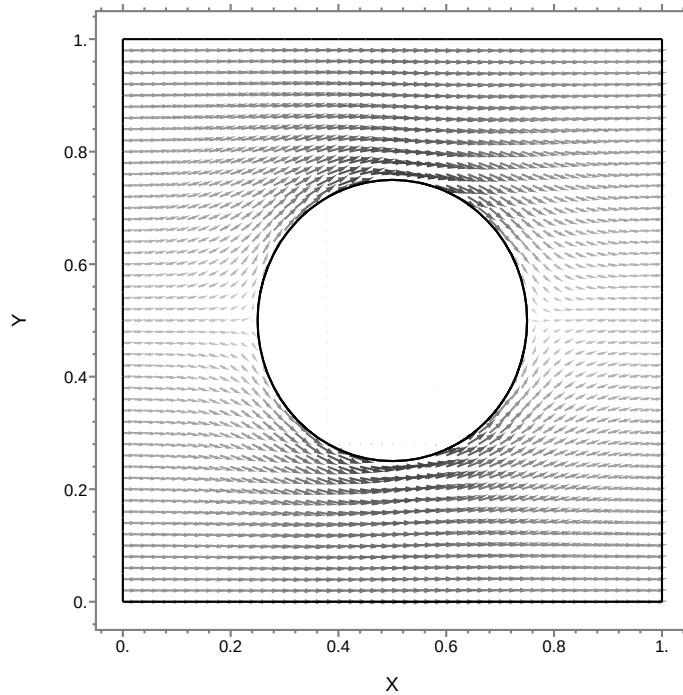
21:50:43 7/11/16
FlexPDE Student 6.36s/W32

temp

max	400.
t:	400.
s:	395.
r:	390.
q:	385.
p:	380.
o:	375.
n:	370.
m:	365.
l:	360.
k:	355.
j:	350.
i:	345.
h:	340.
g:	335.
f:	330.
e:	325.
d:	320.
c:	315.
b:	310.
a:	305.
min	300.

HeatFlow2: Grid#1 P2 Nodes=521 Cells=240 RMS Err= 8.5e-4
Integral= 349.9999

Flujo de calor



21:50:43 7/11/16
FlexPDE Student 6.36s/W32

-K*GRAD(temp)

max	168.
	170.
	160.
	150.
	140.
	130.
	120.
	110.
	100.
	90.0
	80.0
	70.0
	60.0
	50.0
	40.0
	30.0
	20.0
	10.0
	0.00
min	0.16

HeatFlow2: Grid#1 P2 Nodes=521 Cells=240 RMS Err= 8.5e-4

Figura 5.12: Gráfico de distribución de temperatura (arriba) y flujo de calor (abajo) en la transmisión de calor en una superficie cuadrada que tiene un círculo interno aislante.

5.9. ECUACIÓN DE ONDAS

La ecuación de ondas es el prototipo de PDE hiperbólica de segundo orden. Esta ecuación permite describir la propagación de ondas, tales como las ondas de sonido, de luz y de agua. Aparece en diferentes campos, como la acústica, electromagnetismo y dinámica de fluidos. La ecuación de ondas en una dimensión tiene la expresión siguiente, donde c es una constante:

$$\frac{\partial^2 u(x, t)}{\partial t^2} = c^2 \cdot \Delta u(x, t) \quad (5.88)$$

Históricamente, el problema de una cuerda vibrante como la de un instrumento musical fue estudiada por Jean Le Rond d'Alembert, Leonhard Euler, Daniel Bernoulli y Joseph-Louis Lagrange. En 1746 d'Alembert formuló la ecuación de ondas en una dimensión y diez años después Euler formularía la ecuación de ondas en tres dimensiones.

La ecuación de ondas no homogénea es la siguiente:

$$c^2 \cdot \Delta u(x, t) - \frac{\partial^2 u(x, t)}{\partial t^2} = s(x, t) \quad (5.89)$$

siendo c una constante y donde la función $s(x, t)$, llamada frecuentemente la **función fuente**, describe los efectos de las fuentes de onda en el medio que las transmite. Por ejemplo, en una cuerda la función fuente sería la fuerza que mueve dicha cuerda.

A continuación vamos a obtener la ecuación de ondas homogénea en una dimensión, para una varilla delgada elástica, partiendo de principios físicos, de forma análoga a como se dedujo la ecuación del calor. La varilla tiene longitud L , sección transversal constante A y densidad ρ . Llamamos $u(x, t)$ al desplazamiento de la varilla, que depende únicamente de la posición x y del instante de tiempo t . Para derivar la PDE, vamos a considerar las fuerzas en un pequeño segmento de longitud Δx de la varilla, en un instante t (véase la Figura 5.13).

Para derivar la ecuación vamos a emplear los tres principios siguientes:

1. La tensión (σ) es igual a la fuerza por unidad de superficie:

$$\sigma = \frac{F}{A} \quad (5.90)$$

donde A es la superficie donde se aplica la fuerza.

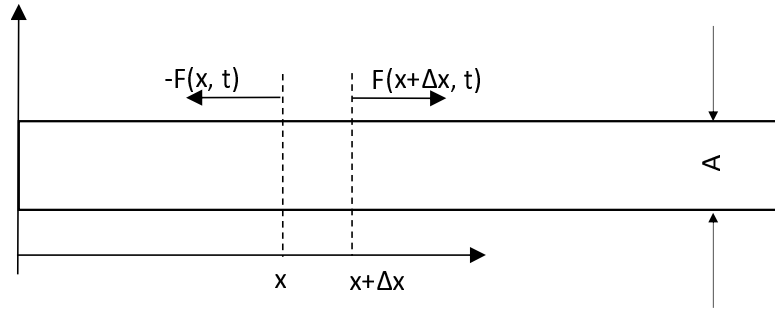


Figura 5.13: Varilla delgada elástica donde se transmiten ondas.

2. Ley de Hooke. Esta ley empírica relaciona la deformación de una barra sometida a un esfuerzo axial con la tensión normal generada por dicho esfuerzo:

$$\sigma(x, t) = E \cdot \frac{\partial u(x, t)}{\partial x} \quad (5.91)$$

donde E se denomina módulo elástico y depende las propiedades elásticas del material, y $u(x, t)$ es la elongación natural de la varilla.

3. Ley de equilibrio, que establece que la tasa de cambio de momento es igual a la fuerza neta:

$$F = \frac{d(m \cdot v)}{dt} \quad (5.92)$$

donde m es la masa del cuerpo, v la velocidad del cuerpo y F la fuerza neta.

Aplicando la Ec. (5.92) al segmento de la varilla, obtenemos:

$$\frac{d}{dt} \left(\rho \cdot A \cdot \Delta x \cdot \frac{\partial u(x, t)}{\partial t} \right) = F(x + \Delta x, t) - F(x, t) \quad (5.93)$$

donde se ha tenido en cuenta que:

$$m = \rho \cdot A \cdot \Delta x \quad (5.94)$$

$$v = \frac{\partial u(x, t)}{\partial t} \quad (5.95)$$

Aplicando la Ec. (5.90), se obtiene la ecuación siguiente:

$$\frac{d}{dt} \left(\rho \cdot A \cdot \Delta x \cdot \frac{\partial u(x, t)}{\partial t} \right) = \sigma(x + \Delta x, t) \cdot A - \sigma(x, t) \cdot A \quad (5.96)$$

Introduciendo la Ec. (5.91) en la Ec. (5.96), obtenemos:

$$\frac{d}{dt} \left(\rho \cdot A \cdot \frac{\partial u(x, t)}{\partial t} \right) = \frac{\frac{\partial u(x+\Delta x, t)}{\partial x} \cdot E \cdot A - \frac{\partial u(x, t)}{\partial x} \cdot E \cdot A}{\Delta x} \quad (5.97)$$

Haciendo el límite cuando $\Delta x \rightarrow 0$ de la Ec. (5.97), se obtiene:

$$\frac{\partial^2 u(x, t)}{\partial t^2} = \frac{\partial^2 u(x, t)}{\partial x^2} \cdot \frac{E}{\rho} = c^2 \cdot \frac{\partial^2 u(x, t)}{\partial x^2} \quad (5.98)$$

La velocidad de ondas c está relacionada con el módulo elástico E y la densidad de la varilla ρ a través de la expresión siguiente:

$$c = \sqrt{\frac{E}{\rho}} \quad (5.99)$$

5.9.1. Ejemplo 1: cuerda fija en sus dos extremos

Supongamos una cuerda homogénea de longitud $L = 1$ m, situada en el plano $X - Y$. La cuerda está fija en sus dos extremos y tiene un valor de velocidad de onda $c = 522$ m/s. En el instante inicial $t = 0$, la cuerda tiene la forma de parábola descrita por la ecuación $u = h \cdot x \cdot (L - x)$, donde $h = 0.01$ y tiene una velocidad inicial nula.

Este problema es equivalente a resolver la siguiente PDE, donde la variable dependiente u representa el desplazamiento de la cuerda en la dirección y :

$$\frac{\partial^2 u}{\partial t^2} - c^2 \cdot \frac{\partial^2 u}{\partial x^2} = 0 \quad \text{en } \Omega = \left\{ x, t : \begin{array}{l} 0 \leq x \leq L \\ t \geq 0 \end{array} \right\} \quad (5.100)$$

con las condiciones de contorno siguientes:

$$u(x = 0, t) = 0 \quad u(x = 1, t) = 0 \quad (5.101)$$

y las condiciones iniciales siguientes:

$$u(x, t = 0) = h \cdot x \cdot (L - x) \qquad \frac{\partial u(x, t = 0)}{\partial t} = 0 \qquad (5.102)$$

FlexPDE sólo puede tratar derivadas temporales de primer orden. Las ecuaciones diferenciales con derivadas temporales de segundo orden se han de separar en dos ecuaciones empleando una variable intermedia. Para poder describir este problema en FlexPDE definimos una variable intermedia v , que es la derivada temporal de la variable u y llegamos al siguiente sistema de PDE:

$$\begin{aligned} \frac{\partial v}{\partial t} - c^2 \cdot \frac{\partial^2 u}{\partial x^2} &= 0 \\ v &= \frac{\partial u}{\partial t} \end{aligned} \quad \text{en } \Omega = \left\{ x, t : \begin{array}{l} 0 \leq x \leq L \\ t \geq 0 \end{array} \right\} \qquad (5.103)$$

La descripción en FlexPDE de este problema se muestra en el Código 5.3. Se ha empleado la cláusula **THRESHOLD** para establecer el rango mínimo de valores de las variables para el cual FlexPDE debería mantener la precisión establecida.

```

TITLE 'Vibraciones en una cuerda'
COORDINATES cartesian1  !Coordenadas 1 dimension
VARIABLES
  u(THRESHOLD = 1e-5)  !Posicion vertical cuerda
  v (THRESHOLD = 1e-4)  !Derivada de u
DEFINITIONS
  c= 522  !Velocidad de onda
  h = 0.01
  L = 1  !Longitud de la cuerda
  Tf = 0.003814  !Periodo fundamental (2L)/c
INITIAL VALUES
  u = h*x*(L-x)  !Forma parabola
  v = 0
EQUATIONS
  u:dt(v)-c^2*dx(x,u)=0
  v: v =dt(u)
BOUNDARIES
REGION 1
  START(0) POINT VALUE(u) = 0
  LINE TO (L) POINT VALUE(u) = 0
TIME 0 TO 0.04
PLOTS
FOR T= 0
  ELEVATION(u) from (0) to (L)
FOR T= 3*Tf/2
  ELEVATION(u) from (0) to (L)
END

```

Código 5.3: Cuerda homogénea fija en sus extremos.

5.9.2. Ejemplo 2: cuerda fija sólo en un extremo

Vamos a modificar el anterior ejemplo, de modo que el extremo derecho de la cuerda se pueda mover libremente en la dirección vertical. Tenemos entonces el siguiente sistema de PDE:

$$\frac{\partial^2 u}{\partial t^2} - c^2 \cdot \frac{\partial^2 u}{\partial x^2} = 0 \quad \text{en } \Omega = \left\{ x, t : \begin{array}{l} 0 \leq x \leq L \\ t \geq 0 \end{array} \right\} \quad (5.104)$$

que transformamos al siguiente sistema de PDE para que sólo existan derivadas temporales de primer orden:

$$\begin{aligned} \frac{\partial v}{\partial t} - c^2 \cdot \frac{\partial^2 u}{\partial x^2} &= 0 \\ v &= \frac{\partial u}{\partial t} \end{aligned} \quad \text{en } \Omega = \left\{ x, t : \begin{array}{l} 0 \leq x \leq L \\ t \geq 0 \end{array} \right\} \quad (5.105)$$

con las siguientes condiciones de contorno:

$$u(x=0, t) = 0 \quad (5.106)$$

$$\frac{\partial u(x=L, t)}{\partial t} = 0 \quad (5.107)$$

y las siguientes condiciones iniciales:

$$u(x, t=0) = h \cdot x \cdot (L - x) \quad (5.108)$$

$$\frac{\partial u(x, t=0)}{\partial t} = 0 \quad (5.109)$$

$$\frac{\partial u(x=L, t=0)}{\partial t} = 0 \quad (5.110)$$

donde $L = 1$ m, $c = 522$ m/s y $h = 0.01$.

La descripción en FlexPDE de este problema se muestra en el Código 5.4. Se ha empleado la cláusula `THRESHOLD` para establecer el rango mínimo de valores de las variables para el cual FlexPDE debería mantener la precisión establecida.

```

TITLE 'Vibraciones en una cuerda'
COORDINATES cartesian1  !Coordenadas 1 dimension
VARIABLES
  u(THRESHOLD = 1e-4)  !Posicion vertical cuerda
  v (THRESHOLD = 1e-5) !Derivada de u
DEFINITIONS
  c= 522 !Velocidad de onda
  h = 0.01
  L = 1 !Longitud de la cuerda
  Tf = 0.007663
INITIAL VALUES
  u = h*x*(L-x) !Forma parabola
  v = 0
EQUATIONS
  u:dt(v)-c^2*dxx(u)=0
  v: v =dt(u)
BOUNDARIES
REGION 1
  START(0) POINT VALUE(u) = 0
  LINE TO (L) POINT LOAD(u) = 0
TIME 0 TO 0.01
PLOTS
FOR T= 0
  ELEVATION(u) from (0) to (L)
FOR T= Tf*0.5
  ELEVATION(u) from (0) to (L)
END

```

Código 5.4: Cuerda homogénea fija sólo en un extremo.

5.10. ECUACIÓN DE LAPLACE

Las ecuaciones de Laplace y Poisson son ejemplos de ecuaciones en derivadas parciales elípticas. Estas ecuaciones, en su forma diferencial, son:

$$\nabla^2 u(x) = 0 \quad \text{Ec. Laplace} \quad (5.111)$$

$$\nabla^2 u(x) = f(x) \quad \text{Ec. Poisson} \quad (5.112)$$

Las aplicaciones de estas ecuaciones incluyen la teoría clásica de campos de potencial y el flujo estacionario de fluidos incompresibles. En electrostática, la función $f(x)$ de la ecuación de Poisson representa una distribución de densidad de carga induciendo un potencial eléctrico $u(x)$. En los flujos de fluidos estacionarios en dos dimensiones $u(x)$ es la velocidad potencial o función de corriente.

Las soluciones de las ecuaciones de Laplace son funciones armónicas de gran importancia en muchos campos de la ciencia, como en el electromagnetismo, la astronomía y la dinámica de fluidos y describen con precisión el comportamiento de los potenciales eléctrico, gravitacional y fluido. En el estudio de la conducción de calor, la ecuación de Laplace representa la ecuación del calor en estado estacionario.

A continuación vamos a derivar la ecuación de Poisson de la electrostática a partir de los siguientes principios físicos.

1. En un medio lineal, isotrópico y homogéneo, la ecuación constitutiva que relaciona la intensidad del campo eléctrico con el desplazamiento eléctrico es:

$$\vec{D} = \epsilon \cdot \vec{E} \quad (5.113)$$

siendo ϵ la permitividad eléctrica del medio.

2. La *ley de Faraday* dice que en ausencia de un campo magnético variable en el tiempo, el rotacional del campo eléctrico es 0. En consecuencia, se puede definir un potencial eléctrico φ cuyo gradiente es el campo eléctrico:

$$\nabla \times \vec{E} = -\frac{\partial \vec{B}}{\partial t} = 0 \implies \vec{E} = -\nabla \varphi \quad (5.114)$$

3. La *ley de Gauss* de la electricidad en forma diferencial relaciona la densidad de carga libre (ρ_f) con la divergencia del vector desplazamiento eléctrico:

$$\nabla \cdot \vec{D} = \rho_f \quad (5.115)$$

Combinando las Ecs. (5.113) y (5.115), se obtiene:

$$\nabla \cdot \vec{E} = \frac{\rho_f}{\epsilon} \quad (5.116)$$

Reemplazando $\vec{E}$ teniendo en cuenta la Ec. (5.114), se obtiene la ecuación de Poisson:

$$\nabla \cdot (-\nabla \varphi) = \frac{\rho_f}{\epsilon} \implies \nabla^2 \varphi = \Delta \varphi = -\frac{\rho_f}{\epsilon} \quad (5.117)$$

Si la densidad de carga libre (ρ_f) es cero, se obtiene la ecuación de Laplace.

A continuación, proponemos un par de ejemplos en 2D de electrostática. Es decir, trataremos con problemas de potencial en sistemas que consideraremos indefinidos, por ejemplo según z , eje a lo largo del cual el sistema se supone tiene simetría de traslación. Utilizaremos, caso de no indicarse otra cosa, unidades del sistema internacional: m para las longitudes, F/m para las permitividades eléctricas absolutas, etc.

5.10.1. Ejemplo 1: condensador 2D relleno con 2 dieléctricos

Supongamos que el condensador está formado por dos electrodos paralelos, situados en $y = 0$ e $y = 1$, de longitud 1 m. El inferior, situado en $y = 0$ y que supondremos se extiende desde $x = 0$ hasta $x = 1$, está a 0 V. El superior está a 100 V. Supondremos que no hay efecto bordes y que el campo en $x = 0$ y $x = 1$ está dirigido verticalmente. Esto equivale a establecer condiciones de contorno homogéneas de tipo Neumann en dichos bordes.

El cuadrado definido entre $(0, 0)$ y $(1, 1)$ está relleno por un medio de permitividad 1, excepto en un rectángulo, que va desde $(0.4, 0.4)$ hasta $(0.8, 0.6)$, que tiene una permitividad de 50.

Se propone efectuar la solución de este problema utilizando el entorno FlexPDE, obteniendo las líneas equipotenciales y el diagrama de campo eléctrico.

La descripción en FlexPDE de este problema se muestra en el Código 5.5.

En la Figura 5.14 se muestran el mallado generado por FlexPDE para este problema.

En la Figura 5.15 se muestran los gráficos de equipotenciales y líneas de campo. Se observa que el dieléctrico de alta permitividad se comporta casi como un metal, con campo eléctrico interior muy pequeño.

```

TITLE 'Ejemplo condensador 2D'
COORDINATES cartesian2
VARIABLES
    V !Potencial electrico
DEFINITIONS !Definicion de parametros
    eps = 1 !Permitividad del medio
EQUATIONS
    div(eps*grad(V))=0 { one possibility }
BOUNDARIES !Definicion del dominio
    REGION 1 !Material con eps = 1
        START(0,0)
        VALUE(V) = 0 LINE TO (1,0) !Potencial 0V
        NATURAL(V) = 0 LINE TO (1,1)
        VALUE(V) = 100 LINE TO (0,1) !Potencial 100V
        NATURAL(V) = 0 LINE TO CLOSE
    REGION 2 !Material con eps = 50
        eps = 50
        START (0.4, 0.4)
        LINE TO (0.8, 0.4)
        LINE TO (0.8, 0.6)
        LINE TO (0.4, 0.6)
        LINE TO CLOSE
PLOTS { save result displays }
    CONTOUR(V) !Potencial
    VECTOR(-dx(V), -dy(V)) !Campo electrico
END
    
```

Código 5.5: Condensador 2D relleno con 2 dieléctricos.

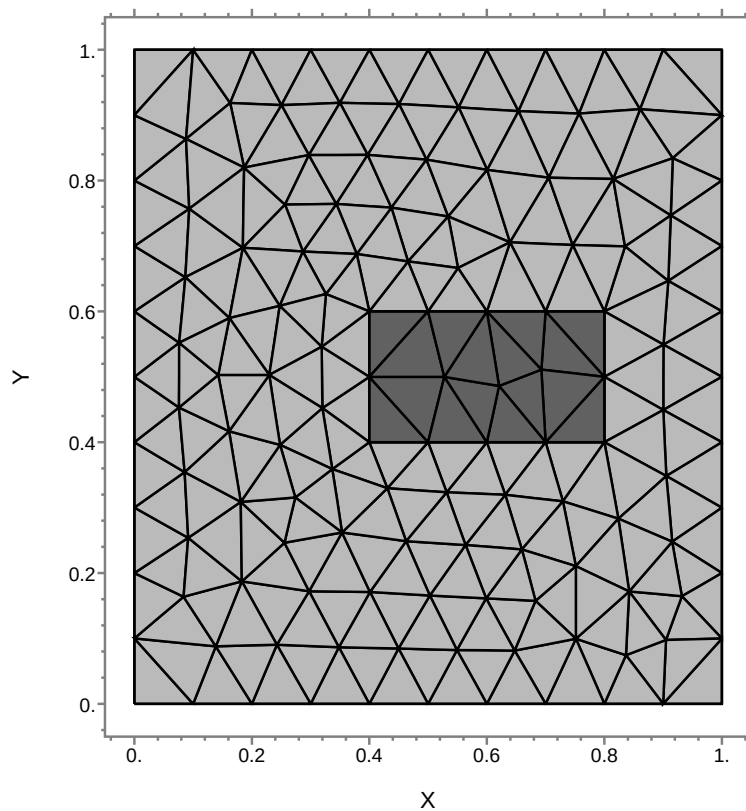
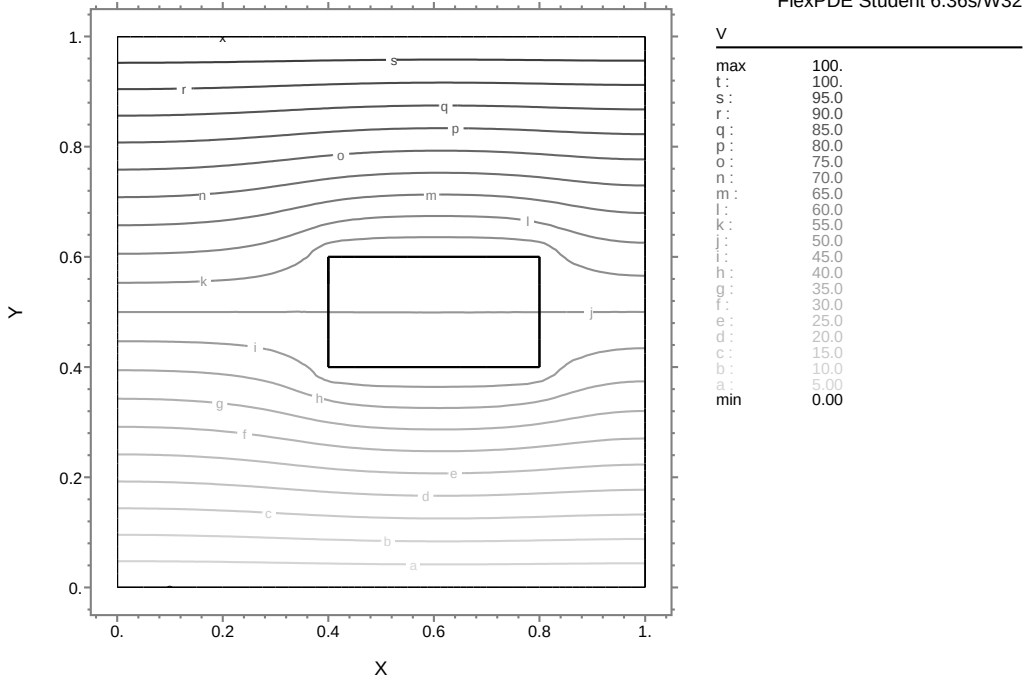


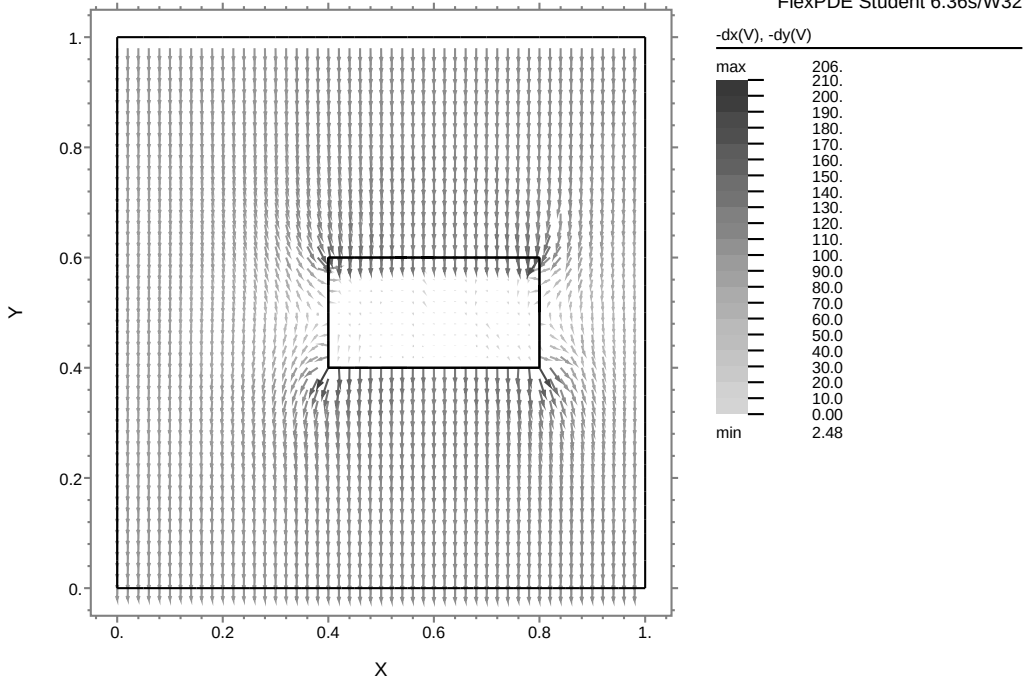
Figura 5.14: Malla generada por FlexPDE en el ejemplo del condensador 2D relleno con 2 dieléctricos diferentes.

Ejemplo 1 Ec. Laplace



dielectrico1: Grid#2 P2 Nodes=537 Cells=248 RMS Err= 0.0018
Integral= 50.00026

Ejemplo 1 Ec. Laplace



dielectrico1: Grid#2 P2 Nodes=537 Cells=248 RMS Err= 0.0018

Figura 5.15: Gráfico de equipotenciales (arriba) y vectorial del campo eléctrico (abajo) correspondiente al ejemplo del condensador 2D relleno con 2 dieléctricos diferentes.

5.10.2. Ejemplo 2: cable coaxial de sección exterior rectangular

Supongamos un cable coaxial cuya sección exterior es rectangular y cuya sección interior es circular. La sección exterior es un cuadrado metálico, de lado 2 m, conectado a 0 V. La sección interior es un círculo metálico centrado, de radio 0.3 m, y conectado a 10 V. El medio entre ambos conductores tiene permitividad 1.

Se propone efectuar la solución de este problema utilizando el entorno FlexPDE, obteniendo las líneas equipotenciales y el diagrama de campo eléctrico.

El Código 5.6 muestra el script realizado en FlexPDE para resolver este problema. La malla generada automáticamente por este entorno se muestra en la Figura 5.16.

Los gráficos obtenidos se muestran en la Figura 5.17. Se observa que el campo eléctrico es perpendicular a la superficie de los conductores, en las proximidades externas de los mismos. En el interior de los conductores no es preciso calcular el potencial, pues daría la solución trivial de potencial constante (campo eléctrico nulo).

```

TITLE 'Cable coaxial'
SELECT
    errlim=1e-3 !Control de precision
    gray !Escala gris de colores
VARIABLES
    U
DEFINITIONS
    eps = 1 !Permitividad
    L=1 !Semilongitud caja
    r0=0.3 !Radio varilla
    U_varilla=10 !Potencial varilla
    Ex=-dx(U) Ey=-dy(U) E=-grad(U) Em=magnitudo(E)
EQUATIONS
    div( eps*grad( U))=0
BOUNDARIES
    REGION 'dominio'
    START 'caja' (-L,-L) VALUE( U)= 0 { Caja Metalica }
    LINE TO (L,-L) TO (L,L) TO (-L,L) CLOSE
    START 'varilla' (r0,0) { Varilla }
    VALUE(U)=U_varilla ARC( CENTER=0,0) ANGLE=360
PLOTS
    CONTOUR(U) !Potencial
    VECTOR(-dx(U), -dy(U)) !Campo electrico
END

```

Código 5.6: Cable coaxial de sección exterior rectangular.

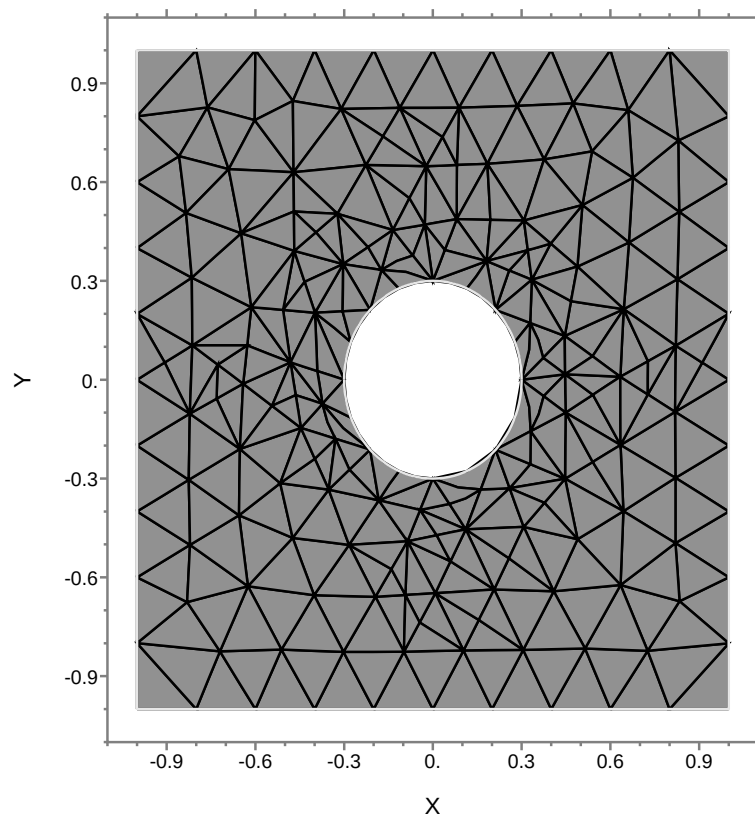
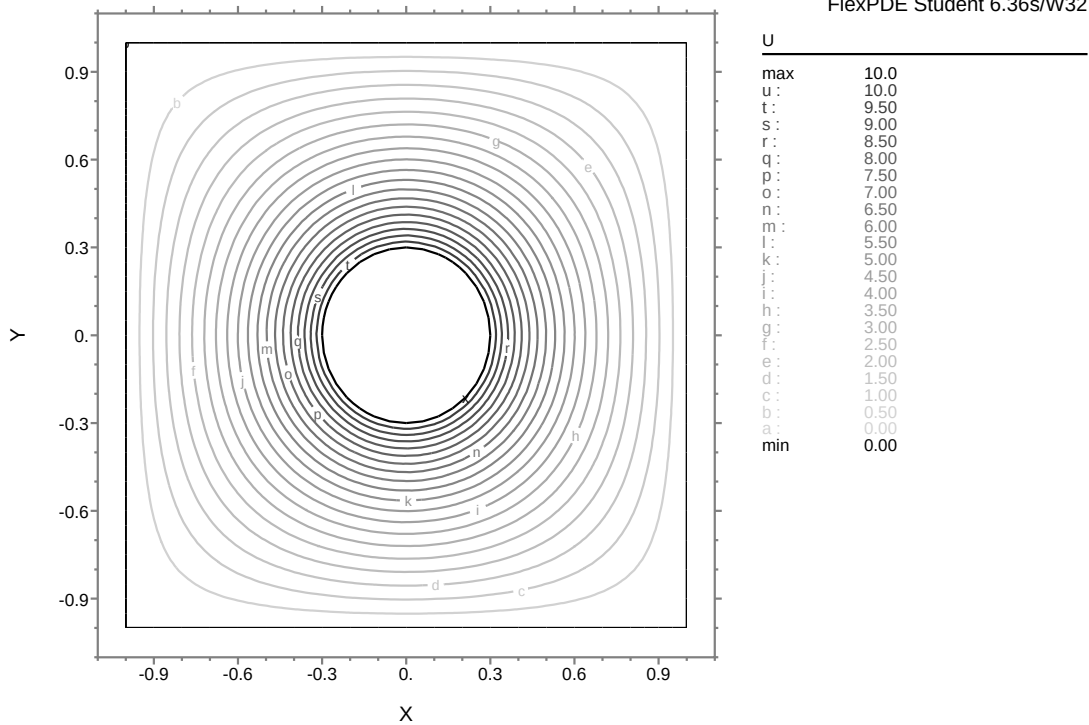


Figura 5.16: Malla generada por FlexPDE en el ejemplo del cable coaxial.

Varilla metalica en caja

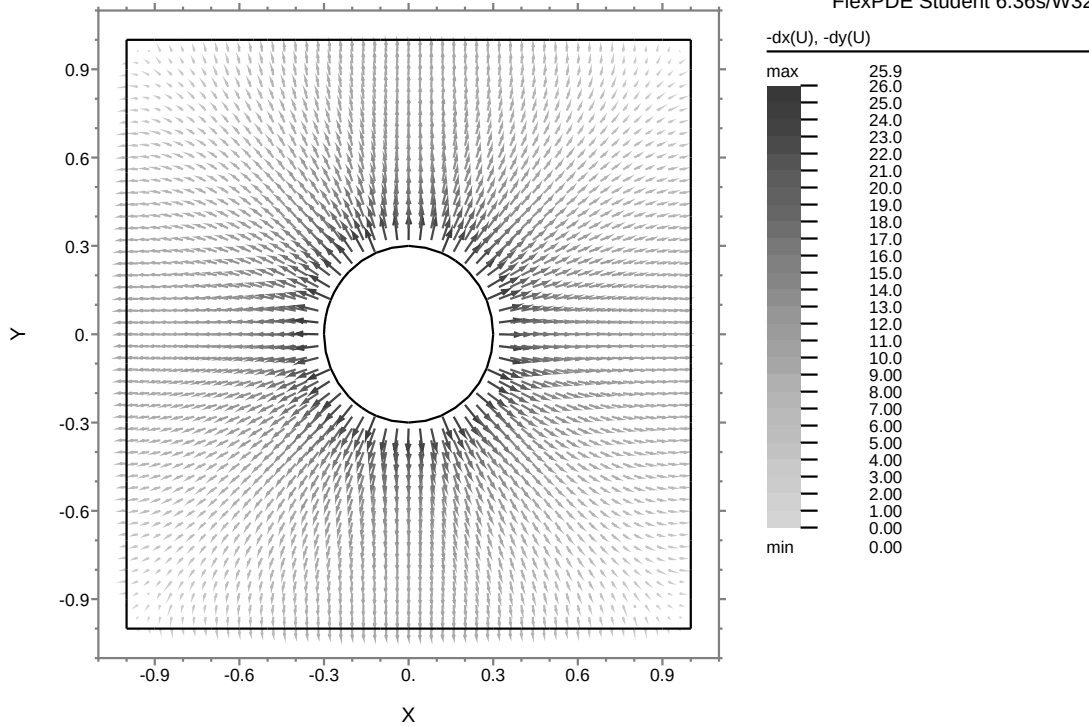
19:13:14 7/11/16
FlexPDE Student 6.36s/W32



ejemplo2Laplace: Grid#2 P2 Nodes=710 Cells=328 RMS Err= 9.e-4
Integral= 10.55077

Varilla metalica en caja

19:13:14 7/11/16
FlexPDE Student 6.36s/W32



ejemplo2Laplace: Grid#2 P2 Nodes=710 Cells=328 RMS Err= 9.e-4

Figura 5.17: Gráfico de equipotenciales (arriba) y vectorial del campo eléctrico (abajo) del ejemplo del cable coaxial.

5.11. LECTURAS RECOMENDADAS

Se recomienda leer (Gockenbach 2011) para profundizar en el conocimiento de los métodos numéricos aplicados a la resolución de PDE. En este libro se pueden encontrar además multitud de ejemplos donde aparecen PDE en diferentes ámbitos. El método FEM viene escrito de una manera muy didáctica en (Guru & Hizioglu 2009). Hay múltiples ejemplos de aplicación del método de diferencias finitas en el ámbito de los fenómenos de transporte en (Burden & Faires 1985).

Para profundizar en el conocimiento del entorno FlexPDE se recomienda acceder a la documentación, libros gratuitos y ejemplos que se encuentran en su sitio web (Inc. 2016).

5.12. EJERCICIOS DE AUTOCOMPROBACIÓN

Ejercicio 5.1

Clasifique cada una de las siguientes dos ecuaciones diferenciales en elíptica, hiperbólica o parabólica, e indique también si es homogénea o inhomogénea.

$$4 \cdot \frac{\partial^2 u}{\partial x^2} + 2 \cdot \frac{\partial u}{\partial x} + \frac{\partial u}{\partial y} + x + y = 0 \quad (5.118)$$

$$e^x \cdot \frac{\partial^2 u}{\partial x^2} + \cos(y) \cdot \frac{\partial^2 u}{\partial x \partial y} - \frac{\partial^2 u}{\partial y^2} = 0 \quad (5.119)$$

Ejercicio 5.2

Se desea determinar la distribución estacionaria de temperatura (*temp*) de una lámina cuadrada delgada de metal, de dimensión de 0.5 m × 0.5 m. La lámina se mantiene a 0 °C en dos lados adyacentes, mientras que la temperatura en las otras fronteras se va incrementando linealmente de 0 °C en una esquina, a 100 °C donde estos lados se encuentran. La conductividad térmica es 1 W/(m·°C). Este problema se expresa matemáticamente como:

$$\frac{\partial^2 temp}{\partial x^2}(x, y) + \frac{\partial^2 temp}{\partial y^2}(x, y) = 0 \quad \text{en } \Omega = \left\{ x, y : \begin{array}{l} 0 \leq x \leq 0.5 \\ 0 \leq y \leq 0.5 \end{array} \right\} \quad (5.120)$$

con las condiciones de contorno siguientes:

$$temp(x = 0, y) = 0 \quad (5.121)$$

$$temp(x, y = 0) = 0 \quad (5.122)$$

$$temp(x, y = 0.5) = 200 \cdot x \quad (5.123)$$

$$temp(x = 0.5, y) = 200 \cdot y \quad (5.124)$$

Escriba un script para obtener un mapa de contorno de la distribución de temperatura empleando FlexPDE. Nótese que la temperatura no se puede llamar T, al estar ese nombre reservado para la variable tiempo.

Ejercicio 5.3

Consideremos el problema en dos dimensiones de un cable coaxial, de sección exterior circular y sección interior rectangular. Supongamos que el interior es un cuadrado metálico, de lado 0.6 m, conectado a 0 V. El exterior es un círculo metálico, centrado de radio 1 m y conectado a 10 V. El medio entre ambos conductores tiene permitividad 1.

Escriba un script en FlexPDE para obtener las curvas de nivel del potencial eléctrico y los vectores correspondientes al campo eléctrico.

Ejercicio 5.4

Consideremos el problema de la conducción del calor en una dimensión en una barra de hierro. La barra de hierro está situada sobre el eje X.

Inicialmente todos los puntos de la misma están a 300 K, salvo el extremo izquierdo de la misma que está a 400 K. En $t > 0$ el extremo izquierdo de la barra se mantiene a 400 K, al situar en dicho extremo de la barra un gran bloque de cobre que se mantiene a 400 K.

Este fenómeno se puede describir por la siguiente ecuación:

$$-K \cdot \frac{\partial^2 temp}{\partial x^2} + rcp \cdot \frac{\partial temp}{\partial t} = 0 \quad (5.125)$$

donde $temp$ es la temperatura, $K = 82$ y $rcp = 3.53 \cdot 10^6$, expresado todo ello en unidades del sistema internacional.

Escriba un script en FlexPDE que describa este problema. Realice la simulación entre los instantes 0 y 3000 s, mostrando el gráfico de la distribución de temperatura en el instante $t = 1800$ s.

Ejercicio 5.5

Consideremos nuevamente el problema de la conducción de calor en una dimensión en una barra de hierro, que en esta ocasión se encuentra completamente aislada. La barra de hierro está situada sobre el eje X.

En el instante inicial, la mitad izquierda de la barra está a 200 K y la otra mitad está a 400 K.

Este fenómeno se puede describir por la ecuación siguiente:

$$-K \cdot \frac{\partial^2 temp}{\partial x^2} + rcp \cdot \frac{\partial temp}{\partial t} = 0 \quad (5.126)$$

donde $temp$ es la temperatura, $K = 82$ y $rcp = 3.53 \cdot 10^6$ en unidades del sistema internacional.

Escriba un *script* en FlexPDE que describa este problema.

Para describir la condición inicial se puede usar la función $ustep(a)$ de FlexPDE. Esta función vale 0 para $x < a$ y vale 1 para $x \geq a$.

Realice la simulación entre los instantes 0 y 100000 s, mostrando el gráfico de la distribución de temperatura en el instante $t = 10000$ s.

Ejercicio 5.6

Consideremos un bloque de cobre de longitud 1 m, que está sometido a una diferencia de potencial entre caras opuestas de 0.2 V. En estas caras tiene una temperatura constante de 300 K. Consideramos que las otras fronteras del bloque de cobre están aisladas térmica y eléctricamente.

Este fenómeno se describe mediante el siguiente sistema de dos PDE:

$$\nabla \cdot (-\sigma \cdot \nabla U) = 0 \quad (5.127)$$

$$\nabla \cdot (-K \cdot \nabla temp) = J \cdot E \quad (5.128)$$

donde la densidad de corriente (J) es:

$$J = \sigma \cdot E \quad (5.129)$$

donde $E = -\nabla U$, siendo U el potencial. El flujo de calor (Q) viene dado por la expresión siguiente:

$$Q = -K \cdot \nabla temp \quad (5.130)$$

La conductividad térmica (K) del cobre tiene el valor 400, en unidades del sistema internacional.

La conductividad eléctrica (σ) del cobre tiene la siguiente dependencia con la temperatura (expresada en unidades del sistema internacional):

$$\sigma = \frac{1}{1.67} \cdot \frac{300}{temp} \quad (5.131)$$

Escriba un script en FlexPDE para obtener un gráfico del potencial eléctrico (U), el flujo de calor (Q) y la temperatura ($temp$) a lo largo de la barra.

Ejercicio 5.7

Consideremos una esfera de corcho que está hueca, con radios interior y exterior de 0.02 m y 0.04 m respectivamente. El interior de la esfera se calienta con un flujo de calor de 100 W y el exterior de la esfera se mantiene a 373 K.

La conductividad térmica (K) del corcho tiene el valor 0.2 W/(m·K).

El flujo de calor viene dado por la ecuación siguiente:

$$Q = -K \cdot \nabla temp \quad (5.132)$$

Escriba un script en FlexPDE para obtener un gráfico del flujo de calor (Q) y la temperatura ($temp$) entre el radio interior y exterior de la esfera. Se trata de un problema unidimensional, para el que se recomienda emplear el sistema de coordenadas `sphere1`.

Ejercicio 5.8

Se tiene un cilindro dieléctrico, con permitividad 7, de radio 2 y altura 2, que en el plano $R - Z$ va desde (0,0) hasta (2,1). El cilindro tiene una distribución de carga que viene dada por la ecuación siguiente:

$$\rho_{vg} = 10^{-6} \cdot e^{-5 \cdot r^2 - 5 \cdot z} \quad (5.133)$$

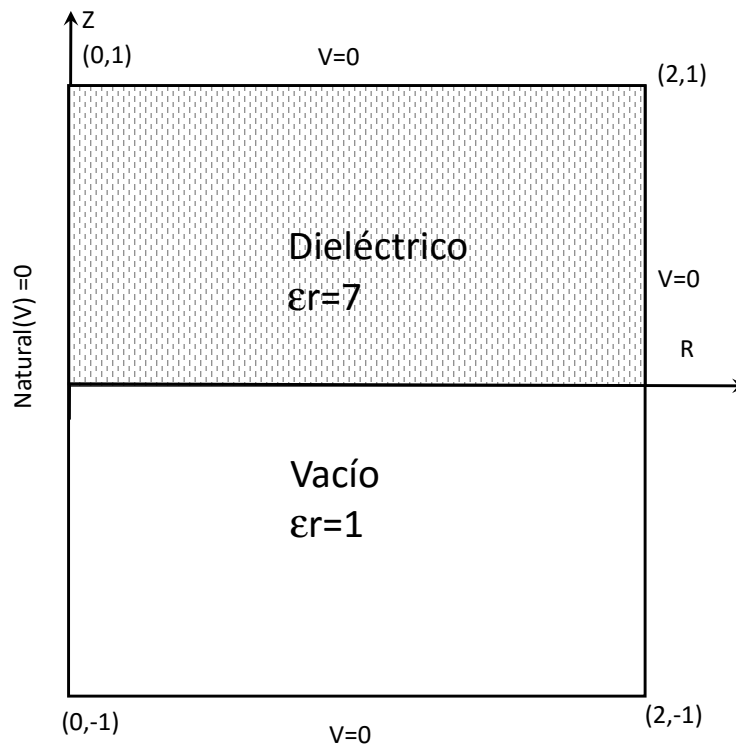


Figura 5.18: Cilindros en el plano $R - Z$.

El cilindro anterior está situado sobre otro de igual geometría, que se extiende desde $(0, -1)$ hasta $(2, 0)$. Este cilindro es vacío, con permitividad 1 y no contiene carga ($\rho = 0$).

La superficie externa del conjunto es metálica y está conectada a potencial 0. Para $r = 0$ el campo va según el eje. En la Figura 5.18 se muestran ambos cilindros en el plano $R - Z$.

La ecuación diferencial que describe este problema es la siguiente:

$$\nabla \cdot D = \rho \quad (5.134)$$

Escriba un script en FlexPDE para obtener la distribución del campo eléctrico y de la densidad de carga en el interior del dieléctrico y del metal. Emplee las coordenadas cilíndricas `ycylinder( 'r', 'z')`.

5.13. SOLUCIONES DE LOS EJERCICIOS

Solución al Ejercicio 5.1

$$4 \cdot \frac{\partial^2 u}{\partial x^2} + 2 \cdot \frac{\partial u}{\partial x} + \frac{\partial u}{\partial y} + x + y = 0 \quad (5.135)$$

es una PDE parabólica e inhomogénea. En esta ecuación $a = 4$, $b = 0$ y $c = 0$. Por tanto, $b^2 - 4 \cdot a \cdot c = 0$ lo que implica que la PDE es parabólica. El término $x + y$ hace que se trate de una ecuación inhomogénea.

$$e^x \cdot \frac{\partial^2 u}{\partial x^2} + \cos(y) \cdot \frac{\partial^2 u}{\partial x \partial y} - \frac{\partial^2 u}{\partial y^2} = 0 \quad (5.136)$$

es una PDE hiperbólica y homogénea. En esta ecuación $a = e^x$, $b = \cos(y)$ y $c = -1$. Por tanto, $b^2 - 4 \cdot a \cdot c = \cos^2(y) + 4 \cdot e^x > 0$ lo que implica que la PDE es hiperbólica.

Solución al Ejercicio 5.2

El *script* se muestra en el Código 5.7 y el gráfico en la Figura 5.19.

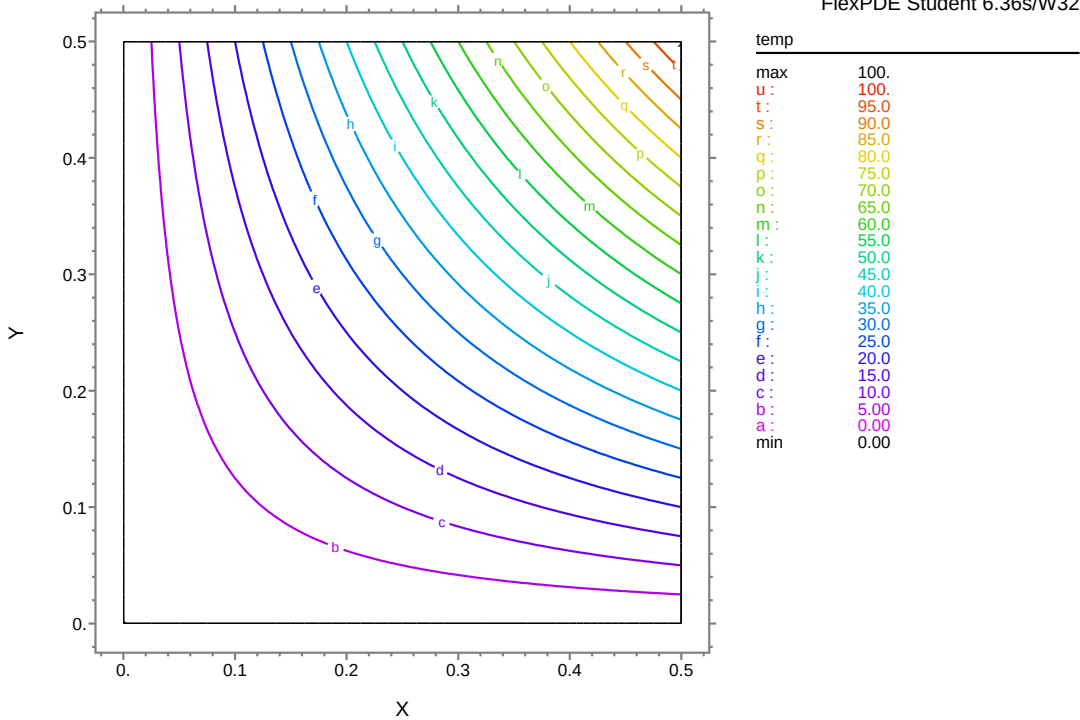
```

TITLE 'Flujo de calor en cuadrado'
COORDINATES cartesian2
VARIABLES
    temp !Temperatura
DEFINITIONS
    tempx = 200*x
    tempy = 200*y
EQUATIONS
    div(grad(temp))=0 !Ec. del calor
BOUNDARIES { Definicion del dominio }
REGION 1 !Hay solo una region
    START(0,0)
    VALUE(temp) = 0 LINE TO (0.5,0)
    VALUE(temp) = tempy LINE TO (0.5,0.5)
    VALUE(temp) = tempx LINE TO (0,0.5)
    VALUE(temp) = 0 LINE TO CLOSE
PLOTS
    CONTOUR(temp)
END

```

Código 5.7: Conducción de calor en un cuadrado metálico.

Flujo de calor en cuadrado



cuadradoT: Grid#1 P2 Nodes=505 Cells=232 RMS Err= 5.e-15
Integral= 6.249935

Figura 5.19: Distribución de temperatura.

Solución al Ejercicio 5.3

Véanse el Código 5.8, y las Figuras 5.20 y 5.21.

```

TITLE 'Cable coaxial'
SELECT
errlim=1e-3 !Control de precision
VARIABLES
U
DEFINITIONS
  eps  = 1 !Permitividad
  L=0.3 !Semilong. lado barra
  r0=1 !Radio tubo
  U_tubo=1 !Potencial tubo
  Ex=-dx(U) Ey=-dy(U)
EQUATIONS
  div( eps*grad( U))=0
BOUNDARIES
REGION 'dominio'
  START 'barra' (-L,-L) VALUE(U)= 0 !barra
  LINE TO (L,-L) TO (L,L) TO (-L,L) CLOSE
  START 'rod' (r0,0) !Tubo
  VALUE(U)=U_tubo ARC( CENTER=0,0) ANGLE=360
PLOTS
  CONTOUR(U) !Potencial
  VECTOR(Ex, Ey) !Campo electrico
END

```

Código 5.8: Cable coaxial de sección exterior circular.

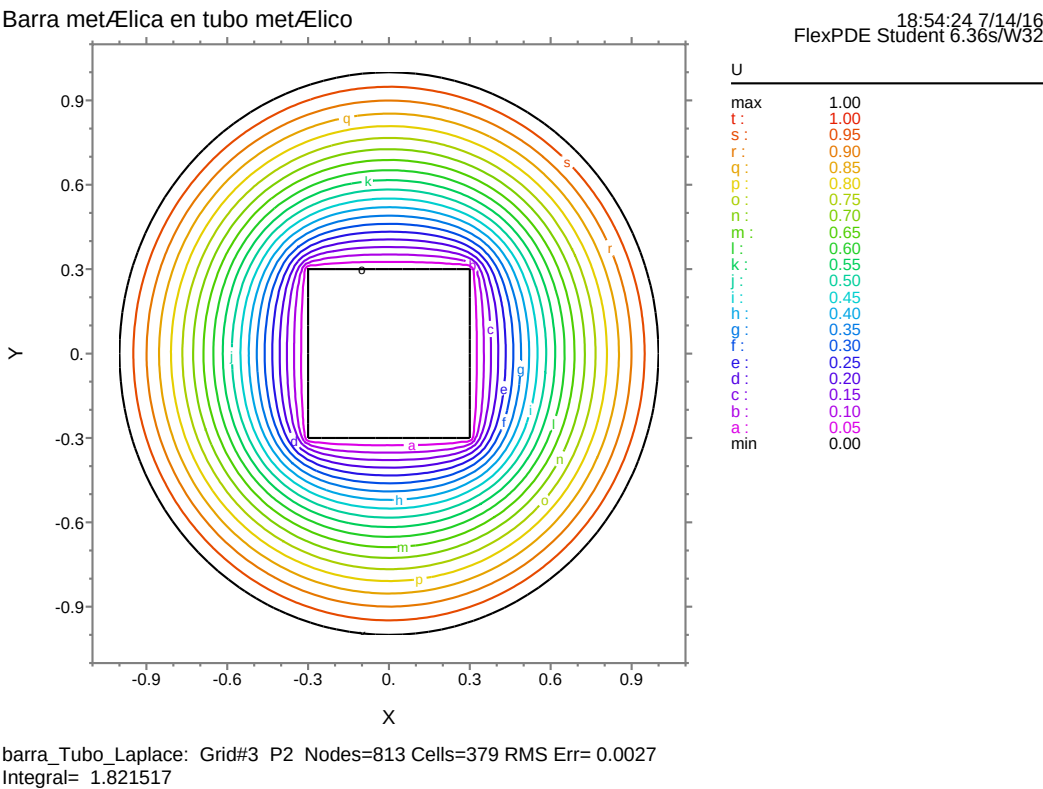


Figura 5.20: Equipotenciales en el cable coaxial.

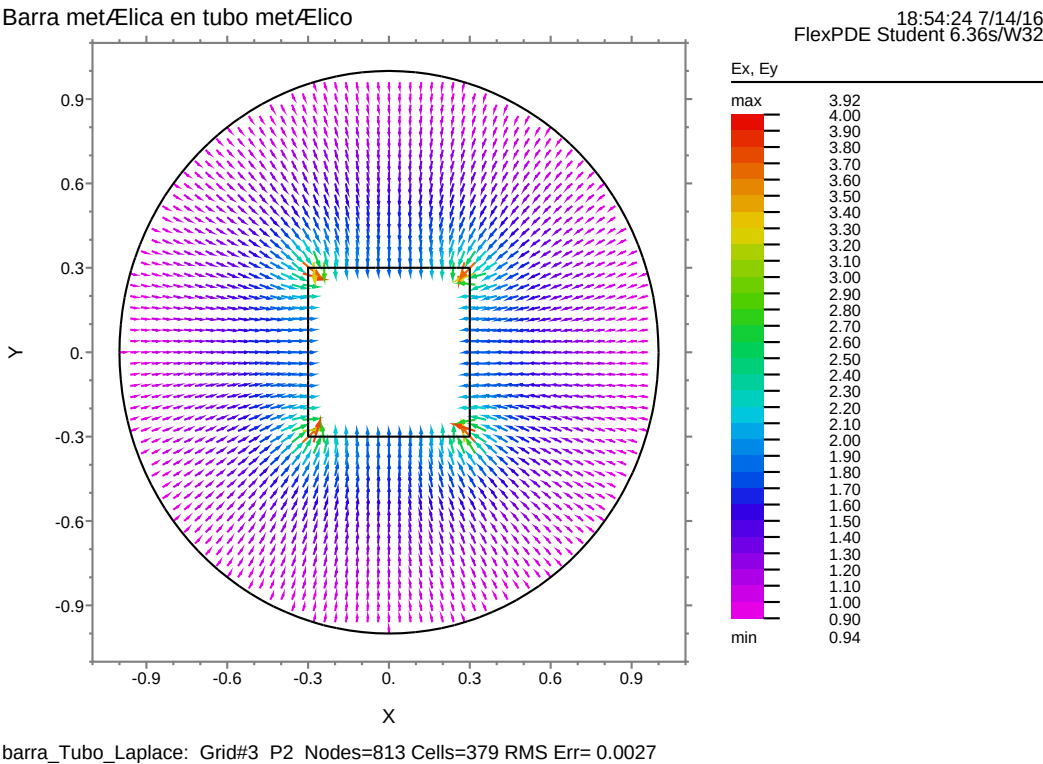


Figura 5.21: Campo eléctrico en el cable coaxial.

Solución al Ejercicio 5.4

El script se muestra en el Código 5.9 y el gráfico en la Figura 5.22.

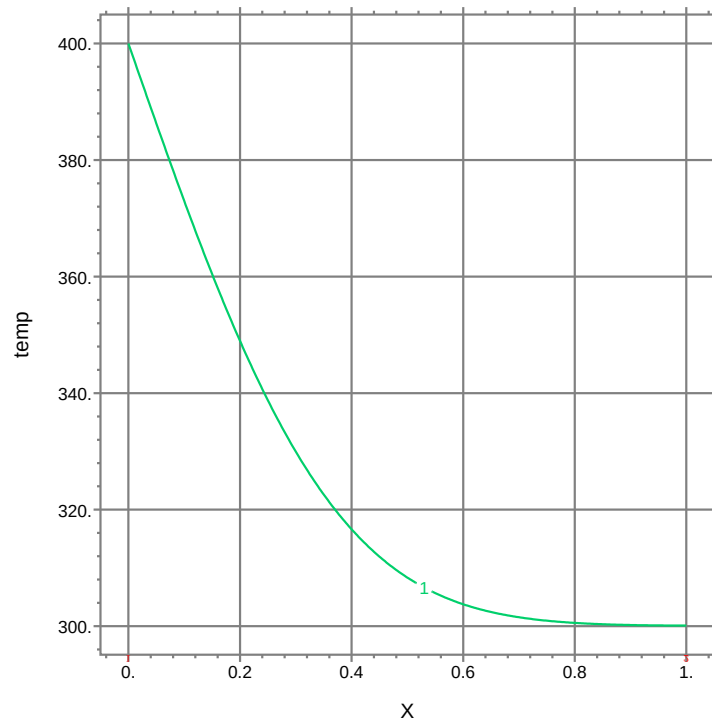
```

TITLE 'Barra de hierro'      { Flujo de calor en una barra de hierro }
SELECT
    errlim=3e-4      spectral_colors
COORDINATES
    cartesian1      { 1D }
VARIABLES    temp
DEFINITIONS
    Lx=1.0      k=-82      rcp=7.87e3*449      { Hierro }
    tempi=300      !Temperatura inicial en t= 0
    temp0=400      !Temperatura de contorno en x = 0
INITIAL VALUES    temp=tempi
EQUATIONS
    k*dxx( temp)+ rcp*dt( temp)=0
BOUNDARIES
    region 'dominio'
START (0) POINT VALUE(temp)=temp0
LINE TO (Lx)
TIME    FROM 0 TO 3000
PLOTS
    FOR t= 1800
    ELEVATION( temp) from (0) to (Lx)
END

```

Código 5.9: Barra de hierro sometida a un salto escalón en la temperatura.

Barra de hierro



21:40:39 7/14/16
FlexPDE Student 6.36s/W32

temp
from (0)
to (Lx)

1: temp

1 2

calorBarra: Cycle=121 Time= 1800.0 dt= 184.14 P2 Nodes=51 Cells=25 RMS Err= 8.e-6
Integral= 323.0704

Figura 5.22: Distribución de temperatura en la barra de hierro en $t = 1800$ s.

Solución al Ejercicio 5.5

El *script* se muestra en el Código 5.10 y el gráfico en la Figura 5.23.

```

TITLE 'Barra de hierro'      { Flujo de calor en una barra de hierro }
SELECT
    errlim=1e-3    spectral_colors
COORDINATES
    cartesian1      { 1D }
VARIABLES    temp
DEFINITIONS
    Lx=1.0    k=-82    rcp=7.87e3*449    { Hierro }
INITIAL VALUES
    temp = 200 + 200*ustep(x-Lx/2)
EQUATIONS
    k*dxx( temp)+ rcp*dt( temp)=0
BOUNDARIES
    region 'dominio'
START (0) POINT LOAD(temp)=0
    LINE TO (Lx)
TIME    FROM 0 TO 100000
PLOTS
    FOR t= 10000
        ELEVATION( temp) from (0) to (Lx)
END

```

Código 5.10: Barra de hierro del Ejercicio 5.5.

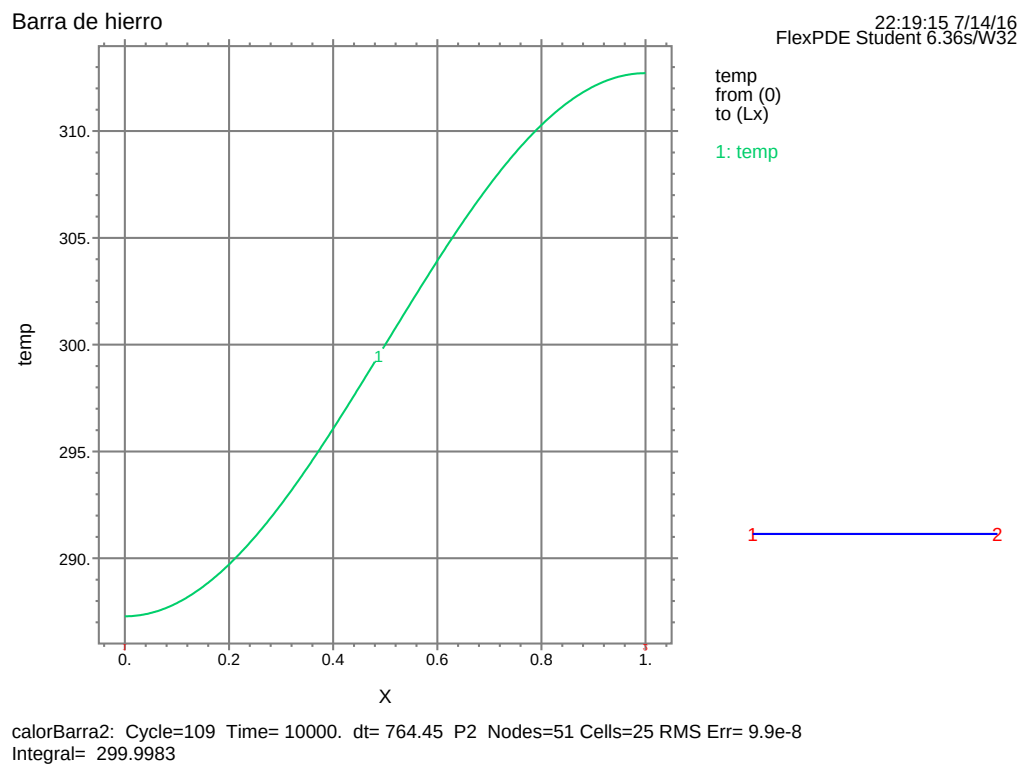


Figura 5.23: Distribución de temperatura en la barra de hierro en $t = 10000$ s.

Solución al Ejercicio 5.6

El script se muestra en el Código 5.11, y los gráficos en las Figuras 5.24 – 5.26.

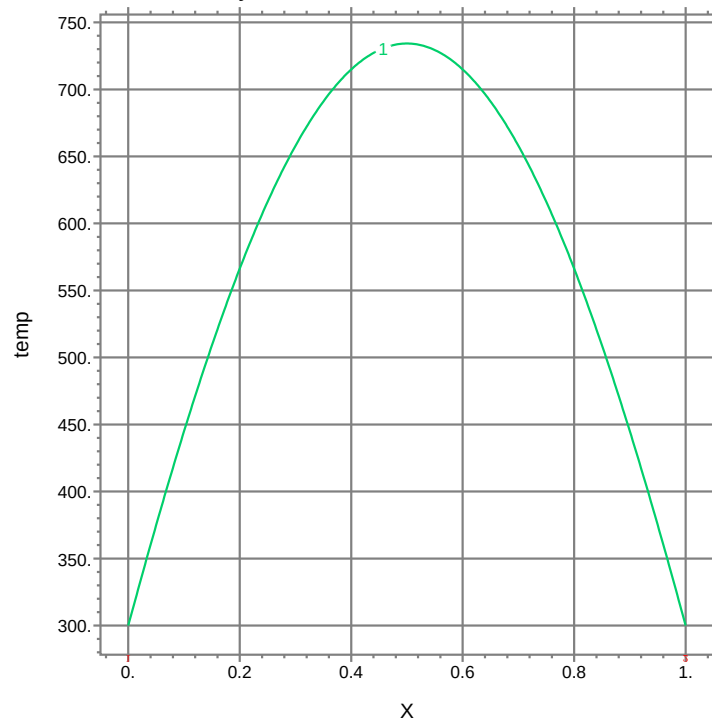
```

TITLE    'Conduccion electrica y termica'
SELECT
    errlim=1e-5      spectral_colors
COORDINATES
    cartesian1      !1 dimension
VARIABLES    U      temp
DEFINITIONS
    Lx=1.0      temp0=300      U1=-0.2
    cond0=1/1.67e-8      cond=cond0*temp0/temp      k=400      !Cobre
    Ex=-dx(U)      Jx=cond*Ex
    fluxd_x=-k*dx( temp)
INITIAL VALUES
    U=U1*x/Lx      temp=300
EQUATIONS
    U:    dx( Jx)= 0      !U es la unica variable dependiente
    temp: dx( fluxd_x)=Jx*Ex      ! A partir de aqui se obtiene temp
BOUNDARIES
    region 'dominio'
    start (0) point value(U)=0    point value(temp)=300
    line to (Lx)    point value(U)=U1    point value(temp)=300
PLOTS
    elevation( temp) from (0) to (Lx)      elevation( fluxd_x) from (0) to (Lx)
    elevation( U) from (0) to (Lx)
END

```

Código 5.11: Conducción eléctrica y térmica en una barra de cobre.

Conduccion electrica y termica



07:53:41 7/18/16
FlexPDE Student 6.36s/W32

temp
from (0)
to (Lx)

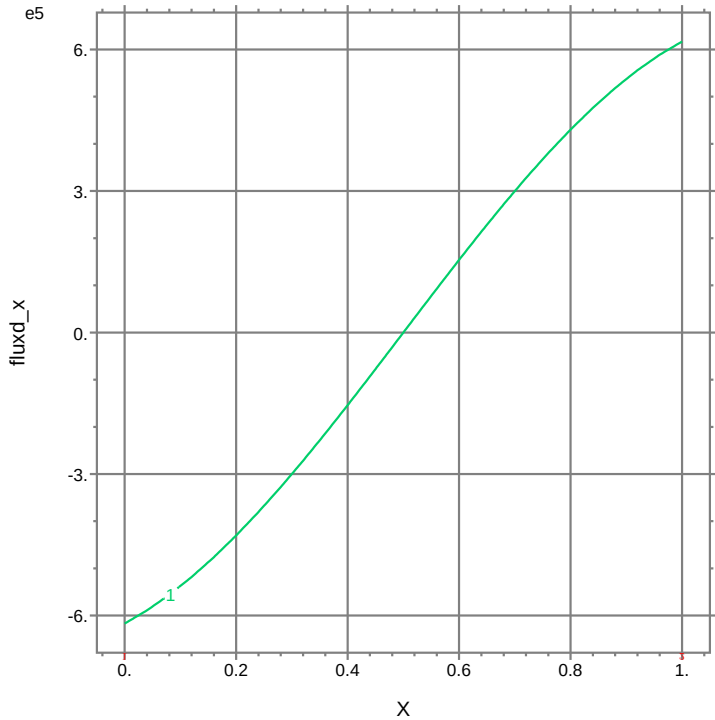
1: temp



condElecTermic: Grid#1 P2 Nodes=51 Cells=25 RMS Err= 3.7e-6
Integral= 582.7940

Figura 5.24: Distribución de temperatura en la barra de cobre.

Conduccion electrica y termica



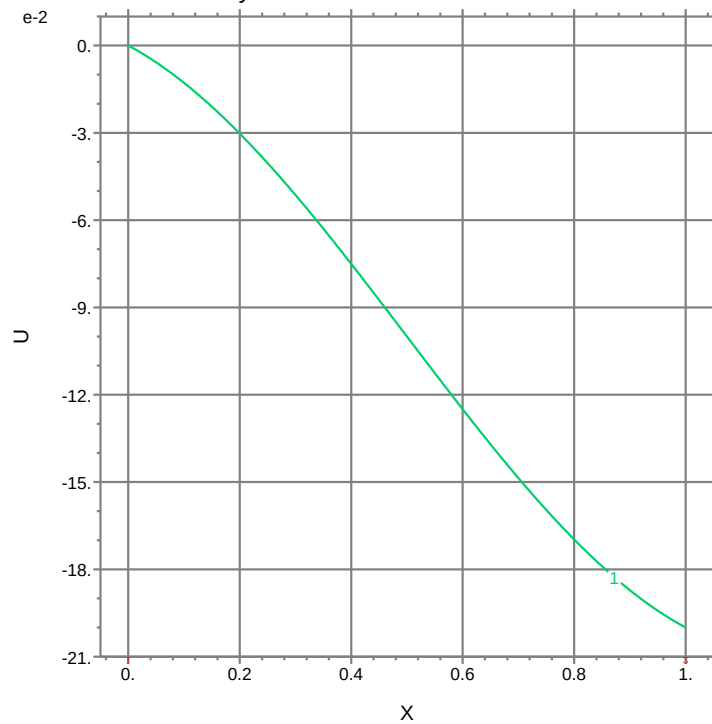
07:53:41 7/18/16
FlexPDE Student 6.36s/W32
fluxd_x
from (0)
to (Lx)
1: fluxd_x



condElecTermic: Grid#1 P2 Nodes=51 Cells=25 RMS Err= 3.7e-6
Integral= -1.277571

Figura 5.25: Distribución de calor en la barra de cobre.

Conduccion electrica y termica



07:53:41 7/18/16
FlexPDE Student 6.36s/W32

U
from (0)
to (Lx)

1: U



condElecTermic: Grid#1 P2 Nodes=51 Cells=25 RMS Err= 3.7e-6
Integral= -0.100000

Figura 5.26: Distribución de potencial en la barra de cobre.

Solución al Ejercicio 5.7

El script se muestra en el Código 5.12, y los gráficos en las Figuras 5.27 y 5.28.

```

TITLE 'Capa esferica'
COORDINATES
  Sphere1
SELECT
  gray
VARIABLES
  temp      !Temperatura
DEFINITIONS
  k=0.2      !Conductividad corcho
  R1 = 0.02   !Radio interior
  R2 = 0.04   !Radio exterior
  fluxQ=-k*grad(temp) !Flujo calor
INITIAL VALUES
  temp = 300
EQUATIONS
  Div(fluxQ) = 0
BOUNDARIES
  REGION 1      !Dominio total
  START(R1)      POINT LOAD(temp)=-100
  LINE TO (R2)    POINT VALUE(temp)=373
PLOTS
  ELEVATION(temp) FROM (R1) to (R2)
  ELEVATION(fluxQ) FROM (R1) to (R2)
END

```

Código 5.12: Flujo de calor a través de una capa esférica.

Flujo de calor a traves de una capa esferica

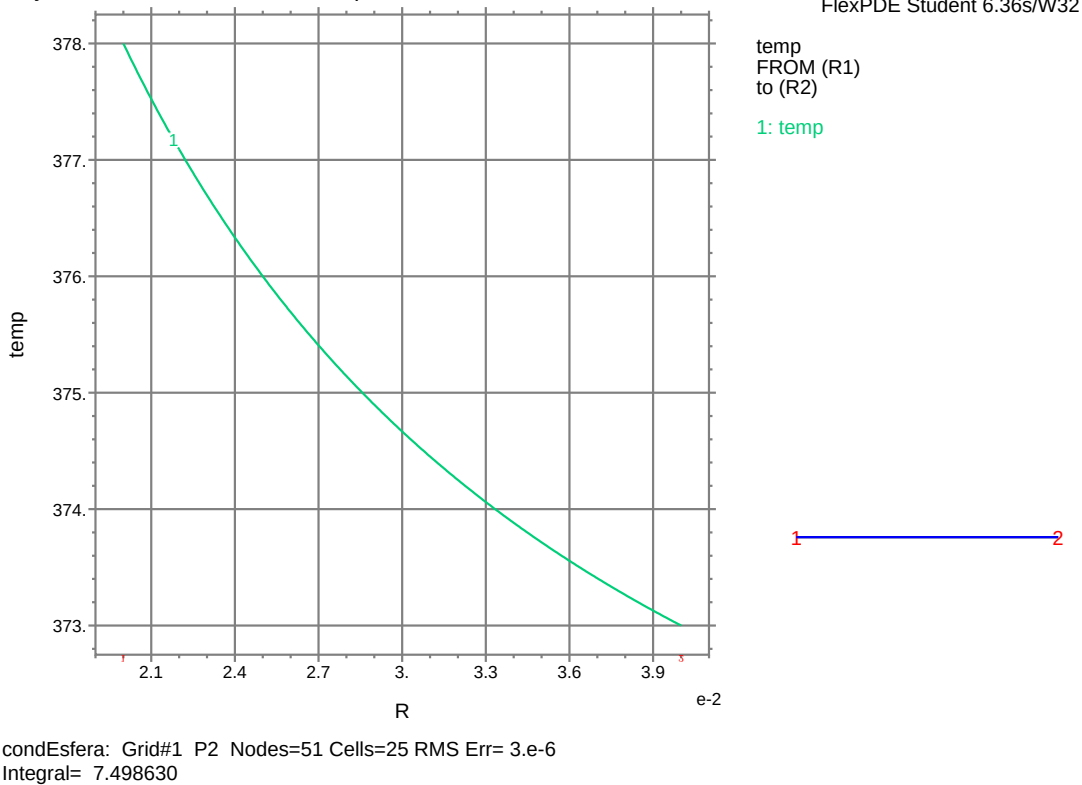


Figura 5.27: Distribución de temperatura en la esfera.

Flujo de calor a traves de una capa esferica

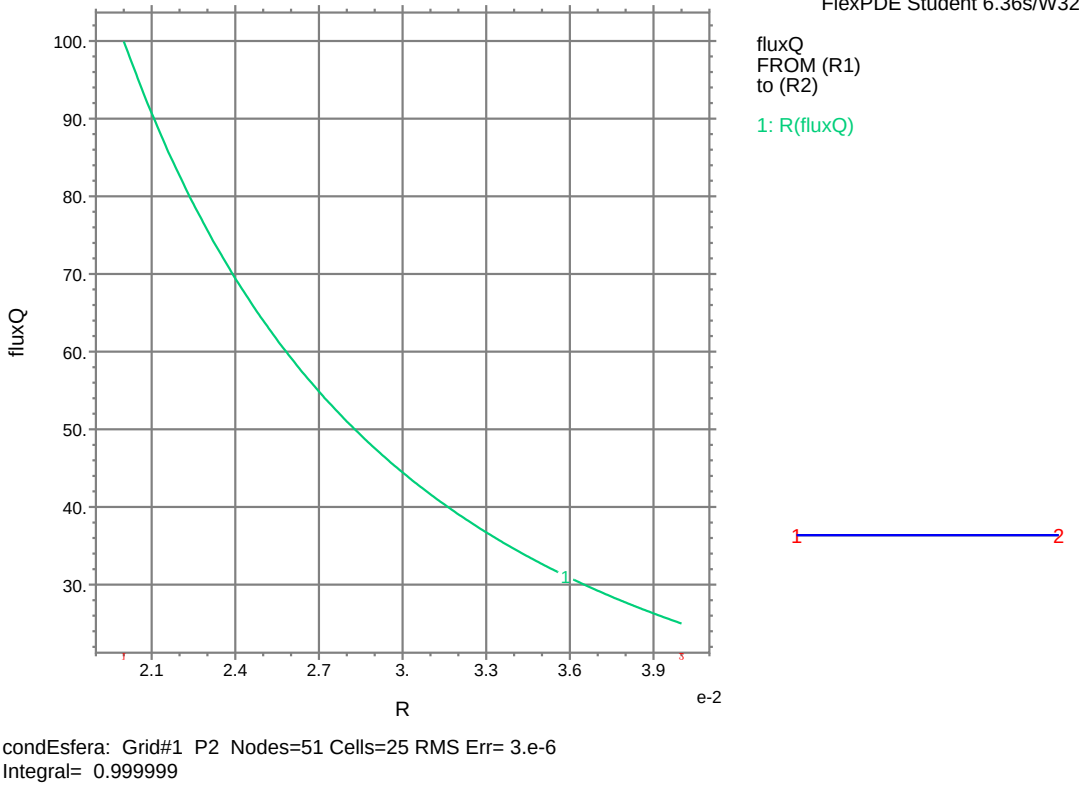


Figura 5.28: Distribución de calor en la esfera.

Solución al Ejercicio 5.8

El script se muestra en el Código 5.13 y los gráficos en las Figuras 5.29 y 5.30.

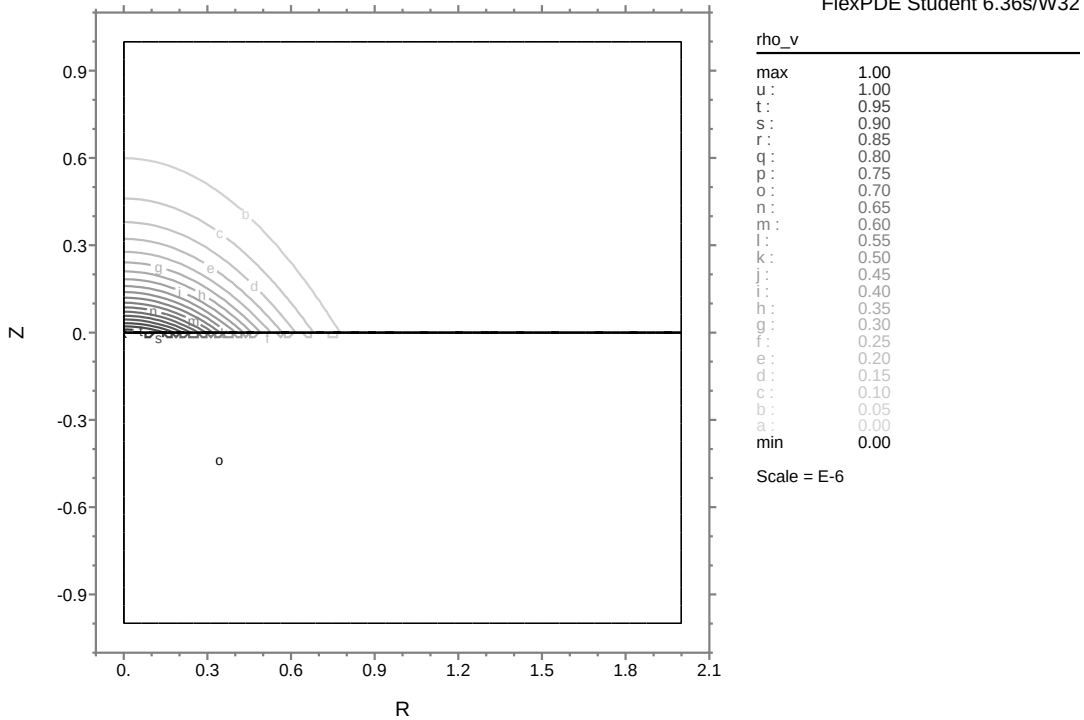
```

TITLE    'Cilindro dielectrico'
SELECT   errlim=1e-4    ngrid=1    gray
COORDINATES  ycylinder( 'r', 'z')
VARIABLES  U
DEFINITIONS
    Lr=2    Lz=1    r0=0.2
    eps0=8.854e-12    epsr    eps=epsr*eps0
    Er=-dr(U)    Ez=-dz(U)    E=-grad(U)    Em=magnitude(E)
    D=eps*E    Dm=magnitude(D)
    rho_v    rho_vg=1e-6*exp(-5*r^2-5*z)    { Densidad de carga dielectrico }
EQUATIONS
    div( D)=rho_v
BOUNDARIES
    region 'metal' epsr=1.0    rho_v=0    { Aire }
        start (0,Lz) natural(U)=0 line to (0,-Lz)
        value(U)=0 line to (Lr,-Lz) to (Lr,Lz) close
    region 'dielectrico' epsr=7.0    rho_v=rho_vg
        start(0,0) line to (Lr,0) to (Lr,Lz) to (0,Lz) close
PLOTS
    contour( rho_v)
    vector( E)
END

```

Código 5.13: Cilindro dieléctrico.

Cilindro dielectrico cargado por un haz



cilindroDielectrico: Grid#4 P2 Nodes=814 Cells=389 RMS Err= 3.1e-4
Vol_Integral= 1.249656e-7

Figura 5.29: Distribución de densidad de carga.

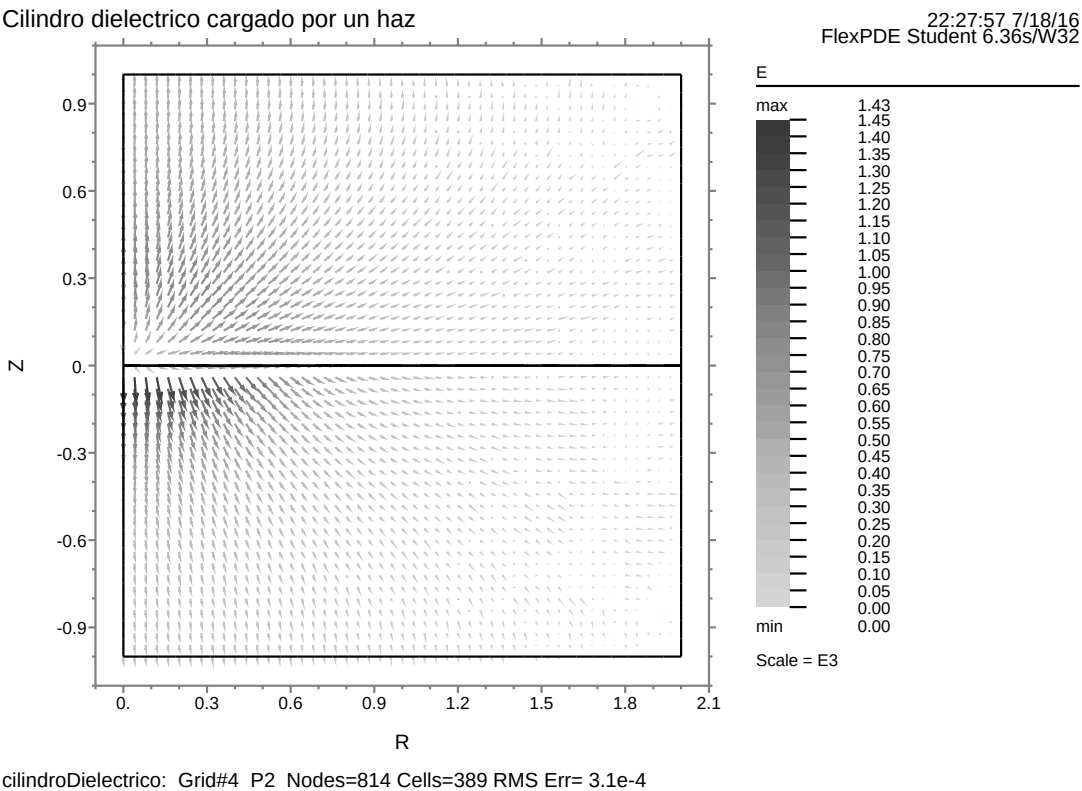


Figura 5.30: Campo eléctrico en el cilindro dieléctrico.

Índice alfabético

- abstracción, 45
- acumulador estadístico, 53
- algoritmo de simulación
 - autómata celular, 62
 - modelo dinámico ODE, 71
 - modelo eventos discretos, 53
 - modelo híbrido, 75
 - modelo tiempo continuo, 305
 - modelo tiempo discreto, 52
- algoritmo genético, 507
- algoritmo integración
 - BDF, 361
 - DASSL, 361
 - Euler explícito, 305
 - inline, 362
 - mixed mode, 363
- AnyLogic, 56, 65
- análisis por reducción, 44
- Arena, 56
- asignación única, 469
- AutoMod, 56
- autoorganización, 64
- autovalor, 342
- autómata celular, 57
 - aplicación, 63
 - célula, 59
 - juego de la vida, 60
 - regla, 59
 - vecindario, 59
- B-chart, 514
- base de datos
 - comportamiento, 38
- base de expansión, 573
- boxplot, 93
- calendario de eventos, 53
- campo
 - escalar, 559
 - vectorial, 560
- CATIA, 142
- causalidad computacional, 143, 144, 291, 449
- chattering, 445
- clase
 - parcial, 146
- cola
 - número de entidades, 124
 - tiempo de espera, 124
- componente
 - primitivo, 168
 - resistivo, 171
- componibilidad, 44
- condensador, 172
- condición
 - de contorno, 170
 - frontera, 564
 - Dirichlet, 565
 - mixta, 565
 - Neumann, 565
- conexión, 252
- controlador PI, 238, 501
- corrección del simulador, 40
- CyModelica, 142
- DAE, 51

- algoritmo de Pantelides, 337
- DASSL, 361
- grados de libertad, 308
- inicialización, 329
- reducción índice, 337
- semi-explícito, 51
- singular, 294
- índice, 318
- índice superior, 318, 327, 456
- DASSL, 361
- derivada numérica, 328
- DEVS, 55
- diagrama, 252
- diagrama de bloques, 291
- diferencias
 - finitas, 567
- difusividad térmica, 596
- diodo, 177
 - ideal, 461
- discretización temporal, 69
- Dymola, 142, 148
 - botón, 150, 151
 - Check, 154, 157, 248
 - Check Model, 165
 - Modelica Text, 149, 157
 - Modeling, 150
 - Simulate, 155, 160
 - Simulation, 150, 154
 - Store in Model, 159
 - Test compiler, 152
 - Translate, 154, 158
 - Change directory, 158
 - comandos experimento, 509
 - compilador C, 149, 152
 - directorio de trabajo, 158, 160
 - document, 508
 - documentación, 149
 - evento, 434
 - eveps, 438
 - experimento, 506
 - fichero
 - .mof, 510
 - dsfinal.txt, 508
 - dsin.txt, 158, 160, 508
 - dsmodel.c, 149, 508
 - dsres.mat, 508
 - dsres.txt, 508
 - dymosim.exe, 149, 158, 160, 508
 - resultados, 447
 - instalación, 149
 - interfaz usuario, 149
 - intervalo
 - de comunicación, 155, 159, 447, 506
 - de integración, 507
 - de simulación, 155, 159
 - list, 509
 - listfunctions, 508
 - método integración, 155
 - New model, 149
 - salida a fichero, 160
 - variable de salida, 159, 160
 - ventana
 - Components, 150
 - de mensajes, 155
 - Modeling, 149
 - Packages, 150
 - para simulación, 151
 - Simulation setup, 152, 155, 159
 - Variable Browser, 155, 158, 160
 - writeMatrix, 512
- EcosimPro, 77
- ecuación, 143
 - algebraica, 50
 - calor, 594
 - diferencial, 78
 - elíptica, 564
 - hiperbólica, 563

- homogénea, 81
- inhomogénea, 81
- lineal, 81
- orden, 79
- ordinaria, 50, 79
- parabólica, 564
- parcial, 50, 79, 80, 563
- en diferencias, 49
- instantánea, 417, 468
- Laplace, 564, 606
- ondas, 601
- Poisson, 607
- redundante, 295
- trivial, 289
- electrostática, 608
- elemento
 - de contorno, 566
 - de volumen, 348
 - finito, 570
- encapsulado de la información, 45, 146
- energía
 - almacenamiento, 172
 - conversión, 174
 - disipación, 171
 - fuelle, 170
 - transporte, 172
- Enterprise Dynamics, 56
- entorno, 170
 - de modelado, 141, 145, 147
 - de simulación, 56
- espacio
 - celular, 59
- estacionario, 505
- evento, 49, 74, 415, 459
 - cadena, 430, 433
 - condición, 415
 - condición de disparo, 74
 - detección, 436, 443, 485
 - disparo, 430
- Dymola, 434
- ejecución, 415, 433
- en el estado, 74, 443
- en el tiempo, 74, 441
- endógenos, 441
- exógeno, 441
- función de cruce, 436
- habilitado, 415
- interactivo, 520
- localización, 76
- noEvent, 444
- orden ejecución, 431
- reinicio, 417, 430
- experimento, 34
- ExtendSim, 56
- factor de Fanning, 537
- FlexPDE, 580
 - .log, 582
 - .pde, 582
 - .pg6, 582
 - botón, 581
 - Domain Review, 581
 - Run Script, 581
 - Show Editor, 581
 - Show Plots, 581
 - condiciones de contorno, 590
- FlexSim, 56
- flujo
 - de calor, 172
- forma
 - débil, 570
 - variacional, 570
- fricción, 171
- fuelle
 - de energía, 170
 - de esfuerzo, 171
 - de flujo, 171
 - ideal, 171

- modulada, 171
- función
 - signo, 444
- fórmula de Blasius, 537
- girador, 176
- GPSS, 56
- grado de libertad, 295, 308, 454
- icono, 252
- inducción, 172
- ingeniería inversa, 38
- inicialización, 495
- integración
 - Adams-Bashforth, 71
 - Adams-Moulton, 71
 - BDF, 73
 - método explícito, 70
 - orden del método, 70
 - por partes, 571
 - RKF45, 73
 - Runge-Kutta, 71
 - tamaño del paso, 69
- interpolación, 225
- interruptor
 - ideal, 449
 - no ideal, 456
- invariante, 75, 414
- JModelica.org, 142
- lazo algebraico, 297, 341, 450
 - manipulación, 342
- lenguaje
 - de modelado, 141
 - de simulación, 56
- ley
 - conservación energía, 169
 - de Faraday, 607
 - de Fourier, 594
 - de Gauss, 607
 - de Hooke, 173, 602
 - de Newton, 172, 204, 218
 - de Ohm, 143
- LMS Imagine.Lab AMESim, 142
- Logo, 66
- Lotka-Volterra, 269
- MapleSim, 142
- marco experimental, 38, 39
- MASON, 65
- MathModelica, 142
- Matlab/Simulink, 77
- matriz
 - BLT, 296
 - de incidencia, 292
 - de incidencia ordenada, 296
 - Jacobiana estructural, 292
 - rala, 293
 - singular, 319
 - sparse, 293
 - tridiagonal, 373
- mezcla agitada, 348
- Model Vision Language, 513
- modelado
 - acausal, 143
 - basado en ecuaciones, 143, 291
 - causal, 144
 - diagrama de bloques, 77
 - jerárquico, 45
 - modular, 44
 - orientado a objetos, 77, 145
 - orientado a procesos, 55
- Modelica, 77, 142
 - .mo, 197
 - .mos, 151, 507
 - abs, 227
 - acceso componentes, 195
 - algorithm, 228
 - partición, 216

- algoritmo, 144
- annotation, 252
- anotación, 147
 - diagrama, 147
 - icono, 147
 - info, 147
- asignación, 144
- assert, 181, 228, 477
- atributo
 - displayUnit, 181, 225
 - final, 183
 - fixed, 153, 181
 - max, 181
 - min, 181
 - quantity, 181
 - start, 153, 181
 - stateSelect, 318
 - unit, 181, 225
- block, 147
- Boolean, 181
- campo físico, 252
- checkModel, 508
- clase, 147
 - parcial, 146, 156, 164
- clear, 509
- comentario en código, 153, 154
- conector, 146
- connect, 196
- connector, 147, 187
 - sin conectar, 197, 248
- constant, 181
- constante, 153
- cross, 214
- der(), 154
- diagonal, 214
- discrete, 181
- div, 227
- encapsulado de la información, 146
- encapsulated, 195
- escritura mensaje, 471
- evento, 145, 459
- experimento, 506
- extends, 188
- fichero script, 151
- fill, 214
- fixed, 360, 496, 500
- flow, 187
- for, 216, 234, 511
- for con iteradores, 213
- función, 145, 227
 - memoria, 228
- función externa, 145, 231
- function, 147, 227
- herencia, 146, 188
 - import, 196
- identity, 214
- if, 190, 450, 460, 506, 511
- import, 195
- inicialización, 495
- initial, 470, 500, 503, 506
- initial equation, 234, 496, 500
- inner, 254, 255, 258
- Integer, 181
- interfaz, 187
- linspace, 214
- LogVariable, 472
- matriz, 212
 - funciones, 213
- max, 214
- min, 214
- model, 147
- Modelica Association, 141
- Modelica Standard Library, 141
- MODELICAPATH, 198
- modularidad, 146
- MSL, 141
 - Constants, 183
 - SIunits, 182

- ndims, 214
- noEvent, 444, 486
- notación punto, 191
- ones, 214
- openModel, 508
- outer, 254, 255, 258
- package, 147, 191
- package.mo, 198
- parameter, 181
- parametrización, 146
- parametrización clases, 199
- partial model, 188
- parámetro, 153
- plot, 508
- pre(), 459
- product, 214
- protected, 188, 221
- rango de índices, 213
- Real, 181
- record, 147, 221
- redeclare, 200
- reinit, 420, 468
- rem, 227
- replaceable, 199
- sample, 470
- scripting language, 507
- sección equation, 154
- sign, 227
- simulateModel, 508
- size, 214
- sqrt, 227
- start, 496, 500
- stateSelect, 359
- String, 181, 471
 - concatenación, 471
 - integerString, 471
 - realString, 471
- submatrices, 213
- subvectores, 213
- sum, 214
- terminación simulación, 470
- terminal, 470
- terminate, 471
- time, 180
- translateModel, 508
- transpose, 214
- type, 147, 182
- valor por defecto, 189
- variable
 - alias, 252
 - atributo, 181
 - de estado, 318
 - protegida, 146
 - tipos predefinidos, 181
- vector, 212
 - funciones, 213
- vectorización
 - de componentes, 234
 - de conexiones, 234
 - de funciones, 227
- when, 467
- within, 198
- zeros, 214
- Modelicac, 142
- modelo, 34, 38, 39
 - algebraico, 50
 - atómico, 45
 - clasificación, 47
 - complejidad, 40
 - compuesto, 45
 - DAE híbrido, 51
 - de eventos discretos, 48
 - de tiempo continuo, 49
 - de tiempo discreto, 48
 - descripción interna, 45
 - determinista, 48
 - dinámico, 48
 - estocástico, 48

- estructura variable, 74, 443, 444, 449
- estático, 48
- fase, 449
- físico, 36
- híbrido, 51, 75, 459
- información estructural, 294
- infradeterminado, 298
- inicialización, 343
- interfaz, 45
- matemático, 36
- mental, 35
- modo, 449
- molecular, 45
- ODE híbrido, 51
- plano, 147
- primitivo, 45
- simplificación, 40
- singular, 295
- sobredeterminado, 296, 299
- stiff, 341
- validación, 46
- verbal, 36
- verificación, 46
- modularidad, 44
- MWorks, 142
- método
 - de la secante, 374
 - de las líneas (MOL), 81
 - de Newton, 375
 - del punto fijo, 375
 - matriz tridiagonal, 373
- NetLogo, 65
- nodo, 169
- notación punto, 197
- número de Reynolds, 537
- ocultación de la información, 45
- ODE, 50, 79
- Omola, 413
- OpenModelica, 142, 161
 - OMEdit, 161
 - botón, 162
 - directorio de trabajo, 167
 - método integración, 163
 - variable de salida, 167
- operador diferencial, 559
 - divergencia, 561
 - gradiente, 560
 - laplaciano, 563
 - rotacional, 562
- palanca, 175
- paradigma
 - modelado físico, 142
- parametrización, 197, 221
- partición, 148, 296
 - Tarjan, 296
- parámetro, 146
- pin, 169
- pistón, 175
- plano de control, 348
- Plant Simulation, 56
- ProModel, 56
- péndulo, 149, 498
- R
 - boxplot, 93
 - c(), 85
 - comentario, 85
 - control del flujo, 99
 - data frame, 88
 - demo, 86
 - descarga, 84
 - directorio de trabajo, 87
 - espacio de trabajo, 87
 - fichero script, 88
 - for, 99
 - gráfico X-Y, 92
 - histograma, 93

- historia comandos, 87
- if-else, 100
- ifelse, 100
- librería, 103
- NA, 97
- ordenar, 97
- paquete, 103
- plot, 86
- salir, 85
- semilla generador, 101
- suma, 98
- while, 99
- Rand Model Designer, 512
- refinamiento, 43
- regla asignación única, 216, 434, 469
- relación
 - de modelado, 38, 40
 - de simulación, 38, 40
- reloj de la simulación, 52
- Repast, 65
- rutina
 - de evento, 55
 - de informes, 55
 - de tiempo, 54
 - inicialización, 54
- Scilab/Scicos, 77, 142
- SIMAN, 56
- Simio, 56
- SIMSCRIPT, 56
- SIMUL8, 56
- simulación, 36
 - de Monte Carlo, 109
 - de tiempo discreto, 52
 - interactiva, 520
 - modelo híbrido, 75, 148, 416
- simulador, 38, 40
 - corrección, 40
- SimulationX, 142
- singularidad estructural, 294, 302
- sistema, 34
 - análisis, 37
 - complejo, 64
 - conocimiento, 36
 - diseño, 37
 - experto, 36
 - fuelle, 37, 38
 - inferencia, 37
 - stiff, 341
- SLAM, 56
- SPICE, 221, 344
- subclase, 189
- superclase, 188
- Swarm, 65
- tabla de transición/salidas, 107
- tacómetro, 175
- tearing, 345
- Teorema
 - de Green, 570
 - de la divergencia, 570
 - integración por partes, 571
- transformador, 175
- trayectoria, 38
 - de entrada, 122
 - de estados, 52
 - de salida, 52, 122
- Unified Modeling Language, 512
- validez, 40
 - estructural, 40
 - predictiva, 40
 - replicativa, 40
- variable
 - across, 146
 - algebraica, 306
 - alias, 290
 - auxiliar, 292

- clasificación, 302
- de entrada, 48
- de estado, 305, 346
- de tiempo continuo, 49
- de tiempo discreto, 49
- effort, 146
- escalar, 210
- esfuerzo, 146
- ficticia, 299
- flow, 146
- flujo, 146
- interactiva, 520
- parámetro, 305
- through, 146, 169
- volumen
 - de control, 234, 348
 - finito, 566
- Witness, 56
- Wolfram SystemModeler, 142

Bibliografía

Analysis, T. (2016), ‘Field simulation software for electrical, thermal and mechanical engineering’.

URL: <http://www.quickfield.com/>

Andersson, M. (1990), *Omola. An Object-Oriented Language for Model Representation*, Licenciate thesis TFRT-3208. Department of Automatic Control. Lund Institute of Technology. Lund. Sweden.

Andersson, M. (1993), *OmSim and Omola. Tutorial and User’s Manual. Version 3*, Department of Automatic Control. Lund Institute of Technology. Lund, Sweden.

Andersson, M. (1994), *Object-oriented Modeling and Simulation of Hybrid Systems*, PhD Thesis, Lund Institute of Tech., Sweden.

Ansys (2016), ‘High frequency electromagnetic field simulation’.

URL: <http://www.ansys.com/Products/Electronics/ANSYS-HFSS>

Åström, K. J., Elmqvist, H. & Mattsson, S. E. (1998), Evolution of continuous-time modeling and simulation, in ‘Proc. of 12th European Simulation Multiconference’, Manchester, UK, pp. 9–18.

Åström, K. J. & Hagglund, T. (1995), *PID Controllers: Theory, Design and Tuning*, ISA Press.

Barton, P. I. (1992), *The Modelling and Simulation of Combined Discrete/Continuous Processes*, Ph.D. Thesis. Department of Chemical Engineering. Imperial College of Science, Technology and Medicine. London.

Barton, P. I. & Pantelides, C. C. (1994), ‘Modeling of combined discrete/continuous proceses’, *AIChE J.* **40**, 966–979.

Becker, R., Chambers, J. & Wilks, A. (1988), *The New S Language*, Wadsworth & Brooks/Cole.

- Bird, R. B., Stewart, W. E. & Lightfoot, E. N. (1975), *Transport Phenomena*, John Wiley & Sons, New York.
- Borshchev, A. (2013), *The Big Book of Simulation Modeling*, AnyLogic North America.
- Bratley, P., Fox, B. L. & Schrage, L. E. (1987), *A Guide to Simulation*, Springer.
- Brenan, K. E., Campbell, S. L. & Petzold, L. R. (1996), *Numerical Solution of Initial-Value Problems in Differential-Algebraic Equations*, SIAM.
- Burden, R. L. & Faires, J. D. (1985), *Análisis Numérico*, Grupo Editorial Iberoamericana.
- Cellier, F. E. (1979), *Combined Continuous/Discrete System Simulation by Use of Digital Computers: Techniques and Tools*, PhD Thesis, Switzerland.
- Cellier, F. E. (1991), *Continuous System Modeling*, Springer-Verlag.
- Cellier, F. E. & Kofman, E. (2006), *Continuous System Simulation*, Springer-Verlag.
- COMSOL (2016), 'Comsol multiphysics'.
URL: <http://www.comsol.com>
- Cutlip, M. B. & Shacham, M. (1999), *Problem Solving in Chemical Engineering with Numerical Methods*, Prentice-Hall.
- Damic, V. & Montgomery, J. (2003), *Mechatronics by Bond Graphs*, Springer.
- Dormido, S. & Urquía, A. (2009a), *Apuntes del curso Modelado de Sistemas Físicos, del programa de doctorado en Ingeniería de Sistemas y Automática*, Dpto. de Informática y Automática, E.T.S. Ing. Informática, UNED.
- Dormido, S. & Urquía, A. (2009b), *Apuntes del curso Simulación de Sistemas Dinámicos, del programa de doctorado en Ingeniería de Sistemas y Automática*, Dpto. de Informática y Automática, E.T.S. Ing. Informática, UNED.
- Elmqvist, H. (1978), *A Structured Model Language for Large Continuous Systems*, PhD Thesis, Lund Institute of Tech., Sweden.
- Elmqvist, H. (1993), Object-oriented modeling and automatic formula manipulation in Dymola, in 'Proc. of SIMS'93, Scandinavian Simulation Society', Kongsberg, Norway.

- Elmqvist, H., Cellier, F. E. & Otter, M. (1993), Object-oriented modeling of hybrid systems, *in* 'Proc. of ESS'93, European Simulation Symposium', Delft, The Netherlands.
- Elmqvist, H., Cellier, F. E. & Otter, M. (1994), Object-oriented modeling of power-electronic circuits using Dymola, *in* 'Proc. of CISS - First Joint Conference of Intl. Simulation Societies', Zurich, Switzerland.
- Elmqvist, H., Cellier, F. E. & Otter, M. (1995), Inline integration: a new mixed symbolic/numeric approach for solving differential-algebraic equation systems, *in* 'Proc. of ESM'95, SCS European Simulation MultiConference', Prague, Czech Republic.
- Elmqvist, H., Mattsson, S. E. & Olsson, H. (2002), New methods for hardware-in-the-loop simulation of stiff models, *in* 'Proc. of 2nd Intl. Modelica Conference', Oberpfaffenhofen, Germany.
- Elmqvist, H., Mattsson, S. E. & Otter, M. (1998), Modelica - the new object-oriented modeling language, *in* 'Proc. of ESM'98, European Simulation Multiconference', Manchester, UK.
- Elmqvist, H., Mattsson, S. E. & Otter, M. (1999), Modelica - a language for physical system modeling, visualization and interaction, *in* 'Proc. of CACSD'99, 1999 IEEE Symposium on Computer-Aid Control System Design', Hawai, USA.
- Elmqvist, H. & Otter, M. (1994), Methods for tearing systems of equations in object-oriented modeling, *in* 'Proc. of ESM'94, European Simulation Multiconference', Barcelona, Spain.
- EPRI (1984), *Modular Modeling System. Theory Manual. MMS-02 Release*, Electric Power Research Institute.
- Fritzson, P. (2015), *Principles of Object-Oriented Modeling and Simulation with Modelica 3.3: A Cyber-Physical Approach*, Wiley.
- Fritzson, P., Aronsson, O., Bunus, P., Engelson, V., Johansson, H., Karstrom, A. & Saldamli, L. (2002), The open source Modelica project, *in* 'Proc. of 2nd International Modelica Conference', Oberpfaffenhofen, Germany.
- Froment, G. F. & Bischoff, K. B. (1979), *Chemical Reactor Analysis and Design*, John Wiley & Sons, New York.
- Gardner, M. (1970), 'The fantastic combinations of John Conway's new solitaire game of life', *Scientific American* **23**(4), 120–123.

- Gockenbach, M. S. (2011), *Partial Differential Equations. Analytical and Numerical Methods*, SIAM.
- Guru, B. & Hizirolu, H. (2009), *Electromagnetic Field Theory Fundamentals*, Cambridge University Press.
- Hecht, F. (2012), ‘New development in freefem++’, *Journal of Numerical Mathematics* **20**.
- Himmelblau, D. M. & Bischoff, K. B. (1992), *Process Analysis and Simulation*, John Wiley & Sons, New York.
- Holmes, M. H. (2007), *Introduction to Numerical Methods in Differential Equations*, Springer.
- Hoover, S. V. & Perry, R. F. (1989), *Simulation. A Problem-Solving Approach*, Addison-Wesley Publishing.
- Huber, R. M. (1994), *Modelado Modular y Jerárquico de Sistemas de Tiempo Continuo*, Apuntes preparados con motivo del XIV Curso de Automática en la Industria, celebrado en La Manga del Mar Menor (Murcia) en Junio de 1994.
- Inc., P. S. (2016), ‘FlexPDE’.
URL: <http://www.pdesolutions.com/index.html>
- Incropera, F. P. & DeWitt, D. P. (1996), *Fundamentals of Heat and Mass Transfer*, John Wiley & Sons, New York.
- Inihov, D., Kolesov, Y. & Senichenkov, Y. (2012), Physical modeling of hybrid systems with Rand Model Designer, in ‘Proc. of 7th Vienna International Conference on Mathematical Modelling’, Vol. 7(1), pp. 49–54.
- Johansson, K. H. (2000), ‘The quadruple-tank process: a multivariable laboratory process with an adjustable zero’, *IEEE Trans. Control Systems Technology* **8**(3), 456–465.
- Kabacoff, R. I. (2011), *R in Action*, Manning Publications Co.
- Karnopp, D. C., Margolis, D. L. & Rosenberg, R. C. (1990), *System Dynamics: A Unified Approach*, John Wiley & Sons.
- Kier, L. B., Seybold, P. G. & Cheng, C.-K. (2005), *Cellular Automata Modeling of Chemical Systems*, Springer.
- Klir, G. J. (1985), *Architecture of Systems Complexity*, Saunders, New York.

- Law, A. M. & Kelton, W. D. (2000), *Simulation Modeling and Analysis*, McGraw-Hill.
- Lawrence, M. F. & Verzani, J. (2012), *Programming Graphical User Interfaces in R*, CRC Press.
- Ljung, L. & Torkel, G. (1994), *Modeling of Dynamic Systems*, Prentice-Hall.
- Luyben, W. L. (1990), *Process Modeling, Simulation and Control for Chemical Engineers*, McGraw-Hill.
- Maindonald, J. & Braun, W. J. (2010), *Data Analysis and Graphics using R - an Example-Based Approach*, Cambridge University Press.
- Martín-Villalba, C. (2007), *Object-Oriented Modeling of Virtual Laboratories for Control Education*, PhD Thesis, UNED, Madrid, Spain.
- Martín-Villalba, C., Urquía, A., Senichenkov, Y. & Kolesov, Y. (2014), ‘Two approaches to facilitate virtual lab implementation’, *Computing in Science and Engineering* **Jan/Feb**, 78–86.
- MathWorks (2016), ‘Partial differential equation toolbox’.
URL: <http://es.mathworks.com/products/pde/>
- Mattsson, S. E., Elmqvist, H., Otter, M. & Olsson, H. (2002), Initialization of hybrid differential-algebraic equations in modelica 2, in ‘Proc. of 2nd Int. Modelica Conf.’, pp. 9–15.
- Mattsson, S. E., Olsson, H. & Elmqvist, H. (2000), Dynamic selection of states in Dymola, in ‘Proc. of Modelica Workshop’, pp. 61–67.
- Mattsson, S. E. & Söderlind, G. (1992), A new technique for solving high-index differential equations using dummy derivatives, in ‘Proc. of IEEE Symposium on Computer-Aided Control System Design’, California, USA.
- Meeker, D. (2016), ‘Finite element method magnetics’.
URL: <http://www.femm.info/wiki/HomePage>
- ModelicaTM (2000), *Modelica - A Unified Object-Oriented Language for Physical Systems Modeling. Tutorial. Version 1.4*, Modelica Association.
- Nilsson, B. (1989), *Structured Modelling of Chemical Processes - An Object-Oriented Approach*, Ms Thesis, Lund Institute of Technology, Sweden.

- Nilsson, B. (1993), *Object-oriented Modeling of Chemical Processes*, PhD Thesis, Lund Institute of Technology, Sweden.
- Otter, M. & Olsson, H. (2002), New features in Modelica 2.0, in ‘Proc. of 2nd International Modelica Conference’, pp. 7.1–7.12.
- Pantelides, C. C. (1988), ‘The consistent initialization of differential-algebraic systems’, *SIAM J. SCI. STAT. COMPUT.* **9**(2).
- Pedgen, C. D., Shannon, R. E. & Sadowsky, R. P. (1995), *Introduction to Simulation Using SIMAN*, McGraw-Hill.
- Railsback, S. F. & Grimm, V. (2012), *Agent-Based and Individual-Based Modeling*, Princeton University Press.
- Ramirez, W. F. (1989), *Computational Methods for Process Simulation*, Butterworths Publishers, Boston.
- Rosenberg, R. C. & Karnopp, D. C. (1983), *Introduction to Physical System Dynamics*, McGraw-Hill.
- Rubio, M. A. (2008), *Modelado Dinámico de Pilas de Combustible en el Contexto del Control Automático*, PhD Thesis, UNED, Madrid, Spain.
- Rumbaugh, J., Jacobson, I. & Booch, G. (2005), *The Unified Modeling Language Reference Manual*, Addison-Wesley.
- Schiela, A. & Olsson, H. (2000), Mixed-mode integration for real-time simulation, in ‘Proc. of Modelica Workshop’, Lund, Sweden, pp. 69–75.
- Schiff, J. L. (2008), *Cellular Automata*, Wiley.
- Senichenkov, Y., Kolesov, Y., Urquia, A. & Martin-Villalba, C. (2013), Simulation of variable structure models using Rand Model Designer, in ‘Proc. of 8th EUROSIM Congress on Modelling and Simulation, Cardiff, Wales, United Kingdom’.
- Sherwin, J. P. . S. (2004), *Handbook of Materials Modeling - Methods*, Springer Netherlands.
- Silebi, C. A. & Schliesser, W. E. (1992), *Dynamic Modelling of Transport Process Systems*, Academic Press.
- Steward, D. V. (1981), *System Analysis and Management: Structure, Strategy and Design*, Petrocelli Books, Inc.

- Tarasov, S. V., Kiptily, D. V. & Lebedev, D. V. (2012), An object-oriented approach to the development of liquid cargo handling simulators in TRANSAS, *in* ‘Proc. of 7th Vienna International Conference on Mathematical Modelling’, Vol. 7(1), pp. 369–373.
- Thoma, J. U. (1990), *Simulation by Bondgraphs*, Springer-Verlag.
- Tyszer, J. (1999), *Object-Oriented Computer Simulation of Discrete-Event Systems*, Kluwer Academic Publishers.
- Urquía, A. (2000), *Modelado Orientado a Objetos y Simulación de Sistemas Híbridos en el Ámbito del Control de Procesos Químicos*, PhD Thesis, UNED, Madrid, Spain.
- Urquía, A. & Martín-Villalba, C. (2013), *Modelado y Simulación de Eventos Discretos*, Editorial UNED.
- Venables, W. N. & Ripley, B. D. (1997), *Modern Applied Statistics with S-PLUS*, Springer.
- Wilensky, U. & Rand, W. (2015), *An Introduction to Agent-Based Modeling*, The MIT Press.
- Wolfram, S. (1986), *Theory and Application of Cellular Automata*, World Scientific, Singapore.
- Zeigler, B. P., Praehofer, H. & Kim, T. G. (2000), *Theory of Modeling and Simulation*, Academic Press.



Juan del Rosal, 14
28040 MADRID
Tel. Dirección Editorial: 913 987 521